



DEVELOPER REFERENCE

*Imagine
That!*

ISBN: 978-0-9825040-1-7

Copyright © 2013 by Imagine That Inc. All rights reserved. Printed in the United States of America.

You may not copy, transmit, or translate all or any part of this document in any form or by any means, electronic or mechanical, including photocopying, recording, or information storage and retrieval systems, for any purpose other than your personal use without the prior and express written permission of Imagine That Inc.

License, Software Copyright, Trademark, and Other Information

The software described in this manual is furnished under a separate license and warranty agreement. The software may be used or copied only in accordance with the terms of that agreement. Please note the following:

ExtendSim blocks (including icons, dialogs, and block code) are copyright © by Imagine That Inc. and/or its Licensors. ExtendSim blocks contain proprietary and/or trademark information. If you build blocks, and you use all or any portion of the blocks from the BPR, Discrete Event, Flow, Item, Mfg, Rate, or Quick Blocks library in your blocks, or you include those ExtendSim blocks (or any of the code from those blocks) in your libraries, your right to sell, give away, or otherwise distribute your blocks and libraries is limited. In that case, you may only sell, give, or distribute such a block or library if the recipient has a legal license for the ExtendSim product from which you have derived your block(s) or block code. For more information, contact Imagine That Inc.

Imagine That!, the Imagine That logo, ExtendSim, Extend, and ModL are either registered trademarks or trademarks of Imagine That Incorporated in the United States and/or other countries. Mac OS is a registered trademark of Apple Computer, Inc. Microsoft is a registered trademark and Windows is a trademark of Microsoft Corporation. GarageGames, Inc. is the copyright owner of the Torque Game Engine (TGE), and the copyright for Stat::Fit® is owned by Geer Mountain Software. All other product names used in this manual are the trademarks of their respective owners. TGE and Stat::Fit are licensed to Imagine That, Inc. for distribution with ExtendSim. All other ExtendSim products and portions of products are copyright by Imagine That Inc. All right, title and interest, including, without limitation, all copyrights in the Software shall at all times remain the property of Imagine That Inc. or its Licensors.

Extend was created in 1987 by Bob Diamond; it was rebranded as ExtendSim in 2007.

Chief architects for ExtendSim 9:

Bob Diamond, Steve Lamperti, Dave Krah, Anthony Nastasi, Cecile Pieper

Graphics, documentation, and production for ExtendSim 9:

Carla Sackett, Pat Diamond, and Kathi Hansen

Imagine That Inc • 6830 Via Del Oro, Suite 230 • San Jose, CA 95119 USA
408.365.0305 • fax 408.629.1251 • info@extendsim.com
www.extendsim.com

Table of Contents

DEVELOPER REFERENCE

TABLE OF CONTENTS

OVERVIEW

Introduction	1
About the Developer Reference.....	2
ExtendSim IDE.....	2
How the Developer Reference is organized.....	2
Additional resources.....	3
Contacting Imagine That Inc. Technical Support.....	3
Parts of a Block.....	5
Table of block parts.....	6
Dialogs.....	7
Exploring an existing block's dialog.....	7
Dialog of the Example block.....	7
Dialog tabs.....	7
Exploring an existing block's structure.....	8
To open a block's internal structure.....	8
Examining the internal structure.....	9
Dialog editor window.....	9
Structure window.....	10
Popups.....	10
Icon pane.....	10
Help pane.....	10
Code pane.....	11
Code completion button.....	11
Variables pane.....	11
Connector pane.....	11
Working with the dialog editor and structure windows.....	11
Dialog items.....	11
Dialog item definition.....	11
Types of dialog items.....	12
Options for dialog items.....	14
Dialog item names and text/titles.....	14
X,Y,W,H.....	14
Display only.....	14
Hide item.....	15
Visible in all tabs.....	15
Add to right click menu.....	15
Tab order.....	15
Number formats.....	15
Working with dialog items.....	16
Adding new dialog items.....	16
Copying dialog items.....	16
Moving dialog items.....	16
Resizing dialog items.....	16
Styles, alignment, and colors for dialog items.....	16
Tool Tips on block dialog and dialog editor windows.....	17

ModL code introduction	18
ModL and other languages	18
ModL structure	18
Icons	18
Creating the icon.....	18
Icon views and model styles.....	19
Model styles	19
Icon views	19
Animation	19
2D animation.....	19
3D Animation.....	20
Connectors.....	20
Connector types	20
Connector options	20
Animation object tool	21
Adding connectors to the icon.....	21
Naming connectors	21
Changing connector types.....	21
Help text	22
ModL Overview.....	23
ModL and other programming languages.....	24
ModL and other languages.....	24
ModL compared to C++	24
ModL compared to other languages	26
ModL feature overview	27
ModL language terminology.....	28
Table of terms	28
Global, local, and static variables.....	29
Layout of a block's ModL code	29
Syntax coloring	29
Type declarations and constant definitions	30
Functions and message handlers.....	30
Code completion	31
Overriding functions and message handlers.....	32
Conditional compilation.....	32
Accessing connectors from a block's code.....	32
Single ("normal") connectors.....	32
Variable connectors	32
Accessing a variable connector.....	32
Setting the number of connectors.....	33
Accessing dialog items from a block's code.....	33
Overview.....	33
Dialog messages	33
Parameters and editable text	34
Dynamic text	34
Check boxes and radio buttons	35
Buttons	36
Data tables and text tables.....	36
Static text (label)	37
Popup menu items	37
Text frame.....	38
Switch	38
Slider.....	38
Meter	38

Embedded object (Windows only)	39
Calendar	39
Using message handlers	39
Example of a message sent during user interaction with dialog	40
Example of a message sent by the application	40
Block to block message.....	40
Accessing code from other languages.....	40
External source code	41

TUTORIAL

Creating a Block	43
Steps for creating a new block	44
Building a block that converts miles to feet	44
Create the block.....	44
Dialog editor window of the Miles block.....	45
Structure window of the Miles block.....	45
Icon	46
Connectors.....	46
Help text.....	47
ModL code.....	47
Compile and save the block.....	48
Test the block.....	48
Adding user interaction and display features.....	48
Dialog items.....	48
Defining a radio button	49
Additional radio buttons	49
Text frames.....	50
Saving the dialog changes	50
Adding parameter fields	50
Adding more static text labels.....	51
Tabs in the dialog.....	51
Adding tabs.....	51
Showing dialog items on all tabs.....	51
Moving dialog items between tabs.....	51
Icon	52
ModL code.....	52
CreateBlock message handler.....	52
Simulate message handler	52
Compile and save the block.....	53
Adding an intermediate results feature	53
Add a button.....	54
ModL code.....	54
Adding 2D animation	54
Changing the icon and adding the animation object	54
Adding code for the animation.....	55
Testing the block.....	56
Other features	56
Colored dialog items	56
Hiding/showing dialog items.....	56
Moving dialog items.....	56
Defining functions	56
Popup menus	57
Linking to a global array or ExtendSim database	57

Block categories.....	57
Variable connectors	58
Connector labels	58
Connector tool tips	58
Column tags and parameter tags	58
Include files.....	58
External source code.....	59
DLLs and Shared Libraries.....	59
Icon views	59
How view order is maintained within classes.....	59

MODL

The ModL Language	61
Names	62
Data types	62
Reals	62
Integers	62
Strings	63
Declarations and definitions	63
Static and local	63
Type declarations.....	64
Constant definitions.....	64
BLANK and NoValue	64
Numeric type conversion.....	65
Real to integer.....	65
NoValue to integer	65
Integer to real.....	65
Integer or real to string.....	66
String to real or integer.....	66
Arrays.....	66
Array declarations.....	66
Fixed and dynamic arrays	66
Arrays as arguments to functions	67
Array-like structures	68
Global arrays	68
Linked lists.....	68
Database tables.....	68
Operators	68
Assignment operators	69
Math operators.....	69
Boolean and magnitude operators	70
Control statements and loops	70
User-defined functions	73
Defining.....	74
Exiting	74
Overriding user-defined functions.....	75
Declaring arrays as arguments for user-defined functions	75
Message handlers	75
Overriding message handlers.....	76
System variables.....	76
Global variables	76
Constants.....	76

Conditional compilation	76
Programming Tools	77
Formatting ModL code and Help	78
Searching and replacing	78
Formatting tips	78
Include files	79
Conditional compilation	79
External source code control	80
How to save or remove external source code	80
Externalizing a block's code	80
Externalizing an entire library's block code	81
The consequences of saving the code externally	81
Using the external source code with code management software	81
Cautions when using the external source code feature	82
Managing non-code parts of blocks	82
Sharing libraries with others	82
Extensions folder	82
Extensions on Windows	82
Extensions on the Mac OS	82
DLLs	83
Overview	83
DLL interface	83
Turning code into a DLL	83
DLL example	84
Shared libraries	85
Sounds	85
Sounds on Windows	85
Sounds on the Mac OS	85
Picture and movie files	86
Naming conventions and limitations for pictures	86
Pictures on Windows	86
Pictures and movies on the Mac OS	86
E3D objects	86
Protecting libraries	86
Programming Techniques	89
Equation block programs	90
Dialogs	90
Changing text in dialogs	90
Changing text as the simulation runs	90
Changing text in response to a user's action	91
Hiding and showing dialog items	92
Moving dialog items	92
Changing the title of a radio button or checkbox	92
Changing and reading parameters globally from a block	92
Remote access to dialog variables	93
Custom remote blocks interfacing with a stand-alone block	94
Custom stand-alone block referencing remote dialog variables	94
Manual data entry	94
Clone drop	94
Shift-click	94
Connectors	96
Variable connectors	96
Deleting connectors or changing connector types	96

Bidirectional connectors	96
Initializing connectors	97
Working with arrays	97
Memory usage of variables, arrays, and items.....	97
Pass by value and reference (pointers)	98
Passing arrays	98
Passing arrays through connectors	99
Passing arrays through global variables	100
Precautions when passing arrays	100
Using passed arrays to make structures	101
Working with global arrays.....	103
Copying arrays using “for” loops	104
Using arrays to import unknown rows of numbers	104
Working with linked lists.....	105
Working with databases.....	105
Using the database API to read and write	105
Registered blocks.....	106
Reserved databases	106
Example	106
Creating and editing.....	106
Reading text blocks as commands.....	107
Passing messages between blocks.....	108
Calling block.....	109
Global function block	109
Changing data while the simulation is running.....	109
Scripting.....	110
OLE and ActiveX Automation	111
COM DLL example.....	111
Controlling Embedded Objects from ModL script.....	111
ExtendSim as an Automation Client	112
ExtendSim as an Automation Server.....	112
Object	112
Topic and Item.....	113
C++ examples.....	113
Retrieving the IDispatch interface	113
Execute	114
Request	115
Poke	115
BlockMsg	116
GetObjectHandle.....	117
Using VBA for ActiveX/OLE Automation	118
Using Visual Basic for ActiveX/OLE Automation.....	118
Animation Using ModL.....	119
2D animation.....	120
Overview.....	120
Deciding how to animate the icon.....	120
Creating 2D animation objects	121
Initializing animation objects	121
Updating the animation object.....	122
Animating hierarchical blocks	122
Showing and hiding a shape	123
Moving a shape	123
Changing a level.....	124
Stretching a shape	125

Showing a picture on an icon	125
Moving a picture along the connection line between two blocks	126
Changing a color	127
Changing text	127
Animating pixels	128
3D animation	128
About the 3D chapter	128
Overview.....	129
The Torque Game Engine and the E3D environment	129
Modes of E3D animation.....	129
Types of 3D ModL functions	131
Distance and time ratios.....	131
References	132
Manipulating object behaviors and appearances	132
E3D object names and labels.....	132
ObjectIDs, userTags and groupTags	132
Object classes	134
Scale and position	134
Rotation.....	135
Skins	135
Mounting.....	137
Smart Mounting	137
MountStack	137
Unmounting	138
Collision	138
Gravity.....	138
Sound	138
Ambient animation	140
Paths, markers, and waypoints.....	141
Defining new 3D objects	142
DTS file format.....	142
Exporting a 3D object file created in another program	143
Scripting and .cs files.....	143
DataBlocks.....	144
DataBlock variables.....	146
Carriable items	148
Buildings and interiors	149
Terrain	149
Simulation Architecture	151
Running a simulation.....	152
How ExtendSim runs continuous simulations.....	152
Pseudocode of the simulation loop for continuous simulations.....	152
Simulation order	153
Initialization.....	153
Step loop.....	155
Final messages	155
Aborting multiple simulations.....	156
How ExtendSim runs discrete event or discrete rate simulations.....	156
Pseudocode of the simulation loop in discrete event/rate simulations	156
Initialization.....	157
Simulation phase	159
Final messages	159
Aborting multiple simulations.....	160
How discrete event blocks and models work	160

Timing for discrete event models.....	160
Scheduling events.....	161
Residence, passing, and decision blocks.....	162
Blocks that post future events.....	162
Residence blocks that do not post future events:.....	163
Zero time events.....	163
Item data structures and indexes.....	164
Real array	164
Cost array.....	164
Integer arrays.....	165
3D array.....	166
Attribute global arrays	166
Basic item messaging.....	167
Push mechanism	168
Pull mechanism.....	169
Passing blocks.....	170
Blocked and query messages.....	170
Blocked messages	171
Query messages.....	171
The Notify message.....	171
Value connector messages.....	172
Message emulation	172
Explicit connector messages	173
Functions in discrete event blocks	174
Globals in discrete event models.....	174
Use of system globals during Simulate message.....	174
Use of Global variables during CheckData or InitSim messages	178
Creating blocks for discrete event models	178
How discrete rate blocks and models work	178
Globals in discrete rate blocks	179
Debugging.....	181
Debugging models	182
Features discussed in the User Guide.....	182
Model profiling.....	182
Adding Trace and Report code	182
Trace code	183
Report code.....	183
Debugging block code without the Source Code Debugger.....	183
Using DebugMsg functions.....	184
Viewing intermediate results	184
Other methods.....	184
Source Code Debugger.....	184
Overview of the debugger	184
Definition of terms	184
Steps for debugging.....	185
Debugger tutorial.....	185
Debugging a block	185
Debugging an entire library.....	187
Setting breakpoints.....	190
Setting conditions	191
Debugger reference	193
Setting a block or library to be in debugger mode	193
Debugger window.....	194
Set Breakpoints window	195

Breakpoints window.....	195
Conditions dialog.....	196
Debugging tips.....	197

REFERENCE

ModL Variables.....	199
System variables	200
Table of system variables	200
Global variables.....	201
Messages and Message Handlers	203
Types of message handlers	204
Simulation messages.....	204
Model Status messages	205
Block Status messages.....	207
Dialog messages	209
Connector messages	210
Block to block messages	211
Dynamic Link messages	212
OLE messages.....	212
3D messages.....	212
ModL Functions	215
ModL function overview.....	216
Code completion	217
Overriding	217
Type conversion of arguments	217
Static data limits.....	217
Function returns	217
Math functions	217
Basic math	217
Trigonometry	219
Complex numbers.....	219
Statistics and random distributions.....	219
Financial	223
Integration	224
Matrices	224
Bit handling	228
Equations.....	228
I/O functions.....	232
File I/O, formatted.....	232
File I/O, unformatted.....	233
Internet access.....	236
Interprocess Communication (IPC)	239
OLE/COM (Windows only).....	241
Mailslot (Windows only).....	249
ODBC.....	250
Serial I/O	253
DLLs and Shared Libraries.....	254
Existing Windows DLLs	254
DLLs for compatibility.....	256
Alerts and prompts.....	256
User inputs.....	258

Animation	259
2D Animation.....	259
Color palette	260
3D Animation.....	264
3D block functions	265
3D time functions.....	266
3D rotation, position, and scale functions	266
3D window functions	267
3D mountStack functions	267
3D path functions.....	268
3D object selection functions	269
3D environment functions.....	269
3D creation and deletion functions	270
3D movement functions	271
3D post function.....	272
3D mounting functions	274
3D animation functions	274
3D name functions	275
3D screenshot functions.....	277
3D distance functions	277
3D skin functions	277
3D vector functions	279
3D user tag functions.....	279
3D tick functions	280
3D object property functions	280
3D script functions	281
3D miscellaneous functions.....	282
Blocks and inter-block communications	282
Block numbers, labels, names, type, position.....	282
Block connectors and connection information	285
Variable connectors	288
Connector tool tips	291
Dialog items.....	291
Block data tables	297
Variable column data tables.....	297
Dynamic data table resizing.....	298
Data table linking.....	298
Formatting individual columns	298
Dynamic linking	302
Dynamic text items	304
Formatting/interactivity using column and parameter tags	305
Column/parameter tag functions.....	308
Dialog item tool tips	309
Block dialogs (opening and closing)	309
Block dialog tabs	310
Messages to blocks (sending and receiving).....	311
Icon classes and views.....	314
Models and notebook.....	314
DE Modeling Using Equation Blocks.....	319
Scripting.....	319
Reporting.....	326
Plotting	326
General plotting.....	327
Scatter plot functions	333
Database functions	333

Error codes.....	333
Creating and deleting database components	334
Selecting parts of a database	336
Copying parts of a database.....	337
Import and export a database	337
Database properties.....	338
Read and write to a database	341
Random data in a database.....	345
Sort and search in a database.....	347
DB address functions	349
Viewing a database.....	350
Linking and notification	351
Arrays, pointers, queues, delay, linked list, and string lookup table functions	352
Dynamic and non-dynamic arrays.....	352
Passing arrays	354
Pointer functions.....	354
.....	355
Global arrays.....	355
Linked lists.....	361
String lookup table functions	366
Queues.....	368
Delay lines	369
Miscellaneous functions	369
Strings.....	369
Attributes	372
Time units	372
Calendar Date functions	373
Date and time, legacy format	375
Timer functions	376
Debugging.....	376
Web and Help connectivity.....	378
Platforms and versions.....	378
Lego Mindstorms control (Windows only).....	379
User-defined functions for ADO.....	380

APPENDIX

Menu Command Numbers	385
Upper Limits.....	397
ASCII Table	401
Cross-Platform Considerations.....	403
Libraries.....	404
Models.....	404
Menu and keyboard equivalents.....	404
Menu and keyboard equivalents for developers	405
Develop menu.....	405
Run menu	405
Transferring files between operating systems.....	405
File name adjustments.....	406
Physically transferring files	406
File conversion	406

Converting model files	406
Converting hierarchical blocks in libraries	407
Libraries	407
Blocks that use the equation functions	408
Extensions	408
IPC or multi-server considerations	408
Cross-platform code	409

INDEX

Overview

Introduction

Where you learn where you need to begin

*“Begin at the beginning,’ the King said, gravely,
‘and go ‘til you come to the end; then stop.’”
— Lewis Carroll*

About the Developer Reference

This Developer Reference is a companion to the ExtendSim User Guide. The purpose of this manual is to reveal the ExtendSim integrated development environment (IDE) so that you can create, modify, and debug ExtendSim blocks.

ExtendSim IDE

The ExtendSim IDE has three components:

- The ModL language for creating logic statements, formulas, and code in equation blocks, as well as for modifying or creating custom-programmed blocks
- A dialog editor for modifying or creating custom block dialogs
- Programming tools, such as include files and external source code capability, that support and simplify your programming efforts

 The Developer Reference presumes that you have read and are familiar with the tutorials in the ExtendSim User Guide.

How the Developer Reference is organized

The Developer Reference is organized into several sections and subsections:

- Overview
 - “Parts of a Block” (which starts on page 6) describes the internal parts (structure) of blocks. This chapter is also helpful for non-developers, to understand how the blocks in their models work.
 - “ModL Overview” (starting on page 24) gives an overview of the “action” part of a block — the ModL code. If you use equation-based blocks — Equation and Optimizer blocks (Value library), Equation(I) and Queue Equation blocks (Item library), or the Buttons block (Utilities library) — you will also find this chapter helpful.
- Tutorial
 - The “Creating a Block” chapter provides a tutorial on how to build a new block; the chapter starts on page 44.
- ModL
 - “The ModL Language” on page 61 describes ModL’s structure and constructs in detail. It presumes you have read the chapters in the Overview module.
 - “Programming Tools” describes all the tools, such as include files and external source code control, that ExtendSim provides to help you program efficiently; it starts on page 78.
 - “Programming Techniques” gives procedures and suggestions for programming using ModL and for interfacing ExtendSim with other languages, such as VBA. That chapter starts on page 89.
 - “Animation Using ModL”, starting on page 119, describes the ExtendSim 2D and 3D capability and features.
 - “Simulation Architecture” gives important information about how ExtendSim runs simulations and how discrete event and discrete rate models and blocks work. The chapter starts on page 151.
 - “Debugging”, which begins on page 184, discusses model and code debugging and shows how to use the ExtendSim source code debugger.

- Reference

Starting on page 200 are three chapters that list and discuss all of the ExtendSim variables, messages, and functions.

- Appendices

- Appendix A: upper limit values for ExtendSim
- Appendix B: an ASCII table
- Appendix C: menu commands and numbers for the ExecuteMenuCommand function
- Appendix D: cross-platform (Windows and Macintosh) considerations

As you will see, programming blocks in ExtendSim is quite easy, especially if you have ever programmed in any language.

Additional resources

Imagine That Inc. provides several resources to support your simulation experience.

- 1) **Getting Started.** The Getting Started model opens when you launch ExtendSim. Use this interface to explore sample models and to view tutorials on building and running models.
- 2) **Online help.** Access the electronic User Guide and Developer Reference by giving the command Help > ExtendSim Help or press F1 on your keyboard. Use tooltips to identify interface elements. Get complete definition of how a block works, including descriptions of its dialog items and connectors, by clicking a block's Help button.
- 3) **Web advice.** FAQs are available at www.ExtendSim.com/support/advice/faq.html.
- 4) **Networking.** Links for ExtendSim user forums, networks, and blogs are at www.ExtendSim.com/support/advice/forum.html. For example, the ExtendSim E-Xchange is a user forum for sharing ideas, insights, and modeling techniques with other ExtendSim users. Use this forum to post issues and solutions, share blocks and models, and to talk directly to other people developing simulations. You must register to join, but access is free and available to all ExtendSim modelers.
- 5) **Complimentary support.** Get technical assistance for installation issues, basic usage questions, and troubleshooting for up to 60 days after purchasing a new product or upgrade.

Contacting Imagine That Inc. Technical Support



You must be a registered customer to receive technical support from our support staff. Register online when you install ExtendSim (Windows only), register online after installation using the Register.exe file in the ExtendSim9\Online Registration folder (Windows only), or mail or fax the registration card that was included in your ExtendSim package (Macintosh only).



Technical support is complimentary for the first 90 days after purchase. After that, you must either subscribe to the ExtendSim Maintenance Plan or purchase per-incident support.

When you contact our support representatives, please provide the following information:

- 1) **ExtendSim serial number, product name, and release number:**

Windows: Located in the Help > About ExtendSim menu command, on the title page of the User Guide, or on the tear-off remainder of your registration card.

Mac OS: Located in the ExtendSim > About ExtendSim menu command, on the title page of the User Guide, or the tear-off remainder of your registration card.

- 2) **Name and contact information** (telephone, email, and/or fax), so we can reply.
- 3) **Type of computer.**
- 4) **Operating system and version.**



Be sure you are using the most current version of ExtendSim; updates are available at our web site (www.extendsim.com)

Overview

Parts of a Block

An introduction to the internals of a block

*“We shape our buildings;
thereafter they shape us.”
— Winston Churchill*

In the User Guide you saw how to use ExtendSim with the blocks that are provided in the package. The Developer Reference teaches how those ExtendSim blocks work and how to build a custom (non hierarchical) block with its own dialog and code. Once you understand what goes into a block, it is an easy task to modify existing blocks and create new ones.

This chapter discusses the ExtendSim integrated development environment (IDE), specifically the parts of a typical ExtendSim block.

📖 Read the Tutorial at the beginning of the User Guide before reading this chapter. Many of the concepts used when building blocks assume an understanding of how blocks are used in models.

Table of block parts

Blocks can have six parts, as listed below and discussed later in this chapter.

Part	Page	Description
Icon	18 46	The icon is the visual representation of the block as seen in a model. Icons are created using the ExtendSim drawing environment or a painting program, or by copying/pasting clip art. Icons can have a <i>model style</i> as well as one or more <i>views</i> within each style. An example of a model style would be Classic, and views within that style might be Forward and Reverse. Icons can display 2D animation and most often have connectors that enable blocks to exchange data.
2D Animation	19 54	Objects for 2D animation are part of the icon. 2D animation can show motion, levels, and values, and can alter an icon's static look. Depending on the block's code, 2D animation can display anywhere in the model, even outside the icon's footprint.
Connectors	20 46	Connectors appear as part of the icon and transmit information to and from each block's ModL code. Connectors can be single or variable. Variable connectors expand and contract, so you can control how many connectors are displayed. (Note that blocks can also transmit information without using connectors by sending messages to other blocks and by using global variables, global arrays, or an ExtendSim database.)
Dialog	7 11	The dialog is the window that appears when you double-click a block's icon. You specify the frames, text, buttons, tables, and entry boxes that go into the dialog (collectively known as <i>dialog items</i> and discussed on page 11), and these determine the dialog's size. You can place all the dialog items in one dialog, or separate them into functional groupings using <i>tabs</i> . The title field at the top of the dialog displays the block's <i>name</i> and the library it resides in. There is a Help button to the left of the bottom scroll bar as well as a text field (for the block's user enterable <i>label</i> field) and a View popup menu (if the block has multiple icon views).
Help text	22	The text that appears when you click the Help button to the left of the dialog's bottom scroll bar. When the Help dialog is open, buttons at the bottom can be used to find a block, or display information about blocks, in all open libraries.
ModL code	18 24	Code is what makes a block work; ModL is the programming language for ExtendSim. ModL code can read and write information via the connectors, dialog, ExtendSim Database, the model environment, and other blocks, as well as control all the parts of a block. Unlike the other block parts, which can be seen by the modeler, ModL code can only be accessed through a block's structure window.

 Objects in the E3D window are not ExtendSim blocks. Instead, they are representations of the ExtendSim blocks that are in the associated 2D model window.

These parts comprise a block's internal structure and are interconnected. For example, the ModL code reads information from the connectors, the Help text is displayed through the dialog, and so on. All of the block's parts can be controlled by ModL code.

Dialogs

Every block has a dialog. A dialog may be as simple as just the OK and Cancel buttons, or it may be quite complex. Setting up a dialog is easy because ExtendSim has a built-in dialog editor.

A dialog contains *dialog items*, a *Help button*, a *block label field*, and a *View popup menu*. It may also contain multiple *tabs*.

Exploring an existing block's dialog

The Simulation Variable block (Value library) is a good example for investigating the internals of a block. So that the original block doesn't get compromised, a copy of that block, named "Example" and stored in the Tutorial library, will be used for this chapter.

 Any changes you make to any block in an ExtendSim library will affect that block in all existing models and will get discarded whenever the library is updated. Thus, it is important that you don't make any changes to the ExtendSim blocks that are shipped with your product. Instead, make a copy of the block and save it in a new library, as was done for this example.

- ▶ Open a new model worksheet.
- ▶ Open the Tutorial library. It is located at \Libraries\Example Libraries.
- ▶ From the Library menu, select the Example block (it is located in the Tutorial library's Inputs category). This will place the Example block on the model worksheet.

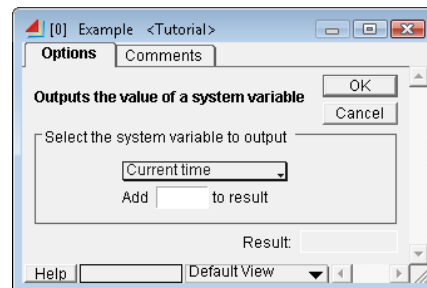
Dialog of the Example block

Double-clicking the icon of the Example block opens its dialog, revealing two tabs:

- An Options tab dealing with the system variables
- A Comments tab for user comments

The frame, text, buttons, and entry boxes that are seen in the tabs are collectively known as *dialog items*.

In each block's dialog, a Help button, block label field, and View popup (if there is more than one icon view) are always located at the left of the bottom scroll bar.



Dialog of Example block

Dialog tabs

Tabs are similar to pages in a dialog and are used to group dialog items by function. Click a tab name at the top of the dialog to change which tab is foremost.

Using menu commands you can:

- Add tabs to a block's dialog
- Add new dialog items to a tab

- Rename or delete the tab
- Move dialog items from one tab to another



In order to delete a tab, the tab must not contain any dialog items. Instead the dialog items must be moved to another tab or deleted. However, see the caution on page 15 about deleting dialog items from blocks that are used in models.

Comments tab

The Comments tab of the Example block has the following dialog items:

- A static text label (“Outputs the value of a system variable”)



By default, dialog items appear on one tab. They can also be set to appear on all tabs, as was done with this text label. By default the OK and Cancel buttons appear on all tabs.

- A dynamic text box, with scrollbars, for entering comments about the block’s use
- An OK button
- A Cancel button

Options tab

The Options tab has the following dialog items:

- Four static text dialog items used as a labels:
 - Outputs the value of a system variable
 - Add
 - to result
 - Result
- A text frame with the text caption “Select the system variable to output”
- A popup menu with several menu items for selecting which system variable to output
- Two parameter fields:
 - One for entering a value to add to the result of the system variable
 - Another that has been set to display the final result but not allow it to be edited
- An OK button
- A Cancel button



As you will see later in this manual, the OK and Cancel buttons, as well as the static text label “Outputs the value of a system variable”, are set to appear on all tabs.

Exploring an existing block’s structure

The internal structure of a block is where the ModL code, dialog items, and other block components are defined.

To open a block’s internal structure

There are four methods you can use to access an existing block’s internal structure:

- 1) Select the block by clicking once on its icon in a model. Then choose Develop> Open Block Structure.
- 2) Right-click the block’s icon in a model. Then select Open Structure.

- 3) Double-click the block's icon in a model while holding down the Alt (Windows) or Option (Mac OS) key.
- 4) Double-click the block's icon in the Navigator or Library Window while holding down the Alt (Windows) or Option (Mac OS) key. (To open the Library Window, choose Open Library Window at the top of a library's menu.)

Examining the internal structure

- Open the internal windows for the Example block using one of the methods from above.

When you look at a block's internals, you see two windows overlapping on the screen.

- By default, the top window is the *dialog editor window* which shows just the block's dialog.
- The window behind it is the *structure window*. This window holds all the structure of the block other than its dialog.

The title bars of the structure and dialog editor windows display the name of the block and the name of the library the blocks resides in.

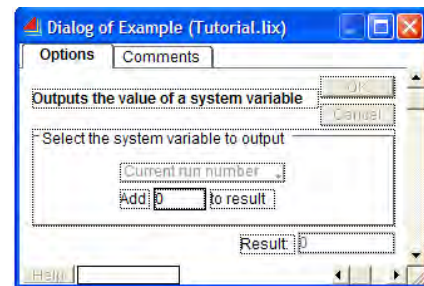
- ☞ Use the menu command Edit > Options > Programming tab to have the default be that the structure window opens in front of the dialog editor window.

Dialog editor window

The dialog editor window for the Example block looks like the screenshot at right.

The block has two tabs: Options and Comments. The Options tab has several dialog items, including a text frame and OK and Cancel buttons. The Help button (which causes the Help text from the structure window to show) and the block label field are located at the left of the bottom scroll bar.

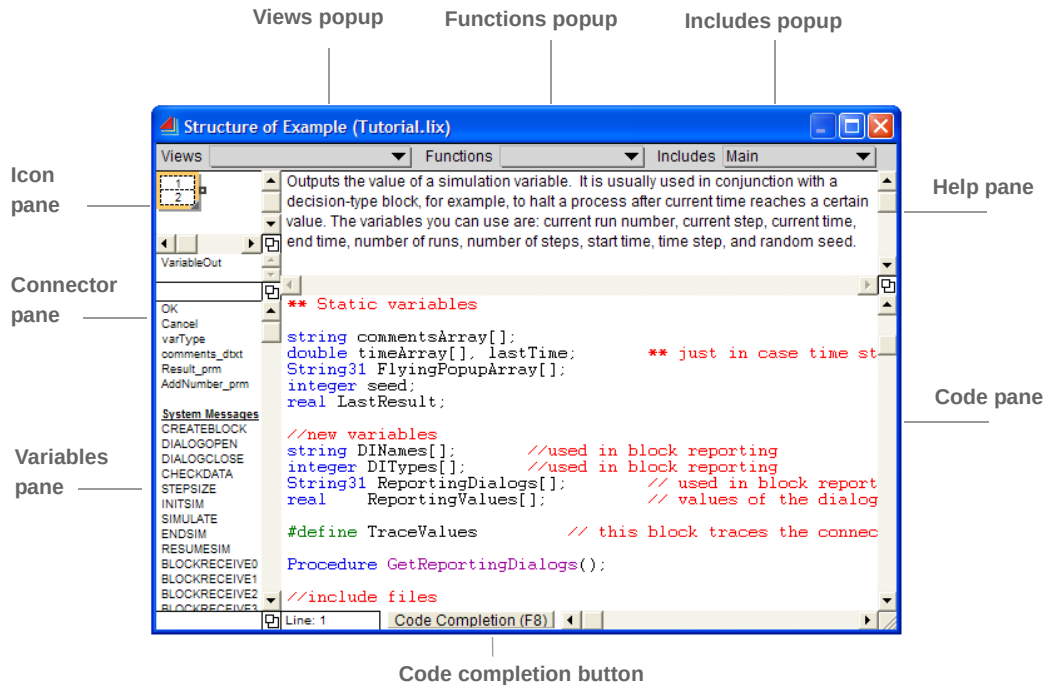
Dialog items and the dialog editor window are discussed in more detail starting on page 11.



Dialog editor window for Example block

Structure window

With the Code pane scrolled down a bit, the structure window of the Example block looks like:

**Popups**

Along the top of the structure window are three popups:

Views

Used to create, edit, and select model styles and icon views. See “Icon views and model styles” on page 19. For more information, see “Icon views” on page 59.

Functions

Selecting a function in this popup will take you to the section of code that uses the function. This popup also displays which function you are editing.

Includes

Used in opening *include* files.

Icon pane

This pane is used to enter and edit the icon(s) for the block, as well as block connectors and animation objects. The ExtendSim drawing tools can be used to create an icon or existing pictures can be pasted into the pane. Each block can have many different icons, depending on the number of model styles and icon views created. See “Icons” on page 18.

Help pane

This pane is used to enter and edit the online Help for the block. After building the block, the block's online Help is easily accessible by clicking the Help button in the lower left of the block's dialog.

Code pane

This pane is used to enter and edit the block's ModL code. As the code is entered, ExtendSim colors the code to make it more readable. As you enter ModL functions, code completion (discussed below) helps you to get the correct spelling and syntax. When compiling the block, this window will show the position of code syntax errors. See "ModL code introduction" on page 18.

To change the *ModL font* used to display the code, change that option in the Programming tab of the Edit > Options dialog.

Code completion button

After typing part of a function or message handler name in the Code pane, click this button (or press F8) and ExtendSim will complete the text. Keep clicking the button until the desired function or message handler appears. The function will be entered with sample arguments.

Variables pane

This pane shows the variable names defined by the block's dialog items. As a reference, it also displays all system variable names and message names.

Connector pane

This pane is used to edit the default connector names so they can more closely represent their function. See "Naming connectors" on page 21.

Working with the dialog editor and structure windows

By default, the dialog editor window opens in front of the structure window. If you instead want the structure window to open in front, select the command Edit > Options > Programming tab and choose *Structure window opens in front*.

You can change the size of any of the panes by dragging any of the resizing  boxes. To print parts of the dialog editor or structure windows use the File > Print command. Choose what you want printed (icon, connector names, dialog box tabs, code text, and/or help text) from the Print dialog that appears.

To close these windows, choose File > Close or click the close box in the upper corner of either window. If you have made any changes to either the dialog editor or structure windows, ExtendSim will prompt you before saving to "Save, Compile if Needed," "Save Without Compiling," or "Discard Changes." Compiling causes changes to be reflected in the code of the block.

Dialog items

The frames, text, buttons, tables, and entry boxes that go into a dialog are collectively known as *dialog items*. When creating a block, the dialog items are defined using the ExtendSim dialog editor. This allows you to select a desired type of dialog item and which options it will have. Each dialog item has its own definition – a specified type, format, options, and so forth. The items in a dialog depend on how the block was defined.

Dialog item definition

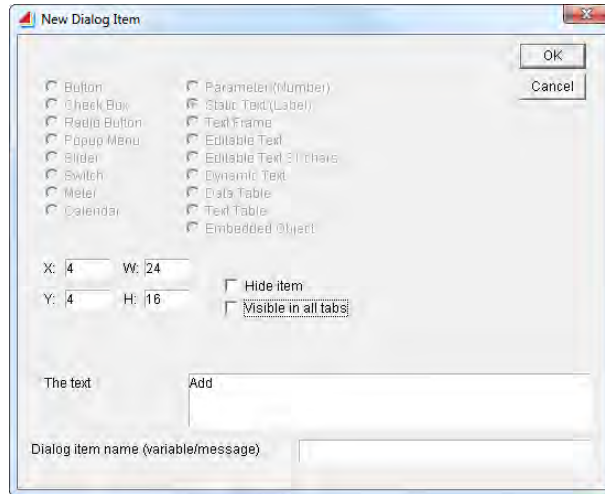
To see the definition of a dialog item:

- ▶ Open the block's internal structure using one of the methods on page 8
- ▶ Double-click the dialog item in the block's dialog editor window

For instance, double-clicking the “Add” static text label in the Example block’s dialog editor window opens the New Dialog Item dialog shown here.

This window displays the definition of the selected dialog item. In this case, the information is:

- The dialog item is of the Static Text (Label) type
- The text that appears in the dialog is “Add”
- There is no dialog item name associated with this dialog item
- The dialog item is located on the dialog at a particular X, Y location and the text has a height of 16 pixels and a width of 24 pixels.



New Dialog Item dialog, showing definition of “Add” label

The New Dialog Item dialog also shows all the possible types of dialog items and their options, as discussed below.

Types of dialog items

The types of dialog items are summarized in the following table. They are discussed in more detail on the specified pages of the next chapter.

Type	Page	Description	Options
Button	36	A button that can be clicked, such as an OK or Cancel button. Buttons can have titles that appear as text inside the button; the titles can be changed using ModL code.	Hide item Visible in all tabs Right click menu Title
Check Box	35	Square buttons that have an “X” in them when they are selected and are empty when they are not. Check boxes can have titles that appear as text to their right; the titles can be changed using ModL code.	Hide item Visible in all tabs Title
Radio Button	35	Round buttons that appear in groups where each group has a unique group number. Only one button in the group can be selected at a time; this causes all the other buttons in the group to become deselected. Radio buttons can have titles that appear as text to their right; the titles can be changed using ModL code.	Hide item Visible in all tabs Groups Title
Popup Menu	37	A shadowed rectangle containing a menu of items. Each item in a popup menu has a title which can be changed using Modl code.	Hide item Visible in all tabs Titles

Type	Page	Description	Options
Slider	38	A control that allows you to select from a range of values by moving a value indicator. You can manually drag the knob to change a value or use ModL code to move the knob to show a value.	Hide item Visible in all tabs
Switch	38	A switch that resembles a light switch. It has two values, 0 and 1 (off and on).	Hide item Visible in all tabs
Meter	38	An output-only item that shows a needle in a meter.	Hide item Visible in all tabs
Calendar	39	Takes an ExtendSim date value and displays it on a calendar.	Hide item Visible in all tabs Tab order
Parameter (Number)	34	Entry box that takes and/or displays a number. Users can dynamically link parameters to a cell in an ExtendSim Database or global array instead of just entering data.	Display only Hide item Visible in all tabs Number formats Tab order
Static Text (Label)	37	Text that appears as a label in the dialog. Can be changed through ModL code but not by the modeler. Does not require a dialog item name.	Hide item Visible in all tabs Text for label
Text Frame	38	Used to group dialog items. Has an optional text caption that will appear at the top of the frame. Does not require a dialog item name.	Hide item Visible in all tabs Text for frame
Editable Text	34	Entry box that takes or displays text. Limited to 255 characters. Users can dynamically link editable text to a cell in an ExtendSim Database or global array instead of just entering data.	Display only Hide item Visible in all tabs Tab order
Editable Text 31	34	Same as Editable Text, above, but limited to 31 characters to save memory.	Display only Hide item Visible in all tabs Tab order
Dynamic Text	34	Entry box that takes or displays text. Limited to 32,000 characters. This item uses an automatically resized dynamic array to store the text. It is useful for larger amounts of text such as for comments or for equations in the Equation and Optimizer blocks.	Display only Hide item Visible in all tabs Tab order
Data Table	36	A two-dimensional table with scrollbars and adjustable column widths (similar to a spreadsheet) for holding numbers. If block code provides for it, modelers can change the number of rows and columns and can dynamically link tables to an ExtendSim Database or global array.	Hide item Visible in all tabs Number formats Title Rows, columns (See Column Tag functions for more options)

Type	Page	Description	Options
Text Table	36	A two-dimensional table with scrollbars and adjustable column widths (similar to a spreadsheet) for holding text. If block code provides for it, modelers can change the number of rows and columns and can dynamically link tables to an ExtendSim Database or global array.	Hide item Visible in all tabs Title (See Column Tag functions for more options)
Embedded Object (Windows only)	39	A placeholder for an OLE ActiveX object which can be manually inserted by the modeler or inserted by the block's ModL code.	Hide item Visible in all tabs

Options for dialog items

Dialog item names and text/titles

Each dialog item can have a dialog item *name* and many types can have a *title* or *text*:

- Dialog item names are the variable names and message names used by the block's code to interact with the dialog, as discussed in "Accessing dialog items from a block's code" on page 33. Names are required for most dialog items; they are optional for text frames and static text dialog items. Dialog item names must start with a letter, cannot contain any non-alphanumeric characters, and can be up to 31 characters in length.
- Depending on the type of dialog item, some dialog items have optional titles or text. Text and titles can be up to 255 characters in length. If defined, titles and text are displayed along with the dialog item in the block's dialog.

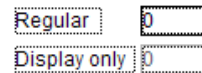
X,Y,W,H

The New Dialog Item window has fields (X, Y, W, and H) for setting the size and position of dialog items as they are created. Their size and position can also be adjusted after they appear on the dialog editor window.

Display only

Under normal circumstances, parameters, editable text, editable text 31, and dynamic text dialog items are editable in the dialog. Checking the *Display only* option means that the dialog item cannot be changed by the modeler in the dialog. Instead, it can only be set through ModL code.

In the dialog editor window, display-only items have a gray outer border instead of the standard black border; this is shown at right. They also appear as dimmed in the block's dialog.



-  Using ModL functions, the state of any dialog item can be dynamically changed between editable and not editable.

Making a dialog item display only prevents modelers from accidentally changing dialog values that the block calculates. For example, the queue blocks (Item library) display information about queue length, average wait time, and so forth. Those parameters are set to only display the results. Display-only dialog items can still be copied and cloned.

Hide item

The Hide item option provides a safe alternative to deleting a dialog item when you no longer want it or want to change its nature in an existing block. For example, if a dialog has a check box option, you might want to change it to a series of radio buttons. You would have to delete the dialog item to do this.

 Deleting a dialog item from a block that is used in any model could disrupt the order of the data in the dialog. The data will have to be reentered for each instance of that block in all models that use it. Instead of deleting the dialog item, hide it using the *Hide item* option.

When the *Hide item* option is selected, the item will not appear in the block's dialog. It will, however, show in the dialog editor window as a dimmed rectangle without text. Hidden dialog items can be moved in the dialog editor window but they cannot be resized. You can also revert a hidden dialog item so that it appears in the dialog. To restore a dialog item from hidden to visible, double-click the item in the dialog editor window and unselect the *Hide item* option.

 You can also temporarily hide and show dialog items using the HideDialogItem function in the dialog item function list that starts on page 291. Also consider using the DIMoveTo and DIMoveBy functions to move dialog items out of the way depending on dialog settings.

Visible in all tabs

If a dialog has tabs, the *Visible in all tabs* option will cause the dialog item to appear on every tab. This is common for static text that describes the purpose of the block.

Add to right click menu

Button items can be added to the menu that appears when a block on the model worksheet is right-clicked. These commands can then be executed without having to open the block's dialog to click the button.

Tab order

The tab key allows you to move between entry boxes on the dialog. Each calendar, parameter, editable text and dynamic text box in a dialog editor window has a tab order number. To change the order, click the Higher and Lower buttons in the New Dialog Item dialog for that item. When the tab order is changed for a dialog item, the tab order for the other dialog items is automatically adjusted.

 Be careful when changing the tab order of dialog items. If the block is used in a model, changing a dialog item's tab order can cause the block's cloned dialog items to become confused. In this case the clone will present itself with multiple question marks; it must be deleted and replaced.

Number formats

For some dialog items you can specify the numeric format of the number as it appears in the box. The options are:

- General
- Currency (2 decimal places)
- Integer
- Scientific
- Percentage

The number format is selected when the dialog item is defined in the New Dialog Item dialog.

Working with dialog items

 In addition to the following information, modifying and creating dialogs, as well as ModL code interaction with dialogs, is described in more detail in “Accessing dialog items from a block’s code” on page 33.

Adding new dialog items

To add a new item to a dialog editor window or to a tab within that window:

- ▶ If the dialog has tabs, select the tab you want to work on to bring it to the front
- ▶ Choose Develop > New Dialog Item
- ▶ Select the type of item you want and choose its options
- ▶ Drag the item to the desired location within the tab or dialog editor window

Copying dialog items

You can copy a dialog item from one tab to different tab within the same block, or from one block to another. To do this, select the dialog item, then use the Edit > Duplicate (or Edit > Copy and Edit > Paste) commands.

When you duplicate or copy/paste a dialog item, any text or title stays the same but a number is appended to the original dialog item name. Unless you change it, the new name, with the appended number, is how the copy of the dialog item will be referenced in ModL code. The name and text/title can be changed by double-clicking the dialog item in the dialog editor window to access its definition.

Moving dialog items

To move an item within a dialog editor window or dialog tab:

- Either select and drag it to the new location within the same window or tab
- Or, double-click it and change its X and Y coordinates in the New Dialog Editor dialog

To move a dialog item from one tab to another, first select the item, then use the Develop > Move Selected Items to Tab command.

ExtendSim places dialog items on a grid; to position them without the grid, press the Alt (Windows) or Option (Mac OS) key as you drag. To select multiple dialog items at once, press the Shift key or drag a frame around the items.

Resizing dialog items

Many dialog items can be resized. To resize an item in a dialog editor window:

- Either select the item and drag one of its handles (the black squares in the corners of the item)
- Or, double-click the item and change its W (width) and H (height) coordinates in the New Dialog Item dialog

Styles, alignment, and colors for dialog items

Formatting can be an aid to comprehension and organization in a block’s dialog. As discussed in more detail below, there are two ways to format dialog items:

- 1) The style and alignment of text and titles can be formatted when the dialog item is defined
- 2) Dialog items can be colorized using functions in ModL code

Formatting while defining the dialog item

The following dialog items can have the style and alignment of their text or titles formatted when they are defined:

- Popup menu item titles
- Static text
- Text frame titles
- Data table titles
- Text table titles

The text or title can be formatted to be:

- Bold (B or b), italicized (I or i) or underlined (U or u)
- Shadow (S or s) or outline (O or o) – Macintosh only
- Left (L or l), right (R or r), or centered (C or c)

You format these dialog items while defining them in the New Dialog Item dialog. This is accomplished by preceding the text or title with the

The text

|Outputs the value of a system variable

Dialog item text formatted as bold

initial of the desired format within angle brackets. For instance, preceding the text by “” would format it as bold, as shown in the screenshot above.

To combine formats, put multiple initials within the angle brackets. For example, text that is bold, italicized, and right adjusted could be formatted by preceding it with “<bir>”. Formatting for dialog item text and titles is not case sensitive and the order of the initials for combined formatting does not matter. Thus bold italics could be <ib>, <BI>, <iB>, and so forth.

 The formatting will not appear in the dialog editor; the text or title only appears as formatted in the block's dialog.

Formatting through ModL code

Using the SetDialogItemColor function, some dialog items can be formatted to be a specific color. Other functions can change the color, making this feature useful for special situations.

When a dialog item has color, each of its component will appear in that color. For example, formatting a popup menu with color causes the text of each item in the menu to be colorized, as well as the border around the popup.

Tool Tips on block dialog and dialog editor windows

As mentioned earlier, each dialog item can have a name. Dialog item names are how the items are referenced in ModL code. Depending on the type of dialog item, a name may be required (such as for a parameter) or may not be necessary (for instance, for static text used as a label).

You may notice that the name of a dialog item is not visible in the dialog editor window or when you look at the block's dialog. If a dialog item has a name, Tool Tips provide that information without you having to access the dialog editor to see the dialog item's definition.

With Tool Tips turned on, resting the cursor above a dialog item displays a small window containing the name of the dialog item; the window will be blank if there is no dialog item name. Tool Tips for the dialog items on block dialogs and dialog editor windows can be turned on or off using the command Edit > Options > Programming tab.

ModL code introduction

ModL is the internal programming language for ExtendSim. A block's ModL code is located in the lower right pane of its structure window.

☞ When you build blocks using ModL, the block's program is compiled to native machine code.

ModL and other languages

ModL is essentially C++ with some enhancements and extensions to make it more robust for simulation modeling. If you are familiar with the C++ programming language or with some other language, you should be able to follow the general logic of the ModL code and get a feeling for how it works. (See page 24 for tables that compare ModL to C++ and other languages.)

ModL structure

If you know a programming language, you will probably recognize the structure of the ModL code when you look at the code pane of a block's structure window. To make it easier to follow the logic of a block's program, ExtendSim colors the code depending on the syntax.

After the copyright and modification history information, the first lines are the declaration of the types of variables used in the code. Lines that begin with `//`, `/**`, or `/*` are comments and are colored red.

The lines of programming code are grouped into sections, where each section is either a *message handler* or a *function*.

- Message handlers begin with a line `"on messageName"`.
- Functions are of the form `"void functionName(type argument, ...)"` or `"type functionName(type argument, ...)"` depending on whether they return a value.

For example, a message handler in every block begins with `"on Simulate"` and the code within the message handler starts and ends with curly braces (`"{"` and `"}"`).

☞ See the "ModL Overview" chapter for an overview of ModL, including messages, functions, and variables. The "The ModL Language" chapter is a reference on the ModL language; message handlers, variables, and functions also have their own reference chapters later in this manual.

Icons

A block's icon is its most obvious aspect since it appears in the model window. You can see the icon in the upper left pane of the structure window. An icon consists of a drawing or group of drawn objects, the block's connectors, and possibly one or more animation objects.

Creating the icon

There are two ways to create an icon:

- With ExtendSim's drawing tools. The ExtendSim toolbar provides a minimal drawing environment for quickly creating icons.
- By pasting drawings from the Clipboard. You can use a painting or drawing program to create an icon. Then paste it into the icon pane.

Use the same tools for selecting objects in the icon pane as you use to select blocks in the model.

☞ ExtendSim automatically provides a grid for placing objects. To use fine placement for objects, override the grid by pressing the Alt (Windows) or Option (Mac OS) key before selecting and moving the object.



Icon pane

Icon views and model styles

As shown on page 10, a block's structure window has a Views popup menu that is used when creating icon views and model styles.

Model styles

If this feature is utilized when the block is created, blocks can have alternate styles that affect their appearance in a model. For instance, blocks could have a model style for one purpose and another model style for a different application or market. In that case, changing the style would cause the model to have a completely different appearance.

Model styles are created and named using the View popup menu. If you select a style that already exists, the icon views in the current style will be appended to the views in the new style.

The style for the model is selected in the command Edit > Options > Model Styles tab. A radio button indicates the style that will be used as a default when a new model is created. (The libraries that ship with ExtendSim use the Classic model style.) Model styles are saved as indexes so you can rename the style and it will still work on a different computer.

Icon views

Within each style, an icon can have many views. For example, a block that represents a flow could be pointed forward in its default view labeled *Forward* and pointed backward in a different view labeled *Backward*. ExtendSim accomplishes this by facilitating the creation of different views of a block's icon, as shown in "Icon views" on page 59.

 All views and styles have the same number of connectors, but connectors can be selectively hidden in a view when it is created or hidden and shown dynamically using ModL functions. ExtendSim automatically adds and deletes connectors from all views when the modeler modifies the number of connectors on a block.

Animation

2D animation

There are several ways to animate a block using 2 dimensional animation objects. Typically, the block's icon would be animated; however, it is also possible to animate outside the icon's footprint. Blocks can show, hide, and change the patterns and colors of text and shapes; move a shape and increase or decrease its size; show a changing level; show a picture or a movie; or move a picture along connection lines between two blocks. The Item and Rate libraries use 2D animation extensively.

In the "Icon pane" screenshot of the Example block shown on page 18, the white rectangles at the center of the icon (labeled with the numbers 1 and 2) are *animation objects*. When the block is created, one or more animation objects can be added in the icon pane of the structure window. The block's ModL code interacts with the object to show animation as the simulation runs. For the Example block, the animation displays which system variable has been selected.

The ExtendSim User Guide gives an overview of 2D animation, including block-to-block animation that flows along the connections in discrete event models. In the Developer Reference, "Adding 2D animation" on page 54 shows how to add 2D animation objects to a custom block and the section "2D animation" on page 120 has a lot more information about programming 2D animation effects.

There are also functions that facilitate customizing animation depending on which model styles and icon views are being selected by the modeler. See the functions for “Icon classes and views” on page 314.

3D Animation

In addition to 2D animation, the ExtendSim Suite package supports 3D animation through ModL code. Unlike 2D animation, which is shown on the model worksheet, 3D animation is displayed in a separate (E3D) window. Functions in the blocks cause objects in the E3D window to be animated; these objects represent the items and blocks of the 2D model. Blocks in the Item library, and some blocks in the Rate library, support 3D animation.

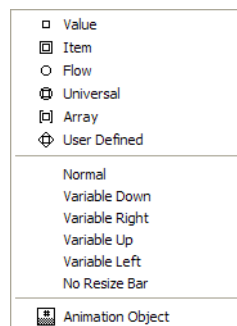
The chapter that starts on page 128 has a lot more information about programming 3D animation objects and effects. When programming your own blocks, there are hundreds of 3D functions for placing objects in the E3D window, object motion, rotation, scaling, and so forth. The complete list of E3D functions starts on page 264.

Connectors

Most blocks have connectors that transmit information to and from the ModL code. Connectors are added to the icon in the Icon pane using tools from the toolbar. When the structure window is the front-most window, the toolbar changes to enable the Icon Tools popup menu shown to the right.

Connector types

The top six tools on the Icon Tools popup are for selecting the type of connector. The ExtendSim User Guide describes the use of the *value*, *item*, *flow*, *universal*, and *array* connectors. The *user-defined* (or “diamond”) connector does not have any “special” properties, and is provided for your convenience for custom applications. For example, if you are designing a new set of blocks and want to be sure that modelers only connect those blocks to each other, you would use this connector. If the modeler tries to connect a diamond connector to a value or item connector, ExtendSim will not let them.



Icon Tools popup

Connector options

The next six tools in the Icon Tools popup menu are options that determine if the connector will be a *normal* connector or a *variable* connector.

Normal and variable connectors are illustrated in the User Guide. By default new connectors are added as a normal (single) connector. Variable connectors act like a row of single connectors, where the row can be expanded or contracted to provide a required number of connectors. The choices in the Icon Tools popup allow you to select either a normal connector or a variable connector that the modeler can expand downward, to the right, upward, to the left, or cannot be manually expanded or contracted (*No Resize Bar*).

In the block's code, you can specify a maximum and minimum number of variable connectors and change the number of connectors dynamically. An example of using normal and variable connectors is the Math block (Value library).

 Regardless of type, each connector can be normal or variable depending on the option selected in the Icon Tools popup. However, the entire row in a variable connector must be of only one type (Value, Item, etc.).

Animation object tool

The Animation Object tool at the bottom of the popup menu, , adds 2D animation objects to a block's icon. This is discussed in “Adding 2D animation” on page 54.

Adding connectors to the icon

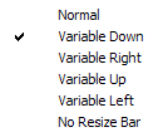
To add a connector to an icon:

- ▶ Click on one of the connector type tools (Value, Item, Flow, Universal, Array, or User Defined)
- ▶ Then click in the icon pane at the desired position

This will place a connector in the icon pane. The default is that the connector is a “normal” connector.

To change the default normal connector to a variable connector:

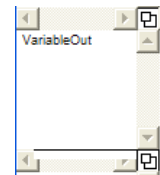
- ▶ Select the connector in the icon pane
- ▶ Choose one of the variable choices in the Icon Tools popup so that it is checked



Naming connectors

Every connector has a unique name. In addition, the name of the connector defines whether it is an input or output connector:

- Connector names must start with a letter, cannot contain any non-alphanumeric characters, and must be less than 32 characters in length.
- Input connectors must end in “In” or “in”.
- Output connectors must end in “Out” or “out”.



Connector pane

Connector names are shown in the connector pane. For example, the name of the connector for the Example block is as shown at right. This block has one “Normal” value output connector named VariableOut.

When you add connectors to the icon, they are all initially input connectors. To make one of these connectors an output connector, change its name to something that ends with “Out.” You can change the connector name to whatever you want, but the name must end in either “In” or “Out.” To change the name of a connector:

- ▶ Select the connector name in the connector pane. ExtendSim highlights that connector in the icon pane so you can identify it.
- ▶ Type a new name or edit the name.
- ▶ Press the enter or return key or click anywhere else in the connector pane to save the edited name.

Changing connector types

If you selected the wrong type of connector (such as a value connector when you wanted an item connector), you can easily change its type:

- ▶ Select the connector on the icon pane by clicking on it.
- ▶ Click on the correct connector type in the toolbar.



When you build blocks it is important that you use the above method to change connector types, rather than deleting a connector. If you delete a connector for a block that is used in any model,

the order of the connections will be disrupted and existing connections to the block might become incorrect.

Help text

You can enter Help text in the upper right pane of the structure. You can also add formatting to the text in the Help window by selecting it and giving commands from the Text menu. This text is about the block and how it can be used in a model. It appears when you click the Help button in the lower left corner of a block's dialog but is not available through the Help menu command.

- By default the block's name and the first sentence of the Help text is displayed as a Tool Tip when the cursor is hovered over the block. You can choose that just the block name, but not the Help text, be displayed. To do this, turn off the "Show additional block information" option in the Edit > Options > Model tab.

Overview

ModL Overview

Creating ModL code and dialogs for custom blocks

*“I must create a system,
or be enslaved by another man’s.”
— William Blake*

This chapter and the others that follow will teach you how to create new blocks, modify existing blocks, or call functions from equation-type blocks such as the Equation and Optimizer blocks (Value library) and the Equation(I) and Queue Equation blocks (Item library).

☞ These chapters only describe creating standard blocks, not hierarchical blocks. Creating hierarchical blocks is described in the User Guide.

As you saw in the preceding chapter, there are many parts of a block, the most complex of which is the ModL code. This chapter shows how a block's code is laid out and the basic ways that ModL code lets blocks interact with other blocks, with their own dialogs, and with the general parameters of a simulation.

☞ This overview of the ModL language is a prelude to, and should be read before, the “The ModL Language” reference chapter that starts on page 61.

ModL and other programming languages

A block's code is written in ModL, a programming language that is much like C or C++ that has been enhanced for simulation purposes.

- **If you have some familiarity with C or C++:** Although ModL is specialized for simulation, it uses many concepts from C++ and you can certainly program in ModL. Note that you do not need to know much C/C++ in order to be completely comfortable in ModL. The table that starts on page 24 lists major differences between ModL and C++.
- **If you program in a language other than C/C++:** ModL will not present too much of a challenge. However, to get the most out of ModL, you should learn C++. The table that starts on page 26 compares common constructs for ModL and for other common languages.
- **If you do not program:** You can still use the equation-based blocks, make some modifications to existing blocks, and build simple blocks without programming experience. However, to get the full benefit from the ModL language, you should have some programming capability.

☞ Whether you know programming or not, the equation-based blocks provide access to ModL functions and variables. This is useful for accomplishing specialized tasks without having to program a block. The equation-based blocks are: Equation or Optimizer (Value library), Equation(I) or Queue Equation (Item library), and Buttons (Utilities library).

ModL and other languages

The next table compares ModL to C++; a second table shows how constructs in ModL compare to constructs in other common languages.

☞ DLLs and Shared Libraries provide a method for incorporating other technologies, such as Visual Basic, Java, or Visual C++, into ExtendSim. For more information, see “DLLs” on page 83.

ModL compared to C++

There are only a few differences between ModL and C++. They are:

ModL	C++
case insensitive	case sensitive
real or double (Mac OS and Windows: 16 significant digits)	double

ModL	C++
integer or long (32 bit)	long
string or Str255 (255 characters maximum)	typedef struct Str255 { unsigned char length; unsigned char str[255]; } Str255;
Str15 (15 characters maximum) Str31 (31 characters maximum) Str63 (63 characters maximum) Str127 (127 characters maximum)	typedef struct StrN { unsigned char length; unsigned char str[N]; } StrN;
Array bounds subscript checking produces error messages when array bounds are exceeded	No array bounds checking
Functions declared using ANSI declaration only (prototype declarations).	Functions can be declared either K&R or ANSI
Functions and message handlers can be overridden.	Not available in C, but available in C++.
"i++;" is a statement. It cannot be used as an expression. See below.	"i++" is a statement which can be used as an expression
Statements cannot be used as expressions. For example "a[++i] = 5;" or "a[i=i+1] = 5;" are not allowed. Instead they must be of the format "i++;", or "i=i+1;" and "a[i] = 5;", or "a[i+1] = 5;"	Expressions can be statements
The ModL "for" statement is: for (statement; boolean; statement)	The C "for" statement is: for (expression; expression; expression)
^ is used as the exponentiation operator (like it is in BASIC and spreadsheets)	^ is used as the exclusive-OR operator in logical expressions
To concatenate a string use: stringVar = stringVar + "abc";	The C equivalent is: strcat(stringVar, "abc")
To convert a number to a string: stringVar = x;	The C equivalent is: ftoa(x, str);
if (stringVar < "abc") { ... }	if (strcmp(stringVar, "abc") < 0) { ... }
Resizable dynamic arrays	Pointers
Linked lists support complex data structures.	Structures
!= or <> are not-equal operators	!= is the not-equal operator

ModL	C++
% or MOD are the modulo operators	% is the modulo operator
Bit handling done by functions (such as BitAnd(n, m))	Bit handling done by operators (such as n&m)
#define, #include, #ifdef, #ifndef, #else, #endif can be used as preprocessor directives to conditionally compile code if a symbol of any type is defined (e.g. constant, variable, connector, function, etc.). There is no macro definition.	Macro definition is allowed in addition to preprocessor directives.

ModL compared to other languages

The following table compares some of the common constructs in ModL to other languages.

 DLLs (Windows) and Shared Libraries (Macintosh) provide a method for linking languages other than ModL to ExtendSim. For more information, see “Accessing code from other languages” on page 40 and “DLLs” on page 83.

ModL	Java	Visual Basic	FORTRAN
real a[10], x;	double[] a = new double[10];	dim a(9) as single	real a(9), x
integer i;	double x; int i;	dim i as integer	integer i
for (i = 0; i < 10; i++) { if (i == 5) x = 3; else x = 5+i; a[i] = x; }	for (i = 0; i < 10; i++){ if (i == 5) { x = 3; } else { x = 5+i; } a[i] = x; }	for i = 0 to 9 if i = 5 then x = 3 else x = 5+i a(i) = x end if next i	do i = 0,9 if (i .EQ. 5) then x = 3 else x = 5+i end if a(i) = x end do
integer b[10][5];	int[][] b = new int[10][5]	dim b(9,4) as integer	integer b(9,4)
switch (i) { case 0: x = 3; break; case 1: x = 5+i; break; case 2: x = 5+i; break; default: x = 0; break; }	switch (i) { case 0: x = 3; break; case 1: x = 5+i; break; case 2: x = 5+i; break; default: x = 0; break; }	select case i case 0 x = 3 case 1 case 2 x = 5+i case else x = 0 end select	select case i case (0) x = 3 case (1,2) x = 5+i case default x = 0 end select

ModL	Java	Visual Basic	FORTRAN
while (i < 10) { x = x+i; i = i+1; }	while (i < 10) { x = x+i; i = i+1; }	while i < 10 x = x+i; i = i+1; wend	do while (i .LT. 10) x = x+i; i = i+1; end do
do { x = x+i; i = i+1; } while (i < 10);	do { x = x+i; i = i+1; }while (i < 10);	do x = x+i i = i+1 loop while i < 10	10 x = x+i; i = i+1; if (i .LT. 10) goto 10
//comments ** comments	//comments	rem comments ' comments	! comments
/* enclose multi-line comments like this */	/* enclose multi-line comments like this */	(NOTE: only to end of current line)	(NOTE: only to end of current line)
strng = strng+"abc";	strng = strng+"abc";	strng = strng&"abc"	strng = strng // 'abc'
strng = x;	strng = double.toString(x);	strng = Str(x)	depends on imple- mentation
if (strng < "abc") ...	if (strng.compareTo("abc") < 0) { ... }	if strng < "abc" then ... end if	if (strng .LT. 'abc') then ... end if

ModL feature overview

As you will see, ModL is very much like C, although it is not as complicated and not case sensitive. ModL also has some enhancements that will help you create block code:

- Block code is organized as message handlers and function definitions, rather than just function definitions. Message handlers and functions can be overridden (e.g. if they are from include files) and can have local variables.
- The names of connectors and dialog items are treated as static variables that can be read or set from within message handlers or functions. The dialog changes take effect immediately.
- Blocks can query, control, and send messages to other blocks, even if they are not connected.
- Blocks can have icon views facilitating the direction of model flow.
- The ModL language has several string data types. Strings can be up to 255 characters and can be concatenated (strung together) with the + (plus) operator.
- ModL has subscript checking, causing the code to abort if you try to access an element beyond the size of an array.
- There are over 1,100 ModL functions for performing general and simulation-specific tasks. In addition, there are many global variables that can be accessed from ModL code and a few pre-defined constants.
- Including 2D animation in a block is easy. You do not need to do any graphics in your block code. Instead, you can position animation objects on the icon and use animation functions to show pictures or text within the objects.

- You can use multi-dimensional arrays (with up to five sets of dimensions). Arrays can be passed to blocks or functions as entire arrays or as individual elements. ModL arrays can be *fixed* (specified size) or *dynamic* (one dimension undeclared). You can use dynamic arrays to hold values when you do not know how many elements will be needed.
- You can create ExtendSim databases, linked lists, and global arrays that are useful as in-memory data repositories. You can also access external ODBC compatible databases (see the Data Access category of blocks in the Value library).
- ModL allows for efficient connectivity with other languages, so you can make use of other features and technologies.
- An external source code feature allows multiple people to efficiently and effectively work on the code of blocks at the same time.

ModL language terminology

Table of terms

The following table describes the terms used in ModL coding.

Term	Description
array	An indexed list of numbers or strings, with indices starting at 0 (zero).
constant	Value that does not change.
data type	The type of storage used for the data: real, integer, or string
E notation or scientific notation	Exponential number specified as a number raised to a power of 10. For example, “6.3E3” means 6,300 and “5E-1” means 0.5.
function	Predefined named group of code instructions with specified arguments that can return a value (void functions don’t return a value) and can be called in a block’s code. Can be overridden so you can define a function in an include file and decide to override it in the block’s code.
global variables	Variables that can be viewed or modified by any block. These variables’ values are preserved between simulation runs and are saved with the model. They are all pre-defined by ExtendSim.
identifier	Name that is typed in.
literal	Number or string that is typed in as a constant.
local variable	Variable that is valid only within the message handler or user-defined function in which it is defined. Note that these are temporary variables and their values are not preserved after exiting a message handler or function.  Do not give local and static variables the same name; local variables with the same names as static variables override the static variables.
message handler	Grouping of code that tells ExtendSim what to do in a particular circumstance that is defined by the message. Can be overridden.
statement	Section of code ending with “;”.

Term	Description
static variable	Variable that is valid throughout the block's code in which it is defined. The values for these variables are preserved and are stored with the model when it is saved.  Use caution when deleting static variables for blocks already used in models. It has the same harmful effect as deleting dialog items (discussed on page 51).
system variable	Variable that is valid in any block in a model. These variables are declared by ExtendSim and can be viewed or modified by any block.
type declaration	Defining a variable as a certain data type.

Global, local, and static variables

Variables define memory that can store a value or values. As seen in the table above, there are several types of ModL variables.

Whether a variable is global, static, or local defines what is known as its *scope*. The scope of a variable determines where it can be accessed by code. The following information is important to keep in mind when using variables in block code.

- A *global* variable's scope is the entire model. In a model, a global variable can be referenced from within any block's code or within equation-based blocks.
- The scope of a *static* variable (a variable declared at the top of a block's Code pane) or *dialog item* variable (the name of a dialog item) is the code of that specific block. This means that a block's static variable cannot be used directly in the code of other blocks or in the equations used in equation-based blocks.
- A *local* variable's scope is just the message handler or user-defined function in which it is declared.

Layout of a block's ModL code

Like C programs, ModL code starts with *type declarations* and *constant definitions*. Because you declare these at the beginning of the code, before any message handlers or user-defined functions, they are considered *static* or permanent. They are, therefore, valid throughout the block's code unless overridden by a *local* variable declaration.

After the type declarations, there are *function* (and *void function*) *definitions* and many *message handlers*. The functions and message handlers are just definitions: they need to be called in order to be executed.

Message handlers are formatted as "On MessageName" and tell ExtendSim what to do in various circumstances; they are usually executed by the application. Functions are called from elsewhere within the block's code. Local variables are declared in message handlers and functions. Message handlers and functions can be overridden by re-declaring any number of times below the first declaration.

Like C, ModL ignores blank lines and indentation. Of course, it is a good idea to indent code with Tab characters and use comments. Single line *comments* are preceded by "//" or "/*". Multi-line comments start with a "/*" and end with a "*/".

Syntax coloring

To make it easier to follow the logic of a block's program, ExtendSim colors the code.

Syntax	Color
Strings	Grey
Comments	Red
ModL keywords, application-defined functions and message handlers	Blue
User-defined functions and message handlers	Purple
Overridden functions and message handlers	Brown
Preprocessor directives	Green

Type declarations and constant definitions

There are three main data types in ModL:

- 1) real or double
- 2) integer or long
- 3) string (Str15, Str31, Str63, Str127, Str255 or String)

For the real/double and integer/long data types, the identifiers are interchangeable. The type declarations and constant definitions are set up like they are in C.

Some typical type declarations might be:

```
real nextTime;
real dataArray[], averages[10], theMatrix[10][10];
str15 str15Array[];
str31 str31Array[];
string strArray[], errorStrings[6], theError;
integer checkUsed, multiArray [][][3][5][10][2];
```

As you can see from the example above, ModL has fixed and resizable arrays, and arrays can be up to five dimensions.

ModL already has some pre-defined variables that can be treated just like other static variables. These system variables are described in the “ModL Variables” chapter that starts on page 200.

Constants can be of any data type (real, integer, or string). A constant definition might be:

```
constant maxPower is 52.5;
```

The type of the constant is defined implicitly by the declaration. In the above example, the type is real or double. However:

```
constant xstr is "x"; // quotes cause the constant to be string type
constant maxPower is 52 // type is an integer or long
```

Structures, data types, declarations, and definitions are described in detail in the chapter “The ModL Language”.

Functions and message handlers

ModL code has functions and message handlers that group the code into sections.

- 📖 ModL functions are listed and described in the chapter “ModL Functions” on page 215; the message handler reference starts on page 204.



Message handlers cannot be used in the equation-entering area of equation-based blocks.

Functions are procedures that do calculations and can be called from different points in the code. Functions can return a value; void functions do not return a value.

Message handlers interpret messages that come from the simulation, from another block, or from user interaction with the dialog. Message handlers begin with a line “on *messageName*”, where *messageName* is the name of the message. The code within the message handler starts and ends with curly braces ({ and }). ExtendSim runs the message handler whenever one of the messages is passed to the block.

The code within the message handlers tell ExtendSim what to do in the specific circumstance. For example, in a continuous model, the code in the “on Simulate” message handler is executed for every step in the simulation. However, the code in the “on InitSim” message handler is only executed once, at the beginning of the simulation.

When you create a block, you can add message handlers that are executed at defined times, such as when the dialog for the block is opened (so you can initialize the dialog's contents), when the simulation is stopped, when a dialog button was clicked, or when the block gets a message from another block. No matter what happens, there is a message handler available to perform an action.

Message handlers are denoted by:

```
on messagename
{
    one or more statements;
}
```

The statements in the body of the message handler are in the same format as C functions. For example, a simple message handler is:

```
on CreateBlock // The modeler added this block to a model
               // from the Library menu
{
    checkUsed = 1; // Static variable declared at top of the code
    myNumber = 1; //Initial setting for dialog item
}
```

The statements in this message handler are executed when the block receives the “CreateBlock” message. The variable “checkUsed” is a static variable for the block, defined at the top of the code. The variable “myNumber” is the name of a parameter dialog item in the block's dialog. This statement initializes the parameter to 1 when the block is created (placed in the model).



Message handlers are discussed more in “Using message handlers” on page 39 and are listed in the chapter “Messages and Message Handlers” that starts on page 203.



You can declare local variables at the beginning of a message handler. However, you should not have a global and a local variable with the same name. The local variable is temporary and loses its value when the message handler is exited. Also, within each message handler, local variables can override static variables. (If a local variable is defined with the same name as a static variable, any references to that name within that routine or message handler will change or reference the local variable, and the static variable will not be modified.)

Code completion

After typing part of a function or message handler name in the code pane, click the Code Completion button at the bottom of the block's structure window (or press F8) and the text will be com-

pleted. This helps with getting the correct spelling and argument list. Keep clicking the button, or pressing F8, until the desired function or message handler appears.

Overriding functions and message handlers

Message handlers and user-defined functions can be overridden by being re-declared any number of times below the first declaration. This is useful in that include files can have basic forms of functions and message handlers which can then be re-declared and overridden in the main block code. See “Include files” on page 79.

Conditional compilation

Specific parts of the code can be compiled depending on the presence of certain *symbols* (variable, connector, function name, dialog item name, or constant). In this case, the code will be either used or not used, depending on whether the symbol is defined in the code. You can also define your own symbols. For more information, see “Conditional compilation” on page 79.

Accessing connectors from a block's code

As discussed in “Naming connectors” on page 21, when you create a block each connector's name must end in either “In” or “Out”. Connector names are used as variables in ModL code.

- ☞ All connector names are real type variables. If you set a connector name to an integer, ExtendSim converts the integer to a real; this conversion (like all conversions) will slow the simulation.
- ☞ Typically, connectors pass values one at a time. As discussed in “Passing arrays” on page 98, connectors can also pass arrays of multiple values. Passing arrays is an easy way to pass more than one piece of information at a time through blocks.

There are several types of connectors as listed on page 20. No matter what their type, connectors can be single (“normal”) or multiple (“variable”).

Single (“normal”) connectors

By default a new connector is added as a single (normal) input connector. That connector can be changed to an output connector by adding “Out” to the end of its name.

- To read from an input connector, use its name in the right side of a statement. For instance, read from an input connector called “firstConIn” with:

```
myNumber = firstConIn;
```

- To set the value of an output connector, assign it a value. For example, to set the value of an output connector called “totalOut”, use:

```
if (myNumber > 0)
    totalOut = 1.0;
```

Variable connectors

Each variable connector is actually a row of connectors where the row expands and contracts. This allows the developer and modeler to control how many connectors are displayed for a particular purpose. They are described in the User Guide.

- ☞ See “Adding connectors to the icon” on page 21 for the steps required to change the default (normal) connector into a variable connector.

Accessing a variable connector

- Use the function `ConArrayGetValue` to read from a variable input connector. Input connector positions start at 0.

- To set the value of an variable output connector, use the function `ConArraySetValue`. Output connector positions start at 0.

Setting the number of connectors

The number of variable connectors can be managed directly within the code with the functions `ConArraySetNumCons` and `ConArrayGetConNumber`.

If the number of variable connectors changes, the application sends two messages to the concerned block. The first message is `ConArrayChanged` followed with the `ConArrayChangedComplete` message.

For an example of how to access variable connectors from a block's code and change the number of connectors based on what is needed, see the code of the Math block (Value library).

- ☞ The entire row in a variable connector must be of only one type (Value, Item, etc.) and must be either an input or an output.

Accessing dialog items from a block's code

Overview

The section “Dialog items” on page 11 introduced dialog items and the dialog editor window.

- The types of dialog items are listed on page 12.
- Options for making a parameter, editable text, or dynamic text item non-editable from the dialog, hiding dialog items so that they don't have to be deleted, and so forth are described in “Options for dialog items” on page 14.
- The section “Working with dialog items” on page 16 has additional information about formatting a dialog item's text, title or number, grouping radio buttons, etc.

The default dialog item names (OK, Cancel, and Comments), as well as the names for any dialog items added to a block, are listed in the Variables pane at the lower left of the block's structure window. Use the Edit > Copy and Edit > Paste commands to copy dialog item names from the Variables pane to use in ModL code.

- ☞ Names are required for most dialog items but not for static text and text frames. If a dialog item is not named, it will not be listed in the Variables pane.

Using their dialog names, you can read and set many kinds of dialog items in the same way as you do connectors. Dialog item names can also be used as message names for use with message handlers.

- ☞ Any dialog name can be used in the ModL code as both a variable name and a message name.

Dialog messages

Message handlers interpret messages that come from the simulation or from modeler interaction with the dialog. There are many types of messages; they are listed in the chapter “Messages and Message Handlers” that starts on page 203.

Dialog messages are the names of dialog items and are sent to the block whenever the dialog is used. When a button in a dialog is clicked or a parameter is unselected (for example, after it has been changed), `ExtendSim` sends a message with the same name as the dialog item to the ModL code.

For example, assume a block has a Count button. When that button is clicked in the block's dialog, ExtendSim sends the "Count" message to the block. If the block has an "on Count" message handler, it will be executed; if not, nothing happens.

Other than static text and text frames, names for dialog items used in a block are listed in the Variables pane at the lower left of the structure window. The ExtendSim messages are also listed in that pane.

Parameters and editable text

Assume that a parameter dialog item has the name "numberOfRecords". In the block's code you could have a statement such as:

```
myNumber = numberOfRecords/100;
```

Dialog items can also be set from inside the ModL code. For instance, to set the value shown in the "numberOfRecords" field to "1000", use the statement:

```
numberOfRecords = 1000;
```

The same methods work for editable text:

```
if (temp > 1500)
    displayHeat = "Hot";
else displayHeat = "Cool";
```

You can choose the *Display only* option in the New Dialog Item dialog for a parameter or editable text dialog item. With this option selected, the item cannot be changed directly in the dialog. It can only be changed in the block's code, using the same techniques just shown.

Dynamic text

Dynamic text items allow up to 32,000 characters, whereas editable text items are limited to 255 characters each. Dynamic text is useful when you need more text area than an editable text item can have. But it is a little more complex to work with because ModL code needs to be written to set it up before it is usable and accessible. This is commonly done in the CreateBlock message handler (which occurs when the block is added to the model), but it can be done at other times to suit the functionality of the block.

Dynamic text items can be accessed directly via the string dynamic array assigned to the dynamic text item or using the dynamic text functions (see "Dynamic text items" on page 304). For an example of using dynamic text items, see the Equation block (Value library).

To declare the string dynamic array:

```
string aStringDynamicArray[]; // the dynamic array declaration
....
```

To set up the dynamic text item in the CreateBlock message handler:

```
myDynamicTextItem = DynamicTextArrayNumber(aStringDynamicArray);
```

To directly access the text:

```
first255Characters = aStringDynamicArray[0];
```

The example above shows a dynamic text dialog item attached to a string array. Dynamic text dialog items can be attached to arrays of any size string.

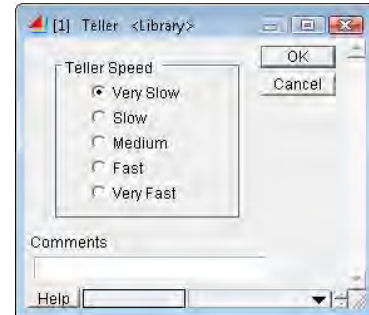
Use the dynamic text functions to find and replace text. See "Dynamic text items" on page 304.

Check boxes and radio buttons

Check box and radio button dialog items return true/false values: true if the check box or radio button is selected, false if not. They also send their dialog item name (as a message name) to the block's code when they are clicked. The code could have an "On DialogItemName" message handler to process the message. The dialog item name could also be used as a variable to query or set the dialog item's value.

For example, if a block represents a teller in a bank, instead of entering a number you could use radio buttons to set the teller's speed. The dialog might look like the one shown at the right.

The five speed radio buttons would have the dialog names VSlow, Slow, Med, Fast, and VFast. To make sure that only one of them can be selected at a time, they must all be defined to be in the same radio group when they are defined. In this example, the group value was left at the default (group 0).



Radio buttons in Teller dialog

To set the variable "theDelay" based on which button was chosen, the code uses the statements:

```
if (VSlow)theDelay = initDelay * 1.25; // v slow is 1.25 normal
if (Slow) theDelay = initDelay * 1.1; // slow is 1.1 of normal
if (Med)  theDelay = initDelay * 1.0; // medium is 1 of normal
if (Fast) theDelay = initDelay * 0.91; // fast is 0.91 of normal
if (VFast)theDelay = initDelay * 0.8; // v fast is 0.8 of normal
```

The "if" statements are executed only if that button value is True (non-zero); of course, only one of them can be true because they are all in the same radio button group. You could also structure the checking with five message handlers, such as:

```
on VSlow // VSlow radio button was clicked
{
    theDelay = initDelay * 1.25;
}
```

Note that in the first instance, the "if" statements would be executed during the simulation, usually in the InitSim message handler. In the second instance, the VSlow button message handler would be executed when you clicked the button labeled "Very slow."

To specify that the "Medium" button should be pre-selected when the block is placed in a model, the CreateBlock message handler contains:

```
Med = TRUE;
```

This also sets the other radio buttons in the group to FALSE.

 When setting the state of a radio button group in ModL code, always explicitly state which button is set to True so that the remaining radio buttons in the group will be set to False. Just setting a radio button to False will not affect the state of the other radio buttons in the group. Thus it is possible to have a condition in which all radio buttons in the group are initially set to False. In most cases, this would be an error condition.

Use the DITitleSet function to change the title of a check box or radio button.

Buttons

When buttons are clicked, they send a message to the block's ModL code. The most common buttons in a block are OK and Cancel, which are handled automatically. If you add other buttons, such as shown later in this chapter, message handlers must be added for those buttons.

To change the title of a button, assign the button's dialog item name as a variable to a string value in the code.

 The changed button title is not stored in the block. Thus the read value of the button's title is always what was originally entered as the title when the dialog item was defined, even if the title is changed by setting it to a different text value in ModL code. If you use the dialog item name in an equation to get a dialog item's title, you will always get the title that was entered when the dialog item was created.

If you change the title of a button, your code must set the title when the modeler opens the block. Do this using the On DialogOpen message handler.

```
on DialogOpen
{
  myButton = "desired button title"; // set it when the dialog opens
}
```

See also “Changing text in response to a user's action” on page 91.

Data tables and text tables

A data table or text table dialog item represents a two-dimensional array of real numbers or text. Tables have an interface that allows modelers to type in any input and also, like all dialog items, allows you to display values generated in the code. If your block code provides for it, modelers can change the number of rows and columns and can dynamically link tables to an ExtendSim Database or global array.

You define the number of rows, number of columns, number format, and headings for the columns, but all of these are changeable with ModL code, including the ability to create and manipulate extremely large tables with up to 255 characters per cell.

The following code fragment shows how to read and write to a data table named “dataTable” that has 4 rows and 3 columns. Data tables are treated as arrays, which are discussed in detail in “Arrays, pointers, queues, delay, linked list, and string lookup table functions” on page 352. As in the C language, array subscripts start at 0, not 1.

```
. . .
// Read the first row, second column cell into myValue.
myValue = dataTable[0][1];

. . .
// Set the fourth row, third column cell to myValue
dataTable[3][2] = myValue;
```

Data tables can be attached to dynamic arrays. They can also have variable columns and the behavior of the columns can be extensively customized. For more information, see the description and functions in “Block data tables” on page 297 and “Formatting/interactivity using column and parameter tags” on page 305.

The text table allows you to type in text, numbers, or both. Since all entries are strings, to use the numbers in ModL code you must first convert them to real values using the StrToReal function.

Static text (label)

Static text appears in a dialog as a label. This is un-editable text, not text that appears in an editable text box. You can give static text a dialog item name that can then be used in the ModL code to change the label, show and hide it, and so forth.

- ☞ The changed label is not stored in the block. Thus the read value of static text is always what was originally entered as the text when the dialog item was defined, even if the text is changed by setting it to a different text value in the code. If you use the dialog item name in an equation to get a dialog item's title, you will always get the text that was entered when the dialog item was created.

If you change the text of a label, your code must set the text when the modeler opens the block. Do this using the On DialogOpen message handler.

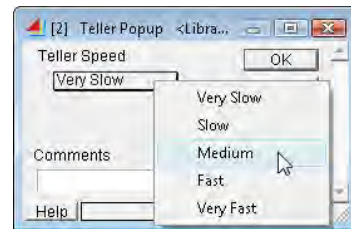
```
on DialogOpen
{
  myLabel = "desired text"; // set it every time the dialog opens
}
```

See also “Changing text in response to a user's action” on page 91.

Popup menu items

When an item in a popup menu is selected, the menu's dialog item name returns a value which is the integer corresponding to the position of the item in the menu, where 1 is the value for the first item in the list. For example, in a 5-item menu, the values of its dialog item name will be set to 1, 2, 3, 4, or 5 based on which menu item the modeler chooses.

Popup menus replace series of radio buttons. For instance, instead of using several radio buttons to represent teller speed, as shown in “Check boxes and radio buttons” on page 35, you could use a popup menu. The dialog would look like the screenshot at right.



Teller dialog with popup menu

This popup menu has the dialog item name “MyMenu”. The five menu items have the titles as shown above. Since “Very slow” is selected, MyMenu is set to 1. If “Medium” were selected, MyMenu would be set to 3.

To set the variable “theDelay” based on which menu item is selected, the code in the InitSim message handler has these statements:

```
if(MyMenu == 1)theDelay = initDelay * 1.25; // v slow is 125% normal
if(MyMenu == 2)theDelay = initDelay * 1.1; // slow is 110% of normal
if(MyMenu == 3)theDelay = initDelay * 1; // medium is normal
if(MyMenu == 4)theDelay = initDelay * 0.91; // fast is 91% of normal
if(MyMenu == 5)theDelay = initDelay * 0.8; // v fast is 80% of normal
```

To specify that, when a block is placed in the model, the “Medium” menu item should be pre-selected, the CreateBlock message handler contains:

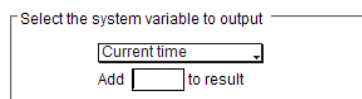
```
MyMenu = 3; // defaults to the “Medium” menu item, third in the list
```

You can use the popup menu's dialog name (for example, *MyMenu*) as the message handler name (for example, *On MyMenu*) to perform specific actions when the modeler selects a menu item. For instance, you could use this to report errors to the modeler, show or hide other dialog items, or cause a sound to play.

Using the column tag functions listed in “Formatting/interactivity using column and parameter tags” on page 305, data tables can have popup menus in their columns. There are also functions to dynamically popup a menu on the fly, whenever the modeler clicks something.

Text frame

A text frame allows you to visually isolate or group dialog items by framing them with a rectangular box. Once a text frame has been added to the dialog editor window, it can be resized and positioned to surround the items of interest. If present, the text frame's title is displayed in its upper left corner, as shown at the right.



Text frame in Example block's dialog

If you give the text frame a name, it can be called as a variable in the block's code. This is helpful for showing and hiding the frame under different circumstances, dynamically repositioning it, or dynamically changing its title.

Switch

A switch looks like a standard light switch, with either a 1 or a 0 displayed.

When you click on the side that is not down, it makes a small clicking sound, changes to the other value, and sends a message to the ModL code.



Switch on (L) and off positions

The switch dialog name always returns either 0 (off) or 1 (on), depending on the value of the switch. You can also control the switch by setting its variable name to 0 or 1 in the code.

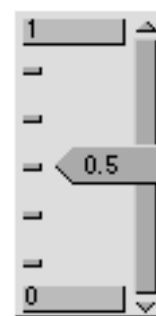
Slider

A slider allows you to enter a value by dragging its knob along its length.

The minimum and maximum values can be set from ModL code; the modeler can change the minimum and maximum values by clicking and editing them in the block's dialog. The current value can be set from the code and can also be set as the model runs by dragging the slider up or down. In this way, you can use the slider both for visual output and for input.

The slider is represented in a three-element array of reals that represent the minimum, maximum, and current values. Thus, if a slider is called “theSlider,” it could be initialized with the lines:

```
theSlider[0]=0.0; // Minimum of 0
theSlider[1]=10.0; // Maximum of 10
theSlider[2]=3.33; // Starting value of 3.33
```



Typical slider

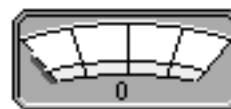
As the model runs, the value of the slider can be checked by reading the third array element:

```
theSetting = theSlider[2];
```

Meter

A meter allows you to view a value on a meter.

You set the minimum, maximum, and current values from the ModL code.



Typical meter

The meter is represented in a three-element array of reals that represent the minimum, maximum, and current values. Thus, if your meter is called “theMeter,” you might initialize it with the lines:

```
theMeter[0] = 0.0;      // Minimum of 0
theMeter[1] = 30.0;     // Maximum of 30
theMeter[2] = 10.0;     // Starting value of 10
```

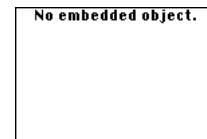
As the model runs, you can set the value shown on the meter in the third array element, such as:

```
theMeter[2] = theSetting;
```

Embedded object (Windows only)

An embedded object is an OLE object that can be chosen via the block code from available types, such as an Excel spreadsheet or chart. The object can then be manipulated by the code of the block. When first placed in a dialog editor window, an embedded object looks like the image at right.

The functions are listed in “Interprocess Communication (IPC)” on page 239 and “OLE/COM (Windows only)” on page 241. Also see the User Guide for information about using embedded objects in models.



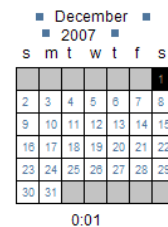
Empty embedded object

Calendar

The Calendar dialog item takes an ExtendSim date value (discussed in “Calendar Date functions” on page 373) and visually displays it in a combination calendar/clock format. For example, assume a calendar dialog item named MyCalendar has been added to a block’s dialog. That dialog item will display December 1, 2007 12:01 am when the following line of code is executed:

```
MyCalendar = 39417.000694444;
```

The dialog item would then look like the screenshot to the right.



Using message handlers

Messages and message handlers were introduced on page 30.

ExtendSim uses a sophisticated messaging architecture to signal blocks into action. While messages can originate either from the ExtendSim application or from individual blocks, it is always a block that is on the receiving end of a message.

Messages could be thought of as falling into one of two categories:

- 1) Messages typically sent from the application to one or more blocks. For instance, when a block is added to the model, while the simulation is running, during interaction with an ExtendSim Database, and so forth.
- 2) Messages typically sent between blocks by calling ModL functions in block code. These messages are sent whether the blocks are connected or unconnected.

There are many types of messages, as summarized in the table on page 203. Different types of messages result in the receiving block doing different types of things. For instance, when you add a block to a model, ExtendSim sends the “CreateBlock” message to the new block. If the block’s code contains a message handler for the “CreateBlock” message (that is, a bracketed set of lines that are preceded by “on CreateBlock”), the code is executed; if not, nothing happens.

When you run a simulation, some messages are sent to all blocks; others are sent only to a specific block. For instance, the “CreateBlock” message is sent only to the block that was added to the

model. However, the “InitSim” message, which tells the blocks that a simulation is starting, is sent to all blocks.

Use the Copy and Paste commands, or drag and drop editing, to copy message names from the Variables pane in a block’s structure window to use in ModL code.

 All ModL messages are listed in the Variables pane at the lower left of the structure window. A complete list and description of ModL messages is in the chapter “Messages and Message Handlers” that starts on page 203.

Example of a message sent during user interaction with dialog

This example shows what happens when the modeler clicks a button in the dialog. The dialog item name of the button is “MyExportButton”. (The title of the button in the dialog is “Export”; it is not used in the code.)

```
// Sent when the modeler clicks a button in the block's dialog
on MyExportButton // The modeler clicked the Export Data button
{
  MyExportFTP(); // Call my function to export the data to the web
}
```

Example of a message sent by the application

The following code uses the CheckData message that is sent by ExtendSim to all the blocks in a model at the beginning of a simulation. If the block has a CheckData message handler, this message tells the block to check the validity of its data before the simulation starts.

```
// Sent by ExtendSim to allow checking data before simulation
on CheckData
{
  if (myParm < 0.0) // this parameter should not be negative
  {
    UserError("Data in " + MyBlockNumber() + " can't be negative.");
    Abort; // Stop simulation and select the offending block
  }
}
```

Block to block message

This is an example where one block tells another block to perform an action. The receiving block is instructed to display, in its dialog, the text sent by the sending block.

```
// Sent from another block telling this block to show a text message
on UserMsg9 // A user-defined message
{
  // GlobalStr9 was set by the sending block who sent this message
  MyDialogVar = GlobalStr9; // Now I can display it in dialog
}
```

Accessing code from other languages

You can use other languages, such as Visual C++, Fortran, C++, and others, to provide functionality to ModL or to make use of a legacy of pre-built code. For instance, you may already have thousands of lines of C++ code which perform a specific calculation. You can add this functionality to your block by recompiling the C++ code as a DLL (dynamic link library for Windows) or Shared Library (external command for Mac OS). DLLs and Shared Libraries are the standard interface technologies for communicating between programming environments. When you want to do the calculation, use ExtendSim’s built-in function calls to call the DLL or Shared Library.

One advantage of this method is that you don't have to program the interface in the external language. Instead, you can leverage ExtendSim's built-in interface and dialog creating capabilities.

- Even though you are using an external language, the resulting block will fit seamlessly into the ExtendSim environment and be indistinguishable from other blocks.

To learn more about DLLs, or to see an example of ModL code that calls a DLL, see "DLLs" on page 83.

External source code

ExtendSim supports a mechanism for saving the source code of individual blocks or libraries of blocks as external files. This external source code feature is useful for situations where multiple developers work on the code of blocks and/or libraries at the same time. For more information, see "External source code control" on page 80.

Tutorial

Creating a Block

Learn how to build an ExtendSim block
and save it in a library

*“Knowledge is of two kinds. We know a subject ourselves,
or we know where we can find information upon it.”*
— Samuel Johnson

Previous chapters showed the external and internal parts of a block. This section shows how to create a continuous block and have it perform useful tasks.

 Please read the previous two chapters before you proceed with this chapter.

For this example, you will create a block that converts miles to feet. The final block, called “Miles”, is located in the menu command Library > Example Libraries > Custom Blocks library under the Math category.

Steps for creating a new block

The steps for creating a new block are:

- ▶ Choose Develop > New Block
- ▶ In the New Block dialog that appears, select a library for the new block:
 - If the library is already open, select the library from the list of open libraries on the left. (The pane on the right of the dialog will show the blocks in the selected library.)
 - To open an existing library, click the Open Library button. Note: This should be a library you have created, NOT a library that ships with ExtendSim.
 - To create a new library, click the New Library button. This process is described below.
- ▶ Once you have selected a library for installation, name the new block.
- ▶ Click *Install In Selected Library*.

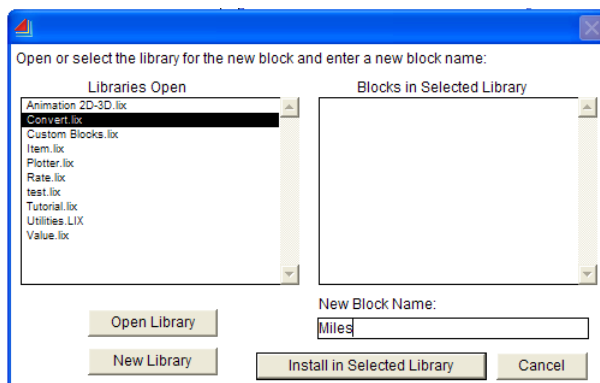
Building a block that converts miles to feet

For this first section, the Miles block will have two value connectors, an input and an output. The block will look at the value of the input connector (the number of miles), multiply it by 5280 (the number of feet in a mile), and copy the result to the output connector.

Create the block

To create the Miles block:

- ▶ Choose Develop > New Block.
The New Block dialog prompts for a library to save the block in.
- ▶ Click the New Library button at the bottom of the dialog.
- ▶ Name the library **Convert** and click Save. Notice that in the New Block dialog, the new library is automatically selected for installing the block.
- ▶ Enter the name **Miles** for the new block, as shown at right.
- ▶ Click the Install in Selected Library button.



Dialog for naming a new block and installing it in a library

ExtendSim opens the default new block dialog editor and structure windows, as seen below.

Dialog editor window of the Miles block

By default, the dialog editor window opens in front of the structure window.

The default window is empty except for the OK and Cancel buttons, a “Comments” label, and an entry box for user-entered comments. The size and location of these dialog items determines the size of the dialog.

To add some explanatory text to the left of the OK and Cancel buttons:

- ▶ Choose Develop > New Dialog Item.
- ▶ In the New Dialog Item dialog, select the **Static Text (Label)** dialog item.
- ▶ Enter **Converts miles to feet** in the text box labeled *The text*. (Entering “” in front of the text formats it as bold.) Since it will not be used, there is no need to enter a name for this dialog item.
- ▶ Click OK. Unless you changed the X and Y coordinates, the label will be placed in the upper left corner of the dialog editor window.
- ▶ Select the label and use the resize boxes at each corner to resize it so that it shows all the text. Or double-click the item and resize it using the H and W coordinates.

✎ ExtendSim automatically puts dialog items and icons on a grid. To drag an object without using the grid, hold down the Alt (Windows) or Option (Mac OS) key while dragging the item.

- ▶ Move the text label down a bit in the dialog:
 - ▶ Either select and drag it to move it
 - ▶ Or, double-click it and change its X, Y coordinates in the New Dialog Item dialog

The dialog editor window should now look like the screenshot here.

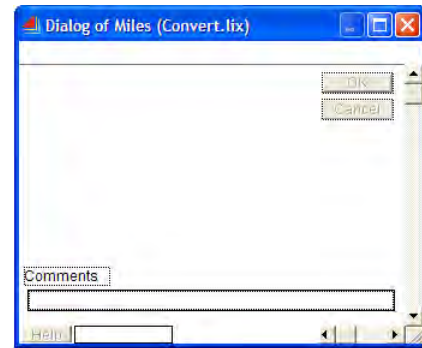
Structure window of the Miles block

- ▶ Bring the structure window to the front.

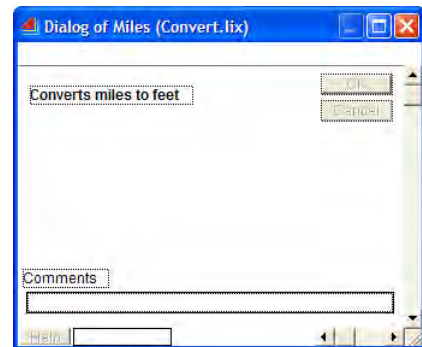
The default structure window has an icon with a text label (New Block) and three pre-defined message handlers in the code pane:

- Simulate
- CheckData
- InitSim

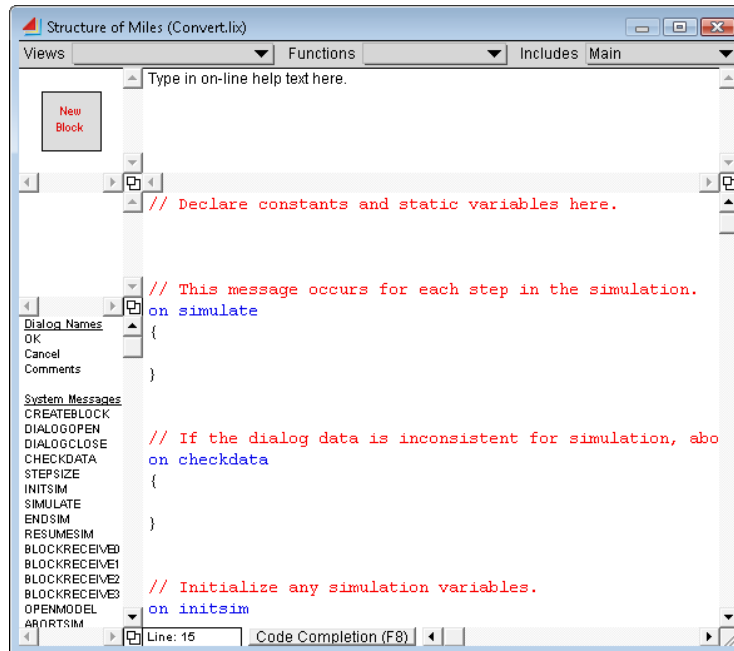
The code of most continuous blocks will use these messages, so they are placed into the code pane by default. Unused messages are ignored. For instance, it is common in discrete event blocks to not use the Simulate message handler so as to optimize the duration of the simulation.



Default dialog editor window



New dialog item



Default structure window

Icon

The Miles block only needs a simple icon. You could draw the icon using the ExtendSim drawing environment or a painting program, or copy clip art and paste it in. For this tutorial, just change the default icon.

In the Icon pane:

- ▶ Double-click the icon's default text (*New Block*) and change it to **Miles->Feet**.
- ▶ Then click somewhere else in the icon pane to deselect the text.
- ▶ Click the icon once to select it, then resize it to accommodate the text. You may also need to resize the text box and the icon pane using their resize boxes.
- ▶ Center the text as desired. (If necessary, you can hold down the Alt or Option key to disable the automatic grid, then drag the text so that it is centered on the icon.)



Icon for new block

The icon should now look like the one shown here. If desired, use the ExtendSim drawing environment and the commands in the Model menu to change the color of the icon and/or text or to change the thickness or color of the border around the icon.

Connectors

The Miles block needs two value connectors, an input and an output. As discussed on page 20, connectors can be either normal (a single connector) or variable (a row of single connectors). The default is that connectors are normal, and that is what is required for the Miles block.

See page 21 for how to change a connector from normal to variable.

Adding the connectors

- ▶ In the Icon Tools popup menu at the far right of the ExtendSim toolbar, select the *Value* connector tool.
- ▶ In the Icon pane, click near the left side of the icon. This places a value input connector at the left of the icon. Notice that the name of the connector (Con1In) is displayed in the Connector pane.
- ▶ Select another *Value* connector from the Icon Tools popup.
- ▶ Click near the right side of the icon to place a value input connector, named Con2In, there.
- ▶ Select each connector and either drag it or use the arrow keys so that the connector touches the icon.



Icon with connectors

To align the tops of the connectors, either select both of them and give the command Model > Align > Top, or use dragging or the arrow keys to move one connector until its Y position (in the Cursor Position field of the ExtendSim toolbar) is the same as the other connector's Y position.

Renaming the connectors and changing an input to an output

When connectors are added to an icon, they are by default all input connectors with the default connector names Con1In, Con2In, and so on. Input connectors must end in "In" and output connectors must end in "Out"; the words "In" and "Out" are not case sensitive.

- ▶ In the Connector pane, click the connector name **Con1In**.
- ▶ Type **MilesIn**, then press the Enter or Return key or click anywhere else in the window. This changes the name of the connector on the left of the icon.
- ▶ With **Con2In** selected in the Connector pane, type **UnitsOut**. Press the Enter or Return key or click anywhere else in the window. This changes the connector on the right to an output connector, as shown.



Help text

Replace the default Help text in the Help pane with some simple text. Even when there is not much to say, it is a good idea to have Help text.

ModL code

The next step is to enter the ModL code. For this block, the action happens in the Simulate message handler. The other default message handlers can be left blank since there is nothing to check or initialize at the beginning of the simulation run.

Type the following ModL code into the Code pane at the lower right of the structure window.

```
on Simulate
{
UnitsOut = MilesIn * 5280.0;
}
```

The spaces on each side of the operators are optional; the semicolon at the end of the statement is **not** optional.

Because ModL is not case sensitive, the connector name "UnitsOut" could just as well have been written as "unitsout" or "unitsOut".

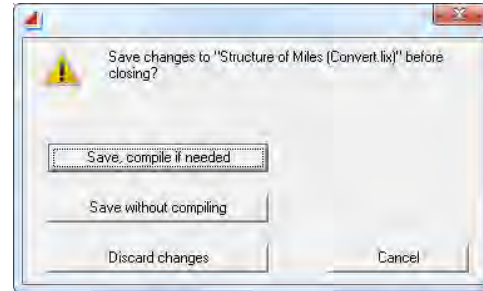
The code means that for each step of the simulation, ExtendSim reads the value at the input connector, multiplies it by the real value 5280.0, and sets the output connector to that product.

Compile and save the block

After the code has been entered:

- ▶ Close the structure window by clicking its close box or giving the command File > Close. ExtendSim opens a dialog for saving changes, seen at right.
- ▶ Click *Save, compile if needed*. This compiles the ModL code and saves the changes to the block in the library.

If there are any compilation errors, ExtendSim will warn you.

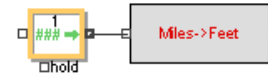


Saving and compiling a block

Test the block

To test the Miles block:

- ▶ Open a new model window if one is not already open.
- ▶ Add your Miles block (Convert library) to the model window.
- ▶ Add a Constant block (Value library; Inputs submenu) to the model and move it to the left of the Miles block.
- ▶ Connect the output of the Constant block to the input of the Miles block.
- ▶ In the Constant block's dialog enter **Constant value: 10**.
- ▶ Add a Plotter, I/O block (Plotter library) to the model and move it to the right of the Miles block.
- ▶ Connect the output of the Miles block to the top input of the Plotter, I/O block as shown here.
- ▶ Run the simulation.



Constant block connected



Test model completed

The value 52800 (5280×10) should be output for the entire length of the simulation. You can verify that the block works with other numbers by entering them in the Constant block and running the simulation. (The plotter automatically rescales at the end of each run.)

Adding user interaction and display features

If moving numbers between input and output connectors was all that ExtendSim did, it would not be a very useful program. As you have seen in the User Guide, robust blocks:

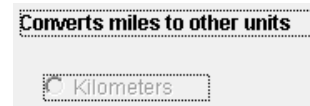
- Let you change their parameters
- Give information about what is happening during the simulation

Dialog items

The enhanced Miles dialog will have buttons for choosing the unit to convert to, a frame to organize the buttons, a parameter field for the input value, and a parameter field to display the result.

Defining a radio button

- ▶ Open the structure and dialog editor windows for the Miles block, using one of the methods listed in “To open a block’s internal structure” on page 8.
- ▶ In the dialog editor window:
 - ▶ Double-click the text **Converts miles to feet**. This displays the dialog item’s definition in the New Dialog Item dialog.
 - ▶ Change the text of the dialog item to read: **Converts miles to other units**. Then click OK to close dialog.
 - ▶ Enlarge the label so all the text is visible. (To do this, either select it and use one of the resize boxes to resize it, or double-click it and change its W coordinate.)
- ▶ With the dialog editor window in front, choose the command Develop > New Dialog Item.
 - ▶ Select Radio Button. ExtendSim gives this first control group a default number of 0.
 - ▶ Enter **Kilometers** as the button title and **Kilometers_rbtn** as the dialog item name. Click OK.
 - ▶ Drag the new button to a place below the first text label.



New radio button added

While not required, coding conventions help when creating blocks. For instance, the button’s dialog item name (Kilometers_rbtn) is more intuitive to work with in the code.

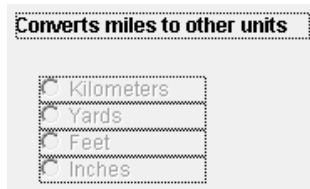
Additional radio buttons

There are two ways to add additional dialog items in the dialog editor window:

- 1) Choose Develop > New Dialog Item, select the desired dialog item in the New Dialog Item dialog, then enter its text/title or name, as applicable.
- 2) Select a dialog item in the dialog editor window and give the command Edit > Duplicate or Edit > Copy/Paste. Then double-click the duplicate dialog item and change its defined text/title and/or dialog item name, if desired.

The second method is only applicable to the same type of dialog item; you cannot use it to change the type of dialog item.

- ▶ Choose one of the methods from above to create three additional radio buttons, each with its own button title (**Yards, Feet, or Inches**) and dialog item name (**Yards_rbtn, Feet_rbtn, or Inches_rbtn**).
- ▶ After the radio buttons have been defined, align their left sides using the command Model > Align > Left, or by coordinating their X coordinates (displayed in the Cursor Position field).



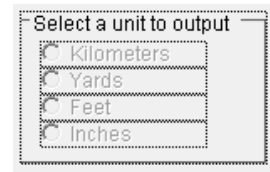
Buttons added and aligned

The dialog editor window should now look like the one shown here.

Text frames

To organize the dialog items on the window:

- ▶ Choose the command Develop > New Dialog Item
- ▶ Add a text frame dialog item
- ▶ For its text, enter **Select a unit to output**. It is not necessary to enter a dialog item name for the text frame.
- ▶ Enlarge the text frame and arrange the radio buttons within it as shown here.



Text frame added

Saving the dialog changes

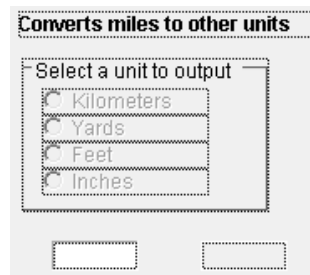
- ▶ Give the command File > Save Block to save your work

 This saves the changes but does not compile the block. It is a quick way to save changes without closing the structure or dialog editor windows.

Adding parameter fields

To add entry fields for the numbers:

- ▶ In the dialog editor window, choose Develop > New Dialog Item.
 - ▶ Select **Parameter (Number)**.
 - ▶ Enter **InNum_prm** for the dialog item name and click OK.
 - ▶ Drag the new parameter field below the text frame.
 - ▶ Shrink the parameter field to about half its default size. (Either select it and use one of the resize boxes to resize it, or double-click it and change its W coordinate.)
- ▶ Define a second parameter dialog item using either of the methods listed in “Additional radio buttons” on page 49.
 - ▶ Enter **OutNum_prm** for its dialog item name.
 - ▶ Select the *Display only* choice. This will cause the parameter value to be displayed in the dialog without being editable. Click OK.
 - ▶ Drag the new parameter field to the right of the first one and shrink it to about half its default size.

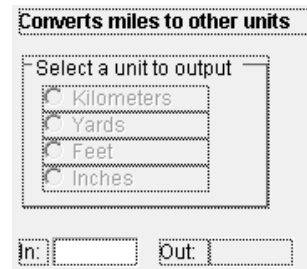


Second entry field added

Adding more static text labels

The next step is to define two more Static Text (Label) dialog items using either of the methods listed in “Additional radio buttons” on page 49.

- ▶ For the first static text item:
 - ▶ Enter **In:** for the text. Click OK.
 - ▶ Shrink the label's width, then move it to the left of the first entry box.
- ▶ For the second static text item:
 - ▶ Enter **Out:** for the text. Shrink the label, then move it to the left of the second entry box.
- ▶ Save and compile the bloc as discussed onpage 48. If there are any compilation errors, Extend-Sim will warn you.



Labels added

Tutorial

Tabs in the dialog

When a block is first created, it does not contain any tabs. For this block, tabs will be used to separate the comments field from the parameters dealing with the conversion.

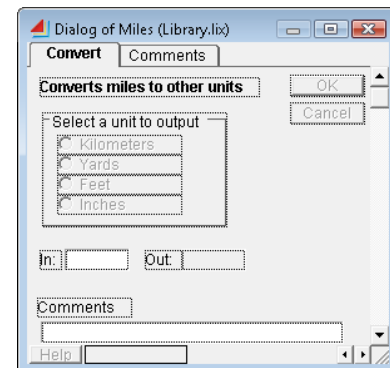
Adding tabs

To add a tab:

- ▶ Open the block's internal structure.
- ▶ With the dialog editor window at the front, choose Develop > New Tab or double-click in the area above the line, and below the title bar, at the top of the dialog editor window.
- ▶ Name the tab **Convert**. Click OK.

To add a second tab for the comments:

- ▶ Create the tab using the same process as above.
- ▶ Name the tab **Comments**. Click OK.



Tabs added

Showing dialog items on all tabs

The only dialog items on the Comments tab are the OK and Cancel buttons. By default, those dialog items appear on all the tabs. To make the *Converts miles to other units* text also appear on both tabs:

- ▶ In the Convert tab, double click the **Convert miles to other units** text to open its definition.
- ▶ Check the **Visible in all tabs** checkbox and click OK.

Moving dialog items between tabs

Since the comments field will be on its own tab, the *Comments* label is no longer necessary.

- ▶ Select the **Comments** label in the Convert tab and delete it.

Use caution when deleting a dialog item. If the block is used in a model, deleting the item could disrupt the order of the dialog's data. The data will have to be reentered for each instance of that block in all models that use it. Instead of deleting the dialog item from a block used in a model,

consider hiding it using the Hide item option. (Since they don't store data, text frames and static text labels may be safely deleted.)

To move the comments field to the Comments tab:

- ▶ Select the comments field in the Convert tab.
- ▶ Choose the command Develop > Move Selected Items to Tab.
- ▶ In the dialog that appears, select the Comments tab.
- ▶ Choose the **Move Items to Tab** button.
- ▶ In the Comments tab, arrange and resize the comments field as desired.
- ▶ Give the command File > Save Block to save changes without closing the windows.

 To add new dialog items to a tab, bring that tab to the front before choosing Develop > New Dialog Item.

Icon

Since the purpose of the block has changed, change the icon.

- ▶ Bring the structure window forward.
- ▶ In the Icon pane, change the text on the icon to **Miles -> ???** and press Enter.

ModL code

The next step is to enter code in the block's Code pane.

CreateBlock message handler

The CreateBlock message handler can specify one of the conversions as the default for the block when it is placed in the model. For instance, to make feet the default, enter this in the Code pane:

```
// feet is the default setting
on CreateBlock // executed when the modeler places a new block
{
  Feet_rbtn = TRUE; // highlight Feet radio button by making it TRUE
}
```

You can put the CreateBlock handler anywhere in the block's code after the declarations.

Simulate message handler

For each step in the simulation, you want to:

- Update the input entry box (the first parameter)
- Check which conversion is being performed
- Multiply the numbers
- Update the output entry box (the display only parameter)

To do this, replace the code in the Simulate message handler with the following:

```
on Simulate          // beginning of the SIMULATE message handler
{
  InNum_prm = MilesIn; /* Display the value input by setting the dialog
                        variable "InNum" to the input connector value */
  if (Kilometers_rbtn) // if kilometers radio button TRUE
    UnitsOut = MilesIn * 1.609344; // set output connector to eqn
  else if (Yards_rbtn) // if yards radio button TRUE
    UnitsOut = MilesIn * 1760.0; // set output connector to eqn
  else if (Feet_rbtn) // if feet radio button TRUE
    UnitsOut = MilesIn * 5280.0; // set output connector to eqn
  else if (Inches_rbtn) // if inches radio button TRUE
    UnitsOut = MilesIn * 63360.0; // set output connector to eqn

  OutNum_prm = UnitsOut; /* Display value output by setting the dialog
                        variable "OutNum" to the output connector value */
}
```

Remember that you can copy the connector and dialog item names from the Dialog Names pane at the left of the structure window.

If the “Yards” radio button is selected in the dialog, the statement “if (Kilometers_rbtn)” evaluates false and its “else” clause is executed. The “if (Yards_rbtn)” statement then evaluates true and executes its statement, and the “else” clauses following it will not be executed. The other radio buttons behave similarly.

The numbers multiplied by MilesIn can be reals or integers. ModL performs all necessary type conversions automatically. However, since connectors are always of type real, you should use reals for values that are used with connectors. That way, ExtendSim will not need to convert integers to reals on each step. (For more information about type conversions, see page 65.)

Compile and save the block

After you have finished writing the code, close the block by clicking its close box. Choose *Save, compile if needed* to compile the new ModL code and save the block in the library.

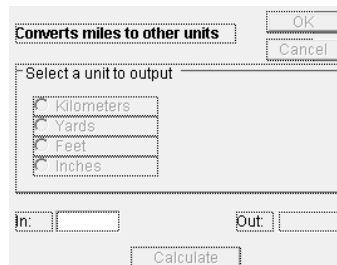
Test the Miles block as you did before, trying all the conversions. Notice that you can keep the Miles dialog open during the test.

Adding an intermediate results feature

You may have noticed something missing from the Miles block: what if you just want to convert a single number without running a whole simulation? As stated earlier, ExtendSim passes dialog messages to a block even if a simulation is not running. It is thus easy to make this block useful even outside of a simulation.

Add a button

- ▶ Open the block's internal structure.
- ▶ Choose a Button item from the New Dialog Item dialog.
 - ▶ Enter **Calculate** for the button title and **Calculate_btn** as the dialog name. Click OK.
 - ▶ Drag this new button below the input and output fields, as shown at right.



New button added

ModL code

- ▶ Add the following message handler somewhere below the Simulate message in the block's Code pane:

```
on Calculate_btn// Display the values
{
  if (Kilometers_rbtn)
    OutNum_prm = InNum_prm * 1.609344;
  else if (Yards_rbtn)
    OutNum_prm = InNum_prm * 1760.0;
  else if (Feet_rbtn)
    OutNum_prm = InNum_prm * 5280.0;
  else if (Inches_rbtn)
    OutNum_prm = InNum_prm * 63360.0;
}
```

- ▶ Save the block and compile the ModL code.

See “Defining functions” on page 56 for how to avoid typing the same code twice.

To test this new functionality:

- ▶ Double click to open the block's dialog. You may need to resize it to show the new dialog items.
- ▶ Type a number in the **In:** entry box.
- ▶ Click Calculate.

Note that the number in the *Out:* entry box is updated with the correct data.

See “Accessing code from other languages” on page 40 for an example of how to do this using a DLL.

Adding 2D animation

The icon of this block indicates that it converts miles to something, but that's not very informative. Displaying text on an icon is a convenient method to show block conditions – in this case, what kind of conversion the block is performing.

The ModL code will ensure that the text is displayed even if the modeler doesn't have Show 2D Animation selected; the display is also independent of the simulation run.

Changing the icon and adding the animation object

- ▶ Open the structure window of the Miles block and bring it to the front.
- ▶ In the Icon pane, delete the ??? part of the icon text.
- ▶ Move the remaining text to the left of the icon to provide more room.

- In the Icon Tools popup menu of the ExtendSim toolbar, select the Animation Object tool; it is at the bottom of the menu.

- In the Icon pane, click once on the icon to the right of the text.
While still holding the mouse down, drag to the right. This creates a rectangular animation object. Since this is the first animation object, it will have a “1” in it.



Animation object added

- Use the mouse to position the animation object within the icon.
You can use the Alt (Windows) or Option (Mac OS) to temporarily turn off gridding, for finer positioning. (You may later need to select the animation object to resize it to the dimensions you want.)

Adding code for the animation

- Add the following message handlers to the end of the ModL code. Each message handler receives a message when the corresponding radio button is clicked.

```
on kilometers_rbtn // Kilometers radio button was clicked
{
  AnimationTextTransparent(1, "KM"); // set animation text
  AnimationShow(1);                // show the animation object
}

on yards_rbtn // Yards radio button was clicked
{
  AnimationTextTransparent(1, "Yards");// set animation text
  AnimationShow(1);                // show the animation object
}

on feet_rbtn // Feet radio button was clicked
{
  AnimationTextTransparent(1, "Feet"); // set up animation text
  AnimationShow(1);                // show the animation object
}

on inches_rbtn // Inches radio button was clicked
{
  AnimationTextTransparent(1, "Inches");// set up animation text
  AnimationShow(1);                // show the animation object
}
```

The ModL compiler will give an error message if code containing “smart quotes” is copied into the Code pane. To fix the problem, replace the copied quotes with new ones in the Code pane.

- *Replace* the code in the “On CreateBlock” message handler to initialize the animation text and set the text to a green color.

```
// feet is the default setting
on CreateBlock // executed when the modeler places a new block
{
  Feet_rbtn = TRUE; // highlight Feet radio button by making it TRUE
  AnimationTextTransparent(1, "Feet"); // set default text value
  AnimationColor(1, 21845, 65535, 52428, 1); // color the text green
  AnimationShow(1); // show the animation object
}
```

- Close the block’s internal structure, saving and compiling the changes.

👁 Notice that the text will show only when a new Miles block is placed on the model worksheet.

Testing the block

To test this new functionality:

- ▶ When you place a new Miles block in a model, the default setting (Feet) should be displayed on its icon.
- ▶ Changing the selected radio button in the block's dialog should cause a corresponding change on the icon.



Icon with animation

The final Miles block is located in the Libraries / Example Libraries / Custom Blocks library within the Math category.

Other features

The preceding example showed how to build a simple block. This section describes some additional ExtendSim features that are useful when creating blocks.

Colored dialog items

By default all dialog items are drawn in the black color. To enhance dialogs, use color-enabling functions (such as `SetDialogItemColor`) from the section “Dialog items” on page 291.

Hiding/showing dialog items

As discussed in “Hide item” on page 15, dialog items have an option that allows them to be hidden. You can also choose to dynamically show and hide dialog items using functions (such as `HideDialogItem`) in the function list that starts on page 291. For example, the Decision block (Value library) has a parameter field for Relax that appears if Use Hysteresis is checked. For more complex dialogs, see “Moving dialog items”, below.

Moving dialog items

You might want to have a dialog item move depending on a block's settings. This is often easier and more organized than placing all the dialog items on top of each other and then hiding and showing them depending on the setting. For instance, the Random Number block (Value library) moves parameter fields based on which distribution the user selects.

Defining functions

You may have noticed that the Simulate and the Calculate message handlers have similar code. One way to be more efficient and reduce errors would be to define a function to calculate the output value and call it in the both the Simulate and Calculate message handlers.

```
Real CalculateOutput(Real inputValue) // Define Real function
{
    if (Kilometers_rbtn)
        Return(inputValue * 1.609344);
    else if (Yards_rbtn)
        Return(inputValue * 1760.0);
    else if (Feet_rbtn)
        Return(inputValue * 5280.0);
    else if (Inches_rbtn)
        Return(inputValue * 63360.0);
}
```

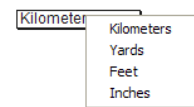
```
On Calculate_btn
{
    OutNum_prm = CalculateOutput(InNum_prm); // Call with input
}

On Simulate
{
    InNum_prm = MilesIn;
    UnitsOut = CalculateOutput(MilesIn); // Call with input con.
}
```

Popup menus

As an alternative to using radio button groups in the Miles block, you could have used a popup menu to allow the modeler to select a unit to output.

See “Popup menu items” on page 37 for an example of how to access a popup menu dialog item from a block’s code. The Simulation Variable block (Value library) is an example of how popup menus can be implemented.



Popup menu

Linking to a global array or ExtendSim database

Dynamic data linking (DDL) creates a link between a data table or parameter dialog item and an internal data repository (global array or ExtendSim database). For instance, the Data Source Create block (Value library) has a data table that is dynamically linked to a global array. To add dynamic linking to a block, see “Data table linking” on page 298 and “Dynamic linking” on page 302.

To simply register a block so that it will be notified if there was a database change, see “Registered blocks” on page 106 and “Linking and notification” on page 351.

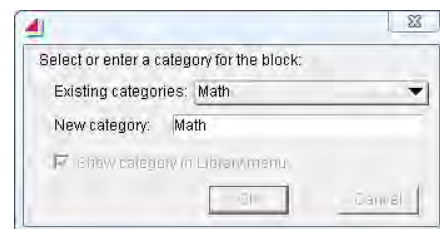
Block categories

It is common to build several blocks to put in a library. You can have all the blocks listed alphabetically in the library, or you can specify a block *category* and cause blocks to be grouped into sub-menus in the Library menu.

Each block has a 15-character string associated with it called the block category. The category is used by ExtendSim to organize blocks into logical groups that perform a similar function. For example, the Math block (Value library) performs basic math functions; therefore, its block type is *Math*. ExtendSim uses the block category in two places: 1) to organize blocks within the Library menu and 2) to organize blocks within the reports. Categories of blocks in libraries and reports are discussed in the User Guide.

To set or edit a block’s category:

- With the block’s structure window open, select the command Develop > Set Block Category.
- In the Category dialog, either select an existing category from the popup menu or define a new one by entering it in the text field.
- Click OK.



Set Block Category dialog

The *Show Category in Library Menu* checkbox is used to determine how ExtendSim will list this block within the Library menu. If checked, the block will be listed in a submenu under the block type under the library name. If the box is unchecked,

ExtendSim will not use the category but will instead list the block alphabetically directly under the library name.

Variable connectors

By default new connectors are added as a normal (single) connector. This works well for simple blocks like the Miles block. But often a block will need several inputs or outputs.

As discussed in “Connector options” on page 20, when you select a type of connector (Value, Item, etc.) you can also select whether the connector is normal or variable. Variable connectors act like a row of normal connectors, where the row can be expanded or contracted to provide a required number of connectors. To work with variable connectors, see the functions on page 288. For an example of how variable connectors are implemented, see the Math block (Value library).

Connector labels

Connector labels are associated with a particular connector and make a block’s inputs and outputs more understandable. They can be positioned, colored, and formatted, and are especially helpful for variable connectors, to distinguish one input or output from the others. Connector label functions start on page 290. Most ExtendSim blocks have one or more connector labels.

Connector labels are defined using the ConnectorLabelsSet function. The function’s arguments allow setting the position and color of the label:

- The positions are: 0 top, 1 right, 2 bottom, and 3 left.
- The color is determined by entering the hue, saturation, and value (HSV) numbers found in the ExtendSim Color tool.

To format a connector label, precede the label text with angle brackets (< and >) that enclose the desired format, using the same techniques shown in “Styles, alignment, and colors for dialog items” on page 16. For instance, a bold, right-adjusted label named “want” would be entered as “<rb>want” in the Code pane.

Connector tool tips

Tool tips can display custom text when the mouse hovers over a connector. This is helpful to show additional information about the connector (including its value during the simulation) and what it can be used for. See “Connector tool tips” on page 291 and the Equation block (Value library).

Column tags and parameter tags

While the Miles block didn’t have any need for them, parameter and column tag functions provide a great amount of control over the display and formatting of parameter dialog items and the columns of data in data tables.

Column and parameter tags allow you to put strings, checkboxes, buttons, dates, popup menus, the infinity character, and so forth into a data table’s cell or a parameter field. They can also be used to hide or disable a column or parameter.

The functions are listed in “Formatting/interactivity using column and parameter tags” on page 305. The Data Init block (Value library) is an example of using column tags.

Include files

Include files are standard header files that are put in ModL code and can contain ModL commands such as definitions, assignments, and functions. The purpose of an include file is to simplify maintenance when several blocks use similar variable definitions and functions.

Like ModL in general, include files allow for the overriding of functions and message handlers and can contain preprocessor statements for conditional compilation. They are discussed more in “Include files” on page 79.

External source code

When multiple developers work on a block or library of blocks, the external source code feature is especially useful. This allow the source code of individual blocks or libraries of blocks to be saved as external files for editing. External source code is discussed on page 41.

DLLs and Shared Libraries

A legacy of code, or programming capabilities that are outside of ModL’s range, can be accessed using DLLs (for Windows) or Shared Libraries (on the Macintosh). See page 40 for more information.

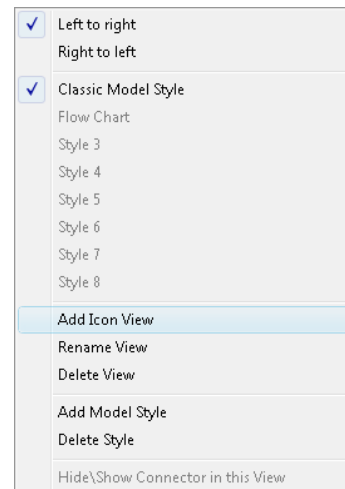
Icon views

Each block has an icon that is its default icon view. You can give a block additional icon views using the Views popup menu. For example, a block could have a Forward and a Reverse view, or different views depending on the type of machine it represents. The modeler selects icon views by right-clicking on a block or using the Views menu to the left of the scrollbar in the block’s dialog.

By default, icon views are created for the Classic model style; you can select a different model style to create icon views for.

The first icon that is created in the block’s structure window becomes the default view. When a view is added, ExtendSim copies the current view into the new view. This gives you something to start with instead of having to redraw all of the icon. The Rotate and Flip commands in the Model menu facilitate creating a new view that represents a flow in a new direction. These commands work only on selected objects.

Views can be deleted or renamed at any time by using the commands in the Views popup menu. An example is the Select Value Out block (Value library).



Views popup menu

How view order is maintained within classes

ExtendSim attempts to maintain view order when model classes are changed. For example, if you have selected a view called *Downward* as the second view in the Classic class and you then change the visible class to Flowchart, ExtendSim will choose the second view in the Flowchart class, assuming it also is the *Downward* view.

ModL

The ModL Language

A detailed description of ModL constructs and structure

*“Knowledge is of two kinds. We know a subject ourselves,
or we know where we can find information upon it.”*
— Samuel Johnson

ModL is the ExtendSim programming language; it is structured much like C++.

☞ A table comparing ModL to C++ starts on page 24; a comparison of ModL to other languages starts on page 26.

This chapter describes the ModL structure and constructs. It is intended as a reference to the ModL programming language; it is not a programming tutorial.

☞ To get the most out of this chapter, you should first read the chapter “ModL Overview”.

The best introduction to programming with ModL is familiarity with programming in C or C++. If you are familiar with programming in some other language, that will help as well. If you have not programmed in any language, a beginning programming class or tutorial would probably be a better starting point than trying to learn programming by building ModL blocks.

Names

Names for variables, constants, and functions can be up to 63 characters. Names can have letters, numbers, and the underscore (_) character; a name must begin with a letter or an underscore (_). Some names are reserved for system use, as described throughout this chapter.

ModL is a case insensitive language. This means that the following identifiers appear the same to the ModL compiler:

Myname **MYNAME** **MyName** **myname**

Data types

There are three main data types in ModL:

- 1) real or double
- 2) integer or long
- 3) string (Str15, Str31, Str63, Str127, Str255 or String)

For the real/double and integer/long data types, the identifiers are interchangeable. The type declarations and constant definitions are set up like they are in C.

Reals

Real numbers are stored as Extended IEEE floating point numbers with 16 significant digits. Numeric literals containing a decimal point or E notation are assumed by the compiler to be real numbers. The range of real values is approximately $\pm 1e\pm 308$.

A ModL real/double variable is the equivalent of a double variable in C++. It occupies 8 bytes of memory.

Integers

Integer numbers are stored as 32-bit integers. The maximum value is 2147483647, and the largest negative value is -2147483648.

A ModL integer /long variable is the equivalent of a long integer variable in C++. It occupies 4 bytes of memory.

☞ ModL treats a non-zero value as TRUE and a 0 value as FALSE. The constant TRUE is defined as 1 and FALSE is defined as 0.

Strings

StrXX may contain up to XX characters and Strings may contain up to 255 characters. String literals (constants) are sequences of characters enclosed by quotes ("..."). The quote character itself may appear in strings by using two adjacent quotes. For example:

```
"It's called ""ExtendSim"""
```

is evaluated as:

```
It's called "ExtendSim"
```

For a string literal that you want to extend past one line of the code, put a back slash (\) character as the last character on the line. For example:

```
longString = "This is a very long string \
literal that is on more than one line";
```

A string variable is the equivalent of an unsigned char x[265] variable in C++. Str255 occupies 256 bytes of memory; smaller string types (StrXX) occupy XX+1 bytes of memory.

ModL strings are not accessed in the same way as strings in C++. To access the contents of a ModL string beyond assigning its contents, you need to use ModL functions.

- ModL strings are internally stored as Pascal style strings, with a leading size byte. In most cases you will not have to worry about the internal storage of the string. However, it does become relevant when you pass a string to outside code through a DLL function call. In that case, the ModL functions DLLPtoCString and DLLCtoPString will be useful - see the function list that starts on page 254.

Declarations and definitions

Static and local

Variables and constants must be declared before being used. Static and constant declarations are made at the beginning of a block's code and local declarations are made at the beginning of a message handler or user-declared function.

The values of static variables are stored in the block and are saved in the model file. Thus, they may be used to store data which must be preserved from one run of a simulation to the next. However, they are not automatically initialized and must be initialized in your block code.

Within each block, and also locally within each function in block code, there is a limit on the total amount of static data that can be declared. If you exceed this limit, the compiler will give an error message when the block code is compiled. The total amount of data that can be defined in static variables in the block, and in local variables within each routine, is 32,767 bytes of data. (See "Type declarations" on page 64 for the amount of memory used by each type of variable.)

This limitation is not an issue for most users because the more common and usually more useful way to allocate large data structures is with dynamic array, global arrays, and database tables. These structure are not subject to, or effected by, the static data limit.

- The advantages of using local variables are that they are not saved with the model and only use memory when the function is called. The disadvantages are that the values of local variables are not remembered after the message handler or function is left, and local variables can override static variables of the same name.

-  Uninitialized static variables can be subtle but serious bugs. The value of an uninitialized variable will often take on a default value of zero. If this is the correct initial value (which it often is) this

can make the code seem like it's working. However, due to the vagaries of uninitialized memory, the variable will sometimes take on other values and cause code to not work in unpredictable ways.

Type declarations

The general forms of the type declarations are:

```
REAL      id, id, .....id;      or  (DOUBLE      id, id, .....id;)
INTEGER   id, id, .....id;      or  (LONG        id, id, .....id;)
STRXX     id, id, .....id;      //Str15, Str31, Str63, Str127, Str255
STRING    id, id, .....id;      or  (Str255      id, id, .....id;)
```

Each variable type consumes a different amount of data, with integers using 4 bytes, doubles using 8 bytes and strings using 256 bytes. At the same time, there is a limit on the total amount of static data that can be declared. Thus if you define a string array as `string x[200]` (which uses 200x256 or 51,200 bytes of memory), it will exceed the local or static data limit of 32,767 bytes and the compiler will report an error condition.

Constant definitions

The general form for a constant definition is:

```
CONSTANT id IS literal;
```

Note that in the constant definition the data type is implied by the format of the literal value:

- For a constant to be real, it must either contain a decimal point or be in E notation
- A string must be enclosed in quotation marks

ModL includes four general-purpose predefined constants; they are discussed on page 76.

Regardless of where they are defined in the ModL code, constants are always static.

BLANK and NoValue

The constant BLANK is a special value that represents “no value”. A NoValue can only be represented by a real variable, not an integer. Technically, it is not a number and appears in a dialog as a blank item. To make a variable a NoValue, assign BLANK to it (`a = BLANK`).

If a number and a NoValue are added, multiplied, divided or subtracted, the answer is always a NoValue. Thus, if any of the values in any equation is a NoValue, the result will always be a NoValue.

If a real value is divided by 0, the answer is a NoValue; the square root of a negative number is also a NoValue. In fact, any operation on numbers that causes an undefined result or an infinity produces a NoValue answer which can be tested with the NoValue function listed in the section “Basic math” on page 217.

To test for a NoValue, do not compare it to BLANK in an “if” statement. Instead, always use the NoValue function:

```
if (myValue == BLANK)      // WRONG! THIS WILL NOT WORK!

if (NoValue(myValue))      // this will always work
```

NoValues can cause unexpected problems when converting reals to integers. It is a good idea to screen for NoValues before converting reals to integers, as discussed in “Numeric type conversion”, below.

Numeric type conversion

Generally, ModL performs all type conversion automatically. Thus, integer values can be assigned to reals, and mixed-type arithmetic can be performed without explicit type conversion beforehand.

 Because conversions are computer intensive, it is best to avoid numerical type conversions. This is particularly true whenever there are repeated or often-used calculations.

ModL converts the arguments of function calls to the type needed by the function. For example:

```
a = cos(integer);
```

Because the cosine function expects the argument to be a real value, the integer is converted to a real value before the function is called.

Real to integer

Like all programming languages, converting from real numbers to integers in ModL is not always exact. If a real number is also an integer, it will be exactly converted. Otherwise, the conversion will cause the fractional value (mantissa) to be truncated. Thus, 0.001*1000.0 may not equal 1 (the integer value), but may equal 0.99999... When this number is converted to an integer, the answer is 0, not 1.

 In an operation between an integer and a real, the result is always a real.

NoValue to integer

As described above, setting a real value into an integer variable has a consistent result, namely the truncation of the real value. This is true in all cases *except* where the real variable contains a BLANK, or NoValue, value.

 If a real contains a BLANK or NoValue value, the NoValue will be too large to fit into an integer. Thus the integer will contain a meaningless value which could cause problems in calculations.

The best method for dealing with this potential problem is to screen the real values for NoValues before assigning them to the integer variable. The following code is an example of how to do this.

```
Real      realV;  
integer   intV;  
  
if (NoValue(realV))  
    intV = 0;           // Can't convert NoValue to integer  
                        // Meaningless result  
else  
    intV = realV; // Can convert real to integer
```

Integer to real

There is no loss of information or special concerns when converting an integer to a real number.

 However, using integer constants for real expressions is dangerous and can give the wrong results.

For example, consider the following code:

```
z = 1/2*a;
```

In this case, z will always equal 0. The integer 1 divided by the integer 2 equals 0 because the result of integer operations must be an integer. The statement should be instead written as:

```
z = 1.0/2.0*a;
```

Integer or real to string

When the equation calls for it, ExtendSim automatically converts reals and integers to string values. If a real variable is a NoValue, the resulting string will be an empty string.

 The result of an operation between any type and a string is a string.

This makes it easy to make strings that contain numeric results, such as:

```
anOutputString = "The answer is " + aRealNum + ".";
```

String to real or integer

The StrToReal function converts a string value to a real number. If the string contains non-numeric characters, the result will be a NoValue.



Do not assign a non-numeric string to an integer. This will assign an unusable value to the integer.

Arrays

You can use real, integer, or string arrays. Any array can have up to five dimensions, that is, up to five subscripts or indexes. Array indexes are limited to 2 billion elements. (There are also other array-structure types: linked lists and global arrays. See “Array-like structures” on page 68.)

Array declarations

The number of dimensions and the magnitude of each dimension are determined by the array's type declaration. The magnitude of each dimension appears in the square brackets in the declaration statement, and there must be one set of brackets for each dimension. The subscripts in an array start at 0. The type declarations are of the form:

```
TYPE id[dim1][dim2]...;
```

Thus the declaration:

```
REAL MyArray[3][4];
```

declares an array of real numbers identified by the name MyArray. This is a two dimensional array, with three rows and four columns.

Individual elements of arrays are treated just like variables in ModL. Thus, for an array declared as

```
integer a[2][3]
```

you can assign the value 4 to the first row in the second column with:

```
a[0][1] = 4;
```

Remember that array subscripts start at 0 and end at 1 less than the number of elements in the declaration (that is, there are n elements in 0 to n-1).

Fixed and dynamic arrays

ModL has both *fixed* and *dynamic* arrays.

Fixed arrays have specified sizes that cannot be changed in the code. The leftmost dimension of dynamic arrays are variable in size and can be assigned a size or resized without changing existing data. Dynamic array size is limited to two billion elements, and you can declare up to 254 dynamic arrays per block. Dynamic arrays can be passed between blocks or used globally. See “Passing arrays” on page 98.

Dynamic arrays are declared in the same manner as fixed arrays, except that their first dimension value is missing. For example:

```
REAL MyArray[ ][4];
```


defines a two-dimensional dynamic array of real numbers. There is a varying number of rows and exactly four columns (indexed 0 through 3).

Dynamic arrays must be static variables, not local ones. Thus, you cannot declare a dynamic array variable in a message handler. However, you can resize dynamic arrays inside of a message handler or in a user-defined function.

There are several functions for sizing dynamic arrays:

Function	Use
DisposeArray	Frees memory when you are finished with the array
DynamicDataTableVariableColumns	Causes the right dimension to be variable. This is only useful for data tables.
GetDimension	Returns the size of the missing left dimension
GetDimensionByName	Returns the size of the missing left dimension based on the block number and the name of the array
GetDimensionColumns	Returns the number of columns in a two dimensional array
GetDimensionColumnsByName	Returns the number of columns in the two dimensional array, based on the block number and the name of the array
MakeArray	Sets the size of the missing left dimension
MakeArray2	Sets the size of the missing left dimension for an array in any block, including a block other than the one calling the code. It allows resizing of dynamic arrays passed in to functions as a string name.

The GetDimension function can be used with any array, not just dynamic arrays. GetDimension returns the value of the first (leftmost) dimension, whereas GetDimensionColumns returns the value of the rightmost dimension in a two dimensional array.

These functions are described fully in “Dynamic and non-dynamic arrays” on page 352. Also see “Variable column data tables” and “Dynamic data table resizing” on page 298.

Arrays as arguments to functions

When passing an array name to a function, it is necessary to differentiate between passing the entire array and passing only an element of the array.

 To pass an entire array, supply only the array name, without subscripts. To instead pass only a single element of an array, use a subscripted array name.

Many ModL functions take arrays as arguments. To pass an array that has the same dimensions as the function needs, simply use the array’s name. For example, the arguments to the AddC function must be arrays with two element; that is, declared such as **real a[2]**. Assume you have three arrays declared as:

```
real x[2], y[2], z[2];
```

You would call the AddC function as:

```
AddC(x, y, z);
```

If you have an array with more dimensions than are needed by the function, you can specify an *array segment* as an argument. An array segment is an array with fewer dimensions than the full

array. Array segmenting can only be done by specifying the leftmost dimensions, not the rightmost dimensions. For example, assume that you had the following declarations:

```
real x[2], y[2];  
real z[50][2];
```

To pass the fifth row of **z** (which contains two elements), to a function such as **AddC** (which only wants an array of one dimension), you would use:

```
AddC(x, y, z[4]);
```

Array-like structures

Global arrays

There is another kind of array that can be set up using functions and is useful for data needed globally throughout the model. *Global arrays* provide a repository for model-specific data. Global arrays are dynamic arrays that, like global variables, can be accessed by any block in the model. However, they differ from global variables in the following ways:

- Global arrays are accessed and managed through a suite of functions.
- You create and dispose global arrays as they are needed. There is no limit to how many global arrays can be associated with a given model.

See “Using passed arrays to make structures” on page 101 for an example of using global arrays to make structures. The global array functions start on page 355.

Linked lists

ModL has a suite of functions to support a linked list data structure. A *linked list* is an internal data structure that allows the construction and manipulation of complex lists of data. These queue-like, multiple type structures maintain internal pointers between the different elements, speeding movement of elements (sorting) around within the list.

- Each structure element can simultaneously contain any number of integer, real, and string data types, allowing the creation of complex sorted structures.
- They are faster than their linear equivalent if their internal sorting functionality is taken advantage of, as in a Queue block (Item library).
- They are owned by individual blocks but can be accessed globally from any block.

See “Linked lists” on page 361 for a suite of functions. See the queue blocks in the Item library or the blocks in the Rate library for examples of using linked lists.

Database tables

ExtendSim database tables are array-like structures. For more information about the ExtendSim database see “Working with databases” on page 105. The list of database functions starts on page 333.

Operators

ModL offers a full set of mathematical and logical operators, as shown below.

Assignment operators

Operator	Description
<i>id</i> = expression;	assignment statement
<i>id</i> += expression;	equivalent to <i>id</i> = <i>id</i> + expression
<i>id</i> -= expression;	equivalent to <i>id</i> = <i>id</i> - expression
<i>id</i> *= expression;	equivalent to <i>id</i> = <i>id</i> * expression
<i>id</i> /= expression;	equivalent to <i>id</i> = <i>id</i> / expression
<i>id</i> ++;	equivalent to <i>id</i> = <i>id</i> + 1
++ <i>id</i> ;	also equivalent to <i>id</i> = <i>id</i> + 1
<i>id</i> --;	equivalent to <i>id</i> = <i>id</i> - 1
-- <i>id</i> ;	also equivalent to <i>id</i> = <i>id</i> - 1

Math operators

Operator	Description
+	addition and concatenation
-	subtraction
*	multiplication
/	division
^	exponentiation. ModL (unlike C) uses ^ to denote exponentiation, as in 2^4.
%	modulo
MOD	modulo

The + operator is used both to add numeric values and to concatenate strings. For example:

```
str = "My name"+" is Ralph";
```

returns the string "My name is Ralph".

```
str = "Time = " + currentTime;
```

returns the string "Time = 5.65" if currentTime equals 5.65.

The % and MOD operators return the remainder after integer division. For example, 5 MOD 2 returns 1 and -5 MOD 2 returns -1. If the MOD operator is used on real values, they will be truncated to integers before the operation is carried out.

 Any operation involving a noValue (BLANK) produces a noValue result. NoValue results appear as blank entries in tabular data and are not reflected in plotted traces at all.

Boolean and magnitude operators

Operator	Type	Description
AND or &&	Boolean (logical)	combination
OR or	Boolean (logical)	conjunction
NOT or !	Boolean (logical)	inverse
!= or <>	Magnitude	not equal to
<	Magnitude	less than
<=	Magnitude	less than or equal to
>	Magnitude	greater than
>=	Magnitude	greater than or equal to
==	Magnitude	equal to

The three standard Boolean operators are not bit-wise logical operators, but only operate on TRUE or FALSE expression values.

The standard magnitude operators return a Boolean value (1 for true and 0 for false) after comparing their arguments.

Operators in an expression are generally evaluated from left to right; however there is a hierarchy of precedence among the operators. The following list is in *descending* order of precedence; within each group, operators have equal precedence.

 Putting any expression in parentheses ensures that it will always be evaluated first.

- 1) ()
- 2) - (negation)
- 3) NOT or !
- 4) ^
- 5) MOD or %, *, /
- 6) + (addition and concatenation), -
- 7) <, >, <=, >=, =, <> or !=
- 8) OR or ||, AND or &&

Control statements and loops

ModL supports a full complement of structured programming control statements. In this table, “boolean” evaluates to TRUE or FALSE. The control statements are:

Statement	Use
Abort	Abort;
Break	Break;
Continue	Continue;

Statement	Use
Do-While	Do STATEMENT; While (boolean); // Note semicolon
For	For (init_assignment; boolean; incr_assignment) STATEMENT;
GoTo	GoTo label; ... label: //note the colon ...
If	If (boolean) STATEMENT;
If-Else	If (boolean) STATEMENT_A; Else STATEMENT_B;
Return	Return; or Return(value or expression);
Switch	Switch (expression) { CASE integerConstant: many STATEMENTS; Break; CASE integerConstant: many STATEMENTS; Break; DEFAULT: many STATEMENTS; Break; }
While	While (boolean) // Note no semicolon STATEMENT;

Multiple statements can be grouped with braces:

```
{
statement;
statement;
...
}   // Note no semicolon here
```

Use the *If* statement for boolean comparisons such as

```

if (total < 0)
{
    isNegative = TRUE;
    findings = abs(total);
}
...

```

You can also use the *If-Else* construct if you have two paths to choose from:

```

if (total < 0)
{
    isNegative = TRUE;
    findings = 100 + total;
}
else
    findings = 100 - total;
...

```

The *FOR* construct lets you set the initial value, continuing boolean condition, and action to take on each step in the parentheses. For example:

```

for (i = 0; i <10; i++)
    a[i] = i;

```

will set a[0] to 0, a[1] to 1, and so on up to a[9].

The *While* loop repeats the statement while the expression is true; the expression is evaluated at the beginning of the loop. Thus,

```

x = 3;
y = 3;
while (y<3)
{
    x++;
    y++;
}

```

would leave x and y set to 3 because the loop is never executed.

The *Do-While* construct tests at the end of the loop, so

```

x = 3;
y = 3;
do
{
    x++;
    y++;
}
while (y<3);

```

would leave x and y set to 4 because the test occurs at the end of the loop, after x and y have already been incremented.

The *Switch* statement is used to check values against integer constants and act on those cases. Note that the integers in the *CASE* statement must be constants, not variables. For instance,

```

switch (numberOfRepeats)
{
  case 0:
    UserError("You didn't run it.");
    wasOK = FALSE;
    break;
  case 1:
    UserError("Thank you, it ran once.");
    wasOK = TRUE;
    break;
  default: // any other number
    UserError("You ran it more than once; please stop.");
    wasOK = FALSE;
    break;
}

```

The *GoTo* syntax is supported only within a message handler or user-defined function. Control is unconditionally transferred to the statements after the label.

The *Abort* statement stops the current message handler. You can use this at any point, even inside loops. For example, in a *DialogOpen* message handler, the *Abort* statement would prevent the dialog from opening. The two most common uses of the *Abort* statement are in the *CheckData* and *Simulate* message handlers

- In *CheckData*, it can be used to abort if the modeler's data is bad.
- In the *Simulate* message handler, it aborts the current simulation if something goes wrong with the calculations. (See the *AbortAllSims* function to abort multiple simulations.)

ModL also provides two control statements for exiting from loops and other control structures:

- *Break* immediately exits an enclosing *For*, *While*, *Do*, or *Switch* construct.
- *Continue* immediately sends control to the next iteration of an enclosing loop.

The *Return* statement is used to exit message handlers and user-defined functions.

- When returning from a message handler or user-defined function that does not return a value, use:

Return;

- When returning from a user-defined function that returns a value, use:

Return(value or expression);

User-defined functions

In addition to the many functions that are included with *ExtendSim*, you can define your own functions and override them by re-declaring them.

User-defined functions make ModL code more readable and allow you to create tools that can be reused. They are defined and called as they are in C and, unless they are in include files, are local to the blocks in which they are defined.

 To make one or more user-defined functions available for use by multiple blocks, create an include file as discussed in "Include files" on page 79.

In addition to the information below, see:

- "Passing messages between blocks" on page 108 for a method of defining "global" functions

- “Pass by value and reference (pointers)” on page 98 for information on passing arguments to functions
- The user-defined ActiveX Data Object (ADO) functions listed on page 380.

User-defined functions have the following form:

```
TYPE (or VOID) name (TYPE id, TYPE id,..., TYPE id) // zero or more arguments
{
optional local variable declarations

zero or more statements

Return(value);
}
```

In these definitions, VOID is a function that does not return any value and TYPE and VOID can be integer, real or string. All arguments are optional. Arrays may be passed as arguments.

User-defined functions can be *recursive*, that is, they can call themselves.

Defining

User-defined functions must be defined before they are used since the ModL compiler needs to know the types of the arguments for type conversion. (Type conversions are discussed on page 64.)

If you define a function that calls another user-defined function, both cannot be defined first. The solution is to have a declaration of the function before it is actually called in the ModL code:

```
TYPE (or VOID) name (TYPE id, TYPE id,..., TYPE id); //Note the ";"
```

This is called a *forward declaration* and tells ModL the types of the function and arguments, and that the function will be defined later. The form is exactly the same as a function definition, but instead of braces, it ends in a semicolon.

Exiting

Exit a user-defined function at any point with a Return statement for Void type, Return(value) for other types, or the Abort statement (see Abort, above).

For example:

```
real MyCalc(real x1, real x2)
{
real sum;          // declare a temp variable

sum = x1+x2;       // the calculation
Return(sum);       // return the sum
}
...
y = MyCalc(a, b); // calc using function
...
```

If Abort is called in a function that was otherwise going to return a value, the code is halted completely and the return value is no longer necessary. That is, the code execution does not return to the calling routine, so the return value of the function is moot.

Overriding user-defined functions

User-defined functions can be overridden by being re-declared any number of times below the first declaration. In this case, the code that is executed is the final version of the routine. In the code pane, the earlier versions of the routine will be colored brown to show that they have been overridden.

Overriding is useful in that include files can have basic forms of functions and message handlers which can be re-declared and overridden in the main block code. See “Include files” on page 79.

Declaring arrays as arguments for user-defined functions

This section shows how to create a function that has arrays as arguments.

If the array size is not fixed, you can declare arrays as arguments to functions with the first (left-most) dimension blank; additional dimensions must have a value. This allows you to pass either kind of array (fixed or dynamic) and any length of array to a user-defined function.

The following is an example of functions that add all of the elements of the rows in a variable-sized array and return the sum.

```

real rowSum(real x[])
{
  integer length, i;           // Temporary variables
  real sum;

  length = GetDimension(x);    // Returns first dimension
  sum = 0.0;                   // Initialize sum

  for (i=0; i<length; i++)     // For all elements
    sum = sum+x[i];            // Add element to sum

  return(sum);
}

on Simulate
{
  real      valuesArray[36], mySum;
  ...
  mySum = rowSum(valuesArray);
  ...
}

```

Message handlers

The format of a message handler is:

```

on MessageName
{
  local variable declarations
  zero or more statements;
}

```

MessageName must be the name of one of the messages listed in the chapter “Messages and Message Handlers”. You exit from a message handler with a Return statement or an Abort statement, as described above.

See “Using message handlers” on page 39 and the list of message handlers starting on page 204.

Overriding message handlers

Message handlers can be overridden by being re-declared any number of times below the first declaration. In this case, the code that is executed is the final version of the message handler. In the code pane, the earlier versions of the message handler will be colored brown to show that they have been overridden.

Overriding is useful in that include files can have basic forms of functions and message handlers which can be re-declared and overridden in the main block code. See “Include files” on page 79.

System variables

System variables give you information about the state of the simulation. You can read or write to these variables, but you should be careful when writing to any of them.

The list of system variables starts on page 200.

Global variables

There are two types of global variables:

- General use global variables have a name that starts with “global”. They can be used any way you want.
- Reserved global variables start with the word “SysGlobal”. These system globals are controlled by the libraries that are included with ExtendSim and are reserved for use with those libraries.

The list of global variables starts on page 201.

Constants

ModL includes four numerical predefined constants:

```
PI = 3.14159265358;
BLANK = (noValue);
TRUE = 1;
FALSE = 0;
```

Setting a dialog parameter to BLANK will make it display as an empty field. BLANK values show nothing in a dialog’s text field and are NoValues for all math calculations.

 Do not compare the constant BLANK with a value to determine if the value is BLANK (that is, NoValue). The result will always be FALSE. Use the NoValue function instead. This function returns a TRUE if the value passed to it is a NoValue.

There are other predefined constants that are specific to functions, such as the pattern constants used with the Animation functions.

Conditional compilation

ModL supports several preprocessor directives to bracket ModL code. This allows specific parts of the code to be compiled depending on circumstances. For more information, see “Conditional compilation” on page 79.

ModL

Programming Tools

Features and capabilities you can use
as you program in ModL

*“In baiting a mousetrap with cheese,
always leave room for the mouse.”
— Hector Hugh Munro*

This and the next four chapters discuss aspects of programming in ExtendSim:

- Programming tools: features such as include files and external source code that support and simplify your programming efforts
- Programming techniques: using ModL functions to accomplish almost anything
- Simulation architecture: how continuous and discrete event models work and why you should know this information
- Animation: programming aspects of 2D and 3D animation
- Debugging

Writing ModL code takes much of the same talents as writing C/C++ programs. However, ExtendSim provides tools to help make writing block code easier and safer, as discussed in this chapter.

Formatting ModL code and Help

This section gives tips on how to edit and format block Help and code.

Searching and replacing

The Edit menu has many commands that help edit a block's code.

- The Find command displays a different dialog depending on whether or not text is selected. If text is selected in the code pane, it lets you define a string to find as well as a replacement string for the Replace command.
- The Enter Selection command gives you an easy method for finding the next instance of some text that is in your code. Select the text before using the Enter Selection command places the selected text in the *Search for* box. The Replace command uses the *Search for* and *Replace with* strings in the Find command's dialog. Thus, you can set up a search and a replace in one dialog. If there is no string in the Replace with: entry box, the text you searched for is deleted (that is, replaced with nothing).

Using the Develop > Go To Line command causes the code pane to scroll to that line number.

These commands are discussed more in the Menu Command Appendix of the User Guide.

Formatting tips

To make it easier to follow the logic, ExtendSim colors different aspects of the code – see “Syntax coloring” on page 29.

The Develop menu's Shift Selected Code Left and Shift Selected Code Right commands let you change the tabbed indentation on lines in ModL code. For example, if you copy lines from another block's code that are at a different indentation level, select them and use the appropriate command to move them to the correct level. These commands only work if you have used the Tab key for indentation.

You can copy names from the variables pane of a block's structure window into the code or the Help panes with the Copy and Paste commands or with drag-and-drop editing.

To change the default font and size for the text of the ModL code, use the Programming tab of the Edit > Options dialog.

The text on icons and the Help text can have character formatting. Select the text and give commands from the Text menu. You can also specify a color for the text from the toolbar's Color tool.

Include files

As discussed on page 73, unless they are in an include file, user-defined functions are local to the block in which they are defined. An include file can contain user-defined functions and predefined constants; the functions and constants are accessible by any block that includes that file. This simplifies programming tasks that are repeated in multiple blocks, such as a library of blocks that use similar variable definitions and functions. Include files are standard header files. ExtendSim allows you to use include files in ModL code; they can contain any ModL commands such as definitions, assignments, and functions.

To start a new include file, be sure a structure window is active and choose the command Develop > New Include File. This opens an untitled text file window. Type the statements you want in this window and choose the File > Save Include File As command. This saves the include file into the ExtendSim7\Extensions\Includes folder. For all platforms, the include file's name *must* end with ".h" (like standard C include files) and the file *must* be either in, or in a subfolder within, the Extensions\Includes folder that resides in the same folder or directory as ExtendSim.

To reference an include file from a block's code, enter a line in the format:

```
#include "filename.h" (or #include "subfolder\filename.h")
```

or

```
#include <filename.h> (or #include <subfolder\filename.h>)
```

For example, if the name of the include file is "New_Defs," you would use the command:

```
#include "New_Defs.h"
```

You can have as many include files as you wish, as long as all of them reside in the Includes folder within the Extensions folder.

For an example of how include files are useful, see the Data Import Export block (Value library) which uses an include file named ADO_DBFunctions. The ModL-coded ADO functions for that include file are described on page 380.

 Include files are also called header files because they are usually included at the top or head of the file. This is the source of their .h extension.

Conditional compilation

With conditional compilation, specific parts of the code will be compiled depending on the presence of certain "symbols". A symbol can be any kind of variable, function name, dialog item name, or constant. The code will be either used or not used, depending on whether the symbol is defined in the code. You can also define your own symbols using #define, as discussed below.

ModL supports several preprocessor directives to bracket ModL code:

- #ifdef. If the symbol is defined, compile the code following the #ifdef until you get to a #else or a #endif.
- #ifndef. If the symbol is not defined, compile the code following the #ifndef until you get to a #else or a #endif.
- #endif. Marks the end of the current preprocessor directive.
- #else. Used with #ifdef or #ifndef to give an alternative set of code to be compiled.
- #define. Used to define your own symbols when you need to change your code based on the existence of that new definition, for example within an include file. After defining a new symbol, you can use it in the #ifdef and #ifndef directives. The syntax of the #define is:

```
#define myNewSymbol // define a new symbol
```

For example, you can create an include file that has code to modify a dialog item. But if that dialog item isn't used in a particular block, the code will be ignored by the preprocessor directives.

The following is an include file generalized for different blocks.

```
...
void AFunction(real anArgument)
{
...
#ifdef myDialogItem // Only if myDialogItem is present
    myDialogItem = anArgument; // display result in the dialog
#endif
}
```

A common situation is to use `#ifdef` to compile a message handler only if a dialog variable exists. For example, an include file may be used by many different blocks, only some of which have the dialog variable `MyValue_prm`. To prevent the compiler from giving an error message when a block does not include the dialog variable, add the following to the include file:

```
#ifdef MyValue_prm
    On MyValue_prm
    {
        //message handler for dialog variable
    }
#endif
```

External source code control

ExtendSim supports a mechanism for saving the source code of individual blocks or libraries of blocks as external files. This external source code feature is useful for situations where multiple developers work on the code of blocks and/or libraries at the same time.

Normally the code of blocks is saved in the library file, along with the blocks' dialog item definitions and icons. When blocks or libraries are recompiled with the external source code option turned on, the library file does not contain the master source code. Instead, the source code is saved in a separate subfolder inside the Libraries folder. It can then be used with a separate source code control or management application.

How to save or remove external source code

Externalizing a block's code

To externalize source code for a block:

- ▶ Open the block's structure
- ▶ Choose the command Develop > External Source Code
- ▶ Close the block's structure window
- ▶ In the dialog that appears, choose *Save, compile if needed*

This causes the block to be recompiled with its source code in an external file.

 In the library's Library Window, the green initials CM (for code management) will be displayed to the right of the block's icon.

To restore the block's code to its structure:

- ▶ Open the block's structure
- ▶ Unselect the command Develop > External Source Code
- ▶ Close the block's structure window
- ▶ Save and compile the block

Externalizing an entire library's block code

To externalize source code for an entire library

- ▶ Open the Library Window for the desired library
- ▶ Give the command Library > Tools > Add External Code to Open Library Windows

This recompiles the library and causes block source code to be placed in an external file.

 In the library's Library Window, the green initials CM (for code management) will be displayed to the right of each block's icon.

To restore the source code to the library file

- ▶ Open the Library Window for the desired library
- ▶ Give the command Library > Tools > Remove External Code in Open Library Windows

The consequences of saving the code externally

Using the external code option for a block will:

- Create a folder named "Source" within the Libraries folder, if the Source folder does not already exist.
- Place a folder with the name of the library inside that Source folder, if that library folder does not already exist.
- Create a source code text file for each block that has been recompiled with external source code and places it in the library name folder. Each file will be named with the block name and end with a .cm extension.
- Cause the green text *CM* (for "code management") to appear on the icons in the library window, for each block that has been recompiled with external source code.
- Create a backup file, with the extension .ck, each time the block is recompiled.

Using the external source code with code management software

The primary reason for using the external source code option is in conjunction with a separate source code control or management software. This allows multiple people to work on the code of the blocks at the same time. For example, Subversion is an open source version-control system that is available for download from the Web. The basic structure would be a situation where Subversion or some other source code control software would maintain an archive on a server and synchronize the source folders on each modeler's machine with that central archive.

Because the source code for each block is saved in a separate .cm file, which is just a text file, the source code control software can maintain the file and synchronize any changes made by the modelers with what is in the central archive.

 Dialogs, icons, and help text are not editable by multiple developers at the same time. See "Managing non-code parts of blocks", below.

Cautions when using the external source code feature

Managing non-code parts of blocks

With external source code turned on, the source code for each block is saved as a text file. This facilitates the management of block code by multiple developers using source code control software. However, the block dialogs, icons, and help text are not saved in the external files. Instead, they are saved in the original library file.

When changes need to be made to block dialogs, icons, or help text, the library should be “locked” using the source code control software. That way, no one else can modify those parts of the block while they are being changed. After the changes have been made, unlocking the library releases the blocks so other developers can work on them.

Sharing libraries with others

When you have recompiled a library with the external code option, you need to be careful about how you manage and share that library. For example, if you give the library to someone else without giving them the source code files, they will receive warning messages when opening the structures of the blocks. Before giving the library to another person, recompile the library with the external source code option turned off.

Extensions folder

When you installed ExtendSim, you also installed an *Extensions* folder in the same folder or directory as the ExtendSim program. The Extensions folder contains various ExtendSim *extensions* and *include files* stored in subfolders: DLLs, E3DObjects, Includes, and Pictures.

Extensions should be stored in the appropriate subfolder. If there is no subfolder for the type of extension you are adding, put it at the top level of the Extensions folder.

 Do not store anything except valid extensions or include files in the Extensions folder. Storing other files there will cause ExtendSim to operate more slowly and could cause functional problems.

Extensions on Windows

On Windows, extensions are files of various types, such as resource files converted from the Mac OS using ExtendSim's MacWin converter utility, WAV, WMF or BMP files, or DLLs. In addition to the files shipped with ExtendSim, you can also add your own extension files to the Extensions folder. For Windows, ExtendSim supports the following types of files: DLLs, WAV files, bitmap files, and WMF files. These are discussed individually later in this chapter.

Except for DLLs, the name referenced by the ModL functions will be the file name. For DLLs, as described below, the name referenced will be the function name of the particular function you are trying to call from the DLL.

Extensions on the Mac OS

On the Mac OS, most extensions are resource files. Resources are either segments of code or pictures that are automatically opened and become accessible when ExtendSim starts up. You can also add your own resource files to the Extensions folder. For Mac OS systems, ExtendSim supports the following types of resources: sounds, picture resources, and QuickTime movies. These are discussed individually later in this chapter.

You access resource files using the ModL functions listed in the ModL Functions chapter. Except for QuickTime movies, the resources must be copied into standalone files and the files copied into ExtendSim's Extensions folder. Except for QuickTime movies, the name referenced by the ModL functions should be the resource name, which is not necessarily the same as the file name. For sim-

plicity, you can name the resource and the file the same name and keep each resource as a separate file (for QuickTime movies the name used to reference it is the file name).

DLLs

As discussed in “External source code” on page 41, ExtendSim provides sets of functions that allow you to call code segments written in a language other than ModL from within a block’s code. This is handy if you want to access existing functions written in another language or solve problems that are difficult in ModL. On Windows, these functions are identified as DLLs or dynamic-link libraries.

Overview

DLLs are libraries of code written and compiled in any language. Their standardized interface provides a method for linking between ExtendSim and languages other than ModL. In order to access these code libraries, they must be stored in ExtendSim’s Extensions directory, discussed above.

The ExtendSim DLL functions allow you to call DLL code libraries from within a block’s code and use that code to perform operations. For example, you can use a DLL to calculate some function, perform a task, or even access Windows API calls. DLLs can also be used to access external devices, or to solve problems that might be difficult or impossible to solve in ModL.

DLL interface

The DLL functions allow you to access existing Windows DLLs. Because they have variable argument lists, these functions allow you to call almost any existing DLL. This interface includes the following functions: DLLLongCFunction, DLLDoubleCFunction, DLLBoolCFunction, DLLVoidCFunction, DLLLongPascalFunction, DLLDoublePascalFunction, DLLBoolPascalFunction, DLLVoidPascalFunction, DLLMakeProcInstance.

See the DLL functions on page 254 for more information about this interface.

Turning code into a DLL

Taking existing code and turning it into an DLL involves the following:

- ▶ Write or edit the DLL code using a Windows compiler that is capable of compiling DLLs. The DLL must be built for 32-bit execution.
- ▶ Modify the DLL code by adding the DLL calling interface.
- ▶ After compiling the code, you will have a DLL file. Place this file in the ExtendSim Extensions\DLLs directory.
- ▶ Restart ExtendSim. This will allow you to call the external code using the DLL functions in your block code.

You should also make note of the following:

- If there is any possibility that your blocks will be used cross-platform (on both Windows and Mac OS systems), the code in those blocks should include checks to allow for the differences between platforms. For more information, see “Cross-Platform Considerations” on page 403.
- Real and integer variables passed from the ModL code to a DLL are passed by value. Real variables are 8-byte double precision and integers are 4-byte long integers.
- Strings and arrays are passed to DLLs as pointers to data that has been allocated by ExtendSim. Modifications to that data will affect the original information in ExtendSim.

- As discussed above, arrays that are passed to a DLL come through as pointers to the original data in ExtendSim. Accessing and modifying the data is fine, but you should not try to resize the pointer. If you do, ExtendSim will not be able to access the data and will probably crash.
- Strings are passed to DLLs from ModL as Pascal strings, not C strings. This means that the string is preceded by a size byte and is not terminated by a zero. For example, if you pass a string to a DLL, the DLL will get a pointer to 256 bytes of data in which the first byte contains the number of characters in the string. See the DLLCtoPString and DLLPtoCString functions in the function list “Existing Windows DLLs” on page 254.
- The most common problem associated with building and using a DLL is making sure that the names of the routines that you want to call are exported and that they are exported without Name Mangling or Name Decoration. Name Mangling is an option for how names are exported from a DLL; it adds information about the arguments to the exported name

☞ When building a DLL for use with ExtendSim, the Name Mangling option should be off.

DLL example

The following code calls a DLL that performs the same function as the ModL code (shown on page 53) that you used to build the Miles block. (Note that you could accomplish the same result, but in a slightly different manner, using a Shared Library for the Mac OS.)

```
// Declare constants and static variables here.
long proc;
on Calculate_btn // Display the values
{
  proc = DLLMakeProcInstance("convert");
  if (proc > 0) // Check if proc>0 before you make the call
    OutNum_prm = DLLDoubleCFunction(proc, inNum_prm,
      kilometers_rbtn, yards_rbtn, feet_rbtn, inches_rbtn);
  else
    Beep(); // Couldn't find the DLL function "convert"
}
// This message occurs for each step in the simulation.
on simulate
{
  InNum_prm = MilesIn; /* Display the value input by setting the dialog
    variable "InNum" to the input connector value */
  proc = DLLMakeProcInstance("convert");
  if (proc > 0) // Check if proc>0 before you make the call
    unitsOut = DLLDoubleCFunction(proc, inNum_prm,
      kilometers_rbtn, yards_rbtn, feet_rbtn, inches_rbtn);
  else
    Beep(); // Couldn't find the DLL function "convert"
  OutNum_prm = UnitsOut; /* Display the value output by setting the
    dialog variable "OutNum" to the output connector value */
}
```

The code of the DLL, written in C++, is:

```
double __export __cdecl convert(double x, long kilometers, long
yards, long feet, long inches)
{
    if (kilometers)
        return(x * 1.609334);
    else if (yards)
        return(x * 1760.0);
    else if (feet)
        return(x * 5280.0);
    else if (inches)
        return(x * 63360.0);
}
```

The ExtendSim\Examples\Developer Tips\DLLs folder has additional examples of writing a DLL in C and calling it from within an ExtendSim block.

Shared libraries

DLLs as described above are a Windows construct. On the Macintosh, an equivalent functionality can be found in Shared Libraries.

All of the DLL functions also support calling Shared Libraries on the Macintosh OS. And most of the discussion about DLLs applies to Shared Libraries. Some differences and notes are:

- As it does on Windows, DLLMakeProcInstance will first attempt to load the library file before trying to find the procedure within the library file. The loading of the library is implemented internally with a call to the API function “dlopen”. The location of the procedure name within the library is done through a call to “dlsym”.
- Shared Libraries on the Macintosh will have a .dylib extension.
- Once you have created a Shared Library that you want to use with ExtendSim, place it into the DLLS folder within the Extensions folder.

Sounds

Depending on your operating system, ExtendSim has access to several sounds. You can play sounds using the Notify block (Value library) or by calling the PlaySound function in the code of a block.

Sounds on Windows

There are three sources of sounds that ExtendSim can access on the Windows system:

- ExtendSim contains a “click” sound extension.
- Your System contains default system sounds such as beep sounds. To see the system sound names, access the Sound device in the Control Panel in the Main program group in Window’s Program Manager. The sounds listed as .WAV files are the system sounds.
- You can also use .WAV files created by other Windows applications or obtained from user groups.

In order to have access to the sounds which are not System sounds, you must copy the .WAV file into the Extensions directory, as discussed above.

Sounds on the Mac OS

There are three sources of sounds that ExtendSim can access on the Mac OS system:

- ExtendSim contains a “click” sound.
- Your System file contains default system sounds such as beep sounds. To see the system sound names, access the Sound device in the Control Panel under the Apple menu. The sounds listed in the Alert Sounds dialog are the system sounds.
- You can also use sound resources (SNDs) made by other Mac OS utilities or obtained from user groups.

In order to have access to the sound resources which are not ExtendSim or System sounds, you must copy them into the Extensions folder, as discussed above.

Picture and movie files

Pictures and movies are used for specialized 2D animation. See the functions in “2D Animation” on page 259, for more detail. To see an example of using pictures and movies for animation, see “Showing a picture on an icon” on page 125. “Menu and keyboard equivalents for developers” on page 405 provides information about converting pictures between Windows and Mac OS operating systems.

3D objects are different from 2D pictures and are discussed on page 86.

Naming conventions and limitations for pictures

You can have up to 300 pictures in the Extensions folder. Pictures with names that start with a “@” will not show up in the block’s animation tab popup menu. This prevents the animation tab menus from being filled with other, larger types of pictures that are not suitable for animation between blocks.

Pictures on Windows

As mentioned in “Extensions on Windows” on page 82, pictures must be stored as a file in the Pictures subfolder of the Extensions folder. The AnimationPicture function will recognize either a .WMF file, a .BMP file (preferred), or a Mac OS picture resource that has been converted using ExtendSim’s MacWin Converter utility.

Pictures and movies on the Mac OS

As mentioned in “Extensions on the Mac OS” on page 82, pictures and movies must be stored as a resource file in the Pictures subfolder of the Extensions folder. PICT resources are not the same as PICT files. In order to use pictures for animation, they must be saved as a resource file with a PICT-type resource and not as a PICT file from a graphics or drawing program. Movies are Quick-Time movies.

E3D objects

The E3DObjects subfolder in the Extensions folder is for custom user 3D objects. The .cs (script) files and the .dts (object) file, as well as other files that the .dts file may need, should be stored in this folder. See “3D animation” on page 128 for additional details and information.

Protecting libraries

If you build your own blocks and you do not want others to have access to your block code, you can protect your libraries by removing the ModL code of the blocks. ExtendSim normally keeps both the ModL code and compiled code in the block; when you remove the ModL code, the compiled code is still there. When a modeler attempts to edit a protected block, ExtendSim displays a dialog message that the block is protected and cannot be opened. The structure and dialog windows for the protected block are never shown.

A protected library can be used the same as any other library except that you cannot view or alter the ModL code. This means that someone using the library has all the functionality of the blocks in that library but no ability to see how the blocks work. This is a convenient way to hide proprietary programming while still giving modelers full access to the power of the block. Protecting the ModL code has the added benefit of preventing someone from changing the icon or the block's help text.

 After a library is protected, *you can never un-protect it*. Before ExtendSim protects a library it makes a copy of the library. However, it is a good practice to make an additional copy of the original and store it away from your computer on a CD or diskette.

The steps for protecting a library are:

- ▶ Select Library > Tools Protect Library.
- ▶ When you give the Protect Library command, ExtendSim warns you that protecting a library's ModL code will permanently and irrevocably prevent access to block code.
- ▶ In the dialog that appears, select and open the library you want to protect.
- ▶ In the **Create new library** text field, give a name to the library. Note that you can only give the same name as the original library if you save the protected library to a new location. This ensures that you are protecting a copy of the library, not the original. Since your models will be expecting the original library name, it is suggested that you save the library to a new location, rather than changing its name.
- ▶ ExtendSim protects the library ModL code by cutting it from the block.

Any copies you make of this protected library will also be protected. You can easily show that a library is protected by opening it and attempting to edit a block in the library using the Develop > Open Block Structure command.

Please note that libraries require the ExtendSim application to run. You may distribute the libraries you build to other ExtendSim owners but, under the terms of your ExtendSim license agreement, you are specifically prohibited from distributing copies of the ExtendSim application or its libraries. The ExtendSim RunTime and Player licenses allow you to distribute a specialized version of ExtendSim so modelers can change parameters and run models that you build, but cannot build their own models. Contact Imagine That, Inc. for more information.

ModL

Programming Techniques

Procedures and suggestions for how to
create and modify ModL code

*“In baiting a mousetrap with cheese,
always leave room for the mouse.”
— Hector Hugh Munro*

This chapter focuses on using ModL functions to create simulation effects and custom blocks in ExtendSim:

- Dialogs and connectors
- Programming with arrays
- Working with ExtendSim databases or linked lists
- OLE and ActiveX automation

Equation block programs

The equation-based blocks are: Equation, Optimizer, Query Equation (Value library); Equation(I), Queue Equation, Query Equation(I) (Item library); and Buttons (Utilities library). The ExtendSim User Guide discusses how to use the equation-based blocks for entering equations and debugging them. However, these blocks can understand more than just the definition of an equation. They are capable of compiling and executing complex ModL programming logic. Think of the equation blocks as a method for including small programs on the fly without having to create a new block.

You can also create your own blocks that take in equations and allow the user to debug them using the functions described in “Equations” on page 228. For example, you might want to do this when building scientific or engineering blocks and can’t predict ahead of time what formulas you will need.

 To enter larger equations or programs into a block’s dialog, see “Equations” on page 228 for a list of equation functions that support up to 32000 characters in a dialog item. Also, see the Equation block (Value library) for an example of using a Dynamic text item and a syntax coloring window to edit a user-entered equation.

 Equation-based blocks can call any of the ExtendSim built-in functions. However, user-defined functions and procedures cannot be defined directly in an equation-based block; they must be defined in an include file.

Dialogs

With a bit of creativity, you can make dialogs that provide a great deal of information about the state of a simulation or that help you debug models-in-progress.

Changing text in dialogs

The Miles block example on page 44 showed the usefulness of keeping a dialog open and watching the numbers in the entry boxes change as the simulation progresses. Dialogs can also be used for displaying text or messages that might change during the simulation.

Changing text as the simulation runs

Changing the text displayed in a dialog is often more useful than displaying alerts since alerts stop the simulation until the modeler clicks one of their buttons.

For example, assume that:

- You are modeling a factory that has three shifts, where the name of the shift (day, night, swing) changes depending on the current time.
- You have a block with a static or editable entry box named *Shift* that you want to modify.
- Simulation times are in minutes.

You could display the time in the Shift dialog item as a number, but then you have to convert the time to the shift name in your head:

```
Shift = (CurrentTime / 60) mod 24;
```

Instead, you could convert the time in the block's code and display text in the dialog:

```
...
string name[2];
name[0] = "Day"; name[1] = "Night"; name[2] = "Swing";
...
Shift = name[((CurrentTime / 60) mod 24) / 8];
```

Changing text in response to a user's action

ModL lets you change the text and titles of dialog items at any time. This allows you to decide what to show based on what is done in the dialog. An example of this is the Random Number block (Value library), which displays different parameter labels depending on the statistical distribution that is selected.

When a control item (radio button, switch, meter, etc.) is clicked, two things happen:

- The value of the radio button's variable name is set to TRUE; switches and checkboxes are toggled from FALSE to TRUE and vice versa; and sliders are changed in value. Plain buttons don't have a value but their titles can be changed.
- ExtendSim sends a message to any message handler with the control item name. This allows the ModL code to take a special action.

For example, assume that you want to show values as either octanes (for gasoline) or cetanes (for diesel fuel). There are two radio buttons that let you decide which unit to show:

```
// ShowOctaneValues radio button was clicked
on ShowOctaneValues
{
    // set static text label above data table
    SetDataTableLabels("dataTable", "Octane Values");

    for (row=0; row<numRows; row++)
        for (col=0; col<numColumns; col++)
            dataTable[row][col] = octaneVals[row][col];
}

// ShowCetaneValues radio button was clicked
on ShowCetaneValues
{
    // set static text label above data table
    SetDataTableLabels("dataTable", "Cetane Values");

    for (row=0; row<numRows; row++)
        for (col=0; col<numColumns; col++)
            dataTable[row][col] = cetaneVals[row][col];
}
```

```
// When the dialog is opened by the modeler
on DialogOpen
{
  // Static labels are not "remembered" when dialog is closed
  // so, when the modeler opens the dialog, restore the titles here

  if (ShowCetaneValues)           // ShowCetaneValues was TRUE
    SetDataTableLabels("dataTable", "Cetane Values");// restore
                                     // title
  else                             // else ShowOctaneValues TRUE
    SetDataTableLabels("dataTable", "Octane Values");// restore
                                     // title
}
```

See also “Buttons” on page 36 and “Static text (label)” on page 37.

 If the octane and cetane arrays were dynamic, the DynamicDataTable function could be used to switch the data table between the two dynamic arrays. This would execute more quickly and require less code.

ModL

Hiding and showing dialog items

ModL code can hide or show certain dialog items depending on a user action or the value of other items. Do this with the HideDialogItem() functions listed in “Dialog items” on page 291. This can be initially done in an item message handler as in the above section. A good example of hiding and showing dialog items is the Create block (Item library).

When the dialog is opened by the modeler, you need to have a DialogOpen message handler to show and hide certain items depending on the values of the controlling items.

Moving dialog items

ModL code can be used to move dialog items around. The DIPosition functions, as well as DIMoveTo and DIMoveBy, can be used to move dialog items. These functions are included in the list that starts on page 291.

Changing the title of a radio button or checkbox

The function DITitleSet is useful for setting the title of a radio button or checkbox. ModL code can thus change the title, and meaning, of the control at any time.

Changing and reading parameters globally from a block

The GetDialogVariable and SetDialogVariable functions can be used to read and set dialog items by name for any block in the model from within one block. When you combine this with the ability to read block names and labels, this feature can be used to build a block that can globally control the model, gather statistics on a class of blocks, or change parameters within all blocks of a certain type.

For example, you can build a block that can set specified Activity blocks (i.e. Activity blocks whose labels include a specific wording, such as ABC) to double their entered delay value. Here is sample code that does this:

```

on doAllButton      // modeler clicked button to change the blocks
{
integer      nBlocks, i;
real        value;
string      name, label, paramStr;

nBlocks = NumBlocks();
for (i=0; i<nBlocks; i++)      // all the blocks in the model
{
    name = BlockName(i);      // get the name of the block
    label = GetBlockLabel(i); // get the block's label

    // look for Activity at beginning (with no case sensitivity)
    // AND look for ABC anywhere in label
    if (StrFind(name, "Activity", FALSE, FALSE) == 0 &&
        StrFind(label, "ABC", FALSE, FALSE) >= 0)
    {
        // Found them. now read the parameter for the delay
        paramStr = GetDialogVariable(i, "waitDelta", 0, 0);

        if (paramStr != "")      // not empty means it was found
        {
            value = StrToReal(paramStr);
            // now set it to double value
            SetDialogVariable(i, "waitDelta", value*2.0, 0, 0);
        }
    }
}
}

```

Remote access to dialog variables

Sometimes it is useful to allow a modeler to use a stand-alone block to get or set the value of a dialog variable in a remote block. For instance, this is how modelers set model factors and get model responses using the Scenario Manager block, discussed in the User Guide xxx crossref.

In ExtendSim there are three mechanisms for interfacing dialog variables between a stand-alone block and a remote block:

- 1) Manually enter the remote block number's and dialog variable name into the stand-alone block
- 2) Clone-drop, using the Clone tool to drag the parameter onto the icon of the stand-alone block
- 3) Shift-click a dialog parameter and select from the popup menu of available options to add that parameter to the stand-alone block. This is especially helpful when one or more of the blocks is within a hierarchical block.

The Find and Replace (Utilities library), and the Optimizer, Scenario Manager, and Statistics blocks (Value library) reference dialog variables in other blocks. And all of the Value, Item, and Rate blocks have interfaces that support this feature. These interfaces are also available to developers, and taking advantage of them requires only minor additions to your custom block code.

▲ printed Developer Reference, page 92

Custom remote blocks interfacing with a stand-alone block

The blocks in the Value, Item, and Rate library have code that allows a stand-alone block to get or set their dialog variables; they support all 3 of the above options. Your custom block can also take advantage of this interface, allowing a stand-alone block to interface with your block's variables:

- Options 1 and 2 will automatically be implemented without any modification to your blocks.
- For option 3, you need to add the following code to your custom block:

► Include `MouseClicked v9.h`

```
#include "MouseClicked v9.h"
```

► Add the following code

```
On DialogClick // called whenever the modeler clicks a dialog item
{
    if(do_keydown_mouse_click())
        return;
}
```

☞ If you have additional code in this message handler, put it after this code.

Custom stand-alone block referencing remote dialog variables

If you create a stand-alone block that will reference (get or set) remote dialog variables in other blocks, you can implement any or all of the above methods for collecting the information from the remote block, as discussed below. (While it is also possible to develop your own interface, using one or more of the `ExtendSim` interfaces will be easier and more consistent with existing blocks.)

Manual data entry

Implementing a manual method for referencing from the list in a custom stand-alone block depends on your particular implementation, interface, and needs. Some blocks, such as Find and Replace (Utilities library), only work with one remote dialog variable at a time. Other blocks, such as Statistics (Value library), have a list of remote dialog variables. For coding examples, look at one of those blocks.

Clone drop

Adding a clone-drop interface to a custom block requires a *DragCloneToBlock* message handler. This is called whenever a clone is dragged to the icon of that block.

Inside of this message handler, call *GetDraggedCloneList*. This function requires two arguments – an integer dynamic array and a string dynamic array. It returns the number of clones (usually 1) dropped onto the block. You then use this information to reference a dialog item in another block.

Shift-click

The shift-click feature is more complicated to implement because it requires using a reserved database (discussed on page 106) and an include file. The reserved database (`_leftClickDB`) provides the necessary information for the shift-click action. The include file (`MouseClicked.h`) has two functions defined in it that are required for managing the reserved database:

- 1) `RegisterBlockInLeftClickDB(String blockNumberDialogVariable, String staticStringVariableForRemoteBlockName, String sStaticStringVariableforRowAndColumn, String popupMenuLabel, integer optionNumber)`. Registers a block in the database of blocks that add a popup to

a shift-click action. This is called in the CreateBlock, PasteBlock and OpenModel message handlers. See the MouseClick v9.h include file for an explanation of the arguments.

- 2) UnRegisterBlockInLeftClickDB. Removes a block from the database of blocks that add a popup to a shift-click action. This should be called in the DeleteBlock message handler.

Coding for Shift-click

The key to the Shift-click functionality is a hidden dialog variable in the stand-alone block. When this dialog variable is set by the remote block, the stand-alone block receives a message that it has been sent a reference to a dialog variable in the remote block. The name of the hidden dialog variable is stored in a table in the reserved database (_leftClickDB) by the RegisterBlockInLeftClickDB function.

The information sent by the remote block must be processed in the message handler for the hidden dialog variable. This information has been set in static variables in the stand-alone block by the remote block. The names of the static variables are set in the RegisterBlockInLeftClickDB function. The information includes the block number, dialog variable name, and database table's row and column. These values can be found in the Dialog and Static variables that are arguments to the RegisterBlockInLeftClickDB function.

In the following code example, the stand-alone block gets:

- The block number for the remote block in the dialog variable AddFactor_prm
- The name of the remote block's dialog variable in the static string variable DialogVarName
- The row and column of the remote dialog variable in the RowColumnDialogVar static string variable

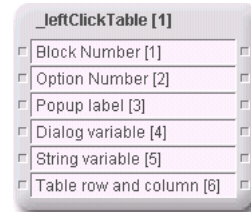
And the popup menu is labeled "Scenario Manager: Add Factor". Because the Scenario Manager has two options (one for adding a factor and one for adding a response), this is option 1.

```
On OpenModel
{
  RegisterBlockInLeftClickDB( "AddFactor_prm","DialogVarName",
    "RowColumnDialogVar", "Scenario Manager: Add Factor", 1);
}
On AddFactor_prm
{
  // Receive message from remote block and process hidden variables
  // AddFactor_prm is the block number of the remote block
  // DialogVarName is the dialog variable in the remote block
  // RowColumn is the row and column in the remote block
}
```

Shift-click example

When the Optimizer block (Value library) is added to a model it creates the reserved database `_leftClickDB`. The entries in the database table cause options, such as *Optimizer: Add Parameter*, to be added to a menu that appears when an appropriate dialog item is Shift-clicked.

Through block code, the Shift-click action causes the selected variable to be used in the Optimizer block. Using this type of architecture makes it easy for developers to add their own block options to the Shift-click menu.



Connectors

Most blocks have connectors. You add and change connectors using the connector tools in the Icon Tools tool at the right of the ExtendSim toolbar.

ModL

Variable connectors

By default new connectors are added as a normal (single) connector. This works well for simple blocks, but often a block will need several inputs or outputs.

As discussed in “Connector options” on page 20, when you select a type of connector (Value, Item, etc.) you can also select whether the connector is normal or variable. Variable connectors act like a row of normal connectors, where the row can be expanded or contracted to provide a required number of connectors. To work with variable connectors, see the functions on page 288. For an example of how variable connectors are implemented, see the Math block (Value library).

Deleting connectors or changing connector types

ExtendSim keeps an ordered list of the connectors on a block. If you delete a connector on a block used in a model, it changes the connector order. This may cause unexpected results – the connectors could become disconnected or connected incorrectly.

 For blocks used in existing models, changing connector order could be a problem. To avoid this problem when you only need to *change* connector types, *do not delete the connector*. Instead, simply select it and click on the correct connector tool. If you have to delete a connector on a block that is used in a model, carefully examine the block's connections.

Bidirectional connectors

All ExtendSim connectors are bidirectional. This is useful if you want blocks to communicate back and forth.

For example, you may want to simulate a network or a bus in which blocks need to both send to, and receive information from, the other members of the network. Because ExtendSim does not allow multiple output connectors to be connected together, you could use only input connectors for all the blocks in the network.

For example, you may also want source and sink connections, where a source block can have its output value changed by the sink blocks that are connected to it. This is useful in simulations where one block's reserves are depleted by the other blocks connected to it.

To implement these features in blocks, assign or modify the value of an input connector. This feature makes input connectors bidirectional. When the value of an input connector is modified, all connected blocks will see this new value when they get their next Simulate message.

The concept of using an input connector for both input and output is shown in the Bidirectional Flows model, which is located in the \Examples\Developer Tips folder. In that model a power sta-

tion supplies energy to cities. Each city block has a single input connector. The block code for the cities subtracts an amount from the input, called “PowerIn”, with the statement:

```
PowerIn = PowerIn - amountUsed;
```

This affects the Power Station block by reducing the output power value stored in its output connector. The power station can check its output connector value and see if it went to 0 or became negative, signifying that its power reserves have been depleted by the towns:

```
if (PowerOut <= 0.0)    // check the reserve power
{
    // Out of power. Tell when this occurred.
    UserError("Brownout occurred at time= "+CurrentTime);
    abort;              // Stop the simulation.
}
```

Initializing connectors

The value of a connector is used to determine whether the block is connected to another block during the CheckData message handler, and thus cannot be changed there. All connectors are initialized by ExtendSim to 0.0 after the CheckData message handlers are executed. To initialize connectors *before* the simulation runs, do it within the InitSim or PostInitSim handler.

For example, to initialize a connector before the simulation runs:

```
On InitSim    // Or use On PostInitSim
{
    conOut = initialValue;
}
```

The blocks in the Item library initialize their connectors in the InitSim handler.

Working with arrays

Since you will typically use arrays to store any data which is more complex than a single variable, you will probably use them fairly often. ExtendSim provides lots of features and functions that use arrays, especially regarding discrete event modeling. Note that, while ModL does not directly support arbitrary user-defined structures, it supports a rich linked list structure (see “Array-like structures” on page 68 and “Linked lists” on page 361) and it emulates other types of structures using arrays of arrays. You will find that working with arrays in ExtendSim is simpler and safer than using the data structures that are available in C.

Memory usage of variables, arrays, and items

For most models, you do not need to worry about how much memory your variables take up. You may need this information in some circumstances, especially if you are using huge arrays.

 There is no overhead for using arrays.

Type	Memory used
Real	8 bytes (double)
Integer	4 bytes (long)
String or Str255	256 bytes per string containing up to 255 characters
Str15	16 bytes per string containing up to 15 characters

Type	Memory used
Str31	32 bytes per string containing up to 31 characters
Str63	64 bytes per string containing up to 63 characters
Str127	128 bytes per string containing up to 127 characters

The total of all static arrays (non-dynamic arrays) and variables in a block, including any static data tables in the block's dialog (which are real arrays, 8 bytes per element), cannot exceed 32,767 bytes. Dynamic data tables are not included in static memory allocation. See "Block data tables" on page 297.

When a user-defined function is called, its local arrays can have up to 32,767 bytes. The size of dynamic arrays are not included in any of the above calculations and can have up to 2 billion elements each.

If your arrays are small, use fixed dimension static arrays since they are easier to declare. If your arrays are large, use dynamic arrays.

Pass by value and reference (pointers)

In C, you can pass variable arguments to functions by value or by reference (pointers). When you pass a variable by value, the value of that variable in the outside environment is not affected by anything that function does. When you pass by reference, however, the function can modify the contents of the variable and those modifications are seen by the outside environment.

In ModL, all non-array variables and single elements of arrays (such as `myArray[i]`) are always passed by value and are therefore never modified. Arrays, however, are always passed by reference (such as "myArray", with no subscripts) and therefore can have their contents changed by a ModL or user-defined function. If you are writing user-defined functions, this feature makes it easy to return more than one value. Simply pass an array to your user-defined function and change the values in the array. All changes you make to the array in your function can be seen in the message handler when the function returns.

Passing arrays

Essentially, a passed array is a pointer assigned to a real variable with the `PassArray` function and it is read from that real variable and converted back to an array with the `GetPassedArray` function. (In ModL, pointers have more information than just the address of data, so real variables are used to hold them.) These functions are listed in "Passing arrays" on page 354. The Passing Arrays model in the Examples\Developer Tips folder illustrates how to pass, receive, and modify an array.

Passed arrays must be dynamic arrays but they can be any type (real, integer, or string). A passed array has the same properties as an array that you use in a single block.

 If you pass arrays, those arrays can only be resized or disposed of in the same block where they were created.

You can use the `SendMsgToBlock` function and the `BlockReceive` message handler to tell the originating block to resize the array. An example of this is the Executive block (Item library).

It is important to note that any changes made to data in a passed array affects all blocks that reference that array, including the block that originated the array. If you want to make a change to a passed array that is not reflected in previous blocks in the model, copy the values from the passed

array to a new array, make changes to that new array, and pass that new array. (You can copy an array quickly with a “for” loop, as described later in this chapter.)

Passing arrays through connectors

The blocks built in this manual pass single values through their connectors. You cannot pass arrays as easily as you can single values, but it is not difficult to add the few functions that let you pass arrays through the connectors.

Because a connector is a real variable, you can pass arrays through connectors. To read a passed array from a connector, use the `GetPassedArray(connector, array)` function. For example, assume that you are receiving an array through the input connector called `ArrayConnectorIn`. Your message handler might look like:

```

real theArray[];
...
on Simulate
{
    if (GetPassedArray(ArrayConnectorIn, theArray)) // is it
    passed
    {
        . . . // Yes, use the array
    }
    else
    {
        . . . // The array did not arrive yet
        // or it is not a passed array
    }
    . . .
}

```

All of the values in the passed array can now be accessed and changed by using the **theArray** variable in your code.

The process for passing multidimensional arrays is exactly the same, with the exception that you need to confirm that the fixed dimensions are the same for both the passing and receiving blocks.

There are two different ways to pass an array to an output connector, depending on whether the array was passed to the block or it originated in the block. If the array was passed to the block, simply set the output connector to the same value as the input connector:

ArrayConnectorOut = ArrayConnectorIn; // pass the old pointer on

To pass an array that originated in the block, use the `PassArray(array)` function. You can assign the value of this function to an output connector (or to a real variable in your ModL code that is then assigned to an output connector). Assume that you had changed the values of **theArray** in the example above and wanted to pass them out the connector named `ArrayConnectorOut`. You would use:

ArrayConnectorOut = PassArray(theArray); // pass the new pointer

If you are just passing an array through a block without looking at it, you should not use `GetPassedArray`. Simply assign the output connector to the value of the input connector, and the real number that holds the passed array will be handled with no overhead:

ArrayConnectorOut = ArrayConnectorIn; // pass the old pointer on

Passing arrays through global variables

You can use any real variable to hold passed arrays. This means that you can pass arrays through the global variables `global0` through `global19` that are available to all blocks. If you want an array to be globally accessible, pass it to one of the global variables and use `GetPassedArray` to interpret the global variable when you want to get at the array values. Discrete event blocks use this technique.

Precautions when passing arrays

The `GetPassedArray` function needs to be called before accessing a passed array. If an array is created at the beginning of the simulation and the size is never changed and the array is not disposed of during the simulation, then `GetPassedArray` only needs to be called once, at the beginning of the simulation.

☞ If you pass arrays NOT during a run, call `GetPassedArray()` in any message handler that uses those arrays before accessing those arrays.

However, `GetPassedArray` must be called again before accessing any passed array that may have changed in size or been disposed of. Trying to access a passed array after the creator block has disposed of or resized it can cause a crash if the `GetPassedArray` function is not called immediately before access is attempted. When you resize or dispose of an array, its memory location may change or otherwise become invalid. A crash can occur because the block that received the array has a pointer to a specific location in memory, and will try to access that memory point even if it no longer is a valid array location. Calling `GetPassedArray` immediately before accessing the array will relink to the correct memory address if the array has been resized and will return a FALSE value if the array has been disposed of.

Since `GetPassedArray` is extremely fast (because it only links the pointer of the array), the safest action is to call it and test its return value before every series of accesses to the array.

☞ An example of what not to do is the following:

Block #1

```
integer x[];
on checkdata
{
  Makearray(x, 10);      // create the array
  global9 = passarray(x);
}

on endsim
{
  DisposeArray(x);      // dispose of the array
}
```

Block #2 //this is not safe!

```
integer x[];
on initSim
{
  GetPassedArray(global9, x);
}

on endSim          //this is not safe!!!!
{
  for(i = 0; I<10; I++) // This is dangerous! Is the array still
    available?
    x[i] = 0;
}
```

The above code could cause a crash if the code in Block #2 is executed later in the simulation than Block #1. The problem occurs because Block #2 tries to access the array it thinks is in the variable x, while the array referenced by x has actually already been disposed of by Block #1.

 It would be safer to use the following approach in Block #2:

```
integer x[];
on initSim
{
  GetPassedArray(global9, x);
}

on endSim
{
  if (getPassedArray(global9, x)) // This is safer. Get and test the
    array first.
  {
    for(i = 0; I<10; I++)
      x[i] = 0;
  }
}
```

For the same reason, it is important to call the function GetArrays() (see “Functions in discrete event blocks” on page 174) in your custom discrete event block code each time before you access the ItemArrays.

Using passed arrays to make structures

ModL does not support structures directly, except for linked lists (see “Linked lists” on page 361). It does, however, allow you to emulate structures using arrays of arrays. This is safer than using pointers and structures in C.

Suppose you want to pass an array consisting of real, string, and integer values. Since arrays can be passed to any real variable, many arrays can be passed into a real array, and that array can be passed through a connector or global variable. The receiving block can get the passed structure array and then get the individual passed data arrays from that array.

The following shows an example of using passed arrays to make structures.

```

// This block makes a structure and passes it.
// Declare the arrays at the top of the code.
real      structureArray[];    // This is the structure
string    stringValues[];     // Holds the strings
real      realValues[];       // Holds the reals
integer    integerValues[];    // Holds the logical values

on InitSim
{
    // Give the arrays a size.
    MakeArray(structureArray, 3);    // Holds reals, strings, long.
    MakeArray(stringValues, 10);    // 10 elements for each array
    MakeArray(integerValues, 10);
    MakeArray(realValues, 10);

    // Pass the arrays to the real structureArray.
    // This needs to be done only once each simulation run.
    structureArray[0] = PassArray(realValues);
    structureArray[1] = PassArray(stringValues);
    structureArray[2] = PassArray(integerValues);
}

on Simulate
{
    // Put data into the realValues, stringValues,
    // and integerValues arrays.
    // For example, set one element of each array.
    realValues[0] = 98.6;
    stringValues[0] = "Octane rating";
    integerValues[0] = TRUE;

    // The data arrays were already passed to the structure-
    // Array.
    // Now, pass the structureArray to the connector.
    // This has to be done here because connectors are active
    // only during "On Simulate".
    DataOut = PassArray(structureArray);
}

```

The following is the receiving block's code. This gets the passed structure array, then gets the individual arrays from the structure array for use in the block:

```

// Declare the arrays at the top of the code.
real      structureArray[];    // This is the structure
string    stringValues[];     // Holds the strings
real      realValues[];       // Holds the reals
integer    integerValues[];    // Holds the logical values

```

```

on Simulate
{
    if (GetPassedArray(ConnectorIn, structureArray))
    {
        // Get the reals
        GetPassedArray(structureArray[0], realValues);
        // Get the strings
        GetPassedArray(structureArray[1], stringValues);
        // Get the logicals
        GetPassedArray(structureArray[2], integerValues);
        // Use the array values
        if (stringValues[0] == "Octane rating")
            . . .
    }
    else
    {
        // The structureArray did not arrive yet
        // or is not an array.
        . . .
    }
}

```

Working with global arrays

As discussed in “Global arrays” on page 68, global arrays provide a repository for model-specific data; they are accessed and managed through a suite of functions listed on page 355. Global arrays can be referenced either by name or index value.

The following is an example of how to create, access, and dispose of a global array:

```

// Declare the arrays at the top of the code.
integer    arrayIndex;    // index value for the global array

on initsim
{
    arrayIndex = GAGetIndex("myGlobalArray");    // see if global array
    already exists
}

```

```

if (arrayIndex < 0) // if global array does not exist...
{
    // Create a three column global array of real numbers named
    // "myGlobalArray". Assign array's index to arrayIndex.
    arrayIndex = GACreate("myGlobalArray", GAREal, 3);
    // Resize array to contain 10 rows of data
    GAREsize("myGlobalArray", 10);
}

on simulate
{
    real realNumber;
    . . .
    // Set second row and column of global array to realNumber
    // (row and columns start at zero)
    GASetReal(realNumber, arrayIndex, 1, 1);
    . . .
    // Read third row and column of global array and assign to realNumber
    realNumber = GAGetReal(arrayIndex, 2, 2);
}

On Endsim
{
    // We are done with myGlobalArray.
    if (GAGetIndex("myGlobalArray") != -1) // Only if myGlobalArray
    still exists,
        GADispose("myGlobalArray"); // dispose of it.
}

```

Copying arrays using “for” loops

Copying the elements of an array to another array is quite easy with the “for” loop construct. The following copies a two-dimensional array that has 100 elements:

```

// Copy array a into array b.

integer    a[10][10], b[10][10];
integer    i, j;

for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
        b[i][j] = a[i][j];

```

Using arrays to import unknown rows of numbers

The Import function reads numbers into a real array. If you do not know how many lines are in the file and you use a fixed-size array, you will lose lines if the array is too small or waste memory space if the array is too long. Instead, use a dynamic array to be sure that you will get all the rows without having to specify the dimension of the rows. The function returns the number of rows read, so you can then reduce the size of the array after the call. For example:

```
integer numRowsRead;
real fileArray[];
string theFileName, thePrompt, theDelim;
...
MakeArray(fileArray, 10000);
numRowsRead = Import(theFileName, thePrompt, theDelim, fileArray);
MakeArray(fileArray, numRowsRead);
```

The second call to MakeArray reduces the size of the array without disturbing the contents in the rows left. Of course, if you import into a two-dimensional array, you have to specify the size of the second dimension.

Working with linked lists

As discussed on page 68, linked lists are complex structures that can enable sophisticated sorting rules. The concepts behind linked lists are beyond the scope of this manual. See “Linked lists” on page 361 for a basic strategy in working with linked lists. Also, see the Queue blocks (Item library) for actual linked list code.

Working with databases

As discussed in the User Guide, an ExtendSim database provides a repository for model-specific data.

Using the database API to read and write

ExtendSim databases can be accessed and managed through the Read and Write blocks in the Value and Item libraries. You can also use the database API (see “Database functions” on page 333) to create databases via execution of a block’s code.

The following is an example of how to create and access a database to store output from a model:

```
// Declare static variables
integer databaseIndex;// index value for the output database
integer tableIndex;// index value for the output table
integer fieldIndex;// index value for the output field
integer recordIndex;// index for setting our record values

on initsim// create the database, table, field if not there
{
// Creates database parts only if it doesn't exist already
DBDatabaseCreate("myDB");
databaseIndex = DBDatabaseGetIndex("myDB");// get index
DBTableCreate("myDB", "myTable");
tableIndex = DBTableGetIndex(databaseIndex, "myTable");
DBFieldCreate("myDB", "myTable", "myField", DB_FIELDTYPE_REAL_GENERAL,
8, FALSE, FALSE, FALSE); // real number field
fieldIndex = DBFieldGetIndex(databaseIndex, tableIndex, "myField");

// create records as we need them during the run
recordIndex = 0;// initialize record index
}

on simulate
{
recordIndex++;// increment our record index first (one based)

// append one record to our table
DBRecordsInsert(databaseIndex, tableIndex, 0, 1);

// write one data value to our database
```

```
DBDataSetAsNumber(databaseIndex, tableIndex, fieldIndex, recordIndex,
myDataValue);
}
```

Registered blocks

Block registration is a method for keeping a block informed when there is a change in the linked data source. Unlike user-defined or code-defined links (which link a particular dialog item to a data source), block registration functions link an entire block to a data source.

The block registration functions start with DBBlockRegister or GABlockRegister (see “Linking and notification” on page 351). Depending on the function chosen, the block gets a LinkContents message if the content of the data source changes or a LinkStructure message if the structure of the data source (such as the name of a table or the location of a field) changes.

An example of block registration for content changes is the Read block (Value library). When Link Alerts is checked in its Options tab, the block registers itself so that it will be alerted if/when changes are made to its source data.

Registered blocks can be located using the Find Links dialog (Edit > Open Dynamic Linked Blocks) discussed in the User Guide xxx crossref.



Use registered blocks judiciously. Due to the extra messaging, a registered block can significantly slow the simulation run.

Reserved databases

ExtendSim databases are internal repositories for storing, managing, and controlling model data. A *reserved database* is a specialized type of ExtendSim database that can be hidden from the modeler. Reserved databases provide database capabilities without the modeler having to use, or even be aware of, the reserved database.

Typically a database would be created by a modeler for a specific model. A reserved database, on the other hand, is usually created by a programmer using the database API.

Example

A common use of a reserved database is to support the architecture of a block. For example, when the Resource Manager block (Item library of ExtendSim AT and ExtendSim Suite) is added to a model it creates a reserved database named `_rM_Database`. There are numerous tables in that database, each focusing on different aspects of the block and how it functions. For instance, the Dialog Colors table, shown here, stores the HSV values of the colors used for text labels in the block’s dialog. Other tables track filtering conditions, store resources with their ranking and skill levels, and so forth. When the modeler enters data and makes selections in the Resource Manager block, these are tracked in the reserved database. This process is invisible to the modeler.



The Optimizer block (Value library) is an example where a reserved database is used to provide a feature for the modeler. This is discussed in “Shift-click example” on page 96.

Creating and editing

Reserved databases are created and edited in much the same manner as you would create or edit any ExtendSim database. Some differences are:

- To notify ExtendSim that the database is to be reserved, enter a leading underscore () at the beginning of the database's name. For example, the name would look like "_AReservedDB".
- To prevent modelers from accidentally writing to reserved databases, they require special write functions. These are listed on page 341. An error message will be displayed if the special write functions are used for non-reserved databases, and vice versa.
- Since they are intended for developers, ExtendSim doesn't support using blocks (such as Read or Write) to access reserved databases. It also doesn't allow modelers to link dialog items to a reserved database.
- When a block that requires a reserved database is added to a model, the code of the block creates the database in the model. In most cases, unless a block that requires a reserved database is placed in the model, the model will not have any reserved databases.
- If a model has reserved databases, they will not be displayed in the Database List or at the bottom of the Database menu unless you first give the command Develop > Show Reserved Databases. By default, this command is not selected. Furthermore, the command is re initialized to off each time the model is opened.
- Even if a reserved database is not listed in the Database List or at the bottom of the Database menu, it is always accessible through ModL functions.



Changing anything in a reserved database is equivalent to changing the code of a block. It is likely to corrupt any blocks that use it.

Reading text blocks as commands

Since every piece of text that you add to a model gets its own number, text on the model worksheet can be accessed in a block's code.

The BlockName function returns the name of a block for the specific block number. However, blocks aren't the only items on the worksheet that have numbers. Because each piece of text gets a number, and the text is equivalent to a block's name, you can use BlockName to read text. This can be useful if you want to see how you have changed some text on the model.

Use this feature to globally change parameters in block dialogs. For example, assume you want to give a command to the model to change a specific parameter in many blocks. Normally, you would have to open all of the blocks and type the new parameter value. Here is a strategy that allows you to type some text, such as "Speed=55", on the model window that will cause all of the speed parameters in all of the blocks to change when the simulation is run. Note that the following function can be put in any block and can be called for each typed value that the code needs:

```

real GetTypedValue(string name)      // User defined function
{
  integer      nBlocks, i, position, nameLength;
  string       typedText, valuePart;
  real         valueFound;

  nameLength = StrLen(name);          // Number of characters
  nBlocks = NumBlocks();              // Number of blocks

  for (i=0; i<nBlocks; i++)           // Loop thru all blocks
  {
    typedText = BlockName(i);         // Get block name or text
                                        // find the "name=" string
    position = StrFind(typedText, name+"=", FALSE, FALSE);
    if (position == 0)                // Must not be part of a larger word
    {
      // Get numeric part of string (skip over "name=")
      valuePart = StrPart(typedText, position+nameLength+1, 255);
      valueFound = StrToReal(valuePart); // Convert to real
      if (noValue(valueFound))
      {
        UserError("The value for "+name+" must be numeric");
        abort; // Stop the simulation
      }
    }
    else
      return(valueFound); // Return the found value
  }

  // No "name=" was found
  UserError("Your typed variable, "+name+", was not found");
  abort; // Stop the simulation
}

// Get your values. If they're not found or are bad,
// GetTypedValue stops the simulation with a message
on Checkdata
{
  // SpeedValue & MassValue are names of dialog parameters
  SpeedValue = GetTypedValue("Speed");
  MassValue = GetTypedValue("Mass");
}

```

Passing messages between blocks

ExtendSim allows blocks to “call” other blocks by sending them a message. Use this feature to cause an action in another block, such as having it perform a calculation or open a plot. Global variables are used to pass arguments to the called block as well as to get results from the called block's actions.

The following code calls another block that calculates some functions when a dialog button is clicked. Since any block can call this block and use its functions, you could think of this as a global function block. See the Executive block (Item library) for an example of a block that has global functions. See the Item library for examples of blocks sending messages using connectors.

Calling block

```
on Button // called when the "Button" is clicked
{
  global0 = myArgumentValue; // Set up argument to the global function
  globalInt1 = 1;           // Number of the Hochmeister function

  // globalInt0 contains block number of the "global function" block
  // UserMsg0 is the message handler that calculates it
  SendMsgToBlock(globalInt0, UserMsg0Msg); // Call it
  myResult = global1; // Get the result of the call
}
```

Global function block

```
// Before simulation runs, set up the block number so that
// other blocks can call it
on CheckData
{
  globalInt0 = MyBlockNumber(); // Use a global integer variable
}

// A message from another block (a "global function" call)

on BlockReceive0
{
  // globalInt1 contains the number of the function

  switch(globalInt1) // Which function did they want?
  {
    case 1: // The Hochmeister function

      // global0 has the argument, result goes into global1
      global1 = cos(global0)^2.0+sin(global0)^2.0;
      break;

    case 2: // The Lemski-Pemski factor
      // global0 has the argument, result goes into global1
      global1 = Sin(global0)/Cos(global0)-Tan(global0);
      break;
  }
}
```

Changing data while the simulation is running

Sometimes you need to change values in a block's dialog while the simulation is running. In most cases, ExtendSim handles this by allowing you to change the value, which is then used immediately in the block's code (usually in the Simulate handler). However, there has to be a special initializing procedure if the block needs to calculate intermediate values based on that new data. ExtendSim provides a mechanism to tell the block that its data has been changed. Then the block can do a recalculation of intermediate data before the simulation resumes.

When any data is changed in a dialog during a simulation, ExtendSim sends a ResumeSim message to that block before the simulation can resume. It also sends a ResumeSimAllBlocks message to all of the blocks in the model. These are optional message handlers because most blocks do not need to recalculate intermediate data, they just use the dialog values directly. But if the block has a

ResumeSim or ResumeSimAllBlocks message handler, it can take some action before the simulation resumes.

Here is an example of a block that needs to do a lengthy calculation before the simulation resumes after a change in parameters. It recalculates the coefficient when the modeler changes the dialog parameter, but doesn't zero out the accumulated value so the simulation can continue properly:

```

real    coeffValue, accum;
real    CalcCoeffValue(real theValue)    // This function does the
                                         // coeffValue calculation
{
  real    coeff;
  integer i;

  coeff = 0.0;                          // Initialize the sum
  for (i=0; i<100; i++)                  // Loop a hundred times
    coeff = coeff+cos(theValue*i);       // Use theValue, calculate coeff
  return(coeff);                          // Done, return it
}

on InitSim    // Initialize values for beginning of simulation
{
  coeffValue = CalcCoeffValue(dialogParameter); // Calc coeff and
  accum = 0.0;                                // init to 0
}

on ResumeSim  // User changed dialogParameter during simulation.
              // Just calculate new coeffValue,
              // don't initialize accumulated value.
{
  coeffValue = CalcCoeffValue(dialogParameter); // Just calc coeff,
                                              // don't zero accum
}

on Simulate   // Calculate new values during the simulation
{
  accum = accum+coeffValue*conIn;           // Fast. Multiply input by coeff
  conOut = accum;                           // Output accumulated value
}

```

Scripting

In ExtendSim, the process of building models typically relies heavily upon modeler interaction. The standard process of placing blocks on the worksheet, connecting them together, and filling in the appropriate dialogs (while graphical and intuitive), requires direct modeler participation. However, models can also be built, modified, and controlled indirectly using ExtendSim's *scripting* features.

Scripting is an extremely powerful feature which allows you to:

- Build, run, and control models from within another application
- Create custom wizards to simplify tasks or interact with modelers
- Develop self-modifying models

By using the scripting functions (see "Scripting" on page 319), you can tell ExtendSim which blocks to place on the worksheet and where, how to connect the blocks together, and what values to use for the block's dialog parameters. In addition, any menu command can be executed by a

function call, for instance to run a model. When used in conjunction with the IPC functions (see “Interprocess Communication (IPC)” on page 239 or “OLE/COM (Windows only)” on page 241), the scripting functions can be used to build and run entire models based on information from another application.

The scripting functions can also provide a means for developing “wizards” – blocks that help the modeler to perform a task by gathering information, then building or modifying a model based upon that information.

You can also develop blocks that help models achieve desired results. You could build artificial intelligence into your models by building blocks that query the model for specific metrics and simultaneously modify the model based on those metrics. For instance, you would use this self-modifying feature to automate the process of changing models in response to simulation results or to build goal-seeking models.

See the Tutorial block (Example Libraries > ModL Tips library > Scripting category) for an example of using the scripting functions.

OLE and ActiveX Automation

ActiveX automation is the process of using ExtendSim’s OLE functions, or the scripting environment of another application, to communicate with, exchange data with, or control another application. ExtendSim functions allow an object from another application, such as an Excel spreadsheet, to be embedded into an ExtendSim model.

 ExtendSim supports ActiveX automation as either an automation server or as an automation client. ExtendSim also supports being a container for embedded objects or ActiveX controls.

Note that almost all of the OLE functions defined in ExtendSim have two versions – one for controlling an embedded object with ExtendSim and one for controlling an outside object through ActiveX Automation. The name and the arguments provide a method of distinguishing between these two versions of the functions:

- The set of functions designed to control embedded objects start with the keyword 'OLE' and include a block number and dialog item name as their first two arguments.
- The functions that can be used for Automation control start with 'OLEDispatch' and will take a dispatch handle as their first argument.

 The Examples\Developer Tips\OLE Automation folder contains examples of C++, Visual Basic, and VBA code for ActiveX Automation. See explanation for those examples on page 113 (C++), page 118 (VBA), and page 118 (Visual Basic).

COM DLL example

The “VB.net COM DLL example” model (Windows only) shows how to interface from ExtendSim with a COM DLL created in VB.net. The COM DLL folder includes the model plus an ExtendSim library with a custom block as well as the source code used to create the COM DLL. ExtendSim is the Client in this automation example; this is the inverse of the Client App example where VB is the Client.

 See the folder ExtendSim9\Examples\DeveloperTips\OLE Automation\VB\ COM DLL

Controlling Embedded Objects from ModL script

A set of functions is available in ModL to access embedded objects. These are described in detail on “OLE/COM (Windows only)” on page 241. They include:

- Functions to insert objects (OLEInsertObject)
 - Functions to invoke methods and set properties of objects; (OLEInvoke, OLEPropertyPut, OLEPropertyGet)
 - And many other miscellaneous functions that allow extensive control of an embedded object
- Embedded objects or ActiveX controls can be embedded directly onto the model worksheet or embedded into dialog items built to be embedded object locations.
- When you are trying to control an object that has been embedded onto the model worksheet, the blockNumber argument is used to determine which object is to be controlled and the dialogItem argument is ignored.
 - When you are trying to control an object or control that has been embedded into a dialog, the block number and dialog item arguments are both used.

ExtendSim as an Automation Client

To use ExtendSim as an Automation Client, start with a call to the function OLECreateObject. This will return a dispatch Handle to the OLE Automation Server. Once you have the base Dispatch Handle for the server, you can then use the functions defined in “OLE/COM (Windows only)” on page 241 to control the object.

ExtendSim as an Automation Server

ExtendSim also supports a simplified version of Automation as a Server. This is the ability for another application to control the ExtendSim application from outside via OLE.

The automation supported in ExtendSim as an Automation Server is five methods:

- Poke
- Request
- Execute
- BlockMsg
- GetObjectHandle

These are used in a fairly simple single object model. Execute, Request, and Poke are the primary means of controlling ExtendSim via ActiveX/OLE Automation. BlockMsg and GetObjectHandle are slightly more obscure, but can also be useful. See “C++ examples” on page 113 for an example of how these methods are used.

Object

For use with C++ or other related languages, the Object is the ExtendSim application, referenced by the following GUID:

{E167B362-7044-11d2-99DE-00C0230406DF}

In Visual Basic, or other environments where ProgIDs are supported, this GUID can also be referenced by the ProgID “**Extend.application**”. ProgIDs are a simplified and easier to remember way of accessing a GUID.



Even though the application name is ExtendSim, for backward compatibility the ProgID is Extend.application. In the future, the ProgID ExtendSim.application will be supported as well, but at the time of this writing the correct ProgID is Extend.application.

Because ModL scripts can be executed with the Execute method, complete control of ExtendSim is available through the following methods:

DispatchID	Method Name	See Page
1	Execute	114
2	Request	115
3	Poke	115
100	BlockMsg	116
101	GetObjectHandle	117

Topic and Item

The Poke and Request methods require two strings (Topic and Item) to identify where the data should be poked or where it should be requested from.

Topic is the name of the worksheet or model (or “system” if you don't need to specify a model). If you specify system, the data will be poked to or requested from the top model.

The Item string is specified as follows:

"VariableName:#BlockNumber:RowStart:ColStart:RowEnd:ColEnd"

RowStart, ColStart, RowEnd, and ColEnd can all be set to zero if the item specified is neither a data table object nor an array.

If the item is not a table, RowEnd and ColEnd can be left off. In this case the string would look as follows:

"VariableName:#BlockNumber:0:0"

Starting in ExtendSim 7.0.2 there are new forms of the Item string which will allow you to poke directly to, or request directly from, an ExtendSim database table or a global array. These forms are as follows:

"GA:#GlobalArrayIndex:RowStart:ColStart:RowEnd:ColEnd"

"DB:#DBIndex:DBTableIndex:RecordStart:FieldStart:RecordEnd:Field-End"

Note that the database case has an extra argument.

C++ examples

The following examples show how to access an IDispatch interface and use the five methods (Poke, Request, Execute, BlockMsg, and GetObjectHandle) in C++. Also see the Examples\Developer Tips\OLE Automation folder.

Retrieving the IDispatch interface

The following C++ sample code shows one possible way you could access an IDispatch interface on an ExtendSim application.

- ☞ The GetActiveObject code attempts to find a running copy of ExtendSim in the ROT (running object table). The CoCreateInstance code creates a new instance of ExtendSim if the running one is not found.

```

CLSIDFromString ("{E167B362-7044-11d2-99DE-00C0230406DF}", &clsid);
hr = GetActiveObject(clsid, NULL, (IUnknown **) &m_pUnknown);

if (hr == S_OK)
{
    // JSL - found an existing instance
    hr = m_pUnknown->QueryInterface(IID_IDispatch, &m_pDisp);
    hr = m_pUnknown->Release();
}
else
{
    theErr = GetLastError();
    MessageBox (NULL, TEXT("GetActiveObject Failed, creating a new
                          Object"), TEXT("OleTest"), MB_OK);

    hr = CoCreateInstance(clsid,          // class ID of object
                          NULL,          // controlling IUnknown
                          CLSCTX_LOCAL_SERVER, // context
                          IID_IDispatch, // interface wanted
                          (LPVOID *) &m_pDisp) ;// output variable

    if (hr != NOERROR)
    {
        theErr = GetLastError();
        MessageBox (NULL, TEXT("Create Instance Not Successful"),
                    TEXT("OleTest"), MB_OK);
        FormatMessage(FORMAT_MESSAGE_FROM_SYSTEM, NULL, theErr,
                    MAKELANID(LANG_NEUTRAL, SUBLANG_DEFAULT),
                    buf, sizeof(buf), NULL);
        MessageBox (NULL, buf, "Error", MB_OK);
        return;
    }
}

```

Execute

The Execute method takes just a single argument that is the code to be executed. The dispID of Execute is 1.

The Execute method is the most flexible method call, as the command that is passed to ExtendSim via this method is just a section of ModL script and can contain anything that can be put into ModL script, including scripting functions. This means that by using the execute method you can build models, run models, or do any of a number of things.

The C++ code you would use to call the execute method would look something like this (note that this code will cause ExtendSim to display a userError statement with the value in userglobal0):


```
// Arguments are all passed as variants
bStr = SysAllocString((WCHAR *) L"userError(userGlobal0)");
VariantInit(&vString);
vString.vt = VT_BSTR;
vString.bstrVal = bStr;

// Set the DISPPARAMS structure that holds the variant.
dp3.rgvarg = &vString;
dp3.cArgs = 1;
dp3.rgdispidNamedArgs = NULL;
dp3.cNamedArgs = 0;

// Call IDispatch::Invoke()
hr = m_pDisp->Invoke(executeID, IID_NULL, LOCALE_SYSTEM_DEFAULT,
                    DISPATCH_METHOD, &dp3, NULL, &ei, &uiErr);
```

Request

The Request method uses Topic and Item, as specified in “Topic and Item” on page 113, and returns a string value. The dispID of Request is 2.

The C++ code you would use to call the Request method would look something like this (this code will return the value present in userGlobal0):

```
// Arguments are all passed as variants
requestVariant = malloc(sizeof(VARIANTARG) *2);
itemStr        = SysAllocString((WCHAR *)
                                L"userGlobal0:#0:0:0:0:0");

VariantInit(&requestVariant[0]);
requestVariant[0].vt = VT_BSTR;
requestVariant[0].bstrVal = itemStr;

topicStr = SysAllocString((WCHAR *) L"system");
VariantInit(&requestVariant[1]);
requestVariant[1].vt = VT_BSTR;
requestVariant[1].bstrVal = topicStr;

// Set the DISPPARAMS structure that holds the variant.
dp2.rgvarg = requestVariant;
dp2.cArgs = 2;
dp2.rgdispidNamedArgs = NULL;
dp2.cNamedArgs = 0;

var.vt = VT_EMPTY;

// Call IDispatch::Invoke()
hr = m_pDisp->Invoke(requestID, IID_NULL, LOCALE_SYSTEM_DEFAULT,
                    DISPATCH_METHOD, &dp2, &var, &ei, &uiErr);

SysFreeString(topicStr);
SysFreeString(itemStr);
```

Poke

The Poke method takes the three arguments Value, Topic, and Item and sets the value specified in the Value argument into the item specified in the Item argument. Topic and Item are specified as

in “Topic and Item” on page 113; Value is the string that is going to be poked into the location specified by Topic and Item. The dispID of poke is 3. Value, Topic, and Item are all strings.

The C++ code you would use to call the Poke method would look something like this (note that this code will set the value of Global0 to 34.5):

```
pokeVariant    = malloc(sizeof(VARIANTARG) *3);
valueStr       = SysAllocString((WCHAR *) L"34.5");
VariantInit(&pokeVariant[0]);
pokeVariant[0].vt = VT_BSTR;
pokeVariant[0].bstrVal = valueStr;

itemStr = SysAllocString((WCHAR *)L"Global0:#0:0:0:0:0");
VariantInit(&pokeVariant[1]);
pokeVariant[1].vt = VT_BSTR;
pokeVariant[1].bstrVal = itemStr;

topicStr       = SysAllocString((WCHAR *) L"system");
VariantInit(&pokeVariant[2]);
pokeVariant[2].vt = VT_BSTR;
pokeVariant[2].bstrVal = topicStr;

// Set the DISPPARAMS structure that holds the variant.
dp.rgvarg      = pokeVariant;
dp.cArgs       = 3;
dp.rgdispidNamedArgs = NULL;
dp.cNamedArgs  = 0;

// Call IDispatch::Invoke()
hr = m_pDisp->Invoke( pokeID, IID_NULL, LOCALE_SYSTEM_DEFAULT,
                     DISPATCH_METHOD, &dp, NULL, &ei, &uiErr);

SysFreeString(topicStr);
SysFreeString(itemStr);
SysFreeString(valueStr);
```

BlockMsg

BlockMsg sends a message to a specific block in the active ExtendSim model. The DispID of BlockMsg is 100. This message will execute a message handler in the block called OLEAutomation.

The BlockMsg method takes two arguments:

- The first is a block number (integer) and specifies the block that is to receive the message.
- The second is a value (integer, real, or string) and can be used to communicate with the block code.

ExtendSim will set the value of one of three globals to be equal to the value you pass in as the Value argument. Which global will be set is based on which type of variable passed in. The example below uses a string variable. The three globals are OLEGlobal, OLEGlobalInt, and OLEGlobalStr.



As reflected in the code below, the integer values used by the blockMsg method are long integers. However, by default an integer variable declared in Visual Basic is a short integer. So VB programmers should take special care to declare the variables as longs in their VB code.

```
msgVariant      = malloc(sizeof(VARIANTARG) *2);
VariantInit(&msgVariant[0]);
msgVariant[0].vt = VT_I4;
msgVariant[0].lVal = 23; // blockNumber
argStr = SysAllocString((WCHAR *) L"userText ");
VariantInit(&msgVariant[1]);
msgVariant[1].vt = VT_BSTR; // could be a long, or a real as well
msgVariant[1].bstrVal = argStr;

// Set the DISPPARAMS structure that holds the variant.
dp.rgvarg      = msgVariant;
dp.cArgs       = 2;
dp.rgdispidNamedArgs = NULL;
dp.cNamedArgs  = 0;

// Call IDispatch::Invoke()
hr = m_pDisp->Invoke( msgID, IID_NULL, LOCALE_SYSTEM_DEFAULT,
                     DISPATCH_METHOD, &dp, NULL, &ei, &uiErr);
SysFreeString(argStr);
```

GetObjectHandle

GetObjectHandle returns the Dispatch Handle value for an embedded object within ExtendSim. This is useful if your outside code is dealing directly with Dispatch Handles, since it will allow you to communicate directly with the embedded object inside the ExtendSim application without going through ExtendSim's interfaces. The DispID of GetObjectHandle is 101.

```
getVariant      = malloc(sizeof(VARIANTARG)*3);
argStr = SysAllocString((WCHAR *) L"Dialog Item Name");
VariantInit(&getVariant[0]);
getVariant[0].vt = VT_BSTR; // could be a long or a real
getVariant[0].bstrVal = argStr;

VariantInit(&getVariant[1]);
getVariant[1].vt = VT_I4;
getVariant[1].lVal = 23; // blockNumber

argStr2 = SysAllocString((WCHAR *) L"model-1.mox");
VariantInit(&getVariant[2]);
getVariant[2].vt = VT_BSTR; // could be a long or a real
getVariant[2].bstrVal = argStr2;

// Set the DISPPARAMS structure that holds the variant.
dp.rgvarg      = getVariant;
dp.cArgs       = 3;
dp.rgdispidNamedArgs = NULL;
dp.cNamedArgs  = 0;

// Call IDispatch::Invoke()
hr = m_pDisp->Invoke( getID, IID_NULL, LOCALE_SYSTEM_DEFAULT,
                     DISPATCH_METHOD, &dp, NULL, &ei, &uiErr);
SysFreeString(argStr);
SysFreeString(argStr2);
```

Using VBA for ActiveX/OLE Automation

On page 113 you saw how to control ExtendSim using C++ and the five methods that ExtendSim supports. ExtendSim can also be controlled using Excel and VBA.

The workbook “Excel Client-Server Model Workbook.xls” (Examples\Developer Tips\OLE Automation\VBA) contains examples of how to communicate with, send data to, and receive data from an ExtendSim model using Excel VBA and OLE automation. For these examples, Excel is the Client application and ExtendSim is the Server application.

 This workbook provides a form-based interface to drive communication between Excel and ExtendSim.

The VBA code in this workbook contains numerous examples that illustrate how to use OLE Automation to remotely interact with an ExtendSim model to perform the following:

- Start and stop the ExtendSim application
- Open and close a model
- Make ExtendSim run asynchronously
- Determine if ExtendSim is running or paused
- Change the end time of a simulation model
- Get a database component, (e.g. table or indexes) from an ExtendSim model's database
- Receive the contents of a database table into a range of worksheet cells
- Poke the contents of a range of worksheet cells into an ExtendSim database table
- Get the path and name of an ExtendSim model
- Get the current time of a simulation run
- Set a dialog parameter value in an ExtendSim model's block

Using Visual Basic for ActiveX/OLE Automation

On page 113 you saw how to control ExtendSim using C++ and the five methods that ExtendSim supports. ExtendSim can also be controlled using Microsoft Visual Basic (VB).

In VB, the syntax for creating or connecting to instances of ExtendSim and for controlling the ExtendSim application is identical to Visual Basic for Applications (VBA) discussed above. The primary difference between VB and VBA is that VB code can be compiled to create executable and DLL files.

The executable file “ExtendSim OLE.exe” (Examples\Developer Tips\OLE Automation\VB\VB Client App) is a VB program that creates an ExtendSim object and uses ExtendSim's execute, poke and request OLE methods to communicate with and control the ExtendSim application. This program starts ExtendSim (if necessary), loads a model, sets a dialog parameter, executes a command, and gets a dialog parameter.

The VB source code and all of the Visual Studio files required to build “ExtendSim OLE.exe” are provided in the same folder as the executable file. View the source code to see examples of how to implement ExtendSim OLE methods and how to create and connect to instances of ExtendSim using Visual Basic.

ModL

Animation Using ModL

Procedures and suggestions for how to
create and modify ModL code

*“In baiting a mousetrap with cheese,
always leave room for the mouse.”
— Hector Hugh Munro*

This chapter is specific to using ModL functions for performing 2D and 3D animation.

2D animation

The functions listed in “2D Animation” on page 259 make it easy to add 2D animation to blocks. This section discusses the general procedures in creating 2D animated blocks, then shows some examples of how you might add 2D animation to the blocks you build.

 Even if Run > Show 2D Animation is not selected, it is possible to display 2D animation.

The Show 2D Animation command (Run menu) only controls whether animation is shown *during the simulation run*. At all other times, 2D animation is always available. For instance, animation is available when the modeler makes any changes in a block's dialog, whether Show 2D Animation is selected or not and regardless of whether the simulation is running. When the command Show 2D Animation is selected, animation is available at all times.

This is a powerful feature because it lets you build blocks which show their initial status or final values without having to turn on 2D animation. For example, if a check box is clicked, the dialog can send an “on myCheckBox” message to the block and the block can animate the change on its icon, as seen for the Miles block in “Adding 2D animation” on page 54.

 3D animation is discussed starting on page 128.

Overview

As shown in “Adding 2D animation” on page 54 and described in detail below, the basic steps for adding 2D animation to a block are:

- ▶ Decide how you want the block to animate, including the shape, color, and pattern that the animation objects should have.
- ▶ Create the 2D animation objects in the icon pane of the block's structure window. Do this manually by placing an animation object in the icon pane or dynamically through block code.
- ▶ Initialize the animation objects in the CheckData or InitSim handlers.
- ▶ Add code to update the animation object at the correct times, including (if necessary) code to slow the display so that it isn't too fast to be seen. For continuous blocks, this code will be in the Simulate message handler. For discrete event or discrete rate blocks, this code will be in the function or message handler appropriate for the data that you want animated. (For example, if you are animating an item entering a block, the code would go in the ItemIn message handler.)

Deciding how to animate the icon

There are several ways to animate a block's icon. You can show and hide text or a shape (such as a rectangle, oval, or level), move a shape within or outside of the icon (even along the connection lines), show a changing level, stretch and reduce a shape's size, show a picture, or change colors, patterns, or text. The AnimationPoly function is especially useful. As shown in the Select Value In and Select Value Out blocks (Value library), it allows the animation of an arbitrary shape.

 Some animations (such as moving or stretching) will cause simulations to run slower than other types (for example, changing color or text).

While it is common that icons are animated, it is also possible to animate in the area around icons. See the Planet Dance model (located in the folder Examples\Continuous\Custom Block Models) for an example of a model that shows animation outside of the icon. Also see the blocks in the Item library for examples of animating from one block to the other, along connection lines.

Creating 2D animation objects

All 2D animation is done through the use of one or more animation objects that you put in the Icon pane. The animation object itself is a resizable rectangle with dotted sides, identified by a unique number. To create an animation object:

- ▶ In the Icon Tools popup menu of the ExtendSim toolbar, select the Animation Object tool; it is at the bottom of the menu.
- ▶ Click in the Icon pane. *While still holding the mouse down*, drag to create a rectangular animation object. Since this is the first animation object, it will have a “1” in it, as shown at right. You will use that object number in all of the animation functions.



Animation object added

Animation objects are always rectangles in the icon pane; the animation functions in the block's code determine the characteristics of the object, for example, the shape and color that will show as the block is animated.

2D animation objects can also be created “on-the-fly” using the AnimationObjectCreate function.

Initializing animation objects

In the block's code, initialize the object in the InitSim message handler. If you always want the animation object visible, even before an animation is run, put the same code in the CreateBlock handler.

The initialization will generally consist of a call to one of the object definition functions (AnimationOval, AnimationRndRectangle, AnimationPoly, and so forth), a call to AnimationColor (to set the color and pattern for the object), and perhaps a call to AnimationShow so that the object is visible at the beginning of the simulation (for example, for showing initial text or color). Here is an example:

```
on initSim
{
  AnimationOval(1);           // Set 1 to oval
  AnimationColor(1, 0, 65535, 65535, 1); // Set 1 to red (HSV)
                                   // solid (pattern 1)
  AnimationShow(1);          // Optional
                                   // makes object visible
}
```

 If you define the animation object in the code as an oval, and resize it on the icon as a square, it will show as a circle. If you define it as an oval and resize it as a rectangle, it will show as an oval.

The color of an object is set with numbers for hue, saturation, and brightness (value), often called HSV. Use the Color tool in the toolbar to determine the HSV values of any color.

The pattern is defined from the Pattern tool. The numbers for the patterns are as shown at right.

If you want a block to see whether or not the Run > Show 2D Animation command is checked (for example, to hide the animation object if animation is not on), use the AnimationOn system variable in the CheckData or InitSim message. It is set to TRUE (1) if animation is on and FALSE (0) if it is not. For example, instead of the preceding initialization, you might have:

```
on initSim
{
  AnimationOval(1); // Set object 1 to oval
  // Set object 1 to red solid (pattern 1)
  AnimationColor(1, 0, 65535, 65535, 1);
  if (AnimationOn) // Animation is on
    AnimationShow(1); // Optional: show 1 now
  else // Animation is not on
    AnimationHide(1, FALSE); // Hide object; it is not
                                // outside of icon
}
```



Numbers for patterns

Updating the animation object

As mentioned earlier, if Show 2D Animation is not selected, animation is still available except during the Simulate message. The Show 2D Animation command only affects 2D animation that is specified in the Simulate message. During Simulate, ExtendSim blocks check the state of the Run > Show 2D Animation command to determine whether or not to perform animation. In the Simulate message handler (for continuous blocks) or in the function or message handler appropriate for the data being animated (for discrete event and discrete rate blocks), put the code that checks whether you want to change the animation object (its position, color, or text). To speed execution of the block, be sure to only call an animation function when the object has changed. If the animation object shows too fast to be seen, you may want to include a call to the WaitNTicks function. Note, however, that this will also slow down the simulation.

Animating hierarchical blocks

To animate a hierarchical block's icon, you have to include, in the hierarchical block's submodel, a block that has code for the animation but no animation object on its icon. Instead, the animation object is created on the hierarchical block's icon.

The block in the submodel is used to read the values from other submodel blocks and control the animation of the hierarchical block's icon based on those values. In order to do this, it references the hierarchical block's animation object using the negative of the object number. For example, if the number of the animation object on the hierarchical block's icon is 2, the block in the submodel would reference animation object -2 throughout its ModL code.

Animating hierarchical blocks is the only time when you would use negative values to reference an animation object. As described in the User Guide, the Animate Value and Animate Item blocks (Animation 2D-3D library) contain code that allows them to animate hierarchical blocks.

The block that controls the animating on a hierarchical block's icon can be deeper than one level in hierarchy. However, searching from the lowest to the highest level, that block will try to animate the first animation object with the correct number that it finds.

- ☞ If more than one hierarchical block has an animation object with the same number, the lowest one above the controlling block will be the one that is animated.

Showing and hiding a shape

An animation object could be displayed if an input value met some condition or became true, or if an item arrived in the block or was being processed. Hiding the animation object would then indicate the opposite condition. You could have a small object near the connector to indicate item arrivals or meeting a condition, like the Select blocks (Value library). Or you could have a larger object indicating a true value or processing, such as the Activity blocks (Item library).

This example uses the animation object drawn above. You want a solid red circle to appear in the icon when a value is true (greater than 0.5) and you want to hide it when the value is false. To do this, initialize the animation object and hide it in InitSim. Then put code in the Simulate message to indicate when and how the object should change:

```
on initSim
{
  AnimationOval(1);           // Set 1 to oval
  AnimationColor(1, 0, 65535, 65535, 1); // Set 1 to red (HSV),
  // solid (pattern 1)
  AnimationHide(1, FALSE);    // Hide object; it is not
  // outside of icon
}

on Simulate // Or appropriate function or message handler
{
  ...;
  if (AnimationOn && ConditionIsTrue) // if both true
    AnimationShow(1);                // Show object now
  else
    AnimationHide(1, FALSE);          // Else hide object; it is not
    // outside of icon
}
```

Moving a shape

Assume that you want the animation object drawn above to move between the original position and a position 20 pixels higher than the original depending on the values received. The input at the connector called "con1in" takes in real values from 0 to 1, with 1 indicating the highest desired input. Since, in screen coordinates, up is considered negative relative to the starting position, you must call AnimationMoveTo with a negative number to move the object up. Your code might be:

```
integer    obj1Loc;           // Save the object's location
real      clipped;
integer    testLoc;
. . .
```

```

on initSim
{
  AnimationOval(1);           // Set object 1 to oval
  AnimationColor(1, 0, 65535, 65535, 1); // Set 1 to red (HSV)
                                   // solid (pattern 1)
  if (AnimationOn)             // Animation is on
    AnimationShow(1);          // Show object now
  else                          // Animation is off
    AnimationHide(1, FALSE);   // Hide object; it is not
                                   // outside of icon
}

on simulate // Or appropriate function or message handler
{
  clipped = Min2(con1in, 1.0); // Highest position is 1
  clipped = Max2(clipped, 0.0); // Lowest position is 0

  testLoc = int(clipped*-20.0); // Scale to range, upwards
  if (testLoc != obj1Loc)       // Do nothing if not change
  {
    obj1Loc = testLoc;
    AnimationMoveTo(1, 0, Obj1Loc, FALSE); // Move to new location
  }
}

```

Changing a level

Sometimes you want to display a changing level, such as in a water tank. For this example, use the animation object drawn above. A simple block that displays a changing level when its input connector varies between 0.0 (lowest level) and 1.0 (highest level) might look like:

```

on InitSim
{
  // Initialize animation object number 1 as a "level" shape
  AnimationLevel(1, 0.0); // Initialize level to low
  AnimationColor(1, 0, 65535, 65535, 1); // Set 1 to red (HSV),
                                   // solid (pattern 1)
  if (AnimationOn)             // Animation is on
    AnimationShow(1);          // Show level now
  else                          // Animation is off
    AnimationHide(1, FALSE);   // Hide level; it is not
                                   // outside of icon
}

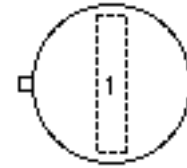
on simulate // Or appropriate function or message handler
{
  AnimationLevel(1, con1in); // Input connector value (0.0 to 1.0)
                                   // controls the height of the level
}

```

To have the level reflect values other than 0 to 1, scale the input values to correspond to that range. See the Holding Tank block (Value library) for an example of changing a level.

Stretching a shape

You can stretch an animation object horizontally or vertically, or both at the same time (circular), relative to its original position on the icon. For example, do this to show the relative size of an item (for instance, based on an attribute value) or to indicate a direction of flow.



Object for stretching

The following code stretches the object vertically inside the icon. This method uses the exact size of the animation object as it is drawn on the icon to determine the boundaries for the stretch. The input at the connector called “con1in” takes in real values from 0 to 1, with 1 indicating the highest desired input. For this example, draw an animation object like the image shown here.

```
integer    origWidth, origHeight;    // Boundaries of the object
integer    pixels;                  // How far to stretch
real       clipped;                 // Amount to change
. . .

on InitSim
{
    AnimationRectangle(1);           // Set 1 to rectangle
    AnimationColor(1, 0, 65535, 65535, 1); // Set 1 to red (HSV),
                                         // solid (pattern 1)
    origWidth = AnimationGetWidth(1, TRUE); // Get original width
    origHeight = AnimationGetHeight(1, TRUE); // Get original height

    if (AnimationOn)                 // Animation is on
        AnimationShow(1);           // Show object now
    else                             // Animation is off
        AnimationHide(1, FALSE);    // Hide object; it is not
                                         // outside of icon
}

on simulate    // Or appropriate function or message handler
{
    . . .
    clipped = Min2(con1in, 1.0);      // Highest position is
    1.0
    clipped = Max2(clipped, 0.0);     // Lowest position is 0.0
    pixels = clipped * origHeight;    // Calculate stretch
    // Subtract the pixels to go upward (negative is up relative to base)
    AnimationStretchTo(1, 0, origHeight - pixels, origWidth, pixels,
    FALSE);
}
```

ModL

To have the shape stretch outside of the icon, set the last argument in AnimationStretchTo to TRUE and increase the value for pixels so it will stretch outside the icon. Stretching or moving outside the icon slows animations considerably and might cause the icon to flash.

See also the AnimationPoly function which can be used for animating an arbitrary shape or for dynamically changing the shape of an icon. Examples are the Select Item In and Select Item Out blocks (Item library).

Showing a picture on an icon

You can also have a picture show on an icon in response to some occurrence in the block. For example, the code for a picture might look like:

```

on initSim
{
// Set object 1 to picture, not scaled
AnimationPicture(1, "MyPicture", FALSE);
AnimationHide(1, FALSE);           // Hide object; it is not
                                   // outside of icon
}
on simulate      // Or appropriate function or message handler
{
. . .
if (ConditionIsTrue)
    AnimationShow(1);              // Show picture
else
    AnimationHide(1, FALSE);       // Hide object; it is not
                                   // outside of icon
}

```

 In order to use a picture, it must be a resource in the System, in ExtendSim, or in a file in the ExtendSim7\Extensions folder as discussed in “Picture and movie files” on page 86.

Moving a picture along the connection line between two blocks

A picture traveling along the connection line between two blocks is an effective way to animate individual items as they flow through a model. As shown above, you can assign a picture to an animation object. By calling `AnimationBlockToBlock`, the animation picture can move from one block to the next along the connection line.

 In order to use a picture, it must be a resource in the System, in ExtendSim, or in a file in the ExtendSim7\Extensions folder as discussed in “Picture and movie files” on page 86.

Arguments for `AnimationBlockToBlock` include the animation object number and the block numbers and appropriate connector numbers of the two blocks you wish to animate the picture between. Typically you do not know how blocks will be connected until the model is built. Therefore you must determine the values of these parameters by using function calls to `MyBlockNumber`, `GetConNumber`, and `GetConBlocks`. You can call the functions from any block, provided it knows which block is sending and which is receiving.

The following code example illustrates how to determine the appropriate parameter values and make a call to `AnimationBlockToBlock`. In this example, `AnimationBlockToBlock` is called from the “sending” block (the block from which the picture will start moving). The picture will move to the “receiving” block connected to the `con1Out` connector of the “sending” block.

```

integer array[][2];
...
on InitSim
{
AnimationPicture(1, "MyPicture");    // Set object 1 to picture
myNumber = MyBlockNumber();          // determine "sending" block #
// determine "sending" connector #
outCon = getConNumber(myNumber, "con1Out");
getConBlocks(myNumber, outCon,array); // what blocks are connected
                                   // to con1Out?
}

```

```

rBlock = array[0][0];           // determine "receiving" block #
rConn = array [0][1];           // determine "receiving" connector #
}

on simulate
{
    // animate picture between blocks
    AnimationBlockToBlock(1,myNumber,outConn, rBlock, rConn, 1.0);
}
    
```

Changing a color

To use a change in color instead of motion, include a call to `AnimationColor` with a variable for one of the hue, saturation, or brightness (value) arguments:

```

on InitSim
{
    AnimationOval(1);           // Set object 1 to oval
    AnimationColor(1, 0, 65535, 65535, 1); // Set 1 to red (HSV),
                                         // solid (pattern 1)
    if (AnimationOn)            // Animation is on
        AnimationShow(1);      // Show object now
    else                        // Animation is off
        AnimationHide(1, FALSE); // Hide object; it is not
                                         // outside of icon
}

on simulate // Or appropriate function or message handler
{
    . . .
    hueVal = hueVal+1000;       // Different colors
    AnimationColor(1, hueVal, 65535, 65535, 1);
    . . .
}
    
```

Changing text

You can also animate by changing text in the animation object created at the beginning of this section.

 Using `AnimationText()` causes a white background around animated text. To not have this, instead use the function `AnimationTextTransparent()` as is done below.

```

string      objText;

on InitSim
{
  AnimationTextTransparent(1, "");      // Set object 1 to blank text
  AnimationColor(1, 0, 0, 0, 1);        // Set 1 to black (HSV),
                                         // solid (pattern 1)
  if (AnimationOn)                      // Animation is on
    AnimationShow(1);                  // Show object now
  else                                  // Animation is off
    AnimationHide(1, FALSE);           // Hide object; it is not
                                         // outside of icon
}

on simulate      // Or appropriate function or message handler
{
  . . .
  if (temp < 1000)                          // Less than 1000
    objText = "Cool";
  else if (temp < 1500)                     // Between 1000 and 1500
    objText = "Med";
  else                                     // Greater than 1500
    objText = "Hot";
  AnimationTextTransparent(1, objText);    // Set the text
  . . .
}

```

Animating pixels

You can generate a rectangle of pixels where each pixel can be a different color based on the value for that pixel. For example, use this to generate a contour map or give visual display of temperatures over a two-dimensional space. For an example of this, see the Mandelbrot model located at \Examples\Continuous\Custom Block Models.

3D animation

The ExtendSim 3D (“E3D”) window provides a fully three-dimensional representation of the world of the model. The objects modeled in the window maintain information about their positions in three dimensions as well as other physical properties of their representations. The window is also tightly integrated with the rest of the ExtendSim simulation engine. Every motion of every object, every aspect of things that happen in the 3D world, can be controlled from the ModL programming language.

☞ 3D animation is only available in ExtendSim Suite.

☞ This chapter presumes you have read the information about using the E3D window and environment in the ExtendSim User Guide.

About the 3D chapter

This chapter provides information for those interested in code development within the E3D development environment. It is organized into the following sections:

- 1) Overview, including a brief discussion of the Torque Game Engine (TGE), upon which the E3D environment is based
- 2) Manipulating existing objects, such as adding a mount point or additional skins
- 3) Building new objects such as people, vehicles, and so forth

The focus of this chapter is on providing a high-level description of the E3D environment, with explanations where the E3D environment differs from the TGE.

In many cases the E3D environment uses standard TGE constructs, and they will not be repeated here. Instead, see the GarageGames website at www.GarageGames.com for complete details about TGE constructs, including DataBlocks and Torque Scripts.

Overview

The Torque Game Engine and the E3D environment

For 3D animation, ExtendSim incorporates its own development environment, a compiled version of the Torque Game Engine (“TGE”), Torque Script files, and numerous 3D functions for communicating with the E3D window. Together this is termed the *E3D development environment*. The TGE is a powerful 3D rendering architecture that GarageGames (www.GarageGames.com) licenses as a source code development environment.

Since the E3D environment is open-source, you can modify the animation capabilities of existing blocks, create entirely new 3D-enabled blocks and 3D objects, and modify object behaviors for specialized applications. ExtendSim has numerous ModL functions for defining the custom behavior of 3D objects and events; the Torque Script scripting environment provides additional control over the animation environment and object behaviors.

Note that the running of a simulation is not necessarily a part of the model’s control of the E3D window content. The Boids model (located in the \Examples\3D Animation folder), is an example of a model that uses custom blocks and does not utilize the simulation loop to control the behavior of the objects in the model.

 If you are serious about developing extensive 3D simulations with ExtendSim, and you want direct access to the TGE API, consider purchasing at least an “indie” license for the TGE. More information about the TGE can be found at www.GarageGames.com. (Even if you don't plan to do any development with the engine, this will allow access to the developer forums.)

Performance considerations

The E3D window is doing a complete textured rendering of a complex 3D environment, so the number of objects being rendered and the capability of the hardware on which the software is run will have an effect on the performance of the E3D window. See the User Guide for some suggestions for improving performance.

Modes of E3D animation

Each model has a saved 3D animation mode – QuickView, Concurrent, or Buffered, as discussed below. These modes are selected in the Run > Simulation Setup > 3D Animation dialog and control aspects of the interaction between the ExtendSim application, the E3D window, and the blocks’ ModL code.

The 3D animation modes are discussed fully in the E3D module of the User Guide. The following is meant to be supplemental to that information.

Quickview

Quickview is the default mode and allows a model to display its behavior in the E3D window without being extensively modified. In this mode most of the blocks at the top level of the 2D model will have a representation in the E3D window and a 3D object's position will correspond to the location of the corresponding block in the 2D model. However, QuickView mode uses the immediate functions (discussed in "Types of 3D ModL functions" on page 131) so only one object will move at a time. This mode allows the modeler to get a quick 3D view of the simulation with minimal customization of the model.

Concurrent

The Concurrent mode is more commonly used for a model that has been designed or modified to specifically support E3D functionality. In this mode, multiple items can be moving in the E3D window at one time and the locations of the blocks in the 2D model don't necessarily correspond to the locations of the objects in the E3D window.

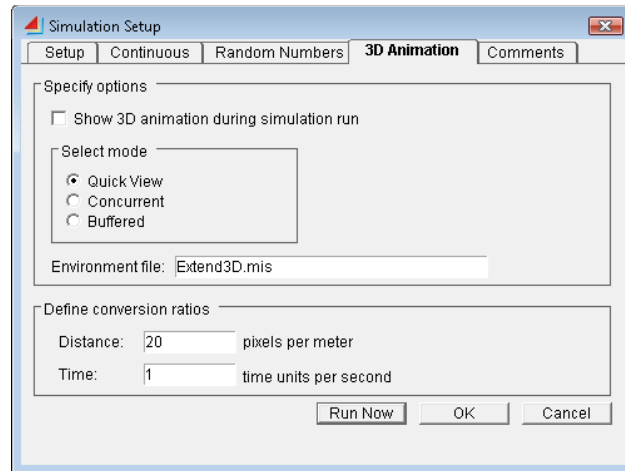
In Concurrent mode events are posted internally with the E3DPost functions. This causes certain events, such as the creation of an object or an object moving, to be put into a queue to be performed later. (For more information about the E3DPost functions, see "Types of 3D ModL functions" on page 131)

An additional aspect of Concurrent mode is that the simulation engine will pause with each E3DPost function call until the currently posted call has begun processing. This assures that the simulation and the E3D window keep in sync, but it adds the additional requirement that the 3D events posted by the model need to be posted in time order (earlier before later). Note that this is handled automatically by the blocks in the Item library.

Buffered

Buffered mode is a variation on Concurrent mode, where the information about what should happen in the E3D window is stored in an internal buffer and replayed later.

As is true of the Concurrent mode, the Buffered mode posts events internally using the E3DPost functions. (For more information about the E3DPost functions, see "Types of 3D ModL functions" on page 131.) However, one advantage of the Buffered mode, which only applies to models built with custom blocks as opposed to models that use the Item library blocks, is that the events posted with the E3DPost functions do not necessarily need to be posted in order. This is because the events are queued in the buffer and sorted.



3D Animation tab in Simulation Setup dialog

Time ratios and 3D event queues for the modes

As discussed in “Distance and time ratios” on page 131, each simulation time unit will by default take one second to display in the E3D window. In Concurrent and Buffered modes the simulation time to real world clock time ratio is carefully respected. In these modes, almost all of the actions the blocks take in the E3D environment are posted using the E3DPost functions (see “Types of 3D ModL functions” on page 131) which specify simulation time as the first argument of the function call. Internally, ExtendSim maintains a list of the posted events and processes them at the correct time. This means that multiple events can happen at the same time and multiple objects can be moving at the same time.

 The event queue maintained by the E3D window is independent from the event queue maintained by the Executive block.

Types of 3D ModL functions

There are two broad categories of functions that manipulate the 3D objects in the E3D window:

- Post functions. Each of these functions starts with E3DPost and contains as its first argument the simulation time value when the function should occur. In many cases, such as in the Item library, this value will be current time. However there is no reason why a custom implementation could not use more distant future values.

As discussed in “Modes of E3D animation” on page 129, the post functions are used in the Concurrent and Buffered animation modes and have the effect of putting the E3D commands into an internal queue. This queue of commands will be executed in time order by the simulation engine. In Concurrent mode the execution will be immediate; in Buffered mode it will be delayed until the user clicks on the Start button.

- Immediate functions. These functions do not have a time value associated with them and they take place immediately. Thus they lose the connection between simulation time and E3D window time that the Post functions have. An example of a model that uses immediate functions rather than Post functions would be the Boids model (Examples\3D Animation), which doesn't even run a simulation.

Distance and time ratios

The distance units in the E3D environment are in meters. The default ratio between the pixels in the 2D model window and the distance units in the E3D window is 20 pixels per meter. This means that for models that use a direct relationship between the two windows, each 20 pixels of distance in the 2D window will translate to one meter of distance in the E3D environment. The ratio can be modified in the Run > Simulation Setup > 3D Animation dialog shown on page 129.

In Concurrent and Buffered modes, time in the E3D window is related to ExtendSim simulation time by the time ratio defined in the Run > Simulation Setup > 3D Animation dialog shown on page 129. By default this ratio is 1 simulation time unit to 1 second of real time. For example, if a simulation is set to take 60 time units, it is in Concurrent mode (using E3DPost calls to establish events in the E3D window), and the E3D window is open, the animation in the E3D window should display for 60 seconds.

 In QuickView mode the time relationship between simulation time and real time is not preserved, so the time ratio defined in the Simulation Setup dialog does not have an effect for that mode.

References

Read the 3D Animation module in the ExtendSim User Guide before doing any 3D development work.

The GarageGames web site (www.GarageGames.com) is useful for finding additional resources when developing in the E3D environment, contacting consultants for custom work, and asking questions about the Torque engine.

There are also several books about the TGE. They tend to be focused on game development but also contain invaluable information about development with the TGE that will in large part translate directly into working with the E3D window. A couple of books to consider are: “3D Game Development All In One” and “The Game Programmers Guide to Torque”.

A complete discussion of the TGE is beyond the scope of this document. Instead, there is extensive information about the Torque scripting environment, as well as the default layout and behavior of TGE scripts, on the GarageGames website at www.GarageGames.com.

Manipulating object behaviors and appearances

Objects in the E3D environment have many properties that define how they look, behave, and interact with the 3D window and with the ExtendSim simulation engine. The following sections define some of these properties, describe how they work within the context of the E3D environment, and how you can change or adjust them.

E3D object names and labels

There are two object name-type properties that are used and referenced in ExtendSim – the name field in the World Editor Inspector and the object label that is displayed along with the object in the E3D window.

Name field

The *Name field* is at the top of the Inspector pane of the World Editor Inspector. This field is an optional text identifier for the object that can be set either in the World Editor or through the E3DSetName function. The name of an object can be used in ModL functions to locate an item by name, rather than numerically with the objectID value (see “ObjectIDs, userTags and groupTags” on page 132.) If used, the name of an object will also appear in the Editor in text drawn on the item following the objectID.

Object label

The other name-like property is the *object label*. This is set with the E3DSetObjectLabel function and produces a visual label that is displayed above the object in the E3D window but not in the World Editor Inspector’s Inspector pane. You can also change the color of the text of the object label.

 To display text in the E3D window without having a visual object associated with it, create a waypoint and set its object label. Waypoints are invisible, but the object label will still be drawn. For instance, this is how the 3D Text block (Animation 2D-3D library) displays text.

ObjectIDs, userTags and groupTags

ObjectIDs, userTags, and groupTags are the numeric properties associated with objects in the E3D window. Many of the 3D functions have arguments that refer to these numeric properties.

ObjectIDs

The objectID is the main identifier that the E3D window uses to identify the objects it needs to support. There is a unique objectID value associated with each object in the E3D window. This

includes not just the objects that are created from ModL code, but also all of the objects that the window uses as the basic building blocks of the 3D world.

- ☞ Names can also be used by ModL functions to locate objects, as discussed in “E3D object names and labels” on page 132.

For example, the Camera object (described in the User Guide) has an objectID. This allows you to call any of the ModL functions that take an objectID value on the Camera and facilitates features like setting the position of the camera dynamically to track a moving object. This is shown in the 3D Animation\Tips\E3D Path model.

For Concurrent and Buffered mode models, which internally call the E3DPost functions instead of the immediate functions (as discussed in “Modes of E3D animation” on page 129), the objectID will not be available at the time the function is called. This is because the objectID is created by the E3D window code when the object is instantiated in the 3D world. In these modes a temporary userTag is created to identify the object, as discussed below.

- ☞ UserTags should be used for all “post” functions. You should only use the objectID when you are sure that the object will exist at the time the function is called.

UserTags

As shown in “Modes of E3D animation” on page 129 and “Types of 3D ModL functions” on page 131, the Concurrent and Buffered modes call the E3DPost functions. At the time the E3DPost function is called, the simulation time when the object will be created may not yet have been reached. So operations on the object are put into a queue to be performed later. This means that a temporary value, a *userTag*, needs to be created that will be used to identify the object before it is created. The userTag is the value that will be returned by the E3DPostCreateObject that schedules the creation of the object.

The userTag is used to identify the object until it is actually created in the 3D window. However, once the object is created and an objectID has been established, the userTag is still available and each object will have both a userTag and an objectID. Thus the object can be referred to by either its userTag or its objectID; they are distinguishable from each other by the fact that userTags are negative and objectIDs are positive.

- ☞ UserTags should be used for all “post” functions. You should only use the objectID when you are sure that the object will exist at the time the function is called.

There are ModL functions to convert userTags to ObjectIDs (see the function list in “3D Animation” on page 264).

GroupTags

GroupTags are another numeric tag on the objects in the E3D window that are specified at the time of object creation and also stay with the object throughout its lifetime. Unlike userTags and objectIDs, groupTag are not unique identifiers but rather are used to define groups of objects that the developer wants to be able to reference as a whole.

A common example of how groupTag are used is in the Item library, where each of the items that travels from block to block is tagged as being part of the same group. This allows the blocks to delete all of these items as a group when one simulation ends and a new one begins, without deleting other objects that need to remain for the next run.

- ☞ By default an object created in the World Editor will be supplied with a groupTag value of 0. The Item library uses the value of 1 for items that travel from block to block.

E3DDeleteAllObjects takes a groupTag argument and will delete all objects in the E3D window with that groupTag number. If you pass in a negative 1 or lower, the function will ignore the groupTag and delete all objects in the E3D Window.

E3DGetObjectNames takes an argument for which kind of objects the function should return. If you pass in a number greater than zero, that number will define a groupTag and the function will return the names of the objects in that group. See the function list in “3D Animation” on page 264 for more detailed information about exactly how these functions operate.

Object classes

ExtendSim defines several classes of objects in addition to the class structure of object types defined by the TGE.

All ExtendSim 3D objects descend from *ExtendBase*, which gives them certain common features as described in “DataBlocks” on page 144. Which class a new object falls into will be defined in its DataBlock. The complete class structure of the objects in the E3D window is beyond the scope of this documentation.

ExtendItems

ExtendItems are intended to represent the items that pass from block to block in an ExtendSim model. They have several features:

- They do not support physics features like gravity, friction, and momentum. This is to preserve the synchronization between the simulation and the 3D animation. See “Gravity” on page 138.
- ExtendItems support the DataBlock variable hasFront, which allows the object to continuously turn toward the direction that it is traveling.
- They support mountStacking and therefore should have a mount0 on the top of the object and a mountPoint on the bottom. See “MountStack” on page 137 for more information.

ExtendPlayers

ExtendPlayer objects (Male and Female) are based on the AIPlayer class in the TGE. Some information about them includes:

- Unlike Torque AIPlayer objects, ExtendPlayer objects do not attempt any AI calculation
- Like ExtendItems, they avoid momentum and friction calculations
- They do support gravity. If created at a Z height of greater than 100.0 they will drop to the ground. See “Gravity” on page 138.

ExtendVehicles

ExtendVehicle objects (Car, AGV, and Forklift):

- Are similar to ExtendPlayer objects in how they differ from standard Torque objects
- Like ExtendPlayer objects, they support gravity but not friction or momentum
- Do not have any AI calculations

Scale and position

The User Guide describes how to visually scale objects in the World Editor and through block dialogs or by changing the numbers in the Inspector pane. You can also scale objects through ModL function calls.

Units of scale are relative to 1 and are associated with each axis, so scaling an object from 1, 1, 1 to 2, 2, 2 will make it twice as big along each axis. If you scale an object from 1, 1, 1 to 1, 1, 2 it will

stretch along the Z-axis but remain the same along the X and Y-axis. So it will just look a bit distorted, but taller.

The numbers used in this example are in the same format as those in the Inspector pane of the World Editor Inspector, although there they are shown without commas. For example you might see “1.1 0.5 1.0” in the Inspector pane, which would be scaling to 110% on the X axis, 50% on the Y axis, and 100% on the Z axis.

Object position uses the same format as scale, and the X, Y, and Z values represent the X, Y, and Z coordinates of the object. Changing the X value from 1 to 2, for example, will move an object 1 meter along the X axis. The default Z values for objects will be a bit above 100.0 since the flat terrain in the default E3D window is at height Z = 100. See “Gravity” on page 138 for more information.

Rotation

The User Guide describes how to visually rotate objects in the World Editor, through block dialogs. Another method of rotating objects would be by changing the numbers in the Inspector pane.

The World Editor Inspector has four numbers for rotation – three factors and an angle in degrees – where each factor represents the x, y, or z axis respectively. Each multiplier determines the amount by which the object is rotated around its particular axis based on the given angle.

For example, “0 0 1 90.0” means that the object is rotated 90 degrees around the Z-axis while “1 0 1 45.0” means that the object is rotated 45 degrees around both the X and the Z-axes. The most common form of rotation in the ExtendSim modeling world is around the Z-axis, as this will lead to rotation of the front of the object on the flat plane.

Skins

Skins is the 3D animators term for the texture files that show the surface details of objects in the 3D world. Many of the 3D objects provided with ExtendSim come with more than one skin

You change which skin is showing on an object either through a block’s Block Animation tab (Item library) or by using ModL functions calls. You can also define a new DataBlock to specify a new version of an object that uses a specific skin (see “DataBlocks” on page 144 and “DataBlock variables” on page 146 for more information about them.)

How skins are defined

ExtendSim supports the naming convention “base.objectName.jpg” which allows developers to add new skins or modify existing ones. This is specified as follows:

- Base is the name of the default skin. To select a new skin, change this part of the name. For example, the Ball object by default will load using the base.ball.jpg skin. To change the ball object to blue, create a blue.ball.jpg and then either call the appropriate function or select the skin on a block dialog.
- ObjectName is usually the same as the name of the object and it must be the same for each skin of a single skin object. For instance, there are several skins in the folder with the Bed object; the first is called base.bed.png and the others are called floral.bed.png, hospital.bed.png, and so on.
- The final part, jpg, is just the extension of the file type.

 Skins in ExtendSim will all be either jpg or png files.

Most of the objects that ship with ExtendSim have a single overall skin that can be changed. The two people objects (Male and Female) however, have two skins each – one for the clothing of the object and a second for the face and hands.

Two skin objects

The two skin objects, Male and Female, have the following skin patterns:

- Base.maleclothes.jpg
- Base.maleskin.jpg
- Base.femaleclothes.jpg
- Base.femaleskin.jpg

This allows the different skins to be combined in arbitrary fashions. For example, a given object in the E3D window could be a Female with these two skin layers: Base.femaleskin.jpg and Medical.femaleclothes.jpg.

Skin setting functions

The ModL function that sets the skins on an object is E3DSetSkin or E3DPostSetSkin, depending on the 3D mode. This function takes two string arguments for the skins.

- For single skin objects, just enter an empty string for the second string.
- In the case of double skin objects, enter both skin names. So, for the example above, call E3DSetSkin once with the first string set to Base and the second set to FemaleSkin, and then a second time with the first string set to Medical and the second set to femaleClothes. (Note that, unless the object had already had its femaleSkin changed from the base, the first call would not be necessary, because the Base skin is already the default.)

Skin DataBlocks

There are four DataBlock variables for specifying skins, as seen in “DataBlock variables” on page 146:

- SkinBaseName1
- SkinBaseName2
- SkinName1
- SkinName2

You can use these to cause special effects, such as overriding whether or not to start with the base skin. For example, setting the skinName1 variable in the DataBlock will cause the object to be created with the skinName1 value in place of the base part of the skinName.



Female with Medical skin

Mounting

Mounting is the process of attaching one 3D object to another at certain points. An example would be mounting a Box object onto a Male object. In this case, the Box object automatically attaches to the mount0 mount node on the Male object. The Male object then adjusts its animation to change its hand position, so it looks like it is holding the Box. And if the Male is given a destination, the Box will be carried along. All of the ModL functions and internal structures are associated with mounting objects, not images.

- See also “Carriable items” on page 148 and the functions for mounting starting on page 274.

Each object has one mountPoint and at least one mount node. These are internal structures of the DTS file (discussed on page 142); if you create custom 3D objects these structures should be added when the DTS file is created.

- An object’s mountPoint is the part of the object that can mount onto another object. An object’s mount nodes are the point or points on the object where other objects can be mounted.

For example, the Ball object that ships with ExtendSim has a mountPoint at its bottom and a single mount node (mount0) at the top. These need to be defined with the names mountPoint, for the point where the Ball will mount another object, and mount0, for the node where other objects can connect to the Ball.

Smart Mounting

The mount functions require you to specify the node on the mount object where the object to be mounted will be placed. This node will usually be node zero (defined in the DTS object as mount0, as discussed above).

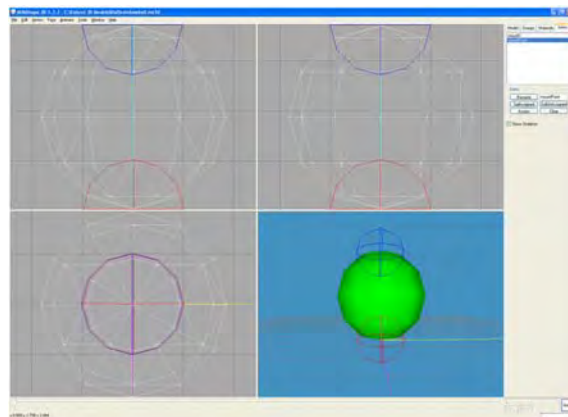
If you instead want the internal logic in the ExtendSim application to select the node that the object will be mounted to, specify that the node should be *smart mounted*. You do this by passing in a negative 1 for the node. ExtendSim will then select the best node to be used based on the type of the mount object and the object to be mounted.

MountStack

One additional type of mounting, specific to ExtendSim, is the mountStack. A *mountStack* is a stack of objects where each object is mounted on top of the previous one. The mountStack functions E3DMountStackInsert, E3DMountStackRemove, E3DMountStackLimit, as well as their Post versions, allow you to maintain a queue of stacked objects with less ModL code than if you used the Mount and Unmount functions. The mountStack functions are on page 267.



Male carrying a Box



Ball in Milkshape

Unmounting

Unmounting an object disconnects the objects from each other but does not change the position of the objects. In the case of the Male and the Box above, unmounting the Box will leave it and the Male in the same position but the Male will drop his hands. And if the Male moves away, the Box will stay where it is.

The unmount functions also have options that allow you to specify a distance and a direction for moving the unmounted object. See the function descriptions for more information.

Collision

E3D objects defined through ModL function calls can have collision enabled or disabled:

- Collision enabled causes the objects to respect each other's boundaries and to stop when they bump into each other
- Collision disabled allows objects to pass through each other

It is a modeling decision whether or not you should enable collision for objects created at locations in your model. Collisions do allow items to line up as if in a waiting line, but in some cases you will not want the objects to be stopped in their travel by colliding with another object.

Gravity

Gravity is implemented in ExtendSim for certain types of objects and not for others.

- The people and vehicle objects will respect gravity. If created at a height or told to travel to a Z location higher than 100, people and vehicles drop down to the ground.
- ExtendItem objects will not be effected by gravity and can travel at a fixed height or even to a different height by simply setting a Z location higher than 100. The Boids example (Examples\3D Animation) shows this – the Boid objects fly through the air by setting 3D destination locations.

 To maintain the constant speeds associated with having travel times match simulation times, the concepts of friction and momentum are not implemented in the objects that travel from location to location in ExtendSim.

Sound

The E3D window supports playing sounds and has a few additions to the Torque Audio system. Sounds can be either:

- Associated with objects in the E3D window. The IdleSound DataBlock variables are an extension to the Torque Audio system so that sounds can be defined and played at automatic times during a model execution. (See the list of sound variables in “DataBlock variables” on page 146.)
- Played by ModL function calls, such as E3DPlaySound.

Sounds are globally enabled or disabled in the Edit > Options > 3D tab.

 Sounds played in the E3D window are 3 dimensional. Thus if you move the camera in the E3D window away from or around a sound source, the volume and direction of the sound will change.

The audio descriptions used in ExtendSim will commonly be either the AudioCloseLooping3d description used by the BoidSound (a looping sound) or the AudioClose3d used by the CarSqueal-Sound (a non-looping sound).

The other way to play a sound in the E3D window is to use the ModL function E3DplaySound. This function, which you could use in an equation-based block if you aren't custom-coding

blocks, specifies the name of an audioProfile and an X, Y, Z location and plays that sound at that location.

An example of an object's ambient sound that is enabled in the E3D window is an Activity block (Item library) when it is represented in the E3D window by the Machine object. If sounds are enabled in the Options tab and the Machine is playing its "running" ambient animation (discussed on page 140), you will hear an active machine sound.

DataBlock of the Machine object

This DataBlock of the Machine object is from ExtendSim7\Extend3D\server\scripts\ExItems.cs.

```
datablock StaticShapeData(Machine)
{
// Mission editor category, this datablock will show up in the
// specified category under the "shapes" root category.
category = "Block";

// Basic Item properties
shapeFile = "~/data/shapes/machine/machine.dts";
mass = 1;
friction = 01;
elasticity = 0.3;
mountPoint = 0;
idleSound[0] = "CarEngineSound";
idleAnimation[0] = "running";
idleSound[1] = "CarSquealSound";
idleAnimation[1] = "blocked";
};
```

This code is an example of associating sounds with objects and their behaviors. The idleSound and idleAnimation variables are two of the custom DataBlock variables available in ExtendSim; the complete list is on page 146. In this example the variables support associating a specific sound with an object's specific ambient animation – the Machine block will play the CarEngineSound while the running animation plays and the CarSquealSound while the blocked animation plays.

 DataBlocks are discussed on page 144 and ambient animation is discussed on page 140.

DataBlock of Boid object

Another example in ExtendSim7\Extend3D\server\scripts\ExItems.cs is the Boid object, which has the following code:

```
// JSL - Boids
datablock AudioProfile(BoidSound)
{
  fileName = "~/data/sound/Boid.wav";
  description = AudioCloseLooping3d;
  preload = true;
};

datablock ExtendItemData(Boid)
{
  // Mission editor category
  category = "ExtendItem";

  // Basic Item properties
  shapeFile = "~/data/shapes/Boids/Boid.dts";
  mass = 1;
  elasticity = 0.2;
  friction = 0.0;
  mountPoint = 0;
 emap = false;

  // JSL - mountLook - look to use when mounted.
  // (For Extend items, they tell the holder to change his/its look.)
  className = "carriableItem"; // class
  mountLook[0] = lookbh;
  unmountLook[0] = look;
  skinname1 = "base";
  hasFront = true;
  idleSound[0] = "BoidSound";
  idleSoundVol[0] = 0.2;
};
```

Differences between the sound examples

A few differences between the Machine and Boids examples are worth noting:

- For the Boids, there is no idleAnimation condition. Thus if the sound is a loop, it will be played continuously; a non-looping sound will play once and quit.
- The ExtendSim idleSoundVol DataBlock defines the volume at which the sound is played. The default, used for the Machine block, is to play the sound at full volume.
- The audioProfile DataBlock is used for the BoidSound. To play a sound in the E3D window you need to define an audioProfile DataBlock which is a standard Torque construct. The file name field defines where the sound file is located and the description field defines how the sound is to be played. More information can be found on the GarageGames website.

Ambient animation

DTS objects (objects that are not large-scale interiors or buildings, as discussed on page 142) can have ambient animations – background behavior such as steam coming from a cup or people breathing. These animations are of two types:

- Those used by the object based on the object type and predefined behaviors for that type. An example is the “run” animation of the people objects. Whenever these objects are moving they have internal code that plays the “run” animation, automatically adjusting the speed of the animation based on the speed at which the object is moving.

- Those played by custom code. An example is the Cup object, which has a “Content” animation. If you start the Content animation running using the E3DAnimationPlay call, the Cup will appear to be full of coffee and will have a wisp of steam rising from it. (To see this, create a Cup object using the 3D Tester model located at \Examples\3D Animation\Tips and activate the Content animation using the controls in the Animation group.)

Stop one ambient animation before starting another

Once you have started an ambient animation using the E3DAnimationPlay function, you might think that you could start another animation using the same function and that would replace the first animation. Due to the nature of the implementation of the animation threads, you should stop the first animation using the E3DAnimationStop function before starting the second animation. In the Cup example, this means that you should stop the “Content” animation before playing the “Empty” animation.

People objects

The people objects have several special ambient animation features:

- An *idle* animation (Root) that is always playing. This animation shows a small movement and essentially makes it look like the people are breathing.
- A *look* animation that is always playing and controls the hand position of the person. See also “DataBlock variables” on page 146 and “Looks” on page 148.
- An overall *body position* which includes sitting, sleeping, and so forth.

Both the look animation and the overall body position animation are automatically set by the 3D engine when the person does certain things. So for the most part you will not need to be specifically setting animation threads on people objects.

Paths, markers, and waypoints

A *path* is a route for an item or other entity that has movement. Each path is a collective object composed of multiple segments, or *markers*, in a predefined order.

A *waypoint* marks a single specific point in the E3D environment; in some ways it is like a path with just one marker. Waypoints are usually used to identify a destination point for a 3D object.

There are a number of ModL functions that reference paths (for example E3DSetTargetPath, which specifies a path for an object to follow). Paths can also be referenced from TorqueScript, which is discussed on page 143.

Markers have locations in three dimensions. Certain kinds of objects (like people and vehicles) will just ignore the Z dimension but the most commonly used type of objects (ExtendItems) does not. Thus you should always set the Z location to the correct value. (See “Object classes” on page 134 for more information on the differences between ExtendItems, ExtendPlayers, ExtendWheeledVehicles, and other types of objects.)

 The height of the ground in the E3D window is 100 meters and the top of objects like the conveyor belt and the machine is 1 meter from the ground, or 101 meters.

Creating a path

Paths can be created through ModL function calls or through the World Editor Creator. For more information about creating paths with the World Editor, see the User Guide. The ModL functions used to create a path are E3DCreatePath, E3DCreateMarker, and E3DAddMarkerToPath; these functions are described on page 268.

By default paths are not visible. Setting a path to visible with the `E3DSetVisibility` function will display the path in the E3D window; this is an extension of the E3D window over standard Torque. You can also set the color of a path using the `E3DpathSetColor` function, which changes how a visible path displays in the E3D window.

Waypoints

A waypoint can be thought of as a path with just one marker; they are often used as destinations for object movement. Like paths and path markers they are invisible in the standard E3D window but visible in the World Editor. There are a number of functions that refer to waypoints. (See the function reference for descriptions.) Waypoints can also be given object labels with the `SetObjectLabel` ModL function, and can therefore be used as a way of displaying text with no visible object on the viewing area of the E3D Window.

A waypoint can be created through ModL code or through the World Editor Creator in the same way as other objects. In the World Editor Creator opening the Shapes > Markers category reveals the `WayPointMarker` object. Clicking on the `WayPointMarker` object in the tree will create a waypoint in the 3D viewing area. See the User Guide for more information about using the World Editor Creator.

 The `E3DSetVisibility` function cannot be used to make a waypoint visible. Waypoints are never displayed in the standard E3D window unless you have added an object label to them as described in the Note on page 132.

Defining new 3D objects

While ExtendSim contains quite a few 3D objects, you might want to add additional ones. Most of the 3D objects supplied with ExtendSim (such as people, items, vehicles and static shapes) are in the DTS format. The following discussion describes how these small-scale 3D objects are defined.

 Buildings or interiors are not usually DTS files. Instead they are DIF files as discussed on page 149.

The steps involved in building a custom 3D object for use in ExtendSim are:

- Create the object in a 3D graphics program that supports exporting the object as a DTS file
- Export the object as a DTS file
- Create a script file/DataBlock for use in ExtendSim
- Add the script and DTS file to the correct places in the ExtendSim folder structure

DTS file format

ExtendSim requires object files to be in the DTS (Dynamix Three Space) format which is specific to the Torque engine. This means that you will either need to work with files that were created specifically for the Torque engine or use one of various conversion utilities (usually a plug-in for a 3D art program) to export a DTS file from a file saved in another format.

ExtendSim DTS file

As an example for developing new objects, see the `Wall.dts` file and the other files associated with it; those files are located in the `Extensions\E3DObjects` folder. (The rest of the 3D objects provided with ExtendSim are located in the `ExtendSim7\Extend3D\Data\Shapes` folder, but new objects should be put into the `Extensions` folder.)

Associated with the `Wall.dts` file are:

- A Wall.cs file. This is a script file that contains the definition of the wall object, mostly in the form of a DataBlock. Script files run when the E3D window loads. (Script files are discussed on page 143 and DataBlocks are discussed on page 144.)

The script for the Wall.cs file is discussed in “Example using Wall object” on page 145.

- The Wall folder which contains the Wall.dts file and the skins of the Wall object. As discussed in “Skins” on page 135, an object’s default skinBaseName1 will be the name of the DTS file. In this example, because the Wall object is called wall.dts, the default skinBaseName1 is base.wall.jpg.

☞ All .cs files must be placed at the top level of the E3DObjects folder within the Extensions folder, since that is the only place ExtendSim knows to look for them.

Adding a DTS file to ExtendSim

Place new script and DTS files within the Extensions\E3DObjects folder, with the script (.cs) files at the top level. As noted above, ExtendSim looks for any .cs files only at the top level of the E3DObjects folder. However, since the .cs file contains information about where the DTS file is, the location of the DTS file is flexible.

☞ If you place a DTS file into the Extensions folder, but do not include a script file containing a DataBlock, you will not be able to place the 3D object into the E3D window.

Exporting a 3D object file created in another program

Object files that have been created for other environments can be used in ExtendSim, but they must first be exported into the DTS format. This is accomplished using DTS exporters.

☞ This discussion is about DTS objects. DIF objects, for buildings and interiors, are discussed on page 149.

DTS exporters are separate pieces of software that are available for several of the popular 3D modeling packages (3DS Max, MilkShape, Maya, etc.). In most cases these modeling applications do not ship with an installed DTS exporter. You must obtain the appropriate DTS exporter from Garage Games and add it to the modeling software package.

The procedure involves opening the object in the 3D modeling application and selecting an export command. This varies depending on which application your files are in.

Exporting a basic shape that doesn't have any custom behavior can be quite straightforward. Objects with more complex behavior, such as ambient animation or custom mountpoints, are more involved. Most of these complex behaviors are defined in different ways for different file formats, so most likely it will require some custom modifications to the 3D object.

Check for more information on the Garage Games website about which products have exporters available and how to install and work with those exporters.

Scripting and .cs files

The TGE includes a sophisticated scripting environment called Torque Script. This environment allows extensive control over what happens in the E3D window, and which objects, sounds, textures and other components are available. The most common use for script files is to define DataBlocks for new 3D objects. You can also add whatever Torque script code you wish.

How to use the scripting environment, the syntax of the scripting language, and even the structure and behavior of the default scripting files is beyond the scope of this documentation. Instead, there

is extensive information about the scripting environment, as well as the default layout and behavior of TGE scripts, on the GarageGames website (www.GarageGames.com).

Objects and so forth

Script (.cs) files placed in the Extensions\E3DObjects folder are custom scripts that get executed when the E3D window is launched. The simplest use for this capability is to just provide DataBlocks for new E3D objects that are to be used in the E3D window. However, there is no reason why you cannot do anything that can be done with TorqueScript in a file placed in this location.

 If you are adding multiple 3D objects it is not required that each object have a separate script file. Instead, you can define multiple DataBlocks in a single script file.

E3D window

The script files that control the behavior of the E3D window are in the Extend3D, Common, and Creator folders within the ExtendSim folder. Much of the content of these files is the same as for default TGE projects but with many customizations and custom features.

Unless you are doing some advanced custom 3D work you will not need to either modify or even look at the contents of the Extend3D, Common, and Creator folders.

Torque script files

Torque script files are text files with a .cs extension that get compiled by the ExtendSim application. The compiled version of a Torque file is called a .dso file. These files will not be placed in the Extensions folder since the script files that are placed in Extensions will be executed directly, as opposed to being compiled. The .dso files are instead located in the Extend3D, Common, and Creator folders.

PathNames in TorqueScript files

Pathnames can have certain special characters in them. There are several standard special characters and ExtendSim defines two new ones.

- The standard characters are “.” and “~”.
 - The “.” character specifies that the path starts from the directory location where the Torque script is executing (the current codeblock\mod path in the Torque environment).
 - The “~” character means the path starts from the root of the Extend3D folder in the ExtendSim folder (extends from the root of the codeblock/mod path in the Torque environment).
- The special characters defined by ExtendSim are “^” and “&”
 - The “^” character refers to a path that starts from the application folder. For example, ^/Extensions refers to the root of the Extensions folder.
 - The “&” character refers to the folder containing the current model.

DataBlocks

A *DataBlock* is an object definition – the code definition of the initial values of properties of a 3D object as well as a definition of the type/class of the object. You could think of a DataBlock as being like the definition of an ExtendSim block in a library. A DataBlock defines the common aspects and behavior of the 3D object, but each instance of the object will have its own individual parameters associated with it.

The file ExItems contains many of the DataBlocks that have been defined for ExtendSim objects; it is located at ExtendSim7\Extend3D\server\scripts\ExItems.cs.

If you are not doing any extensive 3D scripting or adding custom 3D objects into the ExtendSim Extensions folder, you do not need to worry about DataBlocks.

- ☞ If you are adding multiple 3D objects it is not required that each object have a separate script file. Instead, you can define multiple DataBlocks in a single script file.

Types of DataBlocks

When a DataBlock is declared, its type is also declared. While you can define your own types, some typical types of DataBlocks that ExtendSim uses include:

- StaticShapeData
- WheeledVehicleData
- ExtendItemData
- PlayerData
- MissionMarkerData

This definition of the type of the datablock is important, since it will determine what kind of object ExtendSim creates in the 3D window. For simple cases of placing new objects you will most likely use ExtendItemData or StaticShapeData. The deciding factor is if the item is going to be an immobile object (StaticShapeData) or a block-to-block object (ExtendItemData).

DataBlock variables

There are many parameters that can be defined in a DataBlock. DataBlock variables are specific to the type of object that is being defined and can, in fact, be defined dynamically in the DataBlock itself.

- ☞ Defining a new variable in a DataBlock will not have an effect on the object unless the variable is referenced in the script or source code of the object.

Some typical DataBlock variables used by ExtendSim are:

- *Category* defines groupings that are used both in the Mission Editor and in certain ModL functions to separate objects into convenient sets. The most common categories that you would want to use are ExtendItem, Scenery, and Block.
- *ShapeFile* defines the location of the DTS file. For an example, see the ShapeFile of the Wall block “Example using Wall object” on page 145.
- *Scale* defines the default scale of the 3D object. See also “Scale and position” on page 134.

For a list of DataBlock variables that have been specifically defined for ExtendSim or which have been modified from how they are used by the Torque engine, see page 146.

ExtendSim-specific DataBlock additions

ExtendSim defines some custom DataBlock variables which can be set to customize the behavior of objects. These are extensions to the 3D functionality defined in the ExtendSim application.

As shown on page 138, sounds and ambient animation are examples of additional behaviors that are enabled through DataBlocks. See also the list of customized DataBlocks on page 146.

Example using Wall object

The script file for the Wall object mentioned earlier only defines one thing – a DataBlock for the Wall object. The code is:

```
// datablocks
// JSL - Scenery

datablock StaticShapeData(Wall)
{
// Mission editor category, this datablock will show up in the
// specified category under the "Scenery" root category.
category = "Scenery";

// Basic Item properties
shapeFile = "./Wall/Wall.dts";
scale = "1.0 1.0 1.0";
};
```

This script defines a single DataBlock that specifies for the 3D engine some aspects of the behavior of the Wall object and also tells the 3D engine that the Wall object exists.

The first line (datablock StaticShapeData(wall)) is the declaration which has three components:

- The keyword “datablock” defines what follows to be a datablock
- StaticShapeData defines the type of datablock
- Finally, the name of the datablock (Wall) is in parentheses. This is the name that will be used in ExtendSim dialog boxes and script code.

The remaining lines of the DataBlock script define variables as discussed in “DataBlock variables” on page 145. For a simple object like the Wall, defining just three variables is sufficient:

- Category. For the Wall object, the Category is Scenery.
- ShapeFile. The path “./Wall/Wall.dts” indicates that the Wall.dts file is located within a folder named “Wall”. The period (“.”) in the path name signifies that the path starts at the current location; in this case in the Extensions\3DObjects folder where the .cs (script) file is located.
- Scale. As discussed in “Scale and position” on page 134, “1.0 1.0 1.0” specifies that the object is scaled to its standard size in each of the three dimensions.

DataBlock variables

As discussed in “DataBlocks” on page 144, there are many variables supported in DataBlocks. The following tables list a subset of the available variables – those that have been custom developed for ExtendSim or which have custom behaviors compared to the standard Torque DataBlocks.

Custom datablock variables

This table describes variables that have been defined and given custom behaviors for ExtendSim.

Variable	Description
HasFront	This DataBlock flag specifies that the object has a front. When the item is specified as moving in a direction, the object will turn its front in the direction it is facing. If this flag is not defined, it defaults to false.
IdleSound[0-4]	As described in “Sound” on page 138, this flag specifies one of up to five sounds that can be associated with the item idling or with certain ambient animation states.

Variable	Description
IdleSoundAnimation[0-4]	As described in “Sound” on page 138, specifies an animation state to associate with a sound. See above for more information.
IdleSoundVol[0-4]	Specifies the volume level for the idleSound. Volumes range from 0.0 to 1.0.
MountLook[n]	Part of the ambient animation functionality discussed in “People objects” on page 141. The mountLook flag specifies what look a person is supposed to use when this kind of object is mounted on them. The mount functionality is part of the CarriableItem functionality described on page 148.
SkinBaseName1	Defines the baseName for the first skin. An example would be the Male object, where the SkinBaseName1 is maleClothes.
SkinBaseName2	Defines the baseName for the second skin. An example would be the Female object, where the SkinBaseName2 is femaleSkin.
SkinName1	Specifies what the default skin to be used for the object is. Objects with multiple skins will obey the base.shapename naming convention, so this variable tells the object what skin to use if you don’t want the object to start with the base skin.
SkinName2	This DataBlock variable specifies what the default skin for the second skin should be. This is only relevant for objects with multiple skin types. Currently only the people objects have more than one skin type.
StackMountPoint	The mountPoint that will be used for the mountStack functions. This is defined as 1 for the people objects so that mountStacking on them will stack on their arms. However, if people objects stack on each other, they stack on mountPoint 0.
UnmountLook[n]	Part of the ambient animation functionality described on page 140. The UnmountLook flag specifies what look a person is supposed to revert to when this kind of object is unmounted from them. The unmounting functionality is part of the CarriableItem functionality described on page 148.

Modified datablock variables

Some Torque standard datablock variables have been given additional possible values.

Variable	Description
ClassName	Certain items are defined in their DataBlocks as being CarriableItems; see “Carriable items” on page 148.
Category (Block, Extend-Item, Scenery, etc.):	Categories are used in ExtendSim for several of the ModL functions that get information about objects. The functions use categories to distinguish between objects.

Carriable items

Mounting and unmounting objects was discussed on page 137 and page 138, respectively. Certain types of objects in ExtendSim are carriable items which can be carried by a person. This is defined in the DataBlock by setting the ClassName variable to CarriableItem.

A carriable item call causes the look of the person to change when an object is mounted on them. For example, when you mount a Box onto a person, they will move their arms to the 'lookbh' position. (The bh stands for box hold, as discussed on page 148.)

If an object that you expect to be carried by a person is not defined as carriable, people objects will not change their look animation when mounted with the object and the look and feel of the animation will suffer.

Example script

```
datablock ExtendItemData(Box)
{
// Mission editor category
category = "ExtendItem";

// Basic Item properties
shapeFile = "~/data/shapes/box/box.dts";
mass = 1;
elasticity = 0.2;
friction = 0.0;
mountPoint = 0;
emap = false;

// JSL - mountLook - look to use when mounted.
// (For Extend items, they tell the holder to change his/its look.)
className = "carriableItem"; // class
mountLook = lookbh;
unmountLook = look;
};
```

The mounting code will check to see if the object being mounted has a ClassName of CarriableItem and if the object being mounted onto is a person. If both of these are true, it will use the mountLook defined in the DataBlock as the new look for the person object. The unmountLook will be used as the look for the person when the object is unmounted.

Looks

The various looks supported for the carriableItems are:

- Lookbh. This is the “box hold” look associated with carrying a box. The person will spread their arms low and outside the sides of the box.
- Lookph. The “pallet hold” look is associated with the pallet object. This look is similar to the box hold, with a small variation in the position of the hands and arms.
- SuitcaseHold. This hold is for holding a suitcase, single-handed on the person’s right-hand side. This look can be seen in the Airline Security model (Examples\3D Animation).
- SmallHold: This is for holding a small object like a letter or a stack of paper where both hands are brought together in front of the person. This look can be seen in the Bank line models (Examples\Tutorials\E3D Animation).

Buildings and interiors

The 3D objects supplied with ExtendSim are primarily DTS objects. The people, items, vehicles and static shapes are all saved and shipped in the DTS format. If you want to add a new object to ExtendSim and if it is fairly small and self-contained, then you probably want to use the DTS format process described “Defining new 3D objects” on page 142.

If, on the other hand, you want to include in your model a larger-scale object (such as a building or anything else that is enterable), you should not use the DTS format as it is not designed for this use.

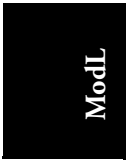
The DIF format is designed for buildings and objects on large scale. As with the DTS format, various editors (such as Quark and Cartography Shop) can export DIF files. TGE owners also get a DIF editor, called Creator, that is specifically designed to develop DIF files.

The creating and exporting process for DIF files can be involved, will be different for each editor, and is beyond the scope of this manual. Information for how to do this can be found on the Garage Games website.

Terrain

ExtendSim's default environment file does not make much use of the terrain capabilities built into the Torque engine. In fact, the blocks in the ExtendSim libraries assume that the terrain will be flat and will be at a height of 100 units. So if you create models using the standard ExtendSim environment and use the blocks in the Item library, you should not change from the flat terrain defined in the default environment file. Changing textures of the terrain in the environment file does not affect the behavior of the ExtendSim blocks.

For certain kinds of custom models, however, the ability to manipulate the terrain in the E3D window might be quite useful. There are several things that can be changed and several mechanisms for how you change them. See the User Guide for information about editing Terrain and changing Terrain Textures.



ModL

Simulation Architecture

Keep this information in mind as you
create and modify ExtendSim blocks

*“In baiting a mousetrap with cheese,
always leave room for the mouse.”
— Hector Hugh Munro*

This chapter describes how the ExtendSim simulation engine works and how discrete event and discrete rate blocks pass messages and resolve logic issues. This is important information when you are creating or modifying blocks, since you need to know:

- How the block will work with the ExtendSim simulation engine
- How the block will work with the other blocks that ship with ExtendSim

Running a simulation

It is useful to understand the steps that ExtendSim takes when it runs a simulation so you can get a feeling for what parts of the process are relevant to models. The following sections give a step-by-step breakdown of how ExtendSim runs simulations and how it decides what to do next.

 Running continuous simulations starts on this page; running discrete event and discrete rate simulations is discussed starting on page 156.

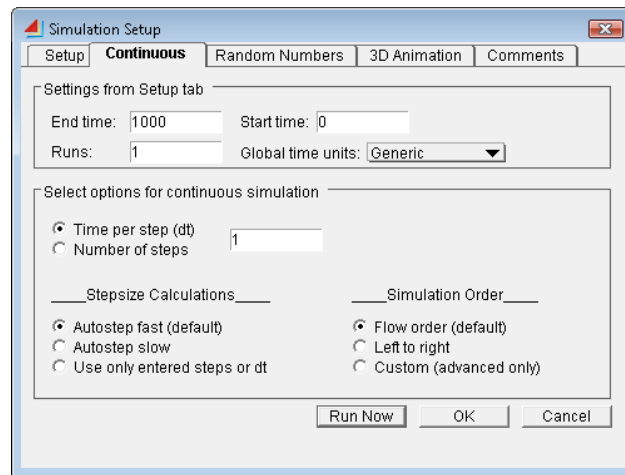
ModL

How ExtendSim runs continuous simulations

ExtendSim keeps many system variables handy so that it can compute how to run the model.

Five of the options in the Continuous tab of the Run > Simulation Setup dialog become variables that are used to determine how the model is run:

Option	Variable
End time	EndTime
Start time	StartTime
Runs	NumSims
Time per step (dt)	DeltaTime
Number of steps	NumSteps



Continuous tab of Simulation Setup dialog

Before anything else happens in the simulation run, these variables are used to calculate an initial value for the DeltaTime or NumSteps variables. The variable that is calculated depends on what is selected in the Simulation Setup dialog: if *Number of steps* is selected, ExtendSim calculates the related variable DeltaTime; if *Time per step* is selected, ExtendSim calculates NumSteps. The formulas used are:

```
DeltaTime = (EndTime-StartTime)/(NumSteps-1);
NumSteps = Floor (((EndTime - StartTime) / DeltaTime) + 1.5)
```

This initial value is the value that the blocks see during the PreCheckData, CheckData, and Step-Size simulation messages.

Pseudocode of the simulation loop for continuous simulations

The following is a *pseudocode* description of ExtendSim's continuous simulation loop. Pseudocode is used by developers to describe the controlling logic of a program in a form that is a cross between English and a programming language.

```

Calculate simulation order of blocks;

for CurrentSim = 0 to NumSims-1
{
  if (continuing saved paused run) // this was saved while paused
  {
    CurrentTime = SavedCurrentTime;
    CurrentStep = SavedCurrentStep;
    Send ContinueSim message to blocks
  }
  else
  {
    CurrentTime = StartTime;
    CurrentStep = 0;
    Send PreCheckDataMsg to blocks; // Prepare for validation
    Send CheckDataMsg to blocks; // Validate block variables
    if (CheckDataMsg aborts)
      Abort Simulation and select bad block;
    Send StepSize to blocks; // Continuous models can change
                          // stepsize
    Send InitSimMsg to blocks; // Initialize block variables
    Send PostInitSimMsg to blocks; // Check initializations
  }

  for CurrentStep = 0 to (NumSteps-1) // In discrete event
  {
    // models, simulation time
    Send SimulateMsg to blocks; // is controlled by the Executive
                          // block, not by this loop.

    if (Abort was encountered)
    {
      Send AbortSim to blocks;
      Jump out of loop to ExecuteEndSims;
    }

    CurrentTime = CurrentTime + DeltaTime;
  }

  Send FinalCalc to blocks; // Performs final stat calculations
  Send BlockReport to reporting blocks; // Report results
  ExecuteEndSims:
  Send EndSimMsg to blocks; // Clean up variables, dispose arrays
}

```

 The above comments indicate one purpose of the specific message handler, but each of them can be used for a variety of purposes, as shown in the table on page 154.

The next several sections discuss the meaning of this pseudocode.

Simulation order

Simulation order defines the sequence in which messages are sent from the ExtendSim application to the blocks in a model. The simulation order section of the pseudocode calculates the simulation order, either flow or left-to-right, as discussed in the User Guide.

Initialization

The pseudocode:

for CurrentSim = 0 to NumSims-1

instructs ExtendSim to execute the contents of the “for” loop NumSims times. You set NumSims in the *Runs* entry box in the Continuous tab of the Simulation Setup dialog. The value of the variable CurrentSim varies from 0 to NumSims-1. This is the logic that defines the number of simulations that will be run. If an Abort statement is executed during a simulation, it will halt the current simulation, increment NumSims by 1, and run any remaining simulations.

When messages are sent

Different messages are sent, depending on whether the run is paused or is a new run:

- If the simulation was previously paused and saved before the run was completed, ExtendSim will send a ContinueSim message to all of the blocks in the model. Since the model was already initialized and only needs to set up its data structures to continue the previous run, the ContinueSim message is sent in lieu of the PreCheckData, CheckData, StepSize, InitSim, and PostInitSim messages that would follow if it were a new run, described below.
- If the run is a new run, the first thing done inside this loop is to send the PreCheckData and CheckData system message to all the blocks in the simulation. These message handlers in the various blocks should check the values of dialog items to see if they are valid. If any of them abort, ExtendSim recognizes that fact and the simulation aborts with the block selected. Connection status of the block can be tested during CheckData: Input connectors that are connected will show a non-zero value, and unconnected input connectors will show a zero value. This is useful in determining if a block’s inputs are actually connected in the model.

Whether it is a new run or a paused run, the StepSize message is then sent to all blocks. In continuous models, StepSize is used to manipulate the DeltaTime variable. The smallest value of DeltaTime specified by any block is found and DeltaTime is set to that value. Note that this DeltaTime overrides the value initially calculated. If AutoStep Slow was selected, the returned value of DeltaTime is then divided by 5 for more accurate simulation results. The Filter blocks (Electronics library) use the StepSize message in this way because they need a calculated value of DeltaTime to get accurate results.

Finally, a new value of NumSteps is determined based on DeltaTime, StartTime, and EndTime.

The formula is:

NumSteps = Floor(((EndTime - StartTime) / DeltaTime) + 1.5)

This value is used to govern the actual simulation loop.

- ☞ Because DeltaTime is used to calculate NumSteps and a rounded value is used for the number of steps, it is possible for the EndTime of the simulation to be exceeded slightly.

The InitSim message is then sent to all blocks. This message handler is used to initialize variables within each block. After the InitSim message has been sent to all blocks, the PostInitSim message is sent and gives the developer a chance to initialize any variables that needed all of the other blocks to be initialized first.

Table of message handler purposes

The following table lists the major purposes of the messages sent during the initialization loop for Value library blocks. Notice that some purposes only apply if the Value library block is used in a discrete event, rather than a continuous, model.

- ☞ While the table lists the major purposes of the message handlers, a purpose can sometimes be accomplished using a different message handler than the one indicated.

Message Handler	Purpose in Value Library Blocks
CheckData	Check validity of parameters Assign position in future events calendar (discrete event models) Check to see which connectors are connected Determine parameter links with database Check for duplicate seeds Determine if dialog variables are cloned
StepSize	Throw and Catch Value blocks create data structures
InitSim	Determine if model is discrete event or continuous Turn off simulate messages in discrete event models Schedule events in discrete event models Initialize variables Calculate local block seed value
PostInitSim	Initialize connector values Schedule events in discrete event models

 For the Data Import Export and Command blocks, initialization message handlers can be selected using options in the blocks' dialogs.

Step loop

The inner loop:

for CurrentStep = 0 to (NumSteps-1)

is where Simulate messages are sent to all of the blocks in the order determined by the connections. In continuous simulations the loop is governed by CurrentStep and NumSteps only. NumSteps is the variable that is used to end the simulation, and the actual number of Simulate messages (steps) that occurs is NumSteps. That is, if 2 is entered for the **Number of steps** option in the Continuous tab of the Simulation Setup dialog, NumSteps becomes 2, the simulation step loop sends Simulate messages to the blocks for CurrentStep equal to 0, and then 1. (If you want to run a simulation for only a single step, set **Number of steps** to 1.)

CurrentTime is incremented by DeltaTime for each step of the loop, but otherwise has no effect on this loop. This means that you can change CurrentTime and DeltaTime without affecting the loop. It also means that you can change CurrentStep or NumSteps to keep the loop running longer or stop it prematurely.

 You can set CurrentTime in any block to any value, although you might want to have some logic in the ModL code to prevent conflicts between blocks setting CurrentTime after other blocks have already changed it. Use the ExtendSim global variables to help establish some safeguards against these conflicts. For instance, this is how the Executive block (Item library) is able to control the progression of time in discrete event models.

Final messages

Upon completion of the simulation, ExtendSim sends out the FinalCalc system message to all blocks. This message handler is used to perform final calculations for all time-dependent statistics.

ExtendSim then sends the BlockReport system message to any blocks that have been selected for a report. This message handler is responsible for writing all report data to the report text file.

Finally, ExtendSim sends the EndSim system message to all blocks. The EndSim message handler is used for any “clean up” activities such as disposing of any dynamic arrays to free up memory.

Aborting multiple simulations

If the Abort statement is executed during the simulation, it only stops the current simulation. If you want to abort all simulations (for example, when you have specified in the Simulation Setup dialog that more than one simulation should be run), use the AbortAllSims() function instead of the Abort statement.

How ExtendSim runs discrete event or discrete rate simulations

ExtendSim keeps many system variables handy so that it can compute how to run the model. Three of the options in the Setup tab of the Run > Simulation Setup dialog become variables that are used to determine how a discrete event or discrete rate model is run:

Option	Variable
End time	EndTime
Start time	StartTime
Runs	NumSims

For discrete event and discrete rate models, the DeltaTime and NumSteps variables (used in continuous simulations and described on page 152) are ignored, because the Executive block (Item library) calculates CurrentTime based on the time of the next event.

Pseudocode of the simulation loop in discrete event/rate simulations

The following is a *pseudocode* description of ExtendSim’s discrete event/discrete rate simulation loop:

```

for CurrentSim = 0 to NumSims-1
{
  if (continuing saved paused run) // this was saved while paused
  {
    CurrentTime = SavedCurrentTime;
    CurrentStep = SavedCurrentStep;
    Send ContinueSim message to blocks
  }
  else
  {
    CurrentTime = StartTime;
    CurrentStep = 0;
    Send PreCheckDataMsg to blocks; // Prepare for validation
    Send CheckDataMsg to blocks; // Validate block variables
    if (CheckDataMsg aborts)
      Abort Simulation and select bad block;
  }
}

```

```

    Send StepSize to blocks;//Continuous models can change
                                // stepsize
    Send InitSimMsg to blocks;// Initialize block variables
    Send PostInitSimMsg to blocks;// Check initializations
}

// simulation Phase is controlled by the Executive block
// Do until simulation end reached
Do While (currentTime < EndTime or number of events)
{
    Determine the block(s) with the next event time
    Set currentTime to the next event time
    Send message to every block whose nextTime == CurrentTime
    Send message to every block that has posted a 0 time event
    (rescheduled)
}

Send FinalCalc to all blocks;
Send BlockReport to all reporting blocks;
ExecuteEndSims:
Send EndSimMsg to all blocks;
}

```

 The above comments indicate one purpose of the specific message handler, but each of them can be used for a variety of purposes, as shown in the table on page 158.

The next several sections discuss the meaning of this pseudocode.

Initialization

The pseudocode:

```
for CurrentSim = 0 to NumSims-1
```

instructs ExtendSim to execute the contents of the “for” loop NumSims times. You set NumSims in the **Runs** field of either the Setup or Continuous tabs of the Simulation Setup dialog. The value of the variable CurrentSim varies from 0 to NumSims-1. This is the logic that defines the number of simulations that will be run. If an “abort” statement is executed during a simulation, the abort will halt the current simulation, increment NumSims by 1, and run any remaining simulations.

When messages are sent

Different messages are sent, depending on whether the run is paused run or is a new run:

- If the simulation run was previously paused and saved before the run was completed, ExtendSim will send a ContinueSim message to all of the blocks in the model. This message is sent in lieu of the following PreCheckData, CheckData, StepSize, InitSim, and PostInitSim messages as the model was already initialized and only needs to set up its data structures to continue the previous run.
- If the run is a new run, the first thing done inside this loop is to send the PreCheckData and CheckData system message to all the blocks in the simulation. These message handlers in the various blocks should check the values of dialog items to see if they are valid. If any of them abort, ExtendSim recognizes that fact and the simulation aborts with the block selected. Connection status of the block can be tested during CheckData: Input connectors that are connected will show a non-zero value, and unconnected input connectors will show a zero value. This is useful in determining if a block’s inputs are actually connected in the model.

In either case, the StepSize message is then sent to all blocks. In discrete event models, this message is used to set up the attribute global arrays (see “Attribute global arrays” on page 166).

The InitSim message is then sent to all blocks. This message handler is used to initialize variables within each block. After the InitSim message has been sent to all blocks, the PostInitSim message is sent and gives the developer a chance to initialize any variables that needed all of the other blocks to be initialized first.

Table of message handler purposes

The following table lists the major purposes of the messages sent during the initialization loop for Item and Rate library blocks, as well as for the Executive block in discrete event and in discrete rate models.

 While the following tables list the major purposes of the message handlers, a purpose can sometimes be accomplished using a different message handler than the one indicated.

Message Handler	Purpose in Item Library Blocks
PreCheckData	Initialize Resource Pools
CheckData	Check validity of parameters Assign position in future events calendar Check to see which connectors are connected Determine parameter links with database Check for duplicate seeds Determine if dialog variables are cloned Check the version of the Executive Get Executive block number Determine if costing is used in the model Initialize DB Equation variables Check Animation Register blocks with ExtendSim database
StepSize	Initialize attribute data structures Initialize Animation Initialize Shifts
InitSim	Initialize variables Initialize attribute animation conversion table Schedule initial events Set local block seed value
PostInitSim	Initialize connector values Check if a downstream block controls the flow of items (blocker) Schedule initial events
Message Handler	Purpose in Rate Library Blocks
CheckData	Update the indexes of the global arrays used for the Rate library system Update information about the flow and value connections If the block starts a section, create the section in the appropriate global array Determine if the type of Shift is On or Off Assign position in future events calendar Check parameters in the block

Message Handler	Purpose in Rate Library Blocks
StepSize	Initialize animation Update the unit conversion factors (flow unit and time unit) Initialize Shifts Initialize attribute data structures in Interchange block
InitSim	Initialize variables (static variables, dialog items) Initialize next event Initialize value connectors Initialize attribute animation conversion table in Interchange block
Message Handler	Purpose in Executive Block
PreCheckData	Dispose Throw Flow and Catch Flow block information (discrete rate models)
CheckData	Initialize system global variables Post Executive block number Send CheckData message through event message connector Check for duplicate random number seeds Create the global arrays used for the Rate system (discrete rate models) Initialize the global variables (discrete rate models) Rebuild and update Throw Flow and Throw Catch info (discrete rate models)
StepSize	Initialize attribute data structures From the beginning, propagate through each section (discrete rate models) Create the Unit Groups in the model (discrete rate models) Create the Lookup List for flow units and time units (discrete rate models) Create and update global arrays linked to the sections (discrete rate models)
InitSim	Initialize variables Pass event arrays to other blocks
PostInitSim	Launch and perform calculation of all effective rates (discrete rate models)

Simulation phase

The Item and Rate libraries rely on the Executive block rather than ExtendSim to control Current-Time and NumSteps. As opposed to continuous simulations, which rely on time being incremented uniformly between each step, discrete event and discrete rate simulations are event based. In discrete event and discrete rate simulations, NumSteps is modified constantly and the Executive block monitors and manipulates CurrentTime based on the events the blocks have posted in the Executive's event queue.

At each simulation step, the Executive searches through a list of event-scheduling blocks and finds the nearest future event time. The Executive then sends a message to each of the event-scheduling blocks whose event time is equal to the nearest event time. This may cause other blocks to post zero time events (rescheduling themselves). The Executive sends a message to each one of the blocks on the zero time event list (see "Timing for discrete event models" on page 160 for a detailed discussion).

Final messages

Upon completion of the simulation, ExtendSim sends out the FinalCalc system message to all blocks. This message handler is used to perform final calculations for all time-dependent statistics.

ExtendSim then sends the BlockReport system message to any blocks that have been selected for a report. This message handler is responsible for writing all report data to the report text file.

Finally, ExtendSim sends the EndSim system message to all blocks. The EndSim message handler is used for any “clean up” activities such as disposing of any dynamic arrays to free up memory.

Aborting multiple simulations

If the Abort statement is executed during the simulation, it only aborts the current simulation. If you want to abort all simulations (for example, when you have specified in the Simulation Setup dialog that more than one simulation should be run), use the AbortAllSims() function instead of the Abort statement.

How discrete event blocks and models work

Discrete event blocks use some fairly complex code. The following explanation should be useful to anyone trying to program their own discrete event blocks or alter the blocks that come in the Item library.

 Always put copies of blocks from the Item library into your own library and alter the copies rather than the originals.

The Make Your Own category of blocks in the Example Libraries > ModL Tips library have sample code and comments that explain how the blocks work. You can use these blocks as templates for your own discrete event blocks.

The following sections discuss the messaging system used to schedule events, transfer items through the model, and resolve logic issues. To understand the details of the different types of messages, some understanding of the simulation engine, the underlying data structures, and event handling is necessary.

Timing for discrete event models

To make a model discrete event, add the Executive block (Item library) to the model worksheet. This block does two things:

- 1) It maintains a data structure of information about the items in the model
- 2) It takes control of the time clock from the ExtendSim application, scheduling events, sending messages to the blocks that scheduled the event, and moving the clock forward to the appropriate time for the next event.

In order to provide true discrete event operation, the simulation clock must move to the exact time of each event. In order to do this, the Executive block uses three dynamic arrays to store future events in the model.

- The TimeArray contains the event times
- TimeBlocks contains the block numbers of the blocks that post events
- TimeEventMsgType stores the constant for the message handler that is called when the event time for a block is reached

The Executive block passes the arrays to the system globals with the statements:

```
SysGlobal0 = passArray(TimeArray);
SysGlobal7 = passArray(TimeBlocks);
SysGlobal13 = passArray(TimeEventMsgType);
```

 An important consequence of the Executive block controlling the time clock in the model is that the variable numSteps, which in a continuous model represents the number of simulation steps that will occur in the total run, is updated only by the Executive block and will always have a value of one greater than the number of simulation steps that have actually occurred.

Scheduling events

Each block that needs to post an event at a specific time in the future needs to add itself to the Executive block's TimeArray, TimeBlocks, and TimeEventMsgType arrays. This is done by reserving a position in the arrays and then setting that position in TimeArray to the next event time (this needs to be done each time a new event time is posted) and TimeBlocks to the block number of the event scheduling block.

The position in the arrays is reserved in the CheckData message handler with the following code:

```
on checkdata
{
  // SysGlobalInt0 is current number of event posting blocks in model
  // Reserves a position in TimeArray & TimeBlocks arrays
  myIndex = SysGlobalInt0;
  SysGlobalInt0 += 1;
}
```

SysGlobalInt0 has been initialized to 0. By incrementing SysGlobalInt0, each block will get a unique value for myIndex.

In the InitSim message handler, the two arrays are passed in from the Executive, the block number is assigned to this block's position in TimeBlocks, and the initial event time is assigned to TimeArray:

```
on initsim
{
  // get the pointer to the TimeArray and TimeEventMsgType arrays
  if(getPassedArray(SysGlobal0, timeArray) > 0)
  {
    // blocks in de models do not get Simulate messages
    GetSimulateMsgs(False);
    // set the first event time to the start of the simulation
    timeArray[MyIndex] = StartTime;
    // get the pointer to the TimeBlocks array
    getPassedArray(SysGlobal7, TimeBlocks);
    // put this block's # in reserved position in TimeBlocks
    TimeBlocks[myindex] = myBlockNumber();
    //Get the pointer to the TimeEventMsgType array
    getPassedArray(SysGlobal13, TimeEventMsgType);
    //reserved position in TimeEventMsgType
    TimeEventMsgType[myIndex] = BlockReceive1Msg;
```

```

    }
    else
        GetSimulateMsgs(True);
}

```

If this block is used in a continuous model, the GetPassedArray call will return 0 and the on Simulate message handler will be called at every simulation step.

When the Executive sends this block a message (at the time specified in TimeArray), the block will receive the message in TimeEventMsgType. If you do not set a value in TimeEventMsgType, the block will receive a BlockReceive0 message.

Put the event code in this message handler and reschedule the block for the next event time:

```

On BlockReceive1
{
    // process event
    ShowTime = CurrentTime;
    // schedule event in the future
    TimeArray[MyIndex] = CurrentTime + EventTime;
}

```

The Event block (ModL Tips library) illustrates the event scheduling procedure.

Residence, passing, and decision blocks

There are three types of item-handling discrete event blocks: *residence blocks*, *passing blocks*, and *decision blocks*:

- 1) Residence blocks are able to contain or hold items for some duration of simulation time. Some residence blocks post events and some do not. Examples of residence blocks are the Queue and Activity blocks.
- 2) Passing blocks pass items through without holding them. These blocks implement modeling operations that are not time based and usually do not post future events. Examples include setting an attribute or getting information from an item (Set or Information blocks). Passing blocks may use the CurrentEvents array to reschedule themselves. In this case they will receive a BlockReceive0 message. Rescheduling is necessary when the passing block needs to return from a message. However, before the simulation clock advances, it also needs to perform some additional processing. In this case a message is sent to the Executive and the block number is posted on the CurrentEvents array. The Executive sends a BlockReceive0 message to every block entered in the CurrentEvents array.
- 3) Decision blocks control the flow of items in the model; they do not post future events. Examples include limiting the number of items (Gate block) or selecting an output (Select Item Out blocks). Note that decision blocks can behave as passing or residence blocks, depending on which options are selected for the particular block.

Blocks that post future events

Each block that posts events places its block number in a slot in the global TimeBlocks during the initSim message handler. The slot number is an index number assigned in the checkData message handler, called "myIndex". Each block also places the time of its next event in a slot in the global TimeArray. For example:


```
TimeArray[myIndex] = nexttime;
```

At the start of a simulation event, TimeArray (the event list) is searched to find the next event time. A third dynamic array, NextTimes, is used to store all of the block numbers that have posted an event at the next event time. The simulation clock is then advanced to the next event time and a message is sent to each block in the NextTimes array, one block at a time.

After receiving its event message, each block processes its own unique code based on that event type. For example, a Convey Item block will attempt to pull in an additional item when the event for an open input occurs, or the Activity block will attempt to push an item out when that item's processing duration has completed. Some blocks will conditionally post an event based on the options selected. An example of this is the Queue, which will post an event if reneging is selected. Blocks do not have to process items for an event to occur – the Clear Statistics block (Value library) generates an event at the clear time when used in a discrete event model.

Residence blocks that do not post future events:

Some of the residence blocks that do not post future events, such as the Queue Matching and Resource Item blocks, also attempt to move items at event times.

If a residence block needs to return from a message but also needs to attempt to pull in or push out items with the same time step, it will send a message to the Executive posting itself on the CurrentEvents array. The Executive sends a BlockReceive0 to all of the blocks listed in the CurrentEvents array. This ensures that the residence block will receive an additional message before the next time step so that additional items can be processed.

Zero time events

Sometimes an event needs to be posted that will occur at the same simulation time. An example of this is an Activity block that has just released its item after the specified time delay. The Activity will need to do two things at the same event time: send the item out to the next block (if that path is not blocked) and pull another item in (if an item is available). Since the Activity can only do one operation at a time, it sends the item to the next block and then posts a “current” event to the Executive. Before the simulation clock advances, the Activity will receive a BlockReceive0 message and will then try to pull another item in.

To post a zero time event, the block assigns SysGlobalInt8 to its block number and sends a BlockReceive3 message to the Executive. The Executive maintains a list of all the blocks that have sent a BlockReceive3 message and before it advances the simulation clock sends a BlockReceive0 message to each of these blocks in turn. In the example below, the “rescheduled” flag is used to prevent the block from posting more than one zero time event at a time:

```
if ( ! rescheduled)
{
    // remember block is scheduled as a zero time event
    rescheduled = TRUE;
    // set the block number for this block
    SysGlobalInt8 = MyBlockNumber();
    // send message to Executive for a zero time event
    SendMsgToBlock(Exec, BLOCKRECEIVE3MSG);
}
```

Item data structures and indexes

The Executive block stores information about each item in a discrete event model in a set of dynamic arrays that it passes into the reserved global variables SysGlobal3, SysGlobal4, SysGlobal6, SysGlobal9, and SysGlobal12 (see “Global variables” on page 201), as well as a set of two global arrays (see “Global arrays” on page 355). The information in all of the arrays is available to every block that needs to access it.

Each item in the model is identified by a unique number that is the index number used to look up the item’s information in the dynamic arrays. This is the item’s *index*. The arrays contain information about the items, such as values, priorities, attribute information, and so on. When an item is passed through an item connector, it is really the index of the item that is passed. Items are indexed from 1 to n-1, where n is the number of slots in the array. Index 0 is not used as an item index, since a connector value of 0 indicates that no item is present.

Passing the index numbers through item connectors allows the blocks to pass a great deal of information with a single number. It also means that all blocks in the model can access any item. Each of the global variables is received into a local array within each block and, after being received, is available in exactly the same way that data in any local array is available.

For example, if there are currently ten items in a simulation, the index number 5 might be passed from one block in the model to the next. When a block receives index number 5, it accesses the information in the Executive block’s arrays for item number 5. For instance, `itemArrayC[5][0]` would tell if the item could accumulate a cost while `itemArrayR[5][1]` would tell the item’s priority. When a block is done with an item, it passes the index value of that item to the next block.

The Executive block maintains two arrays of real information (`itemArrayR[]` and `itemArrayC[]`) and two arrays of integer information (`itemArrayI[]` and `itemArray3D[]`). These arrays are described below:

Real array

The real array `itemArrayR[]`, passed through SysGlobal3, contains the following information:

Slot #	Description
0	Quantity: The number of items that the current item represents. This is used, for example, by the Set block (Item Library). Item quantities are used to copy items in queues and resource blocks. They are also used by certain input connectors (such as on the Activity block) to convey additional information to a block.
1	Priority: Used by the Set, Get, and Queue blocks. Note that in ExtendSim the lower the number (including negatives), the higher the priority.
2	Reserved for future use.

Cost array

The real array `itemArrayC[]`, passed through SysGlobal9, contains information concerning cost resources that are batched with the item. This information is used by residence blocks to calculate the accumulated cost based on the cost rates and the amount of time the item spent in the block.

Slot #	Description
0	Item type: When considering cost, all items can be classified as an item that can accumulate costs (1) or a resource (2).
1	Resource Rate 1: The cost per time unit of a resource batched to the item using the batch block.
2	Batch Number 1: Stores the amount of resource 1 batched to the item.
3	Resource Rate 2: The cost per time unit of a resource batched to the item using the batch block.
4	Batch Number 2: Stores the amount of resource 2 batched to the item.
5	Resource Pool Rate: The accumulated cost per time unit of resources batched to the item using the Queue block.
6	Unused
7	Original Cost: Used in calculating cost when unbatching items using the Unbatch block (Item library).
8	Unused
9	Unused

Integer arrays

There are two integer arrays: itemArrayI and itemArrayI2.

The integer array itemArrayI[][5], passed through SysGlobal4, contains the following information:

Slot #	Description
0	Free row flag. This is used by the Executive block for memory management. It has a value of 1 if the row is free (that is, has no existing item associated with it), and a value of 0 if the row is in use. See the Exit block (Item library) for an example of how to use this to delete an item.
1	Batch ID. This is used by the batching and unbatching blocks to keep track of which items are part of what batch.
2	User-defined integer value. This value is left untouched by the blocks in the Item library.
3	Unused except where needed to provide backwards compatibility with Extend 5 or earlier.
4	Block number where item is.

The new itemArrayI2[][5] is passed through SysGlobal18 and has 5 columns, all of which are reserved:

Slot #	Description
0	Resource Order ID. If an item has requested at least one advanced resource requirement, the integer held in slot 0 represents the record associated with the last requested requirement in the "Resource Orders" database table. Resource Order ID is used with Advanced Resource Management (ARM).
1	Reserved for future use
2	Reserved for future use
3	Reserved for future use
4	Reserved for future use

3D array

The integer array `itemArray3D[][10]` stores information for the ExtendSim 3D animation and is passed through `SysGlobal12`.

 Some of the values are scaled by 100,000 so that they can be stored in an integer, saving space.

Slot #	Description
0	3D objectID. Used to reference the 3D object that represents the item in the E3D window.
1	ID for the first skin of an item.
2	ID for the second skin of an item.
3	Scale of an object. This is based on a factor of 100,000. (An object with a scale of 100000 will be normal sized; a scale of 50,000 will be half sized.)
4	Object rotation in degrees times 100,000.
5	Object Z location times 100,000. The Z locations are based on a ground level of 100, so a Z location in <code>itemArray3D</code> of 10,000,000 would be at ground level.
6	Object type. This is the type of 3D object as returned by the <code>E3DGetObjectType</code> function.
7	Temporary level times 100,000. This stores the Z level for an object before it is placed on a Conveyor or other object that temporarily raises its position.
8	Restore Z level. If true, the temporary Z level will be used to restore the original Z position of the 3D object.
9	Reserved for future use.

Attribute global arrays

There are three global arrays that are responsible for attributes:

- The first (`_AttributeList`) stores attribute names
- The second (`_AttribType`) stores the attribute type (value, string, or db address)
- The third (`_AttribValues`) stores attribute values

The attribute names are stored in a `String15` type global array called "`_AttributeList`". This array is created whenever a block that uses attributes is placed in the model. Its single column contains an

alphabetized list of the attribute names entered into the blocks. All blocks that reference attributes use a popup menu to allow the modeler to select from a list of attributes that have already been defined for the model or to create a new attribute. When the popup menu is clicked, these blocks reference the `AttribList` global array to ensure that all of the attributes defined in the model are available for selection in the popup menus. Each time a new attribute is added, this global array is increased in size by one, the attribute name is appended to the list, and the list is sorted.

The “_AttribType” global array is used to store the attribute’s type at the time the user defines a new attribute. There are 3 types of attributes: value, string, and db address.

While building a model, it is possible to define attributes that end up not being used or referenced in the model. Cleanup of unused attribute names is done at the start of the simulation. The Executive clears the `AttribList` global array in its `StepSize` message handler. Each block in the model that references an attribute name then adds all of its referenced attributes to the `AttribList` global array in their `StepSize` message handlers.

In the `InitSim` message handler, the Executive sorts the new `AttribList` global array (which now includes only attributes which are used in the model). In doing this, each attribute name is assigned a sequential index (the row index) in alphabetical order. Each block then calls the `Attrib_GetColumnIndex` procedure which searches the `AttribList` global array for the attribute that is referenced by the calling block. This index is used when referencing the attribute value during the simulation.

In addition, if either of the two costing attributes (“_cost” or “_rate”, as discussed in the User Guide) are used in the model, they are assigned the index values at the end of the list of user-defined attributes.

The attribute with index 0 stores the animation object for the item.

The third global array used for attributes stores the attribute values for each item. In the Executive’s `InitSim` message handler, the two-dimensional global array “_AttribValues” is created to store the values of the attributes during the simulation. The number of columns is calculated as the “number of user-defined attributes plus one” for the animation attribute and plus two for the costing attributes (if costing is used in the model). The number of rows corresponds to the number of items in the model and is increased if additional items are allocated. The attributes’ values can be referenced by using the attribute index as the column and the item index as the row. For example: the following is pseudocode for setting an attribute:

```
AttribValueIndex = GaGetIndex("_AttribValues");
GaSetReal(Value1,attribValueIndex,itemIndex,AttribIndex);
where:
Value1 = the value of the attribute
AttribValueIndex = The index to the "_AttribValues" global array
ItemIndex = The index of the item
AttribIndex = The index of the attribute
```

Basic item messaging

The actual moving of items between blocks is done through a messaging communication structure using item connectors and connections. This messaging system allows modelers to place blocks in a more intuitive sequence.

Discrete event blocks send messages to each other during the course of a simulation run. These messages are used for communication regarding whether items are available, whether they have been taken, and whether a block is free to receive items.

For single connectors, messages are sent using the functions *SendMsgToInputs(connectorname)* and *SendMsgToOutputs(connectorname)*. For variable connectors, messages are sent using *SendMsgToInputs(connectorname, whichConnector)* and *ConArraySendMsgToOutputs(connectorname, whichConnector)*. Messages are received in message handlers that have the name of the connector that received the message. For example the “On ItemIn” message handler is called when a message is sent to the “ItemIn” connector.

Discrete event blocks have a function in their code called *SendMsg*. This function is just included for clarity. It calls the two ModL functions mentioned above.

The *SendMsgToInputs* and *SendMsgToOutputs* functions only send messages. In discrete event models, so that more information can be sent with each message, the global *SysGlobalInt3* is used as an argument to the messages, and the global *SysGlobalInt0* is used as a return code value. Note that some globals are used to perform different functions during the initialization (*CheckData* and *InitSim*) phases of the simulation run.

During the *Simulate* phase of the run, the meanings of the various values of *SysGlobalInt0* and *SysGlobalInt3* are as follows:

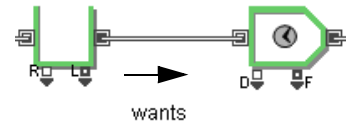
Value	Sending (<i>SysGlobalInt3</i>)	Returning (<i>SysGlobalInt0</i>)
0 (rejects)	(Not sent)	Block rejects item
1 (wants)	Does block want item?	(Not returned)
2 (taken)	Item has been taken	(Not returned)
3 (needs)	Item needs to be taken	Block needs item
4 (query)	What is the index of the next item?	Item index for the next item (0 if no item is found)
5 (notify)	Item has been sent	(Not returned)
6 (blocked)	Is the downstream block blocking us?	(Not returned)
7 (init)	Used during initialization	(Not returned)

Depending on how the message sequence is initiated, items can be either pushed or pulled through the model. It is easiest to illustrate this with a series of simple examples. The following figures show a Queue block connected to an Activity block with a single item capacity.

Push mechanism

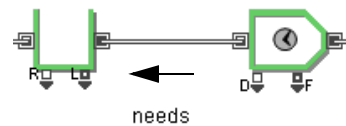
When an item is pushed through the model, the upstream blocks (in this case, a Queue) try to push their items out into any downstream blocks.

For example, assume that an item has just arrived at the Queue and the Queue is attempting to pass that item along to the Activity. The first action is for the Queue to send a *wants* message through its item output connector to the Activity. This is accomplished by setting SysGlobalInt3 to a value of 1 (wants) and calling the function *SendMsgToInputs*. This indicates that the Queue wants to send an item to the Activity.



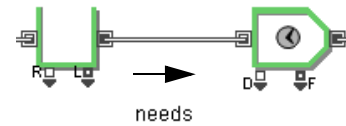
Wants message sent to Activity

The Activity receives the message in its “on itemIn” message handler. If the Activity in the above example is not currently processing an item and thus is idle, it will return a *needs* value to the Queue by setting SysGlobalInt0 to 3 (needs), indicating that the item can be accepted.



Needs value returned from Activity

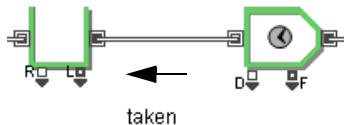
If a *needs* value has been returned from the Activity, the Queue then sends a *needs* message back to the Activity by setting SysGlobalInt3 to 3 (needs) and calling the function *SendMsgToInputs*. At this point the item would be committed to moving from the Queue to the Activity.



Needs message sent to Activity

If the Activity is currently busy processing another item, instead of a *needs* value it will return a *rejects* value. This is accomplished by setting SysGlobalInt0 to 0 (rejects). If the item is rejected, the message sequence will be terminated.

The item is logically moved from one block to the next by transferring its item index over the connection between the blocks. To do this, the Queue sets its output connector value to the item index.



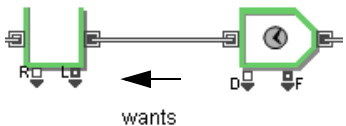
Taken message sent back to Queue

When a block sets its connector value to the item index, the connector value of any connected blocks will automatically be set to that same item index value.

Since the output connector of the Queue is connected to the input connector of the Activity, the two connectors will share the item index value. The Activity then sets its input connector to a negative number and sends a *taken* message back to the queue to indicate that the item has successfully moved. This is done by setting SysGlobalInt3 to 2 (taken) and calling the function *SendMsgToOutputs*. In response to the *taken* message, the queue will update any internal statistics related to the departure of the item.

Pull mechanism

In addition to being pushed, as in the preceding example, items can also be pulled through the model. If they have remaining capacity, downstream blocks try to pull items into their inputs. For example, when the Activity finishes processing the item, it will attempt to pull in another item. To do this, the Activity first sends a *wants* message to the Queue



Wants message sent from Activity

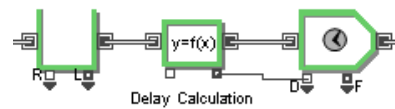
indicating that it is requesting an item. The *wants* message is sent by setting SysGlobalInt3 to 1 (wants) and calling the *SendMsgToOutputs* function.

If an item is available in the Queue, its output connector will be assigned to that item's index value. The Activity will pull in the item and then send a *taken* message back to the Queue by setting SysGlobalInt3 to 2 (taken) and calling the *SendMsgToOutputs* function. If an item is not available in the Queue, a *rejects* value (0) will be returned and the message chain will be terminated.

Passing blocks

Both of the blocks in the above examples are residence blocks and can hold items for some period of time. Passing blocks do not have this ability and must pass the item through in 0 time.

The following example shows an Equation(I) block reading multiple attribute and other property values to calculate the delay needed for the Activity. The Equation(I) block does not affect the messaging communication between the Queue and the Activity. Since it is a passing block, it transfers the initial *wants* and *needs* messages between the Queue and Activity. Thus, any number of passing blocks can be between any blocks that can hold items. Once the item moves into a passing block, it will send a *taken* message to the upstream block.



Passing block between Queue and Activity

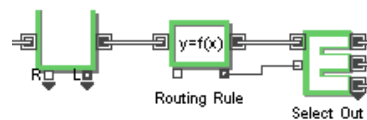
Blocked and query messages

One of the more complex message communication subsystems in this architecture is the communication between blocks when a block needs information about an item that has not yet arrived. This occurs when an item needs to know if it can move downstream before it starts to move, or when it needs to determine which path it will take before it gets to the block that contains the different paths.

If *Predict the path of the item before it enters this block* is selected in its dialog, the Select Item Out block will use a special sequence of messages to determine which direction the item will go before the item leaves any upstream residence blocks.

Traditional discrete event architectures would require dummy resources to overcome these problems. The ExtendSim discrete event architecture, however, is able to determine the path of an item before it moves into the decision logic and is able to block through decision points without any additional modeling components.

In the example at right, the Equation(I) block reads multiple attribute and other property values to calculate the one of three paths for that item to follow (Select Item Out) based on the value of the properties for that item. Without the ability to send a Query message, the Equation(I) block would read the value of the properties on the item, but only after the item has already entered the block. In this case, this could cause a problem as the Select Item Out block may be blocked down the path that the item will need to travel. If the Select Item Out is blocked, and the item has to move into the Equation(I) block to present its property values, then the item will be stuck in the Equation(I)



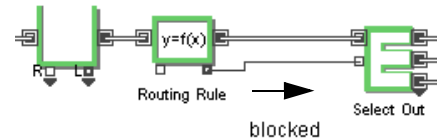
Select Item Out controlled by Equation(I)

block. And since property-manipulating blocks are only meant to pass items, not hold them, the Queue will understate the number of items available.

The solution to this problem is to set up the Equation(I) block to be aware that it is in this situation. The Equation(I) block can then look upstream to see what the next item coming along will be, and not pull in the item until the downstream path is free.

Blocked messages

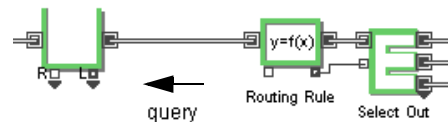
The first part of this process involves sending a *blocked* message downstream from the Equation(I) to see if there are any blocks that could cause this situation. This is accomplished by setting SysGlobalInt3 to 6 (blocked) and calling the *SendMsgToInputs* function. The Equation(I) does this the first time it gets an incoming message (i.e. the first time the Select Item Out block requests a calculated value from the value out connector.) The Equation(I) sends a blocked message from its item output connector in its on AttrbOut message handler. (The on AttrbOut message handler is called when the value out connector on the Equation(I) block gets a message.) If the block downstream is a potential blocker, it will return a TRUE value by setting SysGlobalInt6 to 1 (TRUE). If a TRUE value is returned in response to the message, then the Equation(I) block sets a flag that records that it is blocked.



Blocked message sent to Select Item Out

Query messages

If it has been determined that blocking can occur, each time there is a request for a calculated value, the Equation(I) block sends a *query* message upstream by setting SysGlobalInt3 to 4 (query) and calling the *SendMsgToOutputs* function. This message is essentially a request for information about the next item that is available. The message will be propagated upstream by the blocks until it reaches a block that can contain items, such as a queue.



Query message sent to Queue

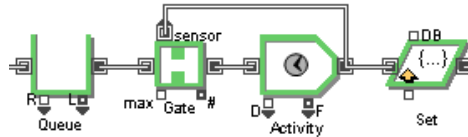
The block responding to the *query* message will check to see what the item index of the next item to be released will be and will return that value to the querying block by setting the global SysGlobalInt0 to the item's index value. The Equation(I) block will then access the property values for that item. From this point on, each time that a property value is requested from the Equation(I) block, it will send out the *query* message and check the property values of the next item. It will not need to re-send the *blocked* message again.

ExtendSim will notify modelers if there are any logical ambiguities that will not allow the model to operate properly. When this occurs, an error message is issued that recommends a course of action that will resolve the ambiguity.

The Notify message

The final message used by this system is the *notify* message. This message is used to notify other blocks that an item has just passed by a specified point in the model. A special sensor connector receives this message. Sensor connectors do not pass items, they monitor the message stream, processing only the *notify* message. Only a few blocks have sensor connectors, although all of the blocks that process items will send the *notify* message through their item output connector by setting SysGlobalInt3 to 5 (notify) and calling the *SendMsgToInputs* function.

In this example, the Gate block limits the number of items in the section of the model between its output connector and the activity's output connector. It uses its sensor connector to determine when an item has passed the activity's output connector.



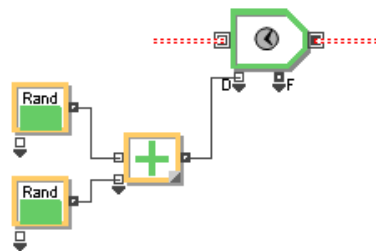
Situation that requires a notify message

As an item travels from the Activity to the Set block, a *taken* message will be sent to the Activity. In response to this message, the Activity will send out the *notify* message. The normal item input connectors in the blocks will ignore the *notify* message, but sensor connectors will respond to it and start processing information about the item. In the above example, when a *notify* message is received by the sensor connector, the Gate block knows that another item has passed by and can allow an additional item into the model section.

Value connector messages

In addition to item connectors, value connectors are used to relay model information. Value connectors pass a single number from one block to another. Examples include a value output such as length of a queue or an input such as a delay time. The use of these connectors allows the combining of blocks that perform numerical calculations to provide a control structure and logical information for the discrete event blocks. This provides additional modeling flexibility without requiring user programming or complicated interfaces.

In the example that follows, two Random Number blocks are added together to specify the delay for an activity. Whenever an item arrives to the activity, the Random Number and Math blocks will need to be recalculated. Whenever a discrete event block detects a condition where an update to the input value of a connector is needed (in this case, an item arriving to the activity), it sends a message out its input value connector (in this case, value input connector "D" in Figure 9) using the *ConArraySendMsgToOutputs()* function. This message propagates backwards via the Math block and causes the Random Number blocks to recalculate their output values so that the Math block can add them and update its output connector value for the Activity block to use.



The sum of two random variables specifying an activity's delay

Message emulation

A default feature ("message emulation") for continuous blocks in a discrete event or discrete rate model causes the messages to be propagated throughout all of the continuous blocks used in the calculation. Message emulation is used whenever the block contains no message handlers for any of its connectors. In that case, the "on simulate" message handler is used as a default message handler for all connectors.

How message emulation works:

- If a message is received on an output connector:
 - Echo to ALL input connectors to get their latest values.
 - Send a SIMULATE message to the block to recalculate values to its output connectors.
- If a message is received on an input connector:

- Echo to all other inputs THAT HAVE NEVER RECEIVED A MESSAGE to get their latest values.
- Send a SIMULATE message to the block to recalculate values to its output connectors.
- Echo message out the outputs.
- If Additional messages are received on any connector while processing current message:
 - DON'T echo these additional messages.
 - Send a SIMULATE message to the block to recalculate its latest values.

This message emulation capability improves performance and reduces redundancy.

The example code below illustrates an On Simulate message handler adding one to its input by assigning the output connector (Con1Out) to the input connector (Con1In) plus one. There are two cases:

- The block received a message from its input connector to recalculate, so it calls its Simulate message handler and then propagates that input connector message by then sending a message via its output connector to any connected block's inputs so they can act on that message.
- The block received a message on its output connector from a downstream block that needs a new value. It first propagates the message backwards through its inputs so upstream blocks can recalculate, then calls its Simulate message handler to recalculate its outputs for the downstream block to use.

Whenever either connector receives a message, the Simulate message handler will be executed and a message will be sent out the other connector through message emulation. This will propagate messages where appropriate.

```
On Simulate
{
  Con1out = Con1In + 1; // calculate the output value
}
```

Explicit connector messages

Overriding message emulation gives you, as a block developer, more flexibility in the behavior of the block. If a generic block has one or more connector message handlers, message emulation is automatically disabled and the connector message handlers are used to perform the calculation instead. In this case, the generic block must send out messages to other value input and output connectors explicitly. For example, a message must be sent out of the output connector whenever a message is received on the input connector, and a message must be sent out of the input connector whenever a message is received on the output connector.

The code below uses message handlers to make a block behave similarly to the message emulation used in the above example. Both examples will perform identically in model operation. Because message handlers have been explicitly specified for the connectors in the example below, message emulation has been automatically disabled.

```

On Con1In
{
    Con10Out = Con1In + 1;
    SendMsgToInputs(Con10Out);
}

On Con10Out
{
    SendMsgToOutputs(Con1In);
    Con10Out = Con1In + 1;
}

On Simulate
{
}

```

Functions in discrete event blocks

The following are some, not all, of the functions to be found in discrete event blocks. They are the ones found in most of the blocks and help give an understanding of how the code is organized.

Function	Description
SendItem	Attempts to pass an item out of the block. First it checks certain block variables to see if an item is available, then it will output the index value of the item and send a message to the receiving block (it calls SendMsg).
GetItem	Attempts to get an item once a block determines that it is ready to get an item. First it checks to see if an item is available, then it gets the index value, negates the connector, and sends a message to the sending block.
PassItem	Performs the actions of both the GetItem and SendItem functions. It is used in blocks that pass items through without delaying them (passing blocks).
SendMsg	Sends the messages out through the connectors. It sends out messages based on the values of its arguments.

Globals in discrete event models

Several reserved global variables (sysGlobals) are used in discrete event models. This table lists the global variables with a brief description of their use during the Simulate message (most have undefined values during the CheckData message). For more information, examine the code of the blocks in the Make Your Own category of the Example Libraries > ModL Tips library or the Executive block and other blocks in the Item library.

Use of system globals during Simulate message

Global	Used In	Definition during Simulate message
SysGlobal0	DE blocks	Used to access the TimeArray of posted events.
SysGlobal1	Blocks with reports	Report file number.

Global	Used In	Definition during Simulate message
SysGlobal2	Blocks with traces	Trace file number.
SysGlobal3	DE blocks	Used to access the real item array of discrete event item data.
SysGlobal4	DE blocks	Used to access the integer item array of discrete event item data.
SysGlobal5	Do Not Use	DO NOT USE. (Was used in versions prior to 7.0.3 to pass the TimeEventMsgType array between the Executive and event scheduling blocks. Use SysGlobal13 instead.)
SysGlobal6	DE blocks	Used to access the string item array of discrete event timer data.
SysGlobal7	DE blocks	Used to access the TimeBlocks array of event posting blocks.
SysGlobal8	Resource pools	Used in communication between the Resource Pool, Queue (in Resource Pool mode), and Resource Pool Release blocks.
SysGlobal9	Costing blocks	Used to access the cost item array for discrete event item data.
SysGlobal10	Global arrays	Communicates the value, row, and column to other global array blocks when a value in a global array has changed.
SysGlobal11	Throw and Catch blocks (Value library)	Passes a value between blocks
SysGlobal12	DE blocks	Used to access the integer item array of discrete event itemArrayI2 data
SysGlobal13	DE blocks	Passes the time event message type array between Executive and event scheduling blocks
SysGlobal14		Unused
SysGlobal15		Unused
SysGlobal16	Blocks with attributes	Passes array containing names of attribute arrays
SysGlobal17	Catch Item	Passes throw block nums array
SysGlobal18		Unused
SysGlobal19	Proof Animation blocks	Passes ProofString as an array
SysGlobal20	Executive	Used in BlockReceive5 as a DB Address argument for the calling block and used by the Executive as the return value.
SysGlobal21		Reserved for v9
SysGlobal22	Item Log Mngr	Used to pass arrays from Item Log Manager to satellite blocks (e.g., the History block).
SysGlobalStr0	Blocking blocks	Name of attribute being checked upstream

Global	Used In	Definition during Simulate message
SysGlobalStr1	Any block	Used as an argument for the block table info (BlockReceive4) message handler.
SysGlobalStr2	Blocks with attributes	Name of new string attribute passed to Executive
SysGlobalInt0	DE blocks	Return code from the messages that discrete event blocks send to each other.
SysGlobalInt1	DE blocks that create items	Index value for the first free row in the item arrays maintained by the Executive block.
SysGlobalInt2	DE blocks	Total number of rows of data that have been allocated to the item arrays. This will always be rounded up to the next allocation level.
SysGlobalInt3	DE blocks	Argument to the messages that discrete event blocks send to each other.
SysGlobalInt4	Blocking blocks and priority	Tells whether the priority is being checked.
SysGlobalInt5	Used for v6 compatibility	Global batch count
SysGlobalInt6	Blocking blocks	Specifies whether or not there is a blocked block (see the Get block).
SysGlobalInt7	DE blocks	ID of the item currently being disposed.
SysGlobalInt8	DE blocks	Used to pass the block number to the Executive when the block is rescheduling itself in the CurrentEvents array.
SysGlobalInt9	Random Number, Holding Tank (Value library)	Flag when the plotter is sending a message. (See the code of the Plotter, Discrete Event and the Random Number blocks for more information.)
SysGlobalInt10	Resource pools	Used in communication between the Resource Pool, Queue (in Resource Pool mode), and Resource Pool Release blocks.
SysGlobalInt11	Select Item Out	Used to control whether or not a new random value is generated in a connected Random Number block.
SysGlobalInt12	Throw and Catch (Item library)	Passes item index in Throw Item and Catch Item blocks.
SysGlobalInt13	Random Number, Equation, and Select blocks	Set to TRUE if a new random number should be generated for a select block.
SysGlobalInt14	Throw and Catch (Item library)	Used by Throw Item and Catch Item blocks to determine which Throw block is sending the message.

Global	Used In	Definition during Simulate message
SysGlobalInt15	Resource pools	Used in communication between the Resource Pool, Queue (in Resource Pool mode), and Resource Pool Release blocks.
SysGlobalInt16	Blocks with costing	Set to TRUE during CheckData message if discrete event model is calculating item costs.
SysGlobalInt17	Blocks with attributes	Holds the number of attributes in a discrete event model.
SysGlobalInt18	Not Used	
SysGlobalInt19	DE blocks	Used as an argument for blockreceive4 (on queueFunction message handler).
SysGlobalInt20	Queues, Resource pools	Used in direct communication with queues and resource pools.
SysGlobalInt21	Queues, Resource pools	Used in direct communication with queues and resource pools.
SysGlobalInt22	Set, Get, Executive	Executive and attribute blocks
SysGlobalInt23	DE blocks	Block number of the Executive
SysGlobalInt24	Create, Set, Get, Executive	Attribute info command number
SysGlobalInt25	Set, Create, Executive	Number of attribute name arrays in a block
SysGlobalInt26	Any block	Block number of item tracing block
SysGlobalInt27	DE blocks	Item index sent to tracing block
SysGlobalInt28	Resource Pool, Queues	List number when resource pool tries to send item. Used in BlockReceive1 message
SysGlobalInt29	Resource Pool, Queues	Set in Queue in PreCheckData to tell Pools the Historical Log has been turned on
SysGlobalInt30	Resource Mngr	Resource Requirement record
SysGlobalInt31	Resource Mngr	Item Index
SysGlobalInt32	Resource Mngr	Pointer to Resource method
SysGlobalInt33	Item library	Block number for Resource Manager
SysGlobalInt36	Resource Mngr	Number of selected resources
SysGlobalInt37	Unique ItemID	Keeps a count of how many items have been created. Used to assign a unique ID to items in ItemArrayI2
SysGlobalInt38	Item logging	Msg type. Tells the remote satellite block what to do in BlockReceive6 for item logging
SysGlobalInt39	Item logging	The table index of the Item Log block's central log table

Global	Used In	Definition during Simulate message
SysGlobalInt40		Global setting for whether or not to track string attribute values

Use of Global variables during CheckData or InitSim messages

Some of the variables in the above table are also used in the CheckData and InitSim message handlers and have a different meaning than when used in the Simulate message handler.

Global	Different definitions during CheckData/InitSim messages
SysGlobalInt0	Number of blocks posting events for the time array.
SysGlobalInt1	Block number of the Executive block.
SysGlobalInt8	Used by the Executive block during CheckData to check for duplicate Executive blocks.
SysGlobalInt11	Used to control random seed initialization.
SysGlobalStr1	Used by equation to send a value to Proof Animation.

Creating blocks for discrete event models

A Make Your Own block (Example Libraries > ModL Tips library > Make Your Own category) is useful for creating blocks that will be used in discrete event models that have item inputs and outputs. However, you may want to create blocks that have value inputs and outputs, but are meant to be used in discrete event models. In that case, do not use the Make Your Own block as a template. Instead, you only need to follow these two rules:

- Add SendMsgToInputs(connName) and SendMsgToOutputs(connName) functions to your Simulate message handler for all input and output connectors on your block. Remember that the argument to SendMsgToInputs is the name of the *output* connector on the block you are creating; likewise, the argument to SendMsgToOutputs is the name of the *input* connector.
- To control how the block receives and sends messages, add at least one message handler (which can be empty) with the name of one of your connectors. Otherwise, the block will emulate connector messages. Thus, if the name of one of the output connectors is “G1Out”, you would add a message handler such as:

```
on G1Out
{
...
}
```

Using options in the Run > Debugging command, you can cause models to display block messages as they run. This gives an idea of how messaging works in discrete event models.

How discrete rate blocks and models work

The blocks in the Rate library are for creating discrete rate models. LP technology, which has global oversight over discrete rate models, as well as messaging in discrete rate models, is discussed in the User Guide.

Globals in discrete rate blocks

Several reserved global variables (sysGlobals) are used in the Rate library blocks. The table below lists those global variables with a brief description of their use in Rate library blocks during the Simulate message (most have undefined values during the CheckData message).

Global	Definition during Simulate message
SysFlowGlobalInt0	Used to propagate the section# (= corresponding to the row# in _FlowSection global array)
SysFlowGlobalInt1	Used to propagate the row# in the _FlowAttValues global array for each out-flow connector that can change attributes
SysFlowGlobalInt2	Used to stop the propagation when the message is received twice in the same block => count the LP calculations made in the Executive
SysFlowGlobalInt3	{1/2/3} Option defined in the Executive block to show or not the bias order above the Merge and Distribute icons using a mode with bias order
SysFlowGlobalInt4	Used to get the type of propagation which is made in FlowBlockReceive0 and FlowBlockReceive3 messages
SysFlowGlobalInt5	Count the number of constraints in an LP calculation
SysFlowGlobalInt6	Row # in siListTCCConnection_GA global array. Used to update when change occurs in Throw Catch connections.
SysFlowGlobalInt7	{0/1} True if the Rate blocks have to update the state starved or blocked after each new calculation of the rates
SysFlowGlobalInt8	Used to inform the block on the number of the section which is concerned by the message handler (FlowBlockReceive0_1_3_4_6_7
SysFlowGlobalInt9	Used to inform which is the sense of the propagation when a message is sent from a block
SysFlowGlobalInt10	Unused
SysFlowGlobalInt11	Used to inform from which connector number the message has been sent (in FlowBlockReceive0 msg)
SysFlowGlobalInt12	Value of the popup menu in the Executive
SysFlowGlobalInt13	Used to inform on the option taken in the Executive for the merge percent option
SysFlowGlobalInt14	Used to inform on the option taken in the Executive for the merge/diverge bias order
SysFlowGlobalInt15	{0/1} True if the definition of the LP area is in process
SysFlowGlobalInt16	Unused
SysFlowGlobalInt17	Unused
SysFlowGlobalInt18	To stop the propagation of the FlowBlockReceive5 message in the block. Each block has to express its constraints only once per LP resolution

Global	Definition during Simulate message
SysFlowGlobalInt19	To know if there is a block in the area which requires the extra calculation of an LP with downstream and upstream differentiation
SysFlowGlobalInt20	To count the number of slacks in the LP
SysFlowGlobalInt21	To inform from which type of connector (Flow or Throw/Catch) number the message has been sent (in FlowBlockReceive0 message)
SysFlowGlobalStr0	Used for the propagation of the name of the flow unit in a section
SysFlowGlobal0	To store the maximum rate defined in the Executive
SysFlowGlobal1	To store the rate precision to be considerate as 0

ModL

Debugging

A guide to debugging models and blocks

*“We have gone beyond the absurd...
our position is ridiculous.”
— John Vacarro*

Debugging models

This section discusses ways to debug models and the code necessary to add Report and Trace features to blocks you build. These methods are also useful when you debug block code.

Features discussed in the User Guide

The Debugging Tools chapter of the User Guide has a list of blocks that are useful for debugging models. These blocks display values, help validate item flow, and speed debugging of a model. That User Guide chapter also contains other information for debugging models such as stepping through a model run and showing simulation order.

Model profiling

Profiling generates a text file showing the percentage of time blocks execute during a simulation. Use model profiling to analyze the amount of time that each block in a model is used. This helps developers who want to determine if they should optimize custom blocks, although non-developers might also use it to find the areas of their models that are most heavily used.

To generate a profile text file, choose the Run > Debugging > Profile Block Code command, then run the model. Be sure to run the model long enough (at least 5 seconds for each block in the model) to compensate for extraneous events and get a good sample. It is also important to not move blocks in the model between runs if you repeatedly run the profile. For example, the profile of a Bank Line model might look like:

Block Name	Block Number	Time (seconds)	Percent
Create	0	2.42	9.30
Queue	2	7.65	29.50
Activity	3	2.27	8.80
Activity	5	1.15	4.40
Activity	6	1.82	7.00
Exit	7	4.15	16.00
Plotter, Discrete Event	8	5.83	22.50
Executive	10	0.62	2.40

You can use the information in this profile to tune your model or to look for anomalous results. For example, if one of the three Activity blocks used a much higher percentage of the time than the other two, but you had expected them to be about the same, you could use that information to see what it was about that block that was different.

Note that only blocks that use 1% or more of the simulation time are shown in the profile. And the percentages are approximate, so the sum of the percentages might not equal 100%.

Profile text files are opened, closed, and edited just like any other text file in ExtendSim.

Adding Trace and Report code

ExtendSim's Trace and Report features can be useful in debugging a model.

 The blocks that ship with ExtendSim contain all the necessary Tracing and Reporting code.

If you create your own blocks, those blocks require additional ModL code to take advantage of the reporting and tracing features. When building a new block, you should include code similar to the following.

Trace code

For continuous blocks (such as those in the Value library), put the following code in the Simulate message handler. For discrete event blocks (such as those in the Item library), put the following code in the departure procedure:

```
// SysGlobal2 is the file reference number for the Debug Trace
// template for trace:   BLOCK NAME   BLOCK NUMBER   CURRENTTIME
if( SysGlobal2 != 0.0 ) // check for open file for TRACE
{
    fwrite(SysGlobal2,"myBlockName block number "+(myBlockNum-
ber()) + ".   CurrentTime:"+currentTime+".", "", True);
    if(getBlockLabel(myBlockNumber()) != "")
        fwrite(SysGlobal1,"Block Label: "+
            getBlockLabel(myBlockNumber()), "", True);
    ....
    ....
}
```

Report code

As noted in “Model reporting” in the ExtendSim User Guide, there are two types of reports: dialogs and statistics. During the On BlockReport message, the *GetBlockReportType* can be used to determine the type of report chosen by the modeler. For both continuous and discrete event blocks, put the following code in the On BlockReport message handler:

```
// SysGlobal1 is the file reference number for the TEXT REPORT
// template for report:   BLOCK NAME   BLOCK NUMBER
if( SysGlobal1 != 0.0 ) // check for open file for REPORT
{
    if (GetBlockReportType() ==0) // 0 is dialogs, 1 is statistics
    {
        fwrite(SysGlobal1,"myBlockName   block number "+
            (myBlockNumber()), "", True);
        if(getBlockLabel(myBlockNumber()) != "")
            fwrite(SysGlobal1,"Block Label: "+
                getBlockLabel(myBlockNumber()), "", True);
        ....
        ....
    }
    if (GetBlockReportType() == 1) // statistics report
    {
        ....
        ....
    }
}
```

The blocks in the Value and Item libraries have report and trace code in them. In addition, their code is set up to cause the Report and Trace files to be formatted in a structured manner.

Debugging block code without the Source Code Debugger

The ExtendSim ModL source code debugger, discussed beginning on page 184, has a lot of advantages in block debugging, offering conditional breakpoints, stack crawl, and viewing variable values in specified blocks. The following methods may also be useful for debugging code.

Using DebugMsg functions

If you don't want to use the Source Code Debugger, you can use the DebugMsg function to insert a breakpoint and monitor variable values as the simulation runs. You specify a message and variable values as this function's string argument. When the function gets called, it displays the argument in an alert.

DebugWrite is the same as the DebugMsg function except that the data is written to a file so the simulation run isn't interrupted. The advantage of using DebugMsg or DebugWrite rather than UserError is that ExtendSim warns if the Debug functions are present when the library is loaded. See the functions in "Debugging" on page 376.

Viewing intermediate results

When developing a block's code, there is an easy way to view intermediate results of calculations without any interruption of the model. Just add an assignment statement:

```
comments = myVar; // myVar will be visible as it changes
```

or add some more information and variables:

```
comments = "myVar = " +myVar+ ", myVar2 = " +myVar2; // more info
```

in your code after some calculations. Then open the block's dialog, tab to the **Comments** field, and run the model. Since "comments" is the comments box (editable text) item in the dialog, any number assigned to it will be immediately visible in the dialog. This is different than using the DebugMsg() function (discussed on page 184) to display data, as it doesn't interrupt the model for each new number displayed.

Other methods

Some of the methods for debugging models, discussed above, are equally useful when debugging block code.

Source Code Debugger

No matter what the language, code often does not work the first time. This section shows how the ModL Source Code debugger can save you time when locating the source of an error in one of your blocks or equations in equation based blocks. It provides a tutorial that shows how to step through lines of source code, examine values of variables, create breakpoints with conditions, and analyze block problems. It also discusses the various Debugger windows and dialogs.

The Source Code Debugger is available both when creating or editing the source code of blocks or equations in equation-based blocks. See "Debugging equations" on page 704 of the ExtendSim User Guide

Overview of the debugger

A source code debugger makes it much easier to determine the causes of block malfunctions. You can watch the execution of the ModL code and see its path and the effects it has on any of the variables used in the block.

Definition of terms

A *breakpoint* is where the debugger will initially stop execution of the block's code and open the debugger window. You can then manually step through lines of code to trace its execution, and examine the effects on the variables defined in that code and in the block's dialog and connectors.

The green *location arrow* points to the currently executing line of code.

The *breakpoint condition* is the TRUE or FALSE boolean decision that causes the breakpoint to break execution only under certain circumstances. This is valuable if there could be too many breaks and an important break only occurs rarely, with certain values of variables.

Step over is used when you want the debugger to execute the line of code with a function call, but not to trace the code of the function.

Step into is used when you want the debugger to trace the actual function call, including the code of the function.

Step out of is used when you want to complete the function or message handler that is currently executing and step back to the calling function.

Stop executing immediately stops the execution of the block and returns to the ExtendSim program.

Stop executing and go to source of block immediately stops the execution of the block, opens the block structure and positions the cursor at the point where execution stopped. This is particularly useful when you have found the error and want to edit the code to make a correction.

See also the descriptions of the Debugger windows and dialogs in “Debugger reference” on page 193.

Steps for debugging

As illustrated in the examples that follow, the steps to debug code are:

- 1) Recompile a block, a library, or several libraries in debugging mode. This opens the Set Breakpoints window.
- 2) Add breakpoints by clicking on the lines in the left margin of the Set Breakpoints window. (To remove a breakpoint, click it again.)
- 3) Add a condition to the breakpoint in the Set Breakpoint window so that it will occur at the appropriate time and model state. Conditional breakpoints are half-filled circles.
- 4) Close the Set Breakpoint window and run the simulation. Or click an item in the block’s dialog to execute the code.
- 5) When the breakpoint is reached, step through the code and examine the variables.
- 6) After you have resolved the problem, remove the breakpoints and remove any debugging code from the libraries.

Debugger tutorial

The following two examples show how to debug 1) a block and 2) an entire library.

- Open the Debug Tutorial model located in the Examples\Tutorials\Programming folder. As shown to the right, the model consists of three connected blocks. These blocks generate data, process the data, and plot the result.



Debug Tutorial model

- Run the model. You should see this error message:

Array exceeded dimensional bounds, variable: array, at line 16. Detected in "[0]Start" block during "DataOut" message at CurrentTime = 8

Debugging a block

- On the model worksheet, right-click the Start block and choose **Set Breakpoints** (or select the block and choose the command Develop > Set Breakpoints).

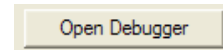
This recompiles the block in debugging mode and opens the block’s Set Breakpoints window.

- ▶ Since the debugger will automatically stop at an error message, you don't need to set breakpoints yet. Close the Set Breakpoints window.

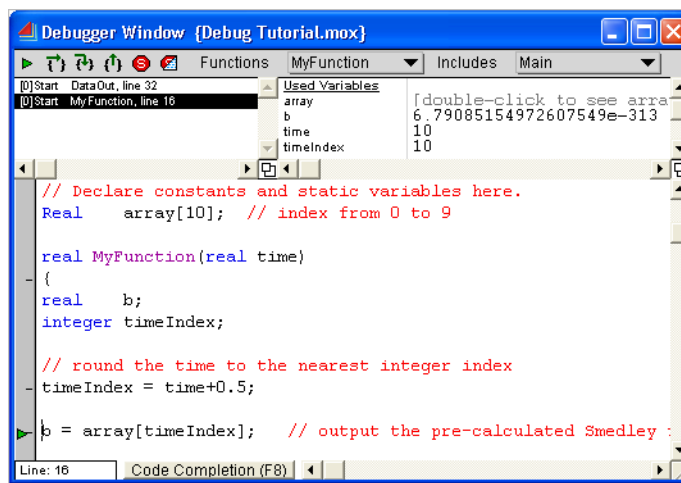
On the model worksheet, the Start block is now surrounded by a red border; its name and information in the Tutorial library's Library Window are also in red. Blocks set in debugging mode will execute more slowly – the red markings are a reminder that you should remove debugging code when finished.



- ▶ Run the model again. Instead of a Cancel button, the error message now has an Open Debugger button.



- ▶ Click the Open Debugger button. This opens the Debugger window shown below.

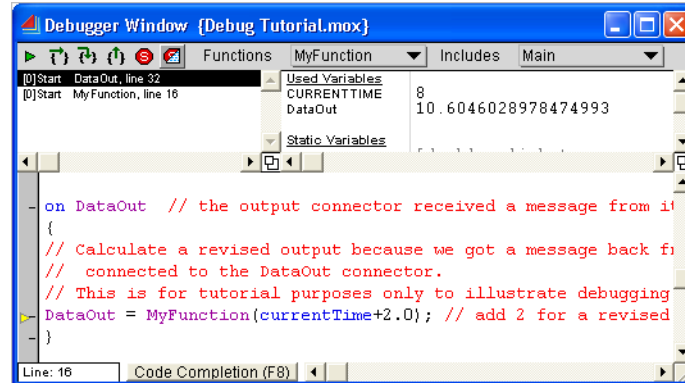


Debugger window after error message

“MyFunction, line 16” is selected in the top left pane, known as the *Call Chain* pane. In the *Breakpoint margin* at the bottom left of the *Source* pane a green *location arrow* points at line 16 (“b = array[timeIndex];”). In the *Variables* pane at top right, notice that the index variable *timeIndex* has a value of 10. Also notice that the variable *Array* is declared at the top of the script with a number of members equal to 10, indexed as 0 to 9.

In order to debug this, you need to find out how *MyFunction* got called and see the chain of events that led to this error message.

- Select the top entry (above the originally selected entry) in the Call Chain pane, shown below.



Debugger window after clicking top entry in Call Chain

Notice that the location arrow in the Breakpoint margin is yellow for this entry, indicating this is the previous entry in the Call Chain. The selected message handler is called *DataOut*, the name of one of the connectors on the block. It indicates that a connector message was received from a block that is connected to the Start block.

To see the code of the block that sent this message, you need to compile it in debugging mode. Then it can be seen in the Call Chain and you can click its entry to see the code that caused the chain of events.

- ☞ The fastest way to ensure that all the blocks in a model are compiled in debugging code is to open the Library Window for each of the libraries used in the model. Then compile the libraries in debugging mode, as discussed below.

Debugging an entire library

- If it is open, close the block's Debugger window
- Open the Tutorial library's Library Window
- ☞ Other than the Tutorial library, close all other Library Windows before you proceed. Otherwise, they will all be set to debugging mode.

- Choose Library > Tools > Add Debug Code to Open Library Windows

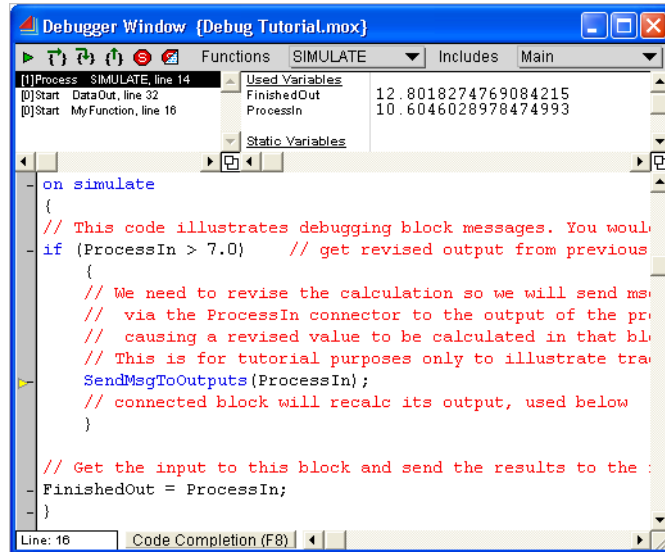
After ExtendSim finishes compiling all of the blocks in the Tutorial library, the three blocks on the Debug Tutorial model will be outlined in red, indicating that they are in debugging mode.

- Run the model again.
- When the error occurs, click the Open Debugger button to access the Debugger window.

Following the Call Chain

The Source Code Debugger allows you to follow the entire chain of events that caused the error.

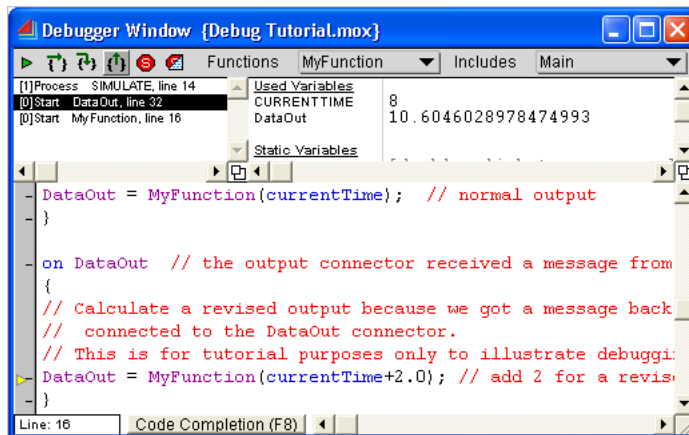
- Click on the top Call Chain entry; the one that started this whole chain of events.



Debugger window with top Call Chain entry selected

In the Source pane's Breakpoint margin, the yellow location arrow points to line 14, as indicated in the Call Chain. There was a SendMsgToOutputs function call from the Process block because its input was greater than 7.0. Its actual value can be seen in the Variables pane as 10.6046.

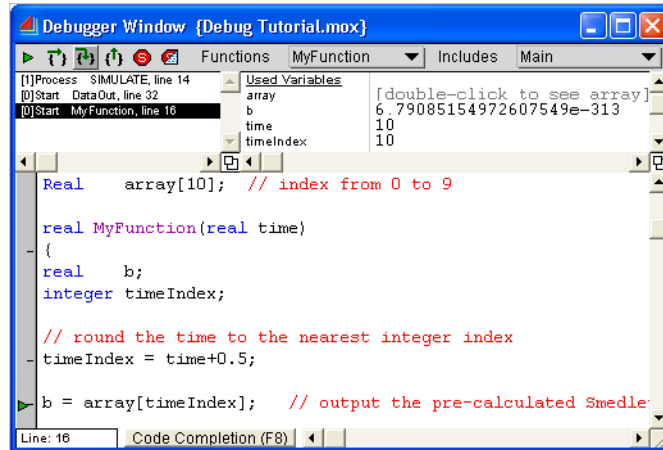
- Click on the second Call Chain entry:



Debugger window with message handler selected

The Variables pane indicates that CurrentTime is 8. Line 32 of the Source pane shows that the code called MyFunction with an argument equal to 10.0 (CurrentTime+2.0).

- Click on the bottom Call Chain item where the error occurred:



Debugger window with error point selected

- Since the error message indicated that Array was indexed beyond its bounds, check Array by double-clicking it in the Variables pane.

In the Array window, notice that the array is indexed from 0 to 9. However, the code tried to index it with a 10, which is outside its bounds, and an error message was generated.

Fixing the block's code

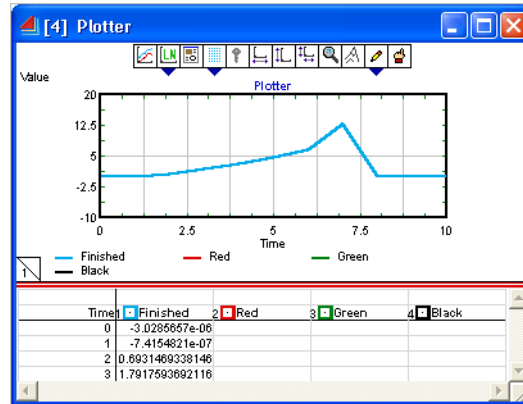
The model is being run from a CurrentTime of 0 to a Current-Time of 10, and the code could be adding 2.0 to the Current-Time in some cases. The way to fix the problem is to increase the size of *Array* so that it can be indexed from 0 to at least 12.

Index	Value
0	-3.0285657e-06
1	-7.4154821e-07
2	0.893146933815
3	1.79175936921
4	3.17805378383
5	4.78749171884
6	6.57925119869
7	8.52516135319
8	10.8046028978
9	12.8018274769

Array window

- Close the Debugger window.
- Open the structure window of the Start block so that you can edit the code:
- Change the declaration of Array[10] to Array[13].
- Close the Structure window and compile the block.

- Try running the model again and observe the plotter:



Plotter showing drop off of output

The plotter shows a drop off in values around time 7. However, the Smedley function is supposed to be increasing. To see what the problem is you need to set breakpoints.

Setting breakpoints

- Right-click the Start block in the model and select **Set Breakpoints**. This opens the Set Breakpoints window.
- To set a breakpoint, click in the Breakpoints margin on the left side of the Set Breakpoints window, at line 16 – the location of the `B = array[timeIndex]` statement.

```
// Declare constants and static variables here.
//Real array[10]; // index from 0 to 9
Real array[13]; // index from 0 to 12

real MyFunction(real time)
{
    real b;
    integer timeIndex;

    // round the time to the nearest integer index
    timeIndex = time+0.5;

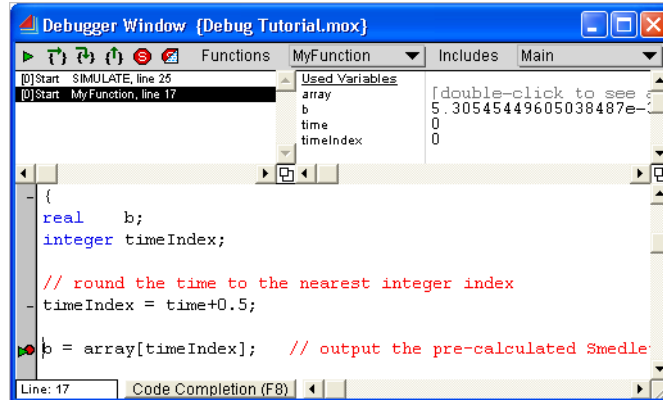
    b = array[timeIndex]; // output the pre-calculated Smedley
}
```

Start block debugging window

A red dot appears where you clicked. Running the model will now cause the simulation to break (pause) at that point.

- Run the model

The Debugger window opens and displays the breakpoint as a red dot.



Breakpoint reached

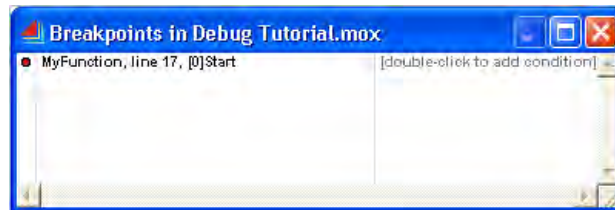
In the Variables pane, timeIndex is 0, which means the simulation is stopped at the beginning.

- ▶ In the toolbar at the top of the Debugger window:
 - ▶ You can either click the green triangle (Continue) 10 times until the breakpoint occurs at timeIndex 10
 - ▶ Or, set conditions as described below

Setting conditions

Clicking until a timeIndex of 10 is reached can get tedious. Instead of clicking each time, set a condition on the breakpoint so the code only breaks when timeIndex is greater or equal to 10.

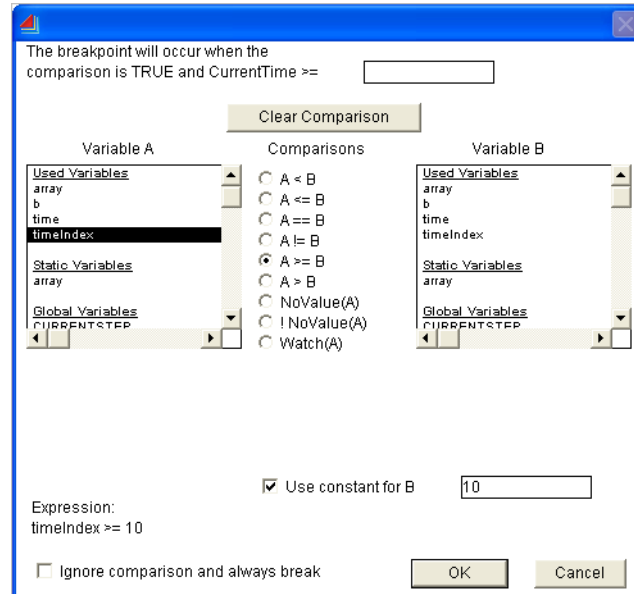
- ▶ Stop the run
- ▶ Choose Develop > Open Breakpoints Window (or bring that window forward if it is open)



Breakpoints window showing the breakpoint

- ▶ The Breakpoints window has two columns. Double-click in the right column that relates to the breakpoint (in this case, there is only one breakpoint).

A new window appears for setting conditions:



Condition window

- In this window, do the following:
 - For Variable A, select **timeIndex**
 - As the comparison, choose the **A>=B** radio button
 - Check the **Use constant for B** check box and enter 10 as the constant
- Close the Conditions window and run the model again
- When the breakpoint occurs, click the *Step over* button in the Debugger toolbar to see the value of b calculated. (Note that, in this case, the *Step over* and *Step into* buttons do the same thing, because there is no function call to step into.)

The calculated value of B is zero!

- Double click the Array variable to look at its values – the result is shown at the right.

In the array, elements 10-12 are zero, indicating that the Smedley values for the new larger array have not been calculated. The code should be changed so that no matter what the size of the array is, it will be filled.

Changing the block's code

- Stop the simulation run
- Open the structure of the Start block
- In the Start block's code, change the Initsim message handler to read:

	0
0	-3.0285557e-06
1	-7.4154821e-07
2	0.693146933815
3	1.79175936921
4	3.17805378383
5	4.78749171884
6	6.57925119889
7	8.52516135319
8	10.6046028978
9	12.8018274769
10	0
11	0
12	0

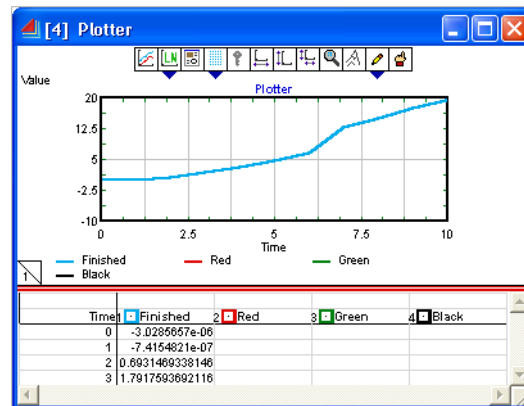
```
on initsim
{
  integer i, length;
  length = GetDimension(array); // get length of array
  // initialize the array so we don't have to recalculate it
  for (i=0; i<length; i++)// use length to limit loop
    array[i] = Log(GammaFunction(i+1));
}
```

Instead of hard coding $i < 10$, this code calls a function to return the array length into a new variable called *length* which is used to limit the calculation loop. Then any time the size of the array is changed the entire array will be filled.

Removing breakpoints

 You can temporarily disable a breakpoint by clicking it once, turning the red dot into an empty circle. You can also remove a breakpoint when you are finished with it.

- ▶ Remove the breakpoint in the Breakpoints window by selecting the red dot and clicking the Delete key.
- ▶ Run the model again to see the correct output:



The well-behaved Smedley function

 Making Array a dynamic array and then resizing it in InitSim would allow the model to be run for any EndTime value without overrunning Array.

Debugger reference

The following sections discuss the Debugger and its windows and dialogs.

Setting a block or library to be in debugger mode

So you can use the Source Code Debugger, ExtendSim has to generate debugging code for a block or blocks. You can either:

- Select a block or blocks on the model worksheet and choose the Develop > Set Breakpoints command (or right-click the block). This automatically generates debugging code for the selected blocks and opens a Set Breakpoints window for each block selected.

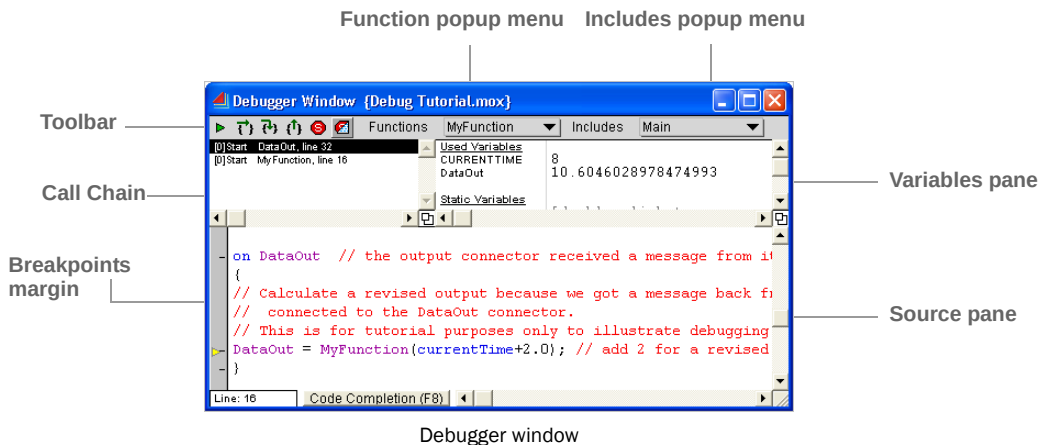
- Open the Library Window for one or more libraries and choose the Library > Tools > Add Debugging Code to Open Library Windows command. (This takes longer than setting a block to debugging mode. But it is useful if you are trying to debug a discrete event model, since the messages will be traceable back to their originators via the Call Chain in the Debugger window.) After the library is in debugging mode, right-click the blocks on the model worksheet that you want to add breakpoints to, or select them and choose the Develop > Set Breakpoints command.

 The only way to access the Debugger window is to run a model with one or more blocks in debugging mode, where there is also either an error message or breakpoints that have been set.

You set a breakpoint in a block's Debugger or Set Breakpoint window by clicking one of the dashes in the left margin. Clicking again toggles the breakpoint off.

When the model is run and execution reaches the breakpoint, the Debugger window will open and show the ModL code, values of variables, and the Call Chain.

Debugger window



Toolbar

There are six tools in the Debugger toolbar. From left to right, they are:

- *Continue* continues execution until the next breakpoint is reached.
- *Step Over* executes a function call without stepping into the function code.
- *Step Into* steps into the function call, tracing execution of the function.
- *Step Out* continues execution until it gets to the caller of the function.
- *Stop* aborts the debugging session.
- *Stop and Go To Code* stops the simulation, opens the block's structure window, and goes to the code being executed.



Debugger toolbar

Popup menus

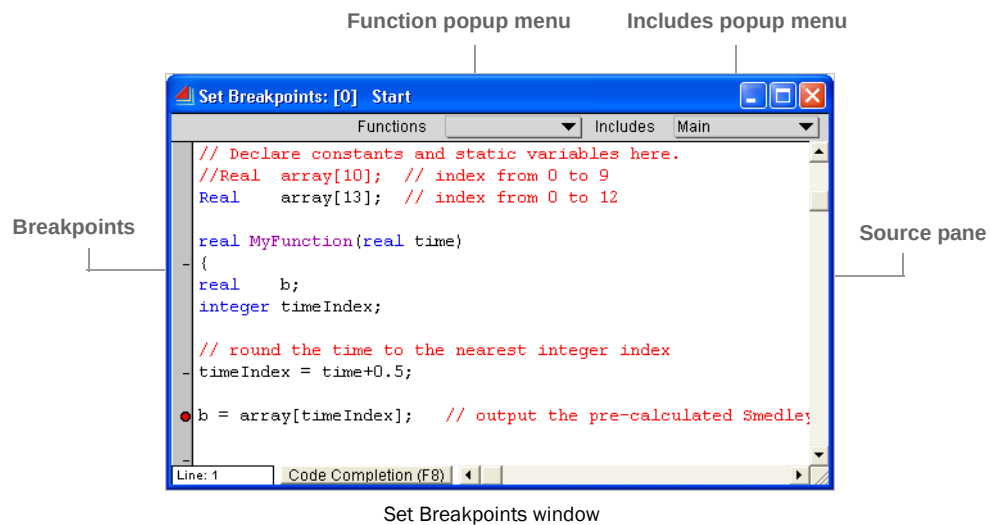
- The Functions popup menu can scroll to any function and shows which function you are in.
- The Includes popup menu shows which include files are used in the block. It allows you to set breakpoints in an include by displaying its contents in the source pane.

Breakpoints

At the left of the window, the Breakpoints margin shows any breakpoints as a red circle. A red half-circle indicates a conditional breakpoint.

To add breakpoints, click on the lines in the Breakpoints margin. To remove a breakpoint, click it again. To add a condition to a breakpoint, use the Breakpoints window, shown on page 195.

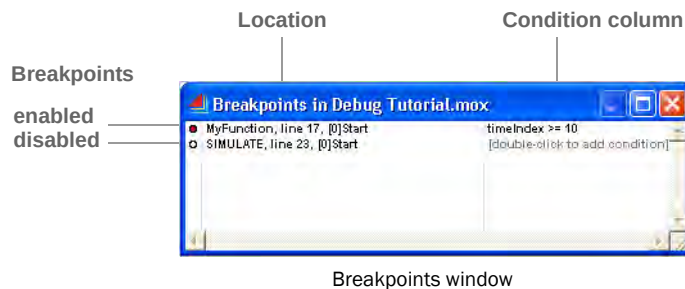
Set Breakpoints window



Model

The popup menus and panes for this window are the same as for the Debugger window on page 194.

Breakpoints window

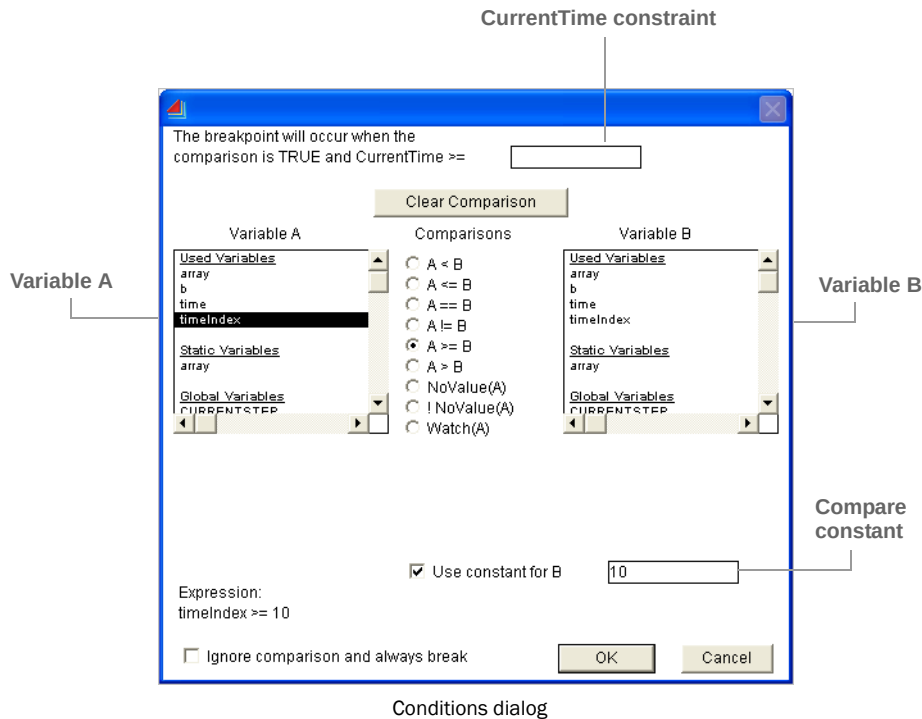


The Breakpoints window is for enabling and disabling, deleting, and adding conditions to breakpoints. For a model with blocks in debugging mode, the Breakpoints window is opened by the command Develop > Open Breakpoints Window.

A red circle indicates an enabled breakpoint. To disable a breakpoint, click the red circle – it becomes an empty circle. To delete a breakpoint, select its location and click the Delete key.

To enable a condition for a breakpoint, double-click its condition column, which opens the Conditions dialog.

Conditions dialog



Conditions allow a breakpoint to be ignored unless the condition is TRUE. This is useful when you get too many breakpoints and you are only interested in a breakpoint that occurs at a specific time or when a variable reaches a specific value.

The Conditions dialog makes it easy to construct a conditional breakpoint with no coding. Just enter a *currentTime* value and any other comparison that might be helpful in narrowing down the problem.

Accessing the dialog

To access the Conditions dialog, double-click in the Conditions column of the Breakpoints window shown on page 195.

To enter a comparison:

- ▶ Select a variable from column A
- ▶ Choose a variable from column B (or check the *Use constant for B* check box and enter a constant in that field)
- ▶ Click the desired comparison operator radio button

You can also click the *Ignore comparison...* check box so that your entered condition will be saved but ignored for the present.

WatchPoints

The WatchPoint condition detects when the variable A changes value and will break to the Debugger whenever that occurs. This is most useful in finding a statement in a different block that is changing a variable incorrectly, and the statement cannot be found easily by normal means.

In order for watchpoints to work correctly, all blocks in a library should be compiled with debugging code turned on. WatchPoints slow model execution as they have to be checked continuously while code is executed.

Arrays

If a variable used in a condition is an array, you need to specify which cell is being referenced. Do this by filling out the Array Indexes dialog. For example, to watch the 2nd row and 3rd column of MyArray, you would enter [1][2] in the Array Indexes dialog.

Debugging tips

- To quickly debug a block, select the block and choose Develop > Set Breakpoints Window. This checks the block structure, recompiles the block for debugging, and opens the Set Breakpoint window.
- To debug the sending of interblock messages, compile the entire library for debugging using the Library > Tools menu. That way the Call Chain will contain the entire chain of messages and it will be obvious which block sent what to whom.
- When finished debugging, you should remove all of the debugging code as it slows execution considerably. To do this, open the Library Windows for the libraries that have blocks in debugging mode and choose Library > Tools > Remove Debug Code in Open Library Windows.
- Use the Debugger to step through code even when you think it is working properly. Try different cases just to make sure that the block is working as you have intended. This will give you confidence in what you have written.

Reference

ModL Variables

A detailed description of the ModL variables
that can be used in your block code

*“Knowledge is of two kinds. We know a subject ourselves,
or we know where we can find information upon it.”*
— Samuel Johnson

This chapter includes a complete list and description of the system and global variables.

- System variables provide information about the state of the simulation
- Global variables are used to pass information between blocks

System variables

System variables give you information about the state of the simulation. They are declared by ExtendSim and can be viewed or modified by any block in a model.

 You can read or write to these variables, but you should be careful when writing to any of them.

Table of system variables

Name	Description
AnimationOn	Tells the state of the Run > Show 2D Animation command. If it is checked, AnimationOn is TRUE (1), otherwise it is FALSE (0). (Note that closed hierarchical blocks always see a value of FALSE until they are opened. This speeds simulations by preventing needless animation when the modeler can't see it anyway.)
AntitheticRandomVariates	If TRUE, ExtendSim's random number functions generate antithetic random numbers.
CurrentScenario	In models where the Scenario Manager is running a series of scenarios, this is the current scenario number. This will be equal to the row in the Scenarios table that is currently providing the factors to the model.
CurrentSense	Used by the sensitivity analysis feature to determine the number of the simulation run for changing sensitivity variables. It is initially set to 0 and is incremented by 1 during each simulation, after ENDSIM. However, you may choose to change CurrentSense to any value you want for whatever reason during ENDSIM – ExtendSim will simply increment it after ENDSIM and use that for its variable calculations. You should not change CurrentSim, and you can always refer to that variable to find the actual simulation number.
CurrentSim	Current simulation number. Its value starts at 0 and increments each step up to NumSims-1. It only has a positive value if you set the Number of runs option in the Simulation Setup or Sensitivity Setup dialogs to a value greater than 1. CurrentSim has a value of -1 when there is no simulation running.
CurrentStep	Current step number. In a continuous simulation, its value starts at 0 and increments each step up to NumSteps. In a discrete event simulation, the value starts at 0 and increments with each event.
CurrentTime	Current time during the simulation. In a continuous simulation, its value starts at StartTime and increments at DeltaTime for each step in the simulation. In a discrete event simulation, the Executive block (Item library) changes the CurrentTime system variable only when processing an event.
DeltaTime	Time increment per step. This value is initialized by the Simulation Setup dialog and represents the basic increment of time used in the simulation. DeltaTime has no meaning in a discrete event or discrete rate simulation.
EndTime	Ending time for the simulation specified in the Simulation Setup dialog. This is the time at which the simulation ends, unless a block stops the simulation with an abort statement or a discrete event or discrete rate simulation runs out of items or events.

Name	Description
GlobalProofStr	Set by the Proof Animation code of a block.
ModernRandom	Tells the state of the random number. If the current random number generator is being used, ModernRandom is 1. If the previous version of the random number generator is being used (for backwards compatibility), ModernRandom is 0. (See "Random numbers" in the ExtendSim User Guide for a discussion about the random number generator.)
MovieOn	This legacy variable is currently unused.
NumScenarios	In models where the Scenario Manager is running a series of scenarios, this is the total number of scenarios that will be run.
NumSims	Number of times the simulation will be repeated, as specified in the Simulation Setup or Sensitivity Setup dialogs.
NumSteps	The total number of steps that will be executed during a continuous simulation. NumSteps is the number of steps entered in the Simulation Setup dialog. NumSteps has no meaning in a discrete event or discrete rate simulation.
OleGlobal, OleGlobalInt, OleGlobalStr	These variables are set by the external application sending an OLEAutomation message to a block. They act as arguments that the block can access when it gets the message.
RandomSeed	Sequence number used to initialize the random number generator; it is set in the Simulation Setup or Sensitivity Setup dialogs. When debugging a simulation, it is sometimes necessary to force the random number generator to produce a repeatable sequence of pseudo-random numbers.
SimDelay	Tells the method of simulation order: 0 for Left to right, 2 for Flow order, 3 for Custom order.
SimMode	0 for manual mode (deltaTime or numSteps values as entered in the Simulation Setup dialog), 1 for autostep fast (use entered values unless model calculates smaller deltaTime), and 2 for autostep slow (divide calculated deltaTime by 5), as specified in the Simulation Setup dialog.
StartTime	Starting time of the simulation at step 0. It is initialized in the Simulation Setup dialog.

Global variables

Global variables are useful for passing information between blocks. They are predefined by ExtendSim and stored within the model. They can be used by any block anywhere in a model. ExtendSim never changes the values of global variables except to initialize them when a new model is created.

There are two types of global variables

- General use global variables have a name that starts with "Global". They can be used any way you want.
- Reserved global variables start with "SysGlobal". These System Globals are controlled by the libraries that are included with ExtendSim and are reserved for use with those libraries.

Name	Type	Use
Global0 through Global19	Real	General
GlobalInt0 through GlobalInt9	Integer	General
GlobalStr0 through GlobalStr9	String	General
SysGlobal0 through SysGlobal29	Real	Reserved
SysGlobalInt0 through SysGlobalInt59	Integer	Reserved
SysGlobalStr0 through SysGlobalStr9	String	Reserved
SysFlowGlobal0 through SysFlowGlobal4	Real	Reserved
SysFlowGlobalInt0 through SysFlowGlobalInt29	Integer	Reserved
SysFlowGlobalStr0 through SysFlowGlobalStr4	String	Reserved

The system globals (the variables that start with “SysGlobal”) are reserved by Imagine That Inc. You can use the SysGlobal variables (for example creating discrete event or discrete rate blocks), but you should only use them in the same way they are used in the blocks that come with Extend-Sim. See “Globals in discrete event models” on page 174 and “Globals in discrete rate blocks” on page 179 for tables that list how the system globals are used during the Simulate, CheckData, and InitSim messages.

Never use the SysGlobal variables for your own purposes; always use the general use globals instead.

Reference

Messages and Message Handlers

A detailed description of the ModL messages

*“Knowledge is of two kinds. We know a subject ourselves,
or we know where we can find information upon it.”*
— Samuel Johnson

Messages and message handlers were introduced on page 39. The following table summarizes each type of ModL message.

Types of message handlers

Message Type	Page	Purpose
Simulation	204	Sent to blocks in simulation order during a simulation run
Model Status	205	Sent to all blocks in a model when the model changes state
Block Status	207	Sent when clicking on a block or connector, when placing, deleting, moving, or pasting a block, or when a model is opened on a different computing platform.
Dialog	209	Sent when the modeler interacts with a block's dialog
Connector	210	Sent via the Connector Message functions, or by the system when a modeler manually changes the number of variable connectors, or when the modeler connects or disconnects a block
Block to block	211	System and user-defined messages sent from a block's ModL code when communicating information to other blocks
Dynamic Link	212	Sent to subscribed blocks when the ExtendSim database or global array part that they are linked to, changes
OLE	212	Sent in response to OLE conditions
3D Animation	212	Sent in response to 3D animation events

Simulation messages

These messages are sent to all blocks during the simulation run *in the following order*:

Message	When sent
ModifyRunParameter	Sent before the simulation run begins to allow changing the Simulation Setup parameters via the SetRunParameter() function (see page 318). Note that this is sent only once even for multiple simulation runs.
SimStart	Sent after ModifyRunParameter and before all other simulation message handlers. See BlockSimStartPriority() function xxx crossref for details.
PreCheckData	Sent to prepare blocks for CheckData, below. Most blocks can ignore this message.
CheckData	This is the best place to check whether all data that is to be used in the block is valid. If the data is bad, you should execute an "abort" statement, so that ExtendSim will select the block and alert you that the data is missing or bad. During this message, connectors that are connected to something else in the model always have a <i>true</i> (any non-zero) value and unconnected ones have a <i>false</i> (zero) value. This makes it easy to check whether a block is connected properly before running a simulation.

Message	When sent
StepSize	After all CheckData messages. If the code needs a particular StepSize, set it here. In continuous models, set the DeltaTime system variable to the maximum step size. ExtendSim queries all blocks and uses the smallest DeltaTime value returned. This message is also used to set up the attribute global arrays in a discrete event model.
InitSim	Just before the simulation starts. If your block's code uses DeltaTime (continuous model only), check it here. Also, allocate any arrays that are dependent on DeltaTime, NumSteps, or any other system variable. This is also a good time to set any static variables, dialog items, or connectors that change when the simulation starts. (Note that ContinueSim, below, is sent instead of InitSim when continuing a saved model's run.)
ContinueSim	This message is sent instead of InitSim if you are continuing a saved model's run. Set up any variables for continuing a saved model run.
PostInitSim	Sent to give blocks another chance to initialize variables that could not be initialized until after the InitSim was sent to all blocks. Most blocks can ignore this message.
Simulate	Every step of the simulation. Note that this message gets sent over and over, not just once. This is where most of the "action" in a block takes place. For instance, you check and change the value of connectors in this message handler. <i>In continuous simulations</i> , the first Simulate message gets sent with CurrentTime=StartTime and CurrentStep=0. For subsequent Simulate messages, ExtendSim adds 1 to CurrentStep and adds DeltaTime to CurrentTime. The last Simulate message occurs when CurrentStep=NumSteps-1 and CurrentTime=EndTime, so each block gets NumSteps Simulate messages. <i>In discrete event and discrete rate simulations</i> , the first Simulate message gets sent with CurrentTime = StartTime and CurrentStep = 0. For subsequent Simulate messages, the Executive block (Item library) controls how CurrentTime is advanced. See the GetSimulateMsgs() function for how to turn off simulate messages in a block under certain conditions.
AbortSim	Sent only if an Abort occurred during the Simulate messages. This notifies the blocks to clean up and lets them know that an abort occurred.
FinalCalc	Sent after the Simulate messages are over and before the BlockReport messages. Used for any final calculations that the block might need before the BlockReport and EndSim messages.
BlockReport	Sent after the FinalCalc messages. If a block receives this message, it has been selected for a report. To organize the reports by block category (see "Block categories" on page 57), ExtendSim cycles through each block type and sends this message to any block selected for a report.
EndSim	At the end of the simulation. Use this to clean up any memory that you used or to reset values. This message gets sent even if the modeler or the block's code aborts the simulation.
SimFinish	Sent after all other simulation message handlers. See BlockSimFinishPriority() function xxx crossref for details.

Model Status messages

When the status of the model changes, these messages are sent to all blocks:

Message	When sent
ActivateModel	Sent to all blocks in a model when that model is brought in front of a different model. This allows blocks to modify their data or appearance if the model is activated.
AnimationStatus	Sent to all blocks in a model when the 2D Animation command changes state. This is useful to change a block's appearance according to the value of the AnimationOn variable.
CloseModel	Sent when the model is being closed, before the decision to save the model is made. This differs from the ModelSave, which is sent only if the modeler decides to save the model.
ModelSave	This message is sent to each block at the beginning of a save. You might use this message handler to dispose of unneeded data before it gets saved.
OldFileUpdate	Before the openModel message, if the file version is older than the application version.
OpenModel	Sent when a model is opened, or to a new hierarchical block that has just been placed on the model worksheet. Use this message handler to set some block variables to values that you only want to reset when a model is opened, not at the beginning of a simulation.
OpenModel2	Sent after all of the blocks have received the OpenModel message, or after the OpenModel message to a new hierarchical block that has just been placed on the model worksheet. Use this message handler to set some block variables to values that you only want to reset when a model is opened, not at the beginning of a simulation.
PauseSimulation	Sent to ALL blocks when the modeler pauses the simulation. Contrast this to ResumeSim which is sent only to the block that the modeler has changed, and ResumeSimAllBlocks sent to all of the blocks in the model.
ResumeSim	This message allows the ModL code to respond to changes before continuing the simulation. It is sent when the modeler: • Either clicks dialog buttons during the simulation • Or, edits a parameter and chooses Run > Resume (or clicks the Resume button in the simulation status bar). Note: This message is sent only to the blocks that have had dialog values edited.
ResumeSimAllBlocks	This message allows the ModL code to respond to changes before continuing the simulation. It is sent when the modeler: • Either clicks dialog buttons during the simulation • Or, edits a parameter and chooses Run > Resume (or clicks the Resume button in the simulation status bar). Note: This message is sent to all of the blocks in the model, as opposed to ResumeSim, above, that is sent only to the blocks edited.
SimOrder-Changed	Sent when the modeler changes connections, connects a new block, deletes connections or blocks, or does anything that changes the simulation order.
SimSetup	Sent when the modeler changes anything in the Simulation Setup dialog.

Block Status messages

These messages are sent to individual blocks:

- When the block is clicked
- When a block is placed, deleted moved, or pasted
- When a block needs to communicate information to the model

Message	When sent
BlockClick	Sent when the modeler clicks on a block. Use GetMouseX(), GetMouseY(), and GetBlockPosition() to find out what portion of the block was clicked. See the Mandelbrot model for an example.
BlockIdentify	Reserved for use by Imagine That Inc.
BlockLabel	Sent to the block that just had its Block Label changed, when the label becomes deactivated. The block can then use the new value of the label.
BlockMove	Upon completion of a move, sent to all blocks that moved. (See “Scripting” on page 319.)
BlockRead	This message must be used only with extreme caution – it can cause a crash if used improperly. It is used to convert version 3.x block data tables to version 4.0 dynamic data tables. See the Information block (Value library) for an example. Call only the GetFileReadVersion() and ResizeDTDuringRead() functions in this message handler.
BlockRight-Click	Sent when a block is right-clicked. Usually used to create a custom popup menu. See the Math block (Value library).
BlockSelect	Sent when a block becomes selected. See also BlockUnselect.
BlockUndelete	Sent to a block when its deletion is undone using the Edit menu Undo command.
BlockUnselect	Sent to a block when it becomes unselected. See also BlockSelect.
Connection-Make	Sent to all newly connected blocks when the modeler makes a connection on the model worksheet. For hierarchical blocks, this is sent to all internal blocks. To prevent the connection from being completed, you can call the <i>Abort</i> statement.
CloneInit	When a clone is placed on the model, sent to the block that owns the clone so that it can be re-initialized. This is useful for static text that needs to be re-initialized when cloned.
EquationCompilePlatform	When an equation tries to execute and it is compiled on the wrong platform, this message handler should just recompile the equation to fix it.
ConnectorRightClick	Sent when the modeler right-clicks a connector. Used, for example, in the Item library to create a popup menu that is used to add History blocks to a connection when that connection is right-clicked.
ConnectorShowHide	Sent when the modeler hides or shows the connectors, either from the toolbar or the Model menu command.
CopyBlock	Sent to all blocks selected before a Copy operation. Useful to dispose of a dynamic array, create a dynamic array, or add a Global Array to the clipboard using the GAClipboard() function, before the copy.

Message	When sent
CreateBlock	When the block is added to a model. This is the place to set initial values for the dialog. Note that this message is only sent to the block when you add it to a model. If you change the CreateBlock code for a block that already is in a model, the changes won't affect the existing blocks, only new blocks added to the model.
DeleteBlock	Sent when the modeler deletes a block from the model. For hierarchical blocks, this is sent to all internal blocks.
DeleteBlock2	Sent to blocks after their connection lines to other blocks have been deleted, right before the block deletion occurs.
DragCloneToBlock	This message gets sent when a modeler drags a clone onto a block and releases the mouse button when the block is highlighted. If you want to get information about the clones, call the GetDraggedCloneList() function. Used in the Optimizer, Statistics, Scenario Manager, and Find & Replace blocks (Value library).
HBlockClose	When a hierarchical block's submodel or structure is closed. For hierarchical blocks, this is sent to all internal blocks.
HBlockFromLibrary	Sent to all enclosed blocks when the H-block from a library is placed on the model. The OpenModel and OpenModel2 messages are then sent to the H-block.
HBlockHelpButton	This message is sent to all the blocks inside a hierarchical block when the H-block Help button is clicked. This can be used to intercept the Help button message for the H-block, and do something of your own implementation instead. Aborting this message will prevent the message from getting to the rest of the blocks in the H-block, and prevent the Help text from opening up.
HBlockMove	Sent to all enclosed blocks when the enclosing H-block is moved.
HBlockOpen	When a hierarchical block is opened, this message is sent to all blocks in its submodel. You can use this to correct animation in a hierarchical block that is opening. If you don't want the hierarchical layout or structure windows to open, execute an Abort statement in the on HBlockOpen message handler in one of the blocks in the submodel.
HBlockSaveToLibrary	Sent to all enclosed blocks when the H-block structure is saved to a library.
HBlockUpdate	Sent to all enclosed blocks when the H-block structure is edited and closed.
HelpButton	Sent when the modeler clicks the Help button in the block's dialog. Executing an Abort statement during this message handler will prevent the ExtendSim Help from opening.
IconView-Change	Sent when the modeler changes the icon view or model class or when a ModL function call from a block changes the icon view or model class. If this message is stopped with an <i>abort</i> statement, the change is not made to that view or class.
MakeSelection-Hierarchical	Sent after a selection of blocks is made into a hierarchical block using the menu command.
MailSlotReceive	Sent repeatedly when there are mailslot messages waiting to be picked up. See the mailslot functions for more information.

Message	When sent
PasteBlock	When the modeler pastes a block onto the model. For hierarchical blocks, this is sent to all blocks within the hierarchical block.
PasteBlock2	Sent after the PasteBlock message has been received by all the blocks pasted.
Plotter0Close, Plotter1Close, Plotter2Close, Plotter3Close	Sent when the modeler closes a plotter window.
TimerTick	Sent by the Timer chore started by the StartTimer function. (See the scripting functions.)

Dialog messages

Dialog messages are the names of dialog items and are sent to the block whenever the dialog is used. When a button in a dialog is clicked or a parameter is unselected (for example, after it has been changed), ExtendSim sends a message with the same name as the dialog item to the ModL code.

For example, assume a block has a “Count” button in its dialog. When that button is clicked, ExtendSim sends the “Count” message to the block. If the block has an “on Count” message handler, it will be executed; if not, nothing happens.

Names for all the named dialog items in a block are listed in the Variables pane at the lower left of the structure window. If a dialog item (such as static text) does not have a dialog item name, it will not be listed in the Variables pane.

Message	When sent
AbortDialog- Message	If the modeler stops the execution of one of your own dialog message handlers or that message handler executes an Abort statement, this message gets sent to the block. Use this as an “exception handler” to clean up after an error occurs.
Cancel	When the Cancel button is clicked. This restores the block to the state it was in the last time it was opened, discarding changes to the dialog. To keep this behavior, do not change the name of the dialog item from “Cancel”.
CellAccept	Sent to a block when the modeler finishes editing any cell in any of the block’s data tables. You can use this to check the data entered in cells in a data table.
DataTable- Hover	Sent to the block when the cursor is hovering over a data table.
DataTableResize	Sent when the datatable resize button is clicked. Use this so the block can be alerted to the new size and inform the modeler if the new size is not acceptable. Use the function WhichDialogItemClicked to determine which datatable resize button has been clicked. Use an Abort statement to prevent the resize.
DataTable- Scrolled	Sent to the owning block when one of its data tables is scrolled.

Message	When sent
DialogClick	Sent when the modeler clicks on a dialog item, before the actual dialog item message is sent to the block. Call WhichDialogItemClicked() to find the name of the item that was clicked. This message is used, for example, to modify the items in a popup menu at the time it is clicked on but before it opens to the modeler.
DialogClose	When the OK or Cancel buttons or the close box are clicked.
DialogItemRefresh	Sent when a dialog item becomes visible, if it was set up by calling the SetVisibilityMonitoring() function.
DialogItemToolTip	Sent when the cursor hovers over a dialog item so that the block can customize the tool tip (e.g format a value in a special way) that the modeler sees.
DialogOpen	When the block's dialog is opened. To display any static text labels that can change based on what is happening in the simulation, set them here. If you don't want the dialog to open, execute an Abort statement – see also Plotter I/O block code.
OK	When the modeler clicks the OK button. No need to do any special handling in this section unless you want to check input data. Use an Abort statement to prevent the dialog from closing if data is not acceptable.
TabSwitch	This message is sent to a block when the block's dialog is switched from one tab to another. This will also be sent when the dialog is first opened, as that is basically treated as a click on the last tab that was previously opened. Use an Abort statement to prevent the tab switch.
YourButton	If you have created a button, radio button, or check box named "YourButton," this message handler is activated when the button is clicked. Use an Abort statement to prevent the action if necessary.
YourItem	Whenever the selection in a dialog is in a parameter or editable text dialog item and you click another item or press the Tab key (taking the selection out of the item), the message handler with that dialog item's name is invoked. This is useful if you want to check the value of the item after it might have been changed. Use an Abort statement to prevent the value from changing.

Connector messages

These messages are sent to individual blocks:

- Via the Connector Message functions
- By the system when a modeler manually changes the number of variable connectors
- When the modeler connects or disconnects a block

 The connector functions start on page 285.

Message	When sent
ConArrayChanged	Sent continuously when a modeler drags a variable connector to increase or decrease the number of connectors in its array. Call ConArrayChangedWhichCon() to return the name of the connector, and then call ConArrayGetNumCons() to find out how many connectors there are in the dragged connector. Abort this message handler to prevent the modeler from adding too many or too few connectors.

Message	When sent
ConArrayChanged-Complete	Sent when the modeler finishes dragging a variable connector, to allow the block to see the final number of connectors chosen.
ConArrayCollapseChanged	Sent when the modeler collapses or expands the variable connectors.
Connection-Break	Sent to all connected blocks when a connection is deleted. Note that a block can receive multiple ConnectionBreak messages when blocks or right angle connections are deleted.
Connection-Click	When a connection line is clicked, sent to all blocks connected so that they can react (e.g. report a value). The function ConnectorToolTipWhich() returns the connector number for that connection.
Connection-Make	Sent to all newly connected blocks when the modeler makes a connection on the model. For hierarchical blocks, this is sent to all internal blocks. The function ConnectorToolTipWhich() returns the connector number for that connection. To prevent the connection from being completed, you can call the Abort statement in this message handler.
Connector-Name	Connector message. When the connector on a connected block receives a message from another block using the connector message functions.
ConnectorRightClick	Sent when the modeler right-clicks a connector. Used, for example, in the Item library to create a popup menu that is used to add History blocks to a connection when that connection is right-clicked.
Connector-ToolTip	Sent when the cursor hovers over a connector so that the block can customize the tool tip (e.g format a value in a special way) that the modeler sees.

Block to block messages

Model type dependent and user-defined messages sent from a block's ModL code to communicate information to other blocks:

Message	When sent
AttribInfo	Sent by the Executive block when attribute information has been changed.
BlockReceive0 through 9	Used by the Discrete Event library as system messages.
ClearStatistics	When a block's statistical variables need to be reset. This message is typically sent by a Statistics block (Value Library). See the Activity block (Item library) for an example of receiving this message.
DEExecutiveArrayResize	Sent by the Executive block to notify Item or Flow blocks that item arrays have been resized.
BlockTableInfo	Sent by some of the blocks to query data table size. Not sent by ExtendSim.
ProofAnimation	This message is used by the blocks that are interacting with Proof Animation to produce proof animation functionality.

Message	When sent
QueueFunction	Sent by the QueueTools block (Utilities library) and the Queue blocks (Item library) to communicate with each other.
ShiftSchedule	Sent by a block to all the blocks in the model when a shift schedule has been changed.
UpdateStatistics	When a block's statistical variables need to be recalculated and updated. This message is typically sent by a Statistics block (Value library). See the Activity block (Item library) for an example of receiving this message.
UserMsg0 through 9	User-defined message that is not used by any ExtendSim libraries. Use the SendMsgToBlock function to send these messages from another block.
FlowBlockReceive0-FlowBlockReceive19	Used by the Rate library to communicate information between blocks.

Dynamic Link messages

When the part they are linked to changes, these messages are sent to individual blocks that are subscribed to an ExtendSim Database or global array.

 Database functions start on page 333; global array functions start on page 355.

Message	When sent
LinkContent	If linked or subscribed to part of an ExtendSim Database or global array, this is sent when the data changes within that part. This facilitates recalculation only when the data changes.
LinkStructure	If linked or subscribed to part of an ExtendSim Database or global array, this is sent when the structure (e.g. number of fields, records, names, etc.) changes within that part. Use the DILinkUpdateInfo() and DILinkUpdateString() functions to find out what changed.

OLE messages

These messages that are sent to an individual block on particular events.

Message	When sent
AdviseReceive	Sent to the block when it receives updated data from an advise conversation (see the functions for “Interprocess Communication (IPC)” on page 239).
OLEAutomation	Sent to a block when the BlockMsg automation method is invoked. See “BlockMsg” on page 116.

3D messages

Sent to an individual block during 3D events.

 The 3D animation functions start on page 264.

Message	When sent
E3DCollision	Sent when two objects are about to collide in the E3D window. Used to control whether or not a collision occurs or otherwise to respond to a collision. (Be aware that this message can occur multiple times for each potential collision.) If a collision is about to happen, the block associated with the 3D object that is about to collide with another 3D object receives an E3DCollision message. The new function E3DMessageInfo() (xxx crossref) allows the programmer to request the 3D object ID of the object that is about to collide and/or the object that is about to be collided with. The E3DCollision message has been defined such that aborting the message will tell the E3D engine that the collision should not happen. This makes the objects noncollidable for the purposes of that collision. The result is that they will visually pass right through each other or, in the event that the collision would have just been the two objects brushing edges, they will just pass right by each other. This functionality is implemented in the Item library to reduce unintentional collisions that could cause objects to move incorrectly in E3D models.
E3DInit	Sent to all blocks in the active model when the 3D window is opened.
E3DStart	Sent to all blocks in the active model when the first 3D post event in a simulation is processed.
E3DFinish	Sent to all blocks in the active model when the last 3D post event in a simulation is processed.
E3DClose	Sent to all the blocks in the active model when the 3D window is closed.
E3DObjectClick	Sent to the block associated with a 3D object when that object is clicked in the 3D window. This can be used in conjunction with the E3DObjectClicked function to respond to clicks on 3D objects.
E3DModeSwitch	Sent to all blocks when the 3D mode (QuickStart, Concurrent, or Buffered) is changed.
E3DReachDestination	Sent to the block that is associated with a 3D object when the object reaches its destination.
E3DCollision	Sent to the block that is associated with a 3D object when the object collides with another object.
E3DObjectMoved	Sent to the block that is associated with a 3D object when that object is moved/resized in the E3D editor.
E3DTick	Sent to the block periodically when the E3DTimer has been started with E3DStartTick.

Reference

ModL Functions

A detailed description of the ModL functions
that can be used in your block code

*“Knowledge is of two kinds. We know a subject ourselves,
or we know where we can find information upon it.”*
— Samuel Johnson

This chapter includes a complete list of the ModL functions. These functions can be called in the blocks that use equations (Equation, Equation(I), Queue Equation, and Optimizer) and when creating new blocks.

ModL function overview

The rest of this chapter is a description of all the functions in ModL, listed by type. Within types, the functions are grouped by category. Within categories, the functions are listed alphabetically.

Function Type	Page	Categories
Math	217	Basic math, Trigonometry, Complex numbers, Statistical and random distributions, Financial, Integration, Matrices, Bit handling, Equations
I/O	232	File I/O (formatted), File I/O (unformatted), Internet Access, Interprocess Communication, OLE/COM, Mailslot, ODBC, Serial I/O, Other drivers, XCMDs, DLLs, Alerts and prompts, User inputs
Animation	259	2D Animation visible on the model worksheet and 3D Animation visible in the E3D window.
Blocks and inter-block communication	282	Block numbers/labels/names/type/position, Block connectors and connection information, Variable connectors, Connector tool tips, Dialog items and dialog items from other blocks, Block data tables, Dynamic linking, Dynamic text items, Dialog item tool tips, Block dialogs (opening and closing), Messages to blocks (sending and receiving), Icon classes and views. Also see Scripting, below
Models and notebooks	314	Models and notebook information, simulation parameters, DE Modeling Using Equation Blocks.
Scripting	319	Building and running a model remotely. Also see “Blocks and inter-block communication” and “Models and notebooks” above
Reporting	326	Block reporting
Plotting	326	General plotting, scatter plots
Arrays, Queues, Delays, Linked lists, and String lookup tables	352	Dynamic arrays, Passing arrays, Global arrays, Queues, Delay lines, Linked list data structures, and String lookup tables
Database	333	Manipulating databases and database data with the ExtendSim database
Miscellaneous	369	Strings, Attributes, Calendar Date and time, Time units, Timer functions, Debugging, Help, Platforms and versions, Lego Mindstorm control

 ModL functions are also listed in the ExtendSim Help command alphabetically and by type, including their arguments. You can copy a function from there to use in your code.

Code completion

When you start to type a function or message handler name in the structure window or include file of a block, you can click the Code Completion button at the bottom of the script pane or press F8 repeatedly until the desired function appears. The function will be entered with sample arguments.

Overriding

Functions can be overridden by being re-declared any number of times below the first declaration. This is useful in that include files can have basic forms of functions which can be re-declared and overridden in the main block code.

Type conversion of arguments

All ModL functions expect their arguments to be the data type specified in the function definition. If you use another type, ModL will automatically convert the argument to the expected type before the function is called. For the functions that take no arguments, the parentheses are still required.

Static data limits

Each function has a limit of 32,560 bytes of data for locally defined static data. Dynamic arrays, global arrays, and database tables are not part of the count and are the more common and usually more useful method used to allocate large data structures.

Function returns

Except for void functions, which do not return values, all functions return values that are real, integer, or string. The type of value returned is indicated in the third column of the function tables as:

Return	Meaning
R	Real or Double (8 byte double)
I	Integer or Long (4 byte long integer)
S	String (up to 255 characters)
V	Void function (no value returned)

Math functions

Basic math

These functions perform numerical operations on their arguments.

Basic Math	Description	Return
Ceil(real x)	Nearest integral real with a value greater than or equal to x . For example: Ceil(1.3) returns 2.0, and Ceil(-1.3) returns -1.0	R
Erf(real x)	Returns the erf value of the variable x . Erf is the Error function, which is a special case of the incomplete gamma function. See Numerical recipes in C.	R
Exp(real x)	e^x	R
FFT(real array[n][2], inverse)	Replaces the array with the real and imaginary parts of the fast Fourier transform (see below).	V

Basic Math	Description	Return
FixDecimal (real value, integer fixFigs)	Sets the number of figures after the decimal point to fixFigs and returns the result.	R
Floor(real x)	Nearest integral real with a value less than or equal to x . For example: Floor(1.3) returns 1, and floor(-1.3) returns -2.	R
GammaFunction(real x)	Gamma function for x . Do not confuse this with the Gamma distribution function.	R
Int(real x)	Nearest integer after rounding toward 0. For example: Int(1.3) returns 1, and Int(-1.3) returns -1.	I
Integerabs(i)	Non-negative integer containing the absolute value of the integer i .	I
Log(real x)	Natural (base e) log of x .	R
Log10(real x)	Base 10 log of x .	R
Log2(real x)	Returns the log base 2 value of the variable x .	R
Max2(real x, real y)	Maximum of the two arguments.	R
Min2(real x, real y)	Minimum of the two arguments.	R
NearlyEqual(real x, real y, integer precision)	Returns true if the x and y arguments are close enough to equal each other. The precision argument specifies the number of significant figures to compare the two numbers.	I
NearlyGreater-Than(real x, real y, integer precision, integer equal)	Returns true if x is greater than y . If <i>equal</i> is TRUE, NearlyEqual() is used, with the <i>precision</i> argument, to give an equal or greater result. If <i>equal</i> is FALSE, x has to be greater than y by the <i>precision</i> number of significant figures. See the NearlyEqual() function.	I
NearlyLessThan(real x, real y, integer precision, integer equal)	Returns true if x is less than y . If <i>equal</i> is TRUE, NearlyEqual() is used, with the <i>precision</i> argument, to give an equal or less than result. If <i>equal</i> is FALSE, x has to be less than y by the <i>precision</i> number of significant figures. See the NearlyEqual() function.	I
NoValue(real x)	1 (True) if x has no value (is blank) or 0 (False) if a value has been assigned to x .	I
Pow(real x, real y)	x^y . The same results may be achieved with the ^ operator, as in x^y . Note that Pow(0,0) is undefined.	R
Realabs(real x)	Non-negative real number containing the absolute value of x .	R
Realmod(real x, real y)	x modulo y (that is, the remainder of x divided by y).	R
Round(real x, integer sigFigs)	Rounds x to the significant figures specified in the <i>sigFigs</i> argument.	R
Sqrt(real x)	Square root of x .	R

The FFT function replaces the real array argument with the real and imaginary parts of the FFT. n must be a power of 2. If inverse is TRUE, the inverse FFT is calculated. The real values are con-

tained in `array[n][0]`, and the imaginary values are contained in `array[n][1]` (these two are denoted together as “`array[n][i]`”). `array[0][i]` is zero frequency. `array[(n/2)-1][i]` is the most positive and most negative frequency. `array[n-1][i]` is the negative frequency just below 0. See the “four1” routine in Press, *Numerical Recipes in C*, for more information on this algorithm.

Trigonometry

These functions assume that the argument represents an angle in radians.

Trig	Description	Return
<code>Acos(real x)</code>	Arccosine of x , where x is any real number between -1 and +1 inclusive.	R
<code>Asin(real x)</code>	Arcsine of x , where x is any real number between -1 and +1 inclusive.	R
<code>Atan(real x)</code>	Arctangent of x , where x is a real number. The value returned is between $-\pi/2$ and $\pi/2$ radians.	R
<code>Atan2(real y, real x)</code>	Arctangent of y/x , where x is non-zero. The value returned is between $-\pi$ and π radians.	R
<code>Cos(real x)</code>	Cosine of angle x .	R
<code>Cosh(real x)</code>	Hyperbolic cosine of angle x .	R
<code>Sin(real x)</code>	Sine of angle x .	R
<code>Sinh(real x)</code>	Hyperbolic sine of angle x .	R
<code>Tan(real x)</code>	Tangent of angle x .	R
<code>Tanh(real x)</code>	Hyperbolic tangent of angle x .	R

Complex numbers

These functions operate on complex numbers composed of two-element real arrays (U, Z, and result), where `U[0]` is the real part and `U[1]` is the imaginary part. All arguments and results are complex numbers expressed as two-element real arrays:

```
real u[2], z[2], result[2];
```

Complex numbers	Description	Return
<code>AddC(result, U, Z)</code>	$\text{result} = U + Z$	V
<code>DivC(result, U, Z)</code>	$\text{result} = U / Z$	V
<code>MultC(result, U, Z)</code>	$\text{result} = U * Z$	V
<code>SubC(result, U, Z)</code>	$\text{result} = U - Z$	V

Statistics and random distributions

These functions can be used both to generate random inputs and to gather statistical information from the results of simulations. In the following functions, the following are used as arguments:

```
real prob, rate, mean, stdDev;
integer nTrials, kthEvent, kthSuccess;
```

Statistics/ distributions	Description	Return
DBinomial(real prob, integer nTrials)	Number of successes out of <i>nTrials</i> , each with a probability of success of <i>prob</i> .	R
DExponential(real rate)	Interval between events. <i>rate</i> is the expected (mean) number of events per period.	R
DGamma(integer kthEvent)	Waiting time to the <i>kthEvent</i> in a Poisson process of mean equal to 1.	R
DLogNormal(real mean, real stdDev)	Positively skewed distribution.	R
DPascal(real prob, integer kthSuccess)	Geometric distribution if <i>kthSuccess</i> equals 1. This returns the number of trials needed for the <i>kthSuccess</i> of an event with probability of <i>prob</i> .	R
DPoisson(real rate)	Number of times an event occurs within a given period. <i>rate</i> is the expected (mean) number of events per period.	R
Gaussian(real mean, real stdDev)	Real random member of a Gaussian (normal) distribution with the specified <i>mean</i> and standard deviation.	R
Mean(real array[], integer i)	Arithmetic mean of the first <i>i</i> members of the single-dimensional array.	R
Random(real i)	Uniform pseudo-random integer in the range 0 to <i>i</i> -1 using the random seed specified in the Simulation Setup dialog. For example, Random(6) returns an integer in the range 0 through 5, inclusive. Random(i) assumes that <i>i</i> is an integer. For the Integer (uniform) function.	I

Statistics/ distributions	Description	Return
RandomCalculate (integer distribu- tion, real arg1, real arg2, real arg3)	Returns a random number given a distribution and up to three argu- ments. If a given distribution does not use all three arguments, a zero should be entered for the unused argument(s). The following numbers are used to define the distribution: Beta 1 Binomial 2 Erlang 3 Exponential 4 Gamma 5 Geometric 6 Hyperexponential 7 Integer Uniform 8 Loglogistic 9 Lognormal 10 Negative Binomial 11 Normal 12 Pearsonv 13 Pearsonvi 14 Poisson 15 Real Uniform 16 Triangular 17 Weibull 18 ExtremeValue1A20 ExtremeValue1B 21 JohnsonSB 22 JohnsonSU23 Laplace24 Rayleigh25 InverseWeibull26 Logarithmic27 Hypergeometric28 Chisquared29 PowerFunction30 Cauchy31 Logistic32 InverseGaussian33 Pareto34	R

Statistics/ distributions	Description	Return
RandomCheck- Param (integer dis- tribution, real arg1, real arg2, real arg3, integer reportError)	Checks that the arguments passed in are valid for the given distribu- tion. The distribution argument is defined using the numbers above. Displays an error message if reportError is TRUE and the arguments are not valid. If reportError is FALSE, the function returns the fol- lowing: 0 Successful -1 Mean must be greater than 0 -2 Probability must be between 0 and 1 -3 Argument must be between 0 and 1 -4 Shape must be greater than 0 -5 Shape2 must be greater than 0 -6 Most likely value must be between Max and Min values -7 Min must be less than Max -8 Arg1 is negative or less than 1.0e-15 -9 Arg2 is negative or less than 1.0e-15 -10 Arg3 is negative or less than 1.0e-15 -11 Arg1 >= Arg2 -12 Arg1 > 1.0 -13 Arg2 > 1.0 -14 Arg3 > 1.0	I
RandomGetModel- SeedUsed()	The actual seed used for the model. If the Simulation Setup dialog had 0 entered for the seed, this will return the actual randomized seed used, not 0. If a non-zero number was entered, this will return that number. This is different than reading the RANDOMSEED global variable, which just shows the actual number entered in the Simulation Setup dialog, including 0, and doesn't show the actual randomized seed used for running the model.	I
RandomGetSeed()	Current seed value or current state of the random number generator. Used for saving and restoring the random state when using different seeds. See RandomSetSeed() below.	I
RandomReal()	Uniform pseudo-random real number x, in the range {0.0 <= x < 1.0} using the random seed specified in the Simulation Setup dialog.	R
RandomSet- Seed(integer i)	Sets the seed value or saved state of the random number generator. Used for saving and restoring the random state when using different seeds. See RandomGetSeed() above.	V
SeedListClear()	Clears the list of the seed values.	V
SeedListRegis- ter(real blockNum- ber, integer seed)	Keeps a list of the seed values. Returns a BLANK for success, mean- ing the item was entered in the list, or returns the blockNumber of the block that already posted the seed value. Note: Block number is a real so we can register DB cells that gener- ate random numbers using a DB attribute. DB attributes will appear as negative numbers.	R
StdDevPop(real array[], integer i)	Population standard deviation of the first i members of the single- dimensional array.	R

Statistics/ distributions	Description	Return
StdDevSample(real array, integer i)	Sample standard deviation of the first <i>i</i> members of the single-dimensional array.	R
TStatisticValue(real probability, integer degreesOfFreedom)	Returns an accurate approximation of the point on the students t-distribution for a given single tail <i>probability</i> and number of <i>degreesOfFreedom</i>	R
UseRandomized-Seed()	Forces a randomized seed for the current simulation run, overriding any fixed seed entered into the Simulation Setup dialog. Must be called in CHECKDATA message handler.	V

See “Random numbers” in the main ExtendSim User Guide for information about how ExtendSim generates random numbers.

Financial

These functions calculate the unknown parameter in loan and annuity calculations given four known parameters. The financial functions use standard financial arguments:

Argument	Meaning
pv	Present value of a cash-flow (real)
fv	Future value of a cash-flow (real)
pmt	Amount of one payment (real)
ratePer	Interest rate per period (real)
nPer	Number of periods (integer)

pv, fv, and pmt treat cash received as a positive value and cash paid out as a negative value.

Note that ratePer must match the length of the periods indicated by nPer. For example, if nPer is the number of months, ratePer is the interest per month.

payAtBegin is a flag variable. If payAtBegin is TRUE (non-zero), payments occur at the beginning of the period. If FALSE (0), payments occur at the end of the period.

Financials	Description	Return
CalcFV(ratePer, nPer, pmt, pv, payAtBegin)	Future value	R
CalcNPER(ratePer, pmt, pv, fv, payAtBegin)	Number of periods	R

Financials	Description	Return
CalcPMT(ratePer, nPer, pv, fv, payAtBegin)	Payment	R
CalcPV(ratePer, nPer, pmt, fv, payAtBegin)	Present value	R
CalcRate(nPer, pmt, pv, fv, payAtBegin)	Interest rate. If the rate cannot be calculated using the values of the arguments, the function returns a noValue (BLANK) result.	R

Integration

These functions integrate a stream of values. For instance, you may want to integrate values coming from inputs during a simulation. In the integration functions, the array used in the integration calculations must be declared a static array of four real values:

```
real array[4];
real initConditions, inputValue, deltaTime;
```

Integration	Description	Return
IntegrateEuler(real array[4], real inputValue, real deltaTime)	Value of a Euler integration in progress. The array must be initialized with the IntegrateInit function. The algorithm is a backward Euler: $out = out + inputValue * deltaTime$.	R
IntegrateInit(real array[4], real initConditions)	Initializes the array for integration. Call this function in the InitSim message handler if you are going to use the integration functions during a simulation. <i>initConditions</i> specifies the starting real value for the integration.	V
IntegrateTrap(real array[4], real inputValue, real deltaTime)	Value of a Trapezoidal integration in progress. The array must be initialized with the IntegrateInit function. The algorithm is a first-order trapezoid: $out = out + deltaTime * (previousInputValue + inputValue) / 2$.	R

Matrices

Many intricate tasks can be written in just a few matrix operations. For example, solving simultaneous equations, curve fitting data, finding the roots of a polynomial, and coordinate transformations may all be done with matrices. For further information about matrices, see *Matrix Methods and Applications* by Groetsch and King, (Prentice Hall, 1988).

The matrices used by these functions are in the form **matrix[m][n]**, where m is the number of rows and n is the number of columns. Vectors are of the form **vector[n]**, where n is the number of elements in the vector. Matrices are supported up to 1500x1500.

The complex numbers returned by the Roots and EigenValues functions are in an array that is declared as **array[n][2]**. This makes array[k][0] the real part, and array[k][1] the imaginary part.

Complex versions of the matrix functions end in the letter C. All arguments to these complex functions are complex.

Declarations for matrices are as follows:

```

real matrix[rows][columns];           // real matrix;
                                         //   also matrixA, matrixB, ma-
trixR
real vector[rows];                     // real vector; also values
real matrixC[rows][columns][2];       // complex;
                                         //   also matrixAC, matrixBC, ma-
trixRC
real vectorC[rows][2];                 // complex; also valuesC
real resultC[2];                       // a complex number
integer n, m;                          // dimensions

```

Rows and columns can be bigger than needed. Just specify desired rows and columns when calling functions.

Matrices	Description	Return
Conju- gateC(matrixRC, matrixAC, integer m, integer n)	Returns the conjugate values of <i>matrixAC</i> in <i>matrixRC</i> . Both <i>matrixAC</i> and <i>matrixRC</i> are <i>m</i> by <i>n</i> by 2 (complex) matrices.	V
Determi- nant(matrixA, inte- ger m)	Value of the determinant of <i>matrixA</i> , which is an <i>m</i> by <i>m</i> matrix. If the matrix is singular, the function returns a NoValue.	R
Determi- nantC(resultC, matrixAC, integer m)	Complex version of Determinant which returns its complex result in <i>resultC</i> . If the matrix is singular, the function returns a NoValue in <i>resultC</i> .	V
EigenValues(Val- uesC, matrixA, inte- ger m)	Eigenvalues of <i>matrixA</i> (<i>m</i> by <i>m</i>) are placed into the complex array <i>ValuesC</i> . <i>ValuesC</i> is of length <i>m</i> by 2 (complex). <i>matrixA</i> may be nonsymmetric. (This routine is based on the EISPACK method of creating a Hessenberg matrix and iterating that matrix into a diagonal matrix through similarity transformations). If the matrix is singular, the function returns TRUE.	I
Identity(matrixR, integer m)	Creates an <i>m</i> by <i>m</i> <i>matrixR</i> with 1s along the diagonal and 0s above and below the matrix diagonal.	V
Identi- tyC(matrixRC, inte- ger m)	Complex version of Identity.	V
Inner(vectorA, vec- torB, integer m)	Value of the inner or dot product of <i>vectorA</i> and <i>vectorB</i> of length <i>m</i> .	R
InnerC(resultC, vectorAC, vec- torBC, integer m)	Complex version of Inner which returns its complex result in <i>resultC</i> .	V

Matrices	Description	Return
LUde- comp(matrixR, matrixA, integer m)	Returns the LU decomposition of input <i>matrixA</i> in <i>matrixR</i> . If the matrix is singular, the function returns TRUE. Since LU factorization in LUdecomp permutes rows to obtain the best pivots, the LU matrix returned is only directly applicable to diagonally dominant matrices. The result of LUdecomp can be used in general once the permutation is accounted for.	I
LUde- compC(matrixRC, matrixAC, integer m)	Complex version of LUdecomp. If the matrix is singular, the function returns TRUE.	I
MatAdd(matrixR, matrixA, matrixB, integer m, integer n)	Returns in <i>matrixR</i> the addition of <i>matrixA</i> to <i>matrixB</i> . <i>m</i> is the number of rows, <i>n</i> is the number of columns.	V
MatAddC(matrixR C, matrixAC, matrixBC, integer m, integer n)	Complex version of MatAdd.	V
MatCopy(matrixR, matrixA, integer m, integer n)	Copies <i>matrixA</i> (of dimension m by n) into <i>matrixR</i> .	V
Mat- CopyC(matrixRC, matrixAC, integer m, integer n)	Complex version of MatCopy.	V
MatInvert(matrixR, matrixA, integer m)	<i>MatrixR</i> is the inverse of <i>matrixA</i> , which is an <i>m</i> by <i>m</i> square matrix. If the matrix is singular, the function returns TRUE.	I
MatIn- vertC(matrixRC, matrixAC, integer m)	Complex version of MatInvert. If the matrix is singular, the function returns TRUE.	I
MatMat- Prod(matrixR, matrixA, integer mA, integer nA, matrixB, integer mB, integer nB)	<i>MatrixR</i> (<i>mA</i> by <i>nB</i>) is created from the product of <i>matrixA</i> (<i>mA</i> by <i>nA</i>) and <i>matrixB</i> (<i>mB</i> by <i>nB</i>). Note that <i>nA</i> must equal <i>mB</i> .	V
MatMat- ProdC(matrixRC, matrixAC, integer mA, integer nA, matrixBC, integer mB, integer nB)	Complex version of MatMatProd.	V

Matrices	Description	Return
MatScalar-Prod(matrixR, matrixA, integer m, integer n, B)	This matrix scalar product creates <i>matrixR</i> by multiplying <i>matrixA</i> by <i>B</i> . <i>MatrixA</i> is <i>m</i> by <i>n</i> , and <i>B</i> is a scalar (single number).	V
MatScalar-ProdC(matrixRC, matrixAC, integer m, integer n, BC)	Complex version of MatScalarProd.	V
MatSub(matrixR, matrixA, matrixB, integer m, integer n)	Returns in <i>matrixR</i> the difference of <i>matrixB</i> from <i>matrixA</i> . <i>m</i> is the number of rows, <i>n</i> is the number of columns.	V
Mat-SubC(matrixRC, matrixAC, matrixBC, integer m, integer n)	Complex version of MatSub.	V
MatVectorProd(vectorR, matrixA, integer m, integer n, vectorB)	Creates an <i>m</i> length vector (<i>vectorR</i>) by multiplying <i>matrixA</i> (<i>m</i> by <i>n</i>) by <i>vectorB</i> (<i>n</i>).	V
MatVector-ProdC(vectorRC, matrixAC, integer m, integer n, vectorBC)	Complex version of MatVectorProd.	V
Outer(matrixR, vectorA, integer m, vectorB, integer n)	Creates an <i>m</i> by <i>n</i> matrix (<i>matrixR</i>) from the product of <i>vectorA</i> and <i>vectorB</i> . <i>vectorA</i> is of length <i>m</i> and <i>vectorB</i> is of length <i>n</i> .	V
OuterC(matrixRC, vectorAC, integer m, vectorBC, integer n)	Complex version of Outer.	V
Roots(real values[][2], real p[], integer n)	Calculates the roots of polynomial <i>p</i> of order <i>n</i>. These roots are returned in the complex array values. The coefficients of the polynomial <i>p</i> are in an array beginning with the coefficient of the second highest power. The coefficient of the highest power is assumed to be 1. For example: $p[] \rightarrow x^n + p[0]*x^{(n-1)} + p[1]*x^{(n-2)} \dots + p[n-1]$ Note that both values and p can be length n or greater. If the matrix of the polynomial is singular, the function returns TRUE.	I
Transpose(matrixR, matrixA, integer m, integer n)	Creates the transpose of <i>matrixA</i> in <i>matrixR</i> . The input matrix is <i>m</i> by <i>n</i> and the result matrix is <i>n</i> by <i>m</i> .	V

Matrices	Description	Return
TransposeC(matrixRC, matrixAC, integer m, integer n)	Complex version of Transpose.	V

Bit handling

The bit-handling functions return the integer value result of the operation. In these functions, bit 0 is the most significant bit and bit 31 is the least significant bit of ModL's 32-bit integers.

Declarations for bit handling are as follows:

integer bitNum, Count;

Bit handling	Description	Return
BitAnd(integer i, integer j)	Bitwise AND of the two integers.	I
BitClr(integer i, integer bitNum)	Sets the bit numbered bitNum in <i>i</i> to 0 and returns the result.	I
BitNot(integer i)	Bitwise NOT of the integer.	I
BitOr(integer i, integer j)	Bitwise OR of the two integers.	I
BitSet(integer i, integer bitNum)	Sets the bit numbered bitNum in <i>i</i> to 1 and returns the result.	I
BitShift(integer i, integer Count)	Shifts <i>i</i> by <i>Count</i> bits. If <i>Count</i> is positive, this shifts to the left (multiply <i>i</i> by 2^{Count}); if <i>Count</i> is negative, it shifts to the right (divide <i>i</i> by 2^{Count}). 0s are shifted in.	I
BitTst(integer i, integer bitNum)	TRUE if the bit numbered <i>bitNum</i> is set to 1, FALSE if <i>bitNum</i> is 0.	I
BitXor(integer i, integer j)	Bitwise Exclusive OR of the two integers.	I

Equations

These functions let you create a block in which the user can enter equations built on ModL code and the ExtendSim built-in functions. (Note that user-defined functions must be defined in include files in order to be used in an equation block.) See “Dynamic text items” on page 304 if you want to implement much larger (text edit boxes are initially limited to 255 characters) user-entered equations (up to 32000 characters). See the Equation block (Value library) for an example of using these and the dynamic text item functions in building your own equation block.

See the EquationSetStatic() and EquationGetStatic() functions to use “remembered” values in your equations.

-  For cross-platform compatibility, if you build blocks that use the equation functions your code needs to detect if the model is being opened on a different platform. Therefore, in addition to your

own specific tests, you should test `dynArrayName` in the `CheckData` message handler of any block that uses the equation functions:

```
On CheckData
{
  if (getDimension(dynArrayName) == 0)
    EquationCompile(...);
  ...
}
```

 Because `ExtendSim` will detect a change of platform and will therefore dispose of `dynArrayName`, the code must check and recompile the equation again when the simulation is run. For a detailed example, see the `Equation` block (Value library).

Equations	Description	Return
<code>EquationCalculate(real input1, ..., real input10, integer dynArrayName[])</code>	Calculates an equation compiled by the <code>EquationCompile</code> function. The arguments can be integer or real values (such as input connectors). This function returns the real value assigned to the output variable. This function should be called whenever you need a new value from the equation, and is usually placed in the <code>Simulate</code> message handler. Also see “Dynamic text items” on page 304 for larger equations than 255 characters. If you need unlimited input and output variables, see <code>EquationCalculateDynamicVariables</code> , below.	R
<code>EquationCalculate20 (real input1, real input2, ... real input19, real input20, integer dynArrayName[])</code>	This works the same as the <code>EquationCalculate</code> function, except it allows 20 inputs. Also see “Dynamic text items” on page 304 for equations larger than 255 characters. If you need unlimited input and output variables, see <code>EquationCalculateDynamicVariables</code> , below.	R
<code>EquationCalculateDynamic(real arrayValues[], integer equationArray[])</code>	Calculates a dynamic text equation using the input arguments from the array <code>arrayValues</code> . Up to 20 input arguments are allowed.	I
<code>EquationCalculateDynamicVariables(real inputDynArray, real outputDynArray, integer codeDynArray)</code>	This function allows unlimited input and output variables. Input-DynArray values and outputDynArray real array are used in the equation like this example: <code>outputsName[i] = inputsName[i]</code> ; Make sure that you have enough elements allocated in the input and output dynamic arrays. See the <code>Equation</code> block in the Values library to see how to use this function.	I

Equations	Description	Return
EquationCompile(string inputVarName1, ..., string inputVarName10, string output VarName, string equation, integer tabOrder, integer dynArrayName[])	Compiles an equation that is typed into an editable text item or string variable in a dialog. The variables <i>inputVarName1</i> through <i>inputVarName10</i> , <i>outputVarName</i> , and <i>equation</i> are all strings; <i>tabOrder</i> is an integer and <i>dynArrayName</i> is an integer dynamic array. The user's equation can use any valid ModL function or statement, including defining new variables. The compiler outputs error messages and puts the insertion point at the error in the dialog item identified by <i>tabOrder</i> if it is a dialog editable text item. The compiler stores the machine code for the compiled equation in <i>dynArrayName</i> . This function should be called only when the equation or variable names are changed. Returns TRUE if there was an error in the equation. Also see "Dynamic text items" on page 304 for larger equations than 255 characters. If you need unlimited input and output variables, see EquationCompileDynamicVariables, below.	I
EquationCompile20 (string inputVarName1, ..., string inputVarName20, string output VarName, string equation, string equation2, integer tabOrder, integer tabOrder2, integer dynArrayName[])	This works the same as the EquationCompile function, except it allows 20 inputs and two equation strings. Also see "Dynamic text items" on page 304 for larger equations than 255 characters. If you need unlimited input and output variables, see EquationCompileDynamicVariables, below.	I
EquationCompileDynamic (str31 varNames[], string dynamicTextArray, integer outputEquation[], string labelOutput, integer tabOrder)	Compiles a dynamic text equation. See EquationCompile20(), as this is similar except that the input variable names are supplied in the array VarNames. VarNames can be up to 20 arguments.	I
EquationCompileDynamicVariables(string inputsName, string outputsName, string equationDynArray, integer codeDynArray, integer tabOrder)	This function allows unlimited input and output variables. InputsName and outputsName are used in the equation like this example: outputsName[i] = inputsName[i]; where i will be the index of that input or output variable. TabOrder is used to select the correct text item when there is an error in the equation. You can do a string substitution in the user's raw equation to put it in this indexed form. See the Equation block in the Values library to see how to use this function.	I
EquationCompileSetStaticArray(integer dynArray)	See the Equation block for use. The dynArray argument can be any type as the equation compile functions set up the array to hold any declared static variables.	V

Equations	Description	Return
EquationDebugCalculate(integer debugEquationIndex, real inputVarValuesArray[], real outputVarValuesArray[])	Calculates the equation using specified input values. The output values for the result will be put into <i>outputVarValuesArray</i> . Returns a TRUE value if an error occurs.	I
EquationDebugCompile(integer debugEquationIndex, string equationCodeArray[], string varsInputArray[], string varsOutputArray[], integer tabOrder)	Converts the equation code and variable names into block form so it can be debugged. <i>DebugEquationIndex</i> is the previously returned value from a previous call to this function for this equation, or -1 (minus one) when this function is called for the first time with this equation. Returns a debug equation index to be used in the other equation debugging functions. <i>TabOrder</i> is the tab order of the text item with the equation. See the equation-based blocks for how to use this function. NOTE: <i>DebugEquationIndex</i> should be set to -1 if EquationDebugDispose() (below) is called for that index OR this function is called for the first time for this block.	I
EquationDebugDispose(integer debugEquationIndex)	Disposes and releases the memory used by the hidden block specified by <i>debugEquationIndex</i> used to debug a particular equation. This does not affect the visible equation block. NOTE: The variable used for <i>DebugEquationIndex</i> should be set to -1 (minus one) after this function is called so it works correctly if EquationDebugCompile(), above, is called after this call.	V
EquationDebugSetBreakpoints(integer debugEquationIndex)	Opens a "Set Breakpoints" window so the user can click to create debugger breakpoints. Returns a True value if an error occurs.	I
EquationGetStatic(integer index)	Used in an Equation type block to allow static values to be "remembered" and used in the equation. Need to define: Real EquationStaticValues[100]; as a static variable at the top of the ModL script for the Equation block (Value library). A shorter name for this function is EqGet(). The user calls this function with an index from 0 to 99 to get the correct static value from this array to use in their equation. Used with EquationSetStatic(), below.	R
EquationIncludeSet(string theIncludeName)	Called right before one of the equationCompile functions, this puts the contents of the specified include file into the compiled equation. Call this for each include desired.	V

Equations	Description	Return
EquationSetStatic (integer index, real value)	Used in an Equation type block to allow static values to be "remembered" and used in the equation. Need to define: <pre>Real EquationStaticValues[100];</pre> as a static variable at the top of the ModL script for this Equation block. A shorter name for this function is EqSet(). The user calls this function with an index from 0 to 99 to set the value in the static array. Then the user can call EquationGetStatic(index), above, to use that value in their equation.	V
IncludeFileEditor(string includeFileName, integer blockNumber)	Tags the specified include file to do the following: If you click the close box, it will not close, but instead will send a message to the block specified by blockNumber. Equation-based blocks use this for their external code editor functionality; see the equation blocks for examples. Returns 0 for success or a negative value to indicate failure.	I
ShowFunctionHelp(integer alpha)	Brings up ExtendSim's Help with a list of the functions and arguments available for the equation functions. If <i>alpha</i> is TRUE, brings up the alphabetical list of functions. If <i>alpha</i> is FALSE, brings up the "Functions by type" list.	V

I/O functions

File I/O, formatted

Use these functions to manipulate formatted text files. These functions let you perform the same tasks as the Import Data and Export Data commands in the File menu as well as lower-level file reading and writing. Text files produced with the output functions must be read in an application that reads text files such as a word processing or spreadsheet program.

In these functions, if the pathname used is the empty string (""), ExtendSim prompts you with a standard open or save dialog. This allows you to determine the correct file name at the time of the simulation. File pathnames for specific files are specified as "driveLetter:\directory\directory\fileName" (Windows) or "volumeName : folder : folder : fileName" (Mac OS) with each level separated by backslashes or colons, as appropriate. If the volumeName and folder names are left off of the pathname, the file name will come from the current folder. Note that names of files can only be 31 characters long (Windows and Mac OS).

The Import and Export functions let you define the column delimiter (separator) character in the file to be read or written. In ExtendSim, Excel, Word, and most other tabular data applications, tabs normally delimit columns, and CRLFs (Windows: carriage returns, line feeds) or CRs (Mac OS: carriage returns) always delimit rows.

The colDelim argument to these functions is a string which specifies the separator character:

""	The empty string (two quotation marks with nothing between them) indicates a tab character
","	A comma character
" "	A space character (multiple spaces are read as one space)

"(any character)"	The character specified will be used as a column separator
-------------------	--

There are two sets of functions. The Import and Export functions are used with files that have numerical data; the ImportText and ExportText functions are used with files that have string data. The functions are:

File I/O (formatted)	Description	Return
Export(string pathName, string userPrompt, string colDelim, real array $\begin{bmatrix} \end{bmatrix}$, integer rows, integer columns)	Writes the contents of a one- or two-dimensional real array or data table to the file. <i>Rows</i> and <i>columns</i> specify the portion of the array to be written and are integers. The function assumes that columns are delimited by the <i>colDelim</i> string character. Single dimension arrays must be read as one column by <i>n</i> rows. The function returns the number of rows written to the file, or 0 if there is an error.	I
ExportText(string pathName, string userPrompt, string colDelim, string array $\begin{bmatrix} \end{bmatrix}$, integer rows, integer columns)	Writes the contents of a one- or two-dimensional string array or text table to the file. <i>Rows</i> and <i>columns</i> specify the portion of the array to be written and are integers. The function assumes that columns are delimited by the <i>colDelim</i> string character. Single dimension arrays are treated as one column by <i>n</i> rows. The function returns the number of rows written to the file or 0 if there is an error.	I
Import(string pathName, string userPrompt, string colDelim, real array $\begin{bmatrix} \end{bmatrix}$)	Reads the numerical data from the file into a one- or two-dimensional real array or data table, then returns the number of rows read (if an error occurs, it returns 0). The function assumes that columns are delimited by the <i>colDelim</i> string character. Single dimension arrays are read as one column by <i>n</i> rows. Note that you should initialize the array before using this function. Otherwise, any values beyond what was read from the file will have the old values of the array.	I
ImportText(string pathName, string userPrompt, string colDelim, string array $\begin{bmatrix} \end{bmatrix}$)	Reads the string data from the file into a one- or two-dimensional string array or text table, then returns the number of rows read (if an error occurs, it returns 0). The function assumes that columns are delimited by the <i>colDelim</i> string character. Single dimension arrays are treated as one column by <i>n</i> rows. Note that you should initialize the array before using this function to prevent any values beyond what was read from the file from having the old values of the array.	I

File I/O, unformatted

These are lower level file I/O functions. You can have up to 200 text (.txt) or HTML (.htm) files open for general reading and writing. The files are specified in the functions with the integer fileNumber that is returned from the FileOpen and FileNew functions. The functions are:

File I/O (unformatted)	Description	Return
CreateFolder(string pathName)	Creates a new folder from the <i>pathName</i> . For Windows, the <i>pathName</i> must use backslashes (\) to separate folder names, "myDrive:\ExtendSim7\myNewFolder". For Mac OS, colons (:) should be used, "myDrive:ExtendSim7:myNewFolder". Returns FALSE if successful, TRUE if there was an error.	I
DirPathFromPathName(string pathName)	Returns the folder pathname part of a complete pathname. Also see FileNameFromPathName(), below. For example: "C:\myfolder\"	S
FileChoose(string defaultFilename, string Prompt)	Pops up the standard file selection dialog, with the prompt and the default file name specified, and returns the file name of the file the user selects. This differs from the fileOpen function, which it otherwise resembles, in that it just returns the file name/path name of the selected file without opening it. This allows the developer to use that name however she chooses.	S
FileClose(integer fileName)	Closes the file when you are finished writing or reading data to or from the file. Files must be closed before they can be used as data in other applications. Call FileClose in the EndSim message handler to close files when the simulation is finished.	V
FileDelete(string pathname)	Deletes the file. Use this with caution because deleted files are not recoverable.	V
FileEndOfFile(integer fileName)	TRUE if the end of file has been reached during the most recent FileRead.	I
FileExists(string pathname)	Returns TRUE if the file exists	I
FileGetDelimiter (integer fileName)	Type of delimiter found after the most recent FileRead. Returns FALSE if a column delimiter (such as a tab character) was found after the data, or TRUE if a CRLF (Windows) or CR (Mac OS) row delimiter was found. Call this immediately after FileRead to find out whether a column delimiter, CRLF, or CR followed the data.	I
FileGetPathName (integer fileName)	Returns the file's path name.	S
FileInfo(string filePathName, integer which)	Returns information about the file specified in the filePathName argument. The <i>which</i> argument specifies what information will be returned: 1: created date 2: modified date Dates are returned as ExtendSim date values.	R
FileIsOpen(string pathName)	Returns TRUE if the file described by the <i>pathName</i> is open.	I
FileNameFromPathName (string pathName)	Returns the filename part of a complete pathname. Also see DirPathFromPathName(), above, to get the path name part of a complete path name.	S

File I/O (unformatted)	Description	Return
FileNew(string pathname, string userPrompt)	Opens a new or existing text (.txt) or HTML (.htm) file for writing and returns a fileNumber. If the <i>pathName</i> is an empty string (""), ExtendSim prompts for a file name, displaying the <i>userPrompt</i> string. If the pathname cannot be found, or the Cancel button has been clicked, FileNew returns FALSE (0). If the file is already open, it returns the file's fileNumber. Note that the FileNew function erases all information from an existing file. To append data to an existing file, use FileOpen. Call FileNew in the InitSim message handler to create files at the beginning of a simulation.	I
FileOpen(string pathname, string userPrompt)	Opens an existing text (.txt) or HTML (.htm) file for reading or writing, and returns a fileNumber for reference. If the <i>pathname</i> cannot be found, or the file is unreadable, or the Cancel button has been clicked, FileOpen returns FALSE. If the file is already open, it returns the file's fileNumber. If the file is written to after using FileOpen, the new data is appended to the end of the file. Call FileOpen in the InitSim message handler to open files at the beginning of a simulation. NOTES: If you call this function with the following strings (e.g. *.TXT) as the pathname, it will change the types of files that the Standard File Dialog will be looking for: *.TXT - text files, *.DAT - data files, *.ATF - Proof trace file, *.LAY - Proof layout file. Note that if <i>pathname</i> is an empty string (""), the user will be prompted for a filename at run time.	I
FileRead(integer fileNumber, string colDelim)	Reads and returns a string read from the file, up to <i>colDelim</i> (a column delimiter character) or to a CRLF (Windows) or CR (Mac OS) delimiter. To ignore the column delimiter, set <i>colDelim</i> to an unused character (i.e. "@"). Reading past the end of file causes an error message. You should test with FileEndOfFile before calling FileRead. See FileGetDelimiter(), above.	S
FileRewind(integer fileNumber)	Resets the file to its beginning so that it can be reread.	V
FileWrite(integer fileNumber, string s, string colDelim, tabCR)	Writes the string or value into the file. If a number is used for s, ModL will automatically convert the number to a string. If tabCR is FALSE, a column delimiter character is written to the file after s; if TRUE, a CRLF (Windows) or CR (Mac OS) delimiter is written after s. If the column delimiter is a plus sign ("+"), no delimiter is written between the strings.	V
GetDirectoryCon- tents (string path, stringArray string- data, longarray longdata)	This function takes two dynamic arrays as arguments, calls MakeArray() for them, and fills them with the names of all the files and sub-directories in the specified folder. The first array will contain the names of all the files/directories, and the second will contain an integer value that will be zero for a file, and one for a folder. It returns the number of row entries in the array. The pathname separator on a mac is ":", on windows a "/".	I

File I/O (unformatted)	Description	Return
GetFileReadMachineType()	Returns the type of machine the currently active model was saved on. (This is only useful if models have been moved from one platform to another. Otherwise, it will be the same as the machine the model is running on.) Type 2 is Windows, 1 is models built on 68k Macs, and 4 is models built on PPC Macs.	I
StripLFs (integer strip)	Sets a flag in ExtendSim that determines if the fileread functions will strip LF characters. This flag defaults to TRUE, so you should call StripLFs(FALSE) if you find that meaningful LF characters are missing from your data.	V
StripPathIfLocal (string pathName)	Strips off the pathname if the file is in the same folder as the current model. For example, this can remove non-portable path names from a filename returned from FileOpen() function. If the file is in the same folder as the model, no pathname is needed.	S

Internet access

These functions allow data and file access and manipulation using internet protocols. The *FTP* block in the I/O category of the ModL Tips library is an example of the use of these functions in FTP access.

There are three connection type handles used in these functions:

SessionHandle	The handle to this entire internet communication session.
FTPHandle	The handle to a specific FTP session.
SearchHandle	The handle to a specific search operation.
InetHandle	Any one of the above handle types.

Internet access	Description	Return
INetCloseHandle(Integer inetHandle)	Closes an InetHandle. This function will close a handle created by InetOpenSession(), InetFTPFileFirstFile(), or InetConnect().	V
INetConnect(integer sessionHandle, string serverName, string userName, string passWord, integer connectionType)	Creates an internet connection type. ConnectionType can be 1:FTP 2:HTTP 3:Secure http Currently FTP connections are the most useful, as we have implemented a suite of FTP Inet functions. Returns a connection handle that needs to be closed with INetClosehandle when the connection is complete.	I
INetFileImportText(integer hFile, string format, stringArray array)	Given an INet handle, (Most likely created by INetOpenURL, above,) this function will import data from the file represented by that handle into the specified array.	I

Internet access	Description	Return
INetFindNextFile(integer searchHandle)	Continues an internet file search by moving the searchHandle on to the next file. Returns a searchHandle.	I
INetFTPCreateDirectory(integer FTPHandle, string targetName)	Creates a folder named targetName in the current folder.	I
INetFTPDeleteFile(integer FTPHandle, string targetName)	Deletes the path\file targetName if it is found.	I
INetFTPExport(integer FTPHandle, string fileName, string delim, real array[], integer rows, integer cols)	This operates much like the Export function with the exception that it writes to a path\file accessed via FTP.	I
INetFTPExportGA(integer FTPHandle, string fileName, string delim, integer GAIndex, integer rows, integer cols)	Writes out the contents of a Global Array to the FTP path\file.	I
INetFTPExportText(integer FTPHandle, string fileName, string delim, string array[], integer rows, integer cols)	This operates much like the ExportText function with the exception that it writes to a path\file accessed via FTP.	I
INetFTPFindFirstFile(integer FTPHandle, string searchFile, integer flags)	Starts a search for files in the default folder. An empty string will find the first file in the current folder with any filename. Returns a search handle that needs to be closed with closehandle when the search is complete. Currently, flags should be set to zero.	I
INetFTPGetCurrentDirectory(integer FTPHandle)	Returns the path to the current FTP directory as a string.	S

Internet access	Description	Return
INetFTPGet-File(integer FTPHandle, string targetName, string fileName, integer failIfExists, integer fileAttributes, integer flags)	Copies a file from the remote site to the local machine. The path\targetname specifies the name of the file to be retrieved. The path\filename specifies the local name where the file should be put. FailIfExists determines what happens if the local file already exists. Currently, flags and file attributes should be set to zero.	I
INetFTPImport(integer FTPHandle, string fileName, string delim, real array [[]])	This operates much like the Import function with the exception that it reads a path\file accessed via FTP.	I
INetFTPImportGA(integer FTPHandle, string targetName, integer format, integer GAIndex)	Reads the contents of an FTP path\file into the Global Array.	I
INetFTPImportText(integer FTPHandle, string fileName, string delim, string array [[]])	This operates much like the ImportText function with the exception that it reads a path\file accessed via FTP.	I
INetFTPPutFile(integer FTPHandle, string fileName, string targetName, integer flags)	Copies a local file to the internet. Path\fileName specifies the name of the local file. Path\targetName specifies the desired name of the file on the internet. Currently, flags should be set to zero.	I
INetFTPRemoveDirectory(integer FTPHandle, string targetName)	Deletes the specified directory path\targetName from the site.	I
INetFTPRenameFile(integer FTPHandle, string targetName, string newName)	Renames the file path\targetName to newName.	I
INetFTPSetCurrentDirectory(integer FTPHandle, string targetName)	Sets the current directory to path\targetName.	I

Internet access	Description	Return
INetGetFind-FileInfo(which)	Returns info about the current file in the search. Currently, which:1 is the only allowed input and returns TRUE if a directory, false if a file.	I
INetGetFindFile-Name()	Returns the name of the current file in the search.	S
INetOpenSession()	Starts an Internet session. Returns a session handle that needs to be closed with closehandle when the session is complete.	
INetOpenURL (integer session, string url)	Given a session handle, (created by the INetOpenSession function,) and a URL, this function will open a connection to the site corresponding to that URL	I

Interprocess Communication (IPC)

Interprocess communication (IPC) provides a standard way in which one application can directly communicate with another. These functions allow ExtendSim to act as a client application by connecting to and requesting data and services from a server application. The server application must support dynamic data exchange (DDE, Windows) or AppleEvents (Mac OS). The IPC functions start with “IPC”. If you are trying to develop on both Windows and Mac OS using IPC, see “IPC or multi-server considerations” on page 408.

IPC & Publish/Subscribe	Description	Return
IPCAdvise(integer conversation, string item, integer block-Number, string dialogItem, integer rowStart, integer colStart, integer rowEnd, integer colEnd)	(Windows only) Starts a DDE Advise loop with the application that is the other side of the specified <i>conversation</i> . This function will return an advise loop id, which needs to be used in the IPCStopAdvise Function when the advise loop is to be terminated.	I
IPCCheckConversation(integer conversation)	Checks the validity of an IPC conversation on Windows. (It always returns TRUE on the Mac.) A TRUE value is returned if the conversation is valid.	I
IPCConnect(string serverName, string topic)	Initiates an IPC conversation between ExtendSim and a server. The <i>serverName</i> argument is the DDE or AppleEvents name of the server you are trying to connect to. (This name needs to be in the format that is appropriate for the platform. For example, Excel on Windows expects “Excel”, on the Mac OS it expects “XCEL”.) The <i>topic</i> argument is typically the keyword “SYSTEM.” You can also use the name of the document you want to communicate with, if you want the conversation to target a specific model. If successfully connected, this function returns an integer value that is used in the other IPC functions as the conversation identifier. If unsuccessful, it returns a zero. Note: the IPCDisconnect function must be used with IPCConnect so that communication is terminated as soon as it is no longer required.	I

IPC & Publish/ Subscribe	Description	Return
IPCDisconnect (integer conversation)	Disconnects the specified <i>conversation</i> . This function is necessary when using IPCConnect, and should be called immediately when communication is no longer required. Returns a zero if the disconnection was successful.	I
IPCExecute(integer conversation, string executeData, string item)	Sends a command to be executed by the server in the specified <i>conversation</i> . The <i>executeData</i> argument is the command to be sent. The <i>item</i> argument is currently not used and should be set to a blank string (""). This function returns a zero if successful, a -1 for a general error, a -2 for a time-out error, a -3 for an invalid connection, and a -4 for an event not handled by the server.	I
IPCGetDocName()	Returns a string that contains the name of the last file opened with the IPCOpenfile() function. This is mostly used to return the name of a file that the user selected when an empty string ("") was passed into the IPCOpenfile function.	S
IPCLaunch(string appName, integer minimized)	Launches the application <i>appName</i> and minimizes it if <i>minimized</i> is TRUE.	I
IPCOpenFile (string fileName)	Opens the file (for example, a spreadsheet) in the Finder using DDE (Windows) or AppleEvents (Mac OS). This function is equivalent to the user double-clicking a file's icon: it causes both the file and the file's application to open. The single argument is a string which is the file name. Note that this function tells the System to open the file (that is, to launch the named application or document) and is not at all related to ExtendSim's FileOpen function. This function will accept a blank string ("") in the <i>fileName</i> argument which will cause a File Open dialog to appear. The user is then prompted to select the file which is to be opened. The function returns zero if successful.	I
IPC Poke(integer conversation, string pokeData, string item)	Sends data to the server in the specified <i>conversation</i> . The <i>pokeData</i> argument is the data to be sent. The <i>item</i> argument indicates where the data is to be put. For example, in Excel the item argument could be "R1C1" indicating that the data is to be sent to the cell at row 1 column 1. Note that the syntax of the item argument is dependent on the server. This function returns a zero if successful.	I
IPC PokeArray(integer conversation, string item, string delim, string array data)	(Windows only) Pokes an array of data. The data from the dynamic array data will be poked to the target application. The return value will be zero for success, and nonzero for failure.	I
IPCRequest(integer conversation, string item)	Returns data from the server in the specified <i>conversation</i> . The <i>item</i> argument indicates where the data is to be taken from. For example, in Excel the item argument could be "R1C1" indicating that the data is to be retrieved from the cell at row 1 column 1. Note that the syntax of the item argument is dependent on the server.	S

IPC & Publish/ Subscribe	Description	Return
IPCRequestArray (integer conversation, string item, string delim, string array data)	(Windows only) Requests an array of data. The dynamic array data will be filled with the results from the request. The return value will be the number of rows of data that were returned.	I
IPCSendCalcReceive(integer product, real sendValue, integer sendRow, integer sendCol, string funcName, integer receiveRow, integer receiveCol)	Sends <i>sendValue</i> to the <i>sendRow</i> , <i>sendCol</i> of a spreadsheet file that is already open. It then executes the macro called <i>funcName</i> , or just recalculates if <i>funcName</i> is an empty string. The function returns the value at <i>receiveRow</i> , <i>receiveCol</i> . <i>Product</i> specifies the spreadsheet you are communicating with: use 1 for Microsoft Excel or 2 for Lotus 123. See also the function IPCSpreadSheetName.	R
IPCSetTimeout(integer timeOut)	Sets the Timeout value for the various IPC functions. This determines how long ExtendSim will wait for the other application to respond to IPC requests. Values are in milliseconds. The default value is 10000. Putting a -1 into this parameter will request Async behavior.	V
IPCServerAsync(integer async)	If <i>async</i> is TRUE, sets a flag in ExtendSim so that ExtendSim will return from further Execute messages immediately instead of waiting for the Execute messages to complete. Useful when used in an Execute message from another application so that the other application can continue to do other tasks while ExtendSim calculates.	V
IPCSpreadSheetName(string spreadSheetName)	Used to establish a default <i>spreadSheetName</i> , and is only used in conjunction with the IPCSendCalcReceive function. The IPCSendCalcReceive function does not take a spreadsheet name as an argument, and Lotus 1-2-3 requires a specific spreadsheet name for DDE communication on Windows.	V
IPCStopAdvise(integer conversation, integer adviseLoopID, integer block)	(Windows only) Stops the specified advise loop within the specified block.	I
UpdatePublishers()	(Mac OS only) Forces an update of publisher information, for all publishers in the model. Note that this departs from Apple's standard interface, where publishers are only updated when the model is saved.	V

OLE/COM (Windows only)

These functions apply to the two variations of interface objects that are supported in ExtendSim, as described below. They also control or communicate with other applications via OLE Automation. Both are types of OLE embedded objects, (and/or ActiveX controls,) that can be inserted into the ExtendSim user interface in different ways. The two variations are:

- Inserting an OLE object at the worksheet level with the Insert Object command in the edit menu. This command will create a new block-like object that represents an instance of the embedded object.
- Inserting an object into a new type of dialog item called an embedded object. This gives the embedded object a dialog variable name that can be used in these functions.

The ModL functions below will allow you to access and communicate with either type of object. The OLE function calls that contain a *blockNumber* and *dialogItem* refer to an embedded object on either the worksheet or in a dialog. In the dialog case, both arguments need to be used. In the worksheet case, just the *blockNumber* is sufficient, and the *dialogItem* should be an empty string (e.g. ""). OLE functions support SafeArray functionality where appropriate.

When these functions are used for OLE Automation, you need to start with the OLECreateObject function. This function will create an OLE object of the type you specify, and return an IDispatch interface on that object. From then on you can use the OLEDispatch functions described below to call methods, or set and get properties on the objects.

Please note that what these functions return, and what effects they have is largely dependent on the implementation of the OLE object/ActiveX Control that you are embedding.

 There are sample blocks in the ModL Tips library in the OLE category that show the syntax of these commands. The blocks are called Excel and Embed, and can be used as OLE scripting tutorials. Embed is a good tool for looking at the methods and properties of a given embedded object.

OLE (Windows only)	Description	Return
OLEGetHelpContext (integer blockNumber, string dialogItem, integer dispID)	Returns the Help context value of the specified dispID.	I
OLEActivate(integer blockNumber, string dialogItem)	Activates the specified embedded object. <i>DialogItem</i> is the dialog variable name in quotes. Returns FALSE if successful.	I
OLEAddRef(integer interfacePtr)	Addrefs the interface specified. Returns the refcount.	I
OLEArrayParam (array data, integer variants)	Puts the data in the array 'data' into a safeArray, packaged as a parameter for an Invoke call. The variants argument determines whether each data element will be packaged in a separate variant or not.	I
OLEArrayParamVariableColumns(short hNum, array datatableName, integer numCols, integer variants)	Specifies that the datatable array passed in will be used as an argument for the next OLEInvoke call. The numCols argument will define how many columns the function defines the data as containing.	I

OLE (Windows only)	Description	Return
OLEArrayResult (array data)	Retrieves the SafeArray result data from an Invoke call, putting the data into the array 'data'.	I
OLEArrayResult-VariableColumns(array datatableName, integer numCols)	Specifies that the datatable array passed in will be filled with the result of the last OLEInvoke call. The numCols argument will define how many columns the function defines the data as containing.	I
OLECreateObject (string objectReference)	This function is the starting point for OLE Automation. It will create an OLE object, or provide an interface to an application if it is already running, and return an Idispatch interface to that object that can be used with the OLEDispatch calls listed below to allow the user to control other applications via OLE Automation. The object reference string is the registry key associated with the object you wish to embed. (As an example, Excel would be excel.application.)	I
OLEDBParam(integer DBIndex, integer tableIndex, integer variants)	Passes the contents of the specified DB table as a safe array parameter.	I
OLEDBResult(integer DBIndex, integer tableIndex)	Fills the specified database table with the results of an Invoke or ParameterGet call that was just made.	I
OLEDeactivate (integer blockNumber, string dialogItem)	Deactivates the specified embedded object. <i>DialogItem</i> is the dialog variable name in quotes. Returns FALSE if successful.	I
OLEDispatchGet-CLSID(integer dispHandle, integer progID)	Returns the CLSID as a string. ProgID is FALSE to return the CLSID, or TRUE to return the program ID.	S
OLEDispatchGet-DispatchName (integer dispHandle, integer which)	Same as the function OLEGetDispatchName below, except it takes a dispatchHandle instead of a blocknumber and a dialog item name.	S
OLEDispatchGet-DispID(integer dispHandle, string theName)	Given a function/variable name, returns the DispID. Same as the function OLEGetDispID below, except it is expecting a dispatchHandle instead of a block number and dialog item name.	I
OLEDispatchGet-Doc (integer IDispatchHandle, string returnDoc[], integer dispID, integer which)	This function is a Dispatch handle version of the OLEGetDoc() function. For a description of the IDispatchHandle argument, see the OLEDispatchGetHelpContext() function.	I

OLE (Windows only)	Description	Return
OLEDDispatchGetFuncIndex(integer dispHandle, integer dispID)	Same as the function OLEGetFuncIndex, except it takes a dispatchHandle instead of a blocknumber and a dialog item name.	I
OLEDDispatchGetFuncInfo(integer dispHandle, integer funcIndex, integer which)	Same as the function OLEGetFuncInfo, except it takes a dispatchHandle instead of a blocknumber and a dialog item name.	I
OLEDDispatchGetHelpContext (integer IDispatchHandle, integer dispID)	This uses an IDispatch handle, usually returned by the OLEDDispatchResult, or OLECreateObject functions defined above. This handle will be the Dispatch interface to an object that is either associated with an embedded object on the worksheet, or in the block dialog, or has been created via OLE Automation in an remote application. Many embedded objects are simple objects that will not have methods that return IDispatch handles on other objects, but some, like an embedded Excel worksheet, for example, contain 'Subobjects' that will need to be referenced in this way. See OLEDDispatchResult() and OLECreateObject(). Same as OLEGetHelpContext(), except uses a handle.	I
OLEDDispatchGetNames (integer IDispatchHandle, Str31 names[], integer dispID)	See OLEDDispatchGetHelpContext(). Same as OLEGetNames(), except uses a handle.	I
OLEDDispatchInvoke (integer IDispatchHandle, integer dispID)	See OLEDDispatchGetHelpContext(). Same as OLEInvoke(), except uses a handle.	I
OLEDDispatchParam (integer dispatchHandle)	Adds a DispatchHandle to the argument list for the next Invoke call. Note: arguments are listed in back to front order. Returns FALSE if successful.	I
OLEDDispatchPropertyGet (integer IDispatchHandle, integer dispID)	See OLEDDispatchGetHelpContext(). Same as OLEPropertyGet(), except uses a handle.	I
OLEDDispatchPropertyPut (integer IDispatchHandle, integer dispID)	See OLEDDispatchGetHelpContext(). Same as OLEPropertyPut(), except uses a handle.	I

OLE (Windows only)	Description	Return
OLEDIs- patchResult()	Returns an IdispatchHandle from the last Invoke call. (If a return value is available.) This handle can be used in the other OLEDIs- patch calls listed above. This handle will be the Dispatch interface to an object that is associated with an embedded object on the work- sheet, or in the block dialog. Many embedded objects are simple objects that will not have methods that return dispatch handles on other objects, but some, like an embedded Excel worksheet, for example, contain 'Subobjects' that will need to be referenced in this way.	I
OLEGAParam (integer arrayIndex, integer variants)	Copies the data in the global array defined by arrayIndex into a Safe- Array, packaged as a parameter for an Invoke call. The variants argument determines whether each data element will be packaged in a separate variant or not.	I
OLEGAResult (arrayIndex)	Retrieves the SafeArray result data from an Invoke call, putting the data into the global array referred to be arrayIndex.	
OLEGetCLSID (integer blockNum- ber, string dialog- Item, integer progID)	Returns the CLSID as a string. ProgID is false to return the CLSID, or TRUE to return the program ID.	
OLEGetDispatch- Name (integer blockNumber, string dialogItem, integer dispID)	<i>DialogItem</i> is the dialog variable name in quotes. Returns the name associated with the specified dispID.	S
OLEGetDispID (integer blockNum- ber, string dialog- Item, string theName)	<i>DialogItem</i> is the dialog variable name in quotes. Returns the Dis- patch ID for the function/variable theName.	I
OLEGetDoc (inte- ger blockNumber, string dialogItem, string returnDoc[], integer dispID, inte- ger which)	<i>DialogItem</i> is the dialog variable name in quotes. Returns the inter- nal documentation from the type library in string array returnDoc. returnDoc will be resized as needed if the text is larger than a single string. This function accesses text that is in the objects Type Library. What information is available, and whether any is available, is object dependent. <i>Which</i> takes the following values. 0: name (DispID property/Method name) 1: doc (Any available Documentation on the DispID.) 2: file name (FileName of the help file associated with the dispID.)	I

OLE (Windows only)	Description	Return
OLEGetFuncIndex (integer blockNumber, string dialogItem, integer dispID)	Returns the function Index used in OLEGetFuncInfo that corresponds to the dispID.	I
OLEGetFuncInfo (integer blockNumber, string dialogItem, integer index, integer which)	<i>DialogItem</i> is the dialog variable name in quotes. Returns function information for the function specified by index. Implementation of this function is object specific, so your results will vary dependent on how the developers of the object have implemented it. if <i>which</i> is 0: INVOKEKIND returns 1 for INVOKE_FUNC returns 2 for INVOKE_PROPERTYGET returns 4 for INVOKE_PROPERTYPUT returns 8 for INVOKE_PROPERTYPUTREF if <i>which</i> is 1: cParams returns Count of total number of parameters if <i>which</i> is 2 : cParamsOpt returns Count of optional parameters if <i>which</i> is 3 : returns DispID (sometimes called memberID). Note that <i>index</i> is not the same as the dispID. It is just a sequential index value from 0 to n-1 where n is the number of functions supported by the object.	I
OLEGetGUID()	Pops up the standard insert item dialog, and returns the GUID of the object you select as a string. The objects that appear on the standard insert item dialog are just those objects that have been defined as 'Insertable' in the registry, and will not necessarily include all of the objects that you can use with ExtendSim.	S
OLEGetInterface (integer blockNumber, string dialogItem, integer whichInterface)	<i>DialogItem</i> is the dialog variable name in quotes. Returns a pointer to an interface on the object. The whichInterface argument currently only supports a zero value for the IDispatchInterface.	I
OLEGetNames (integer blockNumber, string dialogItem, Str31 names[], integer dispID)	<i>DialogItem</i> is the dialog variable name in quotes. Puts the name of the function/variable into the first row of the dynamic array <i>names</i> . The later rows contain the names of any arguments to the function. The return value is the number of names returned.	I
OLEGetRefCount (integer interfacePtr)	Returns the RefCount on the Specified interfacePtr. Note that this does nothing more than an AddRef and a Release, so there is no reason to call this routine if you are already using addref and release.	

OLE (Windows only)	Description	Return
OLEInsertLicensedObject (integer blockNumber, string dialogItem, string guid, integer xPixel, integer yPixel, string licString)	Same as the OLEInsertObject function, below, with the exception of the last argument, which allows you to pass a license string to the object to be inserted. This is used for activeX objects that allow licensed execution as runtimes. See the function OLERequestLicKey for additional information.	I
OLEInsertObject (integer blockNumber, string dialogItem, string guid, integer xPos, integer yPos)	<i>DialogItem</i> is the dialog variable name in quotes. Inserts an object into the indicated location. If you specify the dialog item, and the block number, it will be inserted into that dialog item, ignoring <i>xPos</i> and <i>yPos</i> . If the dialog item name is an empty string, the object will be inserted onto the active worksheet at pixel location <i>xPos</i> and <i>yPos</i> . If <i>xPos</i> and <i>yPos</i> are both 1, the item will be inserted at the "current" position on the worksheet (i.e. the last mouse click or the last created block position).	I
OLEInsertObjectFromFile (integer blockNumber, string dialogItem, string guid, integer xPixel, integer yPixel, string licString, string filePath)	This function inserts an Object into an embedded object dialog item, or onto the worksheet, like OLEInsertObject. It takes the additional Argument <i>licString</i> , the usage of which is defined in OLEInsertLicensedObject. It also takes a <i>filePath</i> Argument, which allows the coder to define an object based on a file, as is done in the standard Object insertion dialog.	I
OLEInvoke (integer blockNumber, string dialogItem, integer dispID)	<i>DialogItem</i> is the dialog variable name in quotes. Invokes (calls) the method/variable specified by <i>dispID</i> . You must call Param functions to set up the arguments to the method. Returns a WIN API error code if it fails, zero if success.	I
OLELongParam (integer value)	Adds an integer <i>value</i> to the argument list for the next Invoke call. Note: Arguments are listed in back to front order. Returns FALSE if successful.	I
OLELongResult()	Returns an integer value from the last Invoke call, if a return value is available.	I
OLEObjectIsRegistered (string clsid)	Checks to see if a specific CLSID is already registered. Returns TRUE if the CLSID is already in the registry and FALSE if it is not.	
OLEPropertyGet (integer blockNumber, string dialogItem, integer dispID)	<i>DialogItem</i> is the dialog variable name in quotes. Gets the property specified by <i>dispID</i> . You must call Result functions to retrieve the value.	I

OLE (Windows only)	Description	Return
OLEPropertyPut(integer block-Number, string dialogItem, integer dispID)	DialogItem is the dialog variable name in quotes. Sets the property specified by dispID. You must call Param functions to set up the arguments to the method.	I
OLERealParam (Real value)	Adds a real <i>value</i> to the argument list for the next Invoke call. Note: Arguments are listed in back to front order. Returns FALSE if successful.	I
OLERealResult()	Returns a real value from the last Invoke call, if a return value is available.	R
OLERelease(integer interfacePtr)	Releases the interface pointer specified. Returns the refCount.	I
OLEReleaseInterface (integer interfacePtr, integer whichInterface)	Releases the interface pointer returned by OLEGetInterface. Returns FALSE if successful.	I
OLERemoveObject(integer block-Number, string dialogItem)	Removes the OLE object from the dialog item.	I
OLERequestLicKey(string guid)	This function requests a license key from a licensed activeX control on your machine. This can be used with OLEInsertLicensedObject, above, to allow the user to retrieve the license key to be used when inserting a licensed runtime activeX Control.	S
OLESetNamedParam (integer paramID)	Specifies that the first named parameter is the parameter ParamID. If the Idispach interface you are using specifies named parameters, then they are the first parameters by definition. All this function does is specify which named parameters you are using. The first time you call it, it specifies what the paramID of the first named parameter is, the second time the second, and so on.	I
OLEStringParam (string value)	Adds a string value to the argument list for the next Invoke call. Note: Arguments are listed in back to front order. Returns FALSE if successful.	I
OLEStringResult()	Returns a string value from the last Invoke call, if a return value is available.	S
OLESupressInvokeErrors(integer supressErrors)	Used to stop those pesky error messages from appearing during an invoke of an OLE object.	V
OLEVariantParam(string value)	Adds a variant pointer value to the argument list for the next invoke call. Note: Arguments are listed in back to front order.	I

OLE (Windows only)	Description	Return
OLEVariantResult(integer which)	Returns a string value from the specified variant pointer argument of the last Invoke call. <i>Which</i> specifies which of the arguments of the invoke call you are referring to, it would be one for the first one entered, two for the second, and so on (If the specified argument was a variant pointer argument.)	S
WinRegSvr32(integer registerObject, string fileName, string dir)	This function runs the RegSvr32 command line tool used is used to register .dll files as components in the windows registry. The registerObject integer is a flag that determines if you want to register (true) or unregister (false) the object. FileName is the name of the dll file. Dir is the path name to the directory containing the dll.	I

Mailslot (Windows only)

Mailslots are a messaging functionality supported by the windows API. They are unidirectional messages that are sent from a given machine to a specified mailslot.

The MailSlotReceive message handler will be called periodically when there is a waiting message available in one of the mailslots created by these functions.

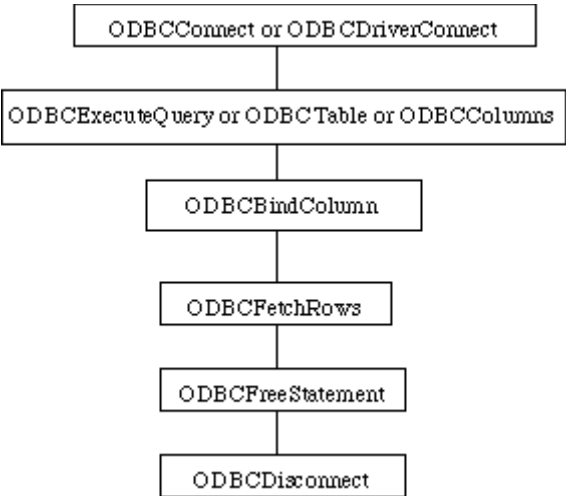
- Because mailslot technology is a “polling” system, there will be a time delay between when a message is sent to when a message is actually received.
- If you send out a single message to a mailslot, the mailslot may receive multiple copies of that message depending on your network configuration. This is a expected behavior of the Microsoft implementation of mailslots. If you need to uniquely identify messages, the simplest way would be to concatenate a unique identifier onto the beginning or the ending of the message and then ignore the duplicates.

See the Windows API documentation for more information about mailslots.

Mailslot (Windows only)	Description	Return
MailSlotClose(integer index)	Closes the specified mailslot. Returns FALSE if successful.	I
MailSlotCreate(string theSlotName)	Creates a mailslot with the specified <i>name</i> . Returns the index number of the mailslot, or a zero if the call failed.	I
MailSlotRead(integer index)	Reads the next message from the specified mailslot. This function will return an empty string if there are no messages waiting.	S
MailSlotSend(string theComputerName, string theSlotName, string message)	Sends a <i>message</i> to the specified mailslot(s). <i>TheComputerName</i> field specifies exactly which machine to send the message to. If this field is a star "*", this message will be broadcast to all mailslots with the specified mailslot name in the primary domain of the sending computer. If this field is a domain name, the message will be sent to all mailslots with the specified name in that domain. The <i>SlotName</i> field must contain the name of the specified mailslot, and cannot be wildcarded. Returns FALSE if successful.	I

ODBC

ODBC stands for Open Database Connectivity. This Microsoft API allows data connectivity between applications that support it, and databases that support it. The following functions allow MODL access to the ODBC API. The following diagram shows the basic sequence of commands that you would use to connect to a database, and retrieve data.



ODBC sequence of operations

If you are trying to change data in the database, or add data, you could substitute the ODBCInsertRows, and/or ODBCSetRows calls for the ODBCExecuteQuery, ODBCBindColumn, and ODBCFetchRows combination. The information listed here is about the MODL implementation of functions that allow access to the ODBC API.

To use these functions effectively, you will need documentation on ODBC, SQL, and the database you are targeting as well as this documentation. The Microsoft documentation on ODBC can easily be found by doing an Exact Phrase search for "ODBC API Reference" on the MSDN Online Search site.

There is a sample block in the ModL Tips library) in the Input/Output category that shows the syntax of these commands. The block is called ODBC, and can be used as an ODBC scripting tutorial, or as a test of the ODBC functionality.

ODBC	Description	Return
ODBCBindColumn (integer statement, integer whichCol, string data[])	Associates an array with a column in the specific dataset. <i>WhichCol</i> defines which column of the dataset you wish to bind the array data with. The contents of that column will be different dependent on dataset, and how it was derived. This does not actually retrieve the data, just specifies where the data will go when it is retrieved. Returns FALSE (0) if unsuccessful. (See ODBCFetchRows below.)	I

ODBC	Description	Return
ODBCColAttribute (integer statement, integer column, integer which)	Returns the value of the specified attribute of the specified column. This function just filters directly through to SQLColAttribute, check the ODBC documentation for additional information. This function can be called as soon as there is a defined dataset.	I
ODBCColumns (integer connection)	Executes a standard query that returns a dataset containing the names of all the valid columns in the data source. Returns a Statement Handle. (Note: it is important to free all statements before disconnecting the connection, otherwise, the disconnection may fail.) Returns FALSE (0) if the query fails. See your ODBC documentation (SQLColumns) for a list of the meanings of the columns of the resulting dataset.	I
ODBCColumns2(integer hdbc, string catalog, string schema, string table, string column)	This function works the same as the ODBCColumns function except that it adds four string arguments. See your ODBC documentation (SQLColumns) for additional information about the meaning and use of these arguments. Please note that the arguments are strings, so you cannot pass in the NULL values that are defined in the SQLColumns specification. For this function, we've defined an empty string "" to be interpreted as a NULL, so just use empty strings for unused arguments where NULL is normally used.	I
ODBCConfigDataSource(integer fRequest, string szDriver, string szAttributes)	Calls the Windows API SQLConfigDataSource() function with the entered arguments	
ODBCConnect (string szDBName, string szUserName, string szPassword)	Connects to an ODBC source. Returns a connection Handle. (Note: It is very important to disconnect this connection before quitting ExtendSim, otherwise a crash may result.) Returns FALSE (0) if the connection fails. You need to have created a valid ODBC Data Source to connect with before using this call.	I
ODBCConnect-Name ()	Returns the string name of the current connection.	S
ODBCCountRows (integer connection, string tableName, string columnName, string whichCondition)	Uses the SQL COUNT statement to return the number of rows in the specified column of the specified table. The statement executed will be SELECT COUNT (ColumnName) FROM "tableName" WHERE whichCondition. ColumnName will default to "*" if it is blank. WhichCondition will specify a selection condition, if it is blank, all rows will be counted. See your SQL documentation for additional information about this query.	I
ODBCCreateTable (Integer connection, string tableName, stringarray columnNames, string columnTypes[])	This function will create a table in the specified database with columns named as the values in the columnnames array, and types as specified in the columntypes array. Returns FALSE (0) if unsuccessful.	I

ODBC	Description	Return
ODBCDriverConnect(string szConnectString)	Connects to an ODBC source, putting up a system source selection dialog. Returns a connection Handle. (Note: it is important to disconnect this connection before quitting ExtendSim, otherwise a crash may result.) Returns FALSE (0) if the connection fails. You need to have created a valid ODBC Data Source to connect with before using this call.	I
ODBCDisconnect (integer connection)	Disconnects the specified connection. Returns FALSE (0) if unsuccessful.	I
ODBCExecuteArray(integer hdbc, string array[])	See the description for ODBCExecuteQuery() below. This function allows a query that contains more than 255 characters by allowing you to pass in an array of strings instead of just one string.	I
ODBCExecuteQuery (integer connection, string theQuery)	Executes the specified SQL query string. Returns a Statement Handle. (Note: it is very important to free all statements before disconnecting the connection, otherwise the disconnection may fail.) Returns FALSE (0) if the query fails.	I
ODBCFetchRows (integer statement)	Fetches the data from the dataset, and stores it in the variables that have been bound by ODBCBindColumn. Returns the number of rows. (See ODBCBindColumn above.)	I
ODBCFreeStatement (integer statement)	Frees the specified Statement Handle. Returns FALSE if successful.	I
ODBCInsertRow (Integer connection, string tableName, string columnNames[], string values[])	This Function add a row of data with the specified values to the indicated table. Returns FALSE (0) if unsuccessful.	I
ODBCKeyword(string word)	Returns a TRUE value if the string word is an ODBC reserved keyword. Otherwise returns a FALSE value.	I
ODBCNumResultCols (integer statement)	Returns the number of resulting columns in the specified dataset. This function will only return useful information after an ODBCExecuteQuery, ODBCTables, or ODBCColumns call has established a dataset.	I
ODBCSetRows (integer connection, string tableName, string columnName, string IDName, string dataArray[], string IDArray[])	Copies the data from the data array into the columnName array, using the IDArray values to determine which cell each item goes into. That is, the variable named in the IDName field will be compared to the values in the IDArray array, and each row of the columnName variable will be updated based on the database row selected by the values in the IDArray array. Returns FALSE (0) if unsuccessful.	I

ODBC	Description	Return
ODBCSetRow- sType(integer hdbc, string tableName, string colName, string IDName, string y[], string x[], integer varTypeX, integer varTypeY)	See the description for the ODBCSetRows function. Whereas the SetRows function defaults the types of the values of both string arrays to SQL_CHAR, the SetRowsType function allows you to specify the types of the variables in the y and x arrays. Type values are: SQL_UNKNOWN_TYPE 0, SQL_CHAR 1, SQL_NUMERIC 2, SQL_DECIMAL 3, SQL_INTEGER 4, SQL_SMALLINT 5, SQL_FLOAT 6, SQL_REAL 7, SQL_DOUBLE 8, SQL_DATETIME 9, SQL_VARCHAR 12, SQL_TYPE_DATE 91, SQL_TYPE_TIME 92, SQL_TYPE_TIMESTAMP 93	I
ODBCSuccessInfo (integer ShowSuc- cessInfo)	Sets a flag that determines if warning error messages are shown, or not.	V
ODBCTables (inte- ger connection)	Executes a standard query that returns a dataset containing the names of all the valid tables in the data source. Returns a Statement Handle. (Note: it is very important to free all statements before disconnecting the connection, otherwise the disconnection may fail.) Returns FALSE (0) if the query is unsuccessful. See your ODBC documentation (SQLTables) for a list of the meanings of the columns of the resulting dataset.	I

Serial I/O

These functions allow ExtendSim to interact with serial ports. You can, for example, set up a simulation that will cause a modem attached to a computer to dial up a remote database, collect data, run a simulation, and send the results to another location over the modem. Likewise, an ExtendSim simulation can be controlled by a modem from a remote location.

- Windows: The port arguments are from 1 through 4 for COM1 through COM4, respectively.
- Mac OS: The port arguments are 0 for serial port A (modem port) or 1 for port B (printer port).

 If you use serial functions in your ModL code, and there is any possibility that your blocks will be used cross-platform (e.g. on both Windows and Mac OS systems), your code should include checks to allow for the differences between platforms. For more information, see “Cross-Platform Considerations” on page 403.

The SerialRead and SerialWrite functions are asynchronous, meaning that they return immediately without waiting for a response from the modem. SerialRead reads from the input buffer, not from the modem, and SerialWrite writes to the output buffer.

Serial I/O	Description	Return
SerialRead(integer port)	Returns a string if there is any read data, or an empty string if data is not available. If there are more than 255 characters in the serial port buffer, it returns the first 255 characters. Successive SerialRead calls will return the rest of the characters in the buffer.	S

Serial I/O	Description	Return
SerialReset(integer port, integer baud, real stop, integer parity, integer data, integer xonXoff)	Sets up one of the serial ports for communications. <i>baud</i> can be 300, 600, 1200, 2400, 4800, 9600, 19200, or 57600; <i>stop</i> can be 1, 1.5, or 2; <i>parity</i> is 0 for no parity, 1 for odd parity, or 2 for even parity; <i>data</i> can be 5, 6, 7, or 8 data bits. Set <i>xonXoff</i> to TRUE for XON/XOFF handshaking or FALSE for CTS handshaking.	V
SerialWrite(integer port, string s)	Writes the string <i>s</i> to the serial port buffer.	V

DLLs and Shared Libraries

ExtendSim can interact with DLLs (Windows only) and Shared Libraries (Macintosh only) through the following functions. The DLLs and Shared Libraries are stored in the \Extensions\DLLs folder.

There are two sets of DLL functions:

- The first set is used for calling existing Windows DLLs. These DLL functions allow a variable argument list and have different calls based on the calling convention of the DLL.
- The second set is designed specifically for cross-platform compatibility with the XCMDs in the Mac OS version of ExtendSim. This set of functions has a simpler interface, but is less flexible in calling convention and argument lists.

Working with DLLs will be a lot easier if you make note of the following:

- DLLs called by ExtendSim must be built for 32-bit execution.
- Variables passed from the ModL code to a DLL are passed by value unless the variables are strings or arrays, in which case they are passed as pointers. Strings are passed to DLLs from ModL as Pascal strings, not C strings. See “DLLs” on page 83, for more information.
- If you use DLL functions in your ModL code, and it is possible that your blocks will be used cross-platform (for example, on both Windows and Mac OS systems), your code should include checks to allow for the differences between platforms. For more information, see “Extensions” on page 408.

Also see “DLLs” on page 83 for more information about using DLLs and the folder \Examples\Developer Tips\DLLs for a DLL example.

Existing Windows DLLs

These function calls are called with variable argument lists. The first argument in each case is the Proc-Address for the procedure. This is returned by the function DLLMakeProcInstance. It is recommended that you call DLLMakeProcInstance once (in On OpenModel or in On InitSim), and save the return value in a variable, rather than call it each time you call the specific function.

Note: It is critical that you match the argument list and the calling convention in the ModL code with the argument list and calling convention in the DLL, otherwise a crash could result.

DLLs (existing)	Description	Return
DLLBoolCFunction(integer procAddress,)	Calls a DLL routine referenced by procAddress and returns a Boolean (translated into an integer by ExtendSim). Accepts a variable argument list. Assumes a C calling convention.	I
DLLBoolPascalFunction(integer procAddress,)	Calls a DLL routine referenced by procAddress and returns a Boolean (translated into an integer by ExtendSim). Accepts a variable argument list. Assumes a Pascal calling convention.	I
DLLBoolStdcallFunction(integer procAddress, ...)	Calls a DLL routine referenced by procAddress and returns a Boolean (translated into an integer by ExtendSim). This function is the same as the other DLL functions except it assumes a StdCall calling convention.	I
DLLCtoPString(string theString)	Converts a C string to a V string that is useable by ModL, as ExtendSim/ModL internally can use only V (Pascal) strings. In some cases pre-established DLLs will expect and return C format strings in the passed parameter string pointer, and this function can convert the string type safely within the pointer space. See DLLPtoCString below to convert back to C strings.	V
DLLDoubleCFunction(integer procAddress,)	Calls a DLL routine referenced by procAddress and returns a real. Accepts a variable argument list. Assumes a C calling convention.	R
DLLDoublePascalFunction(integer procAddress,)	Calls a DLL routine referenced by procAddress and returns a real. Accepts a variable argument list. Assumes a Pascal calling convention.	R
DLLDoubleStdcallFunction(integer procAddress, ...)	Calls a DLL routine referenced by procAddress and returns a real. This function assumes a StdCall calling convention.	R
DLLLoadLibrary(string pathName)	Loads the specified library. This is for people who need to access routines in a DLL that is not present in the Extensions folder. After loading the library, attempts to access DLL routines that are in that library should succeed.	I
DLLLongCFunction(integer procAddress,)	Calls a DLL routine referenced by procAddress and returns an integer. Accepts a variable argument list. Assumes a C calling convention.	I
DLLLongPascalFunction(integer procAddress,)	Calls a DLL routine referenced by procAddress and returns an integer. Accepts a variable argument list. Assumes a Pascal calling convention.	I
DLLPtoCString(string theString)	Converts a V string to a C string that is useable by a DLL, as ExtendSim/ModL internally can use only V (Pascal) strings. In some cases pre-established DLLs will expect and return C format strings in the passed parameter string pointer, and this function can convert the string type safely within the pointer space. See DLLCtoPString above to convert back to V strings.	V
DLLLongStdcallFunction(integer procAddress, ...)	Calls a DLL routine referenced by procAddress and returns an integer. This function assumes a StdCall calling convention.	I

DLLs (existing)	Description	Return
DLLMakeProcInstance(string procName)	Returns the ProcAddress expected by the other calls as an argument. Requires a procedure name. This function will search all open libraries for the named procedure, so it is advisable to call it once, and save the returned value. Function will return a zero if procName was not found.	I
DLLVoidCFunction(integer procAddress,)	Calls a DLL routine referenced by procAddress and returns nothing. Accepts a variable argument list. Assumes a Pascal calling convention.	V
DLLVoidPascalFunction(integer procAddress,)	Calls a DLL routine referenced by procAddress and returns nothing. Accepts a variable argument list. Assumes a Pascal calling convention.	V
DLLVoidStdcallFunction(integer procAddress, ...)	Calls a DLL routine referenced by procAddress and returns nothing. This function assumes a StdCall calling convention.	V

DLLs for compatibility

These functions operate by calling the DLLParam and DLLArray functions once for each parameter you want to pass to the DLL. This puts the parameter information into a structure which is passed to the DLL by the DLLXCMD function, which actually executes the DLL.

The parameters are not retained in memory after the DLLXCMD call, so the parameters must be reestablished before you call the DLLXCMD function again.

These function are equivalents of the XCMD, XCMDParam, and XCMDArray functions. They are designed so existing ModL code that calls XCMD functions does not need to be modified to access a DLL instead of an XCMD.

 The actual argument list and calling convention of the DLL are not variable. Code should be based on one of the DLLXCMD examples shipped with ExtendSim so that these will be correct.

DLLs (compatibility)	Description	Return
DLLXCMD(string procName, integer type)	Executes the procedure referred to by the <i>ProcName</i> and returns a string. Passes in the argument values established by prior calls to DLLParam and DLLArray. <i>Type</i> is ignored.	S
DLLArray(array)	Adds an array argument to the list of arguments.	V
DLLParam(string arg)	Adds a string argument to the list of arguments.	V

Alerts and prompts

These functions can be used to display results and diagnostics as well as to prompt for input data.

Alerts & Prompts	Description	Return
Beep()	Causes the computer to beep using the beep sound selected in the Control Panel.	V
NumericParameter(string message, real default)	Similar to the userParameter function, except that this returns a real value. This function will return a NOVALUE if the user clicks on the cancel button.	R
NumericParameter2(string message1, real default, string message2, real default2, real array[])	Similar to the numericparameter function, except that this function returns two values in a real array declared: real array[2]; The return value of the function itself will be a zero if the user successfully input one or more numbers, or a -1 if they canceled.	
PlaySound(string soundName)	Plays the sound named in the argument. Returns FALSE if no error, TRUE if the sound is not found. See “Sounds” on page 85 for important details.	I
Speak(string s)	(Mac OS only) Speaks the string if the speech manager is present.	V
UserError(string s)	Opens a dialog with an OK button displaying the string s.	V
UserParameter(string prompt, string default)	Opens a dialog to get a value. Displays the <i>prompt</i> string and <i>default</i> string, then returns either the entry typed, or the <i>default</i> string if there was no user entry, or the empty string (“”) if Cancel was clicked. After getting a string with UserParameter, you can convert the string to a real with the StrToReal function described later in the section on strings.	S
UserPrompt(string s)	Opens a dialog with an OK and a CANCEL button displaying the string s. The function returns TRUE if the OK button is clicked and FALSE if the Cancel button is clicked.	I
userPromptCustomButtons(string str, string button1, string button2)	User prompt with the ability to customize the text of the buttons. Returns a 1 (one) if the first button is clicked or 2 (two) if the second button is clicked.	I

Because of the automatic type conversion provided in ModL, and because they provide an easy way to display the contents of variables, these functions are also useful for debugging block code. For example:

```
UserError("X = "+x+", Y = "+y);
```

will display a dialog with something like:

```
X = 5.23, Y = .007
```

This is also an example of using the + operator for concatenating strings.

Also see other useful functions in “Debugging” on page 376.

User inputs

Use these functions to determine the location of mouse clicks and the status of modifier keys during the on blockClick and on dialogClick message handlers.

User inputs	Description	Return
GetModifierKey (integer whichKey)	Returns a 1 if the key is depressed, a 0 if the key is not depressed. Can be used in the on blockClick and on dialogClick message handlers. <i>whichKey 1 = Shift key</i> <i>whichKey 2 = Option (Mac OS) or Alt (Windows) key</i>	I
GetMouseX()	Returns the mouse X position in pixels relative to the model worksheet. Use the GetBlockTypePosition() function to get the coordinates of the block. Can be used in the on blockClick message handler.	I
GetMouseXActiveWindow()	Returns the mouse X position in pixels relative to the active window, for example, a hierarchical submodel window. Use the GetBlockTypePosition() function to get the coordinates of a block in that window. Can be used in the on blockClick message handler.	I
GetMouseY()	Returns the mouse Y position in pixels relative to the model worksheet. Use the GetBlockTypePosition() function to get the coordinates of the block. Can be used in the on BlockClick message handler.	I
GetMouseYActiveWindow()	Returns the mouse Y position in pixels relative to the active window, for example, a hierarchical submodel window. Use the GetBlockTypePosition() function to get the coordinates of a block in that window. Can be used in the on blockClick message handler.	I
HBlockClicked()	This function returns the block number of the hBlock that was last double clicked. This is intended to be called during a On HblockOpen message handler, and will always return a negative one at other times.	V
isKeyDown(integer keyCode)	Returns a True/False value for whether or not the specified key is pressed. The constants for <i>keyCode</i> are the constants for the API call GetKeyState (Windows) or the GetKeys functionality (Macintosh).	I
Lastkeypressed()	The value returned will be the ASCII value of the character entered for keys that have ASCII equivalents. For keys that don't have ASCII equivalents, the value returned is the keycode. This is useful for monitoring what keys the user hits on the keyboard. Used in conjunction with startTimer (See "Timer functions" on page 376) or during a simulation, it will allow live keyboard input.	I
WhichDialogItemClick()	Used in the dialogClick message handler to find the name of the item that received the click. This is used, for example, to modify the items in a popup menu at the time it is clicked on but before it opens to the user.	S

User inputs	Description	Return
WhichDTCell-Clicked(integer rowCol)	Returns the row or column of the cell in the data table that was clicked on. This function should only be used in the on dialogClick message handler, and usually after the whichDialogItemClicked function has determined that a specific data table was clicked on. <i>rowCol 0 = row</i> <i>rowCol 1 = col</i>	I

Animation

2D Animation

Use these functions to implement the ExtendSim 2D animation in a block. Examples of using these functions are shown in “2D animation” on page 120.

For 3D animation, see “3D Animation” on page 264. To instead implement Proof Animation in your blocks, please see the source code for blocks in the Item and 2D-3D Animation libraries.

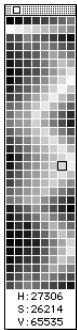
The Show 2D Animation command (Run menu) only controls whether animation is shown *during the simulation run*. At all other times, 2D animation is always available. For instance, animation is available when the modeler makes any changes in a block's dialog, whether Show 2D Animation is selected or not and regardless of whether the simulation is running. When the command Show 2D Animation is selected, animation is available at all times.

In the functions, “obj” is the integer object number of the animation object on the icon. As discussed in “Animating hierarchical blocks” on page 122, hierarchical blocks are animated indirectly (by blocks in the submodel) by referencing the negative of the hierarchical block's object number. If a negative number is used and ExtendSim doesn't find that object in the current enclosing H-block, ExtendSim will search the H-block enclosing that one and so on, until it finds the object or reaches the top level. The outsideIcon value is a value that is set to FALSE unless you want to move or stretch outside the original size of the icon (setting this to TRUE slows down the animation). All measurements are in pixels.

 If you don't want to be limited to using animation objects created on the block's icon when the block was created, and you want to create and delete new animation objects during the run, see the AnimationObjectCreate() and AnimationObjectDelete() functions below.

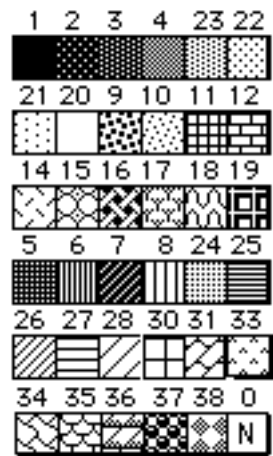
Color palette

AnimationColor specifies the color and pattern of the animated object. The color of an object is set with numbers for hue, saturation, and brightness (value), often called *HSV*. You can see the HSV values for any color on the bottom of ExtendSim's color swatch window:



Color palette

The number for the patterns that appear in the pull-down pattern menu are:



Other pattern numbers

Animation	Description	Return
AnimationBlockTo-Block (integer animationObjNum, integer blockNumFrom, integer conNumFrom, integer blockNumTo, integer conNumTo, real speed)	This function moves an animation object across the connection lines from one block to another. You specify the sending block and connector, and the receiving block and connector, as well as the number of the animation object. The <i>speed</i> value is a relative speed factor. Use a value of 1.0 for normal speed.	V
AnimationBorder(integer obj, integer pixels)	Changes the border size of an animation object. Pixels can be 0, for no border, and up to 20 for a 20 pixel border.	
AnimationBorder-Color(integer obj-Num, integer hue, integer saturation, integer value)	Allows you to specify the color of the border on an animation object. Only effects objects that have a visible border around them.	V

Animation	Description	Return
AnimationColor (integer obj, integer hue, integer sat, integer bright, integer pattern)	Sets the pattern and color of the object using the pattern number, hue, saturation, and brightness.	V
Animation-GetHeight(integer obj, integer getOrig)	Returns the offset of the height of the object. If <i>getOrig</i> is true, gets the original height.	I
AnimationGetLeft (integer obj)	Returns the offset of the left side of the object relative to its original position in the icon.	I
Animation-GetLeftRelative(integer blockNumber, integer objNum, integer original)	Gets the left of the animation object relative to the upper left hand corner of the block icon. See AnimationGetTopRelative for more info.	I
Animation-GetSpeed()	Returns the speed setting, with 1 being slowest and 5 being fastest. See AnimationSetSpeed(), below.	I
AnimationGetTop (integer obj)	Returns the offset of the top side of the object relative to its original position in the icon.	I
AnimationGet-TopRelative(integer blockNumber, integer objNum, integer original)	Gets the top of the animation object relative to the upper left hand corner of the block icon. Compare to AnimationGetTop, which returns the location relative to just the Animation Object location, and will always return zero unless the animation object has been moved with an AnimationMove call. Original specifies if the moved rect (0), or the original unmoved rect (1) should be used.	I
AnimationGet-Width(integer obj, integer getOrig)	Returns the width of the object. If <i>getOrig</i> is true, gets the original width before any stretching.	I
AnimationHide (integer obj, integer outsideIcon)	Immediately hides the object.	V
AnimationLevel (integer obj, real level)	Defines the object as a level whose height varies from 0.0 to 1.0 (based on the <i>level</i> argument).	V
AnimationMoveTo (integer obj, integer leftOffset, integer TopOffset, integer outsideIcon)	Moves the object relative to its original position in the icon. Note: pixel coordinates start at the top/left corner of the animation object and go down and to the right. <i>TopOffset</i> of 0 is the top of the animation object.	V

Animation	Description	Return
AnimationMovie (integer obj, string movieName, real speed, integer play- SoundTrack, inte- ger async)	(Mac OS only) Defines the object as a QuickTime movie, played at the specified speed (relative to 1.0, the normal speed). If <i>play-SoundTrack</i> is true, the soundtrack is played. If <i>async</i> is true, the movie starts and the function returns immediately; if it is false, the function will not return until the movie is finished. The movie must be a file in the ExtendSim7\Extensions folder. Unlike other resources, the “movieName” is its file name, not a resource name.	V
AnimationMovieF- inish(integer obj)	(Mac OS only) Returns TRUE if the QuickTime movie is finished or not currently playing; returns FALSE if the movie is still playing. This is useful if you want to wait for a QuickTime movie to finish playing before you restart it. If you call AnimationMovie with async TRUE, use this function to check the status of the movie at any time.	I
AnimationObject- CopyData(integer objNum, integer objNum2)	This function copies the animation object data, (like type of object, poly points, color, size, etc.) from one object to another one.	I
AnimationObject- Create(integer enclosingH- BlockIfTRUE)	Creates an animation object on the fly. The animation object number for the created object will be returned. If the enclosingH-BlockIfTRUE argument is set to one, or TRUE, the function will create the animation object in the enclosing hblock of the current block. In this case, the returned value will be the negative value of the animation object number. Animation objects create on the fly with this function are not part of the block structure, and will need to be recreated in openModel, or InitSim, or whenever they are used. They will persist until the model window (Including hblock model windows,) is closed, or AnimationObjectDelete is called.	I
AnimationObject- Delete(integer obj- Num)	This function will delete an animation object. This will not affect animation objects that are defined in the block structure, just those created on the fly with AnimationObjectCreate. Use negative obj-Num if in enclosing HBlock.	I
AnimationOb- jectExists(integer objNum)	Returns TRUE if an animation object with the specified <i>objNum</i> exists, and FALSE if it doesn't. For example, you could test if the enclosing hierarchical block has a specified animation object. (i.e. if animationObjectExists(-3) returns a TRUE value, then there is an animation object in the enclosing HBlock, or any number of levels above, with the objNum 3.)	I
AnimationObjectEx- ists2(integer block- Number, integer objNum)	Similar to the AnimationObjectExists function, except that this function includes a block number argument, allowing you to check for the existence of an animation object in a remote block.	I
AnimationOval (integer obj)	Defines the object as an oval.	V

Animation	Description	Return
AnimationPicture (integer obj, string picName, integer scaleToObj)	Defines the object as a picture. If TRUE, the <i>scaleToObj</i> argument scales the picture to the animation object size; if FALSE, the picture will not be scaled. See “Sounds on the Mac OS” on page 85 for important information about pictures.	V
AnimationPixel- Rect(integer obj, integer rows, inte- ger cols, integer intArray[])	Defines the object as a rectangular pixel map of <i>rows</i> height and <i>cols</i> width, where each pixel can have a specified color. ExtendSim normalizes the size of the pixels to conform to the animation object size. <i>IntArray</i> is declared as a dynamic array: integer intArray[];	V
AnimationPixelSet (integer obj, integer row, integer col, integer h, integer s, integer v, integer drawNow)	See AnimationPixelRect, above. Sets a specific pixel to an HSV color (see notes on color, above). <i>Row</i> values start from 0 to rows-1. <i>Col</i> values start from 0 to cols-1.	V
Animation- Poly(integer obj- Num, integer pointCount, inte- ger pointArray[])	Animates a polygon. PointCount is the number of points to be drawn, point array is a two dimensional array of points that contains the points to be animated. The points are defined in pixels from the upper left hand corner of the animation object location.	V
AnimationRectan- gle(integer obj)	Defines the object as a rectangle.	V
AnimationRndRect- angle(integer obj)	Defines the object as a rounded rectangle.	V
AnimationSetDe- layMode(integer trueIsDelay)	If trueIsDelay is TRUE (default), then ExtendSim uses its normal delay, corresponding to the animation speed that the user chose. FALSE removes any animation delay, no matter what speed the user chooses. Use this function to set your own delay for animation and have ExtendSim ignore the animation speed set by the user. See AnimationGetSpeed(), above, to see what speed the user chose.	V
Animation- SetSpeed(integer newSpeed)	Sets the animation speed, with 1 being slowest and 5 being fastest. See AnimationGetSpeed(), above.	V
AnimationShow (integer obj)	Shows a hidden object.	V
Animation- StretchTo(integer obj, integer leftOff- set, integer topOff- set, integer width, integer height, inte- ger outsidelcon)	Stretches the object relative to its original position in the icon.	V

Animation	Description	Return
AnimationText (integer obj, string msg)	Defines the object as text with the string <i>msg</i> . Also see the TextWidth function in <i>Strings</i> , below, that is useful in applying the AnimationStretchTo function to the object so that the text width is accommodated.	V
AnimationTextAlign (integer obj-Num, integer justification)	This function specifies the text alignment for the specified animation object. <i>Justification</i> takes the following values: Left: 0 Right: -1 Center: 1	V
AnimationTextSize(integer obj-Num, integer size)	Allows you to modify the size of animation text. The size argument is a font point size.	V
AnimationTextTransparent(integer objNum, string text)	Exactly the same as the animationText function, except the background behind the text will be transparent. It's normally white for the regular function.	V
DialogPicture (string variable-Name, string pictureName, integer scalePicture)	This function displays the picture <i>pictureName</i> over the static text item <i>variableName</i> . This is used to display a picture on a dialog box. Normally you would create an empty static text item, and use that as the location for displaying this picture. <i>ScalePicture</i> will take a TRUE or FALSE, and determines if the picture is scaled to the space, or not. Returns FALSE if that picture is not available.	I
PictureList (String15 array-Name[], integer type)	Resizes, and fills the dynamic array <i>arrayname</i> with the names of the pictures from the extension folder or the application to be used with the AnimationBlockToBlock() function. Return value is the number of pictures. <i>type</i> : 0 - All pictures in the Extensions folder. 1 - Only AnimationBlockToBlock pictures in the application. 2 - Only AnimationBlockToBlock pictures in the Extensions folder. Pictures with names that start with a "@" character are <u>not</u> AnimationBlockToBlock pictures and are not returned. The @ character prevents pictures from showing up in the block's animation tab popup menu. 3 - All non-AnimationBlockToBlock pictures (pictures that start with an @ character.	I
ProofEncode (string commandLine)	The version of the Proof Animation DLL that works with Extend-Sim requires a numeric value calculated by this function to be passed to it periodically. See the Proof Animation blocks in the Animation 2D-3D library to see how to use this function.	I
ProofEncodeReset()	Resets counters for Proof copy protection.	V

3D Animation

For the 3D functions, all distances are in meters, speeds are in meters per second, and time units are in real time seconds, which correspond to a single simulation time unit.

3D block functions

3D block	Description	Return
E3DBlockToBlock(integer sBlock, integer eBlock, string object-Name, integer object-Type, integer pathID, real speed, integer Self-Deleting)	Using the 3D Block locations set below by E3DSetBlockLocation, this function creates an object and moves it from the location of the sBlock to the location of the eBlock. This function is intended to allow a more complex path between two blocks compared to a straight line – a one step method to emulate 2D AnimationBlockToBlock in the 3D world. ObjectName and ObjectType specify the object to be created, as in E3DCreateObject. See the description of that function for more information. If PathID is nonzero, the function will move the object along the path specified by that ID. If selfDeleting is set to True, the object will be deleted when the animation is completed. If selfDeleting is False, the function will return the ID of the created object.	V
E3DBlockToBlockHeight(real Z)	Specifies a height for the next object to be moved with an E3DblockToBlock call. This is primarily for ExtendItems, as they have a traveling height. People and vehicles will drop to the ground, so this call will not effect them. This procedure should be called before the E3DblockToBlock call.	V
E3DBlockToBlockObjectLabel(label)	Specifies an object label for the next E3DBlockToBlock call. This procedure should be called before the E3DblockToBlock call.	V
E3DBlockToBlockRotation(real x, real y, real z)	Specifies the rotation values for the next E3DblockToBlock call. This procedure should be called before the E3DblockToBlock call.	V
E3DBlockToBlockScale(real x, real y, real z)	Specifies the scale values for the next E3DblockToBlock call. This procedure should be called before the E3DblockToBlock call.	V
E3DBlockToBlockSkin(string skinName, string skinTypeName)	Specifies the string skin names to be used by the next E3DblockToBlock call. This procedure should be called before the E3DblockToBlock call.	V
E3DBlockToBlockSkinByIndex(integer skinIndex1, integer skinIndex2)	Specifies the indexes of the string names to be used by the next E3DblockToBlock call. This procedure should be called before the E3DblockToBlock call. See E3DsetSkinByIndex for more information about the skinIndex values.	V
E3DGetBlockLocation(integer blockN, integer which)	Retrieves the x, y, or z value for the Block location set by E3DsetBlockLocation. The which argument takes the following values: 0: x 1: y 2: z	R
E3DGetObjectBlockNumber(integer objectID)	Gets the block number field on the specified 3D Object.	I
E3DSetBlockLocation(integer blockN, real x, real y, real z)	Sets an x, y, and z location for a block. This information is saved in the model.	V

3D block	Description	Return
E3DSetObjectBlockNumber(integer objectID, integer block-Number)	Sets the blocknumber field on the specified 3d object.	V

3D time functions

3D time	Description	Return
E3DCurrentTime()	Returns currentTime in the 3D windows playback of the model events.	R
E3DMode()	Returns the mode set by E3DsetMode.	I
E3DPauseAllObjects()	Pauses all objects. Pausing an object stops it from moving towards its destination, but records the destination within the object, so it can be resumed later if desired. This is used in the executive to pause all items in a model when the simulation is paused.	I
E3DResumeAllObjects()	Resumes all objects that have been paused, and have a saved resume destination. See E3DPauseAllObjects above.	I
E3DSetMode(integer mode)	This function sets the mode in which the 3D window, and block code, will expect to function. Takes the following values: 0: E3DQUICKVIEWMODE 1: E3DBUFFEREDMODE 2: E3DCONCURRENTMODE	V
E3DSetTimeRatio(real ratio)	Sets the 3DtimeRatio. See the description of E3DtimeRatio for more information.	V
E3Dsleep(integer milliseconds)	Delays for milliseconds. Stops processing MODL code for that integer while the 3D window keeps processing.	V
E3DTimeRatio()	Retrieves the 3D Time ratio. This number sets the correspondence between real-time and simulation time in the 3D window. The default value is 1000. Changing this will speed up, or slow down the Playback of 3D events in the 3D Window. For example, setting the Time Ratio to 2000, will make the 3D window run twice as slow as the default value of 1000. Basically this number is the number of milliseconds in a 3D window time unit.	R

3D rotation, position, and scale functions

3D time	Description	Return
E3DdistanceRatio()	Returns the ratio between pixels and meters set in the Simulation setup dialog.	R

3D time	Description	Return
E3DGetPosition(integer theObject, integer coord)	Returns the X, y, or z position value of the object. Coord takes the following values: 0: x 1: y 2: z	R
E3DGetRotation(integer objectID, integer which)	Returns the rotation information for the specified object. Which can take the following values: 0: x 1: y 2: z	R
E3DSetPosition(integer theObject, real x, real y, real z, dropToGround)	Sets the position of the specified object to the specified location. If dropToGround is true, the object will drop until it touches the ground.	I
E3DSetRotation(integer theObject, real x, real y, real z)	Sets the X, Y, and Z rotation values of the object. Currently only z values are meaningful, they will rotate around the z axis. Values are in radians. (I.e. 2 π will be 100% rotation.)	I
E3DSetScale(integer theObject, real x, real y, real z)	Sets the scale of the object to the specified scale. X, y, and z refer to three different dimensions of the object. By default x, y, and z are 1.0. If you set x, y, or z to values that are different from each other there will be some distortion in the appearance of the object.	I

3D window functions

3D window	Description	Return
E3DNotEnabled()	Checks if the 3D is enabled, or not, and returns why. Return value: 0: E3D enabled 1: Suite not enabled 2: machine not capable 3: 3D files missing	I
E3DOpenWindow()	Opens the 3D Window.	I
E3DUpdate()	Forces an update or redraw of the E3D window. This is not usually necessary.	V
E3DWindow()	Returns TRUE if the 3D Window is open.	I
E3DWindowConnect(d(void)	This function returns a TRUE (1) value if the current model (the one the function is being called from,) is the model connected to the E3D window. It returns a FALSE (0) value otherwise.	I

3D mountStack functions

A mountStack is a stack of objects mounted onto a mount node on some object. These function just simplify stacking multiple objects onto a single mount point.

3D mountStack	Description	Return
E3DMountStackInsert(integer objectID, integer targetID, integer index, integer baseNode)	Add an object to a MountStack. Object will be mounted onto target, at the index specified by index. Index starts at zero for the first object to be mounted on the target. The items in the stack will mount on each other based on the internal stackMountNode of the objects, with the exception of the base of the mountStack, which will use the Node specified by baseNode. If you pass in a -1 for baseNode, the object will use its stackMountNode. If you do not know which node you want to specify, just pass in a negative one for the baseNode.	I
E3DMountStackLimit(integer blockNumber, integer limit)	Determines the limit for mountStacks created by a specified block. The purpose is to reduce the number of objects displayed in the E3D window, enhancing performance. The mountStack limit only renders up to <i>limit</i> objects; after that it indicates the total count of objects as a number on the top of the stack. If not overridden by block code, the default mountStack limit is 10.	I
E3DMountStackRemove(integer objectID, integer index, integer baseNode, integer rule, integer direction, real distance)	Remove an object from a mountStack. Direction and distance are as defined in the Unmount functions. Rule determines how the index value is used. A rule value of zero, means that index is just a zero based index of which object is to be removed. A rule value of one means that index is the Object ID of the object to be removed. BaseNode specifies which node on the base object the mountStack is stacked on. If you pass in a -1 for baseNode, the object will use its stackMountNode. If you do not know which node you want to specify, just pass in a negative one for the baseNode.	I

3D path functions

3D path	Description	Return
E3DAddMarkerToPath(integer path, integer marker)	Adds the specified Marker to the specified path.	I
E3DCreateMarker(real x, real y, real z, integer spline, integer seqNum, integer GroupTag)	Creates a Marker for addition to a path. X, Y and Z specify the location of the marker. Spline specifies how the path will render when viewed in the Editor. If spline is true, the marker will be represented as connected to the other markers, by a curved line, otherwise it will be connected with a straight line. The seqNum argument should be 1 for the first point on the path, and should count up incrementally. If the markers are not sequential, the behavior of the path will be unpredictable. See the creation functions section for more information about the GroupTag.	I
E3DCreatePath(string name, integer looping, integer GroupTag)	Creates a path with the specified name. Looping determines whether the path is looping, or not. If the path is looping, an item that is traveling on the path will go back to the beginning of the path when it leaves the last marker on the path. See the creation functions section for more information about the GroupTag.	I

3D path	Description	Return
E3DGetPathByName(string name)	Returns the ObjectID number for the path with the specified name.	I
E3DPathMarkerLocation(integer path, integer index, integer coord)	Returns the x, y, or z position value of the nth marker on the specified path. Index specifies which marker you want the coordinate value for, starting at one for the first one, and coord specifies which coordinate value you want. This function returns a -10000 when it fails. Coord takes the following values: 0: x 1: y 2: z	R
E3DPathSetColor(integer objectID, integer firstColor, integer secondColor, integer thirdColor, integer RGB)	Sets the color of the specified path. Paths are invisible by default, so this function will seem to have no effect until the E3DSetVisibility function is also called on the path. ObjectID specifies which path is to have its color set. FirstColor, secondColor, and thirdColor have different meanings depending on the value of the RGB argument. If the RGB argument is set to TRUE, firstColor is red, secondColor is green, and thirdColor is blue. If RGB is FALSE, first Color is hue, second is saturation, and third color is value. (See separate discussion of RGB and HSV colors.)	I

3D object selection functions

3D object selection	Description	Return
E3DGetSelectedObject()	Returns the ObjectID for the object currently selected in the 3D window.	I
E3DObjectClicked()	Returns the objectID of the last E3D object clicked.	I
E3DSetSelectedObject(integer object)	Sets the selection in the 3D window to be the object specified.	I
E3DUnselectObject(integer objectID)	Unselects the specified object. This will have no effect if the specified object is not the selected object in the E3D window.	I

3D environment functions

3D environment	Description	Return
E3DBoundingBoxes(integer boundingBoxes)	Sets a flag which decides if objects drawn in the 3d Window should try to draw their bounding boxes. This will only affect ExtendItems, ExtendPlayers, and ExtendVehicles.	I
E3DChooseEnvironment(string Environment-Name)	Sets the Environment file name that is associated with the model.	I
E3DSetDetail(real detailValue)	Sets the level of detail for the objects in the E3D window. This controls the same variable as the LOD control in the options dialog.	V

3D environment	Description	Return
E3DSetTerrain(string terrain, integer which)	Sets the Terrain texture file to be associated with the terrain. Which takes a value of 1-6, which corresponds to which one of the 6 textures the terrain editor defines is to be changed. (By default the terrain is all drawn in texture one (1), so if you just want to change the texture of the default environment file, just use 1 for the which value.	V
E3DSky(integer sky-Setting, integer red, integer green, integer blue)	<i>Red/Green/Blue : 0-255, skySetting:</i> 0: default (Clouds and Skybox) 1: no Clouds (just Skybox) 2: no skybox 3: ceiling (uses color, no clouds or fog)	V
E3DSun(integer sun-Setting)	<i>Red/Green/Blue : 0-255, sunSetting:</i> 0: No visible sun (sunFlare is disabled) 1: Sun is visible, (Sun flare is enabled)	I

3D creation and deletion functions

GroupTag is user-defined identifier to group types of objects. For example you could use 1 to be the type of all block-to-block items that you want to delete at the end of the simulation.

3D creation/deletion	Description	Return
E3DChangeObject(integer objectID, unsigned char *ObjectName)	Changes the specified object into another type of object. ObjectName specifies which type of object to change the chosen object into. Please note that you should only attempt to change objects of a like type into other objects. For example, changing a 'Male' object into a 'Female' object should be fine, because both objects are ExtendPlayer Objects, but changing a 'Male' object into a 'Boid' object is not allowed, and will not work.	I
E3DCopyObject(integer objectID, real x, real y, real z)	Creates a copy of the object referenced by objectID at location x, y, z. This copy will have all the information associated with the object being copied, with the exception of location, and any other objects mounted on it.) If the X, Y and/or Z values are blanks or novalues, then the existing position of the object will be used for that coordinate.	I
E3DCreateObject(string objectName, integer objectType, real x, real y, real z, integer collidable, integer Group-Tag)	Creates an object at location x, y, z. ObjectName specifies what kind of object to create. ObjectType specifies what type of object is to be created. Zero is a special type that tries to create the appropriate type for that particular object. This is almost always what you will want to use from MODL, unless you are custom coding using custom 3D objects.	I

3D creation/deletion	Description	Return
E3DCreateObjectAtWayPoint(string object-Name, integer object-Type, integer waypointID, string name, integer collidable, integer Group-Tag)	Creates an object at waypointID.	I
E3DCreateWayPoint(real x, real y, real z, string name, integer GroupTag)	Creates and returns a waypointID at the specified location.	I
E3DDeleteAllObjects(integer GroupTag)	If tag is ≤ -1 , all objects will be cleared. If tag is greater than -1 , only objects that have that GroupTag will be cleared. Clears all objects and events created from MODL from the 3D Window.	I
E3DDeleteObject(integer theObject, integer deleteMount)	Deletes the specified object. If deleteMount is true, objects mounted to the specified object will be delete.	I

3D movement functions

3D movement	Description	Return
E3DGetDestination(integer objectID, integer coord)	Returns on of the destination coordinants of the specified object. <i>coord</i> 0: x 1: y 2: z	R
E3DGetSpeed(integer objectID)	Returns the speed of the specified object.	R
E3DResumeSpeed(integer objectID)	Resumes the previous speed value of the object. Each object internally saves its old speed when a new speed value is set. This function resets the speed to the last value used.	I
E3DSetDestination(integer objectID, real speed, real x, real y, real z, integer sync, integer selfDeleting)	Sets a destination for the specified E3D object. The types of objects that can have a destination set are ExtendItems, ExtendPlayers, and ExtendVehicles. This excludes scenery and block objects, which do not normally move. The speed argument will be used by to the object as it travels to its destination. X, Y, and Z are the coordinates of the destination. The sync argument determines if the motion of the object will be synchronous, or asynchronous. Synchronous means that the call will wait to return until the specified object reaches its destination. This has a potential danger, in that if the object is blocked, and does not reach its destination, the call will not return, and the code will hang. If selfdeleting is set to true, then the object will delete itself when it reaches its destination.	I

3D movement	Description	Return
E3DSetSpeed(integer objectID, real speed)	Sets the speed of the specified object.	I
E3DSetTargetObject(integer objectID, integer targetID, string name, real speed, integer sync, integer selfDeleting)	Sets a target object for the object. The types of objects that can have a path set are ExtendItems, ExtendPlayers, and ExtendVehicles. This excludes scenery and block objects, which do not normally move. The speed argument will be used by to the object as it travels to its destination. TargetID is the ID of the target object. The sync argument determines if the motion of the object will be synchronous, or asynchronous. Synchronous means that the call will wait to return until the specified object reaches the target. This has a potential danger, in that if the object is blocked, and does not reach its destination, the call will not return, and the code will hang. If selfdeleting is set to true, then the object will delete itself when it reaches the target object.	I
E3DSetTargetPath(integer objectID, integer targetID, string name, real speed, integer sync, integer selfDeleting, integer startingMarker)	Sets a target path for the object. The types of objects that can have a path set are ExtendItems, ExtendPlayers, and ExtendVehicles. This excludes scenery and block objects, which do not normally move. The speed argument will be used by to the object as it travels to its destination. TargetID is the ID of the target path. The sync argument determines if the motion of the object will be synchronous, or asynchronous. Synchronous means that the call will wait to return until the specified object reaches the end of the path. This has a potential danger, in that if the object is blocked, and does not reach its destination, the call will not return, and the code will hang. If selfdeleting is set to true, then the object will delete itself when it reaches the end of the path. The starting-Marker defines which marker starts the path, usually one.	I
E3DSetTargetWaypoint(integer objectID, integer targetID, string name, real speed, integer sync, integer selfDeleting)	Sets a target waypoint for the object. The types of objects that can have a path set are ExtendItems, ExtendPlayers, and ExtendVehicles. This excludes scenery and block objects, which do not normally move. The speed argument will be used by to the object as it travels to its destination. TargetID is the ID of the target waypoint. The sync argument determines if the motion of the object will be synchronous, or asynchronous. Synchronous means that the call will wait to return until the specified object reaches the target. This has a potential danger, in that if the object is blocked, and does not reach its destination, the call will not return, and the code will hang. If selfdeleting is set to true, then the object will delete itself when it reaches the target waypoint.	I
E3DStopAllObjects()	This function stops all objects that have a destination. This would usually be called when the E3D animation is complete, as it will clear destinations, and there will not be a way of resuming the motion.	I

3D post function

Post functions are the equivalent of the similarly named regular E3D function, with the addition of a time argument specifying when the E3D action will occur. See the appropriate E3D function for more information about the meaning of the arguments of the functions. Times are specified in

simulation time units, and the most common value of this argument will be to place the system variable `currentTime` into it.

3D post

<code>E3DPostAnimationPlay(real time, integer objectID, string which, integer forward, integer threadID)</code>
<code>E3DPostAnimationSetPosition(real time, integer objectID, string which, real pos)</code>
<code>E3DPostAnimationStop(real time, integer objectID, string which)</code>
<code>E3DPostChangeObject(real time, integer objectID, string ObjectName)</code>
<code>E3DPostCopyObject(real time, integer objectID, real x, real y, real z)</code>
<code>E3DPostCreateObject(real time, string objectName, integer objectType, real x, real y, real z, integer collideable, integer GroupTag)</code>
<code>E3DPostCreateObjectAtWayPoint(real time, string objectName, integer objectType, integer wayPointID, string wayPointName, integer collideable, integer GroupTag)</code>
<code>E3DPostDeleteAllObjects(real time, integer groupTag)</code>
<code>E3DPostDeleteObject(real time, integer ObjectID, integer deleteMounted)</code>
<code>E3DPostMountObject(real time, integer objectID, integer targetID, integer node)</code>
<code>E3DPostMountStackInsert(real time, integer ObjectID, integer targetID, integer index, integer baseNode)</code>
<code>E3DPostMountStackRemove(real time, integer ObjectID, integer index, integer baseNode, integer rule, integer direction, real distance)</code>
<code>E3DPostObjectPropertySet(real time, integer objectID, integer which, real value)</code>
<code>E3DPostResumeSpeed(real time, integer objectID)</code>
<code>E3DPostRotateTimed(real startTime, real endTime, integer objectID, real startZ, real endZ, integer ticks)</code>
<code>E3DPostSetDestination(real time, integer objectID, real duration, real speed, real x, real y, real z, integer selfDeleting)</code>
<code>E3DPostSetObjectBlockNumber(real time, integer objectID, integer blockNumber)</code>
<code>E3DPostSetObjectLabel(real time, integer objectID, string objectLabel)</code>
<code>E3DPostSetPosition(real time, integer objectID, real x, real y, real z, integer drop)</code>
<code>E3DPostSetRotation(real time, integer objectID, real x, real y, real z)</code>
<code>E3DPostSetScale(real time, integer objectID, real x, real y, real z)</code>
<code>E3DPostSetSkin(real time, integer objectID, string skinName, string skinBaseName)</code>
<code>E3DPostSetSkinByIndex(real time, integer objectID, integer skinIndex1, integer skinIndex2)</code>
<code>E3DPostSetSpeed(real time, integer objectID, real speed)</code>
<code>E3DPostSetTargetObject(real time, integer objectID, real duration, real speed, integer targetID, string name, integer selfDeleting)</code>

3D post

E3DPostSetTargetPath(real time, integer objectID, real duration, real speed, integer targetID, string name, integer selfDeleting)

E3DPostSetTargetWayPoint(real time, integer objectID, real duration, real speed, integer targetID, string name, integer selfDeleting)

E3DPostSetVisibility(real time, integer objectID, integer visible)

E3DPostStopAllObjects(real time, integer sync)

E3DPostUnmountObject(real time, integer objectID, integer targetID, integer direction, real distance)

E3DSetObjectLabel(integer objectID, string name)

3D mounting functions

3D mounting	Description	Return
E3DGetMount(integer objectID)	Returns the objectID for the object that the specified object is mounted to.	I
E3DGetMountedObject(integer objectID, integer node)	Returns the ObjectID for the object that is mounted to the specified object at the specified node.	I
E3DMountObject(integer mountID, integer riderID, integer node)	Mounts the object referred by riderID onto the object referred to by mountID. Node specifies the mountNode to use on the object to be mounted. Node would be set to 0, for objects that have only one mount point. Pass in a -1 if you want Extend to do a smart mount, i.e. to mount a person object to the seat, and a box object to the fork of a forklift, for example.	I
E3DUnmountObject(integer mountID, integer riderID, integer direction, real distance)	Unmounts the rider from the mount. Direction specifies which way to use the optional offset distance. This will move the object a certain distance when the unmount is completed. Distance is in 3D units. (Meters.) Direction: 1: north, 2: west, 3: east, 4: south, 5: up If you pass in a zero for the mountID, the function will unmount the rider from whatever mount it happens to be mounted on.	I

3D animation functions

3D animation	Description	Return
E3DAnimationName(integer objectID, integer which)	Returns the name of the animation on the specified object that corresponds to the 'which' parameter.	S
E3DAnimationPlay(integer objectID, string which, integer backward)	Plays the animation specified by the 'which' parameter on the object specified by the ObjectID parameter. Backward should be set to true of you want the animation to play backward.	I

3D animation	Description	Return
E3DAnimationSetPosition(integer objectID, which, real pos)	Sets the specified animation to the specified position.	I
E3DAnimationStop(integer objectID, string which)	Stops the animation thread specified by the string <i>which</i> . Note that this also stops the Torque animation thread from running. In most cases this doesn't matter, as ExtendSim manages the Torque threads for you. Examples of where it does matter are animations where two sequences use the same parts of the object, or if you have more than eight different animations that you want to run. In either case, you need to stop one thread to successfully run the next.	I
E3DDialogPicture(string variableName, string objectName, real rotation, real rotation2)	Shows the object picture within a dialog's static text item.	I
E3DGetObjectAnimationNames(integer ObjectID, string theArray[])	Gets the object's animation names into theArray.	I
E3DObjectSnapshot(string fileName, integer objectID, real rotation, real rotation2)	Saves a .png file with the object's picture.	I

3D name functions

3D name	Description	Return
E3DGetName(integer objectID)	Returns the name of the specified object. Useful for uniquely naming 3d objects, this is the name used in the E3DgetPathByName, and E3DgetObjectByName functions	S
E3DGetObjectByName(string name)	Returns the objectID for the object with the specified name.	I
E3DGetObjectLabel(integer objectID)	Gets the ObjectLabel of the specified object. (This is the name/label that displays above the object in the 3D Window.) Note for E3D Developers, in the 3D engine this is the ShapeName.	S

3D name	Description	Return
E3DGetObjectList(integer which, string31 nameArray[])	This function returns a list of object type names of all the objects of the type specified by the 'which' argument. Please note that these are not the object names defined by E3DSetObjectName, or the object labels defined by E3DsetObjectLabel, but rather the object type names. I.e.: "Male", "Female", "Ball", and so on. This function is used to identify all the possible types of objects that can be created. NameArray need to be a string31 dynamic array. It will be resized by the function. <i>Which</i> values: -1: all possible objects 0: ExtendItem + ExtendPlayer + ExtendVehicle 1: block 2: scenery 3: ExtendSim 4: ExtendPlayer 5: ExtendVehicle	I
E3DGetObjectNames(integer which, string31 nameArray[])	Fills the passed in string31 array with the names of the E3D Objects in the 3D world. The array should be a string31 dynamic array, which the function will resize based on the number of names to be returned. <i>Which</i> values: 0 or greater: groupTag -1: everything -2: Paths -3: WayPoints -4: ExtendItems -5: Players -6: wheeled vehicles	I
E3DObjectLabelColorSet(integer objectID, integer firstColor, integer secondColor, integer thirdColor, integer RGB)	Sets the color of the object label on the specified object. Nothing will be drawn if the objectLabel is empty, so this function will need to be used in conjunction with E3DSetObjectLabel. FirstColor, secondColor, and thirdColor have different meanings depending on the value of the RGB argument. If the RGB argument is set to TRUE, firstColor is red, secondColor is green, and thirdColor is blue. If RGB is FALSE, first Color is hue, second is saturation, and third color is value. (See separate discussion of RGB and HSV colors.)	I
E3DSetName(integer objectID, string name)	Sets the name of the specified object. This is the name by which the object is referred to in the 3D window. This name will be visible in the editor. Useful for uniquely naming 3d objects, this is the name used in the E3DgetPathByName, and E3DgetObjectByName functions.	I
E3DSetObjectLabel(integer objectID, string shapeName)	Sets the ObjectLabel of the specified object. (This is the name/label that displays above the object in the 3D Window.) Note for E3D Developers, internally in the 3D engine this is the Shape-Name.	I

3D screenshot functions

3D screenshot	Description	Return
E3DPanoramicScreenShot(string fileName)	Creates three screenshot files in the Extend directory. Put together, the screen shots will create a panoramic view of the contents of the 3D window.	V
E3DScreenShot(string fileName)	Creates a screenshot file in the Extend directory. Currently the file type of the screen shot file is PNG. So the file name will be file-name.png.	V

3D distance functions

3D distance	Description	Return
E3DDistance(integer objectID, real x, real y, real z)	Returns the distance from the location of the specified object to the specified location.	R
E3DDistancePath(integer pathID)	Returns the total distance of the path.	R
E3DDistanceToObject(integer objectID, integer targetID)	Returns the distance between the two objects.	R

3D skin functions

3D skin	Description	Return
E3DGetObjectSkinBase(string objectName, integer which)	Returns the SkinBaseNames for the object type specified by objectName. Which specifies if you want base name 1, or base name 2. Currently the only objects that have meaningful base names are the Male and Female object, as discussed under E3DgetObjectName above. I.e. for the female object, calling this function with a which value of 2, will return the string "femaleSkin".	S
E3DGetObjectSkinNames(integer objectID, string baseName, str32 theArray[])	Returns the names of the skins for the object. For most objects, you can leave the basename field blank (""). For objects with multiple skins, it specifies which skin you want to set. Specifically, at the moment, there are two objects that support more than one skin, the female and male figures. For the female, specify "femaleClothes" for her clothes, and "femaleskin" for her skin. For the male, its "MaleClothes" and "maleSkin".	I

3D skin	Description	Return
E3DSetSkin(integer objectID, string skin-Name, string base-Name)	Allows you to set the skin of a 3D object. For most objects, you can leave the basename field blank (""). For object with multiple skins, it specifies which skin you want to set. Specifically, at the moment, there are two objects that support more than one skin, the female and male figures. For the female, specify "female-Clothes" for her clothes, and "femaleskin" for her skin. For the male, its "MaleClothes" and "maleSkin". Returns an error code number (Non-zero) if it fails.	I
E3DSetSkinByIndex(integer objectID, integer skinIndex1, integer skinIndex2)	This function lets you set the skin values of an object by numeric index values. The first skinIndex number modifies the first object skin, and the second modifies the second skin. Note that most objects (currently, all object except the people,) only have one skin, so the second number will be ignored. <i>skinIndex</i> values: -1: make the skin value a random choice 0: keep the skin value the same 1-n: use a specific skin >n: use n	I

3D vector functions

3D vector	Description	Return
E3DForwardAngle(integer objectID, integer targetID)	This function returns the forward angle for the Object. Where zero means directly in front, PI means directly behind.	R
E3DForwardVector(integer objectID, integer which)	This function returns the forward vector for the Object. The forward vector is the vector forward from the Object. The which argument specifies which part of the vector is to be returned. <i>Which</i> values are: 0: X 1: Y 2: Z	R
E3DGetAngleFromVector(real vectorX, real vectorY, real vectorZ, integer which)	This function returns the yaw and pitch angles. The range of yaw is 0 - 2PI. The range of pitch is -PI/2 - PI/2. <i>Which</i> values: 0: yaw 1: pitch	R
E3DGetObjectVector(integer objectID, integer which, real vector[])	This function returns a vector for the object, depending on the value of the which argument. Vector needs to be a predefined single row real array with at least three elements. <i>Which</i> takes the following values: 0: position 1: forward Vector	I
E3DGetVectorFromAngles(real yaw, real pitch, integer which)	This function returns the vector from the specified angles. The range of yaw is 0 - 2PI. The range of pitch is -PI/2 - PI/2. <i>Which</i> values are: 0: X 1: Y 2: Z	R
E3DNormalizeVector(real vector[])	Takes the vector passed in, and normalizes the array. Vector needs to be a predefined single row real array with at least three elements.	I

3D user tag functions

ObjectIDs are unique identifiers defined by the E3D engine to identify each object in the E3D world. Normally functions that create an object, will return a resulting ObjectID. In the case of the Post functions, this is not possible, because the post functions are called before they are executed. A userTag is a value returned by a E3DpostCreate function, as it does not immediately create an object, instead creates it at the specified simulation time.

UserTag values are negative numbers, and objectID values are positive numbers. Objects created from Extend retain their userTags even after the objectID value is created when the object is actually created in the 3D window, and all Extend functions can recognize either.

3D user tag	Description	Return
E3DObjectIDToUserTag(integer objectID)	Returns an UserTag value from a ObjectID. This function is the reverse of the E3DUserTagToObjectID function. See that functions description for more information. UserTag values are negative numbers, and objectID values are positive numbers. All Extend functions can recognize either.	I
E3DUserTagToObjectID(integer userTag)	Returns an objectID value from a userTag. An ObjectID value is the reference ID of a 3D object, as used by the 3D engine. A userTag is a value returned by E3DPostCreateObject, as it does not immediately create an object, instead creates it at the specified simulation time. UserTag values are negative numbers, and objectID values are positive numbers. This function does not usually need to be called, as objects created from Extend retain their userTags even after the objectID value is created when the object is actually created in the 3D window, and all Extend functions can recognize either.	I

3D tick functions

These functions start a timer specifically for the use of the 3D window. The E3DTICK message will be sent to the block periodically once the E3DTimer has been started with E3DStartTick. Unlike the ExtendSim timers, described separately, the E3Dtimer is controlled by the E3D engine. It will be sent on each tick of the E3D engine, which will be frequently enough to update something that needs to move smoothly in the E3D window. If you want a timer that is based on simulation time, you should use the regular ExtendSim Timers.

3D tick	Description	Return
E3DStartTick(integer blockNumber)	blockNumber specifies which block should get the E3DTICK message. If you specify a negative one (-1) for the block number, all the blocks in the model will receive the E3DTICK message.	V
E3DStopTick(integer blockNumber)	Stops the E3D timer for the specific block.	V

3D object property functions

3D object properties	Description	Return
E3DCollisionShow(integer objectID, integer show)	Shows the collision rect for the specified object. This will currently only show up once the object has moved.	V

3D object properties	Description	Return
E3DGetObjectBounds (integer objectID, integer which)	Returns the bounds of the specified object in the 3D world. Which specifies what part of the bounds information is to be returned. Zero through eight return values that are in World Coordinates. World Coordinates are based on the objects position in the E3D World. 10 through 18 return values that are in Object Coordinates. Object Coordinates are based on the object position as if the object were located at the origin. <i>Which:</i> 0: min X, 1: min Y, 2: min Z, 3: max X, 4: max Y, 5: max Z, 6: X length, 7: Y length, 8: Z length, 10: min X (obj Bounds), 11: min Y, 12: min Z, 13: max X, 14: max Y, 15: max Z, 16: X length (obj Bounds), 17: Y length (obj Bounds), 18: Z length (obj Bounds)	R
E3DGetObjectIDs(integer which, integer theArray)	This function fills the passed in array of longs with the object ids of all the 3D objects in the 3D window. The 'which' parameter can be used to specify which objects you want to list. <i>Which:</i> > 0: groupTag, -1: everything, -2: Paths, -3: WayPoints, -4: ExtendlItems, -5: Players, -6: wheeled vehicles	I
E3DGetObjectInfo(integer objectID, integer which)	Returns the specified information about the object. 'Which' takes the following arguments: 0: userTag, 1: objectID, 2: groupTag, 3: savedInEnvironmentFile Flag	I
E3DGetObjectType(unsigned char *ObjectName)	This function returns the type of a 3D object, based on the name. The possible type values are: 1: block, 2: scenery, 3: item, 4: person, 5: vehicle. If the object doesn't fall into any of these categories, a zero will be returned.	I
E3DObjectPropertySet (integer objectID, integer which, real value)	<i>which:</i> 1: collidable. (true/false)	I
E3DSetVisibility(integer objectID, integer vis)	Sets the visibility flag on the item specified by objectID. A vis value of TRUE means the item is visible. (The default.)	I
3D script functions		
3D script	Description	Return
E3DExec(string script)	Executes the contents of the string script through the Torque compiler.	I
E3DExecFile(string path)	Executes a .cs file through the Torque compiler.	I

3D miscellaneous functions

3D miscellaneous	Description	Return
E3DCollisionBlocker(integer objectID, integer blocker)	Collision optimization. Used by the Transport and Convey Item blocks to inform a 3D object that it is in a queuing situation and can optimize its collision calculations by only looking at the object specified in the blocker argument. <i>Blocker</i> should be the object ID of the object that is in front of the specified object in the queue.	V
E3DGetCamera()	Returns the ObjectID for the camera object.	I
E3DLogEvents(integer logFlag)	Turns on (or off) logging of the Events in the E3D window. Logging will write messages to the Console.log file in the ExtendSim7 folder when events occur in the E3D window. Logging is off by default. If logFlag is true (1), it will be turned on.	
E3DMessageInfo(integer which)	Returns information about a message sent by the E3D window to ExtendSim. This function is used in the E3DCollision message handler to request the 3D object ID of the object that is about to collide (<i>which</i> =1) or the object that is about to be collided with (<i>which</i> =2).	I
E3DNeighborDetection(integer objectID, integer which, real distance, real angle, integer sort, integer neighbors[])	This function acts like 'vision' for objects in the E3D world. It detects all the objects within a certain range of a specified item, and returns the number of objects 'seen', as well as filling an array with the Object Ids of the objects. <i>Which</i> is used to specify which types of items are to be detected, see below for possible values. Distance determines how far to detect other objects. Angle determines how broadly to detect the objects, and the angle starts straight ahead and goes both right and left, so 3.14 (Pi radians) will detect everything on both sides, front to back. Sort determines if the object Ids should be sorted on return, and neighbors is the array of objects Ids that are returned. An angle value of zero is special cased to mean don't check the angle. This should have the same affect as an angle value of 3.14. <i>Which</i> : 0: everything 1: ExtendItems	I
E3DObjectExists(integer objectID)	Returns a true (1) if the specified object exists, and a false (0) if it doesn't.	I
E3DPlaySound(string soundProfileName, real volume, real x, real y, real z)	Plays the sound specified by soundProfileName. At the location specified by x,y,z.	I
E3DUses3D()	Returns the value of the uses3D flag from the sim setup dialog for the active model.	I

Blocks and inter-block communications

Block numbers, labels, names, type, position

Blocks, text, and anchor points are numbered uniquely and sequentially from 0 to n-1 as they are added to the model window. The block numbers assigned to each item do not change. If a block or

text is deleted, its number becomes an “unused slot” which is available when another block or text is added to the model.

There are two numbers associated with blocks. *Global numbers* access all blocks, including blocks inside of hierarchical blocks. *Local numbers* access a hierarchical block’s internal blocks and are the same for all instances of that hierarchical block, whereas the global numbers are different for each instance. Blocks are numbered from 0 to NumBlocks-1. Block numbers show in a dialog’s title bar.

Block names are the names you give a block when you create it. For example, “Math” is the name of a block in the Value library. Names appear in the dialog’s title bar. *Block labels* are user-definable names given to a particular block in the model. Block labels are entered in the area to the right of the Help button at the bottom of a block’s dialog. Labels appear at the bottom of the block’s icon.

Block type strings are the categories that appear in the library menu. Examples include *Math* or *Holding* from the Value library.

Block numbers, etc.	Description	Return
BlockName(integer i)	Looks in global slot <i>i</i> and returns either the name of the block, the first 255 characters of text, or an empty string if it is an unused slot. For example, you can use this function to read text information that you have added or modified in a model.	S
BlockRect (integer blockNumber, integer array[], integer useAnimation)	Virtually the same as the getBlockTypePosition function, except for the useAnimation argument. This argument specifies (with a true/false, 1/0 value,) whether or not the animation objects are to be included in the returned rectangle. If useAnimation is true, the rectangle will include the positions of the animation objects that are off the icon in the rectangle, otherwise it will not.	I
FindInHierarchy (string FindBlockName, string FindBlockLabel, string FindDialogName, integer FindDialog)	This function is used by several blocks, including the Catch Item and Throw Item blocks in the Item library, to locate the corresponding block associated with the current block. (For example, to find a Catch corresponding to a Throw, and vice versa.) Please see the Catch and Throw blocks for an example of how to use this function. The function returns the block number of the resulting block found; it returns a -1 if no matching block is found. FindBlockName is the name of the block to be found (e.g. ‘Catch’, or ‘Throw’). FindBlockLabel is the block label of the block to be found. FindDialogName is the name of the dialog item you wish to search for. The FindDialogName field should be filled with the dialog item name, a colon, and then the value of the dialog item that you wish to search for. For example Item:54 will search for the presence of a dialog item with the name “item”, and the value “54”. FindDialog takes the following values: 0: just check the block name, and the block label. 1: just check the block name and the dialog item name and value. 2: check the name, label, and dialog item.	I
FindInHierarchy2(integer blockNumber, string findBlockName, string findBlockLabel, integer findDialog)	The same as the FindInHierarchy function except it takes a block number argument.	I

Block numbers, etc.	Description	Return
GetBlockLabel(integer i)	Returns the label string for the global block <i>i</i> .	S
GetBlockType(integer i)	Returns the block type string for the global block <i>i</i> (e.g. <i>Math</i> or <i>Holding</i>).	S
GetBlockTypePosition(integer i, integer intArray[4])	Returns the type (not the block type string as in GetBlockType() above) and position for the global block <i>i</i>. Types are as follows: 0: empty slot 1: anchor point 2: text 3: block 4: hierarchical block 5: embedded object 6: Slider, Switch, or Meter (from the Model > Controls command) Declare <i>intArray</i> as follows: integer intArray[4]; On return, intarray[0] will contain the top, intarray[1] left, intarray[2] bottom, intarray[3] right position pixel values. To include animation objects, see the blockRect function.	I
GetEnclosingHBlockNum()	Returns the global block number for the enclosing hierarchical block. Returns -1 if there is no enclosing hierarchical block.	I
GetEnclosingHBlockNum2(integer block)	This is the same as GetEnclosingHBlockNum(), except it refers to a global block number.	I
GetStaticNames(integer blockNumber, array names, array types)	Fills the names and types arrays with the names and types of all the static variables defined in the block structure. Names should be an array of strings, and types should be an array of longs. The value of the cells of the types array will be from the following list: 3: LONGTYPE 6: DOUBLETTYPE 7: STRING255TYPE 8: STRING15TYPE 9: STRING31TYPE 10: STRING63TYPE 11: STRING127TYPE	I
GlobalToLocal(integer blockNumber)	Returns the local block number for the specified block if it is contained in an H-block. Returns a -1 if the block is not contained in an H-block.	I
LocalNumBlocks()	Number of internal blocks in the hierarchical block from which this function is called. Blocks are numbered from 0 to LocalNumBlocks()-1.	I
LocalNumBlocks2(integer block)	This is same as LocalNumBlock(), except it refers to a global block number.	I

Block numbers, etc.	Description	Return
LocalToGlobal (integer i)	Converts a local number in the hierarchical block from which this function is called to a global number.	I
LocalToGlobal2 (integer HBlockNum, integer localBlockNum)	Similar to LocalToGlobal, this function will return the global block-number for a local one. The difference is that this function allows you to specify which HBlock in the model is used as the context for the local block number via the first parameter HblockNum.	I
MakeOptimizer-Block (integer true-False)	This function tags the block as an optimizer block so that the Run > Run Optimization command will send a RUN message to that block, assuming there is a "Run" button in the block that will run the optimization and has an "ON RUN" message handler. Call this function in "CREATEBLOCK." See the Optimizer block (Value library).	V
MyBlockNumber()	Global number of the block in which the function is called. Note that this number is the first number shown in parentheses in the block dialog's title.	I
MyLocalBlock-Number()	Local number of the block in which the function is called. Note that this number is the second number shown in parentheses in the block dialog's title.	I
NumBlocks()	Global number of blocks in the model, including text blocks and unused slots. Blocks are numbered from 0 to NumBlocks()-1.	I
SetBlockLabel (integer i, string str)	Sets the label for the global block <i>i</i> to <i>str</i> . Blocks are numbered from 0 to NumBlocks()-1.	V
ShowBlockLabel (integer i, integer show)	Labels are shown by default. To hide a block's label, use this function with the global block number <i>i</i> and FALSE for <i>show</i> .	V

Block connectors and connection information

The following functions give you information about connectors and connected blocks in a model. This helps you manipulate connectors and determine what is connected to a block or to create network lists of your models.

In these functions, "block" is the global block number (see "Block numbers, labels, names, type, position" on page 282), "connName" is the connector's name (without quotes, not a string), "connString" is the connector's name (with quotes), and "conn" is the *ith* connector (0 to *n*-1) in the list of connectors in the structure window's connector pane. To determine the connector number for a given connector name, look in the block's connector pane, counting from the top of the list (which is connector 0). "block" and "conn" are integers.

Also see “Variable connectors” on page 288, below, to see their use and to see the connector labeling functions.

Block connection	Description	Return
GetConBlocks(integer block, integer conn, integer intArray[][2])	Returns the number of blocks attached to the connector. On return, the rows of <i>intArray</i> are used to index the connected blocks (if there are three blocks connected, there are three rows). The first column of <i>intArray</i> contains the global block number of a connected block, the second column contains the connector number of that connected block. Rows and columns are indexed 0 to n-1. <i>intArray</i> is declared as a dynamic array: integer intArray[][2];	I
GetConHBlocks(integer blockNumber, integer iConn, array y[])	Similar to the GetConBlocks() function, above, but returns a list of connected Hblocks. You can also start from a textblock or anchor point if you know the block number (connector numbers are zero for textblocks or anchor points).	I
GetConName(integer block, integer conn)	Name of the connector.	S
GetConnectedTextBlock(integer block, integer conn)	If <i>block</i> is positive, returns the block number of the named connection (bypasses connector text blocks inside an HBlock and goes out HBlock connector) connected to the connector <i>conn</i> of block number <i>block</i> . If <i>block</i> is negative, stay inside the HBlock (can return the connector text block inside of an HBlock). Use the GetBlockName() function to retrieve the text of the named connection.	I
GetConnectedType(name connName)	Tells the type of connector connected to this connector. This is useful for determining what is connected to a universal connector. You can read the result by number or their associated ExtendSim constants: 13 value connector 14 discrete connector 15 universal connector 16 diamond connector 25 flow connector	I
GetConnectedType2(integer block, integer conn)	This is similar to GetConnectedType(), except it refers to a global block number and <i>conn</i> is the <i>ith</i> connector (0 to n-1).	I
GetConnectionColor(integer blockFrom, integer conFrom, integer blockTo, integer conTo, integer colorArray[])	Fills the <i>color</i> array (three integer element) with the hue, saturation, and brightness (HSV) values for the color of the connection line. The HSV color palette is on page 252 of the printed Developer Reference. Returns 0 for success or a negative value to indicate failure.	I

Block connection	Description	Return
GetConnectorPosition(integer blockNumber, integer con, Array)	Returns the position of the specified connector in pixels. Array should be defined as a four row integer array (<code>integer array[4];</code>), which on return from the function call will be filled with the four values: array[0] = top array[1] = left array[2] = bottom array[3] = right	I
GetConnectorType(integer blockNumber, integer con)	Returns the connector type of the specified connector. 13 value connector 14 discrete connector 15 universal connector 16 diamond connector 25 flow connector	I
GetConNumber(integer blockNumber, string conNameStr)	Returns the Connector number of the connector of the specified name.	I
GetEnclosingHBlockCon(integer blockNumber, integer conNum)	Used to find the connector index of the outer connector, given a blockNumber and connector index inside the HBlock that is connected to the HBlock's internal connector text. Returns connector number of the enclosing Hblock's outer connector or -1 if not an Hblock.	I
GetIndexedConValue(integer conn)	Gets the connector's current value. For blocks with many similar connectors, use this in a loop instead of lots of statements like "value[22]=con23In". The connectors are indexed from 0; the indexes are the same as the order of the connector names in the connectors pane.	R
GetIndexedConValue2(integer block, integer conn)	This is same as GetIndexedConValue(), except it refers to a global block number.	R
GetIntermediateBlocks(integer blockNum1, integer conn1, integer blockNum2, integer conn2, array)	Returns number of connected dot blocks, text blocks, and connectors between two blocks. Fills the array with information about the blocks. See GetConBlocks function for definition of the array. If blockNum2 is negative, returns above information between a block (blockNum1) and the text block or connector text block specified by blockNum2 (can be inside a hierarchical block).	I
GetNumCons(integer block)	Number of connectors on the block.	I
GetRightClickedCon()	This function returns the conn number of the last connector that was right clicked on. This function should be called in the CONNectorRIGHTCLICK message.	I
IsConVisible(integer blockNumber, integer conn)	Tests the specified connector for visibility. Returns TRUE if the connector is visible, FALSE otherwise	I

Block connection	Description	Return
MakeFeedback-Block(feedBack)	If <i>feedBack</i> is TRUE, this function causes ExtendSim to terminate the flow order search for this block. For multiple feedback cases, this function prevents feedback from unexpectedly changing the main flow simulation order. For an example, see the Feedback block (Utilities library).	V
NodeGetCurrent-Value(nodeIDIndex)	Returns the currently set value of the connected block connectors. See NodeGetIDIndex(), below.	R
NodeGetIDIndex(blockNumber, conNum)	Returns the nodeID for a connected network of block connectors. Each connected network has a unique nodeID, which can change when additional block connectors are connected or the model is reopened. The nodeID is equivalent to an index of a real value that holds the set value of the connected connectors.	I
SetConnection-Color (integer blockFrom, integer conFrom, integer blockTo, integer conTo, integer hue, integer saturation, integer value)	Sets the color value of the specified connection line. See “Color palette” on page 260 for a description of the hue, saturation, and value arguments. This function returns a TRUE if it succeeds, and a FALSE if it fails.	I
SetConVisibility(integer block-Number, integer conn, integer visible)	Sets the visibility flag on an individual connector. This function can be used by block developers to hide and show connectors based on choices the user has made in the dialog.	V
SetIndexedCon-Value(integer conn, real value)	Sets the connector’s numerical value. For blocks with many similar connectors, use this in a loop instead of lots of statements like “con23Out=value[22]”. The connectors are indexed from 0; the indexes are the same as the order of the connector names in the connectors pane.	V
SetIndexedConValue2(integer block, integer conn, real value)	This is same as SetIndexedConValue(), except it refers to a global block number.	V
SetSelectedConnectionColor (integer hue, integer saturation, integer value)	The setSelectedConnectionColor function sets the color value of all selected connections. See “Color palette” on page 260 for a description of the hue, saturation and value arguments. This function returns a TRUE if it succeeds, and a FALSE if it fails.	I

Variable connectors

These functions are used with a block’s variable connectors, and to interface them with the standard connector functions. Note that all of the single connector functions, above, also work. Where *conn* is specified in the argument list, use ConArrayGetConNumber() to find the actual connector number on the block from the array owner and nth connector in the array. For example, to send a message out of a variable connector, call ConArrayGetConNumber() to get the actual number of

the connector (its index). Then call `SendMsgTo...`() and instead of using the actual connector name (e.g. `FlowIn`), use V7's new feature and use the connector number you got from `ConArray-GetConNumber()`.

These connector functions use a block number and the connector name as a string so you can use them to control other blocks.

Variable connectors	Description	Return
<code>ConArrayChanged-WhichCon()</code>	When the user drags to get more or less variable connectors, the block gets a <code>CONARRAYCHANGED</code> message. This function returns the owning connector name, as a string, that is being dragged. Use this string in the <code>ConArrayGetNumCons()</code> function to get the actual number of connectors during the dragging operation.	
<code>ConArrayGetCollapsed(integer block-Number, string origConName)</code>	Returns True if the specified connector is collapsed, or false if it is not.	I
<code>ConArrayGetCon-Number(blockNum-ber, origConNameStr, nthConn)</code>	Returns the connector number of the nth connector in this array. <code>OrigConNameStr</code> is the owning connector name as a string. <code>NthConn</code> is the index of the array connector, starting from 0 for the owning connector, to the number of connectors in this array minus 1 for the last connector. The returned connector number can be used in the other connector functions.	I
<code>ConArrayGetDirec-tion(blockNumber, origConNameStr)</code>	Returns the array direction as: 0 top, 1 right, 2 bottom, 3 left. If not an array connector or an error, returns -1.	I
<code>ConArrayGetNth-Con(conn)</code>	Given a block connector number, returns the member index of its connector array (e.g. Given the block connector number of <code>ConIn[3]</code> (could be 253), returns 3 meaning <code>ConIn[3]</code>). A -1 returned is an error.	I
<code>ConArrayGetNthCo-n2(blocknum, conn)</code>	Same as <code>ConArrayGetNthCon</code> except it has an additional argument to specify the connector on a different block.	I
<code>ConArrayGetNum-Cons(blockNumber, origConNameStr)</code>	Returns the number of connectors in this array. If there are no added array connectors, returns 1 for the original connector. <code>OrigConNameStr</code> is the owning connector name as a string.	I
<code>ConArrayGetOwner-Con(conn)</code>	Given a block connector number, returns the block connector number of the owning (originating) connector of a connector array (e.g. Given the block connector number of <code>ConIn[3]</code> (could be 253), returns the block connector number of <code>ConIn[0]</code>). A -1 returned is an error.	I
<code>ConArrayGetTotalC-ons(blockNumber)</code>	Returns the total number of connectors in this block, including array connectors. Used to prevent a dragged connector array from adding too many connectors, causing the block to have more than its limit of 255 connectors.	I

Variable connectors	Description	Return
ConArrayGetValue(ConName, nthConn)	Returns the value of the nth connector of the array owned by ConName. ConName can be the name of the connector without quotes or, new for V7, the index from 0 to numCons-1 as a constant or variable (If the compiler detects a connector name, it turns it into an index rather than evaluating its value).	R
ConArrayMsgFrom-Con()	Returns the index of the connector that received the message in an array. For example, if the Con1In message handler received a message, calling this function at the beginning of the message handler would return which connector in this array (from 0 to num-1) received the message.	I
ConArraySendMsgToAllCons(integer origConName, integer nthConn)	Sends a connector message to all connectors connected to the specified Connector. See SendMsgToAllCons.	V
ConArraySendMsgToInputs(integer origConName, integer nthConn)	Sends a connector message to input connectors connected to the specified Connector. See SendMsgToInputs.	V
ConArraySendMsgToOutputs(integer origConName, integer nthConn)	Sends a connector message to output connectors connected to the specified Connector. See SendMsgToOutputs.	V
ConArraySetCollapsed(integer blockNumber, string origConName, integer trueOrFalse)	Sets the collapsed state on the specified connector. True will collapse the connector, and False will uncollapse it.	V
ConArraySetNumCons(blockNumber, origConNameStr, newNumCons, trueToIgnoreConnections)	Call this to set the number of connectors in a variable connector (a 1 for NewNumCons means to only keep the original connector). If trueToIgnoreConnections is TRUE, the user can delete connectors that have connections on them. If trueToIgnoreConnections is FALSE, connectors that have connections on them will not be deleted. This means that connections may prevent all of the desired connectors from being deleted.	V
ConArraySetValue(ConName, nthConn, value)	Sets the value of the nth connector of the array owned by ConName. ConName can be the name of the connector without quotes or, new for V7, the index from 0 to numCons-1 as a constant or variable (If the compiler detects a connector name (not a string), it turns it into an index rather than evaluating its value).	V
ConnectorLabelsGet(blockNumber, origConNum, labelsStrArray[])	Call this to get the connector labels for single connectors or connector arrays. labelsStrArray is any kind of string array (local, static, or dynamic) that will contains all the labels used, one per array element, which will allow up to n labels to be retrieved. Each label is the ith connector in that connector array (0th for single connectors). Returns the number of labels retrieved.	I

Variable connectors	Description	Return
ConnectorLabels-Set(blockNumber, origConNum, numLabels, labelsStrArray[], position, hue, saturation, value)	Call this to set the connector labels for single connectors or connector arrays. labelsStrArray is any kind of string array (local, static, or dynamic) containing all the labels needed, one per array element. The labels can contain combinations of style information such as <biur> for bold, italics, underlined, right adjusted. Positions are: 0 top, 1 right, 2 bottom, 3 left.	V

Connector tool tips

The System message ConnectorToolTip is sent to the block when the cursor hovers over a connector. In that message handler, the connector tool tip functions (below) will allow you to customize and access the tooltip that will appear.

Connector tool tips	Description	Return
ConnectorToolTip-Get(integer blockNumber, integer connectorIndex)	Gets the current string value of a connector tool tip from a block and connector. This is useful if you want to show what block your block is connected to, like the plotter, batch, and select blocks.	S
ConnectorToolTip-Set(stringstring, integer replace)	Allows you to set the string that will be displayed. If the replace flag is true, the default string generated by ExtendSim will be replaced, if it is false, this string will be appended to the default string.	V
ConnectorToolTip-Which()	When the ConnectorToolTip message is received, this function returns the connector index of the connector the cursor is over. This function is also used in the ConnectionMake message to return the connector being connected.	I

Dialog items

These functions work with a block's dialog items and can be used to get and set values from dialog items in other blocks, as well as change dialog item colors, create popup menus, hide and show items, etc.

The functions in this list that include a global block number in their argument list can affect any block in the model, including the calling block if its own block number (from the MyBlockNumber() function) is used.

 Block controls (switch, slider, and meter) can be set or changed via the Get/SetDialogVariable functions or poke/request functionality just the same way other types of blocks can. In the case of the block meter, this is easy to recognize, as this control has a dialog, and therefore all the pieces that you need for setting and getting the values are available. (Block number, and dialog item name.) In the case of the slider and the switch it's not so obvious, as these controls don't have dialogs, and therefore the dialog item name is not readily available. The names for these dialog items follow the pattern of the meter, and therefore are as follows:

- Switch: value – This is whether the switch is on, or off, and just takes a value of 0, or 1.
- Slider: value – The current value of the slider. (The position of the thumb.)
 - lower – The value of the lower bound of the slider.
 - upper – The value of the upper bound of the slider.

For additional formatting options, see “Formatting/interactivity using column and parameter tags” on page 305. These facilitate both specialized formatting and mouse-interactivity with parameters and data tables. Also see “Block data tables” on page 297, “Dynamic text items” on page 304, and “Dynamic linking” on page 302.

Block dialog items	Description	Return
AppendPopupLabels(string variable-NameStr, string theLabels)	This function appends the specified string onto the labels associated with the named popup menu. Popup menu labels can total up to 5100 characters. This function is the method for adding menu labels. <i>VariableNameStr</i> is the dialog item name as a string or in quotes. See SetPopupLabels() below.	I
CreatePopupMenu(string string1, string string2, integer initialSelection)	Creates a Popup at the last location the user clicked. This function can be used in conjunction with the on dialogClick message handler, the whichDialogItemClicked function, and the whichDTCellClicked function to create a popup menu that appears on a dialog item, or data table cell in response to a user's click. See the code of the on dialogClick messagehandler of the Activity block (Item library) for an example of how to use this function. Also see PopupMenuArray(), below.	I
DialogHasEmbeddedObject (integer blockNumber)	Returns the dialog item name if the specified block contains any embedded object dialog items in its dialog. This function will return an empty string "" if there are no dialog items of this type in the dialog.	S
DialogItemVisible (integer blockNumber, string variable-NameStr, integer clones)	Returns a true value if the specified dialog item is visible. If the Clones value is a one (TRUE) this function checks to see if any clones of the specified item are visible; otherwise it checks to see if the primary item is visible.	I
DIGetID(integer blockNumber, string name)	Returns a dialog item ID number. This is used in the LINKCONTENT and LINKSTRUCTURE message handlers to help identify which dialog item is getting the message.	I
DIMoveBy(integer blockNumber, string name, integer y, integer x)	Offsets the dialog item by the values of the y and x parameters.	I
DIMoveTo(integer blockNumber, string name, integer top, integer left)	Moves the specified dialog item to the y and x location specified by the top and left variables.	I
DIMsgNumber(integer blockNumber, string name)	Returns the message number associated with a dialog item. Used to send a dialog item message to a block.	I

Block dialog items	Description	Return
DIPopupButton(integer blockNumber, string name, integer behavesAsButton)	Changes the behavior of the specified popup menu so that it will not show the typical popup behavior when it is clicked, but will instead behave like a button (Sending a message to the MODL code, but not displaying a popup menu). This is used when the block developer wants something that looks like a popup menu, but has a button's behavior.	I
DIPositionGet(integer blockNumber, string name, integer array position)	Gets the position of a dialog item specified by blockNumber, and dialogItemName. The coordinates of the dialog items location will be placed into the integer array position. Position needs to be declared as: integer array[4];	I
DIPositionHome(integer blockNumber, string name)	Resets the location of the dialog item back to the position and size defined in the structure.	I
DIPositionSet(integer blockNumber, string name, integer top, integer left, integer bottom, integer right)	Sets the position of the specified dialog item. This function also includes the bottom and right arguments so you can change the displayed size of the dialog item as well.	I
DisableDialogItem(string variableNameStr, Integer TRUEFALSE)	Disables, or enables the specified dialog item.	V
DisableDialogItem2(integer blockNumber, string name, integer disableEnable)	Disables the specified Dialog Item. This function differs from the DisableDialogItem() function in that it takes a block number argument so you can disable a dialog item from a remote block.	I
DISetFocus(integer blockNumber, string dialogItemName)	Sets the focus on the specified dialog item. Returns 0 for success or a negative value to indicate failure.	I
DITitleGet(integer blockNumber, string dialogItemName)	Returns the string title of the dialog item. For dialog items like radio buttons and check boxes, the title is the text that appears on the dialog. Use this function to retrieve the string if it has been set by the function DITitleSet.	S
DITitleSet(integer blockNumber, string name, string title)	Sets the title of a dialog item. Useful for Dialog items like radio buttons and checkboxes where there is no other way to change the title of the dialog item on the fly.	I

Block dialog items	Description	Return
GetDialogColors (integer blockNumber, integer HSVColorArray[][3])	Copies all the color information from the dialog colors into the color array. Used to save all the colors in a dialog in on dialog close. The HSVColorArray is declared as a dynamic array: integer HSVColorArray[][3];	I
GetDialogItemColor (integer blockNumber, string variableNameStr, integer HSVColorArray[3])	Gets a color value associated with the dialog item.	I
GetDialogItemInfo(integer blockNumber, string variableNameStr, integer which)	Returns TRUE if <i>which</i> qualities are true for the dialog item: 0: exist, 1: hidden, 2: enabled, 3: display only. Returns the values for: 4: dialog item type, 5: rows, 6: columns, 7: width, 8: height. Dialog item types are: 1: button, 2: checkbox, 3: radiobutton, 4: meter, 5: parameter, 6: slider, 7: datatable, 8: edittext, 9: stattext, 12: switch, 13: stringtable, 14: plotpane, 16: popupmenu, 17: embedobject, 18: dynamic text, 19: textframe, 20: calendar, 21: edittext31	I
GetDialogItemLabel (integer blockNumber, string variableNameStr, integer n)	Returns the nth item label. For example, the nth item on a popup menu or the nth column header in a data table. Returns an empty string "" if the wrong type of item or no label is found.	S
GetDialogNames(integer block, string nameArray[], integer typeArray[])	Returns a list of the dialog variables in the specified <i>block</i> . Both <i>nameArray</i> and <i>typeArray</i> are dynamic arrays. This function returns the number of items in the target block's dialog. For each item in the block's dialog, the string array will contain the name of the dialog item and <i>typeArray</i> will contain the type of dialog item. Values for <i>typeArray</i> are: 1:Button, 2:Check Box, 3:Radio Button, 4:Meter, 5:Parameter, 6:Slider, 7:Data Table, 8:Editable Text, 9:Static Text, 12:Switch, 13:Text Table, 16:Popup Menu, 17:Embedded Object, 18:Dynamic Text, 19:Text Frame, 20:Calendar, and 21:Editable Text31 (31 characters)	
GetDialogVariable (integer blockNumber, string variableNameStr, integer row, integer col)	Returns the string value of the named variable (<i>variableNameStr</i> in quotes or string). If the variable is a numeric parameter or a control (such as a checkbox, slider, and so on), you would call StringToReal to convert the returned string to a numeric value. The variable can be any dialog item, static variable, global variable, or dynamic array. <i>Row</i> and <i>col</i> apply to the cells of a data table or text table. For Sliders or Meters, <i>row</i> must be zero and <i>col</i> is 0 for the minimum, 1 for the maximum, and 2 for the value. Row and col are ignored for other types of items.	S

Block dialog items	Description	Return
GetDragged-CloneList (integer blockNums[], string variableNames[])	If you put this function call in a DRAGCLONETOBLOCK message handler, it returns the number of clones dragged onto a block. It also fills the dynamic array parameters with the block numbers and variable names of the clones so you can get and set their values with GetDialogVariable() and SetDialogVariable() functions. This function is used in the Optimizer block (Value library).	I
GetSystemColor (integer whichPart, integer whichColor)	Gets the system colors so that they can be matched in a dialog. Returns whichPart: 1:R, 2:G, 3:B, 4:H, 5:S, 6:V, 7:xxBBGGRR whichColor: 0:Dialog BackGround, 1:ToolTipColor	I
HideDialogItem (string variableNameStr, integer hideShow)	Hides the named dialog item if <i>hideShow</i> is TRUE. <i>DialogNameStr</i> is the dialog item name as a string or in quotes. Returns 0 if it succeeds.	I
HideDialogItem2(integer blockNumber, string name, integer hideShow)	Set the hidden/shown status of the item. See the HideDialogItem() function, with the addition of the block number argument, which allows you to call the function from outside a block. Returns 0 if it succeeds.	I
LastSetDialogVariableString()	Returns the last string value that was set by SetDialogVariable. This is useful if one is setting the value of a dialog item like a popup menu, where the stored value is not the value that the popup menu contains, but rather an additional string variable. In this case, using the WhoInvoked() function, below, and this function you can have the code of the block change the correct string for the dialog item, so the SetDialogVariable function will react as the user expects, even though the data is not directly contained in the dialog item.	S
OpenAndSelectDialogItem (integer blockNumber, string variableNameStr)	Obsolete. Please see OpenAndSelectDialogItem2(), just below this function. This version of the function did not have the row and column indexes to select a cell in a data table.	V
OpenAndSelectDialogItem2 (integer blockNumber, string variableNameStr, integer row, integer col)	Opens the block's dialog, highlighting (selecting) the dialog item corresponding to varName. If the item is a data table, row and col are the indexes. Row and col are ignored if the item is not a data table. If row and col are -1, selects the entire data table.	V
PopupCanceled()	Returns whether or not the last popup menu was canceled. This can be called immediately after a 'flying' popup menu has been created, to determine if the user canceled the popup menu action or not. Canceling would be clicking off the popup menu, without making a selection.	I

Block dialog items	Description	Return
PopupItem- Parse(itemString)	This function parses the string that is passed in, and removes the special characters that are used in certain dialog item contexts. Specifically, it removes the formatting notation (e.g. <RB> for right adjusted, bold), characters that are user for formatting, and returns the stripped string.	S
PopupMenu- AppendAr- ray(string dialogItemName, stringArray array)	Appends the contents of the string array array to the popup menu specified by dialogItemName.	I
PopupMenuArray (string theArray[], integer initialValue)	This function creates a flying popup menu based on the strings in theArray, maximum 20 strings (5100 characters total). Other than the fact that an array of strings is passed in instead of separate strings, it's exactly the same as the CreatePopupMenu function.	I
SetDialogColors (integer blockNum- ber,, integer HSV- ColorArray[][3])	Sets all the colors of the dialog items at once. Usually used with the array from GetDialogColors(), above.	I
SetDialogItem- Color(integer blockNumber, string variable- NameStr, longArray HSVValues)	Sets a color value associated with the dialog item. Some items will not redraw with this color, but instead have their color defined by the operating system settings.	V
SetDialogVariable (integer blockNum- ber, string variable- NameStr, string value, integer row, integer col)	Sets the value of the named variable to the given numeric or string value. The variable can be any dialog item, static variable, global variable, or dynamic array. <i>Row</i> and <i>col</i> apply to the cells of a data table or text table. For Sliders or Meters, <i>row</i> must be zero and <i>col</i> is 0 for the minimum, 1 for the maximum, and 2 for the value. <i>Row</i> and <i>col</i> are ignored for other types of items.	V
SetDialogVariable- NoMsg(integer blockNumber, string variable- NameStr, string value, integer row, integer col)	Same as SetDialogVariable function, but doesn't send a dialog item message to the block.	V
SetPopupLa- bels(string variable- NameStr, string theLabels)	Sets the named popup menu items to <i>theLabels</i> string. The menu items should be separated by semicolons (;). This function is similar to SetDataTableLabels in that the changes made by this call are not permanent within a block's dialog. Changes made by this call are permanent, however, for cloned copies of popup labels. <i>VariableNameStr</i> is the dialog item name as a string or in quotes. See AppendPopupMenuLabels() above to add more labels.	V

Block dialog items	Description	Return
SetVisibilityMonitoring (integer blockNumber, string variable-NameStr, integer monitor)	Turns on monitoring of the visibility of a dialog item. This sets a flag on a dialog item that makes it send a message to the block code (DIALOGITEMREFRESH) when the dialog item (Or one of its clones) becomes visible. This is useful if you have a dialog item whose redraw includes some time-consuming calculation and you want to be able to turn off the calculation unless the dialog item is visible.	I
WhichDialogItem()	Returns the name of the current dialog item in certain contexts. This function can be used in the CELLACCEPT, DATATABLERE-SIZE, DATATABLESCROLLED, DIALOGITEMTOOLTIP, DIALOGCLICK, or dialog item name message handlers.	I
WhoInvoked()	Determines if a dialog item message handler was invoked by a call to SetDialogVariable or by user interaction. Returns one if the invoke was through SetDialogVariable, or zero if through user interaction. Called in the message handler for the dialog item, this is useful if you need the code to react differently when the dialog item value has been set by the SetDialogVariable function.	I

Block data tables

These functions allow ModL code to control the display of data in block dialog data tables. In the following functions, DataTableName is the name of the dialog data table from which you want to retrieve the selection; this name needs to be either a string variable or a string in quotes.

- Starting in ExtendSim V7, you must access all data tables using their dialog item variable name. Even dynamic data tables need to be accessed using the dialog variable name. This was not the case in Extend V6 or earlier, but is a ModL compiler change that is necessary to allow data tables to transparently access linked database or global array data, and to be variable column. Using the dynamic array name will not allow linked or variable column data tables to work.
- If you need large data tables, see the Dynamic data table functions below.

Variable column data tables

The DynamicDatatableVariableColumns function, and the supporting functions that have been built for it, allow the creation and use of a data table with both variable rows and variable columns. The starting point is a call to the DynamicDatatableVariableColumns function which will link a dynamic array to a data table, in much the same way as the DynamicDatatable function, with the addition of allowing the user to define the number of columns that the table supports.

Each time you want to change the number of columns in the data table, you call DynamicDataT-ableVariableColumns again with the new number of columns. The internal implementation of this construct is basically that the link between the data table and the dynamic array contains the infor-mation about the number of columns. This means that you cannot use the dynamic array variable name to refer to the data, as the dynamic array still has the original definition of how many col-umns it has. To reference your data with the variable number of columns, you need to refer to it using the data table variable name, as the data table will use the information in the link to deter-mine how many columns it has. See “OLE/COM (Windows only)” on page 241 for OLE func-tions that fill variable column data tables.

Dynamic data table resizing

The following functions are designed to be support functions for dynamic data table resize functionality. This functionality is implemented partially through block code, and partially through the ExtendSim Application. The basic functionality, as seen on dialog boxes in both the Item and Value libraries, is a button on the lower right hand corner of the data table that can be clicked popping up a dialog that allows the user to specify the resizing options for the data table. The application level implementation is the drawing and behavior of the button on the data table. The presence or absence of the button can be controlled from the block code via the DTGrowButton function described below. When the button is clicked, the block will receive a DATATABLERESIZE message. The code of this message handler is where the majority of the block code controlling the resizing will be executed. Normally the code in this message handler will first be a call to NumericParameter, or NumericParameter2 to determine the desired resizing of the table, followed by the necessary code to implement the resizing. See the Variable Columns data table section above. See the blocks in the Value and Item libraries for examples of this code.

Data table linking

Data tables can be linked to global arrays or an ExtendSim database by the user clicking the Link button at the lower left corner, or using the linking functions below. When the data or structure that they are linked to changes, LINKCONTENT or LINKSTRUCTURE messages are sent to individual blocks that are subscribed to an ExtendSim database or global array. See “Dynamic linking” on page 302.

 Starting in ExtendSim V7, you must access all data tables using their dialog item variable name. Even dynamic data tables need to be accessed using the dialog variable name. This was not the case in Extend V6 or earlier, but is a ModL compiler change that is necessary to allow data tables to transparently access linked database or global array data, and to be variable column. Using the dynamic array name will not allow linked or variable column data tables to work.

Formatting individual columns

For additional formatting options, see “Formatting/interactivity using column and parameter tags” on page 305. These facilitate both specialized formatting and mouse-interactivity with data tables.

Functions

Block data tables	Description	Return
AppendDataTable-Labels (string DTname, string theLabels)	Appends another string of labels to the data table labels list of the data table DTName.	I
DisableDTTabbing (integer blockNumber, string DTname, integer disabledDT)	Disables or enables the tab key functionality for the specified data table.	I
DTGrowButton(integer blockNumber, string dtName, integer showButton)	Shows/hides the resize/grow button on the specified data table. This button is hidden by default, as block code is necessary for the resizing functionality.	I

Block data tables	Description	Return
DTHasDDELink (integer blockNumber, string DTname)	Returns true if the specified data table has an IPC advise link, or a Paste link, associated with it.	I
DTPaneFixed(integer blockNumber, string name, integer fixed)	Changes the behavior of the specified data table, setting or unsetting its internal 'fixed' flag. By default this flag is off, but if it is turned on, the data table will retain its width when the number of columns in the data is reduced, rather than resizing smaller, as it can do in the default case.	I
DTToolTipSet(string caption-String)	In conjunction with the DataTableHover message handler discussed on page 209, this function allows you to show custom tool tips when the cursor hovers over a data table.	V
DynamicDataTable(integer blockNumber, string dataTableName, array dynamicArrayName)	This function attaches a dynamic array to a dialog data table. This allows dynamically resizable data tables, and the ability to change what data is displayed in a data table without recopying the data. You can attach the array to the data table at any time, but it will need to be reattached (dynamicDataTable will need to be called) each time that the dynamic array is resized.	I
DynamicDataTable2(integer blockNumber, string dataTableName, string arrayName)	Has the same behavior as the dynamicDataTable function, with the exception that it can be called from an outside block, and doesn't have to be called from within the block that contains the array. Note that the arrayName argument is the name of the array as a string, not the array name itself.	I
DynamicDataTableVariableColumns(integer blockNumber, string dataTableName, array y, integer rows, integer columns)	Similar to DynamicDatatable, with the addition of adding the specification of how many rows and columns the resulting data table will have. It will correct the data to work with a new number of columns. You must use the variable name of the data table, not the dynamic array, to access the data.	I
DynamicDataTableVariableColumns2(integer blockNumber, string dataTableName, string arrayName, integer rows, integer columns)	Has the same behavior as the dynamicDataTableVariableColumns function, with the exception that it can be called from an outside block, and doesn't have to be called from within the block that contains the array. Note that the arrayName argument is the name of the array as a string, not the array name itself.	I

Block data tables	Description	Return
GetDataTableSelection(string DataTableName, integer integerArray)	<i>IntegerArray</i> is a four element array declared as integer integerArray[4] . On return from the function, <i>integerArray</i> will contain the selection information in the following format: integerArray[0] -- top row integerArray[1] -- bottom row integerArray[2] -- left column integerArray[3] -- right column The integer value returned from the function will be the same as integerArray[0] if there is a valid selection. The value will be a -2 if there is no correct selection, and a -1 if an error of some other type occurred (usually an invalid <i>DataTableName</i> .)	I
GetDTOffset (integer blockNumber, string DTName, integer clone, integer row)	Returns the offset (How many rows or columns the table has been scrolled) for a data table. If row is true, it returns the row offset, otherwise the column offset. If clone is true, it returns the value for the first clone of the item found.	I
RefreshDatatableCells(integer blockNumber, string dataTablename, integer startRow, integer startCol, integer endRow, integer endCol)	This function will redraw the specific cells of the named data table. This needs to be used in conjunction with the dynamic data tables defined by the function above, as they will not automatically update the data displayed during a simulation run when the dynamic array is modified	I
ResizeDTDuringRead(string name, integer oldRows, integer oldCols)	This function will allow you to inform ExtendSim that a data table is now a different size than it was in an earlier version. This should only be called in the BlockRead message handler, and is most useful when used with the GetFileReadVersion function to inform ExtendSim when a Data Table has been redefined. (See GetFileReadVersion for more information.) <i>OldRows</i> and <i>OldCols</i> will contain the original size of the data table.	I
ScrollDTTo(integer blockNumber, string name, integer row, integer col)	Scrolls the named data table to the indicated <i>row</i> and <i>column</i> . (0 successful, 1 failed)	I
SetDataTableCornerLabel (string datatablename, string15 label)	Sets a label to be displayed in the upper left-hand corner of the data table. This area of the table is blank by default. The function will return a zero if the call succeeded, and a value of one if there was an error (most likely an incorrect <i>DataTableName</i>).	I

Block data tables	Description	Return
SetDataTableLabels (string DataTable- Name, string Label- String)	The <i>LabelString</i> variable will be used as a replacement string for the column header string of the specified data table. The format for this string is the same as that used in the dialog editor when creating or modifying a data table dialog item. The function will return a zero if the call succeeded, and a value of one if there was an error (most likely an incorrect <i>DataTableName</i>). Note: The change made by this call is not a permanent change within the block's dialog, you will need to retain the new string in a variable in the block, and call this function multiple times. See the code of the Information block (Item library) as an example of using this function. The change made by this call <i>is</i> permanent for clones of data tables.	I
SetDataTableSelec- tion(name, start- Row, startCol, endRow, endCol, editCell)	This function selects the cells in the named data table or text table. If <i>editCell</i> is TRUE, this function sets up the first cell for entering data.	V
SetDTColumn- Width(integer blockNumber, string name, integer column, integer width, integer doClones)	Sets the specified column in the named data table to the specified width. Please note that column number for this function is zero based starting from the title column, to allow reference to the title column. This means that the first column with data in it is column one, unlike most other functions that reference data table columns.	I
SetDTRowStart (integer blockNum- ber, string name, integer rowStart)	Sets the named data table to have its row numbers start at the indicated value. (i.e the first row number, normally zero, will be <i>row-Start</i>) (0 successful, 1 failed)	I
SortArrayVariable- Columns(short hNum, array datat- ableName, integer numCols, integer numRowsToSort, integer keyColumn, integer increase, integer sortStrin- gAsNumbers)	Sorts the specified array, using the numCols argument to define how many columns the array to be sorted contains.	I
StopDataTableEdit- ing()	Immediately stops the currently selected data table from being edited. A common use would be to call this function in response to a click on the data table, preventing the user from editing the cell but allowing selection to occur.	V
WhichDTCell(inte- ger rowCol)	Returns the Row or Column that is currently active. If rowCol is TRUE this function returns the Column, otherwise it returns the row. This function can be called in the CELLACCEPT, DIALOG-ITEMTOOLTIP, DIALOGCLICK, or dialog item name message handlers.	I

Dynamic linking

Dynamic linking creates a link between a dialog item (data table or parameter) and a data source (global array or ExtendSim database). These links have been substantially enhanced in v7 over how they worked in earlier versions. In v7, the links are live dynamic links that update instantly when there is a change in the data source. These are the functions that allow control over how the links are to be created and controlled, in addition to the ExtendSim user interface that allows the user to create links by just right-clicking on a parameter or clicking the Link button at the lower left of a data table. Also see “Database functions” on page 333 and “Global arrays” on page 355.

To simply register a block so that it will be notified if there was a database change, see “Registered blocks” on page 106 and “Linking and notification” on page 351.

-  Starting in ExtendSim 7, you must access all data tables using their dialog item variable name. Even dynamic data tables need to be accessed using the dialog variable name. This was not the case in Extend V6 or earlier, but is a ModL compiler change that is necessary to allow data tables to transparently access linked database or global array data, and to be variable column. Using the dynamic array name will not allow linked or variable column data tables to work.

Dynamic linking	Description	Return
DILinkClear(integer blockNumber, string dialogItemName)	Clears a link from the specified link if it has one.	I
DILinkInfo(integer blockN, string dialogItemName, integer which)	Returns linking information about the specified dialog item. Values for <i>which</i> : 0 : Link type returns 0: no link, 1: global array, 2: Database, 3: dynamic array 1: DB Index returns the index of the database for the link (if it's a DB link) 2: table/array index returns the value of the index 3: returns column index 4: returns row index 5: user link: if the link was created by a user vs. a function 7: returns dialogItemID to identify a dialog item. See DIGetID function description. 10: ReadOnly: return value is true/false	I
DILinkingDisabled(integer blockN, string dialogItemName, integer disableIfTrue)	This function controls if the specified data table or parameter will allow linking. By default each data table has a link button in its lower left-hand corner and calling this function will allow the block developer to hide this button. This should have the added effect of disabling the link menu command. Return a negative error number or zero if no error.	I

Dynamic linking	Description	Return
DILinkModify(integer blockNumber, string dialogItem, integer which, integer value)	This function is used to modify flags associated with a specific dialog item link. The 'which' argument specifies which flag will be set; value specifies what the value should be set to. <i>which:</i> 0: ReadOnly. Set <i>value</i> to TRUE to make a link 'read-only' 1: InitMsgs. Set <i>value</i> TRUE so link will get messages during INITSIM. 2: SimMsgs. Set <i>value</i> TRUE so link will get messages during SIMULATE. 3: FinalMsgs. Set <i>value</i> TRUE so link will get messages during FINALCALC. 4: AnyMsgs. Set <i>value</i> TRUE so link will get any messages at all. 5: UserLink. Set <i>value</i> TRUE so link was created by the user interface (TRUE), or a MODL function call (FALSE). 11: ShowFieldNames Set <i>value</i> TRUE so link uses the field names for the header rows in the data table.	I
DILinkMsgs(integer sendMsgs)	Turns on and off the sending of messages associated with linking. This function should be used carefully, and should always be turned back on when the code has completed. This will disable all messages sent to <i>any</i> blocks that have dialog items linked to <i>any</i> data source, and many blocks will function incorrectly if these messages are disabled. The primary reason to use this function would be if you are making many changes to a data source, and don't want things to be overwhelmed with too many messages.	V
DILinkSendMsgs(integer dataSourceType, integer DBIndex, integer tableIndex, integer fieldIndex, integer recordIndex)	Sends update messages to all the dialog items linked to this data source. In the Global Array case, the DBIndex is ignored. If you want to send messages to all dialog items linked to a whole table, pass in -1 for the fieldIndex, and -1 for the rowIndex. dataSourceType: 1: Global Array 2: Database	I

Dynamic linking	Description	Return
DILinkUpdateInfo(integer which)	Should be called in the LinkStructure, LinkContent, or dialog item name message handlers. Returns information about which link changed and what the change was. Values for <i>which</i>: 0:Link type (returns 0:no link, 1:global array, 2:database, 3:dynamic array) 1:DB Index returns the index of the database for the link (if it's a DB link) 2:table/array index returns the value of the index 3:returns column index 4:returns row index 5:what changed (see below for values) 6:number of rows or columns changed 7:dialog item ID 8: the block number of the block that changed the data 9: Returns True (1) if link messages are enabled; otherwise returns False (0). 10:returns the index of the database being imported or 0 if no database is being imported. A <i>which</i> value of 5 (what changed) returns 1:data changed, 2:field inserted, 3:field deleted, 4:field renamed, 5:record inserted, 6:record deleted, 7:table or GA deleted, 8:table or GA renamed, 9:DB deleted, 10:DB renamed, 11:link created, 12:link modified, 13:link cleared, 14:DB replaced via DBDatabaseImport(), 15:GA resized, 16:table inserted, 17:field properties modified, 18:field moved, 19:table sorted, 20:Table Properties modified, 21:record ID modified.	I
DILinkUpdateString(integer which)	This companion function to DILinkUpdateInfo returns a string containing information about a change to the link status. This can be called in the same message handlers as DILinkUpdateInfo. which values: 0 : DialogItemName The name of the dialog item associated with the link. 1 : Changed Name. If whatChanged is field or table renamed, for example, the changed name string will be the old name of the field or table.	S
DTHideLinkButton(integer blockN, string dtName, integer hideButton)	This function is replaced by DILinkingDisabled(), above, that works for both data tables and parameters. See that function for a description.	I

Dynamic text items

Dynamic Text items are a new type of dialog item that supports up to 32000 characters of text. They are useful where the 255 character limitation of editable text items is a hindrance. The text is stored in a string dynamic array, declared:

```
string dynTextArray[];
```

For an example of how to use dynamic text items, look at the Equation block (Value library).

Dynamic Text	Description	Return
DynamicTextArrayNumber (string dynamicTextArray[])	Returns the dynamic array index. Used to link the Dynamic Text Item to the correct dynamic array. For example: <pre>myDTextItem = DynamicTextArrayNumber(dynTextArray);</pre> where dynTextArray is declared as above.	I
DynamicTextIsDirty (integer blockNumber, string name)	Returns a true if the text item is dirty (the user has typed text into the dynamic text item or changed its contents in some way).	I
DynamicTextSetDirty (integer blockNumber, string variableNameStr, integer dirty)	Sets the dirty flag to 'dirty' (TRUE or FALSE) on the specified dynamic text item. Useful to set dirty if the block performs a text changing operation outside of the user editing the text.	I
StrFindDynamic (string dynamicTextArray[], string findStr, integer caseSens, integer diacSens, integer wholeWords)	Similar to the StrFind() function, except it searches a dynamic text array.	I
StrFindDynamicStartPoint (string dynamicTextArray[], string findStr, integer caseSens, integer diacSens, integer wholeWords, integer startPoint)	Similar to the StrFind() function, except it searches a dynamic text array. startPoint is the starting character index to search from, starting at zero.	I
StrReplaceDynamic (string dynamicTextArray[], integer start, integer numChars, string replaceStr)	Similar to the StrReplace function, except it replaces text in a dynamic text array.	I

Formatting/interactivity using column and parameter tags

Column tag and Parameter tag values allow block developers to have a great amount of additional control over the display and formatting of columns of data in data tables and of parameters in dialog boxes. The data table and parameter dialog items check which column tags have been set, and draw their data, or respond to clicks appropriately.

The basic mechanism for setting a param tag, or column tag, is to call either the DIParamTagSet or the DTColumnTagSet function. Column and param tags are not saved when the model is closed, so the tags are commonly reset in the OpenFileDialog and CloneInit message handlers.

If you set a column tag that has a value higher than 99, it will be stored as a HeaderTag. HeaderTags are stored independently from column tags, and will store a behavior for the header for that column, not for the individual cells in the table. (See Header tags below for more information.)

In the data table case, column values start at zero, with zero referring to the first column after the row column. Each column has four pieces of information that can be associated with it: Two integer values, the tag itself, a tagOption value, and two string values, the TagString1 and TagString2. ParamTags for parameters have just one number, the paramTag, and the same two strings.

The primary tag sets the default behavior for the column/parameter. The TagOption value is used for specific purposes by some of the tags below, and can also be used to disable the column using the TAGDISABLE value.

The TagString1 value is the lookup name in the case of string Lookup table columns, for other tags it's used as needed. TagString2 is used by some tags. For any tag that displays a string, except the SL tags and the DBFIELD tag, putting a string into the string1, or string2 field will prepend the string from string1, and/or append the string from string2.

The column/parameter tags are listed below. The symbol name definitions can be found in the include file “\Extensions\Includes\ColumnTags v7.h”.

Tag name	Value	Description
Tag_Block_Label	60	Tags the column/parameter as containing block labels.
Tag_Block_Label_Name	62	If the block has a block label it will show that, but if there is no block label, shows the block name (e.g. Activity)
Tag_Block_Name	61	Tags the column/parameter as containing block names.
Tag_Button	21	Tags the column/parameter as containing push buttons. To hide the button in a specific cell, set that data table cell to BLANK.
Tag_CheckBox	20	Tags the column/parameter as containing checkboxes. BLANK is equivalent to FALSE (unchecked).
Tag_Conditional_Popup	42	Tags the column as containing popups conditionally. This functionality is very specific. Each cell/row in the column will show a popup triangle under the following condition: The cell in the column to the left of the column specified must contain a string value that matches the stringTag that was passed in when the coltag was set. If the string value in the cell to the left of a given cell does not contain the string value, that cell will default to a normal editable cell. This functionality is used in the Item Equation block.

Tag name	Value	Description
Tag_Conditional_Popup_Array	43	Tags the column as containing popups conditionally. This functionality is very specific. Each cell/row in the column will show a popup triangle under the following condition: The cell in the dynamic array specified by the tagOption parameter must contain a string value that matches the stringTag that was passed in when the coltag was set. If the string value in the dynamic array cell does not match the string value, that cell will default to a normal editable cell. This functionality is used in the Item Equation block. This functionality is the same as the TAG_Conditional_Popup function except that the comparison strings are kept in a dynamic Array instead of in the adjacent column.
Tag_Date	50	Tags the column/parameter as containing date values.
Tag_Date_Convert	51	Tags the column as containing simulation time values that should be represented as date values. The conversion is done in the application.
Tag_DB_Address	32	Tags the column as containing DB Addresses, but with no popup menu so the block designer can customize the DB part selection process.
Tag_DB_Address_Pop	31	Tags the column as containing DB Addresses. A popup menu appears when the user clicks the data table cell, enabling selection of the DB parts.
Tag_DB_Field	30	Tags the column as containing data from a specified database table field. TagOption should contain the DBIndex, String1, the tablename, and string2 the fieldname.
Tag_Disable	99	Tags the column/parameter as being disabled. Note that this tag can be used in Tag Option to mark a tagged column as disabled as well as the other meaning of the tag in the column tag.
Tag_Head_Tag_Popup	100	Tags the column/parameter header as being a popup menu. Note: this does not change the behavior of clicking on the cell, just the appearance of the cell. The popup behavior needs to be implemented in block code.
Tag_Hide	98	Tags the column/parameter as being hidden.
Tag_Infinity	26	Tags the parameter as containing a checkbox that changes the value to a novalue to represent infinity. If ParamStr and paramStr2 are empty, the behavior is that the dialog item will display the word infinite, and an infinity symbol will be drawn near the checkbox. If there is a string in paramStr, it will be displayed instead of the word infinity, if there is a string in paramStr2, it toggles whether or not the infinity symbol is drawn, so if you want to change the meaning of novalue to something else, just put the meaning in paramStr, and an 'x'. or some thing like that in paramStr2.
Tag_No_Head_Tag	101	Reverts the header tag back to zero, meaning no defined behavior.
Tag_Nothing	0	Tags the column/Parameter as not having a tag. (Note that you can use this with the TagOption or the TagStrings to disable the column, or append/prepend text to the column.)
Tag_Number	71	Used in a string table to format numbers correctly to fit the cell.

Tag name	Value	Description
Tag_Percent	70	Tags the column/parameter as containing percents. Only numerical parameters support this tag.
Tag_Popup	40	Tags the column/parameter as containing popupMenus. Note: this does not change the behavior of clicking on the cells, just the appearance of the cells. The popup behavior needs to be implemented in block code.
Tag_Radio	22	Tags the column/parameter as containing radio buttons.
Tag_SL	10	Tags the column/parameter as containing string lookup values.
Tag_SL_Append	17	Tags the column as containing string lookup values, appended with the string in tagSstring2.
Tag_SL_Pop	11	Tags the column/parameter as containing string lookup values that function as popup menus when clicked.
Tag_SL_Prepend	16	Tags the column as containing string lookup values, prepended with the string in tagString2.
Tag_Str_Flag	80	Tags the column as a numeric column where the colstring will be displayed when the value in a cell is a negative one. Otherwise, the column will be treated as a standard numeric data column.
Tag_Str_Flag_Noedit	81	The same as Tag_Str_Flag, except it doesn't allow editing.

Column/parameter tag functions

Here is a list of the functions used to implement column/parameter tags. The tag values are listed in the table above:

Column/parameter tags	Description	Return
DIParamTagGet(integer blockNumber, string dName)	Returns the ParamTag for the specified parameter.	I
DIParamTagSet(integer blockNumber, string dName, integer tag, string string, string string2)	Sets the specified Parameter tag to the specified value. Values for the tag value are from the ColTag/Param Tag Values table above. The tagString value is currently only used for the string lookup tags, in which case it is the lookup name.	I
DIParamTagStringGet(integer blockNumber, string dName, integer which)	Returns a TagString for the specified parameter. <i>Which</i> should be 0 for the first string and 1 for the second string.	S
DTColumnTagGet(integer blockNumber, string dtName, integer col, integer which)	Returns the column tag for the specified column. which: 0 is columnTag 1 which: 1 is columnTag 2 which: 2 is the header tag	I

Column/parameter tags	Description	Return
DTColumnTagSet(integer blockNumber, string dtName, integer col, integer tag, integer tagOption, string tagString, string tagString2)	Sets the specified column tag to the specified value. Col values start at zero, with zero referring to the first column after the row column. Values for the tag value are from the ColTag/ParamTag Values table above. The tagString value is the lookup table name in the case of string Lookup table columns, for other tags it's used as needed. For many types of tags it's not used. TagString2 is used by some tags as well, for many it's not used, and can just be set to "" the empty string. TagOption specifies an option for the tag. Some tags use this option for specific purposes. Using the flag for TAGDISABLE in this argument will disable most column tags. For any tag that displays a string, except the SL tags and the DBFIELD tag, putting a string into the string1, or string2 field will prepend the string from string1, and/or append the string from string2.	I
DTColumnTagStringGet(integer blockNumber, string dtName, integer col, integer which)	Returns a tagString value for the specified column. Which should be 0 for the first string and 1 for the second string.	S

Dialog item tool tips

The System message DialogItemToolTip is sent to the block when the cursor hovers over a dialog item. In that message handler, the dialog item tool tip functions (below) will allow you to customize and access the tooltip that will appear.

The functions WhichDialogItem(), and whichDTCCell() can be used with this functionality to find which dialog item the cursor is currently hovering over.

Dialog item tool tips	Description	Return
DIToolTipSet(string-string, integer replace)	Allows you to set the string that will be displayed. If the replace flag is true, the default tool tip string will be replaced, if it is false, the default string will be appended to.	V

Block dialogs (opening and closing)

These functions allow the ModL code to control the opening and closing of the block's dialog.

Block dialogs	Description	Return
BlockDialogIsOpen(integer blockNumber)	Returns TRUE if the block dialog is open. Returns FALSE otherwise.	I
CloseBlockDialogBox(i)	Closes the dialog for any block, where i is the global block number. See OpenBlockDialogBox, below.	V

Block dialogs	Description	Return
CloseDialogBox()	Closes this block's dialog from within the ModL code. Put this in the EndSim message handler if you want an open dialog to close at the end of the simulation. See OpenDialogBox, below. NOTE: to prevent the dialog from closing conditionally, use an Abort statement in the DIALOGCLOSE message handler.	V
CloseEnclosing-HBlock()	Closes the hierarchical block in which this block resides, if any.	V
CloseEnclosing-HBlock2(integer blockNumber)	This is same as CloseEnclosingHBlock(), except it refers to a global block number.	V
DialogGetSize(integer theBlockNumber, integer which)	Gets the width or Height of an open dialog. (Returns the width or height of the hBlock Model window if the block number refers to an hBlock.) The 'which' argument takes the following values: 0: width 1: height.	I
Dialog-MoveTo(integer blockNumber, integer x, integer y)	Moves the dialog box to the specified x and y pixel location. If the block is an hBlock, this will move the hBlock submodel window.	I
DialogSetSize(integer blockNumber, integer w, integer h)	Resizes the dialog box of the block. Only works if the dialog is open. If the block is an hBlock, this will resize the hBlock submodel window. W and H are width and height in pixels.	I
MakeDialogModal (integer theBlockNumber, integer TrueFalse)	Makes a block's dialog modal and should be called when the dialog is open. During the dialogOpen message is OK. Calling this function with the <i>TrueFalse</i> flag set to false will turn the dialog back to non-modal behavior.	V
OpenBlockDialog-Box(i)	Opens the dialog for any block, where i is the global block number. You may want to do this if you are telling the user to change a value in a block's dialog. For instance, if you have just put up an alert saying "The value in the 'Height' field of the 'Attic' dialog is negative," you can then open the offending dialog to make the change easier.	V
OpenDialogBox()	Opens this block's dialog to show something visually important to the user. If you want a dialog to always appear in front of any open plotting windows, call this function at the beginning of the Simulate message handler.	V
OpenEnclosing-HBlock()	Opens the hierarchical block in which this block resides, if any.	V
OpenEnclosingHBlock2 (integer blockNumber)	This is same as OpenEnclosingHBlock(), except it refers to a global block number that is within the HBlock.	V

Block dialog tabs

These functions allow ModL code to manipulate block dialog tabs.

Block dialog tabs	Description	Return
DisableTabName (string tabName, integer trueOrFalse)	Disables or enables the specified tab.	V
GetCurrentTab- Name(integer blockNumber)	Returns the currently selected tab name.	S
OpenDialogBox- ToTabName(string tabName, integer blockNumber)	Opens a dialog box to a specified tab.	V
SetDefaultTab- Name (string tab- Name, integer blockNumber)	Sets the block so that the dialog will open at a specific tab name.	V
VariableNameToTa- bName(string var- Name, integer blockNumber)	Returns the name of the tab of a dialog that the indicated dialog item is on.	S

Messages to blocks (sending and receiving)

These functions make it easy to communicate information back and forth between blocks and to execute and modify processes between blocks. They can be used to set up different types of simulations that override ExtendSim's simulation engine (i.e. Item library blocks). Sending messages to blocks can be used to enable global functions. You can put many functions in a custom function block, then use the SendMsgToBlock function to send one of the user-defined messages. Use global variables to select a function and pass parameters to and from the function. Blocks can also send and receive connector messages and propagate them correctly, as discussed in "Basic item messaging" on page 167.

In the functions, *connName* is the name of the connector in the calling block and *block* is the global block number of the receiving block. (See "Block numbers, labels, names, type, position" on page 282.)

Block messaging	Description	Return
BlockSimStartPrior- ity(integer block- Number, integer priority)	Sets the <i>priority</i> value for receiving the SimStart message discussed on "Simulation messages" on page 204. Returns 0 for success or a negative value for failure. The default value is -1 and any block with a <i>priority</i> value of less than 0 will not receive the message. Blocks that have a <i>priority</i> value greater than -1 will receive the message in priority order (lowest to highest).	I

Block messaging	Description	Return
BlockSimFinishPriority(integer block-Number, integer priority)	Sets the <i>priority</i> value for receiving the SimFinish message discussed on “Simulation messages” on page 204. Returns 0 for success or a negative value for failure. The default value is -1 and any block with a priority value of less than 0 will not receive the message. Blocks that have a <i>priority</i> value greater than -1 will receive the message in priority order (lowest to highest).	I
ConnectorMsg-Break()	Called from a block currently receiving a connector message. It prevents any additional connected blocks from receiving that message. This function does not affect the current message handler and it should be called right before returning from the current message handler.	V
DuringHBlockUpdate()	During an HBlock update, some connections are temporarily broken and then reconnected. Returns True if called during an HBlock update so you can determine if ConnectionMake or Connection-Break messages can be safely ignored.	I
GetConnectorMsgs-First (connName)	Makes the specified connector the first in netlist messages. This is used in blocks where it is critical that the messages be received by a specific block first, no matter what the order of connections. Use this function in INITSIM or CHECKDATA. For example, the Status block uses this function.	V
GetMsgSending-Block ()	Returns the blocknumber of the block that sends the currently executing message to the current block.	I
GetSimulateMsgs(integer ifTrue)	In some cases, you may not want a block to get Simulate messages. (See “Residence blocks that do not post future events:” on page 163.) This may be true in some discrete event blocks and in continuous blocks that are used in discrete event simulations. This function prevents blocks from getting Simulate messages and thus speeds up the simulation. The function would usually be called in the InitSim or checkData message handlers. Call this function with FALSE if you do not want the block to get Simulate messages. This is initially set to TRUE for new and existing models, and is always reset to TRUE before a simulation starts.	V
MsgEmulationOptimize(integer ifTrue)	See “Value connector messages” on page 172. This specialized function is used in the Executive block (Item library) to optimize the operation of Value blocks in discrete event models. It prevents the propagation of redundant messages that are produced by Value blocks emulating and propagating connector messages from Item blocks. It does this by not back-propagating (step 1) connector messages to blocks that have ever sent a connector message to that input of this block. This can speed the simulation by reducing the number of messages sent between Item and Value blocks. This behavior is not a default because other types of modeling will fail if not all messages are propagated. This is initially set to FALSE for new and existing models, and is always reset to FALSE before a simulation starts.	V

Block messaging	Description	Return
RestrictConnectorMsgs (integer restrictMessages)	This function enables/disables a flag that restricts the use of connector messages in InitSim, checkdata, and endsim. This is used in the Executive block (Item library), as sending connector messages at the wrong times can cause problems in these libraries.	V
SendConnectorMsgToBlock (integer blockNumber, integer conn)	Sends a connector message to a block number that is not necessarily connected to the block sending the message. <i>Conn</i> is the index for the receiving connector in the connector pane (the index starts at 0).	V
SendMsgToAllCons(connName)	Sends a message to all connectors on other blocks that are connected to <i>connName</i> on the sending block. The connected blocks will get an “on xxx” message where xxx is the connected receiving block’s connector name.	V
SendMsgToBlock (integer block, integer messageConstant)	Sends a message to the specified global block. The <i>message</i> is an ExtendSim constant that corresponds to the message to be sent. The messageConstant is not a string. It is derived by taking the message name and adding MSG after the message name. For example, to send a USERMSG0 to a block, messageConstant would be USERMSG0MSG (not a string). See the chapter “Messages and Message Handlers” on page 203 for all the messages that can be sent. Note: If the block is a hierarchical block, the submodel blocks will not receive the message. Instead, use the <i>SendMsgToHBlock</i> function discussed in this section.	V
SendMsgToHBlock(integer globalBlockNum, integer message)	Sends the message to all internal blocks within the hierarchical block. This function will do nothing if <i>globalBlockNum</i> is not a hierarchical block.	V
SendMsgToInputs(connName)	Sends a message to all input connectors on other blocks that are connected to <i>connName</i> on the sending block. The connected blocks will get an “on xxxIn” message where xxxIn is the connected receiving block’s input connector name.	V
SendMsgToOutputs(connName)	Sends a message to the output connector on the block that is connected to <i>connName</i> on the sending block. The connected block will get an “on xxxOut” message where xxxOut is the connected receiving block’s output connector name.	V
SimulateConnectorMsgs(integer true-False)	This is used to disable the “simulation” of connector messages by Value blocks that have no connector message handlers, as described in “Value connector messages” on page 172. “Simulation” of connector messages is enabled by default for new and existing models, and is always reenabled before a simulation starts. This is a special-purpose function that will probably only be of interest to users who are creating their own libraries that are not being used with the Item library. Use this function in INITSIM or CHECKDATA.	V

Icon classes and views

These functions are used to interrogate and control block icon classes and views. Classes are global (e.g. Flowchart class) and always set by the modeler, but views (e.g. Flow down) can be set by the modeler or the block.

The default class (ExtendSim classic) and view are both zero.

Classes and views	Description	Return
IconGetClass (integer blockNumber)	Gets the current class (0 to 7) of the block, but classes are currently global, so the entire model will have this class.	I
IconGetView (integer blockNumber)	Gets the current view (0 to n) of the block. See IconGetViewName(), below.	I
IconGetViewName (integer blockNumber, integer view)	Returns the name of the view specified.	S
IconSetViewByIndex (integer blockNumber, integer index)	Sets the block's view by index (0 to n).	V
IconSetViewByPartialName (integer blockNumber, string partialName)	Sets the block's view by the partial name entered. For example, "right" will set the icon view to "Right Angle." This is useful when the view names are slightly different for different classes and you want to set the view to a type of view rather than a specific one.	I

Models and notebook

Models, notebook	Description	Return
DuringContinue()	Returns true if the model is being continued from a previous saved run.	I
GetBlockInfo(integer theBlockNumber, integer which)	Returns misc information about a block which can take the following values: 1: invisible --- returns if the block is invisible. 2: scriptedBlock – returns if the block was created via scripting. 3: dialog box is open	R
GetGlobalSimulationOrder(integer blockNumber)	Returns the actual simulation order index of a block.	I
GetLibraryContents (string libName, Str31 blockNames[], Str31 blockTypes[])	Fills the passed-in Dynamic Arrays with the names and types of the blocks in the named library. Returns the number of blocks in the library.	I

Models, notebook	Description	Return
GetLibraryInfo (integer blockNumber, integer specify)	Returns whether a library is of a certain type. <i>specify</i> takes the following values: 1: RunTime (others will be defined in the future)	I
GetLibraryPath- Name (integer blockNumber, integer specify)	Function to return the pathname of a library file, given a block in that library. <i>specify</i> takes the following values: 1: pathname without library name 2: just library name	S
GetLibraryString- Info (integer block- Number, integer specify)	Returns string information about the library that the block comes from. Specify takes the following values: 1: Name (others will be defined in the future)	S
GetLibraryVer- sion(integer block- Number)	This function returns the short version string for the library that includes the block specified by blockNumber.	S
GetLibraryVersion- ByName(string lib- Name)	Returns the library version string.	S
GetModelName()	Returns the string that is the model's file name. This is useful when writing out debugging information or in certain user alerts.	S
GetModelPath (string modelName)	Returns the pathname to the specified model, but does not include the model name. The model needs to be open.	S
GetRunParamete- r(integer which)	Similar to GetRunParameterLong except it returns a real. <i>Which</i> specifies a parameter in the Simulation Setup dialog. In addition to the parameters for GetRunParameterLong , this function returns values for: 1 - endTime 2 - startTime 10 - startDate 12 - endDate	R

Models, notebook	Description	Return
GetRunParameter-Long(integer which)	Gets the specified parameter from the Simulation Setup dialog using the value of <i>which</i> : 3 - numSims 4 - numSteps 5 - random seed 6 - seedControl (value, from 1-3, of the seed popup from the Random Numbers tab) 7 - checkRandomSeeds (value of the duplicate seeds check box from the Random Numbers tab) 8 - timeUnits 9 - calendarDates 11 - “__seed” table database index	I
GetSerialNumber()	Returns the serial number of the unit as a string.	S
GetSimulation-Phase()	Returns the phase of the simulation. 0 - Not currently running a simulation 1 - CheckData 2 - StepSize 3 - InitSim 4 - Simulate (main simulation loop) 5 - FinalCalc 6 - BlockReport 7 - EndSim 8 - AbortSim 9 - PreCheckData 10 - PostInitSim 11 - SimStart 12 - SimFinish 13 - ModifyRunParameter 14 - OpenModelPhase	I
GetWindowsHndl(integer which)	(Windows Only) Returns the Windows API handle of the main window in the ExtendSim application. Currently the <i>which</i> parameter is unused. It should be set to 0 for the Main window handle. This handle is used to pass to a Windows DLL. It is not used in ModL functions.	I
IsSimulation-Paused()	Returns TRUE if the simulation run is paused or is about to pause.	I

Models, notebook	Description	Return
ModelLock(integer lock, integer hBlocksLocked, string password)	The function allows you to lock and unlock the model from MODL code. The Lock argument takes a value of one if the model is to be locked, a value of zero for unlocking. HblocksLocked is a flag that sets whether you want the hBlocks to be locked when locking a model. The password is the password to be set in the locking case, and the password to be compared with in the unlocking case. This function returns a zero if it succeeds, and the following error codes for specific error conditions: -1: Unlock password didn't match. -2: Model is already locked, it cannot be locked again without being unlocked first. -3: lock value out of range.	I
NotebookClose()	Closes the notebook if it is open.	I
NotebookIsOpen()	Returns a true value if the notebook is currently open.	I
OpenNotebook()	Opens the model notebook.	V
PauseSimForSave()	Pauses the currently running active model for saving. This allows the user to save the model while paused and enables the <i>Continue Simulation</i> command discussed in the ExtendSim User Guide in the section <i>Saving Intermediate Results</i> . The model can be saved by the <i>Save</i> menu command or the SaveModel() function below. NOTE: The PauseSim() function does a pause immediate which is useful for debugging, but will not allow saving the model.	V
ResumeSimulation()	Resumes the simulation and returns immediately.	V
RunSetup(trueFalse)	Opens Setup Simulation dialog and sets up a default OK button instead of Run Now button. Returns TRUE if OK is clicked. If <i>trueFalse</i> is TRUE, show RunNow button. If <i>trueFalse</i> is FALSE, hide RunNow button.	I
RunSimulation(trueFalse)	Runs the model. If the argument is TRUE, the function puts up the Simulation Setup dialog, then runs the simulation if the user chooses OK; if the argument is FALSE, the model is run directly. This function returns TRUE if the simulation ran to completion with no errors and was not stopped, otherwise returns FALSE. See the SetRunParameters() function below. NOTE: If called from a block, runs the model until it completes or is aborted, and then returns. If you use the ExecuteMenuCommand() function to call RunSimulation() (e.g. from scripting or OLE automation), it returns immediately after starting the model run.	I
SaveModel()	Saves the model and updates any publishers.	V
SaveModelAs(string fileName)	Saves the active model under the name and path defined in fileName. Returns a zero if successful, a negative number otherwise. Saves after the current block is executed.	I

Models, notebook	Description	Return
SaveTopDocAs(string filePath, integer aSync)	Saves the top document under the file name and path name defined by <i>filePath</i> . If <i>aSync</i> is TRUE, saves after the current block is executed. Similar to SaveModelAs except it will work on whichever the top ExtendSim document is. (Specifically used by the Equation block to save and close Include files.) Returns 0 for success or a negative value to indicate failure.	I
SetBlockSimulationOrder (integer blockNumber, integer newOrder)	Sets the simulation order value of the specified block. This is only useful, or allowed if the model simulation order has been set to custom simulation order. This function returns a zero if successful, and the following error codes if it fails: -1: can't be set during a simulation. -2: newOrder must be greater than zero. -3: no active model document. -4: sim order is not set to custom. -5: no such block.	I
SetModelSimulationOrder (integer neworder)	Sets the type of simulation order to be used in the model. Types are: 0: left to right 1: not used 2: flow 3: custom This function returns a zero if successful, and -1, -2, and -3, as described under SetBlockSimulationOrder.	I
SetRunParameter(real paramValue, integer which)	Sets a single parameter in the Simulation Setup dialog. As an enumerated list, this function is more expandable than SetRunParameters(). The values for <i>which</i> are: 1:endTime, 2:startTime, 3:numSims, 4:numSteps, 5:random seed), 6:seedControl (value, from 1-3, for the seed popup from the Random Numbers tab), 7:checkRandomSeeds (value for the duplicate seeds check box for the Random Numbers tab), 8:timeUnits, 9:calendarDates, 10:startDate, 11: “__seed” table database index.	I
SetRunParameters(real SetEndTime, real SetStartTime, real SetNumSims, real SetNumStep)	Sets the parameters in the Simulation Setup dialog. This function returns the following: 0 Successful -1 End time must be greater than start time -2 Number of steps must be at least one -3 Number of steps must be less than 2000000000 -4 Number of simulations must be at least 1 and less than 32768	I
SpinCursor()	Spins the beachball cursor. Used to alert the user that a time consuming calculation is taking place.	V

DE Modeling Using Equation Blocks

The following function are convenience functions intended to be called from Equation blocks. They have the effect of querying a resource pool block for the number of available resource, or requesting that the Resource Pool block allocate a resource. They are implemented through the sending of messages, and the use of globals. See the code of the resource pool block for more information if desired.

DE Modeling	Description	Return
ResourcePoolAllocate(integer ResourcePoolBlockNum, real NumToAllocate)	Requests the specified Resource Pool block to allocate the specified number of resources.	
ResourcePoolAvailable(integer ResourcePoolBlockNum)	Queries the specified resource pool block for the number of resources that are available.	

Scripting

These functions are useful in building or changing models when called from blocks or from other applications. See also the GetDialogVariable() and SetDialogVariable functions under “Dialog items” on page 291.

See the Scripting blocks in the ModL Tips library for examples of scripting.

Scripting	Description	Return
ActivateApplication()	Brings the ExtendSim application to the foreground. This is primarily used for interapplication/scripting	V
ActivateWorksheet(string sheetName)	Activates, (selects and brings to the front), the named worksheet.	I
AddBlockToClipboard(integer blockNumber)	Adds the indicated block to the clipboard contents without otherwise changing the contents. Returns FALSE if successful.	I
AddBlockToSelection(integer blockNumber)	Adds the indicated block/worksheet object to the selection. This is in contrast to SelectBlock(), which makes the indicated block the entire selection. Returns FALSE if successful.	I
ApplicationFrame(integer frame[], integer inside)	Returns the application frame in global coordinates. The <i>Inside</i> argument specifies if the returned frame should be inside (includes the menubar and window frame) of the application frame window, or the outside. Declare frame as: Integer frame[4]; When the function returns, frame will contain: frame [0] - Top, frame[1] - Left, frame[2] - Bottom, frame[3] - Right	I

Scripting	Description	Return
ChangePreference (integer preference, integer value)	Allows you to change preference values using ModL code. The <i>value</i> argument takes a zero for FALSE or a one for TRUE. The <i>preference</i> argument takes the following values: 8: bitmap plotters 9: bitmap blocks 10: auto library search 12: simulation sound 14: right angle connections 16: unsupported XCMD callbacks beep (Mac OS only) 20: updatepubs (Mac OS only) 21: don't show worksheet tool tips 22: show dialog tool tips 25: don't show hierarchical block drop shadows 26: plotters use patterns. See GetPreference(), below.	V
ChangePreferenceS- tring (integer prefer- ence, string theString)	Change the string-based preferences using this function. The prefer- ences that can be changed with this function are: Default Library Path: 1 Library 1: 2 Library 2: 3 Library 3: 4 Library 4: 5 Library 5: 6 Library 6: 7 Library 7: 8 Default Model Path: 9	V
ClearBlock(integer blockNumber)	Clears the specified block from the worksheet.	V
ClearBlockUndo (integer theBlock- Number)	Clears the block and adds it to the delete task list to allow undoing. Returns FALSE if successful.	I
ClearConnec- tion(integer block- From, integer conFrom, integer blockTo, integer conTo)	Removes the connection between the specified blocks and connector numbers. Returns TRUE if successful.	I
ClearUndo(void)	Clears the Undo list and removes any existing undo tasks from the Edit menu. After executing this function, the Undo command will be disabled.	V
CloneCreate(inte- ger blockNumber, string variable- Name, integer desti- nation, integer left, integer top)	Returns a cloneID for the new clone, used in the other clone script- ing functions. VariableName is the dialog item variable name. Desti- nation is -1 for the top worksheet, -2 for the notebook or the block number of an H-block to put it into its submodel. Left and Top are the pixel coordinates of the left-top corner of the clone.	I

Scripting	Description	Return
CloneDelete(integer cloneID)	Deletes the specified clone. Get the cloneID from the CloneCreate() or CloneFind() functions.	I
CloneFind(integer blockNumber, string variableName, integer n)	Returns a cloneID for the found clone, used in the other clone scripting functions. VariableName is the dialog item variable name. N is the nth instance of the clone specified, in order of creation. If variableName is blank, the nth clone from the block is returned.	I
CloneGetDialogItem(integer cloneID)	Returns the variable name of the dialog item that the specified clone is from.	S
CloneGetDialogItemLabel(integer cloneID, integer n)	Returns the Nth label on a cloned dialog item that has labels. This would be either a Popup menu, or a data table.	S
CloneGetInfo(integer cloneID, integer whatInfo)	For whatInfo, 1:returns the type of the clone, 2:returns the block number of the clone, 3:returns True if the clone is selected and False if it is not. Values for the type of clone (whatInfo:1) are: 1:Button, 2:Check Box, 3:Radio Button, 4:Meter, 5:Parameter, 6:Slider, 7:data table, 8:EditText, 9:StaticText, 12:Switch, 13:String Table, 14:Plotter pane, 15:Plotter data table, 16:Popup Menu, 17:EmbeddedObject, 18:DynamicText, 19:Text frame, 20:Calendar, 21:EditText(31).	I
CloneGetList(integer blockNumber, string variableName, integer cloneIDArray[])	Returns the number of clones in the cloneIDArray array. Fills the dynamic array cloneIDArray with all of the cloneIDs for the variableName. If variableName is blank, all of the clone IDs for that block are returned in the array.	I
CloneGetPosition(integer cloneID, integer positionArray)	PositionArray is declared as an array of 4 integers. It returns the following information: positionArray[0] = cloneRect.top, positionArray[1] = cloneRect.left, positionArray[2] = cloneRect.bottom, positionArray[3] = cloneRect.right.	I
CloneHideDisable(integer cloneID, integer disable, integer disableIftrue)	Disables/enables or hides/shows the specified clone. If <i>disable</i> is True and <i>disableIftrue</i> is True, the clone is disabled. If <i>disable</i> is True and <i>disableIftrue</i> is False, the clone is enabled. If <i>disable</i> is False and <i>disableIftrue</i> is True, the function hides the clone; the clone appears if <i>disable</i> is False and <i>disableIftrue</i> is False. Returns zero if successful; otherwise returns an error code (negative number).	I
CloneResize(integer cloneID, integer top, integer left, integer bottom, integer right)	Resizes the clone to the specified rectangle (position and size).	I

Scripting	Description	Return
CodeExecute(string modlCodeString)	Executes the ModL code in the string <i>modlCodeString</i> . Local variables may be defined. Global variables can be used to input and return values.	V
CreateHBlock (string theName)	Makes the current selection into an H-block with the specified name.	I
DialogRefresh(integer blockNumber)	Forces the dialog box of the specified block to refresh (redraw.) Does nothing if the dialog box is not open or the block doesn't exist. Returns true (1) if it succeeds, zero otherwise.	I
DialogFixed-Size(integer block-Number, integer height, integer width)	Specifies that the dialog box for <i>blockNumber</i> should be fixed to the specified height and width. User resizing of the block dialog will be restricted.	I
DuplicateBlock (integer theBlock-Number)	Makes a copy of the indicated block, and returns the block number of the new block. Returns FALSE if successful.	I
ExecuteMenuCom-mand(integer com-mandNumber)	Executes the specified command. This is functionally the same as selecting the command from the menus. ExtendSim will attempt to perform the command on the active window. If there are multiple windows open (including dialog boxes), you may need to call ActivateWorksheet() to ensure that the correct model is in the active window. See Appendix C for a list of menu command numbers.	V
ExtendMaximize()	Maximizes the ExtendSim Application.	V
ExtendMinimize()	Minimizes the ExtendSim Application.	V
FindBlock (string searchStr, integer which, integer openDialogs, integer wholeWords, integer justBlock-Num)	Finds the first block that matches the string <i>searchStr</i> and returns its block number, optionally opening the dialog or selecting the block. <i>Which</i> specifies what string in the block you want to compare your searchstring to. It takes the following values: BlockLabel:1, BlockName:2, BlockType:3, TextBlockText:4 <i>openDialogs</i> specifies if the block should be just selected, or if the dialog should be opened. It is overridden by the <i>justBlockNum</i> parameter, if it is TRUE. <i>WholeWords</i> specifies if you want the text to try for an exact match, or to match a partial string. The final argument <i>justBlockNum</i> if set to true will set the function to just return a block number, and neither select the block nor open the dialog.	I
FindNext()	Finds the next block that matches the search string specified. Only useful if called immediately after the findBlock function. (Returns a -1 if no matching block is found.)	I
GetAppPath()	Returns a string containing the full path name for the ExtendSim application file. This function is useful for determining the location of files for which only the file's relative location to the ExtendSim application is known.	S

Scripting	Description	Return
GetPreference (integer preference)	Allows you to get preference values using ModL code. The return value is 1 for TRUE and 0 for FALSE. The <i>preference</i> argument takes the following values: 8 = bitmap plotters; 9 = bitmap blocks; 10 = auto library search; 12 = simulation sound; 14 = right angle connections; 16 = unsupported XCMD callbacks beep (Mac OS only); 20 = updatepubs (Mac OS only); 21 = don't show worksheet tool tips; 22 = don't show dialog editor tool tips; 25 = don't show hierarchical block drop shadows; 26 = plotters use patterns; 27 = metric units; 28 = use page info.	I
GetRecentFilePath(integer which)	Returns the full path to the specified recent file from the list of 5 recent files at the bottom of the File menu. The specified file (<i>which</i>) will be a number from 1 to 5.	S
GetUserPath()	Returns the path to the user documents directory. Similar to GetAppPath.	S
GetWorksheetFrame(integer blockNumber, array arrayName)	Returns the frame of the worksheet in the array <i>arrayName</i> , which must be declared integer arrayName[4]. Coordinates are in screen pixels and correspond to the information returned by the GetMouseX(), GetMouseY(), and GetBlockTypePosition() functions. ArrayName[0]:top, ArrayName[1]:left, ArrayName[2]:bottom, ArrayName[3]:right.	I
HBlockTopGet()	Returns the block number of the topmost open HBlock's submodel, or a negative number if there are no HBlocks open <i>above</i> the model worksheet.	I
HBlockUnlink-FromLibrary(integer blockNumber)	Disconnects the specified hBlock from the library it has a link to. This will make the hBlock into a standalone hBlock without a library connection.	V
IsBlockSelected (integer blockNumber)	Returns a TRUE if the indicated block is selected, returns false otherwise. This function will return a -1 if the indicated blocknumber is not a valid block.	I
IsLibEnabled(integer type)	Returns TRUE if the running version of ExtendSim will open a specific type of library. Type values are: 17: OR library, 18: AT library, 19: Suite library	I
IsMenuItemOn(integer whichItem)	Returns TRUE if the given menu command is currently checked. The whichItem argument uses the same numbers as defined in Appendix C for the ExecuteMenuCommand() function.	I
IsMetric()	Returns TRUE if metric is set in the options dialog.	I
LastBlockPlaced()	Returns the block number of the last block placed on the active worksheet.	I
LibraryGetInfoByName(unsigned string libName, string blockName, integer specify)	Function to return information about a library, and blocks in that library. Specify takes the following values; 1: Hblock - TRUE/FALSE	I

Scripting	Description	Return
MakeBlockInvisible (integer global-BlockNum, integer invisible)	Makes the indicated block invisible. (Turns it visible if it was already invisible, and the invisible flag is set to false.) Invisible blocks will not display on the worksheet at all, and cannot be selected by the user. Returns FALSE if successful.	I
MakeConnection (integer blockFrom, integer conFrom, integer blockTo, integer conTo)	Makes a connection line from the From block to the To block.	I
MoveBlock(integer blockNumber, integer xPixel, integer yPixel)	Moves the specified block the specified number of pixels. Coordinates refer to the upper left corner of the block.	I
MoveBlockTo(integer blockNumber, integer xLoc, integer yLoc)	Moves the specified block to the specified location. Coordinates refer to the upper left corner of the block.	I
OpenExtendFile (string fullPath)	Opens the ExtendSim model, library, or text file, using the <i>fullpath</i> name on the disk. This creates a new front window if opening a model or text file. Returns a zero if successful.	I
PlaceBlock(string blockName, string libName, integer xPixel, integer yPixel, integer neighbor, integer side)	Places the named block from the named library onto the active worksheet at the specified location. If the <i>neighbor</i> field is filled in with a block number of a block already on the worksheet, then the <i>xPixel</i> , <i>yPixel</i> values are relative to the location of the <i>neighbor</i> , otherwise if <i>neighbor</i> is -1 (no neighbor) they are absolute worksheet coordinates. The <i>side</i> argument determines how the new block will be placed relative to the old block: 0: Left, 1: top, 2: right, 3: bottom	I
PlaceBlockInH-Block (string blockName, string libName, integer xPixel, integer yPixel, integer HBlockNum)	Places a copy of the named block from the named library in the H-block that is specified by the HblockNum argument. See PlaceBlock in your manual for more information. Returns FALSE if successful.	I
PlaceDotBlock(integer xPixel, integer yPixel, integer neighbor, integer side, integer width, integer hBlockName)	Places a dot block into the worksheet. Arguments are similar to PlaceBlock, and PlaceTextBlock.	I

Scripting	Description	Return
PlaceTextBlock (string text, integer xPixel, integer yPixel, integer neighbor, integer side, integer width)	Places a Text Block with the string text as its content onto the active worksheet at the specified location. The width argument specifies the final width of the text block in pixels. See the description of PlaceBlock above for descriptions of the other arguments.	I
PlaceTextBlockInHBlock(string text, integer xPixel, integer yPixel, integer neighbor, integer side, integer width, integer hBlockNum)	This function places a TextBlock in the Specified HBlock. See the description of PlaceTextBlock for more information.	I
QuickTimeAvailable()	Returns whether or not quicktime is available on the current machine.	I
SelectConnection (integer blockFrom, integer conFrom, integer blockTo, integer conTo)	Selects the connection line associated with the connection between the specified blocks. This function returns a TRUE if it succeeds, and a FALSE if it fails.	I
SetDirty(integer dirty)	Sets/Unsets the “dirty” flag on the active worksheet. A common use for this functionality would be to set the dirty flag to “FALSE” before issuing the ExecutemenuCommand function to close a worksheet. This has the effect of closing the worksheet without querying the user if they want to save, or not.	V
SuppressWorksheetRedraw(integer suppress)	If suppress is TRUE, stops any update drawing of the worksheet until a call to SuppressWorksheetRedraw(FALSE), whereupon the worksheet will be redrawn.	V
UnselectAll()	Unselects all blocks, connections, etc.	V
WinSetForegroundWindow(integer handle)	Sets the application associated with the passed in handle to be the foreground window. Passing in a zero, sets ExtendSim as the foreground window. Windows handles for other applications then ExtendSim will need to be acquired through some means. One example would be querying the Excel object model through OLE/COM to return the object handle for Excel. See the code of the Command block in the Value library for an example of doing this.	V
WinShellExecute(string operation, string fileName, string params, string dir, integer show)	This calls the Windows ShellExecute() function. This specifies all arguments in the ShellExecute() function call but the first (HWND), as ExtendSim supplies that argument internally.	I

Scripting	Description	Return
WorksheetRe- fresh(integer block- Number)	Forces a worksheet refresh (redraw) of the worksheet window containing the specified block.	I

Reporting

These functions are used in the BlockReport message handler to organize reports written by blocks.

Reporting	Description	Return
GetReportType()	Used in BlockReport message handler to get the current report type. Returns 0 for Dialog report, 1 for Statistical report.	I
IsFirstReport()	Returns TRUE if this is the first block of a type getting a BlockReport message. Useful to set up column headers for that block type.	I

Plotting

These functions allow you to provide plotting in any block you create.

Each block can open up to four independent plot pages (numbered 0 to 3), each of which can plot traces (numbered 0, 1, 2 ...) that can be either arrays of points or functions. Each trace can be assigned one of two Y axes, Y and Y2, so that traces of different magnitudes can appear in the same plot.

The following arguments are used in the plotting functions:

Plot argument	Definition
Plot	Number (0 to 3) specifying one of the four possible plot pages
WhichSig	Number (0, 1, 2 ...) specifying one of the plot traces
SigName	Name for the trace
Y	Name of the dynamic array used in the plot. Use MakeArray to allocate Y before using it in a function.
StartTime	X-axis value that corresponds with the first point (y[0]) of the array.
EndTime	X-axis value that corresponds with the last point of the array.
PlotPts	Number of points in the array ready to plot, which does not need to be the number of points actually declared in the array. If there are no points ready to plot (because they have not yet been calculated), use 0 for this value.
UseY2Axis	If TRUE, the trace is plotted to the limits of the Y2 axis instead of the main Y axis.

Plot argument	Definition
Pattern	Pattern of the plot line. You can use the numbers or their associated ExtendSim constants. Note that these are different than the animation patterns. 0: BlackPattern 1: DkgrayPattern 2: GrayPattern 3: LtgrayPattern
Color	Color of the plot traces. You can use the numbers or their associated ExtendSim constants. Note that these are different than the animation colors. 33: BlackColor 205: RedColor 341: GreenColor 409: BlueColor 273: CyanColor 137: MagentaColor
MaxLines	Initial number of rows in the plot's data pane.
IsXLog	If TRUE, the X axis is logarithmic.
IsYLog	If TRUE, the Y axis is logarithmic.
IsY2Log	If TRUE, the Y2 axis is logarithmic.
FunctionName	Name of a real, single argument function. For example, cos(x) should be entered as cos .
NumFormat	Format for the numbers in the data pane. 0: General 1: decimal (2 places) 2: integer 3: scientific (exponential)
LineFormat	Format for traces. 0: connected points 1: rectangular connected points (no interpolation between points) 2: dots
SymFormat	Symbols used on traces. 0 – . ; 1 – □ ; 2 – △ ; 3 – ○ ; 4 – + ; 5 – ■ ; 6 – ▲ ; 7 – ● 8 – # (displays the trace's number); 9 – ✕

General plotting

General plotting	Description	Return
PlotNewBarPoint (integer plot, integer whichSig, integer whichBin, real Value)	This function allows you to set the value of one of the bins in a histogram/barchart chart, rather than adding 1 to it as the PlotNewPoint function will do.	V

General plotting	Description	Return
AutoScaleX(integer plot)	Finds the minimum and maximum X axis values of all installed arrays or installed scatter arrays and adjusts the X axis limits accordingly.	V
AutoScaleY(integer plot)	Finds the minimum and maximum Y values of all installed arrays or installed scatter arrays and adjusts both the Y and Y2 axis limits accordingly.	V
BarGraph(integer plot, integer aBins, real aMin, real aMax)	Sets the number of bins, the minimum value, and maximum value for a bar graph.	V
ChangeAxisValues (integer plot, real xLo, real xHi, real yLo, real yHi, real y2Lo, real y2Hi)	Sets or changes the axis values of the specified plotter to the value specified. Specifying a BLANK for a given value will use the existing value. Returns a TRUE if the function executed successfully.	I
ChangePlotType (integer plot, integer plotType)	Changes the defined plot type of a plotter. This is used when creating custom plotters. The call to change the plotter should be done right after the call to install an axis and before any other of the plotter calls. 1 – linear plot; 2 – scatter plot; 3 – error bars; 4 – strip chart; 5 – worm plot; 6 – bar plot	V
ChangeSignalColor (integer plot, integer whichSig, integer color)	Specifies the color for the trace.	V
ChangeSignalSymbol(integer plot, integer whichSig, integer symFormat)	Specifies the symbol for the trace.	V
ChangeSignalWidth(integer plot, integer whichSig, integer width)	Specifies the width (in pixels) for the trace.	V
ClosePlotter(integer plot)	Closes the plot window, if open.	V
ClosePlotter2 (integer blockNumber, integer plot)	This function just closes the specified plotter window, if it is open.	V
DisposePlot(integer plot)	Disposes of a plot window and all its saved plots. This does not dispose of any installed data arrays.	V
GetAxis(integer plot, real axisValues[9])	Fills the first nine elements of the <i>axisValues</i> array (declared as real <i>axisValues</i> [9]) with the current values of isXLog, xLow, xHi, isYLog, yLow, yHi, isy2Log, y2Low and y2Hi.	V

General plotting	Description	Return
GetAxisName(integer plot, integer whichAxis)	Returns the name of the specified axis. whichAxis: 1 - x 2 - y1 3 - y2	S
GetSignalName(integer plot, integer whichSig)	Returns the name of the signal <i>whichSig</i> .	S
GetSignalValue(integer plot, integer whichSig, real xAxisValue)	Finds the value of the installed trace at <i>xAxisValue</i> . If the trace is an installed array, the value is interpolated between adjacent points. If the trace is an installed function, the function is evaluated at <i>xAxisValue</i> . This does not work for scatter plots because there may be many Y values for a single <i>xAxisValue</i> .	R
GetTickCount(integer plot, integer whichCount)	Returns the number of ticks on the plotter axis. <i>WhichCount</i> is 0 for the X axis, 1 for the Y axis, and 2 for the Y2 axis.	I
GetY1Y2Axis(integer plot, integer whichSig)	Returns an integer specifying which Y axis the signal is plotted against and whether or not the signal is hidden. 1: y1 2: y2 -1: y1, hidden -2: y2, hidden Note: If you are not concerned whether the signal is hidden, take the absolute value of the result.	I
InstallArray(integer plot, integer whichSig, string sigName, real y, real StartTime, real EndTime, integer plotPts, integer useY2Axis, integer pattern, integer color)	Allows the automatic plotting of arrays. The plot window will plot all the points of this array up to plotPts-1. It should be preceded by InstallAxis and eventually be followed by showPlot. The first time arrays are installed they need to be installed in sequential order. The Install Array call is used in two ways in plotter blocks. The first time it is called, it installs the array and sets parameters such as the color and pattern of the line. The second and subsequent times, the formatting options are ignored, and the only action a call to installArray takes is to reinstall the array itself. If you precede a call to InstallArray with a call to RemoveSignal, the formatting information will be used.	V

General plotting	Description	Return
InstallAxis(integer plot, string title, string xName, integer isXLog, real xLow, real xHi, string yName, integer isYLog, real yLow, real yHi, string y2name, integer isY2Log, real y2Low, real y2Hi, integer y2Pattern, integer y2Color, integer maxLines)	Installs both the X and Y axes. If both the <i>y2Low</i> and <i>y2Hi</i> arguments are 0, the Y2 axis is not shown.	V
InstallFunction (integer plot, integer whichSig, string sigName, function-Name, integer useY2Axis, integer pattern, integer color)	Allows the automatic plotting of functions. The plot window will plot all the points of this function corresponding to the x axis values, even if you changed them. It should be preceded by InstallAxis and eventually followed by ShowPlot. Call InstallFunction every time the plot window is shown (before ShowPlot).	V
NumPlotPoints (integer plot, integer points)	Specifies the number of points to be stored for worm and strip plots.	V
PlotNewPoint(integer plot, integer whichSig, integer index, real yValue)	Adds and plots a new <i>yValue</i> point to the installed array. It is useful when you want to see points plotted as they are calculated within a loop (such as during simulations). To use this effectively, first use InstallArray with a value of 0 for the plotPts argument. PlotNewPoint increments the plotPts of the installed array when the point is plotted, and the equation “ <i>y</i> [index] = <i>yValue</i> ” is internally executed, putting the new <i>yValue</i> into the installed array. Note that the first value of index must be 0 when calling PlotNewPoint for a newly installed array.	V
PlotSignalFormat (integer plot, integer whichSig, integer lineFormat, integer numFormat)	Specifies the line format and number format for the trace.	V
PlotterBackground (integer plot, string backName)	Specifies that the named picture should be used as the background for the specified plotter. The picture file must be in the extensions folder. See “Sounds on the Mac OS” on page 85 for more information on how external pictures are used in ExtendSim.	I
Plotter-NameGet(integer plot)	Returns the name of the specified plotter.	S

General plotting	Description	Return
PlotterName-Set(integer plot, string newName)	Sets the name of the specified plotter.	I
PlotterSignalColor-Set(integer plot, integer whichSig, integer Hue, integer Sat, integer Value)	Sets the color of the specified signal. This function will automatically set the color value of the signal to custom, and will then define the custom color values to be the values you pass in.	V
PlotterSignalValueGet(integer plot, integer whichSig, integer whichValue)	Returns a value associate with the specified signal. The whichValue argument specifies which number that should be used. WhichValue: 0: Color, 1: Hue, 2: Sat, 3: Value, 4: Signal Visibility, 5: use Y2Axis, 6: Line Format	I
PlotterSignalValue-Set(integer plot, integer whichSig, integer whichValue, integer value)	Sets the value of a specified aspect of a plotter signal. Which values are the same as for PlotterSignalValueGet.	I
PlotterSquare(integer plot, integer trueFalse)	If true, forces the window to be square. If false, it removes the squaring restriction.	V
PlotterXAxisCalendar(integer plot, integer xAxisCalendar, integer format)	Turns on or off Calendar date behavior on the x axis of the specified plotter. The xAxisCalendar flag set to a true value will set the plotter to redraw with Calendar date values on the x Axis. The format parameter takes the following values: 0: everything 1: Just date, (ignore time) 2: Just time, (ignore date) 3: tight format. (Two digit year, and don't show time value if zero.)	V
PlotterXAxisIs-Time(integer plot, integer xAxisTime)	This function sets a true/false flag in the specified plotter that tells the plotter if the X axis is defined as a time value or not. The primary purpose for which the plotter uses this is for optimization. If the X axis of the plotter is scaled to a certain max value, and the next X value is beyond that value, the plotter knows that it is done drawing because all subsequent X value will be higher than the current values. Without this optimization, the DE plotter, which is based on a scatter plotter, not a continuous plotter, will continue to attempt to draw the rest of the points in the data unnecessarily.	V
PushPlotPic(integer plot)	Pushes the top plot picture, page 1, down to page 2 of the plot window. The oldest plot (page 4) is discarded. This function only works if the plot is showing when the function is called.	V
RefreshPlotter(integer plot, integer wholeframe, integer openPlotterWindow)	Function that will redraw the plotter without opening the plotter window if it is closed. Used for redrawing the clones of a plotter when a change has occurred.	V

General plotting	Description	Return
RemoveSignal(integer plot, integer whichSig)	Removes an installed array or function from a plot so that a new one can be installed.	V
RenamePlotter(integer plot, string new-name)	Renames the plotter.	V
RetimeAxis(integer plot)	Sets the X axis values to the simulation start and end times. This is useful for plotting simulation results without having to reset the x axis values before a simulation is run.	V
RetimeAxisNStep(integer plot, integer nStep)	This function is used to correct axis values when “Plot every nth Point” is selected in the plotter’s dialog. As an example, see the Plotter I/O block in the Plotter library.	V
SetAxisName(integer plot, integer whichAxis, string theName)	Sets the Axis name of the specified Axis. <i>whichAxis</i> : 1-x 2-y1 3-y2	I
SetSignalName(integer plot, integer whichSig, string theName)	Sets the name of signal <i>whichSig</i> .	V
SetTickCounts(integer plot, integer xCount, integer yCount, integer y2Count)	Specifies the number of ticks on the plotter axes in the plot. Specifying -1 as the parameter will use the current value.	V
ShowPlot(integer plot, string plot-Name)	Opens and names the plot window. If the plot has never been used, an empty plot is shown. Additional calls bring the window to the front. <i>plotName</i> is a string that contains the name of the plot. Note that ShowPlot will not change the name of a plot that was typed in the plot dialog.	V
ShowPlot2(integer blockNumber, integer plot)	Shows the specified plotter. The only difference between this function and ShowPlot is the block number argument, which allows you to show a plotter in a remote block.	V
SwitchPlotterRedraw(integer plot, integer direction)	Specifies the direction in which the plot will be redrawn. A <i>direction</i> of 0 specifies left to right, a <i>direction</i> of 1 specifies right to left. This only affects the way that the plot lines are redrawn when the plot is complete.	V

Remember to call the ShowPlot function to replot any changes made by these functions. Be sure to call ShowPlot after all calls that change the plot axis limits or formats.

The structure and contents of plots are saved with the model file, whether they were open or not.

Scatter plot functions

To plot scatter plots (XY plots), you need two additional functions. Scatter plot windows combine two traces previously installed into a plot. The traces are combined into an XY trace, where the first trace supplies the X values, and the second trace supplies the Y values. The values of the *whichSig* argument for the Scatter Plot functions must range from 0 to an even number minus one specifying the highest numbered scatter trace.

Scatter plots	Description	Return
MakeScatter (integer plot, integer <i>whichSig</i>)	Combines the two traces (the first specified by <i>whichSig</i>) into a scatter trace. Both <i>whichSig</i> and <i>whichSig</i> +1 traces must be the same length and must have been installed by InstallArray before MakeScatter is called.	V
PlotNewScatter (integer plot, integer <i>whichSig</i> , integer index, real <i>xValue</i> , real <i>yValue</i>)	Similar to plotNewPoint but for scatter plots. <i>xValue</i> is put into <i>whichSig</i> and <i>yValue</i> is put into <i>whichSig</i> +1.	V

To see how the plotting functions can be used in your own blocks, examine the plotter functions that are used in the code of the various plotters in the Plotter library.

Database functions

These are used to create, read, write, import, export, and delete databases and their components. They are used by blocks to create and manipulate databases during a run. All database indexes are 1 based, so they start at 1 and end at N. Databases and tables maintain their indexes, even when other databases or tables are deleted. Fields and record indexes can change if earlier fields or records are deleted.

 Reserved databases use specially named functions to manipulate them so they don't inadvertently get changed by the user or developer. These functions have the word "Reserved" in their name.

Blocks can be linked to parts of the database, so if the database structure or data changes, they will be notified and can take action. See "Dynamic linking" on page 302 and the LINKSTRUCTURE and LINKCONTENT messages.

Error codes

For database read/write functions, the Database menu command "Read/Write Index Checking" can enable error messages if a read/write function call has illegal indexes. This is a good tool to find illegal indexes and leaving this check on does not impact the speed of a simulation run. Leave it off if it is preventing a legacy model from running. New models have this check on by default.

In most cases, functions return zero if no error, or non-zero error codes when an error occurs:

Error code	Value	Description
no such record	-1	One of the indexes are incorrect.
no such parent	-2	Trying to set a child field to a value that is not a parent value.
not unique error	-3	Trying to set a cell to a non-unique value in a field with the "Unique" property set.

Error code	Value	Description
not unique index	>0	DBSetDataAs... functions will return the record index of the currently existing unique value if you try to set a cell to a non-unique value.
not linked error	-4	Returned by DBDataGetAsNumber() on a <i>list of tables</i> field if a table name found in a database cell doesn't exist and doesn't have an index in the database.

Creating and deleting database components

DB creating/deleting	Description	Return
DBDatabaseCreate(string databaseName)	Creates database databaseName and returns its index. Error: returns negative error code if existing name already used.	I
DBDatabaseDelete(string databaseName)	Deletes entire database. Returns negative error code if DB doesn't exist.	I
DBDatabaseDeleteByIndex(integer databaseIndex)	Deletes entire database using index, returns error code. See also DBDatabaseDelete() in the Developer Reference.	I
DBDatabaseTabDelete(integer databaseIndex, Str255 existingTabName)	Deletes existingTabName tab in databaseIndex database viewer. Returns -1 if error.	I
DBFieldCreate(string databaseName, string tableName, string fieldName, integer fieldFormat, integer decimals, integer unique, integer readOnly, integer invisible)	Creates field fieldName with specified attributes tableName and returns its index. Error: returns negative error code if existing name already used. Field format constants defined: DB_FIELDTYPE_INTEGER_VALUE, DB_FIELDTYPE_INTEGER_BOOLEAN (checkbox), DB_FIELDTYPE_REAL_GENERAL, DB_FIELDTYPE_REAL_SCIENTIFIC, DB_FIELDTYPE_REAL_PERCENT, DB_FIELDTYPE_REAL_CURRENCY, DB_FIELDTYPE_REAL_DATE_TIME, DB_FIELDTYPE_REAL_DB_ADDRESS, DB_FIELDTYPE_STRING_VALUE, DB_FIELDTYPE_TABLELIST	I
DBFieldCreateByIndex(integer databaseIndex, integer tableIndex, string fieldName, integer fieldFormat, integer decimals, integer unique, integer readOnly, integer invisible)	Creates field <i>fieldName</i> using specified indexes, returns error code. See also DBFieldCreate() in the Developer Reference.	I

DB creating/deleting	Description	Return
DBFieldDelete(string databaseName, string tableName, string fieldName)	Delete field. Returns negative error code if field doesn't exist.	I
DBFieldDeleteByIndex(integer databaseIndex, integer tableIndex, integer fieldIndex)	Deletes field using index, returns error code. See also DBFieldDelete() in the Developer Reference.	I
DBRecordsDelete(integer databaseIndex, integer tableIndex, integer startRecord, integer endRecord)	Delete records from a table. Returns negative error code if table doesn't exist.	I
DBRecordsInsert(integer databaseIndex, integer tableIndex, integer insertAtRecord, integer numberRecords)	Insert at record index or append (insertAtRecord is zero) new records to a table, returns tableIndex if ok. Returns negative error code if table doesn't exist.	I
DBRelationCreate(string databaseName, string tableChildName, string fieldChildName, string tableParentName, string fieldParentName)	Create relation, returns negative error code if anything doesn't exist.	I
DBRelationDelete(string databaseName, string tableChildName, string fieldChildName, string tableParentName, string fieldParentName)	Delete relation. Returns negative error code if relation doesn't exist.	I
DBTableCloneToTab(integer databaseIndex, integer tableIndex, string tableName)	Creates the tableName if not there, and clones the table to the tab.	V
DBTableCreate (string databaseName, string tableName)	Creates table tableName and returns its index. Error: returns negative error code if existing name already used.	I
DBTableCreateByIndex(integer databaseIndex, string tableName)	Creates table <i>tableName</i> in current database using indexes, returns negative error code. See also DBTableCreate() in the Developer Reference.	I

DB creating/deleting	Description	Return
DBTableDelete(string databaseName, string tableName);	Delete table. Returns negative error code if Table doesn't exist.	I
DBTableDeleteByIndex(integer databaseIndex, integer tableIndex)	Deletes table using index, returns error code. See also DBTableDelete() in the Developer Reference.	I
DBToolTipsGet(integer databaseIndex, integer tableIndex, integer fieldIndex)	Gets the string value of the tooltip. 1) All indexes good, gets field tooltip. 2) FieldIndex is zero, gets table tooltip. 3) Field and Table index zero, gets database tooltip.	S
DBToolTipsSet(integer databaseIndex, integer tableIndex, integer fieldIndex, string value)	Sets the tooltip to <i>value</i> . 1) All indexes good, sets field tooltip. 2) FieldIndex is zero, sets table tooltip. 3) Field and Table index zero, sets database tooltip.	I

Selecting parts of a database

These functions open a database component selection dialog so the user can select a desired component.

DB selecting	Description	Return
DBChildPopupSelector(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordNum, integer trueForSelectorDialog)	Opens a child field parent value selector similar to clicking on a child field popup arrow, and changes the child value according to which parent value the user selects. If trueForSelectorDialog is TRUE, a selector dialog is shown with a scroll bar, good for a large numbers of parent values. If FALSE, a popup menu is shown, which is good for a small number of parent values. Returns the selected parent record index or zero if cancelled.	I
DBDatabasePopupSelector(real currentDBIndex)	Opens a database selector dialog and selects the currentDBIndex if not BLANK or zero. If BLANK or zero, doesn't select any entries in the list. Returns the selected database index or zero if cancelled.	I
DBFieldPopupSelector(integer databaseIndex, integer tableIndex, real currentFieldIndex)	Opens a field selector dialog and selects the currentFieldIndex if not BLANK or zero. If BLANK or zero, doesn't select any entries in the list. Returns the selected field index or zero if cancelled.	I
DBRecordPopupSelector(integer databaseIndex, integer tableIndex, integer fieldIndex, real currentRecordIndex)	Opens a record selector dialog and selects the currentRecordIndex if not BLANK or zero. If BLANK or zero, doesn't select any entries in the list. Returns the selected record index or zero if cancelled.	I

DB selecting	Description	Return
DBTablePopupSelector(integer databaseIndex, integer tableIndex)	Opens a table selector dialog and selects the currentTableIndex if not BLANK or zero. If BLANK or zero, doesn't select any entries in the list.	
	Returns the selected table index or zero if cancelled.	

Copying parts of a database

DB copy	Description	Return
DBDatabaseCopy(integer fromDatabaseIndex, string newDatabaseName)	Copies entire database to a new database. Returns index of the new database or -1 if error (bad index or same name as existing database).	I
DBTableCopy(integer fromDatabaseIndex, integer fromTableIndex, integer toDatabaseIndex, string newTableName)	Copies and optionally renames an existing table into its current database (must use newTableName) or another database (If newTableName is blank, keeps the existing table name). Returns index of the new table or -1 if error (bad index or same table name as existing table in that database).	I

Import and export a database

DB import/export	Description	Return
DBDatabaseExport(string databaseName, string pathName)	Export database to DB text file pathName. If pathName is a blank string, let the user select a text file. Returns -1 if error.	I
DBDatabaseImport(string databaseName, string pathName)	Import database from text file <i>pathName</i> , replacing the database named <i>databaseName</i> or creating it if it didn't exist. If the database name is in the form "databaseName<delete>", it deletes any left over tables that were not imported in this file. If pathName is a blank string, let the user select a text file. Returns the database index or -1 if it fails. Sends both a LinkStructure message (what changed: DB replaced) and a LinkContent message (what changed: data changed) to linked blocks. See DILinkUpdateInfo() for an explanation of <i>what changed</i> .	I
DBTableExportData(string pathName, string userPrompt, string format, integer databaseIndex, integer tableIndex, integer rows, integer columns)	Exports the table data to a delimited text file. If pathName is a blank string, let the user select a text file. If format is a blank string, uses a tab as delimiter (see Import() functions). Returns -1 if error.	I

DB import/export	Description	Return
DBTableImportData(string path-Name, string userPrompt, string format, integer databaseIndex, integer tableIndex)	Imports a delimited text file into a database table and returns the number of rows imported. If format is a blank string, uses a tab as delimiter (see Import() functions). This function maintains existing relations in the table but will discard relations that violate parent/child data requirements: all child data in a relation must be blank or exist in the parent data set. NOTE: This function automatically resizes the table to the number of rows imported, so you do not need to allocate any records in the table before calling this function.	I

Database properties

DB properties	Description	Return
DBDatabaseExists(integer databaseIndex)	Passing in a database index, returns TRUE if the database exists, FALSE if it doesn't exist. Also see DBTableExists(), DBFieldExists(), and DBRecordExists().	I
DBDatabaseGetIndex(string database-Name)	Returns database index or negative error if not found.	I
DBDatabaseGetName(integer databaseIndex)	Gets the name of the database or blank string if bad index.	S
DBDatabaseRename(integer databaseIndex, string newDatabaseName)	Renames a database. Returns a negative error code if <i>newDatabaseName</i> already exists or the old database doesn't exist.	I
DBDatabasesGetNum()	Because database indexes remain constant even if some databases are deleted, this returns number of database slots of which some could be empty. To count how many actual databases there are, use a loop and count the databases that have a non-blank name.	I
DBDatabaseShowHideReserved(integer showIfTrueHideIfFalse)	Use this to hide or show reserved databases without using the menu command. Reserved databases are discussed on page 106. Note: When the model closes, the reserved databases return to being hidden. So this command must be called again whenever the model is opened.	V
DBDatabaseTabChangeName(integer databaseIndex, Str255 existingTabName, Str255 newTabName)	Changes the name of existingTabName to newTabName in databaseIndex database viewer. Returns -1 if error.	I

DB properties	Description	Return
DBFieldExists(integer databaseIndex, integer tableIndex, integer fieldIndex)	Passing in a database index, table index, field index, returns TRUE if the field exists, FALSE if it doesn't exist. Also see DBDatabaseExists(), DBTableExists(), and DBRecordExists().	I
DBFieldGetIndex(integer databaseIndex, integer tableIndex, string fieldName)	Returns field index or negative error if not found.	I
DBFieldGetName(integer databaseIndex, integer tableIndex, integer fieldIndex)	Gets the name of the field or blank string if bad index.	S
DBFieldGetProperties(integer databaseIndex, integer tableIndex, integer fieldIndex, integer which)	<i>which</i> : 1: fieldType, 2: fieldDecimals, 3: fieldUnique, 4: fieldReadOnly, 5: fieldInvisible, 6: IfFieldID or -1 if error. See the DBFieldCreate() function, above.	I
DBFieldGetPropertiesUsingAddress(real dbAddress, integer which)	This uses a DBAddress (see the DBAddress functions). <i>which</i> : 1: fieldType, 2: fieldDecimals, 3: fieldUnique, 4: fieldReadOnly, 5: fieldInvisible, 6: IfFieldID or -1 if error. See the DBFieldCreate() function, above.	I
DBFieldMove(integer databaseIndex, integer tableIndex, integer fieldIndex, integer newFieldIndex)	Moves field <i>fieldName</i> to <i>newFieldIndex</i> , moving other fields in the process. Returns a negative when indexes are incorrect.	I
DBFieldRename(integer databaseIndex, integer tableIndex, integer fieldIndex, string newFieldName)	Renames a field. Returns a negative error code if the field already exists or the old field doesn't exist.	I
DBFieldSetInitialize(integer databaseIndex, integer tableIndex, integer fieldIndex, integer initFlag, integer initSimFlag, string value)	Set the field's initialization parameters. <i>initFlag</i> : 0 no init (default), 1 init to initDataValue. <i>initSimFlag</i> : 0 init each run (default), 1 init first run of multisim. <i>value</i> : what value to init the field, real or string for string fields. Returns an error code if bad indexes.	I
DBFieldSetProperties(integer databaseIndex, integer tableIndex, integer fieldIndex, integer which, integer newValue)	<i>which</i> : 1: fieldType, 2: fieldDecimals, 3: fieldUnique, 4: fieldReadOnly, 5: fieldInvisible, 6: IfFieldID. Returns -1 if index error. See the DBFieldCreate() function, above.	I

DB properties	Description	Return
DBFieldsGetNum(integer databaseIndex, integer tableIndex)	Returns number of fields. Returns negative error code if no fields or no such table index.	I
DBRecordExists(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex)	Passing in a database index, table index, field index, record index, returns TRUE if the record (database cell) exists, FALSE if it doesn't exist. Also see DBDatabaseExists(), DBTableExists(), and DBFieldExists().	I
DBRecordIDFieldGetIndex(integer databaseIndex, string tableName)	Returns field index of the recordID field, if any. Returns negative error if no fieldID found.	I
DBRecordsGetNum(integer databaseIndex, integer tableIndex)	Returns number of records in a table. Returns negative if no such table index.	I
DBRelationsGetNames(integer databaseIndex, integer relationIndex, string relationNames[])	Returns zero error code and tableChild, fieldChild, tableParent, fieldParent in string relationNames[4]	I
DBRelationsGetNum(integer databaseIndex)	Returns number of relations in the DB. Returns negative error code if no DB.	I
DBTableExists (integer databaseIndex, integer tableIndex)	Passing in a database and table index, returns TRUE if the table exists, FALSE if it doesn't exist. Also see DBDatabaseExists(), DBFieldExists(), and DBRecordExists().	I
DBTableGetIndex(integer databaseIndex, string tableName)	Returns table index or negative error if not found.	I
DBTableGetProperties(integer databaseIndex, integer tableIndex)	Returns the initialization method: 0 is no initialization, 1 is delete all records for all runs, 2 is delete all records for only the first run. Returns -1 if error.	I
DBTableGetName(integer databaseIndex, integer tableIndex)	Gets the name of the table or blank string if bad index.	S
DBTableSetProperties(integer databaseIndex, integer tableIndex, integer initializeMethod)	Sets the initialization method: 0 is no initialization, 1 is delete all records for all runs, 2 is delete all records for only the first run. Returns -1 if error.	I

DB properties	Description	Return
DBTableRename(integer databaseIndex, integer tableIndex, string newTableName)	Renames a table. Returns a negative error code if the table already exists or old table doesn't exist.	I
DBTablesGetNum(integer databaseIndex)	Because table indexes remain constant even if some tables are deleted, returns number of table slots of which some could be empty. To count how many actual databases there are, use a loop and count the tables that have a non-blank name.	I

Read and write to a database

 Note that reading a “List of Tables” field returns the table index if read as a number, and the name of the table if read as a string.

DB read/write	Description	Return
DBDataGetAsNumber(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex)	Reads the value as a number without any formatting. Note that random distributions in cells will return a different random number each time that this function is called. Note: Using this function to read a “List of Tables” field returns the index of the table read.	R
DBDataGetAsNumberParentAltField(real childDBAddress, integer altFieldIndex)	This function, when called with a child DBAddress, allows returning the value from a different field of the parent record. Reads the value as a number without any formatting. Note that random distributions in cells will return a different random number each time that this function is called. Note: Using this function to read a “List of Tables” field returns the index of the table read.	R
DBDataGetAsNumberUsingAddress(real dbAddress)	Using a DBAddress, read the value as a number without any formatting. Note that random distributions in cells will return a different random number each time that this function is called. Note: Using this function to read a “List of Tables” field returns the index of the table read.	R
DBDataGetAsString(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex)	Reads the value as a string at full precision. Percentage format returns the normalized value as “1.00” for database cells that read 100%. NOTE that if the table is a table of tables, this returns the table name (see DBGetDataAsNumber() below. Note that random distributions in cells will return a different random number each time that this function is called. Note: Using this function to read a “List of Tables” field returns the name of the table read.	S

DB read/write	Description	Return
DBDataGetAsStringParentAltField(real child-DBAddress, integer altFieldIndex)	This function, when called with a child DBAddress, allows returning the value from a different field of the parent record. Reads the value as a string at full precision. Percentage format returns the normalized value as “1.00” for database cells that read 100%. NOTE that if the table is a table of tables, this returns the table name (see DBGetDataAsNumber() below. Note that random distributions in cells will return a different random number each time that this function is called. Note: Using this function to read a “List of Tables” field returns the name of the table read.	S
DBDataGetAsStringUsingAddress(real dbAddress)	Using a DBAddress, read the value as a string at full precision. Percentage format returns the normalized value as “1.00” for database cells that read 100%. NOTE that if the table is a table of tables, this returns the table name (see DBGetDataAsNumber() below. Note that random distributions in cells will return a different random number each time that this function is called. Note: Using this function to read a “List of Tables” field returns the name of the table read.	S
DBDataGetDateAsSimTime(integer dbIndex, integer tableIndex, integer fieldIndex, integer recordIndex, integer timeUnits)	Reads a database cell as a date, and converts it to a simulation time value. This allows the user to read a date value directly as a simulation time value, avoiding the conversion process.	R
DBDataGetDateAsSimTimeUsingAddress(real addressValue, integer timeUnits)	Using a dbAddress, reads a database cell as a date, and converts it to a simulation time value. This allows the user to read a date value directly as a simulation time value, avoiding the conversion process.	R
DBDataGetParent(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex, integer parentArray[3])	Get the parent information from a child field. ParentArray can be declared as a local variable array (can be declared in equation blocks): integer parentArray[3]; parentArray[0] returns parent Table index parentArray[1] returns parent Field index parentArray[2] returns parent Record index (or zero if no child value has been set) Note that this function also returns the parent record index (or zero if no child value has been set) (same as parentArray[2]) or a negative error code.	I

DB read/write	Description	Return
DBDataGetParentUsingAddress(real dbAddress, parentArray)	Using a DBAddress, get the parent information from a child field address. ParentArray can be declared as a local variable array (can be declared in equation blocks): <pre>integer parentArray[3];</pre> parentArray[0] returns parent Table index parentArray[1] returns parent Field index parentArray[2] returns parent Record index (or zero if no child value has been set) Note that this function also returns the parent record index (or zero if no child value has been set) (same as parentArray[2]) or a negative error code.	I
DBDataSetAsNumber(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex, real valueDouble)	Write value of field as number to a record. Returns a negative error code. Only sends LINKCONTENT message to block if value has changed. If setting a unique field cell to a non-unique value, returns the record index of the original unique value. This function does not work with reserved databases. See DBDataSetAsNumberReserved(), which works only with reserved databases.	I
DBDataSetAsNumberReserved(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex, real valueDouble)	This function works only with reserved databases. Otherwise, it is the same as DBDataSetAsNumber(), which works only with non-reserved databases.	I
DBDataSetAsNumberUsingAddress(real dbAddress, real valueDouble)	Using a DBAddress, write value of field as number to a record. Returns a negative error code. Only sends LINKCONTENT message to block if value has changed. If setting a unique field cell to a non-unique value, returns the record index of the original unique value. See DBDataSetAsNumberUsingAddressReserved(), which works only with reserved databases.	I
DBDataSetAsNumberUsingAddressReserved(real dbAddress, real valueDouble)	This function works only with reserved databases. Otherwise, it is the same as DBDataSetAsNumberUsingAddress(), which works only with non-reserved databases.	I
DBDataSetAsParentIndex(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex, integer parentIndex)	For a child field, sets the parent index directly rather than with DBPutDataAs... functions trying to set the parent index by finding the data value in the parent. Only sends LINKCONTENT msg to block if value changed. Note that you can set the parentIndex to zero if you want to set the child value to <no value yet>. See DBDataSetAsParentIndexReserved(), which works only with reserved databases.	I

DB read/write	Description	Return
DBDataSetAsParentIndexReserved(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex, integer parentIndex)	This function works only with reserved databases. Otherwise, it is the same as DBDataSetAsParentIndex(), which works only with non-reserved databases.	I
DBDataSetAsParentIndexUsingAddress(real dbAddress, integer parentIndex)	Using a DBAddress for a child field, sets the parent index directly rather than with DBPutDataAs... functions trying to set the parent index by finding the data value in the parent. Only sends LINKCONTENT msg to block if value changed. Note that you can set the parentIndex to zero if you want to set the child value to <no value yet>. See DBDataSetAsParentIndexUsingAddressReserved(), which works only with reserved databases.	I
DBDataSetAsParentIndexUsingAddressReserved(real dbAddress, integer parentIndex)	This function works only with reserved databases. Otherwise, it is the same as DBDataSetAsParentIndexUsingAddress(), which works only with non-reserved databases.	I
DBDataSetAsString(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex, string valueString)	Write value of field as string to a record. Percentage format expects the normalized value as "1.00" for database cells that read 100%. Returns a negative error code. Only sends LINKCONTENT message to block if value has changed. If setting a unique field cell to a non-unique value, returns the record index of the original unique value. See DBDataSetAsStringReserved(), which works only with reserved databases.	I
DBDataSetAsStringReserved(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex, string valueString)	This function works only with reserved databases. Otherwise, it is the same as DBDataSetAsString(), which works only with non-reserved databases.	I
DBDataSetAsStringUsingAddress(real dbAddress, string valueString)	Using a DBAddress, write value of field as string to a record. Percentage format expects the normalized value as "1.00" for database cells that read 100%. Returns a negative error code. Only sends LINKCONTENT message to block if value has changed. If setting a unique field cell to a non-unique value, returns the record index of the original unique value. See DBDataSetAsStringUsingAddressReserved(), which works only with reserved databases.	I
DBDataSetAsStringUsingAddressReserved(real dbAddress, string valueString)	This function works only with reserved databases. Otherwise, it is the same as DBDataSetAsStringUsingAddress(), which works only with non-reserved databases.	I

DB read/write	Description	Return
DBDataSetDateAsSimTime(integer dbIndex, integer tableIndex, integer fieldIndex, integer recordIndex, real simTimeVal, integer timeUnits)	Takes a simulation time value, converts it to a date, and sets a DB cell with that date value. See DBDataSetDateAsSimTimeReserved(), which works only with reserved databases.	I
DBDataSetDateAsSimTimeReserved(integer dbIndex, integer tableIndex, integer fieldIndex, integer recordIndex, real simTimeVal, integer timeUnits)	This function works only with reserved databases. Otherwise, it is the same as DBDataSetDateAsSimTime(), which works only with non-reserved databases.	I
DBDataSetDateAsSimTimeUsingAddress(real addressValue, real simTimeVal, integer timeUnits)	Using a dbAddress, takes a simulation time value, converts it to a date, and sets a DB cell with that date value. See DBDataSetDateAsSimTimeUsingAddressReserved(), which works only with reserved databases.	I
DBDataSetDateAsSimTimeUsingAddressReserved(real addressValue, real simTimeVal, integer timeUnits)	This function works only with reserved databases. Otherwise, it is the same as DBDataSetDateAsSimTimeUsingAddress() which works only with non-reserved databases.	I

Random data in a database

DB random data	Description	Return
DBDataGetCurrentSeed(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex)	Returns current running seed value from a random cell specified by indexes. Returns zero if bad cell.	I
DBDataSetCurrentSeed(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex, integer seedValue)	Sets current running seed value for a random cell specified by indexes. Returns 0 if bad cell.	I

DB random data	Description	Return
DBRandomDistributionGet(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex, string distName, real params[], real empTable[][2])	Gets the cell's distribution, if any assigned. Returns the info on the distribution in the <i>returnParams</i> array. Can use locally or statically declared <i>returnParams</i> (Real <i>returnParams</i>[10]) and <i>empTable</i> (Real <i>empTable</i>[maxEmpiricalrows][2]) or dynamic arrays for <i>returnParams</i>[], <i>empTable</i>[][2]; If using dynamic arrays, no Makearray() needed. You can dispose dynamic arrays after the call if not needed (Local arrays allow use in an Equation block). Case: All database indexes good (ignores <i>distName</i>), return info for a DB cell: 1) Return all info in arrays. Function returns name of distribution or blank if not named. 2) Return distributionIndex of 0 in array if not a random cell. Case: Only databaseIndex is good, rest are zero. Return info for <i>distName</i> named distribution, if it exists: 1) If <i>distName</i> is good name: Return all info for that distribution (useSeed and seedInit will be zero as it is defined for each DB cell). 2) <i>distName</i> doesn't exist: Return string "-1" <i>Note that the returned distributionIndex values are the same as used in the RandomCalculate() function, with these additions:</i> EmpiricalDiscrete 200 EmpiricalStepped 201 EmpiricalInterpolated 202 <i>returnParams</i>[0] = distributionIndex; <i>returnParams</i>[1] = meanDistParam1; <i>returnParams</i>[2] = argDistParam2; <i>returnParams</i>[3] = modeParam3; <i>returnParams</i>[4] = locationParam4; <i>returnParams</i>[5] = lowerLimit; <i>returnParams</i>[6] = upperLimit; <i>returnParams</i>[7] = useSeed; <i>returnParams</i>[8] = seedInit; <i>returnParams</i>[9] = empiricNumRows; <i>empTable</i> contains the empirical distribution array, if any.	S

DB random data	Description	Return
DBRandomDistributionSet(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex, string distributionName, integer distributionIndex, real param1, real param2, real param3, real param4, real lowerLimit, real upperLimit, integer useSeed, integer seedInit, real empTable[][2])	This can be used in several cases: 1) Define or modify a named random distribution without setting a DB cell to it (must use this for empirical distributions): a) Set databaseIndex, other DB indexes should be zero: b) Use distributionIndex to set the distribution type. c) Set remaining params for this distribution. d) Fill empTable if this is an empirical distribution. 2) Set a DB cell to a named random distribution (must use this for empirical distributions): a) Set all DB indexes to point to a DB cell. b) Set distribution name as it is defined in database or via case 1 above. c) Set useSeed and seedInit for this DB cell. Other parameters are ignored as name defines distribution to use. 3) Set a DB cell to an unnamed non-empirical random distribution: a) Set all DB indexes to point to a DB cell. b) Set distribution name to "" (blank string). c) Set distribution index and parameters as in case 1 above. 4) Remove the random distribution from a cell and make it a constant: a) Set all DB indexes to point to a DB cell. b) Set distribution name to "" (blank string). c) Set distribution index to zero. <i>Note that the distributionIndex values are the same as used in the RandomCalculate() function, with these additions:</i> EmpiricalDiscrete 200 EmpiricalStepped 201 EmpiricalInterpolated 202 Returns -1 as error.	I

Sort and search in a database

DB sort/search	Description	Return
DBRecordFind(integer databaseIndex, integer tableIndex, integer notRecordIDFieldIndex, integer startingRecordIndex, integer exactMatch, string findValue)	Finds a record and returns index of record found. If fieldIndex is zero, uses RecordID field for search. startingRecordIndex can be zero or one to start at the first record. To keep searching for more matches, increment the returned record index by one for startingRecordIndex. If exactMatch is FALSE, finds record containing findValue without having to match entire field value. Returns 0 if record not found. Returns negative error if index error. NOTE: Different find functions return different error codes because of legacy concerns.	I

DB sort/search	Description	Return
DBRecordFindMultipleFields(integer databaseIndex, integer tableIndex, integer startingRecordIndex, integer fieldIndex1, string findValue1, integer exactMatch1, integer fieldIndex2, string findValue2, integer exactMatch2, integer fieldIndex3, string findValue3, integer exactMatch3)	Finds a record and returns index of record found. Up to 3 fields can be searched. If only one is searched, make both fieldIndex2, fieldIndex3 zero. If only two are searched, make fieldIndex3 zero. StartingRecordIndex can be zero or one to start at the first record. To keep searching for more matches, increment the returned record index by one for startingRecordIndex. If exactMatch is FALSE, finds record containing findValue without having to match the entire field value. Returns -1 if record not found or index error. NOTE: Different find functions return different error codes because of legacy concerns.	I
DBRecordFindMultipleFieldsArray(integer databaseIndex, integer tableIndex, integer startingRecordIndex, integer fieldIndexArray[], string findValueArray[], integer exactMatchArray[])	Finds a record and returns index of record found. Any number of fields can be searched. Create three arrays that can be local, static or dynamic. For example, to declare local arrays in an Equation block to search 4 fields: <pre>integer fieldIndexArray[4], exactMatchArray[4]; string findValueArray[4];</pre> The arrays can be larger than you need as only nonzero fieldIndexArray members will be searched. The first zero fieldIndex will end the fields searched. StartingRecordIndex can be zero or one to start at the first record. To keep searching for more matches, increment the returned record index by one for startingRecordIndex. If exactMatch is FALSE, finds record containing findValue without having to match the entire field value. Returns -1 if record not found or index error. NOTE: Different find functions return different error codes because of legacy concerns.	I
DBRecordFindNumericalRange(integer databaseIndex, integer tableIndex, integer notRecordIDFieldIndex, integer startingRecordIndex, real lowerDouble, real upperDouble)	Finds a record and returns index of record found where number $\geq$ lowerDouble and $\leq$ upperDouble. To find exactly, make lowerDouble and upperDouble the same. If fieldIndex is zero, uses RecordID field for search. StartingRecordIndex can be zero or one to start at the first record. To keep searching for more matches, increment the returned record index by one for startingRecordIndex. Returns 0 if record not found or index error. NOTE: Different find functions return different error codes because of legacy concerns.	I

DB sort/search	Description	Return
DBRecordFindParentRecordIndex(integer databaseIndex, integer tableIndex, integer childFieldIndex, integer startingRecordIndex, integer findParentIndexValue)	Returns the index of the child record found pointing to <i>startingParentIndex</i> . <i>StartingRecordIndex</i> can be 0 or 1 to start at the first record. To keep searching for more matches, increment the returned record index by 1 for <i>startingRecordIndex</i> . Returns index of found record or 0 for record not found, -1 for indexing error.	I
DBTableSort(integer databaseIndex, integer tableIndex, integer fieldIndex1, integer direction1, integer fieldIndex2, integer direction2, integer fieldIndex3, integer direction3)	Sorts the table using up to three fields and directions. Set the fieldIndexes of the fields you need sorted, and set the other fieldIndexes to zero. For example, if you want to sort only one field, set fieldIndex1 to that field index and set fieldIndex2 and 3 to zero. The direction arguments are TRUE for ascending and FALSE for descending. Returns -1 if there is an error.	I

DB address functions

These functions stuff the database or global array component indexes into a real number. For databases, it includes the database index, table index, field index, and record index. The resulting real number is very useful as an attribute value that can completely describe a database or global array address. The top index limits for DBAddresses are:

- 500 databases
- 10,000 tables
- 1000 fields
- 1,000,000 records

DB address	Description	Return
DBAddressCreate(integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex)	Returns a DBAddress value from the indexes.	R
DBAddressGetAllIndexes(real addressValue, integer returnIndexesArray[4])	Return the indexes from a DBAddress value. ReturnIndexesArray can be declared as a local variable array (can be declared in equation blocks): integer returnIndexesArray[4]; returnIndexesArray[0] returns Database index returnIndexesArray[1] returns Table index returnIndexesArray[2] returns Field index returnIndexesArray[3] returns Record index	
DBAddressGetAsString(real addressValue)	Returns the string representation of a DBAddress value.	S

DB address	Description	Return
DBAddressGetDlg(real theInitVal)	Puts up a dialog for the user to specify the coords of a DBAd- dress. theInitVal is a starting value. BLANK leaves all blank.	R
DBAddressGetDlg2(integer DBIndex, integer tableIndex, integer fieldIndex, integer recordIndex)	Similar to DBAddressGetDlg. Displays a dialog so the user can enter a database address. Note that in addition to entering the DB coordinates as separate values, you can also pass -2 for a DB coordinate, and this will hide that coordinate on the dialog. This allows you to enter just a DB index, for example, or just a DB and table index. Returns a database address.	R
DBAddressGetFrom- String(string dbAd- dressStr)	Returns the real database address when the string argument is of the form "D:T:F:R" where D, T, F, and R are the numerical indexes of the address.	R
DBAddressIncre- mentIndex(real address- Value, integer whichElement, integer incrementValue)	Increments or decrements (negative incrementValue) a DBAd- dress. <i>whichElement</i> : 1: database, 2: table, 2: field, 4: record	R
DBAddressReplaceIn- dex(real addressValue, integer whichElement, integer newValue)	Replaces part of a DBAddress with newValue. <i>whichElement</i> : 1: database, 2: table, 3: field, 4: record	R
DBDatabaseGetIndex- FromAddress(real addressValue)	Returns a database index from the DBAddress value.	I
DBFieldGetIndexFro- mAddress(real address- Value)	Returns a field index from the DBAddress value.	I
DBRecordGetIndex- FromAddress(real addressValue)	Returns a record index from the DBAddress value.	I
DBTableGetIndexFro- mAddress(real address- Value)	Returns a table index from the DBAddress value.	I

Viewing a database

DB viewing	Description	Return
DBDatabaseOpen- Viewer(integer databas- eIndex, string tableName)	Open the table data viewer. If tableName is blank or doesn't exist, open the default database viewer. Returns table index if successful or -1 if error.	I

DB viewing	Description	Return
DBDatabaseOpenViewerToTab(integer databaseIndex, Str255 tableName, Str255 openTabName)	Opens the table data viewer. If tableName is blank or doesn't exist, open the default database viewer. If openTabName exists, opens to that tab. If it is blank or doesn't exist, opens the last clicked tab. Returns table index if successful or -1 if error.	I
DBDatabaseCloseViewer(integer databaseIndex, string tableName)	Close the table data viewer. If tableName is blank or doesn't exist, close the database viewer. Returns table index if successful or -1 if error.	I

Linking and notification

These functions register and unregister blocks from databases, independent of dialog items and their links, so linked blocks can be notified if the data that they are depending on changes in structure or content. This type of linking is different than user linking of parameters and data tables to a database because you don't specify anything to link. If the database changes content (e.g. the value in a cell that you registered changes) you will get a LINKCONTENT message. If the database table that you are linked to changes name or number of fields or rows, you will get a LINKSTRUCTURE message. Please see "Dynamic linking" on page 302, for information on finding out what changed when you get the LINKCONTENT, or LINKSTRUCTURE messages.

 All registration set up by DBBlockRegisterContent() or DBBlockRegisterStructure() is good only for this model session. This disposal of links is done to prevent obsolete links sending unneeded messages that could slow down a simulation.

When the model is closed and reopened, you will have to call these functions again (usually in the OPENMODEL message handler) for all links that you want.

For an overview, see "Registered blocks" on page 106.

DB linking/notify	Description	Return
DBBlockRegisterContent(integer blockNumber, integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex)	This will register the block with the selected part of the database so it will get LINKCONTENT messages when that data is changed. Returns negative code if block not found. Note that for content registration, individual cells as well as whole databases, tables, fields, and records can be registered. If databaseIndex is negative, register all databases. If tableIndex is negative, register all tables. If fieldIndex is negative and recordIndex is negative, register all cells in table. If fieldIndex is negative and recordIndex is good, register whole record. If fieldIndex is good and recordIndex is negative, register whole field.	I

DB linking/notify	Description	Return
DBBlockRegisterStructure(integer blockNumber, integer databaseIndex, integer tableIndex)	This will register the block with the selected part of the database so it will get a LINKSTRUCTURE message when the database or table structure is changed. Returns negative code if block not found. Note for structure registration, only whole databases or whole tables can be registered. If databaseIndex is negative, register all databases for structure changes. If tableIndex is negative, register all tables for structure changes.	I
DBBlockUnregisterContent(integer blockNumber, integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex)	For non-user-linked data, take it out of the registered block list. Return negative code if block not found. If databaseIndex is negative, unregister all databases. If tableIndex is negative, unregister all tables. If fieldIndex is negative and recordIndex is negative, unregister all cells in table. If fieldIndex is negative and recordIndex is good, unregister whole record. If fieldIndex is good and recordIndex is negative, unregister whole field.	I
DBBlockUnregisterStructure(integer blockNumber, integer databaseIndex)	For non-user-linked data, take it out of the registered block list. Return negative code if block not found. If databaseIndex is negative, unregister all databases. If tableIndex is negative, unregister all tables.	I
DBDatatable(integer blockNumber, string datatableName, integer databaseIndex, integer tableIndex, integer showFieldNames)	Link a database table with a data table.	I
DBParameter(integer blockNumber, string dialogItemName, integer databaseIndex, integer tableIndex, integer fieldIndex, integer recordIndex)	Link a parameter with a Database cell.	I

Arrays, pointers, queues, delay, linked list, and string lookup table functions

Dynamic and non-dynamic arrays

The following functions manipulate arrays.

Dynamic arrays are declared as static arrays with their first dimension missing, such as:

```
real, integer, or string array[], array2Dim[][5];
```

- ☞ If you pass a dynamic array as an argument to a user-defined function, you cannot use this local argument name as the argument to the MakeArray or DisposeArray functions, but must instead use the original static declared name.

Dynamic arrays	Description	Return
ArrayDataMove (array y[], integer startIndex, integer rowsToMove, inte- ger targetIndex, integer clearData)	This function moves a specified number of rows of data (rowsToMove) from a specified location (startIndex) in the array to another specified location in the array (targetIndex). If the clearData flag is set to TRUE (1), it will clear out the old data locations in the array. Returns 0 if successful.	I
DisposeArray(array)	Dynamic arrays only. Releases the memory used by a dynamic array. Use this when you have finished using a dynamic array to save memory and model file space. Call this function with the original static declared name of the array (see notes above).	V
FindMinimum(real RealArray, integer ResultArray)	Accepts two dynamic arrays; one real, one integer. It searches the RealArray for the minimum values and returns the minimum value. The ResultArray is filled with the position numbers of the elements that contain the minimum value.	R
FindMinimum- WithThreshold (realArray y, inte- gerArray y2, real threshold)	This function is identical to the FindMinimum function defined in the above, with the addition of the threshold argument. This argument specifies a threshold below which the real values will be ignored. I.e. the minimum value found will always be at the threshold, or above.	R
GetDimension (array)	The size of the first dimension (rows). If <i>array</i> is a dynamic array, this function will return the size of the dynamic dimension. This is useful for determining the maximum subscript (the returned value minus 1) of an array when it is passed to a user-defined function.	I
GetDimensionBy- Name(integer blockNumber, string arrayName)	This function is a variation of the GetDimension function that has two differences. The first is that it takes a block number argument, which specifies which block contains the array, and the second is that it takes the arrayname of the array as an argument rather than the array itself. The combination of these two differences mean that you can call the function from a remote block, query the number of rows in a array, and also pass in a string variable for the arrayname if desired.	I
GetDimensionCol- umns(array)	Returns the number of columns (second dimension in a two dimensional array) in the specified array.	I
GetDimensionCol- umnsByName(inte- ger blockNumber, string arrayName)	Has the same behavior as the getDimensionColumns function, with the exception that it can be called from an outside block, and doesn't have to be called from within the block that contains the array. Note that the arrayName argument is the name of the array as a string, not the array name itself. This also means that the function can be called with a string variable as the second argument, and not a hard coded array name.	I
MakeArray(array, integer i)	Dynamic arrays only. Allocates a dynamic array. <i>i</i> is the desired value of the first (blank) dimension. MakeArray does not clear or disturb data already in the array and can be used to make an array larger or smaller. If you need to initialize an array to 0 or BLANK, this must be done by the block's code. Call this function with the original static declared name of the array (see notes above).	V

Dynamic arrays	Description	Return
MakeArray2(integer blockNumber, string arrayName, integer dim)	Has the same behavior as the MakeArray function, with the exception that it can be called from an outside block, and doesn't have to be called from within the block that contains the array. Note that the arrayName argument is the name of the array as a string, not the array name itself. This allows resizing of dynamic arrays passed in to functions <i>as a string name</i> .	V
SortArray(array, integer numRows, integer keyColumn, integer increase, integer sortStringAsNumbers)	Sorts the array up to <i>numRows</i> . Uses the <i>keyColumn</i> as the sorting column. If <i>increase</i> is TRUE, sorts in ascending order (note that, for purposes of sorting, NoValues are considered larger than any number). If this is a string array and <i>sortStringAsNumbers</i> is TRUE, sorts strings as numbers in <i>keyColumn</i> . This function works with any kind of array, including data tables and text tables.	V

Passing arrays

These functions let you pass dynamic arrays through connectors or global variables. This is discussed in more detail in “Passing arrays” on page 98. Also see the “Passing Arrays” example (Windows: PassArray.mox in the ModLTips folder; Mac OS: Passing Arrays in the ModL Tips folder).

Passing arrays	Description	Return
GetPassedArray(real realVar, array)	Gives access to array values from the input connector or real number <i>realVar</i> that holds the location of a passed array. <i>array</i> must be declared in this block as a dynamic array of the same type as the passed array. After this function is called, <i>array</i> can be used to access and change the values in the passed array. This function returns TRUE if <i>realVar</i> is a passed array from another block and returns FALSE if it is not a passed array.	I
PassArray(array)	Returns the real value that represents the location of a dynamic array declared in this block. This is used to pass the array to an output connector or real variable (such as a global variable). Do not call PassArray to pass an array that has been received by GetPassedArray; see “Passing arrays” on page 98, for more information.	R

Pointer functions

Dynamic array pointer functions allow you to copy the contents of dynamic arrays into an independent data structure that can be created, used by other blocks if desired, and disposed of by any block. You can create a very large number of independent pointers, if desired. They are also useful for creating complex structures of pointers.

 Using a disposed pointer will cause a crash (uses malloc() and free()).

Pointers	Description	Return
PointerFromDynamicArray(dynamicArray)	Copies data from any type of dynamic array to a new pointer created by “malloc()” and returns the pointer. If you pass the pointer to an external DLL, the first integer in the pointer's data is the size of the pointer's data in bytes. Including the size data, the pointer is actually that size plus 4.	I
PointerDispose(integer pointer)	Frees the memory used by the pointer using “free()”. The pointer is now invalid and can cause a crash if used.	V
PointerToDynamicArray(integer pointer, dynamicArray)	Takes a pointer created by PointerFromDynamicArray() and copies it to a dynamic array, so ModL code can access it. This action writes over any old data in that dynamic array. The dynamic array must be the same type as the array in the pointer. The pointer must still be disposed of by PointerDispose().	V

Global arrays

The Global Array functions define and manage global arrays (see “ModL function overview” on page 216). All of the functions except the GAGetxxx functions will return a negative number if they fail. The following numbers have standard meanings:

Value	Meaning
-1	No arrays defined, or array not found
-2	Array index invalid, or array not defined
-3	Row or column reference out of range
-4	Incorrect array type
-5	Name is blank, or too long (> 15 characters)
-7	Wrong type of array
-8	Array length doesn't match global array length

The type of a global array can be defined by the following constants:

Constant	Value
GAReal	1
GAIInteger	2
GAStrng	3
GAStrng15	4
GAStrng31	5

Constant	Value
GAStrng63	6
GAStrng127	7

Global Arrays	Description	Return
DBTabletoGA(integer databaseIndex, integer tableIndex, integer GAIndex)	Copies data from the specified Database table to the specified Global Array. The function makes its best attempt to copy the data over. If the DB table defines multiple fields in different formats, not all the data may be copyable.	I
GABlockRegisterContent(integer blockNumber, integer GAIndex, integer row, integer col)	This function is used in conjunction with the Dynamic Linking functionality in version 7. This function will set up a registration entry for the specified Global Array, such that whenever there is a change in the Global Array, the block will receive ContentChanged messages informing it of the change. This works similarly to the way a dialog item linked to the Global Array would work, except that by calling this function you can establish this kind of link through code without a specific dialog item involved. If you specify the row and column, then the message will only be sent when the specific cell is affected by the change. (Specify -1 if you want the entire array.) See the GABlockRegisterStructure() function below to register to receive structure changes.	I
GABlockRegisterStructure(integer blockNumber, integer GAIndex, integer row, integer col)	This function is similar to the GABlockRegisterContent function, except that it registers to receive StructureChanged messages when the structure of the GA changes, not when the content changes.	I
GABlockUnregisterContent(integer blockNumber, integer GAIndex, integer row, integer col)	Unregisters the specified GA from the Block's dynamic link registry. See GABlockRegister above for information about what it means for a Global Array to be listed in the registry.	I
GABlockUnregisterStructure(integer blockNumber, integer GAIndex, integer row, integer col)	Unregisters the specified GA from the Block's dynamic link registry. See GABlockRegister above for information about what it means for a Global Array to be listed in the registry.	I
GAClipboard (integer arrayIndex)	Copies the array with the specified index into the clipboard, where it will be inserted into the selected model with the next paste.	V
GACopyArray(integer arrayIndex, string newName)	Creates a new Global Array that is a duplicate of the array specified by arrayIndex, with the name <i>newName</i> . The return value of the function is the arrayindex of the new Global Array.	I
GACreate(string name, integer type, integer columns)	This function creates an array with the specified <i>name</i> , and <i>type</i> . Arrays are created with zero rows. Returns the index number.	I

Global Arrays	Description	Return
GACreateQuick (string name, integer type, integer columns)	This function behaves the same way as the GACreate function, with the exception that it will not check the name to see if a global array already exists with that name. The only case where you would want to use this function is where you are creating a large number of GA's, speed is of the essence, and you are sure that there will be no duplication of names. If you do create an array with the same name as an existing array, referencing that array by name will only access the older array.	I
GACreateRandom (integer type, integer columns)	Creates a Global Array with a random name. Useful for quick creation of global arrays in cases where you need to create many arrays quickly.	I
GADDataTable (integer blockNumber, string dtName, integer arrayIndex)	Associates a Global Array with a dialog data table in the same way the DynamicDatatable function associates a dynamic array with one. See the DynamicDatatable() function description for more information.	I
GADeleteRow (integer arrayIndex, integer row)	Deletes a row in the specified Global Array. This will move down the data on rows after the specified row.	I
GADispose (string name)	Disposes the named Array.	I
GADisposeByIndex (integer arrayIndex)	Disposes the Global Array referenced by <i>arrayIndex</i> .	I
GAExport (string pathName, string userPrompt, string format, integer GAIndex, integer rows, integer columns)	Exports data from the Global Array specified by GAIndex directly into a text file. See the Export() function for information about the other arguments.	I
GAFindStringAny (integer arrayIndex, string findString, integer column, integer numRows, integer numChars, integer caseSensitivity)	This function searches a specific string Global Array, referenced by <i>arrayIndex</i> , for the first string that matches the <i>findString</i> . The return value is the index of the array element that contains the first string found. A -1 will be returned if the string is not found.	I
GAGetArray (integer arrayIndex, integer row, array y)	Sets the contents of the array <i>y</i> to the values in the Global Array with the specified index. This will copy the contents of the specified <i>row</i> of the Global Array into the array <i>y</i> . (You need to make sure that the number of elements in the dynamic array match the number of columns in the <i>row</i> of the global array.)	I
GAGetColumns (string name)	Returns the number of columns defined for the named array.	I

Global Arrays	Description	Return
GAGetColumnsByIndex (integer array-Index)	Returns the number of columns defined for the Global Array referenced by <i>arrayIndex</i> .	I
GAGetIndex(string name)	Returns the Index value of the named array.	I
GAGetInfo(integer ArrayIndex, integer Which)	This Global Array function returns the value of some of the Global Array flags for the specified array. Values for <i>Which</i> are: 0:non saving, 1:initializing.	I
GAGetInit-Value(integer array-Index)	Gets the initialization value for the global array. This value is set by GAGetInitValue(), and is used during array initialization.	R
GAGetInteger(integer arrayIndex, integer row, integer column)	Returns the integer value stored at the specific row and column of the array with the specified index.	I
GAGetLong(integer arrayIndex, integer row, integer column)	This function is also known as GAGetInteger. See GAGetInteger for description.	I
GAGetName(integer arrayIndex)	Returns the Name of the array with the specified index.	S
GAGetReal(integer arrayIndex, integer row, integer column)	Returns the real value stored at the specific row and column of the array with the specified index.	R
GAGetRows(string name)	Returns the number of rows defined for the named array.	I
GAGetRowsByIndex (integer arrayIndex)	Returns the number of rows defined for the Global Array referenced by <i>arrayIndex</i> .	I
GAGetString (integer arrayIndex, integer row, integer column)	Returns the string value stored at the specific row and column of the array with the specified index.	S
GAGetString15(integer arrayIndex, integer row, integer column)	Returns the string value stored at the specific row and column of the array with the specified index.	S
GAGetString31(integer arrayIndex, integer row, integer column)	Returns the string value stored at the specific row and column of the array with the specified index.	S

Global Arrays	Description	Return
GAGetString63(integer arrayIndex, integer row, integer column)	Supports the string 63 type, otherwise the same as the GAGetString() function.	S
GAGetType(string name)	Returns the type of the named Array. See table above for type values.	I
GAGetTypeByIndex (integer arrayIndex)	Returns the type of the Global Array referenced by <i>arrayIndex</i> .	I
GAImport(string pathName, string userPrompt, string format, integer GAIIndex)	Imports data from a text file directly into the Global Array specified by GAIIndex. See the Import() function for information about the other arguments.	I
GAInitializing(integer ArrayIndex, integer Initializing)	Sets the initializing flag for the specified global array. The flag determines if the array is automatically initialized during initsim of a model run or not. The Initializing flag takes the following values: 0: don't initialize (default), 1: initialize to 0, 2: initialize to blank (real numbers only.), 3: initialize to specified value. See GASetInitValue(), below.	I
GAInsertRow(integer arrayIndex, integer row)	Inserts a row in the specified Global Array. This will cause all the data on rows after the specified row to be moved up a row.	I
GALastUsedIndex()	Returns the GAIIndex of the last defined Global Array. This can be used to loop through all the Global Arrays in a model.	I
GAMultisim(integer arrayIndex, integer multisim)	Sets a flag on the Global Array that determines how the initialization deals with multiple simulation runs. 0: initialize at beginning of each run, 1: initialize at beginning of first run of a multiple run only.	I
GANonSaving(integer arrayIndex, integer nonSavingArray)	This function flags the specific array as a non saving array. The array will not be written out to the model file when the file is closed. (By default Global arrays are “saving” arrays.)	I
GAParameter(integer blockNumber, StringdialogItem-Name, integer array-Index, integer colIndex, integer rowIndex)	Link a parameter with a Global Array cell.	I
GAPopupMenu (integer arrayIndex, string name, integer rows, integer init, integer flying)	This function copies the strings in the specified Global array into the named Popup menu. This is a utility that allows quick construction of a popup menu. The flying argument is true if the menu is being created “on the fly,” as opposed to adding the array to the existing menu. See the code that controls the <i>Animation</i> tab of any Item library block as an example.	I

Global Arrays	Description	Return
GAPtr (integer arrayIndex)	Returns the memory pointer to the data associated with a particular global array. (For passing data to dll's only, and it should be called only immediately before the DLL call, as memory can move, making the pointer invalid.)	I
GAResize(string name, integer rows)	Changes the number of rows defined for a global array.	I
GAResizeByIndex (integer arrayIndex, integer size)	Changes the number of rows defined for the Global Array referenced by <i>arrayIndex</i> .	I
GASearch(integer GAIndex, integer lValue, real rValue, string sValue, integer whichCol, integer startIndex)	Searches a Global array for the occurrence of the specified value and returns the first index where found. This function can be used to search any type of Global Array. It will search for the value lValue, rValue, or sValue, as appropriate, based on the type of the Global Array and will search in the column specified by the whichCol parameter. The startIndex parameter specifies which row to begin searching on. For second and subsequent searches, just pass the last value returned by the function <i>plus one</i> as the startIndex parameter. You should end the search when the function returns a negative 1, as this will mean that the desired element was not found.	I
GASearch-Count(integer GAIndex, integer lValue, real rValue, string sValue, integer whichCol, integer startIndex)	Returns the number of occurrences in a Global array of the specified value. This function can be used to search any type of Global Array. It will search for the value lValue, rValue, or sValue, as appropriate, based on the type of the Global Array and will search in the column specified by the whichCol parameter. The startIndex parameter specifies which row to begin searching on. You should end the search when the function returns a negative 1, as this will mean that the desired element was not found.	I
GASetArray(integer arrayIndex, integer row, array y)	Sets the contents of the Global Array with the specified index to the values in the array y. This will copy the contents of the array y into the specified row of the Global Array. (You need to make sure that the number of elements in the dynamic array match the number of columns in the row of the global array.)	I
GASetInit-Value(integer array-Index, real value)	Sets the initialization value for the specified global array. This value is used during array initialization.	I
GASetInteger(integer value, integer arrayIndex, integer row, integer column)	Sets the integer value at the specific row and column of the array with the specified index.	I
GASetLong(integer value, integer array-Index, integer row, integer column)	This function is also known as GASetInteger. See GASetInteger for description.	I

Global Arrays	Description	Return
GASetReal(real value, integer array-Index, integer row, integer column)	Sets the real value at the specific row and column of the array with the specified index.	I
GASetString(string value, integer array-Index, integer row, integer column)	Sets the string value at the specific row and column of the array with the specified index.	I
GASetstring15(string value, integer arrayIndex, integer row, integer column)	Sets the str15 value at the specific row and column of the array with the specified index.	I
GASetstring31(string value, integer arrayIndex, integer row, integer column)	Sets the str31 value at the specific row and column of the array with the specified index.	I
GASetString63(string value, integer arrayIndex, integer row, integer column)	Supports the string 63 type, otherwise the same as the GASetString() function.	I
GASort(integer arrayIndex, integer numRows, integer keyColumn, integer increase, integer sortstringAsNumbers)	Sorts the Array with the specified index. Sorts the array up to <i>numRows</i> . Uses the <i>keyColumn</i> as the sorting column. If <i>increase</i> is TRUE, sorts in ascending order (note that, for purposes of sorting, NoValues are considered larger than any number). If this is a string array and <i>sortStringAsNumbers</i> is TRUE, sorts strings as numbers in <i>keyColumn</i> .	I
GAtoDBTable(integer GAIndex, integer databaseIndex, integer tableIndex)	Copies data from the specified GA to the specified Database table. The function makes its best attempt to copy the data over. If the DB table defines multiple fields in different formats, not all the data may be copyable.	I

Linked lists

Linked lists are queue-like, multiple type structures that maintain internal pointers between the different elements. This speeds the moving around of elements (sorting) within the list. Each structure element can simultaneously contain any number of integer, real, Str15, Str31, and Str255 data types, so complex sorted structures can be created. They will be slightly slower to access than their linear equivalent if used as a simple queue (FIFO, or LIFO) and faster than their linear equivalent if their internal sorting functionality is taken advantage of, as in a Queue block (Item library). See that block's code for a good example of using linked lists to sort.

These functions create and manipulate linked lists which are referred to by an index and are associated and stored with a block. Because these functions have a global block number as an argument,

linked lists associated with a specific block can also be accessed globally, from any other block in the model.

The normal sequence for working with linked lists is:

```
ListCreate(...); // create the list structures
ListCreateElement(...); // creates a new empty element
ListSetxxx(...); // sets a field in the element
ListSetxxx(...); // sets another field in the element
... // set the rest of the fields in the element
ListAddElement(...); // adds the new element to the list
... // manipulate the list, adding and deleting elements
... // get elements from the list to use in a calculation
ListDispose(...); // we are done with the list
```

Linked lists	Description	Return
ListAddElement(integer blockNumber, integer listIndex, integer where)	Adds an element previously created with ListCreateElement to the specified queue. <i>where</i>:-2 sorted by preset sortType and field index value <i>where</i>:-1 the front of the list Otherwise, <i>where</i> is an index value and the item will be added after the specified index item. Returns zero for success.	I
ListAddString63s(integer blockN, integer listIndex, integer string63Count)	This function should be called right after ListCreate, if you wish your linked list to contain String63s. It has the same effect as specifying, for example, n String15s in the ListCreate function, it defines the number of string63s that will be present in each element of the Linked List.	I
ListCopyElement(integer blockN, integer listIndex, integer fromIndex, integer targetBlockN, integer targetListIndex, integer targetIndex)	Copies an element from one linked list to another. The first three parameters specify the element in the first list, the next two specify the target list, and the last one specifies where in the new list to copy the element. As with any linked list function that adds an element, you can specify a -2 to mean that the new element should be added in its sorted order.	I

Linked lists	Description	Return
ListCreate(integer blockNumber, integer longCount, integer realCount, integer str15Count, integer str31Count, integer strCount, integer sortType, integer fieldIndex)	Creates a new list with the specified attributes. LongCount, realCount, etc are counts of the number of fields of each of the specified types each list element contains. SortType and fieldIndex determine which field is to be used as the sorting field for the list. sortType:0 don't sort sortType:1 real field is key sortType:2 integer field is key sortType:3 str255 field is key sortType:4 str15 field is key sortType:5 str31 field is key FieldIndex is used to determine which index of the specified type is the key field. Zero is the first field.	I
ListCreateElement(integer blockNumber, integer listIndex)	Creates a new empty element for a list. The normal sequence is ListCreateElement(...); // creates a new empty element ListSetxxx(...); // sets a field in the element ListSetxxx(...); // sets another field in the element ... // set the rest of the fields in the element ListAddElement(...); // adds the new element to the list	I
ListDeleteElement(integer blockNumber, integer listIndex, integer indexToDelete)	Deletes the specified element. Zero is the first element.	I
ListDispose(integer blockNumber, integer listIndex)	Disposes of the specified list and recovers its memory.	I
ListDisposeAll(integer blockNumber)	Disposes all linked lists in a block. Returns TRUE if the block doesn't exist or the lists have already been disposed.	I
ListElementMinMax(integer blockNumber, integer listIndex, integer compareType, integer compareIndex, integer max)	This function searches the list for the maximum or minimum value of the specified value. If the Max flag is true, it will return the index of the element that contains the maximum value of that entry, otherwise it will return the minimum. (Currently just integer and real are implemented. string comparisons are not yet implemented.)	I
ListGetCount(integer blockN)	Returns the count of the number of linked lists the block has defined. Please note that in a similar way to the way GetNumBlocks works for the model worksheet, the number returned by this function can contain 'empty slots'. An 'empty slot' is defined as a list index that specifies a list that has been disposed, or is otherwise not defined. This function can be used to execute a loop that looks at all the linked lists in a block, but you should check each list to confirm that it exists. Because of this aspect of how this functions, you should not call ListGetCount, and assume that the returned value is exactly the number of lists the block supports.	I

364 | ModL Functions

Arrays, pointers, queues, delay, linked list, and string lookup table functions

Linked lists	Description	Return
ListGetDouble(integer blockNumber, integer listIndex, integer elementIndex, integer fieldIndex)	Returns the real (double) value at that element and field index. If elementIndex is passed in as a value less than zero, it refers to the current newly created, but not yet added, item. If elementIndex is zero or greater it is used as an index value into the specified list.	R
ListGetElements(integer blockNumber, integer listIndex)	Returns the number of elements in the specified list.	I
ListGetIndex(integer blockN, string name)	Gets the index of a linked list by its name. Linked lists do not automatically have names. If the name specified is not found, the function will return a negative value as an error code.	I
ListGetInfo(integer blockNumber, integer listIndex, integer infoType)	Returns the specified info about the specified Linked List. InfoType takes the following values: 1: returns the number of real values in the specified list 2: returns the number of integer values in the specified list 3: returns the number of string values in the specified list 4: returns the number of string15 values in the specified list 5: returns the number of string31 values in the specified list 6: returns the number of string63 values in the specified list 10: returns TRUE if this list exists, FALSE if it doesn't	I
ListGetLong(integer blockNumber, integer listIndex, integer elementIndex, integer fieldIndex)	Returns the integer (integer) value at that element and field index. If elementIndex is passed in as a value less than zero, it refers to the current newly created, but not yet added, item. If elementIndex is zero or greater it is used as an index value into the specified list.	I
ListGetName(integer blockN, integer listIndex)	Returns the name of the linked list. List names are new in version 7. Lists do not have a name by default, and will not have a name, until one has been set with the ListSetName function.	S
ListGetString(integer blockNumber, integer listIndex, integer elementIndex, integer stringType, integer fieldIndex)	Returns the string value at that element and field index. If elementIndex is passed in as a value less than zero, it refers to the current newly created, but not yet added, item. If elementIndex is zero or greater it is used as an index value into the specified list. StringType takes the following values: 3: string (str255) field 4: str15 field 5: str31 field	S
ListLastElementIndex(integer blockN, integer listIndex)	Return the index value of the last item added to the specified list (not the end of the list, the last item actually added).	I

Linked lists	Description	Return
ListLocked (integer blockN, integer list-Index, integer locked)	If Locked is true, this function call marks the specified Linked List as locked. This has the effect of making that List not be disposed if ListDisposeAll is called. If the function ListDispose is called explicitly on this list, it will still be disposed, this function only prevents accidental disposal of the list through the ListDisposeAll call. Returns a zero of the call succeeds. Returns a negative error code value if the function fails.	I
ListSearch(integer blockN, integer list-Index, integer searchType, integer searchIndex, integer lVal, real rVal, string sVal, integer startIndex)	Searches the list for the specified value. Search type determines which type to search for, and also which value string is used. StartIndex specifies where in the list to start searching. This function returns the index number of the first list element that matches the search criteria. If you want to search for multiple elements in the same list, just call the function multiple times, using the last index found <i>plus one</i> as the 'startIndex' parameter of the next search. You should end the search when the function returns a negative 1, as this will mean that the desired element was not found.	I
ListSearch-Count(integer blockN, integer list-Index, integer searchType, integer searchIndex, integer lVal, real rVal, string sVal, integer startIndex)	Similar to the ListSearch function, except that this function returns the number of occurrences of the specified value in the list.	I
ListSearchCount-Longs(integer blockN, integer list-Index, integer Array y, integer startIndex)	Functions similarly to the ListSearchCount function, except that the integer Array Y argument is similar to the ListSearchLongs function, below. (I.e. this function will count the number of elements in the list that contain matches for all the integer elements in the integer Array.)	I
List-SearchLongs(integer blockN, integer listIndex, integer Array y, integer startIndex)	Similar to the ListSearch function, except that the type to be searched for is longs, and the function will search for more than one integer at a time within a single element of the list. The integer Array Y argument is a two column array by any number of rows long. These longs are in pair of index followed by search value. This allows you to search a linked list for more than one integer condition at a time. The search will match only list elements where all the longs in the array match.	I
ListSetDouble(integer blockNumber, integer listIndex, integer elementIndex, integer fieldIndex, real value)	Sets the real (double) value at that element and field index. If elementIndex is passed in as a value less than zero, it refers to the current newly created, but not yet added, item. If elementIndex is zero or greater it is used as an index value into the specified list.	I

Linked lists	Description	Return
ListSetLong(integer blockNumber, integer listIndex, integer elementIndex, integer longIndex, integer value)	Sets the integer (long) value at that element and field index. If elementIndex is passed in as a value less than zero, it refers to the current newly created, but not yet added, item. If elementIndex is zero or greater it is used as an index value into the specified list.	I
ListSetName(integer blockN, integer listIndex, string name)	Sets the name of the specified linked list to the name defined by the 'name' parameter.	I
ListSetSort(integer blockNumber, integer listIndex, integer sortType, integer fieldIndex)	ListSetSort allows you to change the sort criteria for the list. Sort-Type and fieldIndex are defined as described above in the ListCreate function. The list will be resorted by this call.	I
ListSetSort2(integer blockN, integer listIndex, integer sortType, integer sortIndex, integer sortType2, integer sortIndex2, integer sortType3, integer sortIndex3)	Sets the sorting criteria for the specified list. This function enhances the existing ListSetSort() function in that there are now multiple sorting criteria. I.e. there is a secondary, and tertiary sort.	I
ListSetString(integer blockNumber, integer listIndex, integer elementIndex, integer stringType, integer stringIndex, string newString)	Sets the string newString at that element and field index. If elementIndex is passed in as a value less than zero, it refers to the current newly created, but not yet added, item. If elementIndex is zero or greater it is used as an index value into the specified list. StringType takes the following values: 3: string (str255) field 4: str15 field 5: str31 field	V

String lookup table functions

String lookup tables are data structures that allow you to associate a string with an integer value. This is used to lookup a string value in the table whenever needed. An example might be associating the strings "Red", "Green" and "Blue" with the values 1, 2 and 3. This allows you to display the string values in a dialog box or popup menu when internally you are storing a numeric value. Because the String Lookup blocks use hash tables internally they will access and convert the integer values to the strings quickly.

These functions are used to implement the string attribute behavior in the Item library.

Note that string lookup table information is stored in the model, but it is not saved when the model is closed. This has a couple of implications. First, if you copy a block that has information that is based on a string lookup from one model to another one where the string lookup is not defined, the results will be unpredictable. The correct work around for this issue would be to make sure that the string lookup is defined in the target model before the block is copied. The sec-

and implication is that, as the string lookup information is not saved when the model is closed, you will need to recreate any necessary string lookups in the model when it is opened. (In the case of the String Attributes, this is done in the executive block during the OpenModel message handler.) Also, see “Strings” on page 369

String lookup	Description	Return
SLClear(string slName)	Clears the specified lookup of all strings, and sets it to not be a string lookup, until SLSet is called on it again.	I
SLCreate(string slName)	Creates a string lookup table with the specified name.	I
SLDelete(string slName)	Deletes a string lookup table with the specified name.	I
SLFlagGet(string slName, integer which)	Returns one of the twenty user defined flag values for the specified string lookup. Each string lookup has twenty flags associated with it. ‘Which’ should take a value from 0 to 19.	I
SLFlagRealGet(string slName, integer which)	Returns one of the twenty real user defined flag values for the specified string lookup. Each string lookup has twenty real flags associated with it. ‘Which’ should take a value from 0 to 19.	R
SLFlagRealSet(string slName, integer which, integer tableAttribute)	Sets one of the twenty real user defined flag values for the specified string lookup. Each string lookup has twenty real flags associated with it. ‘Which’ should take a value from 0 to 19.	I
SLFlagSet(string slName, integer which, integer tableAttribute)	Sets one of the twenty user defined flag values for the specified string lookup. Each string lookup has twenty flags associated with it. ‘Which’ should take a value from 0 to 19. Note that these flags are stored internally as a single byte of data, which means that you can only store values from zero to 127 in the flag. This is normally intended to store Boolean (True/False) values, but you can store values up to 127 if you wish. (Negative values will not be stored.)	I
SLGetCount()	Returns the count of the number of string Lookups defined in a model.	I
SLGetCountStrings(string slName)	Returns the count of the number of strings that are associated with the specified lookup.	I
SLIs(string slName)	Returns a true value if the specified name is a string lookup.	I
SLPopupMenu(string slName, dialog-Item, init, flying)	Either fills a popup menu with the lookup string values, or creates a flying popup menu at the location of the last click, based on whether or not the flying flag is set.	I
SLSort(string slName)	Sorts the strings for the specified lookup alphabetically.	I

String lookup	Description	Return
SLStringAppend(string slName, string stringVal)	Sets a string value for the specified lookup. By default the first string will have value one, the second one value two, and so on. This can be revised by changing the order of the list by either calling SLSort, to sort the list alphabetically, or calling SLRemove to remove a string from the list. The first time this function is called, It will create a string lookup table with the name slName if that string lookup table does not already exist. This is an alternative to calling SLCreate.	I
SLStringGet(string slName, integer index)	Returns the string value of the specified index on a string lookup.	S
SLStringGetIndex(string slName, string string)	This function is the inverse of the SLStringGet, it returns the index value of a string on a string lookup.	I
SLStringInsert(string slName, stringstringVal, integer index)	Inserts the specified string at the specified index.	I
SLStringRemove(string slName, string string)	Removes the specified string from the list of strings associated with the specified lookup.	I

Queues

These functions store queues in a single-dimensional real, dynamic array, that is, an array that is declared with no dimension, such as **real a[]** (see the section above on dynamic arrays). You do not need to use the MakeArray function before calling QueInit. When you have finished with a queue, you should recover the memory it occupies with the

DisposeArray function; this is often done in the EndSim message handler.

In queues, the first member is numbered “0”, the next member “1”, and so on. This means that in an array of n members, the last member is numbered “n-1”. n and i are integers, and x is a real value.

Queues cannot be directly accessed via the array and an index. You must use the following functions to access elements within the queue. Also, see “Linked lists” on page 361.

Queues	Description	Return
GetFront(real array[])	Removes and returns item 0 from the front of the queue. If the queue is empty, NoValue is returned.	R
GetRear(real array[])	Removes and returns item n-1 from the rear of the queue. If the queue is empty, NoValue is returned.	R
PutFront(real array[], real x)	Adds x to the front of the queue.	V
PutRear(real array[], real x)	Adds x to the rear of the queue.	V

Queues	Description	Return
QueGetN(real array[], integer i)	Removes and returns the <i>i</i> th member of a queue. If the <i>i</i> th member does not exist, noValue is returned. This compacts the queue after the <i>i</i> th member is removed so that the <i>i</i> +1st member becomes the new <i>i</i> th member.	R
QueInit(real array[])	Allocates and initializes the array for the queuing functions. Call this procedure in the InitSim message handler for each queue.	V
QueLength(real array[])	Length of the queue. If the queue is empty, the function returns 0.	I
QueSetAlloc (integer alloc, integer realloc)	Specifies the allocation and reallocation constants for the queueing functions. This function allows the user to control how much memory is allocated by the queueing functions to initially allocate memory for item storage, and how much additional to add each time they need to be resized bigger. The default values are 200 for alloc, and 500 for realloc.	V
QueLookN(real array[], integer i)	Value of the <i>i</i> th member of a queue without changing the queue order. If the <i>i</i> th member does not exist, noValue is returned.	R
QueSetN(real array[], integer i, real x)	Sets the value of the <i>i</i> th member of a queue to <i>x</i> without changing the queue order. If <i>i</i> is greater than the length of the queue, an error message informs you and aborts the operation.	V

Delay lines

These functions store delay lines in a single-dimensional real, dynamic array, that is, an array that is declared with no dimension, such as **real a[]** (see the section above on dynamic arrays).

Delay lines are used in continuous simulations to delay values by a constant amount of time. They are like a pipe with values flowing in one end and out the other; the delay time is analogous to the length of the pipe.

Although the delay line functions store delay lines in dynamic arrays, you must not use the Make-Array function to allocate these arrays. Array allocation is handled automatically by the delay line functions. When you have finished with a delay line, you should recover the memory it occupies with the DisposeArray function.

Delay lines	Description	Return
Delay(real array[], real x)	Inserts a new value <i>x</i> , and returns a delayed value from a delay line. The returned value will be the value inserted DelayTime ago.	R
DelayInit(real array[], real DelayTime)	Initializes a dynamic array delay line to <i>DelayTime</i> . Call this procedure in the InitSim message handler for each delay line.	V

Miscellaneous functions

Strings

These functions allow you to change and parse a string into whatever component parts you want. Note that, in ModL, the “+” operator acts as a string concatenation operator. The functions that

require a character position indicate the first character in a string as position 0. In these functions, the arguments *s*, *findString*, and *replaceString* are strings.

Please see “String lookup table functions” on page 366

Strings	Description	Return
ArrayLabel- Parse(integer item, string array)	Parses an array of semicolon delimited strings and treats it as a single long string so you can get the <i>n</i> th label. This is the same parsing that <i>ExtendSim</i> does internally with both the data table labels, and the popup menu strings.	S
FormatString(integer numArgs, string formatString, string value1, string value2, string value3, string value4, string value5, string value6, string value7, string value8)	This function creates formatted strings using string value arguments. The <i>formatString</i> argument is used to define the format of the returned output string. The definition of the format string is based on the C function <i>sprintf</i> . Please refer to a standard C reference for more information on how to define the format string. <i>NumArgs</i> defines the number of value arguments that contain meaningful values.	S
FormatStringReal (integer numArgs, string formatString, real value1, real value2, real value3, real value4, real value5, real value6, real value7, real value8)	This function creates formatted strings using real value arguments. The <i>formatString</i> argument is used to define the format of the returned output string. The definition of the format string is based on the C function <i>sprintf</i> . Please refer to a standard C reference for more information on how to define the format string.	S
NumToFormat(real <i>x</i> , integer maxchar, integer sigFigs, integer format)	Returns the number formatted as a string. <i>X</i> is the input number, <i>maxchar</i> is the maximum number of characters in the returned string, <i>sigFigs</i> is the number of significant figures desired. <i>Format</i> is 0 for general, 1 for currency, 2 for integer, 3 for scientific notation. NOTE: if -1 is used for maxchar, format is ignored and no scientific notation is used even if the value is very small or large. This special output format is needed to be compatible with Proof Animation.	S
RandomString(integer <i>n</i>)	Returns a randomly generated string of <i>n</i> upper case alphabetic characters.	S
RealToStr(real value, integer sig- Figs)	Converts the <i>value</i> , rounded to <i>sigFigs</i> significant figures, to a string.	S

Strings	Description	Return
RealToStrShortest(real value, integer sigFigs, integer alwaysPadZeros)	Converts a double or real variable to a string value, like the RealToStr function. The alwaysPadZeros argument specifies if you want the zero trimming behavior. What this does is to remove any trailing zeros that may have appeared in the string from the sigfigs being higher than the number of actual digits in the resulting value. If you call this function with alwaysPadZeros TRUE, it will behave exactly the same as RealToString.	S
StrFind(string s, string findString, integer caseSens, integer diacSens)	Character position of <i>findString</i> within string <i>s</i> . The first position of a string is 0, so if the <i>findString</i> is not found, StrFind returns -1. If <i>caseSens</i> is TRUE, case is considered in the search. If <i>diacSens</i> is TRUE, diacritical marks are considered in the search.	I
StrGetAscii(string s)	First character of <i>s</i> as an integer value corresponding to the ASCII value.	I
StringCase(string str, integer lowerCase)	Returns <i>str</i> converted to lower case if <i>lowerCase</i> is TRUE, upper case if <i>lowerCase</i> is FALSE.	S
StringCompare(string s1, string s2, integer caseSens, integer diacritical)	Returns -1 if <i>s1</i> < <i>s2</i> , 0 if <i>s1</i> == <i>s2</i> , 1 if <i>s1</i> > <i>s2</i> . If <i>caseSens</i> is TRUE, uses case. If <i>diacritical</i> is TRUE, uses diacritical marks (ä, é, ö, etc.).	I
StringTrim (string input, integer which)	This function trims leading and trailing blank spaces off the input string. Spaces, CR, LF, *and TAB characters will be trimmed. The resulting string is returned. The argument, “which,” takes the following values: 0-both leading and trailing blanks trimmed. 1-leading blanks trimmed. 2-trailing blanks trimmed.	S
StrLen(string s)	Number of characters in the string <i>s</i> .	I
StrPart(string s, integer start, integer i)	Substring of string <i>s</i> , starting at character position <i>start</i> , <i>i</i> characters long. Note that string length is 255 character maximum.	S
StrPutAscii(integer i)	String of length 1 corresponding to the ASCII value of <i>i</i> . This is useful for putting non-printing control characters into a string.	S
StrReplace(string s, integer start, integer i, string replaceString)	The substring of string <i>s</i> starting at start of length <i>i</i> is replaced with <i>replaceString</i> .	S
StrToReal(string s)	Real value converted from string <i>s</i> , ending with the first space or letter found that is not part of the number. If <i>s</i> does not represent a number, the function returns a NoValue.	R

Strings	Description	Return
TextWidth(string theString, integer font, integer face, integer size)	This function returns the width the specified string would draw at in pixels. If font, face, and size are all zero the function will use the default values for the animation text functions.	I
StrPartDynamic(stringarray, integer start, integer numChars)	Same as the strPart function, on a dynamicText dialog item. See “Dynamic text items” on page 304.	S

Attributes

Use these functions in discrete event blocks to work with attribute strings. Attribute strings are formatted as: **AttrName1=val1;AttrName2=val2;...**

 Obsolete. These function are provided for backwards compatibility with version 3.x. New blocks built in version 4.0 should not use these functions.

Attributes	Description	Return
GetAttributeValue(string attrString, string attrName)	Returns the value of the attribute <i>attrName</i> or NoValue if <i>attrName</i> isn't found. If you pass in a blank (empty) string for the <i>attrName</i> variable, function returns the value of the first attribute in the attribute string.	R
RemoveAttribute(string attrString, string attrName)	Returns the attribute string after removing attribute <i>attrName</i> .	S
SetAttribute(string attrString, string attrName, real value)	Adds <i>attrName</i> and value or, if it already exists, changes the <i>attrName</i> to value. Returns the attribute string after adding or changing the attribute value.	S

Time units

These functions convert local time units defined within blocks to the global time unit defined in the Simulation Setup (see “Units of time” in the main ExtendSim User Guide). GetTimeUnits and ConvertTimeUnits use the following values for Time Units.

Time Unit	Value
Generic Time Units	1
Milliseconds	2
Seconds	3
Minutes	4
Hours	5
Days	6

Time Unit	Value
Weeks	7
Months	8
Years	9

Time Units	Description	Return
ConvertTimeUnits (real value, integer fromType, integer toType)	Converts a value from one type of time unit to another.	R
GetTimeUnits()	Returns the currently selected Time Units from the Simulation Setup dialog.	I
SetTimeConstants (real hInADay, real dInAWeek, real dIn- AMonth, real dInA- Year)	Sets the time unit conversion values. These are specified in the Simulation Setup dialog.	V
SetTimeUnits(integer value)	Sets the Time Unit parameter in the Simulation Setup dialog.	I

Calendar Date functions

ExtendSim Calendar Date values are numeric representations of a date value that is the same as is used by Microsoft Excel, and is based on the number of days since January 1, 1900 and the fraction of the current date for the time value. The integer part of the number is the number of days since 1900, and the decimal part of the number is the fraction of the current day. For example, the number 37256.5 would be twelve noon, on January first, 2006. There is a Macintosh version of this date calculation, (also implemented based on the same numbers as Excel,) which bases the first part of the date number on January first, 1904. Make sure that if you are communicating with an outside application, you have selected the same version of these date choices in both applications.

If you have a model saved in ExtendSim, and it has saved date values in one of these formats, running it in the other date setting will give unexpected results. This legacy Macintosh date setting is selectable in the Simulation Setup Dialog. Calendar Dates can only be selected in the simulation setup dialog if the time units for the model are set to seconds or longer, and not to generic time units, or milliseconds.

 Prior to ExtendSim 7, a different time and date format was used, as discussed on page 375.

Date and time	Description	Return
EDCalcDate(integer year, integer month, integer day, integer hour, integer minute, integer second)	Construct a date value from its individual components.	R
EDCalendarDateGet(real startDate)	Opens the calendar input dialog for the user to input a date value and returns that date value. The input dialog will show the value of the parameter startDate as a starting point.	R
EDCalendarDates()	Returns the value of the CalendarDates checkbox in the simulation setup dialog box: FALSE is unchecked, TRUE is checked.	I
EDCalendarShow(true/false)	This function opens or closes the calendar window.	I
EDConvertDate(integer value, integer fromType, real startDate)	Modifies a date value by adding additional time. The additional time added to the date value will be in the value parameter, and what time units it is in will be in the fromType parameter. Note that this function returns a date value, not a number of time units.	R
EDDateToSimTime(real currentTime, integer timeUnits)	Converts from a date value to a simulation time value (e.g. a possible value of currentTime). Putting in a zero for the timeUnits argument will make the function use the currently specified model time units. (see Time Units)	R
EDDateTo-String(real dateValue, integer format)	Converts an ExtendSim date value to a string according to format: 0 : date and time, 1 : Just date (Ignore Time), 2 : Just Time (Ignore Date), 3 : tight format (two digit year, don't show time value if zero.)	S
EDDateValue(real value, integer which)	Gets one part of a date value. <i>Which</i> is the time unit value (see "Time units" on page 372, above). NOTE: Week information is not available in this function.	I
EDDay-OfTheWeek(real currentDate)	Converts from an ExtendSim Date value to an integer value representing the day of the week. Returns a zero for Sunday.	I
EDGetCurrentDate()	Returns the current date equivalent for the CurrentTime simulation variable during a simulation run.	R
EDGetStartDate()	Returns the date value of start time in the Simulation Setup Dialog.	R
EDNow()	Returns the real time current date and time like the Now() function, but as an ExtendSim date.	
EDSimTimeToDate(real simTime, integer timeUnits)	Converts a simulation time value to a date value. A simTime value is a time number in simulation time format. Putting in a zero for the timeUnits argument will make the function use the current model time units. For example in most models, the simTime value for start-Time is zero. Calling with a zero value of simTime will return the ExtendSim Date value for the StartDate value of the model.	R

Date and time	Description	Return
EDStringToDate(string dateString)	Parses a string to retrieve an ExtendSim Date value.	R

Date and time, legacy format

 The following date and time functions were used in releases prior to ExtendSim 7. See “Calendar Date functions” on page 373 for the functions used in ExtendSim 7.

The legacy format stored dates and times in an integer that is the number of seconds since midnight, January 1, 1904.

- The Now function returns the current date and time. To determine the amount of time that has elapsed since the beginning of the simulation, subtract Start Time from Now.
- Use the DateToString and TimeToString functions to see a text representation of a date/time number.
- The CalcDate function creates a single numeric date value from its arguments (year, month, etc.)

Date and time	Description	Return
CalcDate(integer year, integer month, integer day, integer hour, integer minute, integer second)	Returns the numeric time value for the specified information. The arguments are all integers. The <i>year</i> argument is entered as the entire year (e.g. 1993) where years range from 1904 to 2040; <i>months</i> are entered from 1 to 12 (4 for April); <i>days</i> are 1-31; <i>hours</i> are based on the 24-hour clock and go from 0 to 23 (2 in the afternoon would be 14); <i>minutes</i> and seconds are 0-59. For example, February 1, 1994 at 2:00 PM would be (1994, 2, 1, 14, 0, 0).	I
DateToString(integer i)	String containing the date in the localized date format (usually MM/DD/YY) corresponding to integer <i>i</i> .	S
DiffDate(integer firstDate, integer secondDate)	Returns the difference between two date values as a real number which represents the number of days between the two dates.	R
GetBlockDates(integer blockNumber, integer whereFrom, integer whichDate)	Returns the modified and created dates of the block. The <i>whereFrom</i> argument takes a zero for the block or a one for the library. The <i>whichDate</i> argument takes a zero for created or a one for modified. Use the dateToString & timeToString functions to get the string values of the date & time.	I
ModifyDate(integer oldDate, real dateModifier)	Returns the old date value plus the date modifier value. The <i>oldDate</i> value is an ExtendSim integer date value, similar to that returned from the GetBlockDates function, and the <i>dateModifier</i> value is a real number representing a number of days.	I
Now()	Number of the current date and time.	I
TimeToString(integer timeVal)	String containing the time in the localized time format (usually Hour:Minute:Second) corresponding to integer <i>timeVal</i> .	S

Date and time	Description	Return
WaitNTicks(integer numTicks)	Waits for the number of ticks (60ths of a second) before returning. This is useful for slowing simulations down and for synchronizing communication protocols with real time.	V

Timer functions

These allow real time measurements to be set up. The TimerID functions are an extension of the timer functions that allow multiple (Up to 200) timers to run simultaneously. Each timer is referenced by an ID number from 0 to 199. The StartTimer() and StopTimer() functions (without a timerID argument) always refer to timer zero.

Timers	Description	Return
PrecisionTimer()	Returns the current timer count of the highest resolution timer found on the system. Used in conjunction with PrecisionTimerScale(), below.	R
PrecisionTimerScale ()	Returns the numbers of counts per second of the timer made available in the PrecisionTimer function.	I
StartTimer(integer blockNumber, integer waitTicks)	Starts a timer chore that will periodically send messages to either the specified block, or every block in the active model if the blockNumber is zero. This chore is performed when the CPU is idle, so you cannot count on it happening exactly every period, unless the CPU is idle. (e.g. other user/program actions will potentially interrupt the execution of the chore.) If the CPU is idle, the chore will be performed every waitTicks time intervals. Ticks are 60ths of a second, so if you enter 60 the chore will attempt to send out its message every second. The message sent is TIMERTICK. See the Break-out.mox model for an example of timer events.	V
StartTimerID(integer blockNumber, integer waitTicks, integer index)	Starts a timer with an ID tag, which can range from 0 to 199. As with the StartTimer() function, blockNumber specifies which block should receive the TIMERTICK message, and waitTicks specifies how often it should be sent.	V
Stoptimer()	This procedure stops the idle timer chore started by startTimer above.	V
StopTimerID(integer index)	Stops a timer with an ID tag, which can range from 0 to 199.	V
TickCount()	Clock count (in 60ths of a second) since the computer was powered up. This is useful for timing operations.	I
TimerID()	This function is to be used in the TimerTick message handler to find which timer is responsible for triggering the message. Returns the ID number of the timer that is currently activating the message.	I

Debugging

Also see the Abort statement in “Control statements and loops” on page 70 and UserError in “Alerts and prompts” on page 256.

Debugging	Description	Return
AbortAllSims()	Aborts the simulation and all multiple simulations. Note that the Abort statement only aborts the current simulation.	V
AbortAllSimsSilent()	Aborts a multiSim run without an error message.	I
AbortSilent()	Aborts the simulation run without giving any error messages.	V
DebuggerBreakpoint(integer trueFalse)	If called with a true value will act as if a source debugger breakpoint has been set at the line of code where the function is called. This function is useful in debugging the CreatBlock message and in putting a global breakpoint for all blocks of a type.	V
DebugMsg(string errorString)	Operates like the UserError function. The difference is that this function flags the block as having debugging code – the next time a library is opened that contains any blocks having this function, ExtendSim will issue a warning message. This function automatically displays the simulation time and the block number of the block from which the function was called.	V
DebugWrite(integer fileNum, string errorString, delimStr, integer tabCR)	Operates like the FileWrite function. The difference is that this function flags the block as having debugging code – the next time a library is opened that contains any blocks having this function, ExtendSim will issue a warning message.	V
FreeMemory(integer memoryType)	Returns the amount of memory available to ExtendSim. The <i>memoryType</i> argument determines what type of memory will be checked. <u>Windows:</u> 1 - Total memory 2 - Resources (only for Windows 3.1 and Windows 95) <u>Mac OS:</u> 1 - Total memory 2 - Contiguous memory	I
PauseSim()	Pauses the simulation until you choose Run > Resume or click the Resume button.	V
ProfileBlockGet(integer blockNumber)	Returns the block profile results for the specified block. This will only return a meaningful number if the command Profile Block Code is selected. Can be called in finalCalc to get the total profile results for that block or during the simulation to get partial results.	R
SelectBlock(integer trueFalse)	Selects the block and scrolls to it if the argument is TRUE, unselects it if the argument is FALSE.	V
SelectBlock2(integer block, integer trueFalse)	This is same as SelectBlock(), except it refers to a global block number.	V
TraceModeEnableDisable(integer enableIfTRUE)	Call to enable or disable the current trace when desired. Trace mode must be on for this function to work. It returns zero if no error and -1 if trace mode is not on.	V

Web and Help connectivity

See also the ShowFunctionHelp function which brings up a list of ExtendSim's functions and arguments. This function is listed with "Equations" on page 228.

Help	Description	Return
CallHelp(string fileName, integer command, integer data, integer fileType)	Used to load a WinHelp file, an HTML Help file, or a pdf file. (For the WinHelp and HTMLHelp Windows API calls, see the Microsoft documentation for more information.) The fileType flag takes the following values: 0: Calls the WinHelp function (Windows only. Opens compiled Help files. Typically have an extension of .hlp.) 1: Calls HTMLHelp. (Opens compiled Html help files. Typically have an extension of .chm.) 2: Opens a file with a .pdf extension. This function is used to bypass the standard ExtendSim Help system via the following code in the "on helpbutton" message handler: <pre> On helpbutton { CallHelp("C:\helpfile.chm", 1,1,1); Abort; } </pre>	V
OpenURL(string theURL)	Access the specified URL using your computer's default browser. Returns non-zero error code if it fails.	I
ShowBlockHelp(integer block)	Opens the Help dialog, showing the online Help for any global block. For example, use the function GetEnclosingHBlockNum to get the global block number of an enclosing hierarchical block to show its Help under ModL code control.	V
ShowHelp()	Opens the Help dialog, showing the online Help for the block.	V

Platforms and versions

These functions allow you to determine the version number of ExtendSim and the platform on which it is running.

Platforms and versions	Description	Return
GetCurrentPlatform()	Determines the operating system under which ExtendSim is currently running. This function returns 0 for 68K Mac OS, 1 for Power Mac OS or Mac OSX, 2 for Windows 3.1/Win32s, 3 for Windows NT or XP, and 4 for Windows 95, 98, and ME.	I

Platforms and versions	Description	Return
GetExtendVersion(integer which)	Returns a real number in the format 701.1 where 7 is the major version, 1 is the minor version, and 0 is the middle. The value after the point is 1 for an a, 2 for a b, and so on. This final value is always zero for a file read version. Which: 0 : application version 1 : file version	R
GetExtendVersionString()	Returns the version number of ExtendSim as a string.	S
GetExtendType()	Returns the type of the ExtendSim application. Normal: 0 LT/RunTime: 2 Demo/Player: 4	I
GetFileReadVersion()	Returns the version of the file being read, or previously read. It can be used in the On BlockRead message handler to determine the version of the file that is currently in the process of being read. In conjunction with the ResizeDTDuringRead function, it will allow you to inform ExtendSim that a data table has changed size from the size it was in an older version. (This is useful for using the Dynamic data table function without breaking existing models.)	R
GetFileReadVersionString()	Returns the version of the file being read, or previously read, as a string. This is similar to the GetFileReadVersion function, with the difference that the result is a version string, not a real number. NOTE: This function returns the complete version string, in the form 'major version. minor version.bug fix version', unlike the GetFileReadVersion function which just returns the major version. This function will return an empty string for files earlier than version 4.0.	

Lego Mindstorms control (Windows only)

These functions are designed for controlling the Lego Mindstorms robotics kit. They are designed for use with the original Logo kit, not the NXT kit. These functions are intended to be used as part of a direct interface. (I.e. the ExtendSim software is running on the PC, the Control tower is attached to the USB port on the machine, and the ExtendSim model directly controls the robot through the USB tower.) These functions cannot be used for programming the robot, just for direct control from ExtendSim.

Lego Mindstorms	Description	Return
LegoClose()	Stops the communication with the robot.	I
LegoMotor(integer aOn, integer aSpeed, integer bOn, integer bSpeed, integer cOn, integer cSpeed)	aOn, bOn, and cOn specify if the given motor is on or off. A motor that is off will lock. If you want the motor to turn freely, specify that the motor is on, and specify a zero for the speed of the motor. aSpeed, bSpeed, and cSpeed are the speeds of the three motors. The valid values for these range from -8 to +8.	V

Lego Mindstorms	Description	Return
LegoSensor-Block(integer block-Number)	Specifies which block should receive the LEGOSENSOR message. This message is sent whenever the sensor value is non-zero for each sensor that is enabled. By default this block number is set to -1, which means that each block in the model will receive the message.	V
LegoSensorOff(integer which)	Turns off the specified sensor. See LegoSensorOn for more information.	V
LegoSensorOn(integer which)	‘Which’ ranges from 0 to 4. It specifies which sensor is to be turned on. Sensor 1, 2 and 3 are the three sensors of the robot. Sensor 0 is the ‘Alive’ signal from the robot, and sensor 4 is the battery power signal. Sensors take monitoring time, you should avoid turning on, or leaving on, sensors that you don’t need. The sensors can be queried with the LegoSensorRead function, and ExtendSim also sends the LEGOSENSOR message to specified blocks in the model whenever any active sensor is non-zero.	V
LegoSensor-Read(integer which)	Returns the value of the specified sensor.	I
LegoSound(integer sound)	values for sound: 0 : sound off 1 : blip 2 : beep-beep 3 : downward tones 4 : upward tones 5 : low buzz 6 : fast upward tones.	V
LegoStart()	Starts the communication with the robot.	I

User-defined functions for ADO

User-defined functions are custom functions, coded in ModL, that can be declared in a block for local use or declared in an include file for use by multiple blocks. For how to create, use, and override them, see “User-defined functions” on page 73 and “Include files” on page 79.

The following ActiveX Data Object (ADO) functions are used to implement ADO features in the Data Import Export block (Value library). These ModL-coded functions are stored in an include file named ADO_DBFunctions.h. To reference that include file in your block’s code, use a format discussed on page 79, such as #include “ADO_DBFunctions.h”.

ADO Function	Description	Return
ADO_AddRecords(integer ADOApp, string ADO_TableName, integer EX_DBIndex, integer EX_TableIndex)	Inserts an ExtendSim table into an ADO database table. ADO_TableName - name of the table in the ADO database EX_DBIndex - ExtendSim Database index EX_TableIndex - ExtendSim table index Note 1: The tables must have exactly the same structure. Note 2: This is the fastest way to transfer information.	V

ADO Function	Description	Return
ADO_CheckCompatibleFieldType(string ADO-DataType, integer ExtendSimType)	Returns True (1) if the ADO field type is compatible with the ExtendSim field type.	I
ADO_Close(integer ADOAppHandle, integer Force)	Closes the connection to the ADO DLL. Call when done accessing the DLL.	I
ADO_CreateTable(integer ADOApp, string ADO_TableName, string63 ADO_FieldArray[[4])	Creates a table in an ADO database. ADO_TableName - name of the table ADO_FieldArray contains the table names, their type, "Is nullable", and the number of characters	V
ADO_DeleteRecords(integer ADO-App, string ADO_TableName, string Criteria)	Deletes records from an ADO database table. ADO_TableName - name of the table in the ADO database. Criteria - SQL statement indicating which rows to delete. To delete all rows, leave blank.	V
ADO_ExecuteNonQuery(integer ADOApp, string SQLStr)	Executes a Non-query SQL statement. SQLStr - SQL statement	V
ADO_ExecuteQuery(integer ADOApp, string SQLStr)	Executes a Query SQL statement. SQLStr - SQL statement	V
ADO_GetFields(integer ADOApp, String ADO_TableName, string63 ADO_FieldArray[[4])	Gets a list of fields in the database. ADO_TableName - name of the table ADO_FieldArray contains the table names, their type, "Is nullable", and the number of characters. This is returned by the function	V
ADO_GetNumFields(integer ADO-App, string63 ADO_TableName)	Gets the number of fields in table ADO_TableName. ADO_TableName - name of the table in the ADO database	I
ADO_GetNumRows(integer ADO-App, string63 ADO_TableName)	Gets the number of records in table ADO_TableName. ADO_TableName - name of the table in the ADO database	I
ADO_GetNumTables(integer ADO-App)	Returns the number of tables in an ADO Database	I

ADO Function	Description	Return
ADO_GetTableColumns(integer ADOApp, integer EX_DBIndex, integer EX_TableIndex, string ADO_TableName, string63 ADO_Columns[])	Transfers a set of columns to an ExtendSim database table. EX_DBIndex - ExtendSim Database index EX_TableIndex - ExtendSim table index ADO_TableName - name of the table in the ADO database ADO_Columns - array containing the column (field names) to import into the ExtendSim data table Note 1: Allocate records in the ExtendSim table before calling this function. Note 2: The data types for the fields in the ExtendSim table and the ADO table must be compatible.	V
ADO_GetTables(integer ADOApp, string63 ADO_FieldArray[])	Gets the list of tables in the ADO database. ADO_FieldArray contains the table names, their type, "Is nullable", and the number of characters.	V
ADO_OpenConnection(string DatabaseType, string FileName, string UserName, string Password, String Server)	Opens a connection with an ADO database. This is where database information is specified: DatabaseType - Access, SQLServer, MYSql, or XML Filename - name of the database UserName Password Server - name of the database server (not used in Access or XML)	I
ADO_SetTableColumns(integer ADOApp, string ADO_TableName, string63 ADO_Columns[], integer EX_DBIndex, integer EX_TableIndex)	Transfers a set of columns to an ADO database table. ADO_TableName - name of the table in the ADO database ADO_Columns - array containing the column (field names) to import into the ExtendSim data table EX_DBIndex - ExtendSim Database index EX_TableIndex - ExtendSim table index Note: The data types for the fields in the ExtendSim table and the ADO table must be compatible.	V
ADO_Setup()	Sets up the connection to the ADO DLL. Call before accessing the DLL Returns the ADO Application Handle - referred to in other functions as ADOApp.	I
ADO_SQLServerGetServers(string63 ServerInfo[][4])	Returns a list of SQL Server Servers. ServerInfo - array containing the list of the servers. Allocate this array before calling the function.	V
ADO_SQLServerGetDataBases(string63 Server, string63 DBArray[][4])	Returns a list of SQLServer databases. Server - name of the SQL Server DBArray - returns name, size, description of the SQL Server database. The forth column is unused.	V
ConvertADODataType(string ADODataType)	Converts an ADO field type to an ExtendSim Constant. ADODDataType - string containing the type for the ADO type (float, string, int ...)	I

ADO Function	Description	Return
ConvertExtendSim- DataType(integer ExtendSimType)	Converts an ExtendSim constant for data type to SQL string for data type.	S
DB_FieldGetTypeSt ring(integer Extend- SimType)	Returns the string description given an ExtendSim field type.	S

Appendix

Menu Command Numbers

A list of the menu command numbers to be used with
the ExecuteMenuCommand function

*“Obedience alone gives the right to command.”
— Ralph Waldo Emerson*

The `ExecuteMenuCommand(commandNumber)` function executes a specified menu command (see “Scripting” on page 319). This is functionally the same as selecting the command from the menu.

The following is a list of the command numbers that are used with the `ExecuteMenuCommand()` function.

File Menu	Command Number
New	2
New Text File	1601
Open	3
Close	4
Revert Model	7
Save Model	5
Save Model As	6
Update Launch Control	1410
Import Data	2005
Export Data	2006
Import DXF File	2014
Show Page Breaks	2000
Print Setup or Page Setup	8
Print	9
Properties or Get Info	2001
recent files 1-5	1555-1559
Exit or Quit	1

File\Network License menu	Command Number
License Info	3204
Check Out License	3202
Check In License	3203
Remove License	3201

Edit menu	Command Number
Undo	16
Cut	18
Copy	19
Paste	20
Clear	21
Delete Selected Records	1911
Select All	23
Duplicate	2013
Find...	1512
Find Again	1514
Replace	1515
Replace, Find Again	1516
Replace All	1517
Enter Selection	1513
Create/Edit Dynamic Link	1929
Open Dynamic Linked Blocks	2075
Sensitize Parameter	2046
Open Sensitized Blocks	2019
Paste DDE Link	9005
Delete DDE Link	9006
Show DDE Links	9007
Refresh DDE Links	9008
Insert Object...	9009
Design Mode	9011
Show Clipboard	22
Options	9060
Text Menu	Command Number
Plain Text	30

Text Menu	Command Number
Bold	31
Italic	32
Underline	33
Outline (Macintosh)	34
Shadow (Macintosh)	35
Condensed (Macintosh)	36
Extended (Macintosh)	37
Justify Left	41
Justify Center	42
Justify Right	40
Border	5000
Transparent	5100

Text\Size Menu	Command Number
Other	3000

Library Menu	Command Number
Open Library	4500
Close Library	4501
New Library	4503

Library\Tools Menu	Command Number
Open All Library Windows	1534
Compile Open Library Windows	1530
Compile Selected Blocks	1531
Add Debug Code to Open Library Windows	1533
Remove Debug Code in Open Library Windows	1532
Add External Code in Open Library...	1577

Library\Tools Menu	Command Number
Remove External Code in Open...	1576
Protect Library	2063
Set Library Version	2064
Convert Library to RunTime Format	2061
RunTime Startup Screen Editor	2062

Model Menu	Command Number
Make Selection Hierarchical	1501
New Hierarchical Block	1502
Open Hierarchical Block Structure	3047
Show Named Connections	2011
Hide Connections	2012
Hide Connectors	2071
Rotate Shape	1310
Flip Horizontally	1311
Flip Vertically	1312
Lock Model	2028
Use Grid	2070
Show Block Labels	2049
Show Block Numbers	2025
Show Simulation Order	2018
Set Simulation Order...	1700

Model\Align Menu	Command Number
Left	7401
Right	7403
Top	7404
Bottom	7406

Model\Border Thickness Menu	Command Number
0	1314
1	1315
2	1316
3	1317
4	1318
5	1319

Model\Shape Fill/Border Menu	Command Number
Fill Color	1322
Border Color	1323

Model\Connection Lines Menu	Command Number
Non Right-Angle Connections	5002
Right-Angle Connections	5001
Right Pointing Arrows	1208
Left Pointing Arrows	1209
View Using Defaults	1250
Thin Line	1201
Medium Line	1202
Thick Line	1203
Hollow Line	1204
Solid Line	1205
Dashed Line	1206
New Connections Are Black	1210
New Connections Use Color	1211

Model\Controls Menu	Command Number
Slider	1301
Switch	1302
Meter	1303

Model\Change Model Style Menu	Command Number
Classic	16001
Flowchart	16002
Style 3	16003
Style 4	16004
Style 5	16005
Style 6	16006
Style 7	16007
Style 8	16008

Database Menu	Command Number
New Database	1901
Import New Database	1904
Export Database	1905
Rename Database	1902
New Table	1906
Import Tables	1903
Export Selected Tables	1907
Rename Table	1908
Edit Table Properties	1932
New Database Tab	1922
Rename or Delete Database Tab	1923
Clone Selected Tables to Tab	1920
Remove Selected Tables from Tab	1921

Database Menu	Command Number
Append New Field	1913
Insert New Field	1912
Edit Field Properties	1933
Append New Records	1910
Insert New Records	1909
Read/Write Index Checking	1931

Develop Menu	Command Number
New Block	1503
Open Block Structure	2047
Rename Block	1509
Set Block Category	1535
Compile Block	1508
Generate Debugging Info	1524
External Source Code	1575
New Dialog Item	1510
New Tab	1551
Rename or Delete Tab	1552
Move Selected Items to Tab	1550
New Include File	1520
Open Include File	1521
Delete Include File	1522
Shift Selected Code Left	1518
Shift Selected Code Right	1519
Go To Line	1523
Go To Function/Message Handler	1536
Match Braces	1537
Match IFDEF/ENDIF	1538
Show Reserved Databases	1928

Develop Menu	Command Number
Set Breakpoints...	1525
Open Breakpoints Window	1403
Open Debugger Window	1404
Continue	1526
Step Over	1527
Step Into	1528
Step Out	1529

Run Menu	Command Number
Run Simulation	6000
Continue Simulation	6003
Run Optimization or Scenarios	6002
Simulation Setup	6001
Prioritize Front Model	6004
Use Sensitivity Analysis	2027
Show 2D Animation	2020
Add Connector Line Animation	2081
Add Named Connection Animation	2080
Show 3D Animation	2082
Launch Proof	3030
Launch StatFit	3040
Show Movies (Macintosh)	2026
Generate Report	2050
Add Selected To Report	2051
Add All To Report	2055
Remove Selected From Report	2052
Remove All From Report	2053
Show Reporting Blocks	2054
Stop	30000

Run Menu	Command Number
Pause	30001
Step	30003
Resume	30002

Run\Report Type Menu	Command Number
Dialogs Report	2057
Statistics Report	2058

Run\Debugging Menu	Command Number
Pause At Beginning	2033
Step Each Block	2024
Step To Next Animation	2032
Step Entire Model	2023
Show Block Messages	2021
Only Simulate Messages	2022
Scroll To Messages	2030
Generate Trace	2040
Add Selected To Trace	2041
Add All To Trace	2045
Remove Selected From Trace	2042
Remove All From Trace	2043
Show Tracing Blocks	2044
Profile Block Code	2029
Show Debug Messages	2034

Help Menu (Apple Menu)	Command Number
About ExtendSim	256
ExtendSim Help	257

Block Dialog	Command Number
OK	100
Cancel	101

Simulation Status Bar	Command Number
Stop	30000
Pause	30001
Resume	30002
Step	30003
Slower Animation	30004
Faster Animation	30005

Appendix

Upper Limits

A list of the maximum numbers of things
that you can do at one time

“The thing I am most aware of is my limits.”
— *André Gide*

Note: Some limits depend on available memory.

Steps in a simulation run	2 billion
Number of simulations in a multiple run	2 billion
Block name or label length, characters	31
Blocks in a model	2 billion
Blocks in a library	200
Libraries open at one time	40
Text files open at one time	200
Databases per model	2 billion
Tables per database/fields per table/records per table	2 billion/1,000/2 billion
Output connectors in a model (nodes)	2 billion
Connectors per block	255
Length of a block's ModL code (characters)	4 megabytes
Dialog items in a block	1024
2D animation objects per block	255
Dynamic arrays (each array)	2 gigabytes
Number of array dimensions	5
Maximum index for array dimensions	2 billion elements total
Dynamic arrays per block	255
Columns in a table	255 (data table); 255 (text table)
Total table size (cells) per block for static data tables	3260 (data table); 2030 (text table)
Total table size (cells) each, for dynamic data tables	2 billion
Variable name length: dialog item msgs, connector names	63
Variable name length: ModL local, static variables	127
Maximum popup menu length	5100 characters or 20 strings
User defined function arguments	127
Nested loops	32
Maximum, minimum of real numbers	$\pm 1\text{E}\pm 308$
Maximum, minimum of integer numbers	$\pm 2,147,483,647$
Significant figures in real calculations	16 (double)
Number of attributes for discrete event item	500
Static and local variable declaration limit (See page 63)	32,767 bytes

Appendix

ASCII Table

To help you determine the values of
the ASCII characters

*“I would sooner read a timetable or catalogue than
nothing at all. They are much more entertaining
than half the novels that are written.”*
— W. Somerset Maugham

This table shows the ASCII values from 00 to 127, which are the same for Windows and Mac OS. Values above 127 are not part of the standard ASCII set and vary depending on the font.

00	NUL	32	space	64	@	96	`
01	SOH	33	!	65	A	97	a
02	STX	34	"	66	B	98	b
03	ETX	35	#	67	C	99	c
04	EOT	36	\$	68	D	100	d
05	ENQ	37	%	69	E	101	e
06	ACK	38	&	70	F	102	f
07	BEL	39	'	71	G	103	g
08	BS	40	(	72	H	104	h
09	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	S	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	ETB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

Appendix

Cross-Platform Considerations

File conversion, file name comparisons, and keyboard shortcuts
for the Windows and Macintosh operating systems

*“There never were, since the creation of the world,
two cases exactly parallel.”
— Lord Chesterfield*

Libraries

In most cases, libraries should be placed in the ExtendSim7\Libraries folder; they can be placed in subfolders within that folder. Optionally, libraries needed by a model can also be at the model file location. Libraries should never be placed anywhere outside of either the Libraries folder or the folder that contains the model file that uses the library.

Windows: ExtendSim supports library file names up to 64 characters. Library names must end in the “.LIX” extension.

Mac OS: Library names can be up to 31 characters long and usually end in “Lib”, although this is not required.

Models

For Windows, ExtendSim supports long file names. All Windows models must end in the “.MOX” extension. Mac OS model names can be up to 31 characters.

Menu and keyboard equivalents

The following table lists some common actions and keyboard shortcuts under the Windows and Mac OS systems. The User Guide contains pictures of the ExtendSim menus, including the keyboard equivalents for the menu commands.

Action	Menu>Command	Windows keyboard	Mac OS keyboard
Open a model	File > Open	CTRL+O	CMND+O
Open a library	Library > Open Library	CTRL+L	CMND+L
Save a model	File > Save Model	CTRL+S	CMND+S
Close the active window	File > Close	CTRL+W	CMND+W
Run a simulation	Run > Run	CTRL+R	CMND+R
Stop a simulation	Run > Stop	CTRL+period (.)	CMND+period (.)
Stop a library search		CTRL+period (.)	CMND+period (.)
Select a parameter for sensitivity analysis	Edit > Sensitize Parameter (Select parameter first)	Hold down the CTRL key and click once on the parameter	Hold down the CMND key and click once on the parameter
Open a hierarchical block's structure to edit the icon, etc.	Develop > Open Block Structure (Select block icon first)	Hold down the Alt key while double-clicking on the block's icon	Hold down the Option key while double-clicking on the block's icon
Open a library block's structure to edit the Modl code or icon	Develop > Open Block Structure (Select block icon first)	Hold down the Alt key while double-clicking on the block's icon	Hold down the Option key while double-clicking on the block's icon
Remove the grid when aligning drawing objects, connectors, etc.		Hold down the Alt key while moving the object	Hold down the Option key while moving the object

Action	Menu>Command	Windows keyboard	Mac OS keyboard
Proportionately scale drawing object		Hold down the Shift key while resizing the object	Hold down the Shift key while resizing the object

Menu and keyboard equivalents for developers

Develop menu

Command	Windows	Macintosh
Compile Block	Ctrl+K	Cmnd+K
New Dialog Item	Ctrl+I	Cmnd+I
Shift Selected Code Left	Ctrl+[	Cmnd+[
Shift Selected Code Right	Ctrl+]	Cmnd+]
Go To Line	Ctrl+'	Cmnd+'
Match Braces	Ctrl+J	Cmnd+J
Match IFDEF/ENDIF	Ctrl+I	Cmnd+I

Run menu

Command	Windows	Macintosh
Run Simulation	Ctrl+R	Cmnd+R
Simulation Setup	Ctrl+Y	Cmnd+Y
Add Selected To Report	Ctrl+6	Cmnd+6
Remove Selected From Report	Ctrl+7	Cmnd+7
Stop	Ctrl+.	Cmnd+.
Pause	Ctrl+/	Cmnd+/
Step	Ctrl+,	Cmnd+,
Resume	Ctrl+-	Cmnd+-
Debugging > Add Selected To Trace	Ctrl+8	Cmnd+8
Debugging > Remove Selected From Trace	Ctrl+9	Cmnd+9

Transferring files between operating systems

The Windows and Mac OS versions of ExtendSim are cross-platform compatible. For example, if you build a model or a library on your Windows computer, you can move it to a Mac OS computer and ExtendSim will read it, and vice versa.

There are three considerations when transferring ExtendSim files between Windows and Mac OS systems: file name adjustments, physical transfer, and file conversion.

- Models and libraries developed in an older version and different platform may not transfer successfully. It is strongly recommended that you upgrade to the latest version on the source platform, re-

save the structure of any hierarchical blocks stored in libraries (see “Saving hierarchical blocks to a library” on page 609) and recompile any libraries that you have created. Then re-save your models before transferring your files.

File name adjustments

- **Windows to Mac OS:** If you are transferring files from a Windows computer to a Mac OS, you do not need to change the file name or delete the extension; the Mac OS system can read names up to 31 characters long.

 It is important that you do not delete the MOX extension from a Windows model file name before transferring the model to your Mac OS system. The MOX extension is required so ExtendSim can identify the file as a Windows model. After ExtendSim has converted the Windows model to Mac OS format, save the model under the same name or a new name.

- **Mac OS to Windows:** If you transfer files from a Mac OS to a Windows computer, you may need to change the name of your file before you transfer it. ExtendSim supports file names of up to 64 characters for Windows. File names must end in a three-character extension (the extensions are “.MOX” for ExtendSim model names, “.LIX” for library names, and “.TXT” for text file names). To change your file names to Windows format, change the name of the file *on the Mac OS computer* using ExtendSim’s Save As command (if the file is open) or using the Finder (if the file is not open).

Physically transferring files

Once you have made any necessary file name adjustments, physically transfer the files between Windows and Mac OS computers. This process depends on your system resources and is independent of ExtendSim. For example, you might copy the file onto a memory stick. Or you could send the files directly from one computer to the other if the computers are networked.

 Libraries and extensions created on a Mac OS computer need to be converted *before* being physically transferred to the Windows system, as discussed below.

File conversion

Depending on the type of file, file conversion may be handled automatically by ExtendSim or may involve using a conversion application. As discussed below, ExtendSim automatically converts model files when they are opened on a different platform. If you program your own blocks, the ExtendSim for Mac OS package includes a file conversion utility which you can use to convert libraries and picture resource extensions from one operating system format to another. Other extensions that you build, such as QuickTime movies, DLLs, and Shared Libraries require more extensive conversion. Include files are, of course, already cross-platform compatible.

Converting model files

The Windows version of ExtendSim can read ExtendSim model files created on the Mac OS as long as the name format is correct, as discussed above. The Mac OS version of ExtendSim can read ExtendSim model files created under Windows without file name modification (in fact, as discussed in the Note above, the MOX extension should not be removed.) When you open a model file that was created on another operating system, ExtendSim will notify you that it is converting the file from that system to the current one. Once you save the model file, it will be in the format of your current operating system.

If your models (including hierarchical blocks) use libraries that you have created yourself, and you have changed the name of those libraries, ExtendSim will not be able to locate the library. In this

case, ExtendSim will ask you to find and select the correct library as described in the User Guide. Keeping all your libraries in the Libraries folder will make this search process easier. Saving the model will cause the new libraries to be used from then on.

For model files that have blocks that access text files (such as the Read block from the Value library), you may need to change the name of the text file that is being read to conform to platform requirements, as discussed above. Be sure to also change the name of the file in the Read block's dialog to correspond to the new file name.

- ☞ The first time you run a model that has been transferred from one operating system to another, any Equation blocks in the model will recompile to the format of the new system at the beginning of the simulation run. Messages that report this process may appear too quickly for you to read.

Converting hierarchical blocks in libraries

If you have a hierarchical block saved in a library *and* you have renamed any of the libraries of the blocks *inside* the hierarchical block (for example, to comply with Windows format), you need to update the hierarchical block's information so that it can locate the renamed libraries. The easiest way to do this is to drag hierarchical blocks from their libraries, place them on a worksheet, and update their structure, as discussed below.

- ☞ This is only required for hierarchical blocks saved in libraries; hierarchical blocks saved only in a model get updated with the model.

When you add a hierarchical block from a library to a model worksheet, the hierarchical block causes ExtendSim to open the libraries of the blocks inside it. Since you have renamed those libraries, ExtendSim will not be able to locate them. In this case, ExtendSim will ask you to find and open the correct libraries. Note: keeping all your libraries in the Libraries folder will make this search process easier.

If you save the model worksheet that contains the hierarchical block, the location of the renamed libraries is saved for the model only. Before you close the model worksheet, you also need to update the hierarchical block's library information. To do this, open the hierarchical block's structure window and then close it, causing the hierarchical block's Save dialog to appear. In the dialog, choose **Also save to library**. This process is described in the User Guide.

Libraries

The libraries that come with your ExtendSim package are already formatted correctly for your operating system. However, if you build your own libraries, and want to transfer them to a computer running a different operating system, you must convert them to the appropriate operating system format. You do this *on the Mac OS computer* using the Libraries > Tools > MacWin Conversion command (see the User Guide for more information). This converts libraries to the specified operating system format, ensures that the file names are properly formatted, and so forth.

After the libraries have been converted to Windows or Mac OS format, physically transfer them to the target computer (that is, keep them on the Mac OS or transfer them to a Windows computer). Then recompile the libraries under the target computer's operating system using the Library > Tools > Compile Open Library Windows command.

The MacWin Conversion converts libraries to either Windows or Mac OS format. After conversion, you must recompile the library on the target computer. When you do this, the library will compile to native code for the target system. For example, if you compile on a Windows computer, the library will be in native Windows mode. If you compile on a Power Mac OS computer, the library will be in native Power Mac OS mode.

Blocks that use the equation functions

If you build blocks that use the equation functions, your code needs to detect if the model is being opened on a different platform. See the “Equations” on page 228 for more information.

Extensions

Extensions are files (such as pictures and DLLs) that can be accessed by ExtendSim to fulfill specialized tasks. Like libraries, the extensions that come with your ExtendSim package are formatted correctly for your operating system. However, if you build your own extensions, and you want to transfer your extensions or blocks to a computer running a different operating system, you will need to do some conversion:

- **Pictures:** As discussed “Pictures on Windows” on page 86, ExtendSim for Windows accepts three kinds of pictures: WMF (Windows MetaFiles), BMP (Bitmap), and a Mac OS picture resource file that has been converted to Windows format. As discussed in “Pictures and movies on the Mac OS” on page 86, ExtendSim for the Mac OS accepts only picture resources. To convert Mac OS picture resource files to Windows format, use ExtendSim’s MacWin Converter utility on the Mac OS. To convert Windows pictures to Mac OS format, use a graphics conversion application.
- **Sounds:** Shareware utilities are available to convert Mac OS sound resources (SNDs) to Windows sound files (.WAV) and vice versa.
- **DLLs and Shared Libraries:** On Windows the DLL functions will search the ExtendSim Extensions folder for a DLL file. For the Mac OS those same calls will search for a Shared Library file.

IPC or multi-server considerations

The ModL language can only be compiled by ExtendSim. ExtendSim will execute ModL code in the same way regardless of the platform (Windows or Macintosh) on which it runs.

However, ExtendSim is capable of communicating with other applications through ModL code, and some of these applications may not have a consistent interface across platforms. The following is a discussion of some of the things to consider when writing ModL code that is intended to be cross-platform compatible.

The interprocess communication (IPC) functions allow ExtendSim to act as a client application and request data and services from server applications. When using the IPC functions, it is important to remember that the syntax of some of the function arguments are dependent both on the server application that ExtendSim is communicating with and the platform that ExtendSim is running on. For example, the *serverName* argument of the IPCConnect function (which defines the application you want to connect ExtendSim to) must be in the format appropriate for that application and platform. The *serverName* for Microsoft Excel on the Windows operating system would be “Excel,” while on the Mac OS operating system it would be “XCEL.” This type of information can be found in the documentation of the server application.

When writing ModL code that utilizes IPC functions and is meant to be cross-platform compatible or capable of operating with multiple server applications, it is necessary to define the function arguments according to the platform application *prior* to calling the function. This can be accomplished by using the platform variables PLATFORMPOWERPC, and PLATFORMWINDOWS) and using dialog items to define the server application (see blocks in the IPC library for examples).

For instance, the following code establishes a conversation between ExtendSim (the client) and Excel (the server) on either the Windows or Mac OS operating system:

```
if (PlatformPowerPC)
    serverName = "XCEL";
if (PlatformWindows)
    serverName = "Excel";
IPCCConnect (serverName, theSpreadsheet);
```

The following is a list of functions that contain arguments where the syntax of the argument depends on the server application (the dependent argument is shown in italics):

- IPCCConnect(*serverName*, topic)
- IPCExecute(conversation, *executeData*, item)
- IPCPoke(conversation, pokeData, *item*)
- IPCRequest(conversation, *item*)

Cross-platform code

The following ModL constants return TRUE or FALSE depending on the platform: PLATFORMMAC; PLATFORMWINDOWS; PLATFORMPOWERPC. You can use these constants in an If statement to make code be cross-platform capable. For example:

```
if (PLATFORMWINDOWS)
    windows specific code
else if (PLATFORMMAC)
    macintosh specific code
```


Symbols

- `_`(underscore) character 106
- `_leftClickDB` 94, 96
- `.cs` files 142
- `**` 29
- `*/` 29
- `/*` 29
- `//` 29
- `#define` 79
- `#else` 79
- `#endif` 79
- `#ifdef` 79
- `#ifndef` 79
- `%` operator 69

Numerics

- 2D animation
 - adding to a block 54
 - between blocks 126
 - changing a level 124
 - color 122, 127
 - for hierarchical blocks 122
 - functions 259
 - hiding a shape 123
 - hierarchical blocks 122
 - moving a shape 123
 - object 55
 - objects 19
 - overview 19
 - pattern 122
 - pictures 125
 - pixels 128
 - programming 120
 - shapes 121
 - showing a shape 123
 - stretching a shape 125
 - text 127
- 3D animation
 - 3D window 128
 - ambient animation 140
 - Buffered mode 130
 - buildings 149
 - clock 131
 - collision 138
 - Concurrent mode 130
 - creating a new object 142
 - DataBlocks 144
 - DIF format 149

- distance ratio 131
- DTS 142
- E3D environment 129
- event queue 131
- ExtendItems 134
- ExtendPlayers 134
- ExtendVehicles 134
- functions 264
- gravity 138
- ground height 141
- GroupTags 133
- idleSoundVol 140
- immediate functions 131
- interiors 149
- markers 141
- messages 212
- modes 129
- mount nodes 137
- mounting 137
- mountPoint 137
- mountStack 137
- Name field 132
- new objects 143
- object classes 134
- object label 132
- object tops 141
- objectID 132
- overview 128
- paths 141
- performance considerations 129
- pixels to distance ratio 131
- position of object 135
- post functions 131
- QuickView mode 130
- rotation of objects 135
- scale of objects 134
- script file 142
- scripting environment 143
- skins of objects 135
- sound 138
- terrain 149
- time ratio 131
- time ratios 131
- Torque Game Engine 129
- units 134
- unmounting 138
- userTag 133
- Wall.cs file 142
- waypoint 142
- 3D animation functions 274

- 3D array 166
- 3D block functions 265
- 3D creation, deletion functions 270
- 3D distance functions 277
- 3D environment functions 269
- 3D event queue 131
- 3D messages 212
- 3D miscellaneous functions 282
- 3D mounting functions 274
- 3D mountStack functions 267
- 3D movement functions 271
- 3D name functions 275
- 3D object functions 269
- 3D path functions 268
- 3D post functions 131, 272
- 3D properties functions 280
- 3D screenshot functions 277
- 3D script functions 281
- 3D skin functions 277
- 3D tick functions 280
- 3D time functions 266
- 3D user tag functions 279
- 3D vector functions 279
- 3D window functions 267

A

- Abort statement 70, 160
 - in CheckData 73
 - in Simulate 73
- AbortAllSims 160, 377
- AbortAllSimsSilent 377
- AbortDialogMessage 209
- aborting multiple simulations 156, 160
- AbortSilent 377
- absolute value 218
- Acos 219
- ActivateApplication 319
- ActivateModel 206
- ActivateWorksheet 319
- ActiveX
 - Automation 112
 - BlockMsg 116
 - C++ examples 113
 - client 112
 - controls 241
 - Execute 114
 - GetObjectHandle 117

- Poke 115
- Request 115
- Visual Basic 118
- Visual Basic examples 118
- ActiveX Automation 112
- ActiveX Data Objects functions 380
- Add External Code to Open Library Windows
 - command 81
- add to right click menu 15
- AddBlockToClipboard 319
- AddBlockToSelection 319
- AddC 219
- ADO functions 380
- ADO_AddRecords 380
- ADO_CheckCompatibleFieldType 381
- ADO_Close 381
- ADO_CreateTable 381
- ADO_DeleteRecords 381
- ADO_ExecuteNonQuery 381
- ADO_ExecuteQuery 381
- ADO_GetFields 381
- ADO_GetNumFields 381
- ADO_GetNumRows 381
- ADO_GetNumTables 381
- ADO_GetTableColumns 382
- ADO_GetTables 382
- ADO_OpenConnection 382
- ADO_SetTableColumns 382
- ADO_Setup 382
- ADO_SQLServerGetServers 382
- ADO_SQLServerGetDatabases 382
- AdviseReceive 212
- alert functions 256
- Alt key 258
- alternate views 59
- ambient animations 140
- animation
 - 2D 120
 - 2D functions 259
 - 3D 128
 - 3D functions 264
- animation object (2D) 121
- animation object (3D) 132
 - ambient animation 140
 - buildings 149
 - classes 134
 - collision 138

- creating a new 142
- creating new 143
- exporting 143
- ExtendBase 134
- ExtendItems 134
- ExtendPlayers 134
- ExtendVehicles 134
- gravity 138
- GroupTags 133
- interiors 149
- mounting 137
- mountStack 137
- objectID 132
- people 141
- rotation 135
- scale 134
- skins 135
- sound 138
- unmounting 138
- userTag 133
- animation object tool 21
- AnimationBlockToBlock 126, 260, 264
- AnimationBorder 260
- AnimationBorderColor 260
- AnimationColor 121, 127, 261
- AnimationGetHeight 261
- AnimationGetLeft 261
- AnimationGetLeftRelative 261
- AnimationGetSpeed 261
- AnimationGetTop 261
- AnimationGetTopRelative 261
- AnimationGetWidth 261
- AnimationHide 123, 261
- AnimationLevel 124, 261
- AnimationMoveTo 123, 261
- AnimationMovie 262
- AnimationMovieFinish 262
- AnimationObjectCopyData 262
- AnimationObjectCreate 121, 262
- AnimationObjectDelete 262
- AnimationObjectExists 262
- AnimationObjectExists2 262
- AnimationOn 122, 200
- AnimationOval 121, 262
- AnimationPicture 263
- AnimationPixelRect 263
- AnimationPixelSet 263
- AnimationPoly 120, 121, 263
- AnimationRectangle 263
- AnimationRndRectangle 121, 263
- AnimationSetDelayMode 263
- AnimationSetSpeed 263
- AnimationShow 121, 123, 263
- AnimationStatus 206
- AnimationStretchTo 125, 263
- AnimationText 127, 264
- AnimationTextAlign 264
- AnimationTextSize 264
- AnimationTextTransparent 127, 264
- AntitheticRandomVariates 200
- AppendDataTableLabels 298
- AppendPopupLabels 292
- AppleEvents 239
- ApplicationFrame 319
- arguments
 - arrays as 67, 75
 - data type expectations 217
 - of function calls (converting) 65
 - pass by value or reference 98
- array segment 67
- ArrayDataMove 353
- ArrayLabelParse 370
- arrays 66
 - array segment 67
 - as arguments 67, 75
 - copying using FOR loops 104
 - definition 28
 - dimensions 66
 - disposing 100
 - disposing of global 103
 - dynamic 66
 - fixed 66
 - functions 352
 - global arrays 68, 103
 - import function 104
 - in discrete event modeling 160
 - memory usage 97
 - passing 99, 354
 - passing (precautions) 100
 - subscripts 66
 - working with 97
- ASCII value table 402
- Asin 219
- ASP license
 - form-based interface 118
- assignment operators 69

- Atan 219
- Atan2 219
- AttribInfo 211
- AttribType array 167
- AttributeList array 166
- attributes
 - arrays for 166
 - functions 372
- AttribValues array 167
- AudioClose3d 138
- AudioCloseLooping3d 138
- audioProfile 140
- automation
 - methods 112
- automation, OLE
 - client 112
 - server 112
- AutoScaleX 328
- AutoScaleY 328

B

- BarGraph 328
- basic math functions 217
- Beep 257
- bidirectional connectors 96
- Bidirectional Flows model 96
- Binomial distribution function 220
- BitAnd 228
- BitClr 228
- bit-handling functions 228
- bitmap pictures 86, 408
- BitNot 228
- BitOr 228
- BitSet 228
- BitShift 228
- BitTst 228
- BitXor 228
- BLANK 64, 76
- block
 - compile command 405
- block number 285
- block parts
 - animation 6
 - connectors 6
 - dialog 6
 - dialog items 6, 11
 - Help button 6
 - help text 6
 - icon 6
 - labels 6
 - ModL code 6
 - name 6
 - structure window 10
 - tabs 6
- block status messages 207
- block to block messages 211
- BlockClick 207, 258
- BlockDialogIsOpen 309
- blocked message 170
- BlockIdentify 207
- blocking the flow 170
- BlockLabel 207
- BlockMove 207
- BlockMsg 112, 116
- BlockName 107, 283
- BlockRead 207, 300
- BlockReceive 211
- BlockReceive0 163
- BlockReceive3 163
- BlockRect 283
- BlockReport 155, 160, 205, 326
- BlockRightClick 207
- blocks
 - 2D animation between 126
 - adding connectors 46
 - animating in 2D 54
 - categories 57
 - communicating with each other 93
 - communication 93, 282, 285, 291, 311
 - compiling 48
 - creating 44
 - creating an icon 46
 - data table functions 297
 - dialog items 33
 - dialog tab functions 310
 - dialogs 7, 309
 - dynamic text functions 304
 - equation-based 90
 - Executive 156
 - for debugging 182
 - general description 6
 - help text 22
 - hierarchical (animating) 122
 - icon class and view functions 314
 - icons 18
 - intermediate results 53

- labels 282
- labels (length) 398
- Make Your Own 160
- messaging functions 311
- ModL code 18
- names 282
- names (length) 398
- new 44
- numbers 282, 285
- parts of a block 6
- popup menus 57
- programming discrete event 160
- programming discrete rate 178
- protecting 86
- red border for debugging mode 186
- registered 106
- remote communication 93
- structure 6
- submenus in library 57
- that don't post future events 163
- that post events 162
- types of discrete event 162
- BlockSelect 207
- BlockSimFinishPriority 312
- BlockSimStartPriority 311
- BlockTableInfo 211
- BlockUndelete 207
- BlockUnselect 207
- Boid object DataBlock 139
- Boolean operators 70
- Break statement 70, 73
- breakpoints
 - condition 185
 - definition 184
 - disabling 193
 - margin 186
 - removing 193
 - setting 190
 - setting conditions 191
 - window 195
- Breakpoints window 195
- Buffered mode 129, 130
- button
 - code completion 11, 31
 - creating 54
 - dialog item 12
 - message 210
 - titles 12
 - titles (changing) 36, 92

- Buttons block 90

C

- C to Pascal string function 63
- C++ 40
- CalcDate 375
- CalcFV 223
- CalcNPER 223
- CalcPMT 224
- CalcPV 224
- CalcRate 224
- calendar 13
- calendar date
 - functions 373
- Call Chain pane 186
- CallHelp 378
- Cancel 209
- case sensitivity 62
- CASE statement 72
- categories of blocks 57
- Category 146, 147
- Ceil 217, 218
- CellAccept 209
- ChangeAxisValues 328
- ChangePlotType 328
- ChangePreference 320
- ChangePreferenceString 320
- ChangeSignalColor 328
- ChangeSignalSymbol 328
- ChangeSignalWidth 328
- changing parameters globally 92
- check box 12
 - programming for 35
 - titles (changing) 92
- CheckData 152, 154, 157, 204
- classes of 3D objects 134
- ClassName 147
- ClearBlock 320
- ClearBlockUndo 320
- ClearConnection 320
- ClearStatistics 211
- ClearUndo 320
- client application 408
- CloneCreate 320
- CloneDelete 321
- clone-drop 93

- code for a custom stand-alone block 94
- CloneFind 321
- CloneGetDialogItem 321
- CloneGetDialogItemLabel 321
- CloneGetInfo 321
- CloneGetList 321
- CloneGetPosition 321
- CloneHideDisable 321
- CloneInit 207
- CloneResize 321
- CloseBlockDialogBox 309
- CloseDialogBox 310
- CloseEnclosingHBlock 310
- CloseEnclosingHBlock2 310
- CloseModel 206
- ClosePlotter 328
- ClosePlotter2 328
- CM 80
- cm extension 81
- code
 - code completion 11, 31
 - conventions 49
 - example 47
 - introduction to ModL 18
 - language overview 24
 - layout 29
 - management 80
 - structure window 45
 - type declarations 30, 62
- code completion 31
 - button 11, 31
 - F8 31
 - for functions 217
- code management 80, 81
- code pane 11
- CodeExecute 322
- coding conventions 49
- color 122, 127, 260
 - HSV 122
- color palette 260
- colored dialog items 56
- column separators 232
- column tag
 - functions 305
- column tags 58
- COM DLL example 111
- comments
 - multi-line 29
 - single line 29
 - syntax coloring 30
- compile
 - conditionally 26, 76, 79
- Compile Block command 405
- complex number function 219
- ConArrayChanged 210
- ConArrayChangedComplete 211
- ConArrayChangedWhichCon 289
- ConArrayCollapseChanged 211
- ConArrayGetCollapsed 289
- ConArrayGetConNumber 289
- ConArrayGetDirection 289
- ConArrayGetNthCon 289
- ConArrayGetNthCon2 289
- ConArrayGetNumCons 289
- ConArrayGetOwnerCon 289
- ConArrayGetTotalCons 289
- ConArrayGetValue 290
- ConArrayMsgFromCon 290
- ConArraySendMsgToAllCons 290
- ConArraySendMsgToInputs 290
- ConArraySendMsgToOutputs 290
- ConArraySetCollapsed 290
- ConArraySetNumCons 290
- ConArraySetValue 290
- concatenation 69, 70, 257
- Concurrent mode 130
- conditional breakpoint dialog 196
- conditional compilation 26, 76, 79
- ConjugateC 225
- ConnectionBreak 211
- ConnectionClick 211
- ConnectionMake 207, 211
- connector messages 168, 210
- connector names
 - used as variables in ModL 32
- connector pane 11
- connector tool tips 291
- ConnectorLabelsGet 290
- ConnectorLabelsSet 291
- ConnectorMsgBreak 312
- ConnectorName 211
- ConnectorRightClick 207, 211
- connectors 32
 - adding 21, 46
 - aligning 47

- bidirectional 96
- changing 96
- changing an input to an output 47
- changing the type of 21
- checking in CheckData 204
- deleting 96
- functions 285
- initializing 97
- labels 58
- messages 168, 173
- naming 21, 32
- No Resize Bar 20
- normal 21
- normal vs variable 20
- pane 21
- renaming 47
- tool tips 58
- tools 20
- types 20
- User Defined 20
- variable 20, 21, 32, 58, 96
- ConnectorShowHide 207
- ConnectorToolTip 211
- ConnectorToolTipGet 291
- ConnectorToolTipSet 291
- ConnectorToolTipWhich 291
- constant definitions 29
- constants 63, 76
 - definition 28, 64
 - names 62
- Continue statement 70, 73
- Continue tool 194
- ContinueSim 154, 157, 205
- control 38
- control statements 70, 73
- conversion
 - of numeric types 65
- ConvertADODataType 382
- ConvertExtendSimDataType 383
- ConvertTimeUnits 373
- CopyBlock 207
- copying
 - dialog items 16
- Cos 219
- Cosh 219
- cost array 164
- costing attributes 167
- CreateBlock 208

- CreateFolder 234
- CreateHBlock 322
- CreatePopupMenu 292
- cross-platform compatibility 405
- cross-platform development 408
- CurrentScenario 200
- CurrentSense 200
- CurrentSim 154, 157, 200
- CurrentStep 155, 200
- CurrentTime
 - changed by Executive block 200

D

- data
 - checking in CheckData 204
 - consumption by type of variable 64
 - linking to databases 57
 - linking to global arrays 57
 - managing data in complex models 105
 - repository 105
- data consumption 64
- data tables 13, 36
 - column tags 58
 - functions 297
- data types 30, 62
 - definition 28
- Database
 - Read/Write Index Checking 333
- database errors
 - no such parent 333
 - no such record 333
 - not linked error 334
 - not unique error 333
 - not unique index 334
- database functions 333
- Databases (ExtendSim)
 - accessing 105
 - creating 105
- databases (ExtendSim)
 - _character at beginning of name 106
 - copying functions 337
 - creating/deleting functions 334
 - DB address functions 349
 - functions 333
 - import/export functions 337
 - linking/notify functions 351
 - no such parent error 333
 - no such record error 333

- not linked error 334
- not unique error 333
- not unique index error 334
- number of databases per model 398
- properties functions 338
- random data functions 345
- read/write functions 341
- registering blocks 57, 302
- reserved 106
- selecting functions 336
- sort/search functions 347
- viewing functions 350
- working with 105
- DataBlock
 - audioProfile 140
 - definition 144
 - types 145
 - variables 145, 146
- DataBlocks
 - audioProfile 140
- DataTableHover 209
- DataTableResize 209
- DataTableScrolled 209
- date functions (legacy) 375
- DateToString 375
- DB_FieldGetTypeString 383
- DBAddressCreate 349
- DBAddressGetAllIndexes 349
- DBAddressGetAsString 349
- DBAddressGetDlg 350
- DBAddressGetDlg2 350
- DBAddressGetFromString 350
- DBAddressIncrementIndex 350
- DBAddressReplaceIndex 350
- DBBlockRegister 106
- DBBlockRegisterContent 351
- DBBlockRegisterContents 106
- DBBlockRegisterStructure 106, 352
- DBBlockUnregisterContent 352
- DBBlockUnregisterStructure 352
- DBChildPopupSelector 336
- DBDatabaseCloseViewer 351
- DBDatabaseCopy 337
- DBDatabaseCreate 334
- DBDatabaseDelete 334
- DBDatabaseDeleteByIndex 334
- DBDatabaseExists 338
- DBDatabaseExport 337
- DBDatabaseGetIndex 338
- DBDatabaseGetIndexFromAddress 350
- DBDatabaseGetName 338
- DBDatabaseImport 337
- DBDatabaseOpenViewer 350
- DBDatabaseOpenViewerToTab 351
- DBDatabasePopupSelector 336
- DBDatabaseRename 338
- DBDatabasesGetNum 338
- DBDatabaseShowHideReserved 338
- DBDatabaseTabChangeName 338
- DBDatabaseTabDelete 334
- DBDataGetAsNumber 341
- DBDataGetAsNumberParentAltField 341
- DBDataGetAsNumberUsingAddress 341
- DBDataGetAsString 341
- DBDataGetAsStringParentAltField 342
- DBDataGetAsStringUsingAddress 342
- DBDataGetCurrentSeed 345
- DBDataGetDateAsSimTime 342
- DBDataGetDateAsSimTimeUsingAddress 342
- DBDataGetParent 342
- DBDataGetParentUsingAddress 343
- DBDataSetAsNumber 343
- DBDataSetAsNumberReserved 343
- DBDataSetAsNumberUsingAddress 343
- DBDataSetAsNumberUsingAddressReserved 343
- DBDataSetAsParentIndex 343
- DBDataSetAsParentIndexReserved 344
- DBDataSetAsParentIndexUsingAddress 344
- DBDataSetAsParentIndexUsingAddressReserved 344
- DBDataSetAsString 344
- DBDataSetAsStringReserved 344
- DBDataSetAsStringUsingAddress 344
- DBDataSetAsStringUsingAddressReserved 344
- DBDataSetCurrentSeed 345
- DBDataSetDateAsSimTime 345
- DBDataSetDateAsSimTimeReserved 345
- DBDataSetDateAsSimTimeUsingAddress 345
- DBDataSetDateAsSimTimeUsingAddressReserved 345
- DBDatatable 352
- DBFieldCreate 334
- DBFieldCreateByIndex 334

- DBFieldDelete 335
- DBFieldDeleteByIndex 335
- DBFieldExists 339
- DBFieldGetIndex 339
- DBFieldGetIndexFromAddress 350
- DBFieldGetName 339
- DBFieldGetProperties 339
- DBFieldGetPropertiesUsingAddress 339
- DBFieldMove 339
- DBFieldPopupSelector 336
- DBFieldRename 339
- DBFieldSetInitialize 339
- DBFieldSetProperties 339
- DBFieldsGetNum 340
- DBinomial 220
- DBParameter 352
- DBRandomDistributionGet 346
- DBRandomDistributionSet 347
- DBRecordExists 340
- DBRecordFind 347
- DBRecordFindMultipleFields 348
- DBRecordFindMultipleFieldsArray 348
- DBRecordFindNumericalRange 348
- DBRecordFindParentRecordIndex 349
- DBRecordGetIndexFromAddress 350
- DBRecordIDFieldGetIndex 340
- DBRecordPopupSelector 336
- DBRecordsDelete 335
- DBRecordsGetNum 340
- DBRecordsInsert 335
- DBRelationCreate 335
- DBRelationDelete 335
- DBRelationsGetNames 340
- DBRelationsGetNum 340
- DBTableCloneToTab 335
- DBTableCopy 337
- DBTableCreate 335
- DBTableCreateByIndex 335
- DBTableDelete 336
- DBTableDeleteByIndex 336
- DBTableExists 340
- DBTableExportData 337
- DBTableGetIndex 340
- DBTableGetIndexFromAddress 350
- DBTableGetName 340
- DBTableGetProperties 340
- DBTableImportData 338
- DBTablePopupSelector 337
- DBTableRename 341
- DBTableSetProperties 340
- DBTablesGetNum 341
- DBTableSort 349
- DBTabletoGA 356
- DBToolTipsGet 336
- DBToolTipsSet 336
- DDE (dynamic data exchange) 239
- DE Modeling Using Equation Blocks 319
- Debug Tutorial model 185
- debugger 184
 - arrays 197
 - arrow 184
 - breakpoint margin 186
 - breakpoints 184
 - call chain pane 186
 - conditional breakpoint 196
 - location arrow 184, 186
 - red border around blocks 186
 - removing breakpoints 193
 - setting breakpoints 190
 - setting conditions 191, 196
 - source pane 186
 - toolbar 194
 - tutorial 185
 - variables pane 186
 - WatchPoint conditions 197
 - window 194
- debugger windows
 - conditions dialog 196
 - toolbar 194
- DebuggerBreakpoint 377
- debugging 182, 184
 - block code using UserError 257
 - blocks for 182
 - functions 376
 - functions using alerts and prompts 256
 - models 182
 - reporting 182
 - source code (see "debugger") 184
 - stepping 182
 - tips 197
 - using DebugMsg function 184
- DebugMsg 184, 377
- DebugWrite 184, 377
- decision blocks 162
- declaring types 64

- DEExecutiveArrayResize 211
- Delay 369
- delay line functions 369
- DelayInit 319, 369
- DeleteBlock 208
- DeleteBlock2 208
- delimiters 232
- DeltaTime 152, 154, 155, 200
- Determinant 225
- DeterminantC 225
- Develop command 44
- Develop menu keyboard equivalents 405
- DExponential 220
- DGamma 220
- diagnostic functions 256
- diagnostics 256
- dialog editor window 9, 45
 - printing 11
- dialog functions 297, 304, 309
- dialog items 7, 13, 33
 - add to right click menu option 15
 - buttons 12, 36
 - buttons (creating) 54
 - calendar 13
 - changing text and titles 91
 - check box 12
 - check boxes 35
 - color 17, 56
 - copying 16
 - data tables 36
 - deleting 51
 - display only option 14, 50
 - dynamic text 13, 34
 - editable text 13, 34
 - editable text 31 character limit 13
 - embedded objects 14, 39
 - formatted text or titles 17
 - formatted with color 17
 - formatting 16, 17
 - functions 291
 - grid 16
 - hide item option 15
 - hiding 92
 - linking to ExtendSim databases 57
 - linking to global arrays 57
 - list 12
 - messages 33, 209
 - meter 13, 38
 - move 56
 - moving 16, 92
 - names 14, 33
 - new 45
 - number formats for 15
 - parameter 13
 - parameter dialog items 34
 - parameter fields 13
 - parameters 50
 - popup menu 12
 - popup menus 37, 57
 - radio button 12, 48
 - radio buttons 35
 - show and hide 56
 - showing 92
 - slider 13, 38
 - static text 37
 - static text label 13
 - styles 17
 - switch 38
 - switch control 13
 - tabs 7, 51
 - text 14
 - text frame 13, 38, 50
 - text table 14
 - text tables 36
 - titles 14
 - tool tips 309
 - types of 12
 - visible in all tabs 51
 - visible in all tabs option 15
 - W and H coordinates 14, 16
 - X and Y coordinates 14, 16
- dialog messages 209
- dialog names
 - used as message names 33
 - used as variable names 33
- dialog tab functions 310
- dialog variables
 - getting and/or setting remotely 93
 - remote interface with 93
- DialogClick 210, 258, 292
- DialogClose 210
- DialogFixedSize 322
- DialogGetSize 310
- DialogHasEmbeddedObject 292
- DialogItem 210
- DialogItemRefresh 210
- DialogItemToolTip 210
- DialogItemVisible 292

- DialogMoveTo 310
- DialogOpen 210
- DialogPicture 264
- DialogRefresh 322
- dialogs
 - exploring 7
 - functions for opening and closing 309
 - programming tips 90
 - tabs 7
 - text (changing) 90
 - window for dialog editor 9
- DialogSetSize 310
- DIF format 149
- DiffDate 375
- DIGetID 292
- DILinkClear 302
- DILinkInfo 302
- DILinkingDisabled 302
- DILinkModify 303
- DILinkMsgs 303
- DILinkSendMsgs 303
- DILinkUpdateInfo 304
- DILinkUpdateString 304
- dimensions of an array 66
- DIMoveBy 292
- DIMoveTo 292
- DIMsgNumber 292
- DIParamTagGet 308
- DIParamTagSet 308
- DIParamTagStringGet 308
- DIPopupButton 293
- DIPositionGet 293
- DIPositionHome 293
- DIPositionSet 293
- DirPathFromPathName 234
- DisableDialogItem 293
- DisableDialogItem2 293
- DisableDTTabbing 298
- DisableTabName 311
- discrete event programming
 - arrays 160
 - blocked and query messages 170
 - continuous blocks 172
 - explicit connector messages 173
 - functions 174
 - how ExtendSim runs DE models 156
 - item data structures 164
 - item messaging 167
 - Make Your Own blocks 160
 - message emulation 172
 - notify message 171
 - overview 160
 - pseudocode of the simulation loop 156
 - pulling 169
 - pushing 168
 - scheduling events 161
 - SysGlobal variables 174
 - system variables 156
 - types of blocks 162
 - zero time events 163
- discrete rate programming 178
- DISetFocus 293
- display only option 14, 34, 50
- DisposeArray 67, 353
- DisposePlot 328
- distributions
 - Binomial function 220
 - Exponential function 220
 - Gamma function 220
 - Gaussian function 220
 - LogNormal function 220
 - Mean distribution function 220
 - Pascal function 220
 - Poisson function 220
 - Random function 220
 - RandomCalculate function 221
 - RandomCheckParam function 222
 - RandomGetModelSeedUsed function 222
 - RandomRead function 222
 - students t 223
 - TStatisticValue function 223
- DITitleGet 293
- DITitleSet 293
- DIToolTipSet 309
- DivC 219
- division by integer 69
- DLL functions 254
- DLLArray 256
- DLLBoolCFunction 255
- DLLBoolPascalFunction 255
- DLLBoolStdcallFunction 255
- DLLCtoPString 255
- DLLDoubleCFunction 255
- DLLDoublePascalFunction 255
- DLLDoubleStdcallFunction 255
- DLLLoadLibrary 255

- DLLLongCFunction 255
- DLLLongPascalFunction 255
- DLLLongStdCallFunction 255
- DLLMakeProcInstance 256
- DLLParam 256
- DLLPtoCString 255
- DLLs 83, 254, 408
 - example code 84
 - interface 83
- DLLVoidCFunction 256
- DLLVoidPascalFunction 256
- DLLVoidStdcallFunction 256
- DLLXCMD 256
- DLogNormal 220
- dlopen 85
- dlsym 85
- double 217
- double (data type) 30, 62
- Do-While 71, 72
- DPascal 220
- DPoisson 220
- DragCloneToBlock 94, 208
- drawing tools 18
- drivers (cross-platform) 408
- DTColumnTagGet 308
- DTColumnTagSet 309
- DTColumnTagStringGet 309
- DTGrowButton 298
- DTHasDDELink 299
- DTHideLinkButton 304
- DTPaneFixed 299
- DTS (Dynamix Three Space) format 142
- DTS exporters 143
- DTS format 143
- DTToolTipSet 299
- DuplicateBlock 322
- DuringContinue 314
- DuringHBlockUpdate 312
- dylib extension 85
- dynamic arrays 66, 352
 - functions 352
 - number per block 398
- dynamic data exchange (DDE) 239
- dynamic data link functions 302
- dynamic data link messages 212
- dynamic data linking 57
- Dynamic Link Libraries (DLLs) 83, 254, 408

- dynamic text 13, 34
 - add to right click menu 15
 - display only 14
 - display only option 14
 - functions 304
 - hide item 15
 - visible in all tabs 15
- DynamicDataTable 299
- DynamicDataTable2 299
- DynamicDataTableVariableColumns 67, 297, 299
- DynamicDataTableVariableColumns2 299
- DynamicTextArray 305
- DynamicTextIsDirty 305
- DynamicTextSetDirty 305
- Dynamix Three Space format 142

E

- E (scientific) notation
 - definition 28
- E3D environment 129
 - definition 129
- E3D window 128, 144
- E3DAddMarkerToPath 268
- E3DAnimationName 274
- E3DAnimationPlay 141, 274
- E3DAnimationSetPosition 275
- E3DAnimationStop 141, 275
- E3DBlockToBlock 265
- E3DBlockToBlockHeight 265
- E3DBlockToBlockObjectLabel 265
- E3DBlockToBlockRotation 265
- E3DBlockToBlockScale 265
- E3DBlockToBlockSkin 265
- E3DBlockToBlockSkinByIndex 265
- E3DBoundingBoxes 269
- E3DChangeObject 270
- E3DChooseEnvironment 269
- E3DClose 213
- E3DCollision 213
- E3DCollisionBlocker 282
- E3DCollisionShow 280
- E3DCopyObject 270
- E3DCreateMarker 268
- E3DCreateObject 270
- E3DCreateObjectAtWayPoint 271
- E3DCreatePath 268

E3DCreateWayPoint 271
E3DCurrentTime 266
E3DDeleteAllObjects 134, 271
E3DDeleteObject 271
E3DDialogPicture 275
E3DDistance 277
E3DDistancePath 277
E3DdistanceRatio 266
E3DDistanceToObject 277
E3DExec 281
E3DExecFile 281
E3DFinish 213
E3DForwardAngle 279
E3DForwardVector 279
E3DGetAngleFromVector 279
E3DGetBlockLocation 265
E3DGetCamera 282
E3DGetDestination 271
E3DGetMount 274
E3DGetMountedObject 274
E3DGetName 275
E3DGetObjectAnimationNames 275
E3DGetObjectBlockNumber 265
E3DGetObjectBounds 281
E3DGetObjectByName 275
E3DGetObjectIDs 281
E3DGetObjectInfo 281
E3DGetObjectLabel 275
E3DGetObjectList 276
E3DGetObjectNames 134, 276
E3DGetObjectSkinBase 277
E3DGetObjectSkinNames 277
E3DGetObjectTypes 281
E3DGetObjectVector 279
E3DGetPathByName 269
E3DGetPosition 267
E3DGetRotation 267
E3DGetSelectedObject 269
E3DGetSpeed 271
E3DGetVectorFromAngles 279
E3DInit 213
E3DLogEvents 282
E3DMode 266
E3DModeSwitch 213
E3DMountObject 274
E3DMountStackInsert 137, 268
E3DMountStackLimit 137, 268
E3DMountStackRemove 137, 268
E3DNeighborDetection 282
E3DNormalizeVector 279
E3DNotEnabled 267
E3DObjectClick 213
E3DObjectClicked 269
E3DObjectExists 282
E3DObjectIDToUserTag 280
E3DObjectLabelColorSet 276
E3DObjectMoved 213
E3DObjectPropertySet 281
E3DObjectSnapshot 275
E3DOpenWindow 267
E3DPanoramicScreenShot 277
E3DPathMarkerLocation 269
E3DPathSetColor 269
E3DPauseAllObjects 266
E3DPlaySound 282
E3DplaySound 138
E3DPostAnimationPlay 273
E3DPostAnimationSetPosition 273
E3DPostAnimationStop 273
E3DPostChangeObject 273
E3DPostCopyObject 273
E3DPostCreateObject 133, 273
E3DPostCreateObjectAtWayPoint 273
E3DPostDeleteAllObjects 273
E3DPostDeleteObject 273
E3DPostMountObject 273
E3DPostMountStackInsert 273
E3DPostMountStackRemove 273
E3DPostObjectPropertySet 273
E3DPostResumeSpeed 273
E3DPostRotateTimed 273
E3DPostSetDestination 273
E3DPostSetObjectBlockNumber 273
E3DPostSetObjectLabel 273
E3DPostSetPosition 273
E3DPostSetRotation 273
E3DPostSetScale 273
E3DPostSetSkin 136, 273
E3DPostSetSkinByIndex 273
E3DPostSetSpeed 273
E3DPostSetTargetObject 273
E3DPostSetTargetPath 274

- E3DPostSetTargetWayPoint 274
- E3DPostSetVisibility 274
- E3DPostStopAllObjects 274
- E3DPostUnmountObject 274
- E3DReachDestination 213
- E3DResumeAllObjects 266
- E3DResumeSpeed 271
- E3DScreenShot 277
- E3DSetBlockLocation 265
- E3DSetDestination 271
- E3DSetDetail 269
- E3DSetMode 266
- E3DSetName 276
- E3DSetObjectBlockNumber 266
- E3DSetObjectLabel 132, 274, 276
- E3DSetPosition 267
- E3DSetRotation 267
- E3DSetScale 267
- E3DSetSelectedObject 269
- E3DSetSkin 136, 278
- E3DSetSkinByIndex 278
- E3DSetSpeed 272
- E3DSetTargetObject 272
- E3DSetTargetPath 272
- E3DSetTargetWayPoint 272
- E3DSetTerrain 270
- E3DSetTimeRatio 266
- E3DSetVisibility 281
- E3DSky 270
- E3Dsleap 266
- E3DStart 213
- E3DStartTick 280
- E3DStopAllObjects 272
- E3DStopTick 280
- E3DSun 270
- E3DTick 213
- E3DTimeRatio 266
- E3DUnmountObject 274
- E3DUnselectObject 269
- E3DUpdate 267
- E3DUserTagToObjectID 280
- E3DUses3D 282
- E3DWindow 267
- E3DWindowConnected 267
- EDCalcDate 374
- EDCalendarDateGet 374
- EDCalendarDates 374
- EDCalendarShow 374
- EDConvertDate 374
- EDDateToSimTime 374
- EDDateToString 374
- EDDateValue 374
- EDDayOfTheWeek 374
- EDGetCurrentDate 374
- EDGetStartDate 374
- editable text 34
 - 31 character limit 13
 - add to right click menu 15
 - dialog item 13
 - display only 14, 34
 - hide item 15
 - visible in all tabs 15
- editable text 31 character 13
- EDNow 374
- EDSimTimeToDate 374
- EDStringToDate 375
- EigenValues 225
- Embed block 242
- embedded objects 14, 39, 111, 242
- endif 79
- EndSim 156, 160, 205
- EndTime 152, 154, 156, 200
- Enter Selection command 78
- entry boxes 13
- Equation block 90, 228, 407
- equation functions 228
- Equation(I) block 90
- equation-based blocks 2, 24, 90
 - user-defined functions 90
- EquationCalculate 229
- EquationCalculate20 229
- EquationCalculateDynamic 229
- EquationCalculateDynamicVariables 229
- EquationCompile 230
- EquationCompile20 230
- EquationCompileDynamic 230
- EquationCompileDynamicVariables 230
- EquationCompilePlatform 207
- EquationCompileSetStaticArray 230
- EquationDebugCalculate(231
- EquationDebugCompile 231
- EquationDebugDispose 231
- EquationDebugSetBreakpoints 231

- EquationGetStatic 231
- EquationIncludeSet 231
- equations 90
- EquationSetStatic 232
- Erf 217
- Euler integration 224
- evaluating expressions 70
- event
 - list 163
 - posting 162
 - scheduling 160, 161
- event clock 161
- Excel block 242
- Excel interface 118
- Execute 112, 114
- ExecuteMenuCommand 322
- Executive block 156, 312
 - controlling simulation 159
 - controlling timing 160
 - relationship to 3D animation 131
- Exp 217
- Exponential distribution function 220
- exponential number 28
- Export 233
- ExportText 233
- expressions
 - evaluating 70
- ExtendBase 134
- ExtendIPlayers 134
- ExtendItems 134
- ExtendMaximize 322
- ExtendMinimize 322
- ExtendSim
 - databases 105
 - upper limits 398
- ExtendVehicles 134
- extensions
 - converting 408
 - DLLs 83
 - folder 82
 - Macintosh 82
 - pictures 86
 - Shared Libraries 85
 - sounds 85
 - Windows 82
- external source code 41, 59, 80
 - command 80
 - source folder 81

F

- F8 (code completion) 31
- FALSE 76
- fast Fourier transform 217
- FFT 217
- file conversion 405, 406
- file format
 - pictures 408
- file I/O functions 232, 233
- file names 404
- FileChoose 234
- FileClose 234
- FileDelete 234
- FileEndOfFile 234
- FileExists 234
- FileGetDelimiter 234
- FileGetPathName 234
- FileInfo 234
- FileIsOpen 234
- FileNameFromPathName 234
- FileNew 235
- FileOpen 235
- FileRead 235
- FileRewind 235
- files
 - transferring 405
- FileWrite 235
- FinalCalc 155, 159, 205
- financial functions 223
- Find command 78
- FindBlock 322
- FindInHierarchy 283
- FindInHierarchy2 283
- FindMinimum 353
- FindMinimumWithThreshold 353
- FindNext 322
- FixDecimal 218
- fixed arrays 66
- Floor 218
- flow order 201
- FlowBlockReceiveN 212
- For construct 71, 72
- formats
 - DIF 149
 - DTS 143
 - number formats 15

- of 3D files 142
- FormatString 370
- FormatStringReal 370
- formatted text 17
- formatted titles 17
- formatting
 - dialog items with color 17
- form-based interface 118
- Fortran 26, 40
- forward declaration 74
- free() 354
- FreeMemory 377
- friction 138
- function popup 20
- functions 215
 - 2D 259
 - 3D animation 264, 274
 - 3D block 265
 - 3D creation, deletion 270
 - 3D distance 277
 - 3D environment 269
 - 3D miscellaneous 282
 - 3D mounting 274
 - 3D mountStack 267
 - 3D movement 271
 - 3D name 275
 - 3D object 269
 - 3D path 268
 - 3D post 272
 - 3D properties 280
 - 3D rotation, position, scale 266
 - 3D screenshot 277
 - 3D script 281
 - 3D skin 277
 - 3D tick 280
 - 3D time 266
 - 3D user tag 279
 - 3D vector 279
 - 3D window 267
 - alerts, prompts 256
 - animation 2D 259
 - animation 3D 264
 - AppleEvents 239
 - arrays as arguments to 67
 - attributes 372
 - bit-handling 228
 - block numbers 282
 - calendar date 373
 - column tag 305
 - connections 285
 - data tables 297
 - database 333
 - database copying 337
 - database creating/deleting 334
 - database DB address 349
 - database import/export 337
 - database linking/notify 351
 - database properties 338
 - database random data 345
 - database read/write 341
 - database selecting 336
 - database sort/search 347
 - database viewing 350
 - date (egacy) 375
 - DDE 239
 - debugging 376
 - definition 28, 31
 - delay line 369
 - diagnostic 256
 - dialog 309
 - dialog items 291
 - dialog tabs 310
 - distributions 219
 - DLLs 254
 - dynamic arrays 352
 - dynamic data linking 302
 - dynamic text 304
 - equation 228
 - file I/O, formatted 232
 - file I/O, unformatted 233
 - financial 223
 - global 108
 - help 378
 - icon class and view 314
 - integration 224
 - internet access 236
 - Interprocess communication (IPC) 239
 - labels 282
 - linked list 361
 - Mailslots 249
 - math 217
 - matrix 224
 - messaging blocks 311
 - models 314
 - names 62, 282
 - netlist 285
 - notebook 314
 - ODBC 250
 - OLE 241

- overriding 32
- overriding (syntax coloring) 30
- passing arrays 354
- plotting 326
- pointers 354
- queue 368
- recursive 74
- returns 217
- scripting 319
- serial I/O 253
- Shared Libraries 254
- statistical 219
- string 369
- text files 232, 233
- time (legacy) 375
- time units 372
- timer 376
- trigonometric 219
- types 216
- user-defined 73, 98
- user-defined (syntax coloring) 30
- web 378

future value 223

G

- GABlockRegister 106
- GABlockRegisterContent 356
- GABlockRegisterContents 106
- GABlockRegisterStructure 106, 356
- GABlockUnregisterContent 356
- GABlockUnregisterStructure 356
- GAClipboard 356
- GACopyArray 356
- GACreate 356
- GACreateQuick 357
- GACreateRandom 357
- GADataTable 357
- GADeleteRow 357
- GADispose 357
- GADisposeByIndex 357
- GAExport 357
- GAFindStringAny 357
- GAGetArray 357
- GAGetColumnsByIndex 358
- GAGetColumns 357
- GAGetIndex 358
- GAGetInfo 358
- GAGetInitValue 358
- GAGetInteger 358, 360
- GAGetLong 358
- GAGetName 358
- GAGetReal 358
- GAGetRows 358
- GAGetRowsByIndex 358
- GAGetString 358
- GAGetString15 358
- GAGetString31 358
- GAGetString63 359
- GAGetType 359
- GAGetTypeByIndex 359
- GAImport 359
- GAInitializing 359
- GAInsertRow 359
- GALastUsedIndex 359
- Gamma distribution function 220
- GammaFunction 218
- GAMultisim 359
- GANonSaving 359
- GAParameter 359
- GAPopupMenu 359
- GAPtr 360
- GAResize 360
- GAResizeByIndex 360
- GAsearch 360
- GAsearchCount 360
- GASetArray 360
- GASetInitValue 360
- GASetInteger 360
- GASetReal 361
- GASetString 361
- GASetstring15 361
- GASetstring31 361
- GASetString63 361
- GA Sort 361
- GAtoDBTable 361
- Gaussian 220
- Gaussian distribution function 220
- GetAppPath 322
- GetAttributeValue 372
- GetAxis 328
- GetAxisName 329
- GetBlockDates 375
- GetBlockInfo 314
- GetBlockLabel 284

GetBlockName 286
 GetBlockPosition 207
 GetBlockType 284
 GetBlockTypePosition 284, 323
 GetConBlocks 126, 286
 GetConHBlocks 286
 GetConName 286
 GetConnectedTextBlock 286
 GetConnectedType 286
 GetConnectedType2 286
 GetConnectionColor 286
 GetConnectorMsgsFirst 312
 GetConnectorPosition 287
 GetConnectorType 287
 GetConNumber 126, 287
 GetCurrentPlatform 378
 GetCurrentTabName 311
 GetDataTableSelection 300
 GetDialogColors 294
 GetDialogItemColor 294
 GetDialogItemInfo 294
 GetDialogItemLabel 294
 GetDialogNames 294
 GetDialogVariable 92, 294
 GetDimension 67, 353
 GetDimensionByName 67, 353
 GetDimensionColumns 67, 353
 GetDimensionColumnsByName 67, 353
 GetDraggedCloneList 94, 295
 GetDTOffset 300
 GetEnclosingHBlockCon 287
 GetEnclosingHBlockNum 284
 GetEnclosingHBlockNum2 284
 GetExtendType 379
 GetExtendVersion 379
 GetExtendVersionString 379
 GetFileReadMachineType 236
 GetFileReadVersion 207, 300, 379
 GetFileReadVersionString 379
 GetFront 368
 GetGlobalSimulationOrder 314
 GetIndexedConValue 287
 GetIndexedConValue2 287
 GetIntermediateBlocks 287
 GetItem 174
 GetLibraryContents 314
 GetLibraryInfo 315
 GetLibraryPathName 315
 GetLibraryStringInfo 315
 GetLibraryVersion 315
 GetLibraryVersionByName 315
 GetModelName 315
 GetModelPath 315
 GetModifierKey 258
 GetMouseX 207, 258, 323
 GetMouseXActiveWindow 258
 GetMouseY 207, 258, 323
 GetMouseYActiveWindow 258
 GetMsgSendingBlock 312
 GetNumCons 287
 GetObjectHandle 112, 117
 GetPassedArray 98, 100, 354
 GetPreference 323
 GetRear 368
 GetRecentFilePath 323
 GetReportType 326
 GetRightClickedCon 287
 GetRunParameter 315
 GetRunParameterLong 316
 GetSerialNumber 316
 GetSignalName 329
 GetSignalValue 329
 GetSimulateMsgs 312
 GetSimulationPhase 316
 GetStaticNames 284
 GetSystemColor 295
 GetTickCount 329
 GetTimeUnits 373
 GetUserPath 323
 GetWindowsHndl 316
 GetWorksheetFrame 323
 GetY1Y2Axis 329
 global arrays 68
 AttribType 167
 AttributeList 166
 AttribValues 167
 functions 355
 working with 103
 global numbers 282
 global variables
 definition 28
 for passing messages 108
 list 201

- scope 29
- SysGlobals 174
- use during Simulate 174
- use during CheckData or InitSim 178
- GlobalProofStr 201
- GlobalToLocal 284
- goal seeking 110
- GOTO statement 71, 73
- grid
 - dialog item 16
 - overriding for icon movement 18
- groups
 - for radio buttons 12
- GroupTags 133

H

- HasFront 146
- HBlockClicked 258
- HBlockClose 208
- HBlockFromLibrary 208
- HBlockHelpButton 208
- HBlockMove 208
- HBlockOpen 208
- HBlockSaveToLibrary 208
- HBlockUnlinkFromLibrary 323
- HBlockUpdate 208
- header files 79
- help 378
 - changing text 22
 - functions 378
 - getting technical support 3
 - pane 10
- HelpButton 208
- hide item 15
- HideDialogItem 295
- HideDialogItem2 295
- hiding dialog items 56, 92
- hierarchical blocks
 - animation 122
 - library (converting) 407
- HSV 122, 260

I

- icon
 - alternate views 59
 - animating in 2D 54
 - block part 6
 - building for Miles block 46
 - creating 18
 - functions for class and view 314
 - grid 18
 - pane 10
 - red border around block 186
 - showing a picture on 125
 - tools 20
 - views 19, 59
- icon class and view
 - functions 314
- Icon Tools popup 20
- icon views
 - view order 59
- IconGetClass 314
- IconGetView 314
- IconGetViewName 314
- IconSetViewByIndex 314
- IconSetViewByPartialName 314
- IconViewChange 208
- IDE 6
- identifier
 - definition 28
- Identity 225
- IdentityC 225
- IdleSound 146
- IdleSoundAnimation 147
- IdleSoundVol 147
- idleSoundVol 140
- IEEE floating point numbers 62
- IF statement 71
- ifdef 79
- If-Else statement 71, 72
- ifndef 79
- Imagine That, Inc. 3
- immediate functions 131
- Import 233
- Import function 104
- ImportText 233
- include files
 - definition 58
 - MouseClicked 94
 - New Include File command 79
 - referencing 79
 - Save Include File command 79
- include popup 10
- IncludeFileEditor 232
- index of items 164

- INetCloseHandle 236
- INetConnect 236
- INetFileImportText 236
- INetFindNextFile 237
- INetFTPCreateDirectory 237
- INetFTPDeleteFile 237
- INetFTPExport 237
- INetFTPExportGA 237
- INetFTPExportText 237
- INetFTPFindFirstFile 237
- INetFTPGetCurrentDirectory 237
- INetFTPGetFile 238
- INetFTPImport 238
- INetFTPImportGA 238
- INetFTPImportText 238
- INetFTPPutFile 238
- INetFTPRemoveDirectory 238
- INetFTPRenameFile 238
- INetFTPSetCurrentDirectory 238
- INetGetFindFileInfo 239
- INetGetFindFileName 239
- INetOpenSession 239
- INetOpenURL 239
- initializing connectors 97
- InitSim 154, 158, 205
- Inner 225
- InnerC 225
- InstallArray 329
- InstallAxis 330
- InstallFunction 330
- Int 218
- integer
 - division of an 69
 - function return 217
 - maximum value 62
 - to real 65
 - to string 66
- integer (data type) 30, 62
- integer array for SysGlobal 165
- Integerabs 218
- integrated development environment 6
- IntegrateEuler 224
- IntegrateInit 224
- IntegrateTrap 224
- integration functions 224
- interface 118
- internet access functions 236

- Interprocess communication (IPC)
 - cross-platform development 408
 - functions 239
- IPCAdvise 239
- IPCCheckConversation 239
- IPCConnect 239, 409
- IPCDisconnect 240
- IPCExecute 240, 409
- IPCGetDocName 240
- IPCLaunch 240
- IPCOpenFile 240
- IPCoke 240, 409
- IPCokeArray 240
- IPCRequest 240, 409
- IPCRequestArray 241
- IPCSendCalcReceive 241
- IPCServerAsync 241
- IPCSetTimeOut 241
- IPCSpreadSheetName 241
- IPCStopAdvise 241
- IsBlockSelected 323
- IsConVisible 287
- IsFirstReport 326
- isKeyDown 258
- IsLibEnabled 323
- IsMenuItemOn 323
- IsMetric 323
- IsSimulationPaused 316
- item
 - array 164
 - priority 164
 - quantity 164
- item index 164
- Item library
 - programming 160
- itemArray arrays 164
- itemArray3D
 - passed by SysGlobal12 166
- itemArrayC
 - passed by SysGlobal9 164
- itemArrayI
 - passed by SysGlobal4 165
- itemArrayI2 165
- itemArrayR
 - passed by SysGlobal3 164
- items
 - flow is blocked 170

- pulling 169
 - pushing 168
- ## J
- Java 26
- ## K
- keyboard shortcuts 404
 - for developers 405
 - keywords
 - colors 30
- ## L
- label 13
 - label functions 282
 - labels on connectors 58
 - LastBlockPlaced 323
 - LastKeyPressed 258
 - LastSetDialogVariableString 295
 - left to right order 201
 - LegoClose 379
 - LegoMotor 379
 - LegoSensorBlock 380
 - LegoSensorOff 380
 - LegoSensorOn 380
 - LegoSensorRead 380
 - LegoSound 380
 - Lib extension 404
 - libraries
 - converting 407
 - file name size 404
 - number of blocks per 398
 - protecting 86
 - size 398
 - source folder 81
 - transferring between platforms 407
 - window 9
 - LibraryGetInfoByName 323
 - limits of ExtendSim 398
 - Link Alerts 106
 - Link Contents 106
 - Link Structure 106
 - LinkContent 212
 - linked list functions 361
 - linked lists
 - sorting 68
 - LinkStructure 212
 - list of tables 334, 341, 342
 - ListAddElement 362
 - ListAddString63s 362
 - ListCopyElement 362
 - ListCreate 363
 - ListCreateElement 363
 - ListDeleteElement 363
 - ListDispose 363
 - ListDisposeAll 363
 - ListElementMinMax 363
 - ListGetCount 363
 - ListGetDouble 364
 - ListGetElements 364
 - ListGetIndex 364
 - ListGetInfo 364
 - ListGetLong 364
 - ListGetName 364
 - ListGetString 364
 - ListLastElementIndex 364
 - ListLocked 365
 - ListSearch 365
 - ListSearchCount 365
 - ListSearchCountLongs 365
 - ListSearchLongs 365
 - ListSetDouble 365
 - ListSetLong 366
 - ListSetName 366
 - ListSetSort 366
 - ListSetSort2 366
 - ListSetString 366
 - literal
 - definition 28
 - LIX extension 404
 - local numbers 282
 - local variables 29, 31, 63
 - definition 28
 - LocalNumBlocks 284
 - LocalToGlobal 285
 - LocalToGlobal2 285
 - location arrow 184, 186
 - Log 218
 - Log10 218
 - Log2 218
 - logical operators 70
 - LogNormal distribution function 220

- long 217
- long (data type) 30, 62
- LUdecomp 226
- LUdecompC 226

M

- Machine object DataBlock 139
- Macintosh
 - file conversion 406
 - keyboard shortcuts 404
- Macintosh to Windows 405
- MacWin Conversion command 407
- magnitude operators 70
- Mailslot functions 249
- MailSlotClose 249
- MailSlotCreate 249
- MailSlotRead 249
- MailSlotReceive 208, 249
- MailSlotSend 249
- Make Your Own block 174, 178
- Make Your Own blocks 160
- MakeArray 67, 105, 353
- MakeArray2 67, 354
- MakeBlockInvisible 324
- MakeConnection 324
- MakeDialogModal 310
- MakeFeedbackBlock 288
- MakeOptimizerBlock 285
- MakeScatter 333
- MakeSelectionHierarchical 208
- malloc() 354
- markers 141
- MatAdd 226
- MatAddC 226
- Match Braces command 405
- Match IFDEF/ENDIF 405
- MatCopy 226
- MatCopyC 226
- math functions 217
- math operators 69
- MatInvert 226
- MatInvertC 226
- MatMatProd 226
- MatMatProdC 226
- matrix functions 224
- MatScalarProd 227

- MatScalarProdC 227
- MatSub 227
- MatSubC 227
- MatVectorProd 227
- MatVectorProdC 227
- Max2 218
- Mean 220
- memory usage when programming 97
- menu command shortcuts 404
 - for developers 405
- message emulation 172
- message handlers 18
 - 3D 212
 - block status 207
 - block to block 211
 - connector messages 210
 - constants 313
 - definition 28, 31
 - dialog messages 33, 209
 - dynamic data link 212
 - example 31
 - format 75
 - model status 205
 - OLE 212
 - overriding 32, 76
 - overriding (syntax coloring 30
 - purpose during DE initialization loop 158
 - purpose during initialization loop 154
 - Simulation Messages 204
 - types 39, 204
 - use during DE initialization loop 158
 - use during initialization loop 154
- message name 33
- messages 108
 - 3D 212
 - AbortDialogMessage 209
 - ActivateModel 206
 - AdviseReceive 212
 - AnimationStatus 206
 - AttribInfo 211
 - block status 207
 - block to block 211
 - BlockClick 207
 - BlockIdentify 207
 - BlockLabel 207
 - BlockMove 207
 - BlockRead 207
 - BlockReceive 211
 - BlockReport 205

BlockRightClick 207
 BlockSelect 207
 BlockTableInfo 211
 BlockUndelete 207
 BlockUnselect 207
 Button 210
 Cancel 209
 CellAccept 209
 CheckData 204
 ClearStatistics 211
 CloneInit 207
 CloseModel 206
 ConArrayChanged 210
 ConArrayChangedComplete 211
 ConArrayCollapseChanged 211
 ConnectionBreak 211
 ConnectionClick 211
 ConnectionMake 207, 211
 connector messages 210
 ConnectorName 211
 ConnectorRightClick 207, 211
 ConnectorShowHide 207
 ConnectorToolTip 211
 ContinueSim 205
 CopyBlock 207
 CreateBlock 208
 DataTableHover 209
 DataTableResize 209
 DataTableScrolled 209
 DEExecutiveArrayResize 211
 DeleteBlock 208
 DeleteBlock2 208
 DialogClick 210
 DialogClose 210
 DialogItem 210
 DialogItemRefresh 210
 DialogItemToolTip 210
 DialogOpen 210
 DragCloneToBlock 94, 208
 dynamic data link 212
 E3DClose 213
 E3DCollision 213
 E3DFinish 213
 E3DInit 213
 E3DModeSwitch 213
 E3DObjectClick 213
 E3DObjectMoved 213
 E3DReachDestination 213
 E3DStart 213
 E3DTick 213
 EndSim 205
 EquationCompilePlatform 207
 FinalCalc 205
 FlowBlockReceiveN 212
 functions for communicating 311
 GetDraggedCloneList 94
 HBlockClose 208
 HBlockFromLibrary 208
 HBlockHelpButton 208
 HBlockMove 208
 HBlockOpen 208
 HBlockSaveToLibrary 208
 HBlockUpdate 208
 HelpButton 208
 IconViewChange 208
 InitSim 205
 LinkContent 212
 LinkStructure 212
 list 204
 listed in variables pane 11
 MailSlotReceive 208
 MakeSelectionHierarchical 208
 model status 205
 ModelSave 206
 ModifyRunParameter 204
 netlist 312
 OK 210
 OldFileUpdate 206
 OLE 212
 OLEAutomation 212
 OpenModel 206
 OpenModel2 206
 PasteBlock 209
 PasteBlock2 209
 PauseSimulation 206
 PlotterNClose 209
 PreCheckData 204, 205
 ProofAnimation 211
 QueueFunction 212
 ResumeSim 206
 ResumeSimAllBlocks 206
 sent by a block 40
 sent by application 40
 sent during user interaction 40
 ShiftSchedule 212
 SimFinish 205
 SimOrderChanged 206
 SimSetup 206
 SimStart 204
 Simulate 205

- simulation 204
- StepSize 205
- TabSwitch 210
- TimerTick 209
- types of 39
- UpdateStatistics 212
- UserMsg0-9 212
- meter 13, 38
- methods
 - for automation 112
- Miles block 44
- Min2 218
- MOD operator 69
- model
 - appearance 19
 - artificial intelligence 110
 - converting for cross platform use 406
 - cross-platform 404
 - debugging a 182
 - discrete event (running) 156
 - file names 404
 - functions 314
 - goal-seeking 110
 - how discrete event models work 160
 - Macintosh file name 404
 - names 404
 - profiling 182
 - programming 314
 - self-modifying 110
 - size 398
 - status messages 205
 - styles 19
 - text blocks as commands 107
 - timing for discrete event 160
 - Windows file name 404
- ModelLock 317
- ModelSave 206
- ModernRandom 201
- modes for 3D animation 129
- ModifyDate 375
- ModifyRunParameter 204
- ModL
 - case sensitivity 62
 - code completion 31
 - code conventions 49
 - code example 47
 - code part of block 6
 - compared to C++ 24
 - compared to Java, Visual Basic, FORTRAN 26

- compiled to machine code 18
- conditional compilation 76, 79
- connector names as variables 32
- constant definitions 30, 62
- data types 30, 62
- external source code 41, 59, 80
- font 11
- functions and message handlers 30
- introduction 18
- keywords (syntax coloring) 30
- language terminology 28
- layout 29
- overview of the language structure 24
- preprocessor directives 76, 79
- structure window 45
- syntax coloring 29
- type declarations 30, 62
- user-defined functions (syntax coloring) 30
- modulo 218
- momentum 138
- mount nodes 137
- MountLook 147
- mountPoint 137
- MouseClicked v8.h 94
- MouseClicked.h 94
- Move Selected Items to Tab command 52
- MoveBlock 324
- MoveBlockTo 324
- MovieOn 201
- movies 262
- moving dialog items 56, 92
- MOX extension 404
- MsgEmulationOptimize 312
- MultC 219
- multiple simulations 201
- multiple statements 71
- MyBlockNumber 126, 285
- MyLocalBlockNumber 285

N

- Name field for 3D animation 132
- name functions 282
- names
 - constants 62
 - dialog item 14
 - functions 62
 - reserved 62
 - variables 62

- names of blocks
 - length of name 398
- NearlyEqual 218
- NearlyGreaterThan 218
- NearlyLessThan 218
- netlist messages 312
- New Block dialog 44
- New Dialog Item command 16, 45, 405
- New Dialog Item dialog 12
- New Include File command 79
- NextTimes 163
- No Resize Bar 20
- no such parent 333
- no such record 333
- NodeGetCurrentValue 288
- NodeGetIDIndex 288
- normal connectors 21
- not linked error 334
- not unique error 333
- not unique index 334
- notebook functions 314
- NotebookClose 317
- NotebookIsOpen 317
- notebooks
 - functions 314
- notify message 171
- NoValue 64, 69, 218
 - converted to integer 65
- Now 375
- number of periods 223
- numbers 62
- NumBlocks 285
- numeric conversion 65
- NumericParameter 257
- NumericParameter2 257
- NumPlotPoints 330
- NumScenarios 201
- NumSims 152, 154, 156, 157, 201
- NumSteps 152, 155, 201
- NumToFormat 370

O

- object
 - for Automation 112
- object label for 3D animation 132
- objectID 132

- objects
 - animation 120
- ODBC functions 250
- ODBCBindColumn 250
- ODBCColAttribute 251
- ODBCColumns 251
- ODBCColumns2 251
- ODBCConfigDataSource 251
- ODBCConnect 251
- ODBCConnectName 251
- ODBCCountRows 251
- ODBCCreateTable 251
- ODBCDisconnect 252
- ODBCDriverConnect 252
- ODBCExecuteArray 252
- ODBCExecuteQuery 252
- ODBCFetchRows 252
- ODBCFreeStatement 252
- ODBCInsertRow 252
- ODBCKeyword 252
- ODBCNumResultCols 252
- ODBCSetRows 252
- ODBCSetRowsType 253
- ODBCSuccessInfo 253
- ODBCTables 253
- OK 210
- OldFileUpdate 206
- OLE
 - automation client 112
 - automation server 112
 - BlockMsg 116
 - C++ examples 113
 - Execute 114
 - GetObjectHandle 117
 - IDispatch interface 113
 - messages 212
 - methods 112
 - Poke 115
 - Request 115
 - VB.net DLL example 111
 - Visual Basic 118
 - Visual Basic examples 118
- OLE functions 241
- OLEActivate 242
- OLEAddRef 242
- OLEArrayParam 242
- OLEArrayParamVariableColumns 242

- OLEArrayResult 243
- OLEArrayResultVariableColumns 243
- OLEAutomation 212
- OLECreateObject 243
- OLEDBParam 243
- OLEDBResult 243
- OLEDeactivate 243
- OLEDispatchGetCLSID 243
- OLEDispatchGetDispatchName 243
- OLEDispatchGetDispID 243
- OLEDispatchGetDoc 243
- OLEDispatchGetFuncIndex 244
- OLEDispatchGetFuncInfo 244
- OLEDispatchGetHelpContext 244
- OLEDispatchGetNames 244
- OLEDispatchInvoke 244
- OLEDispatchParam 244
- OLEDispatchPropertyGet 244
- OLEDispatchPropertyPut 244
- OLEDispatchresult 245
- OLEGAParam 245
- OLEGAResult 245
- OLEGetCLSID 245
- OLEGetDispatchName 245
- OLEGetDispID 245
- OLEGetDoc 245
- OLEGetFuncIndex 246
- OLEGetFuncInfo 246
- OLEGetGUID 246
- OLEGetHelpContext 242
- OLEGetInterface 246
- OLEGetNames 246
- OLEGetRefCount 246
- OleGlobal 201
- OleGlobalInt 201
- OleGlobalStr 201
- OLEInsertLicensedObject 247
- OLEInsertObject 247
- OLEInsertObjectFromFile 247
- OLEInvoke 247
- OLELongParam 247
- OLELongResult 247
- OLEObjectIsRegistered 247
- OLEPropertyGet 247
- OLEPropertyPut 248
- OLERealParam 248
- OLERealResult 248
- OLERelease 248
- OLEReleaseInterface 248
- OLERemoveObject 248
- OLERequestLicKey 248
- OLESetNamedParam 248
- OLEStringParam 248
- OLEStringResult 248
- OLESupressInvokeErrors 248
- OLEVariantParam 248
- OLEVariantResult 249
- Open a library command 404
- Open a model command 404
- Open Block Structure command 8
- Open Library Window command 9
- Open structure command 404
- OpenAndSelectDialogItem 295
- OpenAndSelectDialogItem2 295
- OpenBlockDialogBox 310
- OpenDialogBox 310
- OpenDialogBoxToTabName 311
- OpenEnclosingHBlock 310
- OpenEnclosingHBlock2 310
- OpenExtendFile 324
- OpenModel 206
- OpenModel2 206
- OpenNotebook 317
- OpenURL 378
- operations
 - between any type and a string 66
 - between reals and integers 65
- operators 68
- Optimizer block 90
- Option key 258
- Outer 227
- OuterC 227
- overriding
 - message handlers 76
- overriding functions 32
 - syntax coloring 30
- overriding message handlers 32
 - syntax coloring 30
- overriding the grid 18
- overriding user-defined functions 75

P

panes

- changing size 11
- code 11
- connector 11
- help 10
- icon 10
- variables 11

parameter

- add to right click menu 15
- adding to a dialog 50
- changing globally from a block 92
- changing through text on the model 107
- dialog item 13, 34
- display only 14, 34
- hide item 15
- tags 58
- visible in all tabs 15

parameter fields 13

parameter tags 58

Pascal distribution function 220

Pascal to C string function 63

pass by value or reference 98

PassArray 98, 354

passing array functions 354

passing arrays 99, 354

- connectors 99
- precautions 100
- using globals 100

passing blocks 162

passing messages 108

PassItem 174

PasteBlock 209

PasteBlock2 209

paths

- creating in 3D 141

paths for 3D animation 141

pattern 122

pattern palette 122

PauseSim() 377

PauseSimForSave 317

PauseSimulation 206

payment 223

PI 76

PictureList 264

pictures 86, 125, 263, 408

- bitmap 86, 408

naming conventions 86

Windows MetaFiles 86, 408

pixels

- animating 128
- coordinates 261
- measurement 259

PlaceBlock 324, 325

PlaceBlockInHBlock 324

PlaceDotBlock 324

PlaceTextBlock 325

PlaceTextBlockInHBlock 325

PLATFORMPOWERPC 408

PLATFORMWINDOWS 408

PlaySound 257

PlotNewBarPoint 327

PlotNewPoint 330

PlotNewScatter 333

PlotSignalFormat 330

plotter functions 326

PlotterBackground 330

PlotterNameGet 330

PlotterNameSet 331

PlotterNClose 209

PlotterSignalColorSet 331

PlotterSignalValueGet 331

PlotterSignalValueSet 331

PlotterSquare 331

PlotterXAxisCalendar 331

PlotterXAxisTime 331

pointer functions 354

PointerDispose 355

PointerFromDynamicArray 355

pointers 98

PointerToDynamicArray 355

Poisson distribution function 220

Poke 112, 115

popup menus

- dialog item 12, 37
- in block dialogs 57

PopupCanceled 295

PopupItemParse 296

PopupMenuAppendArray 296

PopupMenuArray 296

position of 3D object 135

Post functions 131

posting events 162

PostInitSim 154, 158

- Pow 218
- PreCheckData 154, 157, 204, 205
- PrecisionTimer 376
- PrecisionTimerScale 376
- preprocessor directives 26, 76, 79
 - syntax coloring 30
- present value 223
- procedures 73
- Profile Block Code command 182
- ProfileBlockGet 377
- profiling 182
- programming
 - 2D animation 120
 - arrays 97
 - changing parameters globally 92
 - code search and replace 78
 - debugging 184
 - dialogs 90
 - discrete event blocks 160
 - discrete rate blocks 178
 - DLLs 83, 254
 - extensions 82
 - include files 79
 - profiling 182
 - Report 182
 - scripting 110
 - Shared Libraries 254
 - sounds 85
 - techniques 78
 - text as commands 107
 - Trace 182
 - viewing debugging data 184
 - viewing intermediate results 184
- programming languages
 - C++ 24
 - C++ example for Automation 113
 - Fortran 26
 - Java 26
 - used for OLE and ActiveX 111
 - VBA for Automation 118
 - Visual Basic 26
 - Visual Basic for Automation 118
- prompts 256, 319
- Proof Animation
 - numerical format 370
- ProofAnimation 211
- ProofEncode 264
- ProofEncodeReset 264
- protecting libraries 86

- pseudocode 152, 156
- pulling items 169
- pushing items 168
- PushPlotPic 331
- PutFront 368
- PutRear 368

Q

- QueGetN 369
- QueInit 369
- QueLength 369
- QueLookN 369
- Query Equation block 90
- Query Equation(I) block 90
- query message 170
- QueSetAlloc 369
- QueSetN 369
- Queue Equation block 90
- queue functions 368
- QueueFunction 212
- QuickTime 262
- QuickTimeAvailable 325
- QuickView mode 130

R

- radians 219
- radio button
 - defining in a dialog 48
 - dialog item 12
 - groups 12
 - programming for 35
- radio control 35
- Random 220
- Random distribution function 220
- random number generator 201
- RandomCalculate 221
- RandomCheckParam 222
- RandomGetModelSeedUsed 222
- RandomGetSeed 222
- RandomReal 222
- RandomSeed 201
- RandomSetSeed 222
- RandomString 370
- rate 223
- Rate library
 - programming 178

Read/Write Index Checking 333
 real
 function return 217
 to integer 65
 to string 66
 real (data type) 30, 62
 Real (uniform) function 222
 real array 164
 Realabs 218
 Realmod 218
 RealToStr 370
 RealToStrShortest 371
 recursive functions 74
 red border 186
 RefreshDatatableCells 300
 RefreshPlotter 331
 RegisterBlockInLeftClickDB 94
 registered blocks 106
 registering blocks 57, 302
 RemoveAttribute 372
 RemoveSignal 332
 RenamePlotter 332
 Replace command 78
 replacing text 78
 reporting
 programming 182
 Request 112, 115
 reserved database 106
 _character 106
 blocks that use 106
 residence blocks 162
 ResizeDTDuringRead 207, 300
 resource files 82
 Resource Order ID 166
 ResourcePoolAllocate 319
 ResourcePoolAvailable 319
 RestrictConnectorMsgs 313
 ResumeSim 109, 206
 ResumeSimAllBlocks 109, 206
 ResumeSimulation 317
 RetimeAxis 332
 RetimeAxisNStep 332
 Return statement 71, 73
 returns
 function 217
 Roots 227
 rotation of 3D animation objects 135

Round 218
 Run a simulation command 404
 Run menu keyboard equivalents 405
 runs
 multiple 200, 201
 RunSetup 317
 RunSimulation 317

S

Save a model command 404
 Save Block command 50
 Save Include File As command 79
 SaveModel 317
 SaveModelAs 317
 SaveTopDocAs 318
 scale for 3D animation objects 134, 146
 scatter plots 333
 scientific notation
 definition 28
 scope 29
 script file 142
 scripting 110
 scripting environment 143
 scripting functions 319
 ScrollDTTo 300
 searching text 78
 SeedListClear 222
 SeedListRegister 222
 SelectBlock 377
 SelectBlock2 377
 SelectConnection 325
 self-modifying 110
 SendConnectorMsgToBlock 313
 SendItem 174
 SendMsg 168, 174
 SendMsgToAllCons 313
 SendMsgToBlock 313
 SendMsgToHBlock 313
 SendMsgToInputs 168, 169, 171, 178, 313
 SendMsgToOutputs 168, 169, 178, 313
 sensitivity analysis
 CurrentSense variable 200
 sensor connector 172
 serial I/O functions 253
 serial ports 253
 SerialRead 253

- SerialReset 254
- SerialWrite 254
- server application 408
- Set Block Category command 57
- Set Breakpoints window 185, 195
- SetAttribute 372
- SetAxisName 332
- SetBlockLabel 285
- SetBlockSimulationOrder 318
- SetConnectionColor 288
- SetConVisibility 288
- SetDataTableCornerLabel 300
- SetDataTableLabels 301
- SetDataTableSelection 301
- SetDefaultTabName 311
- SetDialogColors 296
- SetDialogItemColor 296
- SetDialogVariable 92, 296
- SetDialogVariableNoMsg 296
- SetDirty 325
- SetDTColumnWidth 301
- SetDTRowStart 301
- SetIndexedConValue 288
- SetIndexedConValue2 288
- SetModelSimulationOrder 318
- SetPopupLabels 296
- SetRunParameter 318
- SetRunParameters 318
- SetSelectedConnectionColor 288
- SetSignalName 332
- SetTickCounts 332
- SetTimeConstants 373
- SetTimeUnits 373
- SetVisibilityMonitoring 297
- ShapeFile 146
- Shared Libraries 85, 254
- Shift key 258
- Shift Selection Left command 78
- Shift Selection Right command 78
- Shift-click 93
 - _leftClickDB 96
 - code for a custom stand-alone block 94
 - code in blocks used remotely 94
- ShiftSchedule 212
- Show additional block information option 22
- Show Animation command 120, 259
 - AnimationOn variable 200
- Show Reserved Databases command 106
- ShowBlockLabel 285
- ShowFunctionHelp 232
- ShowHelp 378
- showing dialog items 56, 92
- ShowPlot 332
- ShowPlot2 332
- SimDelay 201
- SimFinish 205
- SimMode 201
- SimOrderChanged 206
- SimSetup 206
- SimStart 204
- Simulate 155, 205
- SimulateConnectorMsgs 313
- simulation
 - stop command 404
- simulation messages 204
- simulation order 201
- simulation pseudocode 156
- Simulation Setup command 152
- simulations
 - automating 110
 - changing data while running 109
 - internals 152
 - messages 204
 - methods 152
 - multiple 200, 201
 - number of runs (maximum) 398
 - order when running 201
 - paused 154
 - pseudocode 152, 156
 - running 152
 - running with Equation/Equation(I) block 407
 - stopping multiple 156, 160
 - time 160, 200
- Sin 219
- Sinh 219
- sink connectors 96
- sizes 97
- SkinBaseName1 136, 147
- SkinBaseName2 136, 147
- SkinName1 136, 147
- SkinName2 136, 147
- skins of 3D objects 135
- SLClear 367
- SLCreate 367

- SLDelete 367
- SLFlagGet 367
- SLFlagRealGet 367
- SLFlagRealSet 367
- SLFlagSet 367
- SLGetCount 367
- SLGetCountStrings 367
- slider 13, 38
- SLIs 367
- SLPopupMenu 367
- SLSort 367
- SLStringAppend 368
- SLStringGet 368
- SLStringGetIndex 368
- SLStringInsert 368
- SLStringRemove 368
- smart mounted 137
- SNDs 86
- SortArray 354
- SortArrayVariableColumns 301
- sorting
 - linked lists 68
- sounds 85, 257, 408
- source code
 - external 59
- source code debugger (see "debugger") 184
- source connectors 96
- source folder 81
- Source pane 186
- Speak 257
- Speech Manager 257
- SpinCursor 318
- Sqrt 218
- StackMountPoint 147
- StartTime 152, 154, 156, 201
- StartTimer 376
- StartTimerID 376
- statements
 - control 70
 - definition 28
 - multiple 71
- static data limits 63
- static text 13, 37
- static variables 31, 63
 - definition 29
 - limits on data size 63
 - scope 29
 - uninitialized 63
- statistics functions 219
- Status block 312
- StdDevPop 222
- StdDevSample 223
- step into 185
- Step Into tool 194
- step loop 155
- Step out of 185
- Step Out tool 194
- step over 185
- Step Over tool 194
- stepping through the model 182
- steps
 - number of 200
 - number of (maximum) 398
- StepSize 152, 154, 158, 205
- Stop a simulation command 404
- Stop and Go To tool 194
- Stop executing 185
- Stop tool 194
- StopDataTableEditing 301
- Stoptimer 376
- StopTimerID 376
- Str127 30, 62
- Str15 30, 62
- Str255 30, 62
- Str31 30, 62
- Str63 30, 62
- StrFind 371
- StrFindDynamic 305
- StrFindDynamicStartPoint 305
- StrGetAscii 371
- string 217
 - concatenation 70, 257
 - concatenation of a 69
 - declaring 63
 - function return 217
 - functions 369
 - literals 63
 - syntax coloring 30
 - to integer 66
 - to real 66
- String data type 30, 62
- string functions 369
- StringCase 371
- StringCompare 371

- StringTrim 371
- StripLFs 236
- StripPathIfLocal 236
- StrLen 371
- StrPart 371
- StrPartDynamic 372
- StrPutAscii 371
- StrReplace 371
- StrReplaceDynamic 305
- StrToReal 36, 371
- structure
 - open command 404
- structure window 9, 10, 45
 - printing 11
- Structure window opens in front option 11
- structures 68, 97
 - using passed arrays 101
- structures (pointers) 354
- styles of models 19
- SubC 219
- submenus for blocks 57
- subscripts of an array 66
- SuppressWorksheetRedraw 325
- Switch 72
- switch 13, 38
- Switch statement 71
- SwitchPlotterRedraw 332
- syntax coloring 29
- SysFlowGlobal0 180
- SysFlowGlobal1 180
- SysFlowGlobalInt0-21 179
- SysFlowGlobalStr0 180
- SysGlobal 76
- SysGlobal variables 174, 201
 - during CheckData or InitSim 178
 - during Simulate message 174
- SysGlobal0 160, 161, 174
- SysGlobal1 174, 183
- SysGlobal10 175
- SysGlobal11 175
- SysGlobal12 164, 175
 - passing itemArray3D 166
- SysGlobal13 160, 161, 175
- SysGlobal14 175
- SysGlobal15 175
- SysGlobal16 175
- SysGlobal17 175
- SysGlobal18 175
- SysGlobal19 175
- SysGlobal2 175, 183
- SysGlobal20 175
- SysGlobal21 175
- SysGlobal22 175
- SysGlobal3 164, 175
 - passing itemArrayR 164
- SysGlobal4 164, 175
 - passing itemArrayI 165
- SysGlobal5 175
- SysGlobal6 164, 175
- SysGlobal7 160, 161, 175
- SysGlobal8 175
- SysGlobal9 164, 175
 - passing itemArrayC 164
- SysGlobalInt0 161, 168, 169, 171, 176, 178
- SysGlobalInt1 176, 178
- SysGlobalInt10 176
- SysGlobalInt11 176, 178
- SysGlobalInt12 176
- SysGlobalInt13 176
- SysGlobalInt14 176
- SysGlobalInt15 177
- SysGlobalInt16 177
- SysGlobalInt17 177
- SysGlobalInt18 177
- SysGlobalInt19 177
- SysGlobalInt2 176
- SysGlobalInt20 177
- SysGlobalInt21 177
- SysGlobalInt22 177
- SysGlobalInt23 177
- SysGlobalInt24 177
- SysGlobalInt25 177
- SysGlobalInt26 177
- SysGlobalInt27 177
- SysGlobalInt28 177
- SysGlobalInt29 177
- SysGlobalInt3 168, 169, 171, 176
- SysGlobalInt30 177
- SysGlobalInt31 177
- SysGlobalInt32 177
- SysGlobalInt33 177
- SysGlobalInt36 177
- SysGlobalInt37 177

- SysGlobalInt38 177
- SysGlobalInt39 177
- SysGlobalInt4 176
- SysGlobalInt40 178
- SysGlobalInt5 176
- SysGlobalInt6 171, 176
- SysGlobalInt7 176
- SysGlobalInt8 163, 176, 178
- SysGlobalInt9 176
- SysGlobalStr0 175
- SysGlobalStr1 176, 178
- SysGlobalStr2 176
- system variables
 - definition 29
 - list 200

T

- tab character 232
- tab delimited files 232
- Tab key 15, 78
- tab order 15
- table list 334, 341, 342
- tabs 7
 - moving dialog items to 52
 - new 51
- TabSwitch 210
- Tan 219
- Tanh 219
- technical support
 - information to provide 3
- text 107
 - changing programmatically 90
 - formatted for dialog items 17
 - tables 14, 36
- text as commands 107
- text files
 - functions 232, 233
 - include files 79
 - numerical data 233
 - programming 233
 - string data 233
- text frame 13, 38, 50
- text tables 14, 36
- TextWidth 372
- TGE 129
- TickCount 376
- time
 - current time 200
 - time functions 375
 - time functions (legacy) 375
 - time in simulation 200
 - time units
 - functions 372
 - TimeArray 160, 163
 - TimeBlocks 160
 - TimeEventMsgType 160
 - timer functions 376
 - TimerID 376
 - TimerTick 209
 - TimeToString 375
- titles
 - formatted for dialog items 17
- titles of buttons 12
- tool tips 58
 - connector 291
 - dialog item 309
 - on block dialogs 17
 - on dialog editor 17
- tools
 - animation 20
 - connectors 20
 - in the Debugger window 194
- topic and item 113
- Torque Game Engine 129
- Torque Script 143
- TraceModeEnableDisable 377
- tracing 182
- transparent text 127
- Transpose 227
- TransposeC 228
- Trapezoidal integration 224
- trigonometry functions 219
- TRUE 76
- TStatisticValue 223
- tutorial
 - source code debugger 185
- type conversion 65
 - function arguments 217
- type declarations 29, 64
 - definition 29

U

- uninitialized static variables 63
- UnmountLook 147

- UnRegisterBlockInLeftClickDB 95
- UnselectAll 325
- UpdatePublishers 241
- UpdateStatistics 212
- upper limits 398
- Use 372
- UseRandomizedSeed 223
- user-defined functions 73
 - ADO functions 380
 - exiting 74
 - include files 79
 - overriding 75
 - recursive 74
- user-defined procedures 73
- UserError 184, 257
- UserMsg0-9 212
- UserParameter 257
- UserPrompt 257
- userPromptCustomButtons 257
- userTag 133

V

- value connector messages 172
- variable connectors 20, 21, 32, 58, 96
 - No Resize Bar 20
- variable name 33
- VariableNameToTabName 311
- variables
 - data consumption 64
 - global 201
 - list 200
 - local 28, 63
 - memory usage 97
 - names 62
 - pane 11, 186
 - scope 29
 - static 29, 63
 - SysGlobal 201
 - system 200
- Variables pane 186
- VB.net 111
- VB.net COM DLL example 111
- VBA example 118
- version control 41, 80
- view order 59
- views popup 10, 19, 59
- visible in all tabs 15

- Visible In All Tabs checkbox 51
- Visual Basic 26, 118
- Visual C++ 40
- void 217
- void functions (procedures) 28

W

- W (width) and H (height) coordinates 16
- WaitNTicks 122, 376
- Wall.cs file 142
- WatchPoint condition 197
- waypoint 142
- web functions 378
- WhichDialogItem 297
- WhichDialogItemClicked 209, 210, 258, 292
- WhichDTCell 301
- WhichDTCellClicked 259
- While loop 72
- While statement 71
- WhoInvoked 297
- Windows
 - file conversion 406
 - keyboard shortcuts 404
 - MetaFiles 86, 408
- Windows to Macintosh 405
- WinRegSvr32 249
- WinSetForegroundWindow 325
- WinShellExecute 325
- wizards 110
- WorksheetRefresh 326

X

- X and Y coordinates 16

Z

- zero time event list 159